THE SEMESTER BOOKS

Book 3 · Design System & Building

Design system · Architecture · Engineering · Source code

One of three books that together cover everything about Semester: Book 1 Company, Brand & Operating · Book 2 Product · Book 3 Design System & Building. Compiled from *The Main Semester design system*.

Edition of 21 October 2026 · Pre-seed, founder-led · Large figures are labelled sample data

Contents

OVERVIEW

Executive summary 5
The system in numbers 7
Design principles 9
Foundations explained 10
Component library guide 12
Patterns and governance rules 15
Architecture 16
How it is built 18
What's next 20
Reader's guide 21
Glossary 22

DESIGN SYSTEM

Design system brand book (README) 25
Design tokens 34
Foundations 44
Brand specimens 49
Components (light) 52
Patterns 93
Guidelines 96
Architecture and tooling cards 102
Every component, light and dark 106
Design language 114
Figma mapping and design-system baseline 120

BUILDING

Architecture guide 129
Three-tier architecture and API contract 132
Architecture cards 136
Architecture explorer 138
Developer guide 149
Engineering playbook 151
Product and engineering operations 154
QA test plan 155
Accessibility test plan 158
Build handoff 160
Execution plan, step by step 165
Code packages 186
Claude Code toolkit 190
Tooling cards 195
AI workflow router and benchmark lab 197
Prototype notes 201

Prototype-to-codebase map 203
What's next 205
Document library index 211

SOURCE CODE

Token and component stylesheets 215
Component types 226
Component sources 233
semester-core package 249
semester-platform: API, security, migrations and tests 254
Workflow router and benchmark app 267
Claude Code toolkit scripts and build scripts 289

ARCHIVE

Archive: the pre-migration cards and specimens 298

SOURCE RESEARCH

Semester build Architecture 324
Build a read-model architecture dashboard for your 327
Build an interactive CQRS read-model dashboard for 348
Interactive CQRS simulator: model event lag, proje 379
Front-end compliance audit: WCAG 2.2 criteria, FTC 405
scan 421
now do same thing as it applies to front end and c 437
building semester design system tell me everything copy 457
building semester design system tell me everything 544
provide comand to claud code to excute on this and 619
audit [https:::github.com:harrisonjrubin7-cmyk:sem (3) copy 629

BOOK 3

Design System & Building

The complete design system and how Semester is architected, built, tested and shipped.

Executive summary

Semester describes itself as the native Student Operating System for the whole educational journey: one governed, accessible platform for a student's academic, personal, campus, financial and career life, plus the institutional workspaces (faculty, advisor, registrar, finance, IT and security) that serve them. The design system in this book is the set of rules, tokens, components and patterns that make those surfaces look and behave like one product. Its character is calm, scholarly and source-aware: near-black chrome, warm off-white content, a restrained Semester blue reserved for action and focus, and a controlled vocabulary that keeps official, connected, personal and AI-generated information visibly distinct.

The company is early. The README gives its status as of October 2026: pre-seed, founder-led and bootstrapped, with a first paid pilot signed with Vanderbilt University (dates not set). HECVAT Full, a VPAT/ACR, SAML single sign-on and insurance are in progress, and large figures anywhere in the system are labelled sample data. Read the book with that in mind. Much of it is a design-level specification and hand-off package, not a record of shipped and tested software, and the sources say so repeatedly.

A few principles recur in every document. Name the source of every fact and rank sources honestly. Never let colour carry meaning alone; every status has a glyph and a word. Say what an action will do before it does it, with a way back. Default to private and ask for consent one decision at a time. Keep the institution as system of record and keep student records away from any AI model. Be respectful rather than celebratory: no streaks, ranks or confetti. These rules are written into tokens, components and checks rather than left to taste.

The system is concrete. One `tokens.json` holds 112 colour tokens plus spacing, radius, shadow, size, layer, leading and tracking tokens, with twelve type roles on three typefaces. It ships 56 documented components (57 exports counting Mark), a 108-glyph icon set, logo files, 33 specimen cards, eight written guidelines and 28 UI kits. The student app covers 64 destinations in seven navigation areas, with five canonical destinations: Today, My Path, Search, Plan and Me.

The design system is subordinate to the real codebase. The README names `harrisonjrubin7-cmyk/semester` (branch `main`) as ground truth, and the build documents repeat "repo beats design": extend what exists and record whether each item was found, extended or new. The app's token pipeline runs from `lib/look.ts` to `styles/tokens.css` to `lib/tokenexport.ts` to a generated `semester.tokens.json`, and a Claude Code toolkit checks the design project against it. The documents list the differences. The main one is palette: the app's rule is "one colour for attention, the rest in words and glyphs", while the founder's brief asked for Semester blue and distinct source hues. The system keeps the code's grounds, type, radii, motion and contrast floors, adopts the brief's palette, and treats the source hues as a proposal not yet in code.

Architecturally the product has three tiers: an experience platform, an education intelligence platform and an institutional infrastructure platform. Underneath is Postgres with row-level security and a tenant identifier on every row. The browser calls only server-side `api.*` functions, writes pass through audited commands and an outbox into read models stamped with version and freshness, and one AI gateway checks scope and policy, blocks student records, labels answers AI-assisted and logs evidence.

It is built through `handoff/BUILD.md`, a single entry point telling Claude Code or Codex to work one branch and one pull request per stream, from stream 00 (repository audit) to stream 13 (platform maturity). The package supplies nine pure-TypeScript decision modules, a database and API package with eight migrations, a Next.js reference app and a Figma

toolkit, plus a playbook, QA plan and accessibility plan. It is candid that the platform code has not been run, typechecked or tested. The closing chapter records the backlog and one open decision: whether the palette moves toward the indigo, emerald, amber and rose proposed in an outside brief.

The system in numbers

Fact	Value	Source
Colour tokens	112 total	tokens.json
Colour tokens by family	source semantics 12; status semantics 10; status primitives 14; blue ramp 9; neutral ramp 9; Ink 6; Parchment 6; Graphite 3; surfaces 6; text 6; chrome 6; brand 6; action 5; sterling 4; brass 4; borders 3; focus, accent, scrim 3	tokens.json
Themes	2 (Parchment light, Ink dark)	tokens.json
Grounds and accents in the app	13 grounds, 11 accents; this project ships Ink and Parchment	README; colors cards
Typefaces	Barlow, Barlow Condensed, Cinzel, system mono	tokens.json
Font files	6	tokens.json; fonts/
Type roles and scale	12 roles; 14-step scale from 28px to 10.5px	tokens.json; type-scale card
Line-height and tracking tokens	12 and 4	tokens.json
Layout spacing tokens	11 (8px grid plus padding)	tokens.json
Density steps	7 (2, 4, 6, 8, 10, 12, 16px) at 3 densities (1, 0.86, 0.74)	spacing card; guideline 06
Radius, shadow, size, layer tokens	4, 6, 11 and 8	tokens.json
Breakpoints	3 tiers: under 760px, 760 to 1099px, 1100px and up	guideline 07
Page padding	18, 20 and 30px by tier	guideline 07
Motion durations	130, 180, 240 and 280ms	motion card
Components	56 documented, 57 exports with Mark	components/; index.d.ts
Components by group	Actions 4; AI 2; Data 5; Forms 8; Glyph 3; Navigation 4; States 11; Surfaces 8; Trust 3; Governance 7; core primitives 2	README
Icons	108 glyphs, 1.5px stroke, 24px grid	README
StatusChip keys; SourceBadge sources	21; 10	component READMEs; index.d.ts
Provenance ladder	6 rungs plus 4 display-only kinds	guideline 01
Specimen cards; guideline documents	33; 8	guidelines/
UI kits; templates	28 folders; 21 folders	ui_kits/; templates/
Student app	64 screens, 26 fully recreated, 7 areas, 5 destinations	README
Master catalog	51 roles, 673 screens, 319 workflow steps (17 workflows), 88 systems, 82 documents	README
Architecture	3 tiers; 20 tables; 15 API domains; 10 API checks; 5 freshness states; 5 ADRs	architecture docs

Fact	Value	Source
Migrations; core modules	8 (010 to 080); 9	BUILD.md; semester-core README
Security findings fixed in 050	11	semester-platform README
Execution streams	14 (00 to 13)	execute/00-INDEX.md
CI jobs that block merge	7	Engineering Playbook
QA role journeys; severities	14; 4	QA Test Plan
Assistive technologies tested	5	Accessibility Test Plan
Figma mapping	7 collections, 48 semantic rows, 12 primitive rows, 11 components	FIGMA-MAPPING.md
Toolkit skills; router workflows	3; 12	toolkit and router READMEs

The sources disagree in a few places. Components number 56 in the README, 57 in NEXT-SESSION and 64 in BUILD.md; this book uses 56 documented and 57 exported. The Figma README says 49 semantic variables but the table has 48 rows. Role counts are 51 in the catalog, 49 in the role-pages note, and the Role lens covers 14 roles in the README but 22 in NEXT-SESSION. The execute index says "12 stream prompts" but lists 00 to 13. The front end is Vite and React in the playbook, baseline and explorer, but the Architecture Guide says Next.js; BUILD.md reconciles this with a separate `web/` Next.js project if `app/` is Vite. Environments number three in the architecture card and four in the guide.

Design principles

1. Name the source and rank it honestly. Facts show their origin in a controlled vocabulary. Stale ranks below "yours" on purpose, because an unchecked copy of something that may have moved is worth less than a guess that knows it is a guess. Example: a deadline shows a SourceBadge "Official", naming the Registrar and "Updated 2h ago".

2. Colour is never the only carrier. Every state is glyph, word and tone. Hues are desaturated stones used as ink, hairline and an 8 percent wash. Example: forced-colours mode thickens the solid provenance chip's border to 3px.

3. Say what will happen first. Consequences appear before confirm: what happens, exactly what, what stays, how to take it back. Example: "You leave CHEM 101 and a W is recorded on your transcript. It cannot be undone here."

4. The institution stays the system of record. Semester never calls itself the official gradebook, registrar or payment system. Example: "Semester does not process the payment; TouchNet does."

5. Student records never reach an AI model. Enforced in the gateway, not by convention. Example: the QA plan tests that a question containing a student ID is blocked, logged and answered with a friendly message.

6. Default private, one decision at a time. Decline has equal weight to accept. Example: "Only what you tick here can be shared."

7. Respectful, not celebratory. No streaks, ranks, confetti or personas. Example: "Registration submitted. Next: review your fall schedule."

8. Border first, shadow second. Hierarchy is hairline, then surface step, then shadow. Example: cards have a 1px border, 6px radius, no shadow, and are never nested.

9. Design all seven states. Loading, empty, partial or stale, offline, error, permission and success. Example: "Your draft is safe on this device."

10. One primary action per decision region. Example: PageHeader allows a kicker, one H1, one line of purpose, a source badge and at most one primary button.

11. Disclose in three layers. Immediate decision, explanation and action, then advanced detail. Example: "Your registration opens Tuesday at 8:00 AM. You have one hold." is layer 1.

12. Reuse; do not invent. Modules may not invent colours, spacing, status words or source labels. Example: states come from `SOURCES` and `STATUSES` in `components/trust/vocabulary.js`, never hand-typed.

13. Status, not claims. Never claim a certification not held. Example: "working toward WCAG 2.2 AA; ACR in preparation", and roadmap items carry "Planned", "Pilot" or "Preview" chips.

Foundations explained

Colour: grounds. Ink is the near-black family (`ink-bg` #090a0e, `ink-panel` #12141a, `ink-raise` #22262f, text #eceef2). Parchment is the warm off-white family (`parchment-bg` #f4f1ea, panel #fbf9f4, text #1b1a17). Content defaults to Parchment and flips to Ink under `[data-theme="dark"]` with tonal elevation, so surfaces lighten as they rise. Chrome (rail, tab bar, console bar) is always Ink. The production app offers 13 grounds and 11 accents (Sterling, Brass, Copper, Jade, Slate, Pewter, Industry, Oxblood, Moss, Indigo, Old gold); this project ships Ink and Parchment. Accents identify courses and action and never encode success or error.

Colour: roles and hues. Six surface, six text and three border tokens map onto the ramps; borders are hairlines at about 9, 16 and 34 percent ink on light. Action is Semester blue: blue 700 (#416180) on light and blue 400 (#94bce3) on dark, with hover and press one or two ramp steps away and a blue-600 focus ring on light. Four source states each have ink, line and wash tokens: official (#2c455d on light, #b5d9fd on dark), connected (#2d5c61, #9fcbcd), personal (#4a4b4f, #b9b9bd) and AI (#544a7f, #bcb2df). Status hues are pending (#7a5310, #e2c27a), danger (#a3301f, #f0907a) and success (#3e6141, #a9c9a3). Lines run about 40 to 50 percent alpha and washes 6 to 12. Sterling is for the mark and chrome highlights (a brushed gradient is used on wordmark text only); brass is an editorial accent, using its shade stop (#695a33) for text on light.

Colour: contrast. Contrast is measured against every surface of a ground. Body and UI text need 4.5:1, large text 3:1, and icons, borders, focus and chart marks 3:1. Tertiary text is about 3.4 to 3.7:1 and is for metadata only, never anything a student must act on.

Type. Barlow carries reading at 16px on a 1.55 line, with controls at 14px. Barlow Condensed 600 carries headings, labels and buttons at -0.015em. Cinzel is for rare editorial display of 42 to 56px and never appears in warnings, provenance, consent or accessibility text. System mono is for IDs and timestamps. The twelve roles run from editorial (42px) through display (26), page (24), section (20), card title (18), body (16), control (14), compact (12), label and caption (11), kicker (10.5, tracked 0.08em caps) and numeric (22, tabular). Measure is about 66 characters, sentence case is universal, and sizes scale with `--text-scale`, which reader settings move from 0.94 to 1.18.

Spacing and density. Layout uses an 8px grid (8, 16, 24, 32, 40, 48, 64, 80, 96px) with page padding of 18px widening at 760 and 1100px. Inside components, seven steps (2 to 16px) are multiplied by density 1, 0.86 or 0.74. Targets are 44px by default, 40px for icon buttons, 32px for compact rows and a 24px floor in crowded composites with keyboard parity.

Corners. "Drawn" corners are 3px for controls, 6px for cards and 10px for sheets, with a 999px pill for floating bars. Chips stay at 3px.

Elevation. Border first, shadow second. Light grounds use three ink-tinted shadows; dark grounds use three lift tokens. Shadows appear only on floating and modal layers. Eight stacking layers run from 0 to a 1500 curtain. Only the tab bar and focus bar are translucent (14px blur), with an opaque fallback under reduced transparency, increased contrast, Low stimulation and forced colours. Scrims are 50 percent Ink.

Motion. Hover and press take 130ms, state changes 180ms, panels 240ms and sheets 280ms, with progress at 180ms linear and the ease `cubic-bezier(.22,1,.36,1)`. Sheets rise 8px and fade. There is no bounce, parallax, shimmer, spinner, looping decoration or press scaling. All motion is zeroed by `prefers-reduced-motion` and `data-calm`, and no meaning depends on it.

Layout and breakpoints. Below 760px a labelled bottom TabBar carries the five destinations. From 760 to 1099px a 72px compact rail appears. From 1100px the SideRail is 248px (272px at very wide). A 44px SystemContextBar sits on top with location, term, Command-K search and sync state. Pages use a `minmax(0,1.6fr) minmax(300px,1fr)` grid that stacks below 1000px, with the next action leading the main column and "Needs you", context and policy in the aside. Reading measure is 66ch and operational pages cap at 1180px. The student app is mobile-first and offline-safe; staff and console are desktop-first; the public site goes single column under 860px; everything reflows at 320px and 200 percent zoom.

Accessibility floors. The target is WCAG 2.2 AA, a design requirement and explicitly not a conformance claim. Each page has one main landmark, one H1 and a skip link. Focus never lands under sticky chrome. Inputs are native with persistent labels, and errors preserve input. Overlays trap focus, close on Escape and return focus. Reader settings cover text size, density, line height, letter and word spacing, reading width (52 to 82 characters), an Atkinson Hyperlegible option, three motion modes, Increase contrast and 13 grounds. Status, provenance, consent text and accessibility controls are brand-exempt: body-grade type only and never restyled.

Component library guide

Each component has a README, type declarations and a live preview. All are bundled under `window.SemesterDesignSystem_e9f3ba` and typed together in `components/index.d.ts`.

Actions

- **Button.** One primary per decision region; 44px, or 32px with `size="sm"`. Variants: primary, secondary, ghost and danger (red ink and hairline, never a red fill). Props: `variant`, `size`, `icon`, `loading`, `disabled`.
- **IconButton.** Icon-only control; `label` (its accessible name and tooltip) is required.
- **QuickActions.** Two to five labelled starting actions for empty and onboarding states.
- **ActionPreview.** What happens, exactly what, what stays, taking it back. Props: `says`, `exactly`, `doesNotChange`, `recovery` (undo, request or none). Place it inside a Dialog for consequential actions.

AI

- **AIResponse.** A governed answer labelled AI-assisted with scope, numbered sources, confidence, cost and policy. Consequential `actions` must open a Dialog and ActionPreview.
- **ProvenanceChips.** Origin chip plus at most two cues and an age, spoken as one sentence.

Data

- **DataTable.** Compare columns; use a list when scanning or acting on rows. Props: `columns`, `rows`, `caption`.
- **Fields.** Label and value list for one record.
- **DecisionTrail.** Timeline of who did what, when and under which authority; states done, current, waiting, blocked.
- **MetricTile.** One figure with unit, source and age; never an outcome claim.
- **ProvenanceChart.** Bars encode provenance by pattern: solid, outline, hatched, dotted. The title states the finding.

Forms

- **TextField.** Visible label, hint and error; the placeholder is an example, never the label.
- **Select.** Native select in Semester's palette.
- **Segmented.** Two to four exclusive presentation choices, not navigation.
- **Choice.** Radio or checkbox row with a sentence on the consequence, for sharing decisions.
- **Switch.** On/off setting that shows the word On or Off.
- **Combobox.** Filtered listbox for courses, people and rooms.
- **DateField.** Native input that speaks the date back as "Thu 9 Oct 2026" via `said`.
- **VisibilityPicker.** The one privacy question with four answers: Only me, Course, Selected collaborators, Public portfolio. Offer only what the module can honour; it never grants access.

Glyph

- **Icon.** From the 108-glyph set; `label` makes it a named image.

- **Mark** and **Wordmark.** The three-slab mark and the SEMESTER lockup; `metal` paints sterling for dark chrome.
- **SectionLabel** and **Avatar** (core primitives). The tracked-caps kicker with optional count, and initials in a tinted square, never photos.

Navigation

- **SideRail.** Persistent 248px Ink rail; the current row has a blue wash.
- **TabBar.** Mobile bottom navigation, always labelled.
- **SystemContextBar.** Location, term, search, sync state.
- **CommandPalette.** Global search and commands with keyboard selection.

States

- **EmptyState.** Why it is empty plus one action.
- **LoadingState.** Structural placeholders and a spoken "Loading"; no spinner.
- **ErrorState.** Plain-language failure with a required recovery and optional reference; keeps the user's input.
- **SuccessState.** Acknowledgement pointing to what is next.
- **ProgressState.** Determinate wait with Cancel or Retry.
- **StepStatus.** Named steps (waiting, working, done, failed) so a system never fakes certainty.
- **PermissionNotice.** What changed about access, why, and where it is controlled.
- **OfflineStrip.** What still works and how many changes are queued.
- **SaveState.** Quiet "Saving, Saved" live region.
- **Notice.** One-line service notice.
- **UndoToast.** Confirmation with Undo for reversible work; never for consequential or financial actions.

Surfaces

- **ObjectCard.** The universal decision card: eyebrow, title, provenance, why, metadata, one primary and one secondary action.
- **ContextBar.** The object a screen is about, with states and actions.
- **NextSteps.** Ordered safe actions with one line of why.
- **Dialog.** Modal confirmation; focus lands on Cancel; `destructive` uses a red-ink button.
- **Sheet** (with **SourceDetails**). Bottom sheet or window; home of Source and details.
- **FocusBar.** Floating pill for focus sessions.
- **ScreenGuide.** "About this screen" disclosure at every screen's foot.
- **OpenIn.** Words-only links to every place an object matters.

Trust and provenance

- **SourceBadge.** Glyph, word and tone for origin; `onReport` adds "Report incorrect information".
- **SourceLine.** A thin brass rule from a statement to its source chip; used only where a real source exists.
- **StatusChip.** One of 21 shared status keys, always glyph and word.

Governance

- **PolicyBadge.** Policy name, version and owner; opens policy text in a Sheet.
- **ConsentBadge.** Private, shared (recipient, expiry), expired, revoked or required.
- **HealthBadge.** Healthy, watch, at risk or unknown, with driver and freshness.
- **ApprovalBanner.** Two-person approval state with buttons suited to the viewer.
- **ErrorSummary.** Form errors at the top (WCAG 3.3.1), each linking to its field.
- **SupportHandoff.** Hand-off to the owning office with channel, hours and status, without editing the official record.
- **PageHeader.** Kicker, one H1, purpose, source and at most one primary action.

The component library document marks FreshnessBadge, AuthorityBadge and RiskBanner as "Partial" and NotificationCenter as "Pattern only". The seven governance components closed gaps in October 2026.

Patterns and governance rules

Provenance ladder. Strongest first: official, connected, made (built by the app from the student's sources), yours, sample, stale. Display-only kinds are AI-assisted, estimated, needs review and external. Made and AI differ on purpose, since an extraction can misread and a draft can sound right while wrong. Estimates and AI are styled to be noticed, because students over-trust them. Freshness reads "Updated 3 days ago" or "Update time not recorded", never "just now" for an unknown time.

AI answer anatomy. Every answer shows the AI-assisted label, the allowed data scope (changeable before asking), numbered sources, confidence and limits, request cost, the governing policy, and actions that open a preview first. StepStatus shows the work. Never say "AI-powered", "intelligent", "knows you" or "accurate". When an instructor restricts AI, the course shows "Course AI policy, set by instructor" and disallowed actions are disabled with a reason, not hidden.

Consent. Private by default, never implying guardian access. Each ask states what, with whom, why, how long and how to undo. Before sharing, the student previews the exact payload and expiry. Shares are revocable beside where they are made, access history is visible, and protected categories (counselling, health, disability services) show a disabled switch with the reason.

Consequential actions.

Kind	Pattern
Reversible and personal	Do it, then UndoToast
Leaves Semester or visible to others	Dialog with ActionPreview
Academic record	In-place Review then Confirm, stating where the request lands
Financial	Dialog naming amount, method, processor and refund route
Institutional high-risk	Two-person duty, fresh MFA within 15 minutes, audit first

Dialog titles are questions naming the object, buttons name outcomes, focus lands on the safe choice, and recovery is required. A SuccessState receipt follows.

State matrix. Every data region designs loading, empty, partial or stale, offline, error, permission and success.

Data trust panel. Critical screens show Source, Updated, Authority, Status, Policy (versioned) and a path to official detail. Delayed data blocks destructive actions until confirmed at the source.

Content templates. The status sentence is the unit of voice: what is true, what it means for you, what you can do. Errors give what failed, what is safe, the way out and a reference such as SYNC-4F2A. Incident notices list what is affected, what is not (only if verified), what to do and the next update, and never say "minor" or "isolated". Onboarding starts from what the student has, asks consent one item at a time, uses no streaks, and always offers Skip. Dates read "Thu 9 Oct · 23:59". Emoji are never used.

Architecture

The architecture documents are labelled design-level. The Architecture Guide (v0.1, October 2026) says to "verify everything against the repo", so this chapter describes the intended architecture, not audited running software.

Three tiers. Tier 1, the experience platform ("Apple-like"), holds the Student OS, Course Studio, advisor, registrar, family, career, company site and operations surfaces. Tier 2, the education intelligence platform ("Google-like"), holds identity and education graphs, search, the AI gateway, policy and workflow engines, provenance, analytics and human escalation. Tier 3, the institutional infrastructure platform ("Microsoft-like"), holds the tenant control plane, RLS, SSO and SCIM, approvals, audit, the integration gateway (LTI, OneRoster, Edu-API), migration, feature flags, billing, trust, SLOs, backup and a developer ecosystem. The borrowing table says what not to copy: closed ecosystems, ad-tech incentives on student data, configuration overload and treating model output as official truth. A ten-step deployment ladder runs from development to multi-campus expansion, and each step needs a security review, accessibility review, rollback plan and release evidence. The integration doctrine is to connect first and replace a domain only where the institution chooses, in five phases from Connect to Education OS; phase one is "Now (paid pilot)".

Layers and data. The front end is the student app, the staff and console routes on the same shell, the design system and an offline queue. Middleware refreshes sessions, resolves the tenant, checks role by screen by grant, rate-limits (harder for AI), writes the audit event before the action, and applies per-tenant flags and kill switches. The server layer offers `api.*` read functions, `private.*` command functions requiring an idempotency key, edge functions, integrations and an outbox worker. The database is Supabase Postgres with RLS on every table. The design-level schema has 20 tables, including tenants, memberships, role_grants, enrollments, grades, holds, shares, approvals, audit_log, domain_outbox_events, ai_requests and integrations. Schemas are `core`, `edu` (student data), `community`, `market`, `ops` (company data, no student identifiers) and `audit`.

Read models, CQRS and freshness. A write succeeds against the authoritative table, an outbox event is written in the same transaction, a worker applies the projection idempotently, version and freshness are stamped, caches are invalidated, the browser gets a targeted refetch signal and the UI renders the read model. The `private`, `read_model` and `api` schemas keep that boundary visible. Freshness drives UI policy: live (up to 5 seconds) and recent (up to 60) behave normally; delayed (over 60) shows a badge and re-checks the source before consequential actions; failed or rebuilding shows the last good state and holds them. Rebuilds are blue-green behind a parity check. Realtime is limited to approvals, incidents, P0 and P1 support, rollout state and invalidation. The UI is optimistic: it shows "Saving" at once, confirms only when the projection version advances, and rolls back with a reason otherwise.

AI gateway. The gateway is the single egress to models (ADR-003). A request passes identity and scope, policy (course AI policy, institution switches, FERPA boundary), routing to an approved model, an answer labelled AI-assisted with sources and confidence, and an evidence log. Student records are blocked at the policy step. The allow-list admits course content, the student's own typed text and public catalog data, and blocks anything from `edu.*`. The starting code runs authentication, policy, gate, content scan, provider and audit, with a redaction module that blocks IDs, SSN, GPA, grades, transcripts, dates of birth and card numbers in prompts, a usage log that stores no prompt content, and a red-team suite that is a launch gate.

Environments. Local uses synthetic seed data. Preview deploys per pull request on synthetic data. Staging mirrors the institution's configuration with synthetic data and test IdP accounts. Production holds real records under the DPA with separate keys, named operators, MFA and logged access. Data is never copied between environments, and secrets live in

Vercel and Supabase settings.

API contract standard. APIs are organised by authority and risk. "High-impact writes never mutate raw tables from the browser; they go through capability-checked server commands." Fifteen domains are defined (identity, tenant, academic, learning, workspace, AI, student success, campus, career, family, integrations, commercial, trust, operations, platform), each with a write-control model. Every API answers ten questions: identity, scope, purpose, authority, policy, validation, idempotency, audit, freshness and error recovery. Examples include `POST /v1/academic/registration-requests` and `POST /v1/family/grants`.

Security model. Five principles: the institution is the system of record; student records never reach an AI model; company roles never see student data; consequential actions are previewed, attributed and logged; source and freshness show on every record. RLS checks the tenant against the JWT claim, service-role keys live only in workers, every query runs as the user, and every mutation writes an audit row. Marketplace payments go through a licensed processor, so Semester never holds funds. High-risk actions need a second approver and fresh MFA. A review of migrations 010 to 040 found 11 issues, including a critical one (staff self-inserting as CEO) and a high one (changing a profile's institution), fixed in migration 050, which must be applied before any data. Migrations 060 to 080 add audited command functions, the AI gateway tables, private storage and delegations. Five ADRs record the main choices: RLS as primary authorisation, read models over live SIS queries, the single AI egress, processor-based payments, and a separate `ops` schema.

Operations. The explorer records staged tenant rollouts, forward-only migrations with a parity check, monitoring of errors, projection lag, sync failures and AI spend, on-call response of 1 hour (P0) and 4 hours (P1), daily backups with point-in-time recovery and a quarterly restore test, quarterly access reviews, and postmortems within five days.

How it is built

Stack. The toolkit baseline (repository commit 5eba494, 5 October 2026) describes the real app as Vite, React and TypeScript under `app/`, with plain CSS, about 273 component files, about 95 lazy screens, 19 style contract tests, a 143-pairing contrast test and 12 CI workflows. Hosting is Vercel and Supabase; validation is Zod; tests are Vitest. The platform and router packages are Next.js App Router with Supabase.

Packages. `semester-core` is dependency-free, I/O-free TypeScript with nine tested modules: provenance, nextaction, access, freshness, consequence, aigateway, approvals, scoring and pilot. `semester-platform` holds database and API code for community, marketplace, ops and audit: migrations 010 core, 020 community, 030 market and 040 ops, then 050 to 080 for security, commands, AI and storage, plus API routes, validation and RLS, security, command and red-team tests. Its README says: "Not yet run, typechecked or tested." `workflow-router-app` is the AI Workflow Router and Benchmark Lab: Next.js, strict TypeScript, Supabase magic-link auth, Zod server actions, two migrations with organisation-scoped RLS, a seed of 12 workflows and Vitest suites; its README says checks "have not been run here". All code files carry a `.txt` suffix, which `install.sh` strips without overwriting.

Migrations and tests. Migrations run 010 to 080 in order, renumbered after the repository's latest, and applied ones are never edited. Each ships with RLS policies and tests in the same pull request (own tenant allowed, other tenant denied, company roles denied on student tables, expired grant denied), proved red then green. Test layers are unit, RLS, AI red-team, Playwright end-to-end and accessibility. `verify.sh` must end with "ALL GATES PASSED".

Execution plan. Each stream uses a branch `semester/<stream>`, ends with a pull request and a report, and stops for review.

Step	Stream	Gate
00	Repo audit and design-system sync	CI green; baseline approved
01	Database, tenancy, RLS, core logic	RLS red-then-green
02	Identity, SAML SSO, onboarding	Test login per pilot role
03	Student OS	Five states per screen; accessibility 95 or more
04	Staff and academic workspaces	Two-person approvals tested
05	Institution console, integrations	Zero unexplained mismatches
06	Governed AI gateway	Red-team suite passes
07	Community, career, family	Revocation immediate
08	Operations command center	Company-role isolation test
09	Security, privacy, accessibility	Restore drill timed
10	Pilot launch	Every go/no-go gate evidenced
11	Public site and grow	Every claim sourced
12	One place	No regulated step re-implemented
13	Platform maturity	Scorecard re-scored from evidence

The pilot-critical path is 00, 01, 02, 03, 06, 05, 09, 10. Humans alone bind insurance, sign the DPA, submit HECVAT Full, obtain attorney review, supply the Vanderbilt IdP details, approve the baseline and each pull request, approve cutover and choose the AI provider.

Engineering playbook. Three rules never bend: RLS on every table with a proving test; the browser calls `api.*` only; student records never reach an AI provider. `main` is always deployable, pull requests stay under about 400 lines, and the founder approves anything touching auth, RLS, the AI gateway or billing. Seven CI jobs block merge. Releases are tagged, smoke-tested in staging, backed up before migrations and watched for 30 minutes, and none ship in launch week, finals or registration opening except S1 and S2 fixes. Features ship behind per-tenant flags with kill switches. Dependency alerts are reviewed weekly. The product and engineering operations page admits gaps in a PRD template, ADR log and experiment registry.

QA and accessibility. The QA plan uses one synthetic account per role and 14 end-to-end journeys, such as a registrar override needing a second approver and a support user unable to read student tables. Defects run S1 (data exposure) to S4 (cosmetic). The accessibility plan targets WCAG 2.2 AA with an ACR on the VPAT 2.x template, runs axe in CI, and requires manual keyboard, screen-reader (NVDA, JAWS, VoiceOver, TalkBack), zoom, spacing, target and motion checks on pilot paths each release.

Claude Code and Figma toolkit. The toolkit adds three skills (`audit-semester-design-sync` , `build-semester-ui` , `create-semester-component`), an `.mcp.json` for the remote Figma MCP server with no credentials, a CLAUDE.md section, a zero-dependency audit script, a report script, Vitest proofs and `design-system:check` scripts, with no new dependency, Tailwind or Storybook. Authority runs from `look.ts` to `tokens.css` to `tokenexport.ts` to the generated JSON to `FIGMA-MAPPING.md` to Figma, and Figma is never written to. Figma names stay "proposed" until a file is connected.

What's next

The founder's top-priority ask is: "Every role in Semester. Finish, generate and produce everything I need to run, grow, build, operate, maintain, secure and launch Semester." NEXT-SESSION.md records launch, secure, build, operate, run and grow documents as done, plus one page for each of 49 catalogued roles.

Open work, in order: replace archetype screens with bespoke designs, highest traffic first; regenerate the master catalog documents; export the investor, institution and internal decks to PPTX; deepen registration readiness, support SLA and SIS reconciliation screens; build Course Studio and the moderator workspace; write the AI governance policy, data inventory, incident response and pilot delivery documents; and run the Claude Code toolkit in the real repository. An administrator manual and tutorials are requested twice and remain to do, as does a batch of device and media capabilities (camera, microphone, location, notifications, offline, multi-device sessions, personalisation) with consent, fallback and revoke paths.

One decision is open: indigo with emerald, amber and rose from an outside brief, or Semester blue with stone hues. The rules to keep are that nothing is verified without code, tests and release evidence; company roles never see student records; status replaces certifications not held; and external numbers are cited. The `github.md` note records that `docs/design-system/` and the `workflow-lab/` app have landed upstream, while the toolkit's `.mcp.json`, skills, scripts and baseline had not.

Reader's guide

- **Design system README.** What Semester is, voice, visual foundations, iconography, components and UI kits.
- **Tokens.** The reference generated from `tokens.json`.
- **Foundations.** Specimen cards for colour, type, spacing, density, layout, motion and data visualisation.
- **Brand specimens.** Logo system, mark, wordmark, iconography rules and voice.
- **Components, light.** Every component on the Parchment ground.
- **Components, light and dark.** The same components on Parchment and Ink.
- **Patterns.** Provenance ladder, AI anatomy, consent, consequential actions, state matrix, Source Line.
- **Guidelines.** Eight written deep dives, from provenance to content templates.
- **Design language.** The eight documents in `docs/system/design`.
- **Figma mapping.** Collections, variables and component mappings.
- **Architecture guide.** Principles, components, data model, flows, environments and ADRs.
- **Three-tier and API.** The three tiers, deployment ladder and API contract standard.
- **Architecture cards.** Layers, read-model pipeline, AI gateway, environments.
- **Architecture explorer.** OS map, 20-table schema, CQRS pipeline, lag simulator and phases.
- **Developer guide.** Setup, conventions, branching, testing and the new-feature checklist.
- **Engineering playbook.** Layout, CI, environments, releases and definition of done.
- **Product and engineering ops.** The company-OS page on owner, KPIs and approval thresholds.
- **QA plan.** Test levels, role journeys, negative tests and gates.
- **Accessibility plan.** WCAG 2.2 AA checks, assistive-technology matrix and ACR.
- **Build handoff.** `handoff/README.md` and `BUILD.md`.
- **Execution plan.** The master index and stream prompts 00 to 13.
- **Code packages.** READMEs for semester-core, semester-platform and the router app.
- **Toolkit.** The Claude Code and Figma toolkit documents.
- **Tooling cards.** Four summary cards for the tooling.
- **Workflow lab.** The AI router and benchmark prototype.
- **Prototype notes.** What is sample data and what is stored only on the device.
- **Prototype-to-codebase map.** The `github.md` screen map and sync history.
- **What's next.** The NEXT-SESSION backlog.
- **Source-code appendices.** `tokens.css` and `bundle.css`; component sources; semester-core; semester-platform (API routes, migrations, tests); the workflow-router app.

Glossary

Term	Meaning
Ask Semester	The AI assistant; no human name or persona, always labelled AI-assisted
AI gateway	Single egress to models; checks scope and policy, blocks records, logs evidence
Approval banner	Component showing two-person approval state
Audit before action	The audit event is written first, then the action applies
Authority	The office entitled to decide a fact, such as the Registrar
Batch N	The founder's plan to produce everything needed to run and launch Semester
Brand-exempt zones	Status, provenance, consent text and accessibility controls, never restyled
Break-glass	Emergency privileged access needing a reason and review
Calm mode	`data-calm`; zeroes motion and quiets decoration
Canonical destinations	Today, My Path, Search, Plan, Me
Chrome	The always-dark rail, tab bar and console bar
Connected	Provenance rung for a recently checked account the student connected
Consent badge	Private, shared, expired, revoked or required state
Consequence copy	Text stating what an action does, before confirm
Core	`semester-core`, nine tested decision modules
CQRS	Writes to authoritative tables, reads from projected models
Decision trail	Timeline of who did what under which authority
Density	Multiplier of 1, 0.86 or 0.74 on component spacing
DPA	Data processing agreement governing production data
Drawn corners	3px controls, 6px cards, 10px sheets
Freshness	Live, recent, delayed, failed or rebuilding
Ground	Base surface family: Ink or Parchment (13 in the app)
HECVAT	Vendor assessment toolkit; HECVAT Full is in progress
Idempotency key	Makes a retried command safe
Ink	The near-black ground, #090a0e
Kicker	10.5px tracked uppercase label
Made	Rung for content built by rule-based extraction from student sources
Master catalog	51 roles, 673 screens, 319 steps, 88 systems, 82 documents
Outbox	Events written in the same transaction as a change
Parchment	The warm off-white ground, #f4f1ea
Policy badge	Names governing policy, version, owner
Provenance	Origin and trust level of a fact
Read model	Projected table stamped with version and freshness

Term	Meaning
Repo beats design	Existing repository code wins over design proposals
RLS	Row-level security in Postgres
Semester blue	Action colour: #416180 light, #94bce3 dark
SIS	Student information system, the institution's record
Source badge	Glyph, word and tone showing origin
Source Line	Thin brass rule linking a statement to its source
Stale	Rung for data from a connection not checked lately
Status sentence	What is true, what it means, what you can do
Stream	One numbered unit of the execution plan
Tenant	An institution's isolated space; `tenant_id` on every row
Three-layer disclosure	Decision, explanation, advanced detail
Two-person duty	Requester and a different approver, fresh MFA, audit first
VPAT/ACR	Accessibility template and conformance report; in preparation
Visibility picker	The four-audience "Who can see this?" control
Wash	Low-alpha tint behind chips
WCAG 2.2 AA	Accessibility target, not a conformance claim
Yours	Rung for student-entered, unchecked information

PART 1

Design system

Tokens, foundations, every component in light and dark, patterns and the deep-dive guidelines.

Brand book

Semester is the native Student Operating System for the complete educational journey — one governed, accessible platform for a student's academic, personal, campus, financial, career and lifelong life, plus the institutional workspaces (faculty, advisor, registrar, finance, IT/security) that serve them. Surfaces include Student Today, Calendar, Tasks, Courses, Study, Write, Degree, Registration, Grades, Campus, Dining, Housing, Family, Career, Billing, Support, the AI Assistant, and the Institution Console.

The system is calm, scholarly and source-aware: near-black chrome, warm off-white content, restrained Semester blue for action and focus, and a controlled vocabulary that keeps **official, connected, personal and AI-generated** information visibly distinct.

Status (Oct 2026): pre-seed, founder-led, bootstrapped; first paid pilot signed with Vanderbilt University (dates not set). HECVAT Full, VPAT/ACR, SAML SSO and insurance are in progress. Large figures anywhere in this system are labelled sample data.

Sources

- **Codebase (ground truth):** https://github.com/harrisonjrubin7-cmyk/semester (branch `main`). Key files read: `DESIGN-SYSTEM-GUIDE.md`, `docs/COLOR-AND-DARK-MODE-SPEC.md`, `app/src/styles/{tokens,typefaces,industry,unity}.css`, `app/src/lib/{look,status,source}.ts`, `app/src/components/{Brand,Icons,icons.data,mark.data,SourceBadge,StatusNotice,StartToday}.tsx`, `app/src/components/unity/*`, `app/src/components/console/*`. Also read: `docs/BRAND-PLATFORM.md` (voice, naming, Source Line), `enrollment/StudentRegistration.tsx`, `gradebook/StudentGrades.tsx`, `AdvisorMeeting.tsx`. Explore the repo further — especially `docs/design/`, `company-site/` and `app/src/components/` — to build closer to the real product.
- **Uploads:** `uploads/download-1.png` (lit silver logo mark on Ink), `uploads/Semester build Architecture.pdf`, `uploads/audit [...].pdf/.md` (repo audit).
- **Brief:** the user's written design direction (source-status hues, Semester blue, warm off-white content). Where it diverges from the code it is noted below.

Reconciling brief and code

The app's own rule is "one colour for attention, the rest in words and glyphs", with a sterling/brass metal accent. The brief asks for a Semester blue action colour and distinct source hues. This system **keeps the code's grounds, type, radii, motion, glyph+word status vocabulary and contrast floors**, and **adopts the brief's palette** using hues pulled from the code's own measured ramps (Industry blue → Semester blue; Jade/Moss/Indigo/Old gold → teal/green/violet/amber). Every chip still carries a glyph and a word; colour is never the only carrier.

CONTENT FUNDAMENTALS

From `docs/BRAND-PLATFORM.md` (draft) and in-app copy. **Personality:** a good registrar's assistant crossed with a precision instrument — exact, calm, candid, warm sparingly, self-possessed. **Public descriptor:** "the student action platform" ("Student OS" is internal only — it implies replacement). **Assistant name:** *Ask Semester* — no human name, face or persona; always labelled AI-assisted.

The status sentence (the unit of Semester voice): *what is true → what that means for you → what you can do.* "Your Canvas calendar hasn't updated since Tuesday. Deadlines below may have moved. Reconnect it, or check the course page."

Prefer / avoid: "helps you organise" not "revolutionises"; "alongside your school's systems" not "replaces / all-in-one"; "AI-assisted, source-linked" not "AI-powered / intelligent / magic"; name the control, never "secure / private / compliant / certified"; "planned / in private beta / pilot" not "launching soon"; "can / is designed to" not "guarantees / always / 100%"; "student" not "user / learner". Destinations are always Today, My Path, Search, Plan, Me.

Consequence copy from the product: "Confirming puts you on the waitlist, number 3 in line. You are not enrolled until a seat comes to you." · "You leave CHEM 101 and a W is recorded on your transcript. It cannot be undone here." · "Worked out by Semester from released grades only; your instructor's final grade is the one that counts." · "Only what you tick here can be shared. Nothing else — not your study history, grades, notes or anything else — ever is."

- **Voice:** plain, precise, quietly warm. Speaks to the student as **"you"**; the product never says "we" about itself in-UI ("Semester has not checked it with your institution"). First person only in the assistant's own answer ("I'd place them…").
- **Casing:** Sentence case everywhere — buttons, titles, chips ("Needs confirmation", "Sync trouble"). UPPERCASE only for tracked kickers/eyebrows and the wordmark.
- **Say what, why, who decides.** Every state explains itself: *"You are offline. You can keep working — 2 changes will sync when you are back."* Errors say what is safe: *"Your draft is safe on this device."*
- **Name the source.** "Institution verified", "Imported", "Student entered", "Estimated", "AI-assisted", "Needs review". Never "verified" unless it was.
- **Consequences before confirm:** What happens / Exactly what / What stays the same / Taking it back. *"This can be reversed on request: ask the Registrar before Oct 20."*
- **Respectful, not celebratory:** no streaks, ranks, exclamation marks or confetti. *"Registration submitted. Next: review your fall schedule."*
- **Never overclaim:** don't call Semester the official gradebook, registrar or payment system; label demo data "Sample".
- **Numbers:** tabular figures; dates as "Thu 9 Oct · 23:59"; middle dot `·` separates metadata.
- **Emoji:** never. Unicode glyphs are used as status marks: ◆ official, ○ yours, ✦ AI, ⇄ connected/conflict, ? review, ! attention, ✓ done, ⊘ offline, … working, ◌ sample.

VISUAL FOUNDATIONS

- **Colour:** Ink (#090a0e family) for all chrome — rail, tab bar, console environment bar. Parchment (#f4f1ea / panel #fbf9f4) for content. Dark mode = Ink content via `[data-theme="dark"]` (tonal elevation: surfaces lighten as they rise). Semester blue 700 `#416180` is the one action/fill colour on light, 400 `#94bce3` on dark. Brass is an editorial

accent only. Source/status hues are desaturated stones, used as ink + hairline + 8% wash — never as large fills.

- **Type:** Barlow (body 16/1.55, controls 14), Barlow Condensed 600 (headings, buttons, labels, -0.015em), Cinzel (editorial display, 42–56px, rare), system mono (IDs, timestamps). Kickers: 10.5px, 0.08em tracked caps, secondary ink. Body measure ≈ 66ch.
- **Spacing:** density-aware 2px steps `--sp-1…7` (2–16px) inside components; 8px grid `--space-*` for layout. Page padding 18 / 20 / 30 by breakpoint. 44px default targets, 24px floor in crowded composites.
- **Backgrounds:** flat grounds only. No photography backgrounds, no gradients except the brushed sterling text gradient on the wordmark/chrome text. No patterns, textures or illustrations.
- **Borders & cards:** hierarchy by hairline first (9 / 16 / 34% ink), surface step second, shadow third. Cards: 1px border, 6px radius, 16px padding, no shadow, never nested. A card groups a *decision*, not a paragraph.
- **Corners:** "Drawn" 3 / 6 / 10px (control / card / sheet). Pills only for floating bars; chips are 3px.
- **Shadows:** light — ink-tinted `0 1px 2px` / `0 3px 10px` / `0 12px 32px`; dark — `--lift-1…3`. Used for floating and modal layers only.
- **Transparency & blur:** only on the tab bar and focus bar (14px blur), always with an opaque fallback under reduced transparency, increased contrast, Low stimulation and forced colours. Scrims are 50% Ink.
- **Motion:** 130ms hover/press, 180ms state change, 240ms panel, 280ms sheet; ease `cubic-bezier(.22,1,.36,1)`. Sheets rise 8px + fade. No bounce, parallax, shimmer, spinners or looping decoration. All zeroed by `prefers-reduced-motion` and `data-calm`.
- **Hover:** a blue 8% wash on buttons/rows, hairline darkens on inputs, text steps from secondary → primary. **Press:** 14% wash or one ramp step darker on primary. No scale/shrink.
- **Focus:** 2px blue-600 outline, 2px offset; selected list rows use a 3px leading edge + weight, never colour alone.
- **Imagery:** the only brand image is the lit silver mark on Ink (cool, metallic, no grain). No stock photography or AI imagery in-product.
- **Layout:** fixed dark side rail (248px) on desktop, labelled bottom tab bar on mobile, persistent system context bar with ⌘K search on top. Operational content ≤1180px; one H1 per screen.

ICONOGRAPHY

- The app ships its **own inline icon set** (`app/src/components/icons.data.ts`): Lucide-like, 24×24 grid, **1.5px stroke, round caps/joins, no fills** (one exception: the favourite star fills when on). 108 glyphs, copied programmatically to `assets/icons/*.svg` and to `components/icons/iconShapes.js` for the `<Icon>` component.
- Default size 19px in rows/rails, 21px in the tab bar, 15–17px inside buttons. Always `currentColor`.
- `assets/icons-sprite.svg` is the app's public sprite; logo files: `assets/logo-horizontal(-reversed).svg`, `logo-stacked(-reversed).svg`, `logo-brass.svg`, `logo-mono-black/white.svg`, `favicon.svg`, `logo-cobrand.svg` (see Brand → Logo system), `assets/logo-mark.svg` (flat, currentColor, generated from `mark.data.ts`), `assets/logo-mark-tile.svg` / `-maskable.svg` / `-512.png` (lit silver app icons), `assets/logo-mark-silver.png` (uploaded render).
- Unicode glyphs carry status (see Content). Emoji are never used. Icon-only controls always have an accessible name. If a glyph you need is missing, use the matching **Lucide** icon at stroke-width 1.5 (CDN) and flag it.

Components

Sources in `components/<group>/Name.jsx`; each component's guide and types in `components/<Name>/README.md` + `components/<Name>/<Name>.d.ts`, styled by `components/bundle.css`. Namespace: `window.SemesterDesignSystem_e9f3ba`.

- **core:** Button, IconButton, Icon, Wordmark (+ Mark), SectionLabel, Avatar
- **forms:** TextField, Select, Segmented, Choice, Switch, Combobox, DateField, VisibilityPicker
- **trust:** SourceBadge, SourceLine, StatusChip, ProvenanceChips, AIResponse (shared `vocabulary.js`)
- **feedback:** EmptyState, LoadingState, ErrorState, SuccessState, ProgressState, StepStatus, PermissionNotice, OfflineStrip, SaveState, Notice
- **surfaces:** ObjectCard, ContextBar, NextSteps, ActionPreview, Dialog, ScreenGuide, OpenIn, QuickActions
- **overlays:** Sheet (+ SourceDetails), UndoToast, FocusBar
- **navigation:** SideRail, TabBar, SystemContextBar, CommandPalette
- **data:** DataTable, Fields, DecisionTrail, MetricTile, ProvenanceChart
- **governance:** PolicyBadge, ConsentBadge, HealthBadge, ApprovalBanner, ErrorSummary, SupportHandoff, PageHeader

Intentional additions (no single source file): `AIResponse` (combines the app's AI disclosure rules into one frame), `Switch` / `Choice` (the app uses native inputs in `.visibility-choice`), `SideRail` (generalises `desk/Sidebar.tsx`), `CommandPalette` (from `unity/Combobox` styling + the brief's ⌘K), `DecisionTrail` (from `unity/DecisionTrail` + console approvals), `SourceLine` (the brand platform's proposed signature device, §2.6).

UI kits

All views follow the codebase's seven navigation areas (`app/src/lib/navareas.ts`): **Today** (what now?), **Plan** (what's coming?), **Learn** (how do I get the work done?), **Help** (who can help?), **Campus** (what's happening, where?), **Progress** (how am I doing?), **You** (settings). Every one of the 64 destinations in `app/src/screens.tsx` appears in the student App directory; 26 are fully recreated, the rest open an outline screen with their area and primary action.

- `ui_kits/student/` — Student (desktop). Today, Notifications · Calendar, Registration, Career, Billing · Courses, Grades, Study, Ask Semester, Write, Sheets, Decks · Support, Family · Campus, Maps, Mail, Community · Me, Degree, Advising · Settings (Appearance), Connect, Privacy & data · App directory.
- `ui_kits/staff/` — Faculty gradebook, TA grading queue, Advisor, Registrar desk, Finance student accounts, Student affairs support requests.
- `ui_kits/console/` — Institution console: Overview, Approvals, Support queue, Integrations, Security operations, Audit log.
- `ui_kits/external/` — Applicant, Guardian (family portal), Alumni, Employer, Marketplace partner.
- `ui_kits/mobile/` — Phone-width app on Parchment and Ink with the five-destination tab bar.

Audience coverage: students ✓ · applicants ✓ · faculty ✓ · TAs ✓ · advisors ✓ · registrars ✓ · finance ✓ · student affairs ✓ · IT/security ✓ · support/ops ✓ · guardians ✓ · alumni ✓ · employers ✓ · marketplace partners ✓.

AI Workflow Router & Benchmark Lab

- `ui_kits/workflow-lab/` — interactive prototype (Internal tools card): 6-input router with forced-v0 rules, confidence, reasons, limits, escalation, prompts, GitHub plan, Supabase prompt, MD/JSON export; sortable 16-column feature matrix; 12 canonical workflows; benchmark grid with persistence tests, persistence ceiling, 0–5 grading, weighted /5 and /100 scores, leaderboard, CSV/Markdown export. Runs are saved on this device only and labelled prototype mode.
- `handoff/workflow-router-app/` — the production app source: Next.js App Router + TypeScript strict + Tailwind + Supabase (@supabase/ssr, magic link, middleware refresh), Zod, Server Actions, export route handlers, migrations `001_core_workflow_router.sql` / `002_benchmark_grading.sql` with org-scoped RLS (owner/admin/member/viewer), `seed.sql` with the 12 workflows, honest manual adapters, persistence + source-structure probes, vitest suites and an RLS test plan. Code files carry `.txt`; run `install.sh`. Not yet typechecked, linted, tested or built.

Master control & company site

- `ui_kits/master-control/` — the hub: product front end, console & back end, business OS, company site & marketing, design system & engineering — every kit, deck, document and hand-off one click away.
- `ui_kits/company-site/` — homepage following the brand platform: hero, "status is part of the product" map (use now / pilot / later), value props, institutions, trust (status, not certifications), footer.

Role lens

- `ui_kits/role-lens/` — every one of the 64 screens through 14 roles (student, applicant, guardian, alumni, faculty, TA, advisor, registrar, finance, student affairs, IT/security, support, employer, partner): access level (full · scoped · shared · read · audited · none), how the screen renders for that role, and its backend (sources, tables, RLS, API). Design-level model in `roles.js`; verify against migrations.

Business workspace

- `ui_kits/business/` — gap analysis, sortable launch tracker (56 items, CSV), 90-day roadmap, pilot/contract checklist, risk register, hiring plan, fundraising readiness; `business-data.js` is the single record. `Operating Pack.html` is the printable document.
- Decks as templates: `templates/investor-deck`, `templates/institution-deck`, `templates/internal-plan-deck`.
- Inputs: `uploads/Business Master Checklist…pdf`, `uploads/now what else…pdf`, founder answers (LLC, solo, bootstrapping, signed paid pilot with DPA/FERPA, HECVAT Full, SAML SSO, VPAT, insurance; Vercel + Supabase; AI never sees student records).

Claude Code toolkit (`handoff/claude-code-toolkit/`)

Repo-ready files that wire this system into Claude Code + Figma MCP without changing the app's architecture: two skills (`audit-semester-design-sync` , `build-semester-ui`), `.mcp.json` , `docs/FIGMA-MAPPING.md` (collections, 49 semantic + 12 primitive variables, 11 components, and this project's proposal tokens marked as *not in code*), a zero-dependency audit + report script, focused vitest proofs, and the three `design-system:*` package scripts. `PROMPTS.md` has the exact Claude Code commands. Every mapped path was checked against `app/design-tokens/semester.tokens.json` .

Design System tab, group by group

Each group in the tab lists what it holds, where it lives, and the rule to keep.

Foundations

- **Colors**: `guidelines/colors-*.html` , `dataviz.html` . Ink chrome, Parchment content, Semester blue 700/400, metals, source semantics on both grounds, provenance-by-pattern data viz. *Rule:* colour is never the only carrier.
- **Type**: `guidelines/type-*.html` . Barlow body, Barlow Condensed headings, Cinzel display (rare), mono for IDs. *Rule:* sentence case, 66ch measure, tabular figures.
- **Spacing**: `guidelines/spacing.html` , `archived/guidelines/radii.html` , `layout.html` , `states-interaction.html` . 2px density steps, 8px grid, corners 3/6/10, breakpoints, hover/press/focus. *Rule:* no scale or bounce on press.
- **Brand**: `components/LogoSystem/preview.html` , `components/LogoMark/preview.html` , `components/WordmarkLockup/preview.html` , `components/Iconography/preview.html` . Mark, wordmark, **logo system** (horizontal, stacked, reversed, brass, mono, favicon, co-brand lockup in `assets/logo-*.svg`), icon set. *Rule:* Semester leads in co-brand lockups, partner logo at equal cap height, never recolour the mark.

Components (`components/<Name>/` , namespace `window.SemesterDesignSystem_e9f3ba`)

- **Actions**: Button, IconButton, QuickActions, ActionPreview. Primary = one per view, consequence copy before confirm.
- **AI**: AIResponse, ProvenanceChips. Always labelled AI-assisted, sources shown, human confirms the action.
- **Data**: DataTable, Fields, DecisionTrail, MetricTile, ProvenanceChart.
- **Forms**: TextField, Select, Segmented, Choice, Switch, Combobox, DateField, VisibilityPicker.
- **Glyph**: Icon, Mark, Wordmark. 1.5px stroke, `currentColor` , accessible names on icon-only controls.
- **Navigation**: SideRail, TabBar, SystemContextBar, CommandPalette (⌘K).
- **States**: Empty, Loading, Error, Success, Progress, StepStatus, Permission, Offline, SaveState, Notice, UndoToast.
- **Surfaces**: ObjectCard, ContextBar, NextSteps, Dialog, Sheet, FocusBar, ScreenGuide, OpenIn.
- **Trust & provenance**: SourceBadge, SourceLine, StatusChip. The official / connected / personal / AI distinction lives here.
- **Patterns** (`guidelines/pattern-*.html`): AI answer anatomy, provenance ladder, consent & sharing, consequential actions.

Product

- **Student app**: `ui_kits/student/`. All 64 destinations across Today, Plan, Learn, Help, Campus, Progress and You, with Tweaks and live Ask Semester.
- **Mobile**: `ui_kits/mobile/`. Five-destination tab bar on Parchment and Ink.
- **Product**: `ui_kits/community/`, the community and live marketplace: feed, groups, events, Q&A, clubs and leaders, mentorship, campus marketplace with live drops, messages, privacy and safety. Campus-verified only; never touches student records.
- **Staff workspaces**: `ui_kits/staff/`. Faculty, TA, advisor, registrar, finance, student affairs.
- **Institution console**: `ui_kits/console/`. Overview, approvals, support, integrations, security, audit.
- **External audiences**: `ui_kits/external/`. Applicant, guardian, alumni, employer, marketplace partner.
- **Forms & interfaces**: `ui_kits/forms/`. 12 working forms, role capabilities, API, command and event contracts.
- **Wireframes**: `ui_kits/wireframes/`. Every screen × role in low-fi.

Maps of the whole

- **Master control**: `ui_kits/master-control/`. The hub; every kit, deck, document and hand-off is one click away. *Rule:* every new file gets linked here.
- **Master catalog**: `ui_kits/master-catalog/`. 51 roles, 673 screens, 319 workflow steps (17 workflows), 88 systems and 82 docs, each with a status.
- **Atlas**: `ui_kits/atlas/`. Each role's features, screens and functions.
- **Maps card**: `guidelines/maps-of-the-whole.html`. **Documents card**: `guidelines/company-documents.html` (Business group). **Iconography rules**: `guidelines/iconography-rules.html` (Brand group).
- **Role lens**: `ui_kits/role-lens/`. Access level, rendering, tables, RLS and API per role × screen.

Architecture

- **Architecture / Education OS architecture**: `ui_kits/architecture/`. OS map against legacy systems, full stack and data (20 tables), CQRS read-model pipeline, lag simulator, AI gateway, migration phases. *Rule:* the institution stays the system of record; AI never receives student records.

Company

- **Business**: `ui_kits/business/`. Workspace (gap analysis, launch tracker, roadmap, pilot checklist, risks, hiring, fundraising), Operating Manual, Operating Pack, Market Research, Vanderbilt one-pager, invite email, campus map.
- **Operations**: `ui_kits/ops/`. 12 consoles: social media control, finance, marketing, sales and RevOps, customer success, support, trust & safety plus community and marketplace ops, legal and compliance, people, IT and security, data, partnerships. Role-gated. *Rule:* everything is labelled sample data and contains no student PII.
- **Investor**: `ui_kits/investor/`. Investor and board portal.
- **Company site**: `ui_kits/company-site/`. Home, institutions, pilot, pricing, trust, company.
- **Compliance**: `ui_kits/compliance/`. WCAG 2.2, FTC proof rules, release gate. *Rule:* state your status; never claim a certification you don't hold.

- **Product brief package**: `docs/brief/`. Index, one-pager, FAQ. Overview deck and role sheets are next.

- **Architecture cards**: `guidelines/arch-*.html` — stack layers, read-model pipeline & freshness, AI gateway, environments.

Tooling

- **Tooling cards**: `guidelines/tooling-*.html` — Claude Code toolkit, semester-core, semester-platform, AI workflow router & benchmark lab.
- **Internal tools**: `ui_kits/workflow-lab/`, the AI workflow router and benchmark lab.
- **Tooling**: `handoff/claude-code-toolkit/`, `handoff/semester-core/`, `handoff/workflow-router-app/`. See `handoff/README.md`.

Templates (`templates/`, entry `<Slug>.dc.html` + `ds-base.js`)

- Decks: company-deck (14 slides), investor-deck, institution-deck, internal-plan-deck.
- Screens: student-screen (rail + context bar + main/aside), console-screen (environment bar + DataTable + DecisionTrail).
- Reports: pilot results report.

What's next

See `NEXT-SESSION.md` for the ordered backlog: render every missing screen, capability map, workflow runner, site operations, the full public site, personalization capabilities, formal documents, the rest of the brief package, and the PPTX exports.

Index

- `styles.css` — entry; `@import`s only.
- `tokens.json` (compiled to `tokens.css`) and `components/bundle.css`; the original `tokens/` stylesheets are in `archived/tokens/`.
- `fonts/` — Barlow 400/500/700, Barlow Condensed 400/600, Cinzel (OFL).
- `assets/` — logos, app icons, `icons/` (108 SVGs), sprite.
- `guidelines/` — specimen cards (Colors, Type, Spacing, Brand, Patterns) and deep-dive docs:
- `01-provenance-and-trust.md` — the ladder, made vs AI, where cues go
- `02-ai-governance.md` — Ask Semester, AIResponse anatomy, course policy
- `03-consequential-actions.md` — undo vs preview vs two-person duty
- `04-consent-and-family.md` — default private, one decision at a time, access history
- `05-institutional-surfaces.md` — context bars, tables, authority scope
- `06-accessibility.md` — contrast, structure, reader settings, brand-exempt zones
- `07-layout-and-responsive.md` — tiers, shell, page grid, destinations
- `08-content-templates.md` — status sentence, empty/error/success, incidents, onboarding
- `components/`, `ui_kits/` — see above.

- `assets/notes/SKILL.from-standalone.md`, `github.md`, `thumbnail.html`.

Caveats: Atkinson Hyperlegible (the app's legibility body option) and Latin-Extended font subsets were not copied. The production app offers 13 grounds × 11 accents; only Ink and Parchment ship here.

Where the system lives

- **Tokens:** `tokens.json` is the one source for colours, type, spacing, radii, shadows, sizes, layers, leading and tracking; `tokens.css` is generated from it. Values that are not tokens (the sterling gradient, calc type scales, density steps, motion) and the base and component styles are in `components/bundle.css`.
- **Components:** the bundle is `components/bundle.js` (namespace `window.SemesterDesignSystem_e9f3ba`); each component has `components/<Name>/README.md`, `<Name>.d.ts` and a live `preview.html`; all types together are in `components/index.d.ts`.
- **Moved aside:** the old stylesheets, the original `_ds_bundle.js`, group cards, `_ds_manifest.json` and specimen pages now covered by tokens are in `archived/`.

Design tokens

Content defaults to Parchment; [data-theme="dark"] flips content to Ink. Chrome (rail, tab bar, console header) is always Ink. Source and status hues always travel with a word and a glyph.

Colour — primitives

	TOKEN	VALUE	USAGE
	`ink-void`	`#040507`	Ink ground ramp: near-black, cooled. Default dark ground and always-dark chrome.
	`ink-bg`	`#090a0e`	Ink ground ramp: near-black, cooled. Default dark ground and always-dark chrome.
	`ink-panel`	`#12141a`	Ink ground ramp: near-black, cooled. Default dark ground and always-dark chrome.
	`ink-hero`	`#191c23`	Ink ground ramp: near-black, cooled. Default dark ground and always-dark chrome.
	`ink-raise`	`#22262f`	Ink ground ramp: near-black, cooled. Default dark ground and always-dark chrome.
	`ink-fg`	`#eceef2`	Ink ground ramp: near-black, cooled. Default dark ground and always-dark chrome.
	`parchment-void`	`#e8e4dc`	Parchment ground ramp: warm off-white. Default light content ground.
	`parchment-bg`	`#f4f1ea`	Parchment ground ramp: warm off-white. Default light content ground.
	`parchment-panel`	`#fbf9f4`	Parchment ground ramp: warm off-white. Default light content ground.
	`parchment-hero`	`#ffffff`	Parchment ground ramp: warm off-white. Default light content ground.
	`parchment-raise`	`#ffffff`	Parchment ground ramp: warm off-white. Default light content ground.
	`parchment-fg`	`#1b1a17`	Parchment ground ramp: warm off-white. Default light content ground.
	`graphite-bg`	`#16181c`	Graphite: alternate dark ground.
	`graphite-panel`	`#1f2229`	Graphite: alternate dark ground.
	`graphite-raise`	`#333843`	Graphite: alternate dark ground.
	`sterling`	`#d4d9e2`	Semester color.
	`sterling-bright`	`#f6f8fb`	Sterling metal, for the mark and chrome highlights.
	`sterling-deep`	`#9ca3b2`	Sterling metal, for the mark and chrome highlights.
	`sterling-shade`	`#4c5561`	Sterling metal, for the mark and chrome highlights.
	`brass`	`#d8c79a`	Semester color.

	TOKEN	VALUE	USAGE
	`brass-bright`	`#f2e7c8`	Brass metal, for editorial accents.
	`brass-deep`	`#b09f76`	Brass metal, for editorial accents.
	`brass-shade`	`#695a33`	Brass metal, for editorial accents.
	`blue-100`	`#eef6ff`	Semester blue (the Industry ramp): interactive focus and trust.
	`blue-200`	`#d6ebff`	Semester blue (the Industry ramp): interactive focus and trust.
	`blue-300`	`#b5d9fd`	Semester blue (the Industry ramp): interactive focus and trust.
	`blue-400`	`#94bce3`	Semester blue (the Industry ramp): interactive focus and trust.
	`blue-500`	`#749dc4`	Semester blue (the Industry ramp): interactive focus and trust.
	`blue-600`	`#597ea3`	Semester blue (the Industry ramp): interactive focus and trust.
	`blue-700`	`#416180`	Semester blue (the Industry ramp): interactive focus and trust.
	`blue-800`	`#2c455d`	Semester blue (the Industry ramp): interactive focus and trust.
	`blue-900`	`#1d2d3d`	Semester blue (the Industry ramp): interactive focus and trust.
	`neutral-100`	`#f5f5f8`	Industry neutral ramp.
	`neutral-200`	`#e7e7ea`	Industry neutral ramp.
	`neutral-300`	`#d4d4d7`	Industry neutral ramp.
	`neutral-400`	`#b7b7ba`	Industry neutral ramp.
	`neutral-500`	`#98989b`	Industry neutral ramp.
	`neutral-600`	`#7a7a7d`	Industry neutral ramp.
	`neutral-700`	`#5d5d60`	Industry neutral ramp.
	`neutral-800`	`#424244`	Industry neutral ramp.
	`neutral-900`	`#2b2b2d`	Industry neutral ramp.
	`official-ink`	`#2c455d`	Official status primitive: text on light grounds.
	`official-tint`	`#b5d9fd`	Official status primitive: text on dark grounds.
	`connected-ink`	`#2d5c61`	Connected status primitive: text on light grounds.
	`connected-tint`	`#9fcbcd`	Connected status primitive: text on dark grounds.
	`personal-ink`	`#4a4b4f`	Personal status primitive: text on light grounds.
	`personal-tint`	`#b9b9bd`	Personal status primitive: text on dark grounds.
	`ai-ink`	`#544a7f`	Ai status primitive: text on light grounds.
	`ai-tint`	`#bcb2df`	Ai status primitive: text on dark grounds.

	TOKEN	VALUE	USAGE
	`pending-ink`	`#7a5310`	Pending status primitive: text on light grounds.
	`pending-tint`	`#e2c27a`	Pending status primitive: text on dark grounds.
	`risk-ink`	`#a3301f`	Risk status primitive: text on light grounds.
	`risk-tint`	`#f0907a`	Risk status primitive: text on dark grounds.
	`success-ink`	`#3e6141`	Success status primitive: text on light grounds.
	`success-tint`	`#a9c9a3`	Success status primitive: text on dark grounds.
	`chrome-text-secondary`	`rgba(236,238,242,.64)`	Chrome (rail, tab bar, console bar): always dark.
	`chrome-line`	`rgba(236,238,242,.10)`	Chrome (rail, tab bar, console bar): always dark.
	`chrome-active`	`rgba(148,188,227,.14)`	Chrome (rail, tab bar, console bar): always dark.
	`scrim`	`rgba(4,5,7,.5)`	Backdrop behind sheets and dialogs.

Colour — semantic (light · dark)

LIGHT · DARK	TOKEN	LIGHT	DARK	USAGE
	`surface-sunken`	`{parchment-void}`	`{ink-void}`	Wells and recessed areas below the canvas.
	`surface-canvas`	`{parchment-bg}`	`{ink-bg}`	Page background.
	`surface-base`	`{parchment-panel}`	`{ink-panel}`	Panels and cards on the canvas.
	`surface-quiet`	`#f1ede4`	`#161820`	Quiet fills: hover rows, secondary panels.
	`surface-raised`	`{parchment-hero}`	`{ink-hero}`	Raised surfaces: inputs, popovers, hero cards.
	`surface-overlay`	`{parchment-raise}`	`{ink-raise}`	Overlays: menus, sheets, dialogs.
	`text-primary`	`{parchment-fg}`	`{ink-fg}`	Body and heading text on every surface-* token.
	`text-secondary`	`rgba(27,26,23,.68)`	`rgba(236,238,242,.64)`	Supporting text on surface-canvas, surface-base and surface-raised.
	`text-tertiary`	`rgba(27,26,23,.52)`	`rgba(236,238,242,.42)`	Placeholders and least-important meta text. Contrast note: about 3.4–3.7:1 on the surfaces in both themes, below 4.5:1 for body text; keep it to large text (24px+) or non-essential hints, and use text-secondary for anything a person must read.
	`text-disabled`	`rgba(27,26,23,.52)`	`rgba(236,238,242,.42)`	Disabled control text.
	`text-on-action`	`#ffffff`	`#08090c`	Text on action-primary fills.
	`text-link`	`{blue-700}`	`{blue-300}`	Inline links on surface-* grounds.

LIGHT · DARK	TOKEN	LIGHT	DARK	USAGE
	`border-subtle`	`rgba(27,26,23,.09)`	`rgba(236,238,242,.08)`	Hairline dividers.
	`border-default`	`rgba(27,26,23,.16)`	`rgba(236,238,242,.14)`	Default control and card borders.
	`border-strong`	`rgba(27,26,23,.34)`	`rgba(236,238,242,.30)`	Hover borders and emphasized edges.
	`action-primary`	`{blue-700}`	`{blue-400}`	Primary button fill; one per decision region.
	`action-primary-hover`	`{blue-800}`	`{blue-300}`	Primary button hover.
	`action-primary-press`	`{blue-900}`	`{blue-500}`	Primary button pressed.
	`action-secondary`	`rgba(65,97,128,.08)`	`rgba(148,188,227,.10)`	Secondary button hover wash.
	`action-secondary-hover`	`rgba(65,97,128,.14)`	`rgba(148,188,227,.18)`	Secondary button pressed wash.
	`focus-color`	`{blue-600}`	`{blue-400}`	Focus ring on every interactive element.
	`accent-editorial`	`{brass-shade}`	`{brass}`	Editorial accent (kickers, rules) in brand moments.
	`source-official`	`{official-ink}`	`{official-tint}`	Text and glyph color for the official source state. Every state also carries a word and a glyph.
	`source-official-line`	`rgba(44,69,93,.45)`	`rgba(181,217,253,.40)`	Border for the official source state. Every state also carries a word and a glyph.
	`source-official-wash`	`rgba(44,69,93,.08)`	`rgba(181,217,253,.08)`	Background wash for the official source state. Every state also carries a word and a glyph.
	`source-connected`	`{connected-ink}`	`{connected-tint}`	Text and glyph color for the connected source state. Every state also carries a word and a glyph.
	`source-connected-line`	`rgba(45,92,97,.45)`	`rgba(159,203,205,.40)`	Border for the connected source state. Every state also carries a word and a glyph.
	`source-connected-wash`	`rgba(45,92,97,.08)`	`rgba(159,203,205,.08)`	Background wash for the connected source state. Every state also carries a word and a glyph.
	`source-personal`	`{personal-ink}`	`{personal-tint}`	Text and glyph color for the personal source state. Every state also carries a word and a glyph.
	`source-personal-line`	`rgba(74,75,79,.40)`	`rgba(185,185,189,.34)`	Border for the personal source state. Every state also carries a word and a glyph.
	`source-personal-wash`	`rgba(74,75,79,.06)`	`rgba(185,185,189,.06)`	Background wash for the personal source state. Every state also carries a word and a glyph.
	`source-ai`	`{ai-ink}`	`{ai-tint}`	Text and glyph color for the ai source state. Every state also carries a word and a glyph.
	`source-ai-line`	`rgba(84,74,127,.45)`	`rgba(188,178,223,.42)`	Border for the ai source state. Every state also carries a word and a glyph.

LIGHT · DARK	TOKEN	LIGHT	DARK	USAGE
	`source-ai-wash`	`rgba(84,74,127,.08)`	`rgba(188,178,223,.09)`	Background wash for the ai source state. Every state also carries a word and a glyph.
	`status-pending`	`{pending-ink}`	`{pending-tint}`	Text and glyph color for the pending status state. Every state also carries a word and a glyph.
	`status-pending-line`	`rgba(160,112,24,.50)`	`rgba(226,194,122,.45)`	Border for the pending status state. Every state also carries a word and a glyph.
	`status-pending-wash`	`rgba(196,146,48,.12)`	`rgba(226,194,122,.09)`	Background wash for the pending status state. Every state also carries a word and a glyph.
	`status-danger`	`{risk-ink}`	`{risk-tint}`	Text and glyph color for the danger status state. Every state also carries a word and a glyph.
	`status-danger-line`	`rgba(163,48,31,.45)`	`rgba(240,144,122,.45)`	Border for the danger status state. Every state also carries a word and a glyph.
	`status-danger-wash`	`rgba(163,48,31,.08)`	`rgba(240,144,122,.09)`	Background wash for the danger status state. Every state also carries a word and a glyph.
	`status-success`	`{success-ink}`	`{success-tint}`	Text and glyph color for the success status state. Every state also carries a word and a glyph.
	`status-success-line`	`rgba(62,97,65,.45)`	`rgba(169,201,163,.42)`	Border for the success status state. Every state also carries a word and a glyph.
	`status-success-wash`	`rgba(62,97,65,.08)`	`rgba(169,201,163,.08)`	Background wash for the success status state. Every state also carries a word and a glyph.
	`status-neutral`	`{text-secondary}`	`{text-secondary}`	Text and glyph color for the neutral status state. Every state also carries a word and a glyph.
	`chrome-surface`	`{ink-bg}`	`{ink-bg}`	Chrome (rail, tab bar, console bar): always dark.
	`chrome-surface-raised`	`{ink-panel}`	`{ink-panel}`	Chrome (rail, tab bar, console bar): always dark.
	`chrome-text`	`{ink-fg}`	`{ink-fg}`	Chrome (rail, tab bar, console bar): always dark.
	`brand-canvas`	`{surface-canvas}`	`{surface-canvas}`	Brand role shared with the company site.
	`brand-surface`	`{surface-base}`	`{surface-base}`	Brand role shared with the company site.
	`brand-ink`	`{text-primary}`	`{text-primary}`	Brand role shared with the company site.
	`brand-muted`	`{text-secondary}`	`{text-secondary}`	Brand role shared with the company site.
	`brand-accent`	`{action-primary}`	`{action-primary}`	Brand role shared with the company site.
	`brand-focus`	`{focus-color}`	`{focus-color}`	Brand role shared with the company site.

Typography

Barlow for product reading, Barlow Condensed for headings, labels and controls, Cinzel for editorial display moments only, system mono for identifiers and timestamps. Sizes scale with —text-scale (see components/bundle.css).

FAMILY	STACK
`display`	'Cinzel','Barlow Condensed',Georgia,serif
`heading`	'Barlow Condensed',system-ui,sans-serif
`body`	'Barlow',system-ui,sans-serif
`mono`	`ui-monospace,'SF Mono',Menlo,Consolas,monospace`

FONT FILE	FAMILY	WEIGHT
`fonts/Barlow-400-latin.woff2`	Barlow	400
`fonts/Barlow-500-latin.woff2`	Barlow	500
`fonts/Barlow-700-latin.woff2`	Barlow	700
`fonts/BarlowCondensed-400-latin.woff2`	Barlow Condensed	400
`fonts/BarlowCondensed-600-latin.woff2`	Barlow Condensed	600
`fonts/Cinzel-latin.woff2`	Cinzel	400 600

Roles

`role-editorial` · 42px / 1.1 · 500 · display
Editorial headlines on the company site; Cinzel moments only.

Your lab report is due Thursday

`role-display` · 26px / 1.15 · 600 · heading
Display numbers and hero headings in product.

Your lab report is due Thursday

`role-page` · 24px / 1.2 · 600 · heading
Page titles (h1).

Your lab report is due Thursday

`role-section` · 20px / 1.2 · 600 · heading
Section headings (h2).

Your lab report is due Thursday

`role-card-title` · 18px / 1.25 · 600 · heading
Card titles.

Your lab report is due Thursday

`role-body` · 16px / 1.55 · 400 · body
Body copy.

Your lab report is due Thursday

`role-control` · 14px / 1.2 · 600 · heading
Button and control labels.

Your lab report is due Thursday

`role-compact` · 12px / 1.45 · 400 · body
Dense rows and compact UI.

Your lab report is due Thursday

`role-label` · 11px / 1.3 · 500 · body
Field labels and meta.

Your lab report is due Thursday

`role-caption` · 11px / 1.45 · 400 · body
Captions.

Your lab report is due Thursday

`role-kicker` · 10.5px / 1.3 · 500 · body
Uppercase kickers above headings.

YOUR LAB REPORT IS DUE THURSDAY

`role-numeric` · 22px / 1.15 · 600 · heading
Metric values (tabular numerals).

Your lab report is due Thursday

Spacing

8px macro grid. The density-aware —sp-1…7 steps stay in components/bundle.css (they multiply by —density).

TOKEN	VALUE		USAGE
`space-1`	8px		8px macro grid for layout gaps and section spacing.
`space-2`	16px		8px macro grid for layout gaps and section spacing.
`space-3`	24px		8px macro grid for layout gaps and section spacing.
`space-4`	32px		8px macro grid for layout gaps and section spacing.
`space-5`	40px		8px macro grid for layout gaps and section spacing.
`space-6`	48px		8px macro grid for layout gaps and section spacing.
`space-8`	64px		8px macro grid for layout gaps and section spacing.

TOKEN	VALUE		USAGE
`space-10`	80px		8px macro grid for layout gaps and section spacing.
`space-12`	96px		8px macro grid for layout gaps and section spacing.
`page-pad`	18px		Page and chrome padding at narrow widths; widens at 760px and 1100px.
`chrome-pad`	18px		Page and chrome padding at narrow widths; widens at 760px and 1100px.

Radius

"Drawn" corners, the app default.

TOKEN	VALUE		USAGE
`r-sm`	3px		Drawn corners: controls.
`r-md`	6px		Drawn corners: cards.
`r-lg`	10px		Drawn corners: surfaces and sheets.
`shape-pill`	999px		Pills and chips.

Shadow

Border first, shadow second.

TOKEN	VALUE		USAGE
`lift-1`	`0 1px 2px rgba(0,0,0,.4)`		Lift on dark grounds. Border first, shadow second.
`lift-2`	`0 1px 2px rgba(0,0,0,.45),0 6px 18px rgba(0,0,0,.35)`		Lift on dark grounds. Border first, shadow second.
`lift-3`	`0 1px 3px rgba(0,0,0,.5),0 12px 36px rgba(0,0,0,.5)`		Lift on dark grounds. Border first, shadow second.
`shadow-sm`	`0 1px 2px rgba(43,43,45,.14)`		Ink-tinted shadow on light grounds. Border first, shadow second.
`shadow-md`	`0 3px 10px rgba(43,43,45,.16)`		Ink-tinted shadow on light grounds. Border first, shadow second.

TOKEN	VALUE	USAGE
`shadow-lg`	`0 12px 32px rgba(43,43,45,.22)`	Ink-tinted shadow on light grounds. Border first, shadow second.

Sizes and targets

TOKEN	VALUE	USAGE
`rail-w`	`248px`	Side rail width.
`rail-w-compact`	`72px`	Collapsed side rail width.
`layout-measure`	`100%`	Default content measure.
`layout-operational`	`1180px`	Max width of operational (console) pages.
`target-min`	`24px`	Smallest hit target.
`target-compact`	`32px`	Compact control height (dense rows).
`target-icon`	`40px`	Icon button size.
`target-primary`	`44px`	Standard control height.
`focus-ring-width`	`2px`	Focus outline width.
`focus-ring-offset`	`2px`	Focus outline offset.
`blur-chrome`	`14px`	Backdrop blur behind translucent chrome.

Layers (z-index)

TOKEN	VALUE	USAGE
`layer-base`	`0`	Stacking order: base.
`layer-raised`	`1`	Stacking order: raised.
`layer-sticky`	`20`	Stacking order: sticky.
`layer-chrome`	`21`	Stacking order: chrome.
`layer-overlay`	`80`	Stacking order: overlay.
`layer-menu`	`90`	Stacking order: menu.
`layer-skip`	`100`	Stacking order: skip.
`layer-curtain`	`1500`	Stacking order: curtain.

Leading (line height)

TOKEN	VALUE	USAGE
`leading-none`	`1`	Line height: none.
`leading-display-xl`	`1.08`	Line height: display-xl.
`leading-display-lg`	`1.1`	Line height: display-lg.
`leading-display`	`1.15`	Line height: display.
`leading-display-sm`	`1.2`	Line height: display-sm.
`leading-display-xs`	`1.25`	Line height: display-xs.
`leading-tight`	`1.3`	Line height: tight.
`leading-tight-plus`	`1.35`	Line height: tight-plus.
`leading-normal`	`1.45`	Line height: normal.
`leading-relaxed`	`1.5`	Line height: relaxed.
`leading-relaxed-plus`	`1.55`	Line height: relaxed-plus.
`leading-loose`	`1.6`	Line height: loose.

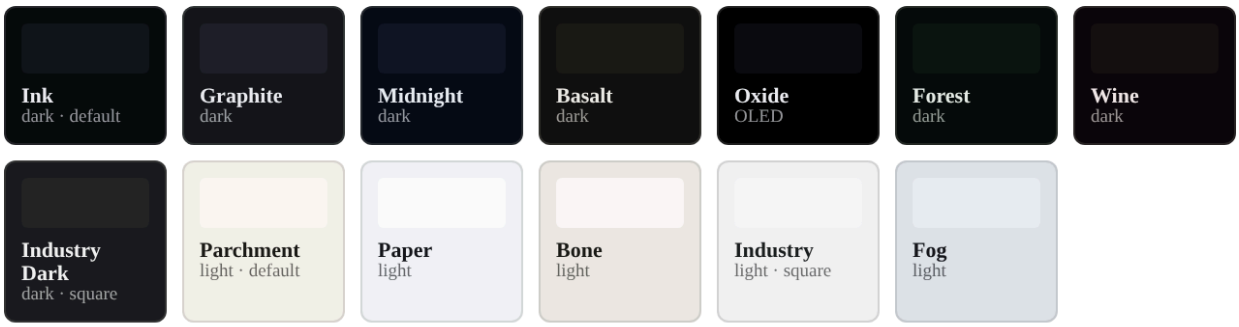
Tracking (letter spacing)

TOKEN	VALUE	USAGE
`tracking-heading`	`-0.015em`	Heading letter spacing.
`tracking-kicker`	`0.08em`	Kicker (uppercase label) letter spacing.
`tracking-caps`	`0.1em`	All-caps letter spacing.
`tracking-wordmark`	`0.22em`	Wordmark letter spacing.

Foundations

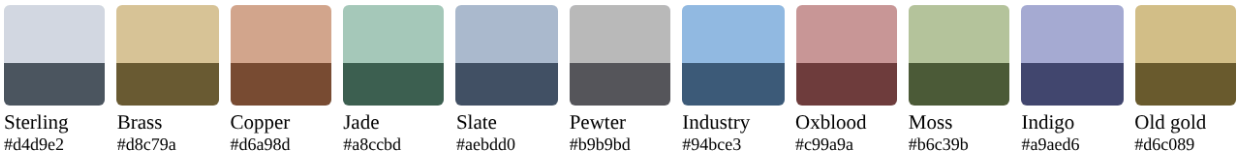
All 13 grounds (reader setting)

The app lets each reader choose a ground; this system ships Ink + Parchment



The 11 metal accents

base (on dark) and shade (text on light) — desaturated metals and stones



Industry → Semester blue in this system. Accent colours identify courses and action; they never encode success or error.

Metals

Sterling (wordmark/chrome), Brass (editorial accent)



Components on Ink

The same components with [data-theme=

!Assignment · Action required

Lab report 3: Enzyme kinetics

◆Official · BIO 210Institution verified by BIO 210

Due Thu 9 Oct · 23:59

Open draft View rubric

Submit Save draft

✦AI-assistedWritten with Semester's assistant. It is not an official answer — check anything you act on.

⇄ConnectedSynced from a connected account or feed. Semester has not checked it with your institution.

!Hold on account

⊘You are offline. You can keep working — 2 changes will sync when you are

Data visualisation encoding

Provenance by pattern: solid, outline, hatched, dotted — survives greyscale

Official / connected	Yours	Estimated / AI-derived	Out of date
data-enc="solid"	data-enc="outline"	data-enc="hatched"	data-enc="dotted"

Full type scale

Every —type-* step with its job; all × —text-scale

--type-2xl	28px	The figure a screen is about
--type-xl	26px	The one heading on a screen
--type-display-lg	24px	Screen title
--type-display	22px	Figure read at a glance
--type-display-sm	20px	Section heading
--type-display-xs	19px	Card or item title
--type-lg	18px	A card’s own title
--type-md	17px	List rows, the thing you tap
--type-base	16px	Body
--type-control	14px	Inputs and buttons
--type-sm-plus	12.5px	A caption with something to say
--type-sm	12px	Captions, second-rank prose
--type-xs	11px	Labels, kickers, meta
--type-2xs-plus	10.5px	Caps kicker over a section

Editorial display — Cinzel

Major statements and editorial moments only

The whole of the semester.

Fall Term · Northfield University

--font-display · Cinzel 400–600 · 42–56px · leading 1.08

Headings — Barlow Condensed

Page, section, card, and control labels

Page title · 24	**Today, Tuesday 7 October**
Display · 26	**Degree progress**
Section · 20	**This week in BIO 210**
Card · 18	**Registration opens tomorrow**

Body — Barlow

Product reading, forms, operational text

Body 16 / 1.55. Your lab report is due Thursday. The methods section is drafted; results and discussion remain. Start with the data table — it is the part the rubric weighs most.

Row 17 — the thing you tap

Control 14 — inputs & buttons

Caption 12 — second-rank prose

Meta 11 — labels and timestamps

Kickers, numerals & mono

Caps kickers, tabular figures, identifiers

Official · Updated today
Begin today

3.62 GPA · 94 / 120 cr

req_7Hk2Q9
2026-10-03 14:02 UTC

Spacing scales

Density-aware 2px steps for UI · 8px grid for layout

sp-1	2px
sp-2	4px
sp-3	6px
sp-4	8px
sp-5	10px
sp-6	12px
sp-7	16px

space-1	8px
space-2	16px
space-3	24px
space-4	32px
space-6	48px
space-8	64px

Density settings

Comfortable 1 · Snug .86 · Tight .74 — scales the 2px steps, never targets

COMFORTABLE · 1

Assignment
Lab report 3

Due Thu · 23:59

SNUG · 0.86

Assignment
Lab report 3

Due Thu · 23:59

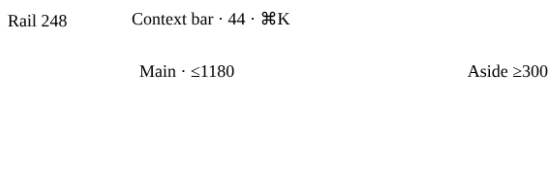
TIGHT · 0.74

Assignment
Lab report 3

Due Thu · 23:59

Layout & breakpoints

Rail + context bar + content; page padding by tier



<760	Tab bar · page-pad 18 · rail 0
760–1099	Compact rail 72 · pad 20
≥1100	Rail 248 · pad 30
READING	66ch measure
OPS MAX	1180px

Motion

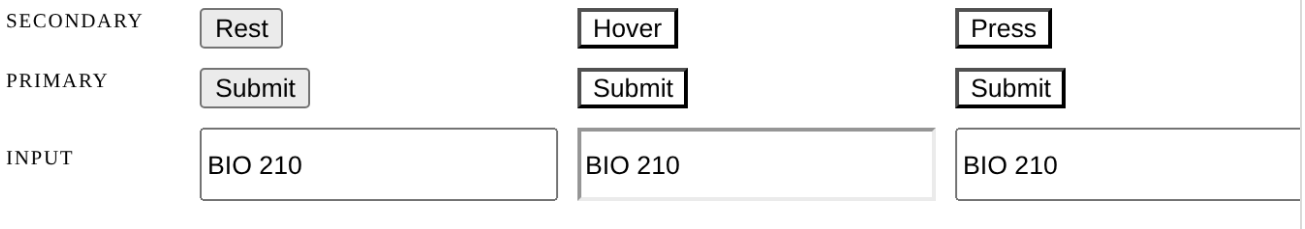
Role durations · zeroed by reduced motion and Less motion

fast	standard	slow	sheet	progress
130ms	180ms	240ms	280ms	180ms linear
hover, press	save, insert	panel	sheet, dialog	progress

ease cubic-bezier(.22,1,.36,1) · emphasized (.2,.8,.2,1) · no bounces, no parallax, no looping decoration

Interaction states

Rest, hover, press, focus, disabled — no scale, no bounce

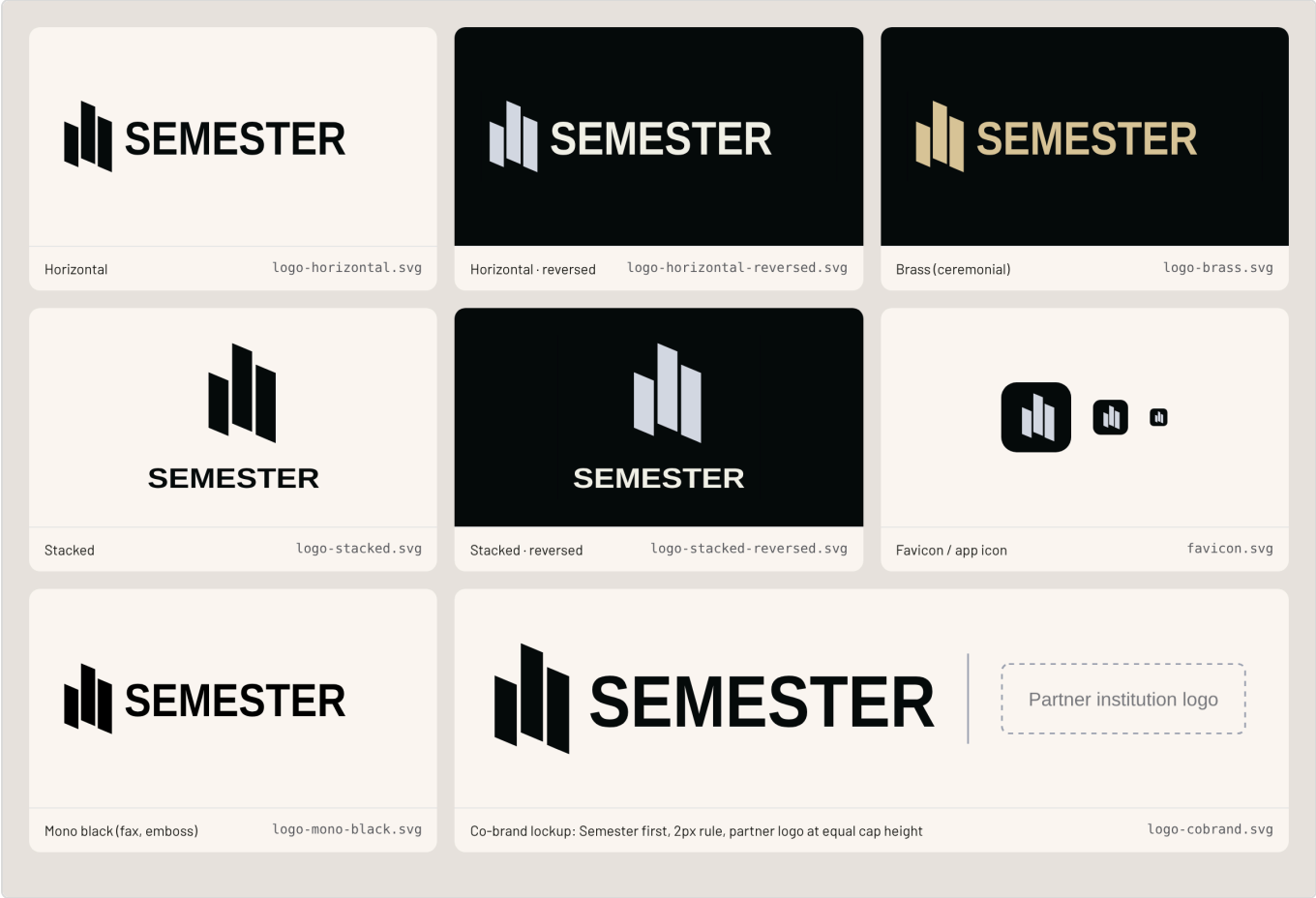


Hover = 8% blue wash · Press = 14% / one ramp step darker · Focus = 2px blue-600 ring, 2px offset · Disabled = 45% opacity + not-allow

Brand

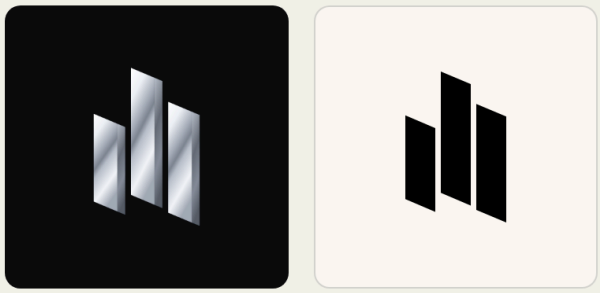
Logo system

Horizontal, stacked, mono, reversed, favicon, co-brand



Logo mark

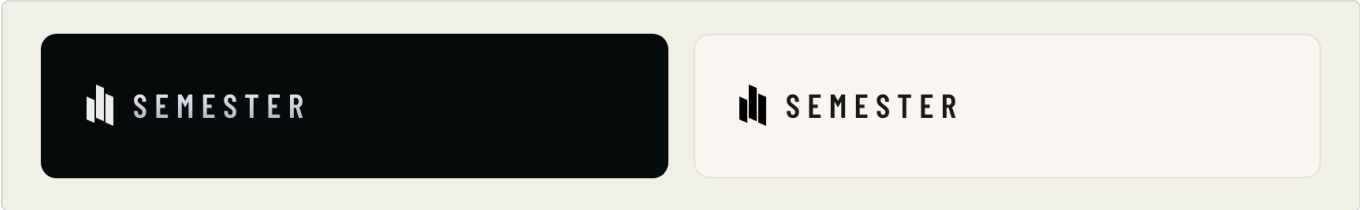
Three slabs — a term rising, cresting, running out



Lit silver on Ink for app icons and splash; flat currentColor everywhere in-app so it follows the ground. Never recolour into the accent, add glow, or place on photography.

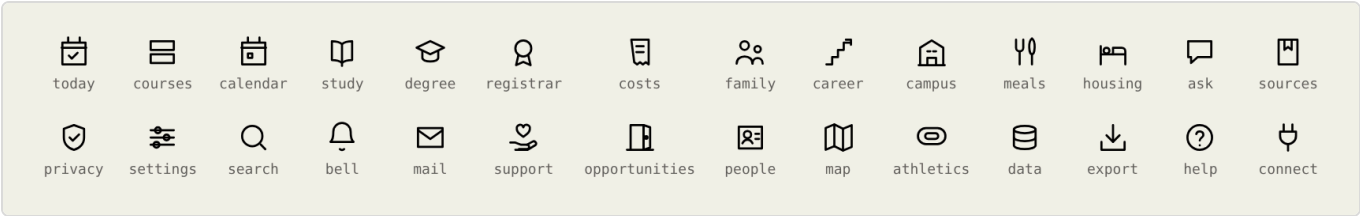
Wordmark lockup

Mark + SEMESTER in Barlow Condensed, tracked



Iconography

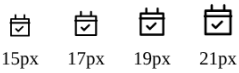
Lucide-like, 24 grid, 1.5 stroke, round caps — the app's own set



Iconography rules

Sizes, stroke, colour, accessible names, status glyphs, files

Sizes



15–17 in buttons · 19 in rows and rails · 21 in the tab bar

Drawing

24×24 grid, 1.5px stroke, round caps and joins, no fills (favourite star fills when on). Always currentColor. 108 glyphs.

Missing glyph
Matching Lucide icon at stroke 1.5, flagged

Status glyphs

◆ official ○ yours ✦ AI

⇄ connected ? review ! attention

✓ done ⊘ offline … working ◌

sample

No emoji. Icon-only controls always have an accessible name.

Voice — prefer / avoid

From docs/BRAND-PLATFORM.md §1.7

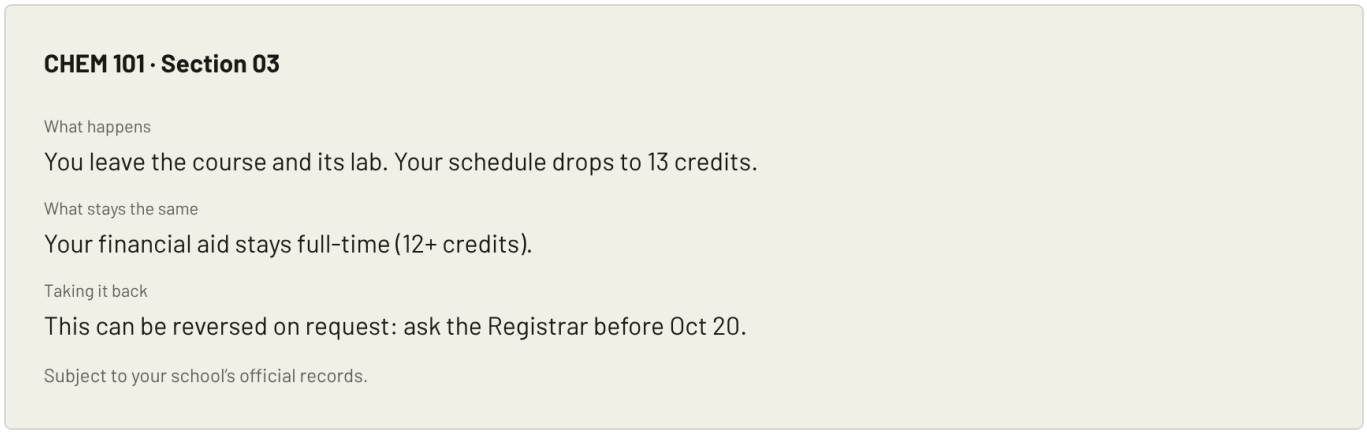
✓ helps you organise · turns … into	✕ revolutionises · transforms · supercharge
✓ alongside your school's systems	✕ replaces · all-in-one · everything school
✓ AI-assisted, source-linked · Ask Semester	✕ AI-powered · intelligent · knows you · magic
✓ built with controls for [named control]	✕ secure · private · compliant · certified
✓ planned · in private beta · in a pilot	✕ launching soon · coming Spring
✓ can · is designed to	✕ guarantees · always · never fails · 100%
✓ student	✕ user · learner
✓ Your Canvas calendar hasn't updated since Tuesday. Deadlines may have moved. Reconnect it.	✕ Oops! Something went wrong 😕

Components

Every component in `components/bundle.js`, rendered from its preview. Each entry gives its guide (README.md) and its props (from `components/index.d.ts`).

Actions

ActionPreview



What is about to happen, said before it happens — What happens / Exactly what / What stays / Taking it back.

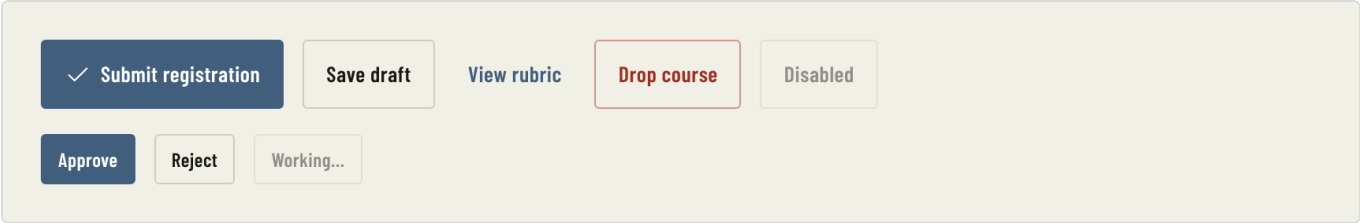
```
<ActionPreview subject="Drop CHEM 101" says="You leave the course and its waitlist." recovery=
{{kind:"request",how:"ask the Registrar before Oct 20."}} />
```

Put inside Dialog for consequential, irreversible or financial actions.

PROPS

```
import * as React from 'react';
/**
 * What is about to happen, said before it happens — What happens / Exactly what / What stays / Taking it back.
 */
export interface ActionPreviewProps { subject?: string; says: React.ReactNode; exactly?: React.ReactNode;
doesNotChange?: React.ReactNode; recovery: { kind: "undo" | "request" | "none"; how?: string }; subjectTo?:
React.ReactNode; provenance?: React.ReactNode; }
export declare function ActionPreview(props: ActionPreviewProps): JSX.Element;
```

Button



Action button — one primary per decision region; 44px tall by default, condensed-caps-free label in Barlow Condensed 600.

```
<Button variant="primary" icon="check">Submit registration</Button>
```

primary (Semester blue fill), secondary (hairline), ghost (text), danger (red ink, hairline — never a red fill). size="sm" = 32px for dense rows.

PROPS

```
import * as React from 'react';
/**
 * Action button — one primary per decision region; 44px tall by default, condensed-caps-free label in Barlow
Condensed 600.
 * @startingPoint section="Actions" subtitle="Action button — one primary per decision region; 44px tall by
default, condensed" viewport="700x400"
 */
export interface ButtonProps { variant?: "primary" | "secondary" | "ghost" | "danger"; size?: "md" | "sm";
icon?: string; iconAfter?: string; block?: boolean; loading?: boolean; disabled?: boolean; children?:
React.ReactNode; onClick?: (e: any) => void; type?: "button" | "submit"; }
export declare function Button(props: ButtonProps): JSX.Element;
```

IconButton



Icon-only control with a required accessible name (also its tooltip).

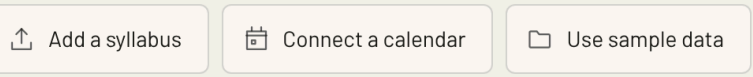
```
<IconButton icon="bell" label="Notifications" />
```

40px (md) / 32px (sm). Never ship an icon-only control without `label`.

PROPS

```
import * as React from 'react';
/**
 * Icon-only control with a required accessible name (also its tooltip).
 */
export interface IconButtonProps { icon: string; label: string; size?: "md" | "sm"; pressed?: boolean; onClick?:
(e: any) => void; }
export declare function IconButton(props: IconButtonProps): JSX.Element;
```

QuickActions



Row of 2–5 labelled starting actions for empty and onboarding states.

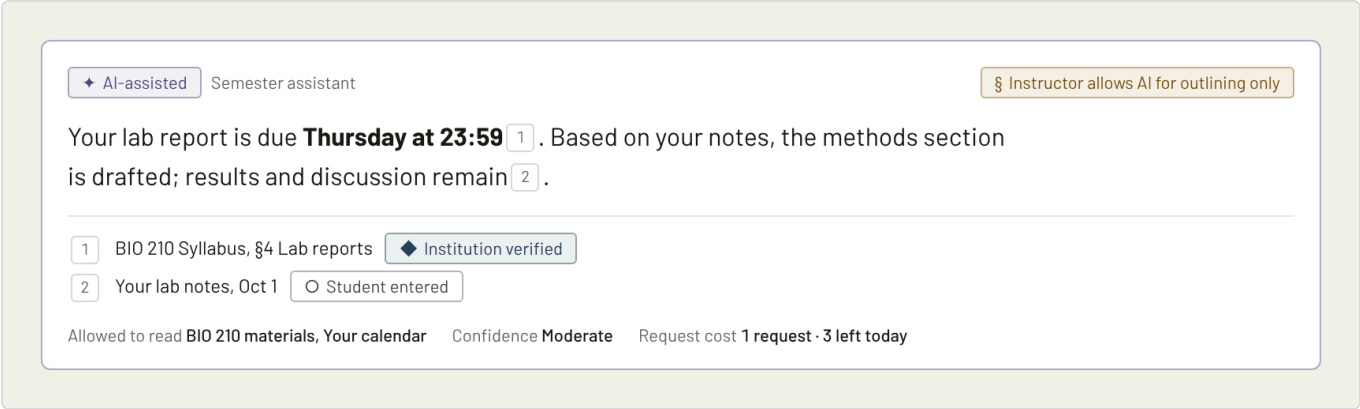
```
<QuickActions actions={[{label:"Add a syllabus",icon:"import"}]} />
```

PROPS

```
import * as React from 'react';
/** Row of 2–5 labelled starting actions for empty and onboarding states. */
export interface QuickActionsProps { actions: { label: string; icon?: string; onClick?: () => void }[]; }
export declare function QuickActions(props: QuickActionsProps): JSX.Element;
```

AI

AIResponse



A governed assistant answer — labelled AI-assisted with data scope, numbered sources, confidence, request cost and policy.

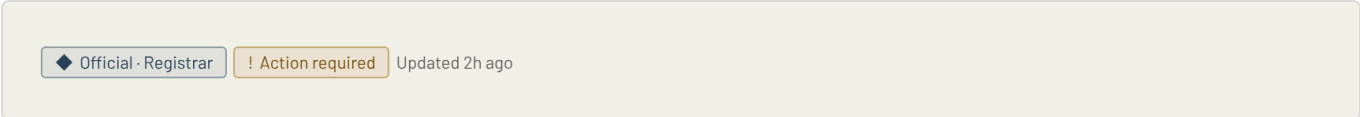
```
<AIResponse scope={["BIO 210 materials"]} sources={[{title:"Syllabus",kind:"official"}]}
confidence="Moderate">…</AIResponse>
```

Use `.ai-cite` spans for inline [n] citations. Consequential actions go in `actions` and must open a Dialog/ActionPreview before running.

PROPS

```
import * as React from 'react';
/**
 * A governed assistant answer — labelled AI-assisted with data scope, numbered sources, confidence, request
cost and policy.
 * @startingPoint section="AI" subtitle="A governed assistant answer — labelled AI-assisted with data scope,
numbered sou" viewport="700x400"
 */
export interface AIResponseProps { children: React.ReactNode; scope?: string[]; sources?: { title: string;
kind?: string }[]; confidence?: string; cost?: string; policy?: string; actions?: React.ReactNode; model?:
string; }
export declare function AIResponse(props: AIResponseProps): JSX.Element;
```

ProvenanceChips



A fact's origin chip plus at most two cues and its age, spoken as one sentence.

```
<ProvenanceChips origin="official" by="Registrar" cues={["action-required"]} age="Updated today"
/>
```

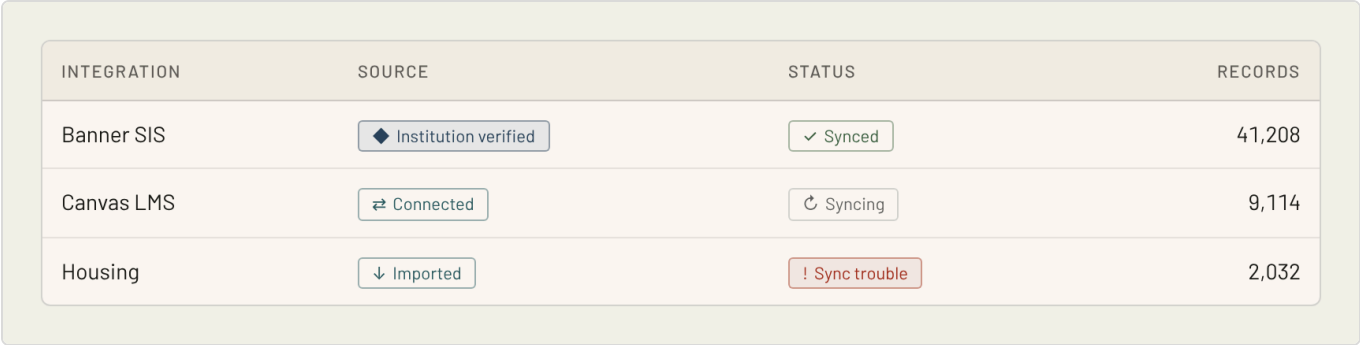
Origin always shows; only two cues render — the rest belong in Source & details.

PROPS

```
import * as React from 'react';
/**
 * A fact's origin chip plus at most two cues and its age, spoken as one sentence.
 */
export interface ProvenanceChipsProps { origin: string; by?: string; cues?: string[]; age?: string; }
export declare function ProvenanceChips(props: ProvenanceChipsProps): JSX.Element;
```

Data

DataTable



Crisp comparison table with caps headers, hairline rows, tabular numerals.

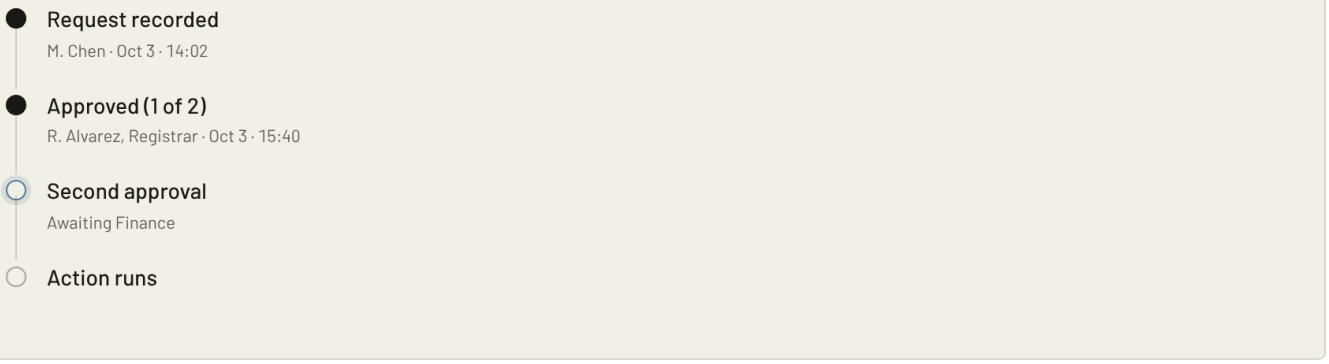
```
<DataTable caption="Holds" columns={[{key:"name",label:"Hold"}]} rows={[{name:"Immunization"}]}
/>
```

Use a list when scanning/acting on rows; a table only to compare columns.

PROPS

```
import * as React from 'react';
/**
 * Crisp comparison table with caps headers, hairline rows, tabular numerals.
 * @startingPoint section="Data" subtitle="Crisp comparison table with caps headers, hairline rows, tabular
numerals." viewport="700x400"
 */
export interface DataTableProps { columns: { key: string; label: string; numeric?: boolean; render?: (row: any)
=> React.ReactNode }[]; rows: any[]; caption?: string; }
export declare function DataTable(props: DataTableProps): JSX.Element;
```

DecisionTrail



Auditable timeline of who did what, when, and under which authority.

```
<DecisionTrail items={[{title:"Request recorded",who:"M. Chen",when:"Oct 3 · 14:02"}]} />
```

PROPS

```
import * as React from 'react';
/**
 * Auditable timeline of who did what, when, and under which authority.
 */
export interface DecisionTrailProps { items: { title: string; who?: string; when?: string; detail?: string;
state?: "done" | "current" | "waiting" | "blocked" }[]; label?: string; }
export declare function DecisionTrail(props: DecisionTrailProps): JSX.Element;
```

Fields



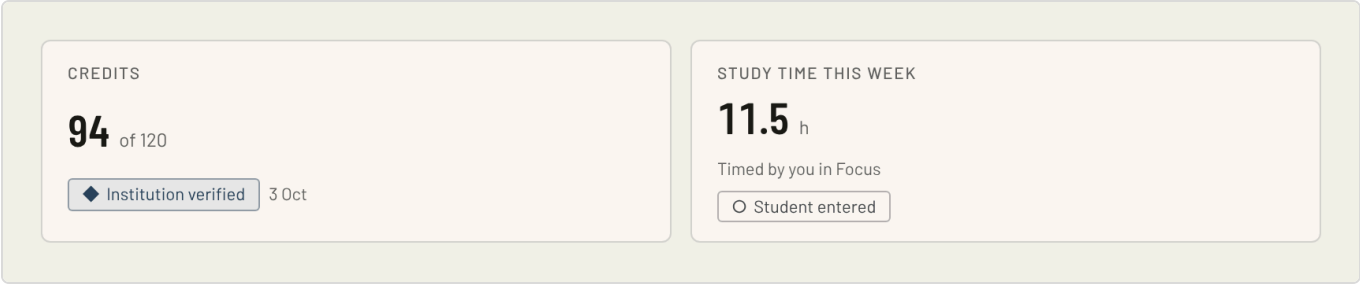
Label/value definition list for records and context.

```
<Fields items={[{field:"Environment",value:"Production"}]} />
```

PROPS

```
import * as React from 'react';
/**
 * Label/value definition list for records and context.
 */
export interface FieldsProps { items: { field: string; value: React.ReactNode; mono?: boolean }[]; label?:
string; }
export declare function Fields(props: FieldsProps): JSX.Element;
```

MetricTile



One figure worth reading first, with unit, source and age.

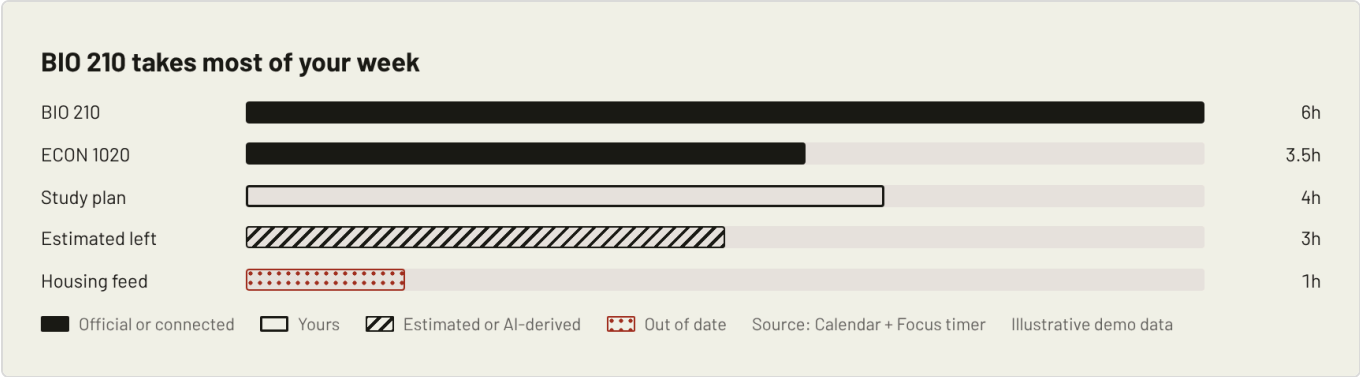
```
<MetricTile label="Credits" value="94" unit="of 120" source="official" updated="3 Oct" />
```

Never an outcome claim (GPA lift, time saved).

PROPS

```
import * as React from 'react';
/** One figure worth reading first, with unit, source and age. */
export interface MetricTileProps { label: string; value: string; unit?: string; source?: string; updated?:
string; delta?: string; }
export declare function MetricTile(props: MetricTileProps): JSX.Element;
```

ProvenanceChart



Bar chart that encodes provenance: solid official/connected, outline yours, hatched estimated/AI, dotted stale.

```
<ProvenanceChart title="Hours this week, by course" rows={[{label:"BIO
210",value:6,source:"personal"}]} unit="h" />
```

Title states the finding; every bar labelled directly; "Illustrative demo data" when `demo`.

PROPS

```
import * as React from 'react';
/** Bar chart that encodes provenance: solid official/connected, outline yours, hatched estimated/AI, dotted
stale. */
export interface ProvenanceChartProps { title: string; rows: { label: string; value: number; source?: string }
[]; unit?: string; max?: number; source?: string; demo?: boolean; }
export declare function ProvenanceChart(props: ProvenanceChartProps): JSX.Element;
```

Forms

Choice



Radio/checkbox row with a sentence explaining the consequence — used for visibility and sharing choices.

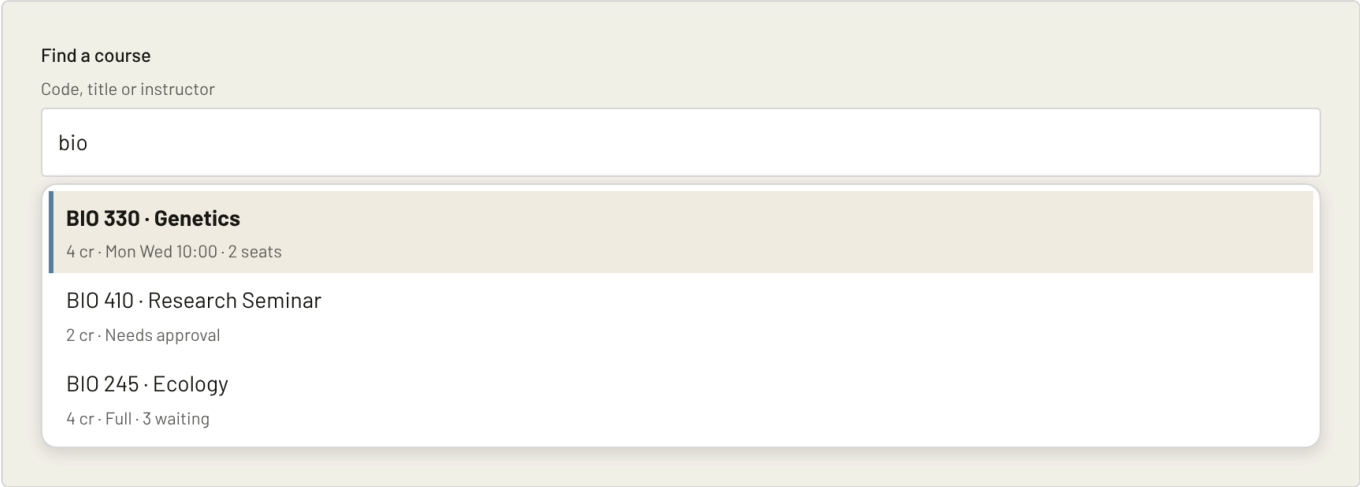
```
<Choice name="vis" label="Only me" about="Nobody else can see this." checked />
```

24px+ target, 44px row. Use for consent/sharing where the consequence must be read.

PROPS

```
import * as React from 'react';
/**
 * Radio/checkbox row with a sentence explaining the consequence — used for visibility and sharing choices.
 */
export interface ChoiceProps { type?: "radio" | "checkbox"; label: string; about?: string; checked?: boolean;
onChange?: (e: any) => void; name?: string; value?: string; disabled?: boolean; }
export declare function Choice(props: ChoiceProps): JSX.Element;
```

Combobox



Text box with a filtered listbox beneath — course search, people, rooms.

```
<Combobox label="Find a course" options={[{label:"BIO 330 · Genetics",detail:"4 cr · MW 10:00"}]}
/>
```

Selected row = 3px leading edge + weight. Escape closes; Enter picks.

PROPS

```
import * as React from 'react';
/** Text box with a filtered listbox beneath — course search, people, rooms. */
export interface ComboboxProps { label: string; hint?: string; options: { label: string; detail?: string }[];
value?: string; onChange?: (q: string) => void; onSelect?: (o: any) => void; placeholder?: string; defaultOpen?:
boolean; }
export declare function Combobox(props: ComboboxProps): JSX.Element;
```

DateField



Native date/time input with the chosen date spoken back unambiguously.

```
<DateField label="Due" value="2026-10-09" said="Thu 9 Oct 2026" />
```

Always show `said` in "Thu 9 Oct 2026" form so locale order can't mislead.

PROPS

```
import * as React from 'react';
/** Native date/time input with the chosen date spoken back unambiguously. */
export interface DateFieldProps { label: string; hint?: string; value?: string; onChange?: (e: any) => void;
withTime?: boolean; said?: string; }
export declare function DateField(props: DateFieldProps): JSX.Element;
```

Segmented



2–4 mutually exclusive view choices (e.g. Cards / Rows / Timeline).

```
<Segmented label="Feed" options={["Cards","Rows","Timeline"]} value="Cards" />
```

Selected = inverted ink. 32px tall; reserved for presentation choices, not navigation.

PROPS

```
import * as React from 'react';
/**
 * 2–4 mutually exclusive view choices (e.g. Cards / Rows / Timeline).
 */
export interface SegmentedProps { options: (string | { value: string; label: string })[]; value: string;
onChange?: (v: string) => void; label: string; }
export declare function Segmented(props: SegmentedProps): JSX.Element;
```

Select



Native select drawn in Semester's palette.

```
<Select label="Term" options={["Fall 2026","Spring 2027"]} />
```

Use native semantics; don't imitate a select with a menu.

PROPS

```
import * as React from 'react';
/**
 * Native select drawn in Semester's palette.
 */
export interface SelectProps { label: string; hint?: string; options: (string | { value: string; label: string
})[]; value?: string; onChange?: (e: any) => void; id?: string; }
export declare function Select(props: SelectProps): JSX.Element;
```

Switch



On/off setting that speaks its state as a word beside the track.

```
<Switch label="Share grades with Alex" checked />
```

role="switch"; On/Off text is always visible.

PROPS

```
import * as React from 'react';
/**
 * On/off setting that speaks its state as a word beside the track.
 */
export interface SwitchProps { checked?: boolean; onChange?: (v: boolean) => void; label: string; disabled?:
boolean; }
export declare function Switch(props: SwitchProps): JSX.Element;
```

TextField



Labelled input/textarea with associated hint and error — the label is always visible.

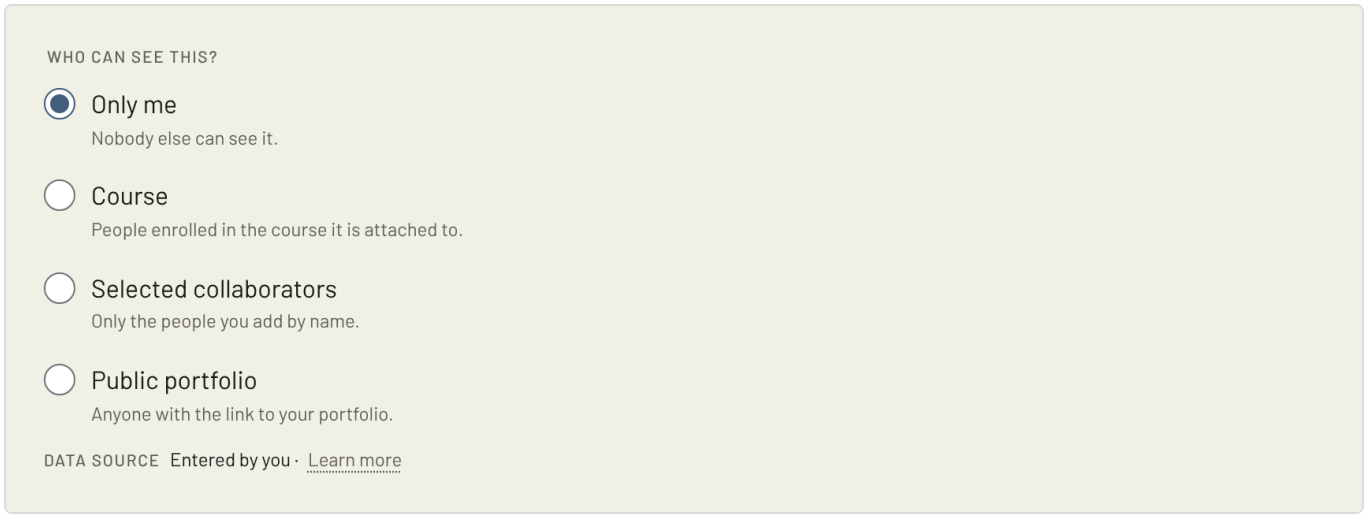
```
<TextField label="Preferred name" hint="Shown to classmates and instructors." />
```

`error` sets aria-invalid and a "!"-prefixed message. Placeholder is an example, never the label.

PROPS

```
import * as React from 'react';
/**
 * Labelled input/textarea with associated hint and error — the label is always visible.
 * @startingPoint section="Forms" subtitle="Labelled input/textarea with associated hint and error — the label
is always vis" viewport="700x400"
 */
export interface TextFieldProps { label: string; hint?: string; error?: string; multiline?: boolean; id?:
string; value?: string; placeholder?: string; onChange?: (e: any) => void; disabled?: boolean; }
export declare function TextField(props: TextFieldProps): JSX.Element;
```

VisibilityPicker



"Who can see this?" — the one privacy pattern, with the same four answers in every module.

```
<VisibilityPicker value="only-me" allowed={["only-me","course"]} />
```

Only offer audiences the module can actually honour. `locked` draws the statement alone. Presentation only — never grants access itself.

PROPS

```
import * as React from 'react';
/** "Who can see this?" — the one privacy pattern, with the same four answers in every module. */
export interface VisibilityPickerProps { value?: "only-me" | "course" | "collaborators" | "portfolio";
onChange?: (v: string) => void; allowed?: string[]; locked?: boolean; origin?: "student-entered" | "institution-
provided" | "connected-system"; onLearnMore?: () => void; }
export declare function VisibilityPicker(props: VisibilityPickerProps): JSX.Element;
```

Glyph

Avatar



Initials in a tinted square that follows the corner setting; profile glyph when no name.

```
<Avatar name="Maya Lindqvist" size={40} />
```

No photos, never derived from an email address.

PROPS

```
import * as React from 'react';
/** Initials in a tinted square that follows the corner setting; profile glyph when no name. */
export interface AvatarProps { name?: string; size?: number; }
export declare function Avatar(props: AvatarProps): JSX.Element;
```

Icon



Semester line glyph (Lucide-like, 24 grid, 1.5 stroke) from the app's own icon set — 108 names, see assets/icons/.

```
<Icon name="calendar" size={19} />
```

`label` makes it a named image; otherwise decorative. Names: today, courses, calendar, study, degree, registrar, costs, family, career, campus, meals, housing, ask, sources, privacy, settings, search, bell…

PROPS

```
import * as React from 'react';
/**
 * Semester line glyph (Lucide-like, 24 grid, 1.5 stroke) from the app's own icon set — 108 names, see
assets/icons/.
 * @startingPoint section="Glyph" subtitle="Semester line glyph (Lucide-like, 24 grid, 1.5 stroke) from the
app's own icon s" viewport="700x400"
 */
export interface IconProps { name: string; size?: number; label?: string; strokeWidth?: number; style?:
React.CSSProperties; className?: string; }
export declare function Icon(props: IconProps): JSX.Element;
export declare const ICON_NAMES: string[];
```

Wordmark



The Semester lockup — three-slab mark + SEMESTER in Barlow Condensed caps; also exports `Mark`.

```
<Wordmark size={20} metal />
```

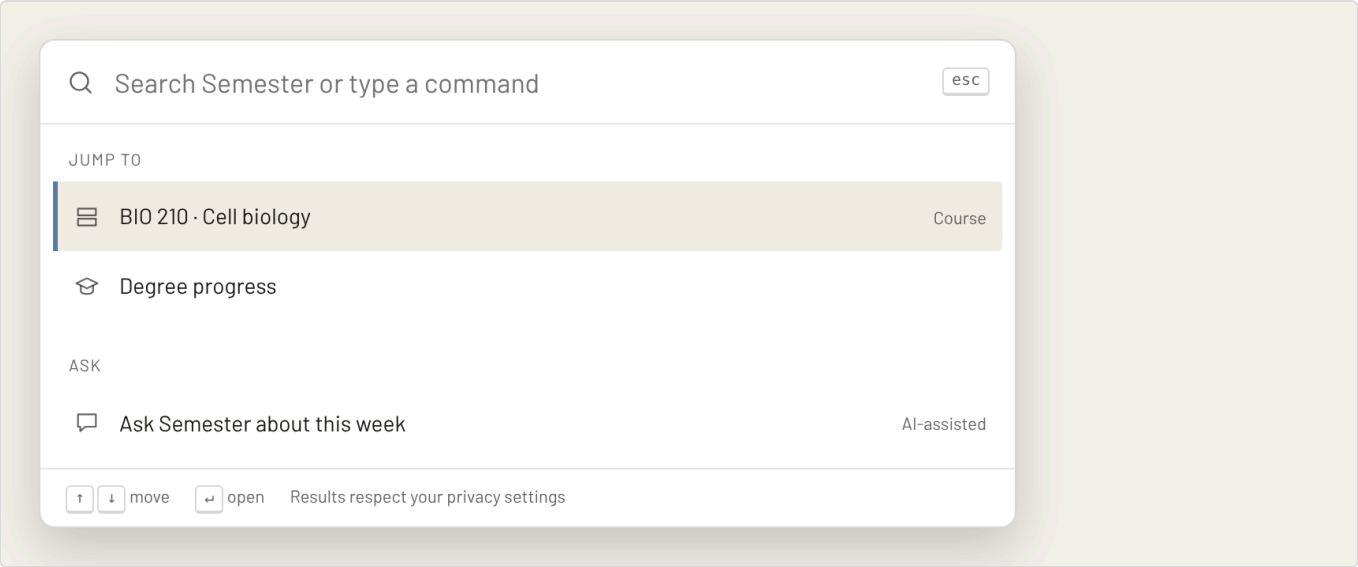
`metal` paints brushed sterling for dark chrome. `<Mark size={24}/>` alone for tiles/favicons.

PROPS

```
import * as React from 'react';
/**
 * The Semester lockup — three-slab mark + SEMESTER in Barlow Condensed caps; also exports `Mark`.
 */
export interface WordmarkProps { size?: number; metal?: boolean; showWord?: boolean; style?:
React.CSSProperties; }
export declare function Wordmark(props: WordmarkProps): JSX.Element;
export declare function Mark(props: { size?: number; style?: React.CSSProperties; className?: string }):
JSX.Element;
```

Navigation

CommandPalette



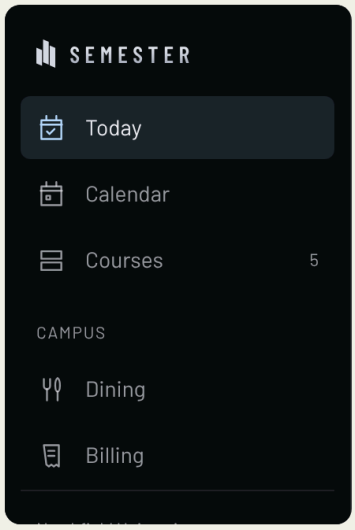
Global search and command palette (⌘K) with keyboard selection.

```
<CommandPalette groups={[{label:"Jump to",items:[{label:"BIO 210",icon:"courses"}]}]} />
```

PROPS

```
import * as React from 'react';
/**
 * Global search and command palette (⌘K) with keyboard selection.
 */
export interface CommandPaletteProps { groups: { label: string; items: { label: string; icon?: string; detail?:
string }[] }[]; placeholder?: string; onSelect?: (item: any) => void; }
export declare function CommandPalette(props: CommandPaletteProps): JSX.Element;
```

SideRail



Desktop side rail on dark Ink chrome — persistent, quiet, labelled.

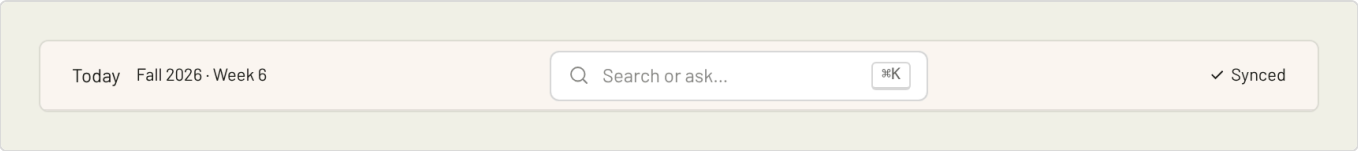
```
<SideRail current="today" groups={[{items:[{id:"today",label:"Today",icon:"today"}]}]} />
```

248px wide. Current row = blue-tinted wash + blue glyph.

PROPS

```
import * as React from 'react';
/**
 * Desktop side rail on dark Ink chrome — persistent, quiet, labelled.
 * @startingPoint section="Navigation" subtitle="Desktop side rail on dark Ink chrome — persistent, quiet,
labelled." viewport="700x400"
 */
export interface SideRailProps { groups: { label?: string; items: { id: string; label: string; icon: string;
count?: number }[] }[]; current?: string; onNavigate?: (id: string) => void; foot?: React.ReactNode; brand?:
boolean; }
export declare function SideRail(props: SideRailProps): JSX.Element;
```

SystemContextBar



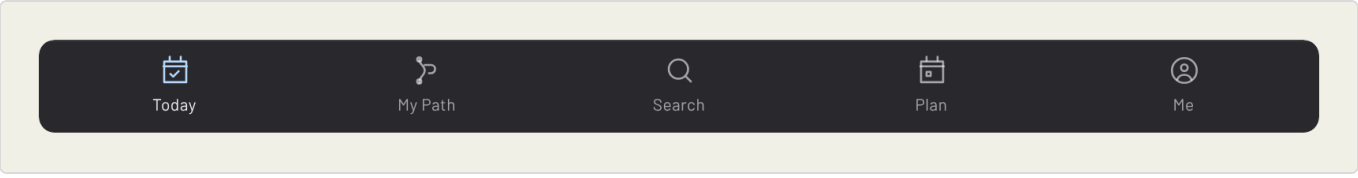
Persistent top bar: location, term, ⌘K search, sync state, quiet tools.

```
<SystemContextBar location="Today" term="Fall 2026 · Week 6" sync="synced" />
```

PROPS

```
import * as React from 'react';
/**
 * Persistent top bar: location, term, ⌘K search, sync state, quiet tools.
 */
export interface SystemContextBarProps { location?: string; term?: string; sync?: string; onSearch?: () => void;
actions?: React.ReactNode; }
export declare function SystemContextBar(props: SystemContextBarProps): JSX.Element;
```

TabBar



Mobile bottom navigation for the five canonical destinations, always labelled.

```
<TabBar current="today" items={[{id:"today",label:"Today",icon:"today"}]} />
```

Frosted Ink only where blur is supported; opaque otherwise.

PROPS

```
import * as React from 'react';
/**
 * Mobile bottom navigation for the five canonical destinations, always labelled.
 */
export interface TabBarProps { items: { id: string; label: string; icon: string }[]; current?: string;
onNavigate?: (id: string) => void; }
export declare function TabBar(props: TabBarProps): JSX.Element;
```

States

EmptyState



Explains why a list is empty and offers one relevant next action.

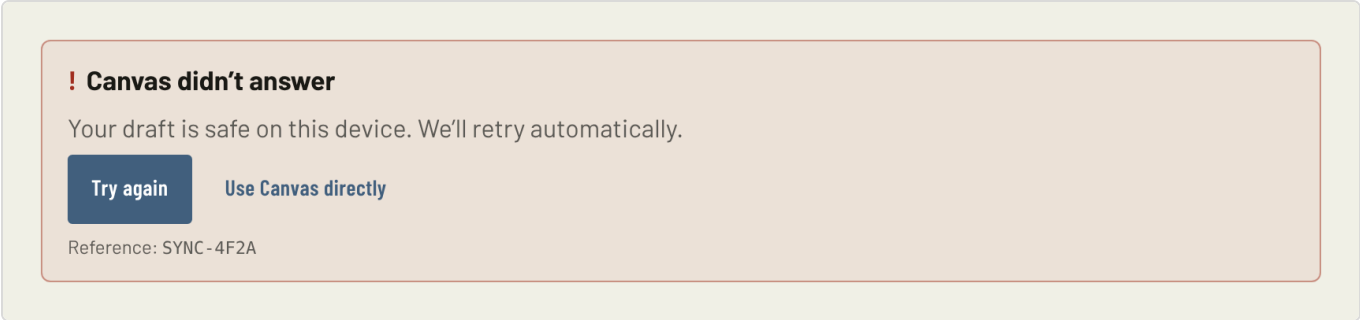
```
<EmptyState title="No tasks due this week" body="Tasks you add or that courses publish appear
here." action={<Button>Add a task</Button>} />
```

Dashed hairline frame; never decorative illustration.

PROPS

```
import * as React from 'react';
/**
 * Explains why a list is empty and offers one relevant next action.
 * @startingPoint section="States" subtitle="Explains why a list is empty and offers one relevant next action."
viewport="700x400"
 */
export interface EmptyStateProps { title: string; body?: string; action?: React.ReactNode; }
export declare function EmptyState(props: EmptyStateProps): JSX.Element;
```

ErrorState



Plain-language failure with a required recovery action and an optional support reference.

```
<ErrorState title="Canvas didn’t answer" body="Your draft is safe on this device."
reference="SYNC-4F2A" />
```

role="alert"; preserve the user's input, separate retryable failure from permission.

PROPS

```
import * as React from 'react';
/**
 * Plain-language failure with a required recovery action and an optional support reference.
 */
export interface ErrorStateProps { title: string; body: string; recoverLabel?: string; onRecover?: () => void;
secondaryLabel?: string; onSecondary?: () => void; reference?: string; }
export declare function ErrorState(props: ErrorStateProps): JSX.Element;
```

LoadingState

Still structural placeholders plus a spoken "Loading …" — no spinner or shimmer.

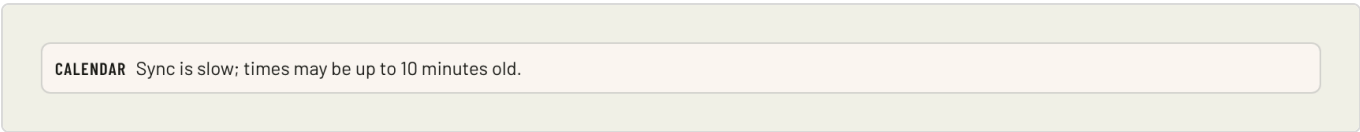
```
<LoadingState what="your courses" />
```

Only when the final shape is known; otherwise say what is happening in words.

PROPS

```
import * as React from 'react';
/**
 * Still structural placeholders plus a spoken "Loading …" — no spinner or shimmer.
 */
export interface LoadingStateProps { what?: string; bars?: number[]; }
export declare function LoadingState(props: LoadingStateProps): JSX.Element;
```

Notice



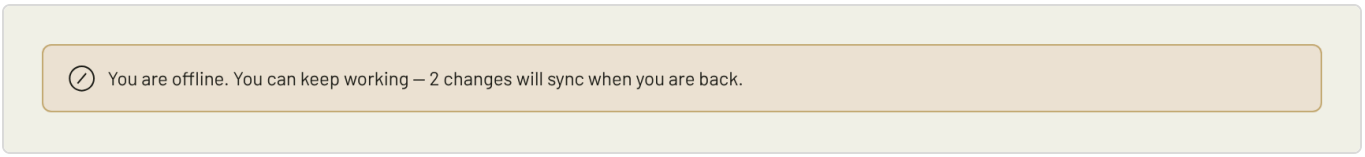
One-line service or context notice (role=status).

```
<Notice head="Calendar">Sync is slow right now; times may be up to 10 minutes old.</Notice>
```

PROPS

```
import * as React from 'react';
/**
 * One-line service or context notice (role=status).
 */
export interface NoticeProps { head?: string; children: React.ReactNode; }
export declare function Notice(props: NoticeProps): JSX.Element;
```

OfflineStrip



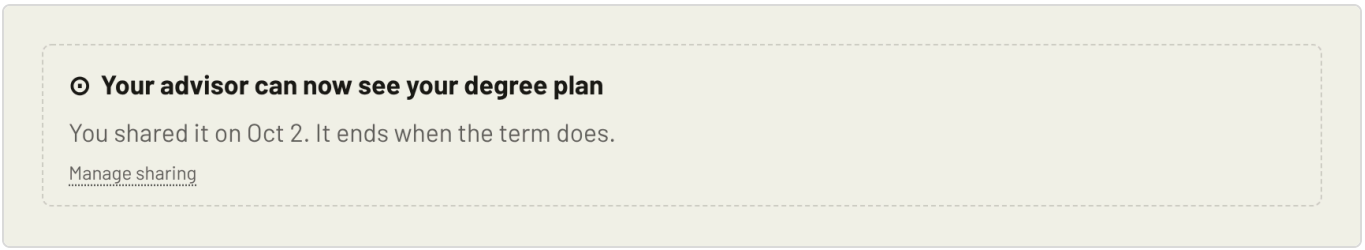
Offline strip that says what still works and that changes are queued.

```
<OfflineStrip queued={2} />
```

PROPS

```
import * as React from 'react';
/**
 * Offline strip that says what still works and that changes are queued.
 */
export interface OfflineStripProps { queued?: number; syncs?: boolean; }
export declare function OfflineStrip(props: OfflineStripProps): JSX.Element;
```

PermissionNotice



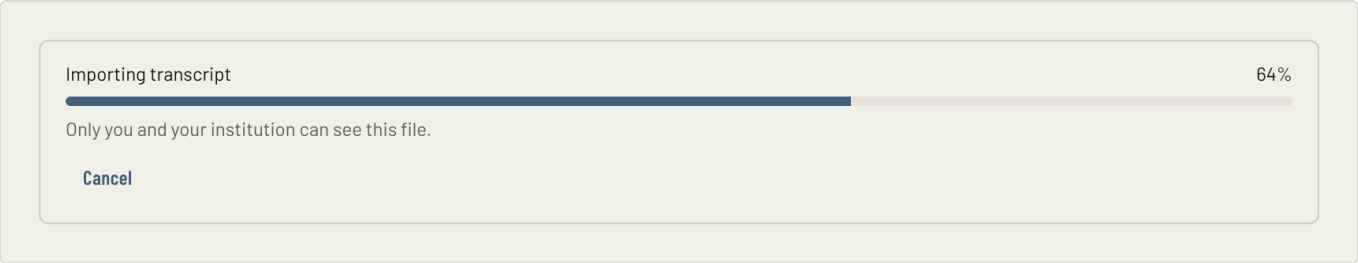
States what changed about access, why, and where it is controlled.

```
<PermissionNotice changed="Your advisor can now see your degree plan" why="You shared it on Oct
2." controlLabel="Manage sharing" />
```

PROPS

```
import * as React from 'react';
/**
 * States what changed about access, why, and where it is controlled.
 */
export interface PermissionNoticeProps { changed: string; why: string; controlLabel?: string; onControl?: () =>
void; }
export declare function PermissionNotice(props: PermissionNoticeProps): JSX.Element;
```

ProgressState



A determinate wait (upload, import) with real

```
<ProgressState label="Importing transcript" done={64} total={100} note="Only your institution and you can see this file." />
```

`failed` swaps the bar for the reason + Retry.

PROPS

```
import * as React from 'react';
/**
 * A determinate wait (upload, import) with real <progress>, percentage in words, Cancel/Retry.
 */
export interface ProgressStateProps { label: string; done?: number; total?: number; note?: string; failed?: string; onCancel?: () => void; onRetry?: () => void; }
export declare function ProgressState(props: ProgressStateProps): JSX.Element;
```

SaveState



The quiet "Saving… → Saved" line near the work, in a polite live region.

```
<SaveState status="saved" />
```

offline/conflict/sync-error announce assertively.

PROPS

```
import * as React from 'react';
/**
 * The quiet "Saving… → Saved" line near the work, in a polite live region.
 */
export interface SaveStateProps { status?: string; }
export declare function SaveState(props: SaveStateProps): JSX.Element;
```

StepStatus



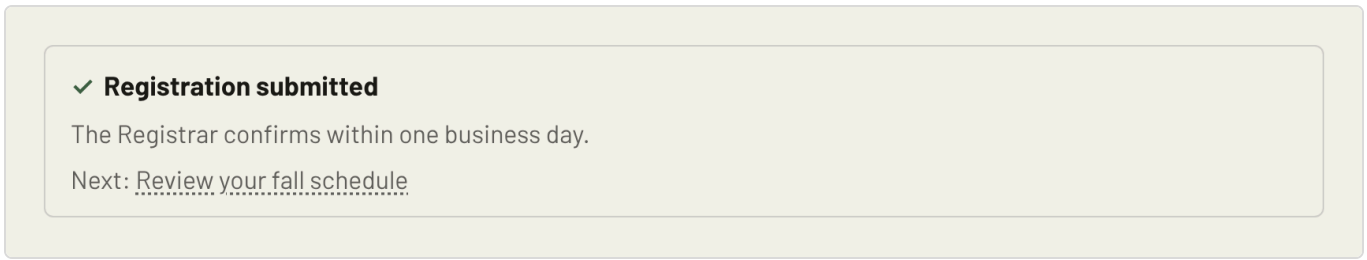
A process in named steps — shows the AI or system working instead of faking certainty.

```
<StepStatus steps={[{label:"Reading sources",state:"done"},{label:"Drafting",state:"working"}]}
/>
```

PROPS

```
import * as React from 'react';
/**
 * A process in named steps — shows the AI or system working instead of faking certainty.
 */
export interface StepStatusProps { steps: { label: string; state: "waiting" | "working" | "done" | "failed" }[];
label?: string; }
export declare function StepStatus(props: StepStatusProps): JSX.Element;
```

SuccessState



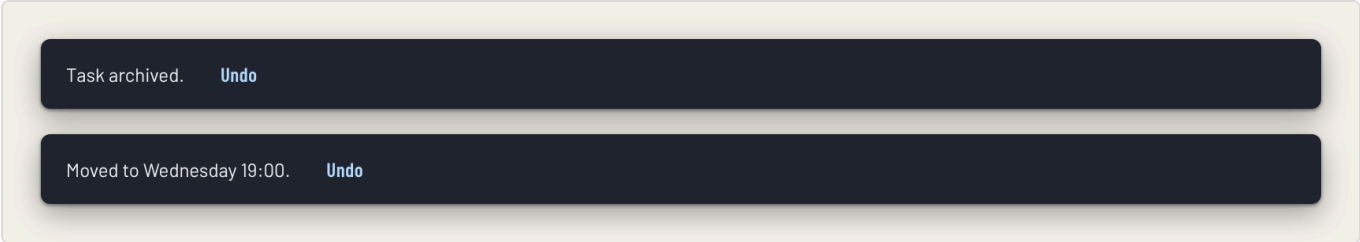
Respectful acknowledgement that points at what is next — no confetti, streaks or ranks.

```
<SuccessState title="Registration submitted" nextLabel="Review your fall schedule" />
```

PROPS

```
import * as React from 'react';
/**
 * Respectful acknowledgement that points at what is next — no confetti, streaks or ranks.
 */
export interface SuccessStateProps { title: string; body?: string; nextLabel?: string; onNext?: () => void; }
export declare function SuccessState(props: SuccessStateProps): JSX.Element;
```

UndoToast



One-sentence confirmation with Undo for reversible work — instead of an "are you sure?" dialog.

```
<UndoToast message="Task archived." onUndo={() => {}} />
```

Ink surface on every ground. Never for consequential or financial actions — those use Dialog + ActionPreview.

PROPS

```
import * as React from 'react';
/** One-sentence confirmation with Undo for reversible work — instead of an "are you sure?" dialog. */
export interface UndoToastProps { message: string; onUndo?: () => void; undoLabel?: string; }
export declare function UndoToast(props: UndoToastProps): JSX.Element;
```

Surfaces

ContextBar



The object a screen is about, with its states and actions.

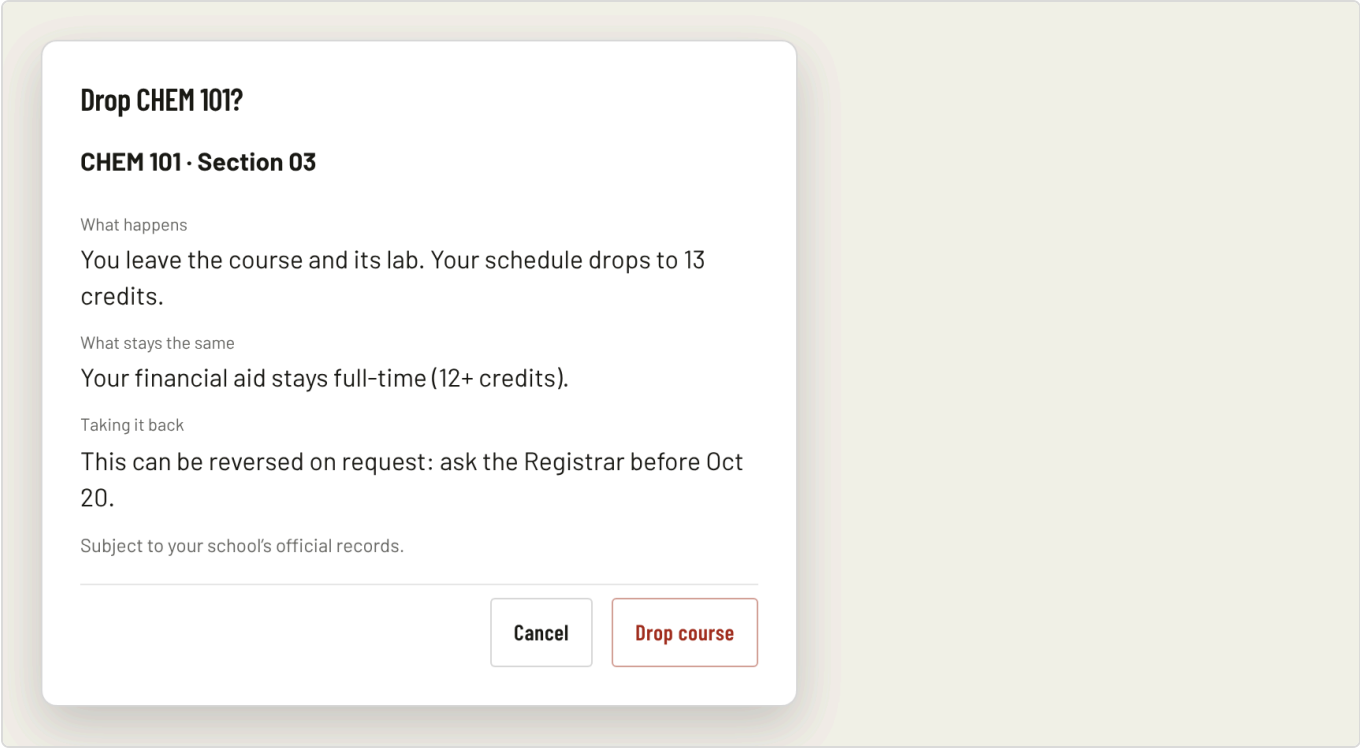
```
<ContextBar kicker="BIO 210 · Assignment" title="Lab report 3" states={<StatusChip
status="draft"/>} />
```

Inside an existing panel it drops its own frame.

PROPS

```
import * as React from 'react';
/**
 * The object a screen is about, with its states and actions.
 */
export interface ContextBarProps { kicker?: string; title: string; states?: React.ReactNode; actions?:
React.ReactNode; }
export declare function ContextBar(props: ContextBarProps): JSX.Element;
```

Dialog



Modal confirmation — focus lands on Cancel, Escape closes, names object + consequence + recovery.

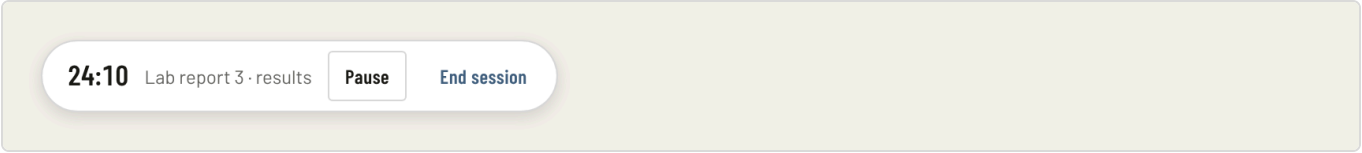
```
<Dialog title="Submit registration?" confirmLabel="Submit"><ActionPreview … /></Dialog>
```

`destructive` uses red-ink button. `inline` renders without scrim (for docs).

PROPS

```
import * as React from 'react';
/**
 * Modal confirmation — focus lands on Cancel, Escape closes, names object + consequence + recovery.
 */
export interface DialogProps { open?: boolean; title: string; children?: React.ReactNode; confirmLabel?: string;
cancelLabel?: string; onConfirm?: () => void; onCancel?: () => void; destructive?: boolean; inline?: boolean; }
export declare function Dialog(props: DialogProps): JSX.Element;
```

FocusBar



Floating pill during a focus session: time left, task, Pause, End.

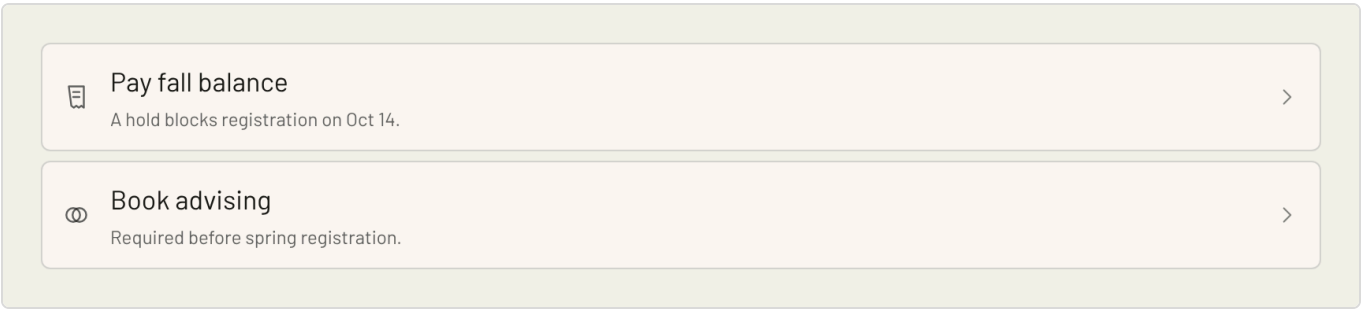
```
<FocusBar task="Lab report 3 · results" remaining="24:10" />
```

The only frosted overlay besides the tab bar; opaque under reduced transparency.

PROPS

```
import * as React from 'react';
/** Floating pill during a focus session: time left, task, Pause, End. */
export interface FocusBarProps { task: string; remaining?: string; onPause?: () => void; onEnd?: () => void;
breakDue?: string; }
export declare function FocusBar(props: FocusBarProps): JSX.Element;
```

NextSteps



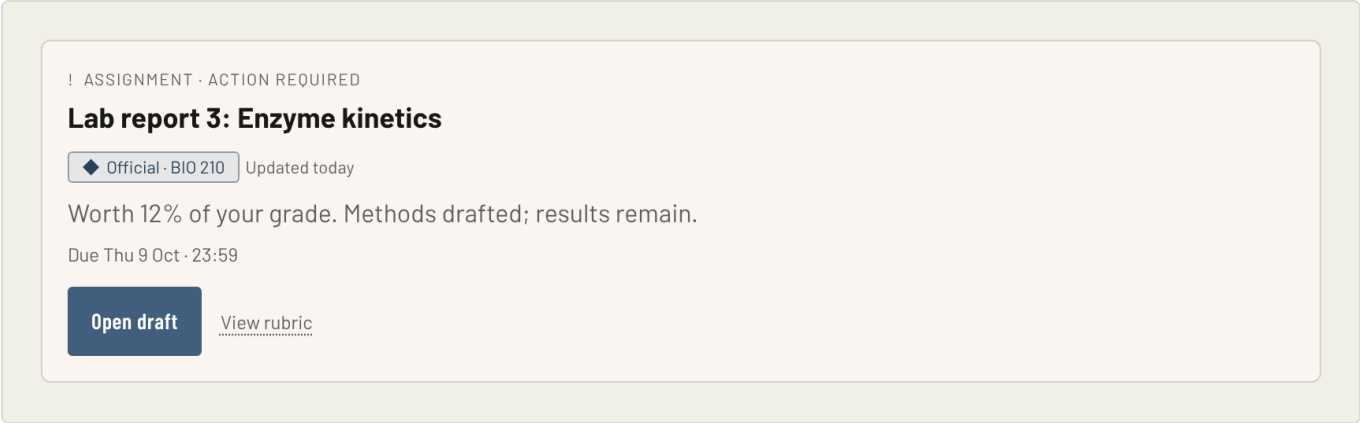
Ordered list of next safe actions, each with one line of why.

```
<NextSteps steps={[{label:"Pay fall balance",why:"A hold blocks registration on Oct
14.",icon:"costs"}]} />
```

PROPS

```
import * as React from 'react';
/**
 * Ordered list of next safe actions, each with one line of why.
 */
export interface NextStepsProps { steps: { label: string; why?: string; icon?: string; aside?: React.ReactNode;
onClick?: () => void }[]; }
export declare function NextSteps(props: NextStepsProps): JSX.Element;
```

ObjectCard



The universal decision card: eyebrow → title → provenance → why → metadata → one primary + one quiet secondary.

```
<ObjectCard kind="Assignment" title="Lab report 3" provenance={{origin:"official",by:"BIO 210"}}
explanation="Worth 12% of your grade." metadata="Due Thu 9 Oct · 23:59" primary={{label:"Open
draft"}} secondary={{label:"View rubric"}} />
```

A card groups a decision, not every paragraph. Never nest cards.

PROPS

```
import * as React from 'react';
/**
 * The universal decision card: eyebrow → title → provenance → why → metadata → one primary + one quiet
secondary.
 * @startingPoint section="Surfaces" subtitle="The universal decision card: eyebrow → title → provenance → why →
metadata → one" viewport="700x400"
 */
export interface ObjectCardProps { kind?: string; statuses?: string[]; title: string; provenance?: { origin:
string; by?: string; cues?: string[]; age?: string }; explanation?: string; metadata?: string; primary?: {
label: string; onClick?: () => void }; secondary?: { label: string; onClick?: () => void }; tone?: "pending" |
"danger"; level?: 2 | 3 | 4; children?: React.ReactNode; }
export declare function ObjectCard(props: ObjectCardProps): JSX.Element;
```

OpenIn



"Open in" row — the way from one object to every place it matters (Plan, Study, Calendar).

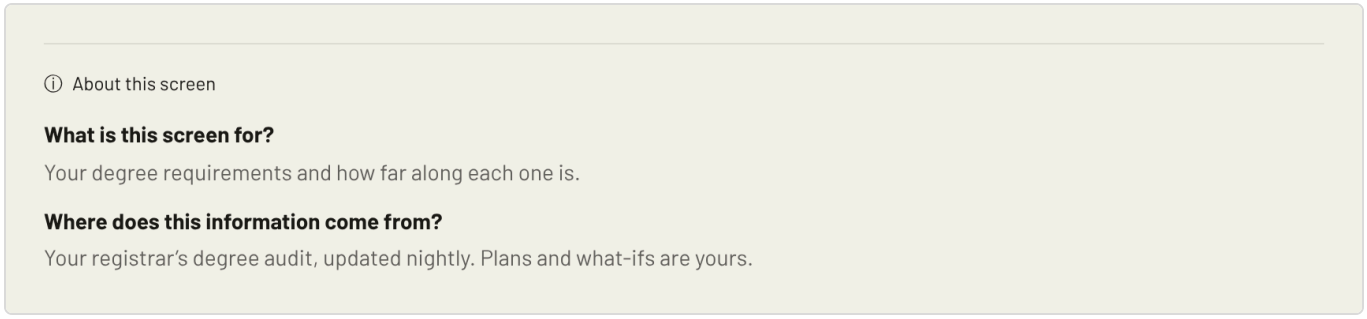
```
<OpenIn about="BIO 210" targets={["Plan","Study","Calendar"]} />
```

Words, never icons alone.

PROPS

```
import * as React from 'react';
/** "Open in" row — the way from one object to every place it matters (Plan, Study, Calendar). */
export interface OpenInProps { targets: string[]; about: string; onOpen?: (t: string) => void; }
export declare function OpenIn(props: OpenInProps): JSX.Element;
```

ScreenGuide



"About this screen" disclosure at the foot of every screen — what it is for, where its data comes from, who decides.

```
<ScreenGuide items={[{q:"Where does this information come from?",a:"Your registrar’s degree
audit, updated nightly."}]} />
```

PROPS

```
import * as React from 'react';
/** "About this screen" disclosure at the foot of every screen — what it is for, where its data comes from, who
decides. */
export interface ScreenGuideProps { items: { q: string; a: string }[]; open?: boolean; }
export declare function ScreenGuide(props: ScreenGuideProps): JSX.Element;
```

SectionLabel

The small uppercase kicker that heads a section, with an optional right-aligned count.

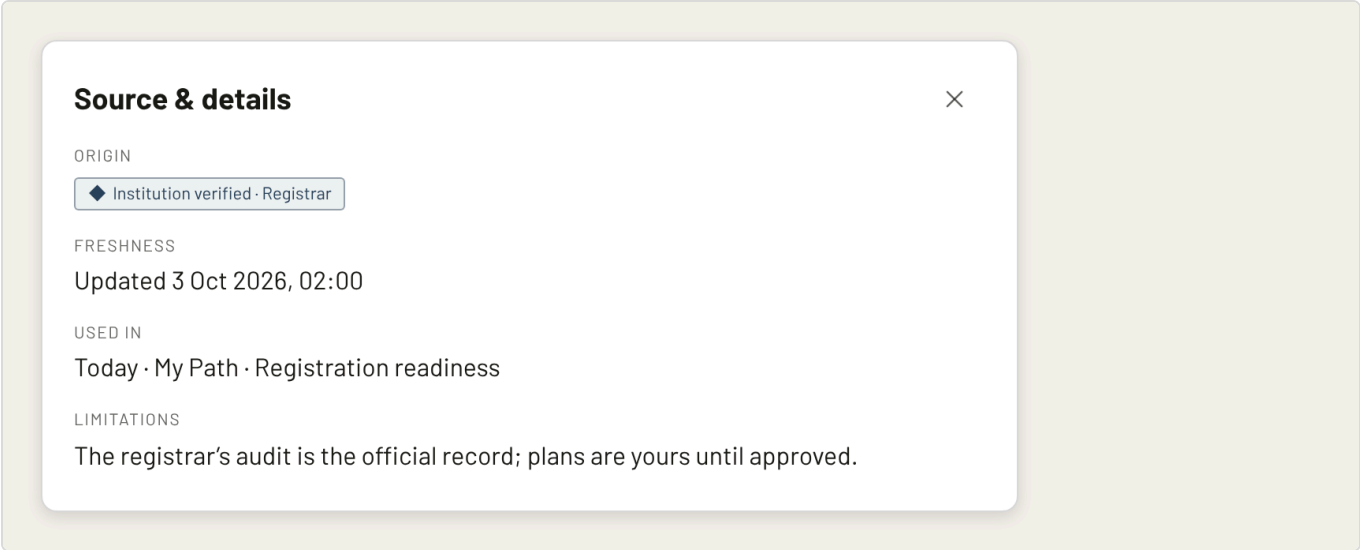
```
<SectionLabel aside="3">Begin today</SectionLabel>
```

Tracked 0.08em caps at 10.5px, secondary ink.

PROPS

```
import * as React from 'react';
/**
 * The small uppercase kicker that heads a section, with an optional right-aligned count.
 */
export interface SectionLabelProps { children: React.ReactNode; aside?: React.ReactNode; as?: "h2" | "h3" |
"h4"; style?: React.CSSProperties; }
export declare function SectionLabel(props: SectionLabelProps): JSX.Element;
```

Sheet



Bottom sheet on mobile, window on desktop; the home of "Source & details". Also exports `SourceDetails`.

```
<Sheet title="Source & details"><SourceDetails items={[{label:"Origin",value:"Official · Registrar"}]} /></Sheet>
```

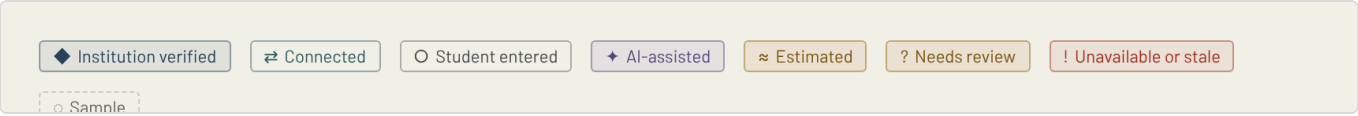
Rows: Origin, Freshness, Used in, Who can see this, Limitations, Report.

PROPS

```
import * as React from 'react';
/** Bottom sheet on mobile, window on desktop; the home of "Source & details". Also exports `SourceDetails`. */
export interface SheetProps { title: string; children?: React.ReactNode; onClose?: () => void; inline?: boolean; actions?: React.ReactNode; }
export declare function Sheet(props: SheetProps): JSX.Element;
export declare function SourceDetails(props: { items: { label: string; value: React.ReactNode }[] }): JSX.Element;
```

Trust & provenance

SourceBadge



Where a fact came from, beside the fact — controlled vocabulary (glyph + word + tone).

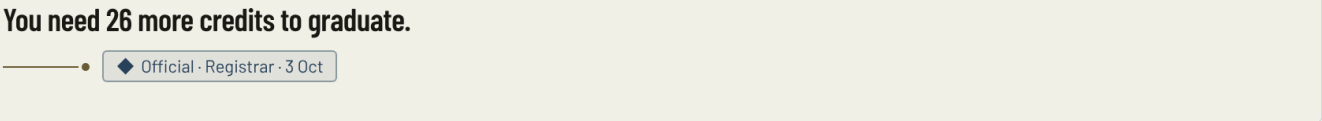
```
<SourceBadge source="official" by="Registrar" updated="Updated 2h ago" />
```

official navy (solid), connected teal, personal graphite, ai violet (always labelled), estimated/review amber, stale red, sample dashed. `onReport` adds "Report incorrect information".

PROPS

```
import * as React from 'react';
/**
 * Where a fact came from, beside the fact — controlled vocabulary (glyph + word + tone).
 * @startingPoint section="Trust & provenance" subtitle="Where a fact came from, beside the fact — controlled
vocabulary (glyph + word + " viewport="700x400"
 */
export interface SourceBadgeProps { source: "official" | "connected" | "imported" | "personal" | "ai" |
"estimated" | "review" | "stale" | "external" | "sample"; updated?: string; by?: string; onReport?: () => void;
}
export declare function SourceBadge(props: SourceBadgeProps): JSX.Element;
```

SourceLine



The Source Line — a thin brass/editorial rule from a statement to its source chip; Semester's signature device (docs/BRAND-PLATFORM.md §2.6).

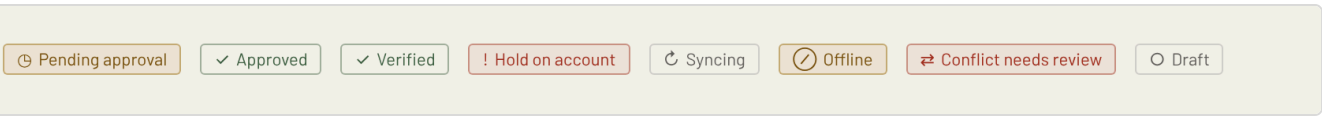
```
<SourceLine source="official" by="Registrar" when="3 Oct"><h2>You need 26 more credits to graduate.</h2></SourceLine>
```

Never decorative: use only where a real source label exists. The chip carries the word; the line is reinforcement.

PROPS

```
import * as React from 'react';
/** The Source Line — Semester's signature device: a thin editorial rule from a statement to its source chip. */
export interface SourceLineProps { children: React.ReactNode; source?: string; by?: string; when?: string }
export declare function SourceLine(props: SourceLineProps): JSX.Element;
```

StatusChip



One status chip — glyph + word + tone from the shared status table.

```
<StatusChip status="pending" />
```

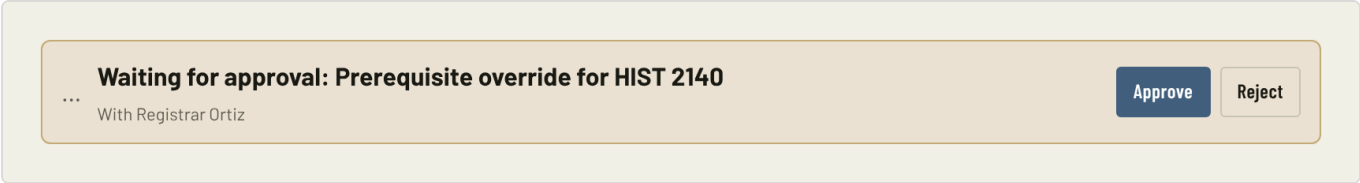
Keys: saving, saved, syncing, synced, offline, queued, conflict, sync-error, needs-confirmation, action-required, pending, approved, rejected, verified, hold, faculty-approved, course-provided, updated-today, draft, live, beta.

PROPS

```
import * as React from 'react';
/**
 * One status chip — glyph + word + tone from the shared status table.
 */
export interface StatusChipProps { status: string; label?: string; }
export declare function StatusChip(props: StatusChipProps): JSX.Element;
```

Governance

ApprovalBanner



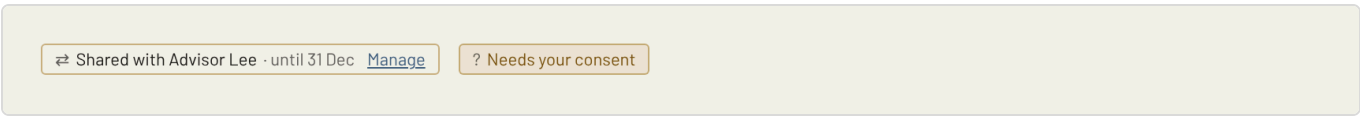
Approval state for a consequential action (two-person rule): required → pending → approved/rejected, with the right buttons for the viewer. Pair with ActionPreview and DecisionTrail.

```
<ApprovalBanner state="pending" what="Prerequisite override for HIST 2140" approver="Registrar Ortiz" onApprove={a} onReject={r} />
```

PROPS

```
import * as React from 'react';
export interface ApprovalBannerProps { state?: "required" | "pending" | "approved" | "rejected"; what: string; approver?: string; requestedBy?: string; onRequest?: () => void; onApprove?: () => void; onReject?: () => void; }
export declare function ApprovalBanner(props: ApprovalBannerProps): JSX.Element;
```

ConsentBadge



Consent state for a student-owned item: private, shared (with recipient and expiry), expired, revoked, or consent required. Always glyph + words.

```
<ConsentBadge state="shared" recipient="Advisor Lee" expires="31 Dec" onManage={open} />
```

PROPS

```
import * as React from 'react';
export interface ConsentBadgeProps { state?: "private" | "shared" | "expired" | "revoked" | "required"; recipient?: string; expires?: string; onManage?: () => void; }
export declare function ConsentBadge(props: ConsentBadgeProps): JSX.Element;
```

ErrorSummary



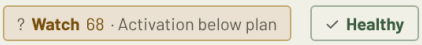
Summary of form errors at the top of a form (WCAG 3.3.1). Each item links to and focuses its field by id. Move focus to the summary on failed submit.

```
<ErrorSummary ref={r} errors={[{ field: "email", message: "Enter a full email address" }]} />
```

PROPS

```
import * as React from 'react';
export interface ErrorSummaryProps { errors: { field: string; message: string }[]; title?: string; }
export declare const ErrorSummary: (props: ErrorSummaryProps & { ref?: any }) => JSX.Element | null;
```

HealthBadge



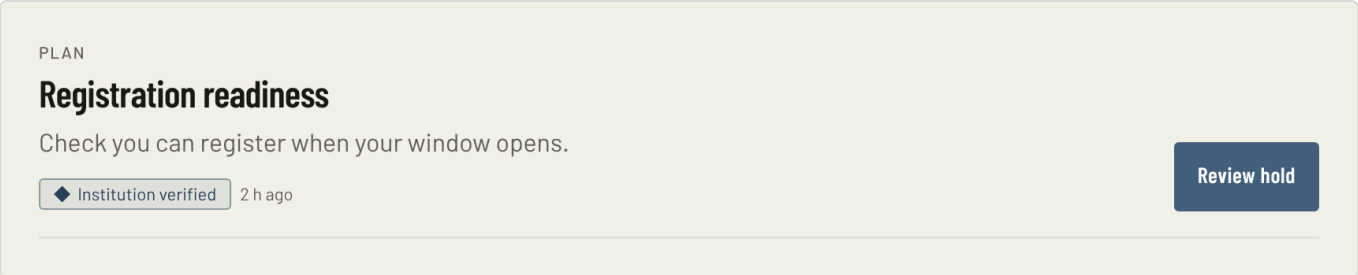
Health band for a tenant, integration or customer: band + optional score + top driver + freshness. Explains, never just colours.

```
<HealthBadge band="watch" score={68} driver="Activation below plan" updated="2 h ago" />
```

PROPS

```
import * as React from 'react';
export interface HealthBadgeProps { band?: "healthy" | "watch" | "atrisk" | "unknown"; score?: number; driver?:
string; updated?: string; }
export declare function HealthBadge(props: HealthBadgeProps): JSX.Element;
```

PageHeader



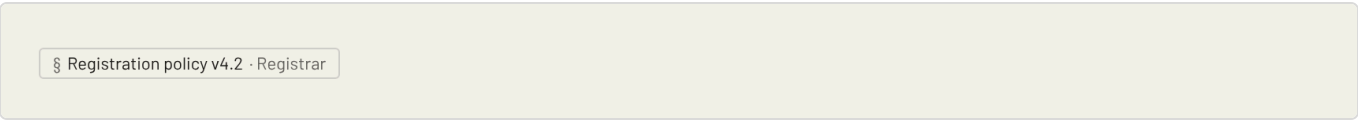
The universal page head from the UI grammar: kicker, one H1, one line on what the page helps you do, source/freshness, and at most one primary action.

```
<PageHeader kicker="Plan" title="Registration readiness" purpose="Check you can register when your window opens." source={<SourceBadge source="official" updated="2 h ago" />} actions={<Button variant="primary">Review hold</Button>} />
```

PROPS

```
import * as React from 'react';
export interface PageHeaderProps { kicker?: string; title: string; purpose?: string; source?: React.ReactNode; actions?: React.ReactNode; }
export declare function PageHeader(props: PageHeaderProps): JSX.Element;
```

PolicyBadge



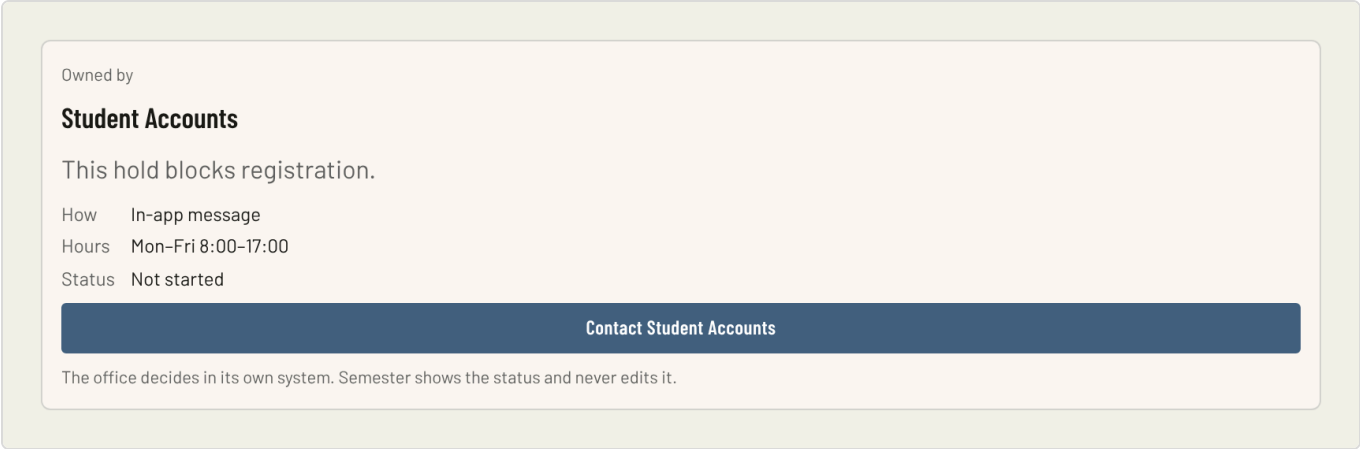
Names the policy that governs a fact, action or AI answer: name, version, owner. Clickable when onOpen is set (opens the policy text in a Sheet).

```
<PolicyBadge name="Registration policy" version="4.2" owner="Registrar" />
```

PROPS

```
import * as React from 'react';
export interface PolicyBadgeProps { name: string; version?: string; owner?: string; onOpen?: () => void; }
export declare function PolicyBadge(props: PolicyBadgeProps): JSX.Element;
```

SupportHandoff



Official hand-off to the owning office (holds, aid, accommodations, counseling). Names the owner, the channel and hours, and tracks status without editing the official record.

```
<SupportHandoff office="Student Accounts" reason="This hold blocks registration." channel="In-app message" hours="Mon–Fri 8–5" onStart={go} />
```

PROPS

```
import * as React from 'react';
export interface SupportHandoffProps { office: string; reason?: string; channel?: string; hours?: string; status?: "ready" | "sent" | "received" | "resolved"; reference?: string; onStart?: () => void; }
export declare function SupportHandoff(props: SupportHandoffProps): JSX.Element;
```

Patterns

AI answer anatomy

Label · scope · sources · uncertainty · cost · policy · human-confirmed action

✦AI-assistedWritten with Semester's assistant. It is not an official answer — check anything you act on.Semester assistant§Outlining allowed in BIO 210

About 3 hours remain on Lab report 31. Results is the largest gap2.

1. 1BIO 210 Syllabus §4◆Institution verifiedConfirmed by your institution's own system.
2. 2Your lab notes, Oct 1○Student enteredYou typed this in. Semester has not checked it with your institution.

1. Always labelled AI-assisted — never a persona
2. Allowed data scope, named
3. Numbered sources with their own provenance
4. Confidence stated, and what it can't know
5. Request cost and remaining allowance
6. Course/institution policy that applies
7. Consequential actions open a preview first

Provenance ladder

Strongest first; stale ranks below yours on purpose (lib/where.ts)

0	◆Institution verifiedConfirmed by your institution's own system.	Your institution's own record, read from its system.
1	⇄ConnectedSynced from a connected account or feed. Semester has not checked it with your institution.	From a calendar or account you connected, checked recently.
2	○Made hereSemester built this from sources you gave it — a rule-based extraction, not an AI draft.	The app built this from sources you gave it.
3	○Student enteredYou typed this in. Semester has not checked it with your institution.	You entered this yourself. Nothing has checked it.
4	○SampleIllustrative data. Not live.	Shipped as a demonstration. Not your semester.
5	!Unavailable or staleSemester cannot confirm that this information is current.	From a connection not checked lately. It may have moved.

Source Line

Signature device — a statement, a rule, its source. Only where a real source exists

You need 26 more credits to graduate.

The chip carries the word; the brass/editorial line is reinforcement only. Never decorative — absent source, absent line. Works on screens, decks, social cards and reports.

Consent & sharing

One decision at a time · default private · visible history · revoke anywhere

ASK

Share final grades with Jonas?

Course letter grades once posted by the Registrar. Not assignment scores, feedback or anything else.

Share final grades | Not now

OnFinal grades · shared since Aug 28

SHOW WHAT HAPPENED

1. Jonas viewed final grades
   Guardian · Sep 30 · 08:02
2. You turned on Final grades
   You · Aug 28 · 16:40
3. Invitation accepted
   Jonas Lindqvist · Aug 28 · 16:31

Consequential action

Review → preview → confirm → receipt. Reversible work gets Undo instead

Withdraw from CHEM 101?

CHEM 101 · Section 03

What happens
: You leave CHEM 101 and a W is recorded on your transcript.

What stays the same
: Your other courses and aid status (13 credits, full-time) stay as they are.

Taking it back
: This can’t be undone. Ask the Registrar if this was a mistake.

Subject to your school’s official records.

RULES

· Name the object in the title and both buttons.

· Focus lands on the safe choice.

· Destructive = red ink + hairline, never a red fill.

· State recovery: undo · on request · can’t be undone.

· Financial and record changes always preview.

AFTER

✓Withdrawn from CHEM 101

A W is recorded. Your schedule is now 13 credits.

State matrix

Every data region designs all seven states

LOADING

Loading this screen…

EMPTY

No tasks this week

Add a task

PARTIAL / STALE

HousingRoom data may be 6 hours old.

OFFLINE

⊘You are offline. You can keep working — 2 changes will sync when you are back.

ERROR

!Canvas didn’t answer

Your draft is safe.

Retry

PERMISSION

⊙Only your advisor can edit this

Set by the Registrar.

SUCCESS

✓Registration submitted

Next: Review your spring week

Guidelines

Provenance and trust

Source: `docs/TRUST-CUES-AND-SOURCE-PRESENTATION.md`, `app/src/lib/where.ts`, `lib/status.ts`, `lib/source.ts`.

The ladder (strongest first)

Rank	Key	Word	Sentence
0	official	Official / Institution verified	Your institution's own record, read from its system.
1	connected	Connected	From a calendar or account you connected, checked recently.
2	made	Made here	The app built this from sources you gave it (rule-based extraction).
3	personal (yours)	Yours / Student entered	You entered this yourself. Nothing has checked it.
4	sample	Sample	Shipped as a demonstration. Not your semester.
5	stale	Out of date	From a connection not checked lately. It may have moved.

Display-only kinds: **AI-assisted, source-linked** (a model's draft), **Estimated** (Semester's arithmetic), **Needs review**, **External**.

- `stale` ranks below `yours` on purpose: an unchecked copy of something that may have moved is worth less than a guess that knows it is a guess.
- `made` ≠ `ai`. An extraction can misread; a draft can be wrong while sounding right. Different glyphs (○ vs ✦), different sentences.
- Estimates and AI are styled to be **noticed** (amber/violet wash) — the opposite of "official looks louder", because they are what students over-trust.
- Never claim `official` for anything the institution did not send.

Where cues go

1. The eyebrow or state row of an ObjectCard / ContextBar — never louder than the primary action.
2. A SectionLabel aside (`<SourceBadge>` beside "Released grades").
3. The **Source & details** sheet: Origin · Freshness · Used in · Who can see this · Limitations · Report incorrect information.
4. Charts: by pattern (solid / outline / hatched / dotted), plus a source line and "Illustrative demo data" when applicable.

Rules for new work

1. Say a state through `SOURCES` / `STATUSES` in `components/trust/vocabulary.js`; never hand-type "Official" or "Synced".
2. Anything with a source gets a Source & details route.
3. Anything a model drafted is `ai`, lists its sources and limitations, and is editable before it is kept.
4. Freshness is written as "Updated 3 days ago", or "Update time not recorded" — never "just now" for an unknown time.

AI governance in the interface

Source: `docs/BRAND-PLATFORM.md` §6.1, `TRUST-CUES-AND-SOURCE-PRESENTATION.md` (AI presentation), `components/StudyStudio.tsx`.

Stance: assistive, source-linked, reviewable, interruptible. A tool, not a person. Name: **Ask Semester**. No human name, avatar face, sparkle orb or "I" in marketing.

Every AI answer shows (AIResponse)

1. The label **AI-assisted** (violet chip, ✦) — always, wherever it speaks.
2. **Allowed data scope** ("Allowed to read BIO 210 materials, Your calendar"). The student can change scope before asking.
3. **Numbered sources** with their own provenance chips; inline `.ai-cite` markers.
4. **Confidence** and what it can't know ("time estimates are mine, not your instructor's").
5. **Request cost** and remaining allowance.
6. **Policy** that applies (course AI policy set by the instructor; institution switches).
7. **Actions** — anything that writes, sends, shares, pays or registers opens Dialog + ActionPreview first.

Show the work

Use StepStatus ("Reading sources → Drafting → Matching quotations → Ready"), marking the step that failed. Never fake an instant, certain answer. No shimmer, no typing animation.

Words

Say: "Ask Semester drafts a plan from the sources you gave it. Review it before you use it." · "It can be wrong. Check the source linked beside each answer." Never: "AI-powered", "intelligent", "knows you", "accurate", "no hallucinations", "handles it for you", "Ask anything".

Course policy

When an instructor restricts AI (e.g. outlining only), show it on the course workspace (object card "Course AI policy · set by instructor") and as a policy chip on every answer in that course; disable the disallowed actions with a reason rather than hiding them.

Consequential actions

Source: `components/unity/ActionPreview.tsx`, `enrollment/StudentRegistration.tsx`, `console/Approvals.tsx`.

Kind of action	Pattern
Reversible, personal (archive, move, rename)	Do it, then **UndoToast** ("Task archived. Undo"). No dialog.
Leaves Semester or someone else sees it (share, send, submit)	**Dialog + ActionPreview**: What happens / Exactly what / What stays the same / Taking it back
Academic record (enroll, drop, withdraw, regrade)	Two-step **Review → Confirm** in place; state where the request will land ("number 3 in line"; "a W is recorded")
Financial (pay, refund, waive)	Dialog + ActionPreview naming amount, method, processor ("Semester does not process the payment; TouchNet does"), refund route
Institutional high-risk (release hold, change role, break-glass)	**Two-person duty**: request → approve (≠ requester) → act; MFA fresh within 15 min; audit event written first

Writing the dialog

- Title is a question naming the object: "Drop CHEM 101?"
- Buttons name the outcome: "Keep CHEM 101" / "Drop — confirm". Focus lands on the safe one.
- Destructive uses red ink + red hairline (`variant="danger"`), never a solid red fill.
- Recovery is required: `undo` · `request` (with how) · `none` (with what to do if it was a mistake).
- Afterwards: a SuccessState receipt with what changed and "Next:" — never "All set!" while anything is pending.

Consent, sharing and family access

Source: `components/unity/Visibility.tsx`, `AdvisorMeeting.tsx`, brief §5 (Family and Guardian Portal).

- **Default private.** Guardians, advisors and classmates see nothing by default. Never imply default guardian access.
- **One decision at a time.** Each ask states what, with whom, why, for how long, and how to undo. Decline is equal weight.
- **The same question everywhere:** VisibilityPicker — Only me · Course · Selected collaborators · Public portfolio. Offer only the audiences the module can honour.
- **Show exactly what leaves.** Before a share, preview the payload ("will see exactly this, as 'Maya Lindqvist', until 5 Nov 2026"). Private notes are never shared or exported.

- **Expiry and revoke.** Shares end when the student says; revoke and delete live next to every share.
- **Access history.** DecisionTrail of who viewed what, when. Students see when a share was opened.
- **Protected categories** (counseling, health, disability services) are never shareable automatically; show the switch disabled with the reason.
- **Guardian register:** the consent boundary is the hero of the screen — what's shared, since when, by whose choice.

Institutional surfaces

Source: `components/console/*`, `institutional/*`, brief §4.

- **Context first.** Every console page opens with Environment (word + shape: ◆ Production), Scope, Operator, Role (grants), MFA freshness, Session, Support access. Staff workspaces carry a scope bar: Role · Scope · Signed in · "Every record view is logged".
- **Tables over cards** for anything compared across rows (integrations, audit, requests). Caps headers, hairline rows, tabular numerals, mono for IDs and correlation ids.
- **Fields** (label/value dl) for a single record.
- **DecisionTrail** for anything with authority: who, when, under which grant.
- **Errors separate cause classes:** retryable failure vs permission vs configuration. Name the system and the consequence for students ("Students see a stale label on Housing").
- **Implementation honesty.** Use "Planned", "Pilot", "Preview" chips beside the capability — never a roadmap presented as live.
- **Authority scope is always visible** for faculty and advisors (Can see / Can approve / Cannot). No copy suggests the tool judges staff or ranks students.

Accessibility contract

Target WCAG 2.2 AA (a design requirement, not a conformance claim).

Contrast — measured against every surface of a ground, not the flattering one

Element	Requirement
Body / UI text	≥ 4.5:1
Large text (≥24px or ≥18.66px bold)	≥ 3:1
Icons, borders, focus, chart marks	≥ 3:1 vs neighbour
Tertiary ("faint") ink	metadata only, never information a student must act on
Brass	base stop on dark only; shade stop (#695a33) for text on light

Structure & input

- One `main`, one visible H1, skip link; heading levels describe structure.
- Keyboard order follows reading order; focus never lands under sticky chrome (scroll margins 96 / 84px).
- 44px ordinary targets; 24px floor only in crowded composites with keyboard parity. Icon-only controls always named.
- Native inputs; persistent labels; hints and errors associated via aria-describedby; errors preserve input.
- Overlays: focus trap, Escape, outside dismissal, focus return; triggers report aria-expanded.

Never colour alone

Every state: glyph + word + tone. Provenance origin has a second carrier in fill (solid / outline / dashed); forced colours thicken the solid chip's border to 3px.

Reader settings the tokens respond to

Text size 0.94–1.18× (`--text-scale`) · Density 1 / .86 / .74 · Line height 1.35–1.95 · Letter/word spacing · Reading width 52–82ch · Body face incl. Atkinson Hyperlegible · Motion: Follow device / Less motion / Low stimulation (also removes gradients, shadows, quieter badges) · Increase contrast raises faint/dim/hairline alphas · 13 grounds + Match my device.

Motion

All roles zero under reduced motion and `data-calm`. No essential meaning depends on motion. No flashing, no parallax, no looping decoration, no countdown theatre.

Brand-exempt zones

Status/warnings, provenance, consent text and accessibility controls: body-grade type only (never Cinzel), never restyled by tenants, campaigns or themes, never animated away or hidden behind hover.

Layout and responsive behaviour

Tier	Width	Navigation	Page pad
Compact	< 760px	Labelled bottom TabBar: Today · My Path · Search · Plan · Me	18px
Medium	760–1099px	Compact rail (72px, icons + tooltip names)	20px
Wide	≥ 1100px	SideRail 248px (272 at very wide) with group labels	30px

- **Shell:** SideRail (Ink) │ SystemContextBar (44px: location · term · ⌘K · sync · tools) over a scrolling page.

- **Page:** pagehead (kicker · H1 · one-line purpose · ≤2 actions) → `cols` grid `minmax(0,1.6fr) minmax(300px,1fr)` → main column leads with the next action; aside holds "Needs you", context, policy. Stacks to one column under 1000px.
- **Measures:** reading 66ch; operational content ≤ 1180px.
- **Tables** keep row identity and the first action on compact screens; the table region scrolls, not the document.
- **Five canonical destinations** are the spine: Today, My Path, Search, Plan, Me. Specialist tools open in context (OpenIn), never as a sixth global product.
- **Workspace modes** (presentation only): Guided · Focused (navigation steps back, FocusBar is the way out) · Detailed (source sentences up front) · Accessibility.

Content templates

From `docs/BRAND-PLATFORM.md` §7.

Status sentence — what is true → what it means for you → what you can do.

> "Your Canvas calendar hasn't updated since Tuesday. Deadlines below may have moved. Reconnect it, or check the course page."

Empty state — why it's empty + one action. "No grades released yet. Your instructor has not released any BIO 210 grades for Fall 2026. Grades appear here once they do." [Work out what you need]

Error — what failed, what's safe, the way out, a reference. "Canvas didn't answer. Your draft is safe on this device." [Try again] [Use Canvas directly] Reference: SYNC-4F2A

Success — acknowledge, then Next. "Registration submitted. Next: review your spring week."

Permission change — what changed, why, who controls it, where to change it.

Incident — Status · What is affected · What is NOT affected (only if verified) · What it means for you · What you can do now · What we know / don't know · Next update (only if operations can keep it). Never "minor", "isolated", "out of an abundance of caution".

Onboarding — "Semester starts with what you have. Pick one: add a syllabus, connect a calendar, or use sample data." → first real next step with its source label → consent one at a time → no streaks, no points, Skip always available.

Dates — "Thu 9 Oct · 23:59" in UI; "3 Oct 2026" in documents. Always unambiguous; time zone when it crosses one.

Architecture and tooling

Maps of the whole

Master control, catalog, role lens — how the pieces point at each other

Master control

ui_kits/master-control/ — product, console & back end, business OS, company site, design system & engineering

Rule
Every new file is linked here

Master catalog

51 roles · 673 screens · 319 workflow steps (17 workflows) · 88 systems · 82 docs

Rule
Status from evidence: code, tests, release

Role lens

64 screens × 14 roles: rendering, sources, tables, RLS, API

Access levels
full · scoped · shared · read · audited · none

Rule: company roles (investor, CS, RevOps) never see student records. Role model in roles.js is design-level; verify against migrations.

Stack layers

Front end → middleware → API → database, with the rule for each

Front end

Student app
Five destinations, 64 screens

Staff + console
Same shell, role-scoped routes

Design system
Tokens → semantic CSS → components

Offline
Queue mutations, replay on reconnect

Middleware

Auth proxy
Refresh session; reject expired

Tenant resolver
Host/SSO → tenant_id on every request

Capability check
role × screen × grant first

Audit writer
Event written before the action runs

Server / API

Query RPCs
api.* reads of read models

Commands
private.* fns; idempotency key

Integrations
LTI 1.3, OneRoster, SIS read, SAML

AI gateway
Scope, policy, citations, spend log

Database

Supabase Postgres
RLS on every table

tenant_id
On every row

private.*
Authoritative, never exposed

read_model.*
Projections + version + freshness

Read-model pipeline & freshness

Write → outbox → projection → refetch; lag decides what the UI allows

WRITE → READ

1 write		2 outbox		3 project		4 stamp		5 invalidate		6 signal		7 render
Authoritative write	→	Event, same transaction	→	Worker applies, idempotent	→	Version + freshness	→	Cache namespaces	→	Targeted refetch	→	Server read model

FRESHNESS → UI POLICY (FRESHNESS.TS)

State	Lag	Display	Consequential actions
Live	≤ 5 s	Normal	Allowed if approvals satisfied
Recent	≤ 60 s	Normal	Normal guardrails
Delayed	> 60 s	Delay badge + age	Re-check source first
Failed / Rebuilding	—	Last good + notice	Held

AI gateway

Six checks between a question and an answer — student records never leave

Request		Identity + scope		Policy		Routing		Answer		Evidence
Student, staff or institution asks	→	Who; which data they may read	→	Course AI policy, institution switches, FERPA boundary	→	Approved model per task; search, calendar, materials	→	Labelled AI-assisted, sources, confidence	→	Citations, cost, outcome logged

Environments

Local, staging, production on Vercel + Supabase — data never crosses

Environment	Hosting	Data	Access
Local	Supabase CLI + Next.js dev	Synthetic seed	Engineers
Staging	Vercel preview + Supabase staging	Synthetic + institution test IdP accounts	Engineers, institution IT testers
Production	Vercel production + Supabase production (separate keys)	Real student records under the DPA	Named operators, MFA, access logged

Rule: no copying data between environments; secrets set per environment. Cutover steps: docs/launch/Cutover Plan.html

Document library by stream

docs/ — what exists per stream and what is next

Stream	Built	Next
Brief	One-pager, FAQ, role sheets, overview deck	Real screenshot on slide 4
Guides	Architecture, developer, user	Admin manual, tutorials
Launch	Runbook, cutover plan, day-1/week-1 support	—
Secure	10 policies; HECVAT, SOC 2, pen test, insurance readiness	Stream index
Legal (drafts)	Terms, privacy, community, subprocessors	MSA, order form, DPA, cookie, accessibility
Build	—	Engineering playbook, QA and accessibility test plans

Rule: formal documents are doc_page files under docs/<stream>/, linked from docs/index.html. Legal stays draft until attorney review.

semester-core

Workflow decision logic as pure TypeScript — handoff/semester-core/

Module	Decides	Rendered by
provenance.ts	Source rank, conflicts, freshness label	SourceBadge, ProvenanceChips
nextaction.ts	Order of a student's next actions	Today, NextSteps
access.ts	Role × screen, share expiry, break-glass	Role lens, route guards
freshness.ts	Lag → display / safe / destructive	Read models, lag simulator
consequence.ts	Undo / preview / two-person	Dialog, ActionPreview
aigateway.ts	Scope + policy before any model call	AIResponse
approvals.ts	Two-person duty state machine	Console approvals
scoring.ts	Weighted 0–5 grading	Workflow lab
pilot.ts	Launch readiness	Business gap analysis

No dependencies, no I/O, vitest proof per module. Files end in .txt; strip on copy to app/src/lib/core/.

semester-platform

Migrations 010–040 with RLS, API routes and tests — handoff/semester-platform/

010 Core	020 Community	030 Market	040 Ops
core.institutions, core.profiles, audit.events	groups, members, posts, reports, events, rsvps	listings, offers, drops, bids	staff, accounts, invoices, social_posts, settings

supabase/migrations
*.sql.txt → .sql

→

app · lib
validation, 2 API routes

→

tests
RLS + unit

→

npx supabase db reset
then npm test

Status: written to the architecture guide. Not yet run, typechecked or tested.

AI Workflow Router & Benchmark Lab

Prototype in ui_kits/workflow-lab/ → production source in handoff/workflow-router-app/

Prototype · ui_kits/workflow-lab/

Router
6 inputs → tool, confidence, reasons, limits, escalation, prompts

Matrix
16-column feature matrix, 12 canonical workflows

Benchmark
Persistence tests, 0–5 grading, weighted /5 and /100, leaderboard, CSV/MD export

Saved on this device only; labelled prototype mode.

Production · handoff/workflow-router-app/

Stack
Next.js App Router, TypeScript strict, Tailwind, Supabase (@supabase/ssr, magic link), Zod, Server Actions

Data
001_core_workflow_router, 002_benchmark_grading; org-scoped RLS (owner · admin · member · viewer); seed with 12 workflows

Tests
vitest suites, RLS test plan

Run install.sh. Not yet typechecked, linted, tested or built.

Claude Code toolkit

Token pipeline, files and commands — handoff/claude-code-toolkit/

lib/look.ts
tokensFor — every value
→
styles/tokens.css
semantic names
→
lib/tokenexport.ts
buildTokenExport
→
semester.tokens.json
generated · never hand-edit
→
FIGMA-MAPPING.md
Figma variables · checked

ADDS TO THE REPO

.claude/skills/audit-semester-design-sync/ read-only audit

.claude/skills/build-semester-ui/ build rules

.mcp.json Figma remote MCP

docs/FIGMA-MAPPING.md parity table

app/scripts/design-system-audit.mjs 4 checks

app/scripts/design-system-report.mjs CI markdown

app/src/lib/designsystem.test.ts proves each check

RUN

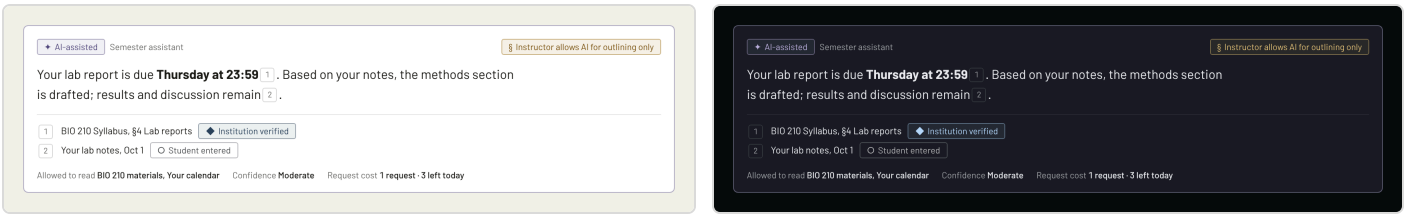
```
claude mcp add --scope project \
  --transport http figma https://mcp.figma.com/mcp
/audit-semester-design-sync app/src
cd app && npm run design-system:check
```

No new dependency, token source, Tailwind, Storybook or package.

Every component, light and dark

Each component in Parchment (light, left) and Ink (dark, right).

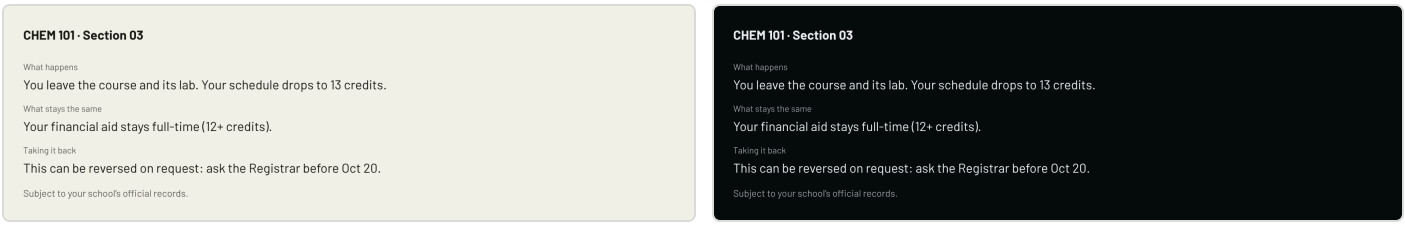
AIResponse AI



ProvenanceChips AI



ActionPreview ACTIONS



Button ACTIONS



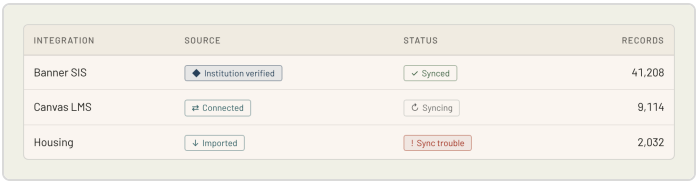
IconButton ACTIONS

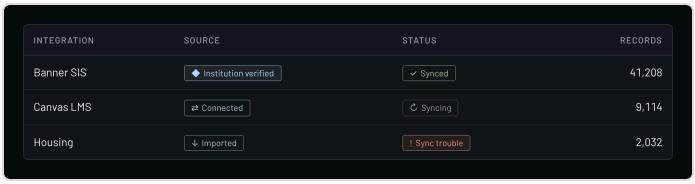


QuickActions ACTIONS



DataTable DATA





DecisionTrail DATA



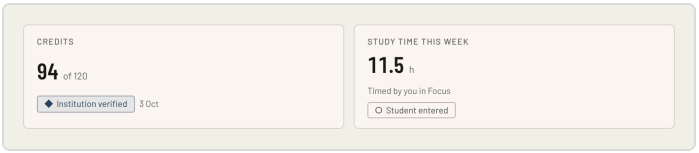


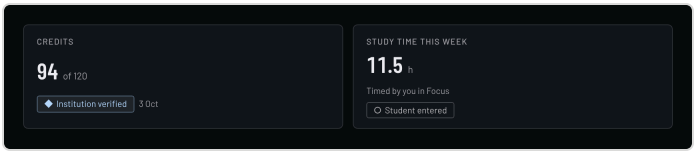
Fields DATA



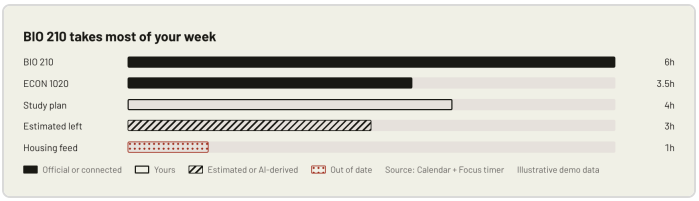


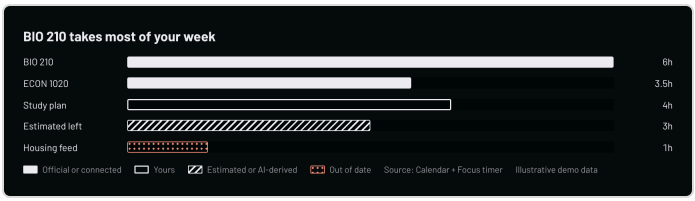
MetricTile DATA





ProvenanceChart DATA



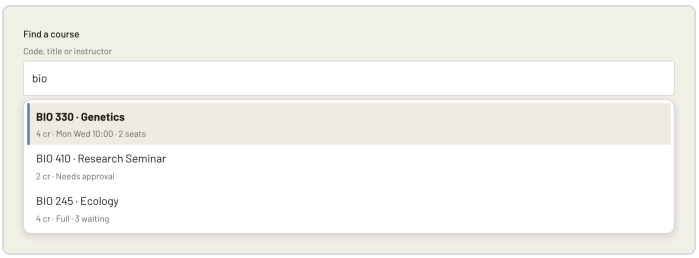


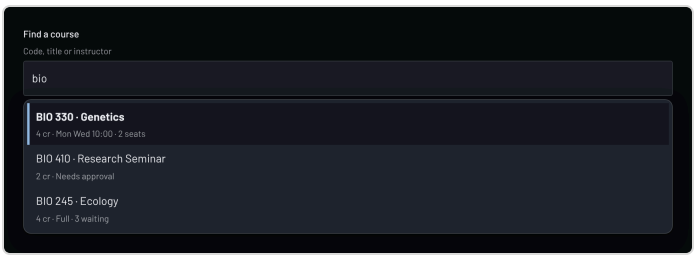
Choice FORMS



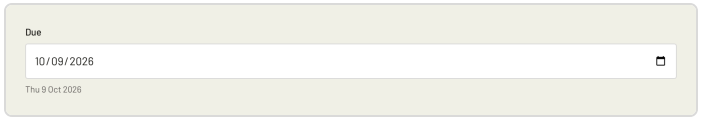


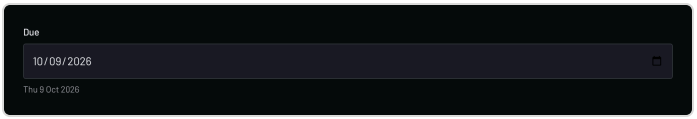
Combobox FORMS





DateField FORMS





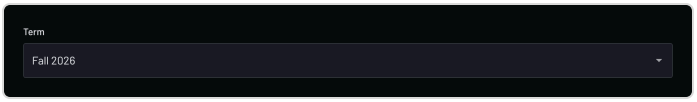
Segmented FORMS





Select FORMS





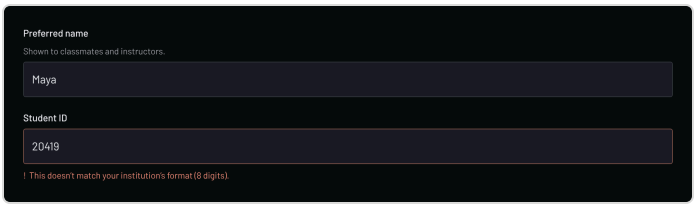
Switch FORMS



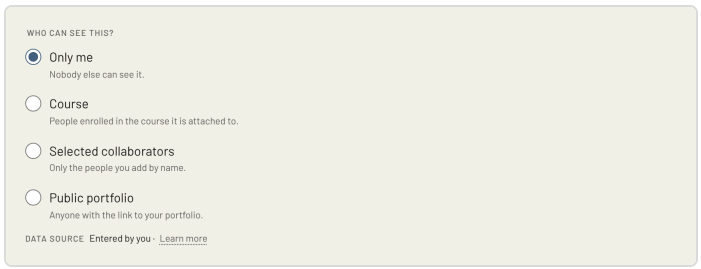


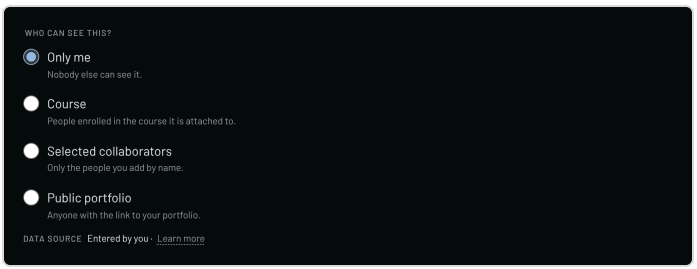
TextField FORMS





VisibilityPicker FORMS





Avatar GLYPH





Icon GLYPH

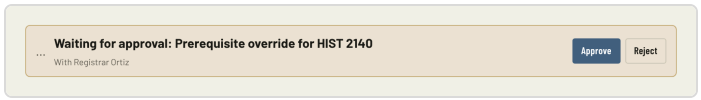


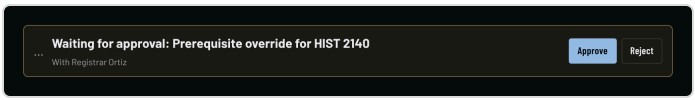
Wordmark GLYPH



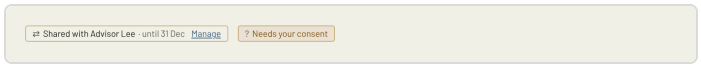


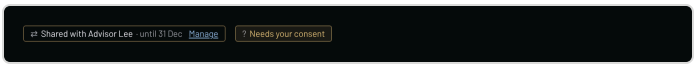
ApprovalBanner GOVERNANCE





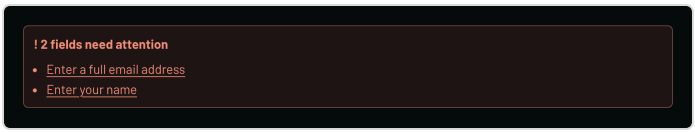
ConsentBadge GOVERNANCE





ErrorSummary GOVERNANCE



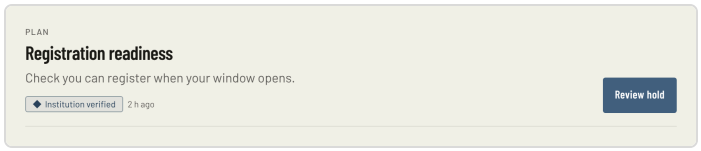


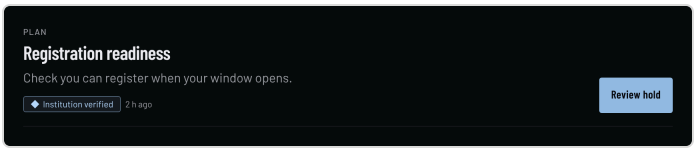
HealthBadge GOVERNANCE





PageHeader GOVERNANCE

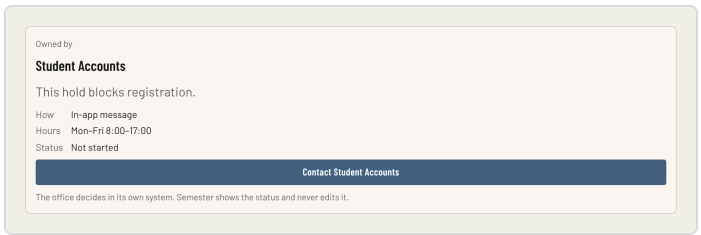


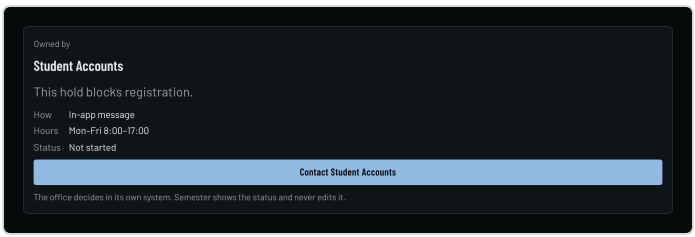


PolicyBadge GOVERNANCE

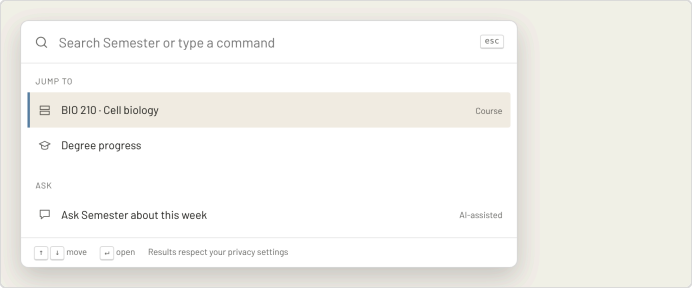


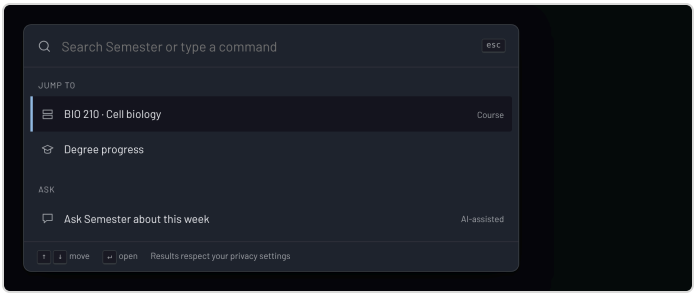
SupportHandoff GOVERNANCE



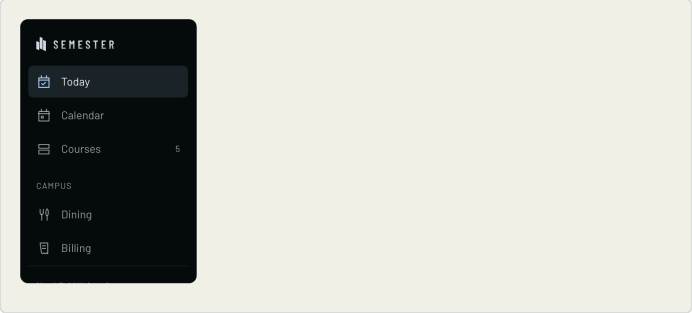


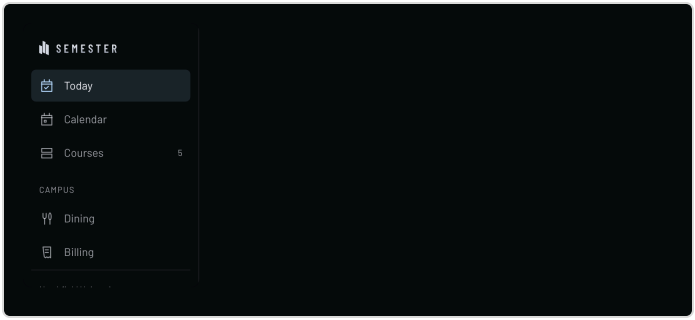
CommandPalette NAVIGATION



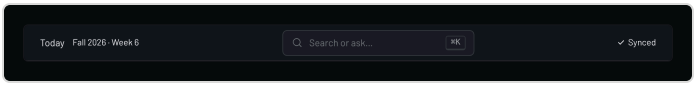


SideRail NAVIGATION

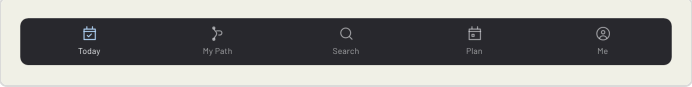


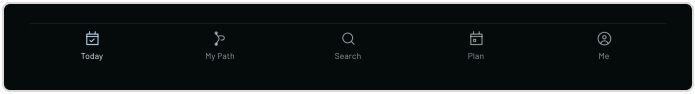


SystemContextBar NAVIGATION

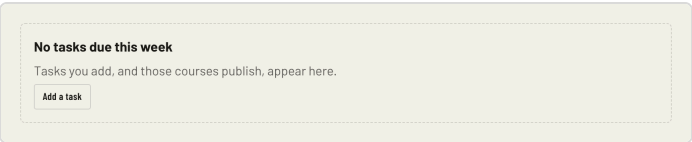


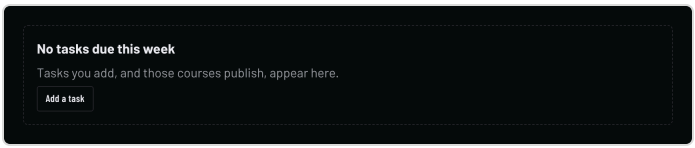
TabBar NAVIGATION



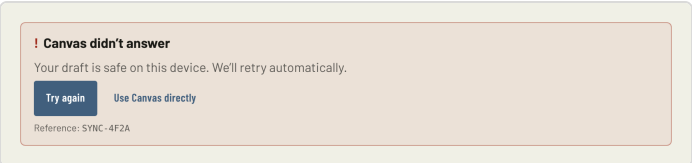


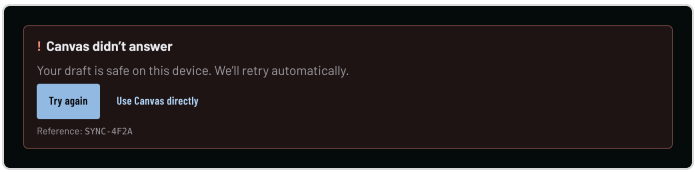
EmptyState STATES





ErrorState STATES





LoadingState STATES



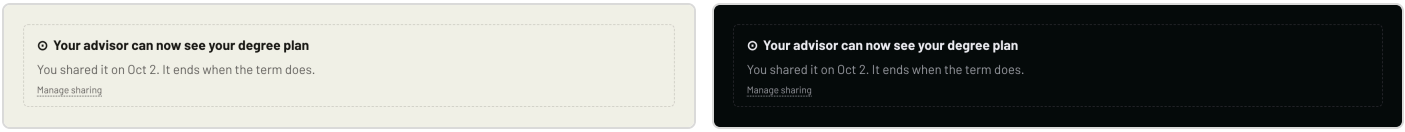
Notice STATES



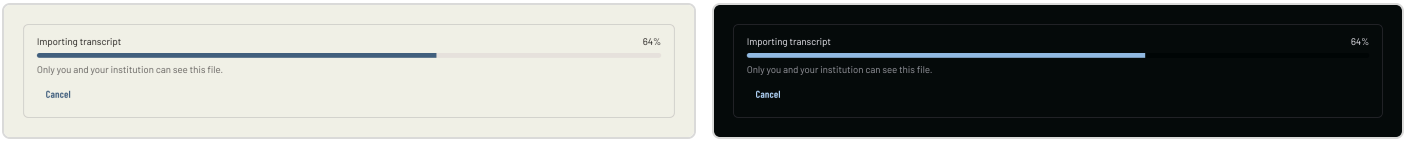
OfflineStrip STATES



PermissionNotice STATES



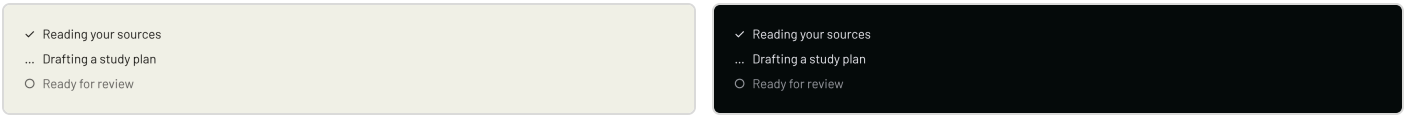
ProgressState STATES



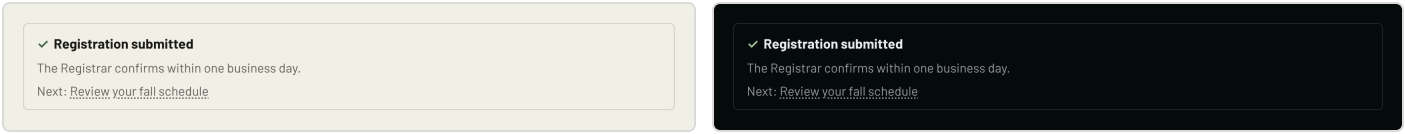
SaveState STATES



StepStatus STATES



SuccessState STATES

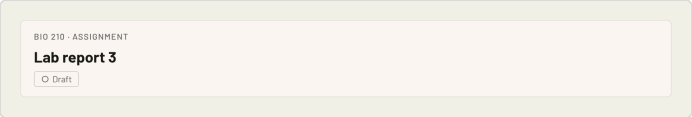


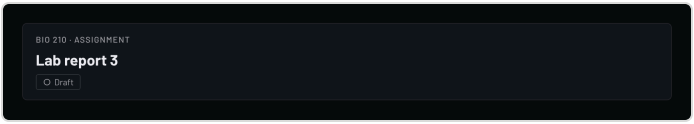
UndoToast STATES



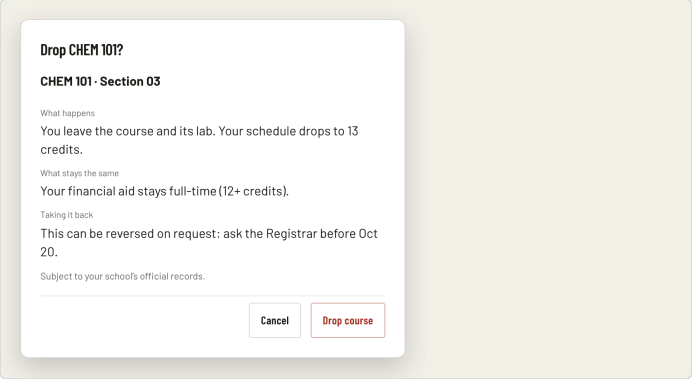


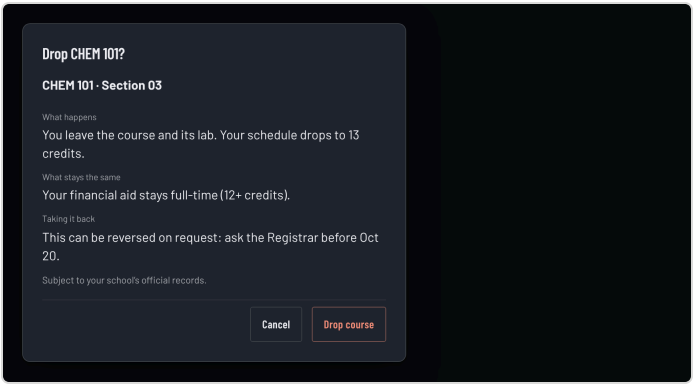
ContextBar SURFACES





Dialog SURFACES

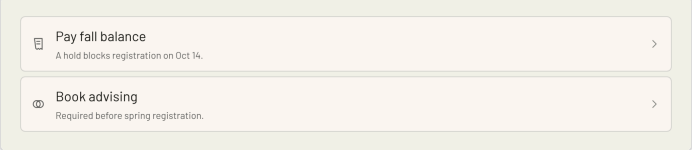


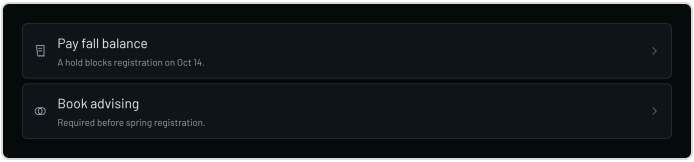


FocusBar SURFACES

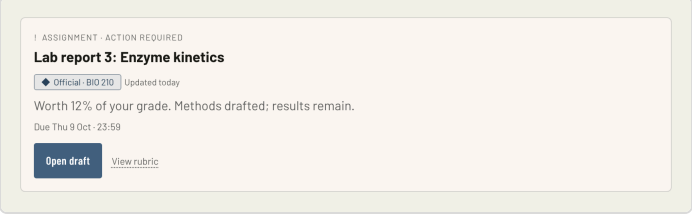


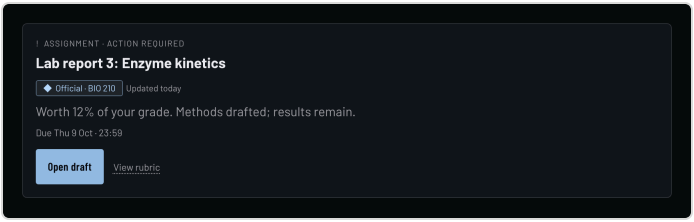
NextSteps SURFACES





ObjectCard SURFACES

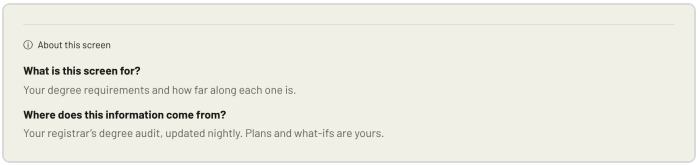


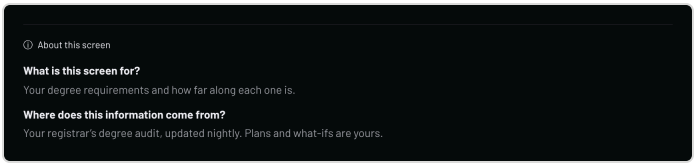


OpenIn SURFACES



ScreenGuide SURFACES

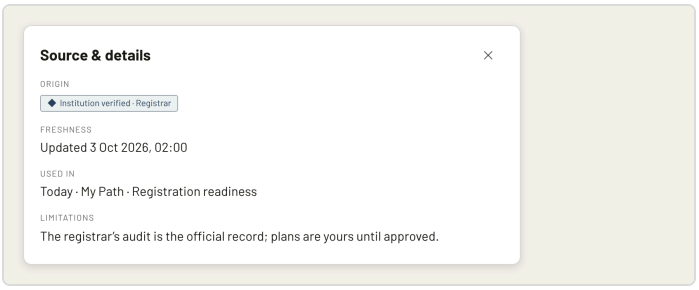


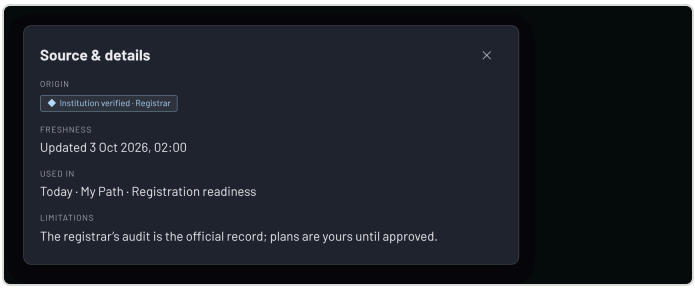


SectionLabel SURFACES



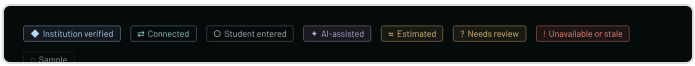
Sheet SURFACES





SourceBadge TRUST & PROVENANCE



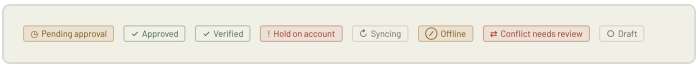


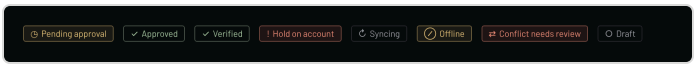
SourceLine TRUST & PROVENANCE





StatusChip TRUST & PROVENANCE





Design language

docs/system/README.md

Design, intelligence, workflow and data system

One brand language · one source-authority language · one action language · one policy language · one privacy language · one error-recovery language · one accessibility standard · one workflow grammar · one education graph = one Semester OS.

- design/SEMESTER_BRAND_IDENTITY.md
- design/SEMESTER_VISUAL_LANGUAGE.md
- design/SEMESTER_UI_GRAMMAR.md
- design/SEMESTER_COMPONENT_LIBRARY.md
- design/SEMESTER_INTERACTION_LANGUAGE.md
- design/SEMESTER_MOTION_AND_FEEDBACK.md
- design/SEMESTER_DATA_TRUST_UI.md
- design/SEMESTER_RESPONSIVE_STRATEGY.md
- algorithms/SEMESTER_ALGORITHM_CATALOG.md
- algorithms/SEMESTER_ALGORITHM_GOVERNANCE.md
- algorithms/SEMESTER_NEXT_ACTION_RANKING.md
- algorithms/SEMESTER_WORKLOAD_MODEL.md
- algorithms/SEMESTER_REGISTRATION_READINESS.md
- algorithms/SEMESTER_SEARCH_RANKING.md
- algorithms/SEMESTER_AI_ROUTING.md
- workflows/SEMESTER_WORKFLOW_ARCHITECTURE.md
- workflows/SEMESTER_WORKFLOW_CATALOG.md
- data/SEMESTER_DATA_FLOW_ARCHITECTURE.md
- data/SEMESTER_DATA_CLASSIFICATION.md
- data/SEMESTER_SOURCE_AUTHORITY.md
- data/SEMESTER_PROVENANCE_MODEL.md

Next 25 implementation actions

1. Decide brand hue: keep Semester blue or move to indigo
2. Build PolicyBadge
3. Build ConsentBadge
4. Build HealthBadge
5. Build SupportHandoff
6. Build ApprovalBanner

7. Build ErrorSummary
8. Promote PageHeader to a component
9. Add data-trust panel to every critical screen
10. Wire freshness.ts to SourceBadge
11. Ship next-action ranking with explanation UI
12. Ship registration readiness states
13. Write algorithm governance records for the 6 pilot algorithms
14. Bias/fairness review for caseload and risk signals
15. Evaluation dataset for next-action ranking
16. Search permission filter + authority ranking
17. AI router transparency panel
18. Implement the SemesterWorkflow type in the repo
19. Universal workflow state machine
20. Data classification tags on every table
21. Provenance columns on canonical tables
22. Analytics event taxonomy (no PII)
23. axe + keyboard tests per component
24. Screen consistency audit against the UI grammar
25. Regenerate docs/master from the repo

`docs/system/design/SEMESTER_UI_GRAMMAR.md`

UI grammar

Semester design, intelligence, workflow and data system · 2026-10-05 · Source: uploads/expand further on brand identy algorthims workflow.pdf, reconciled with this design system (readme.md, tokens/, components/). Where they differ, this design system wins and the difference is listed.

> Semester should make an institutionally complex world feel personally clear.

Universal page shell

Context bar (logo, location, search ⌘K, notifications, profile) → primary navigation (Today · Path · Plan · Courses · Workspace · Support · Career; rail on desktop, tab bar on mobile) → page title + what it helps you do + source/freshness → Layer 1 → Layer 2 → Layer 3. Built from SystemContextBar, SideRail, TabBar and the student-screen template.

Three-layer disclosure

Layer	Purpose	Example
1	Immediate decision	“Your registration opens Tuesday at 8:00 AM. You have one hold.”
2	Explanation and action	“The hold is from Student Accounts. Here’s why it matters and who can help.”

Layer	Purpose	Example
3	Advanced detail	Source system, policy text, history, export, integration state, audit trail

A student never needs Layer 3 for a normal daily action. Staff use Layer 3 often but start collapsed.

Every screen answers

- Where am I?
- What is this screen for?
- What matters now?
- What can I do?
- What is official?
- What is stale or uncertain?
- What happens if I take this action?
- Who can help?
- Where is deeper detail?

Standard panels

Data trust: Source · Updated · Authority · Status · Policy (with version) · Action (view official details). Built from SourceBadge + SourceLine + Fields.

AI answer: Answer · Sources · What this means · Suggested next step · Limits and uncertainty · Course/institution policy · Give feedback · Ask a person / official office. Built from AIResponse + ProvenanceChips.

Error: What happened · What data may be affected · What you can do now · Whether it saved · Whether it will retry · Who can help · Reference ID. Built from ErrorState.

`docs/system/design/SEMESTER_COMPONENT_LIBRARY.md`

Component library

Semester design, intelligence, workflow and data system · 2026-10-05 · Source: uploads/expand further on brand identy algorthims workflow.pdf, reconciled with this design system (readme.md, tokens/, components/). Where they differ, this design system wins and the difference is listed.

> Semester should make an institutionally complex world feel personally clear.

Required shared components mapped to this design system (`components/`, namespace `SemesterDesignSystem_e9f3ba`).

Required	In this system	Status
AppShell	Shell templates (student-screen, console-screen, platform-shell)	Have
ContextBar	ContextBar, SystemContextBar	Have

Required	In this system	Status
GlobalNavigation	SideRail	Have
MobileNavigation	TabBar	Have
GlobalSearch / CommandPalette	CommandPalette	Have
PageHeader	PageHeader	Have
SourceBadge	SourceBadge	Have
FreshnessBadge	SourceBadge updated prop + freshness.ts	Partial
AuthorityBadge	SourceLine by/authority	Partial
PolicyBadge	PolicyBadge	Have
ConsentBadge	ConsentBadge (+ VisibilityPicker input)	Have
StatusBadge	StatusChip	Have
HealthBadge	HealthBadge	Have
RiskBanner	Notice	Partial
ActionCard	ObjectCard	Have
ActionCenter	NextSteps	Have
WorkflowStepper	StepStatus	Have
DecisionCard	ActionPreview + Dialog	Have
EvidenceDrawer	Sheet + SourceDetails	Have
AuditTimeline	DecisionTrail	Have
DataTable	DataTable	Have
EmptyState / LoadingState / ErrorState	EmptyState, LoadingState, ErrorState	Have
OfflineState	OfflineStrip	Have
ForbiddenState	PermissionNotice	Have
SupportHandoff	SupportHandoff	Have
AIAnswerPanel	AIResponse	Have
CitationPanel	ProvenanceChips	Have
ActionPreview	ActionPreview	Have
ApprovalBanner	ApprovalBanner	Have
FormField	TextField, Select, Combobox, DateField, Choice, Switch	Have
ErrorSummary	ErrorSummary	Have
Modal	Dialog, Sheet	Have
Toast	UndoToast	Have
NotificationCenter	Student Notifications screen	Pattern only

Gaps closed (Oct 2026): PolicyBadge, ConsentBadge, HealthBadge, SupportHandoff, ApprovalBanner, ErrorSummary, PageHeader in components/governance/.

Every component defines

Semantic token usage · accessibility requirement · responsive behaviour · loading/empty/error/offline/forbidden states · analytics event · source/freshness display · test coverage. Recorded in each component's `.prompt.md` and `.d.ts`.

Modules may not invent their own colours, spacing, status vocabulary, permission behaviour or source labels.

`docs/system/design/SEMESTER_INTERACTION_LANGUAGE.md`

Interaction language

Semester design, intelligence, workflow and data system · 2026-10-05 · Source: uploads/expand further on brand identy algorthims workflow.pdf, reconciled with this design system (readme.md, tokens/, components/). Where they differ, this design system wins and the difference is listed.

> Semester should make an institutionally complex world feel personally clear.

Situation	Rule
Alerts	Only actionable; one primary per view; glyph + words; never colour alone
Official data	Source + authority + freshness shown; official items outrank derived ones
AI uncertainty	Confidence and limits stated; sources numbered; human or office offered
Approvals	Preview first, consequence in words, second approver when consequential, audit before apply
Policy restrictions	Say which policy, who owns it, what you can do instead
Recovery after errors	Say whether it saved, whether it retries, who can help, reference ID
Consent	Recipient, category, purpose, duration, items; revoke always visible
Destructive actions	Undo where possible; otherwise a request path; never silent

---

`docs/system/design/SEMESTER_MOTION_AND_FEEDBACK.md`

Motion and feedback

Semester design, intelligence, workflow and data system · 2026-10-05 · Source: uploads/expand further on brand identy algorthims workflow.pdf, reconciled with this design system (readme.md, tokens/, components/). Where they differ, this design system wins and the difference is listed.

> Semester should make an institutionally complex world feel personally clear.

Event	Behaviour	Token
Hover / press	Wash, no scale	130 ms
State change	Direct	180 ms
Panel	Subtle	240 ms
Sheet / modal	Rise 8 px + fade, direct	280 ms
Save state	Always visible (SaveState)	—
Completion	Calm check, no confetti	—

Event	Behaviour	Token
Loading	Non-distracting skeleton (LoadingState), no spinners	—
Reduced motion / calm mode	All motion zeroed	prefers-reduced-motion, data-calm

---

docs/system/design/SEMESTER_RESPONSIVE_STRATEGY.md

Responsive strategy

Semester design, intelligence, workflow and data system · 2026-10-05 · Source: uploads/expand further on brand identy algorthims workflow.pdf, reconciled with this design system (readme.md, tokens/, components/). Where they differ, this design system wins and the difference is listed.

> Semester should make an institutionally complex world feel personally clear.

Surface	Strategy
Student	Mobile-first; five-destination tab bar; offline-safe for student-owned data
Staff / institution / ops	Desktop-first; dense tables; rail + context bar
Public site	Fluid; single column under 860 px
All	Reflow at 320 px; 200% zoom; 44 px targets (24 px floor in dense composites)

---

docs/system/design/SEMESTER_DATA_TRUST_UI.md

Data trust UI

Semester design, intelligence, workflow and data system · 2026-10-05 · Source: uploads/expand further on brand identy algorthims workflow.pdf, reconciled with this design system (readme.md, tokens/, components/). Where they differ, this design system wins and the difference is listed.

> Semester should make an institutionally complex world feel personally clear.

Every critical screen shows: Source · Updated · Authority · Status · Policy (versioned) · Action. Example: "Source: official institution record · Updated 2 hours ago · Authority: Registrar · Status: current · Policy: registration policy v4.2 · View official details."

Freshness states come from semester-core freshness.ts (live ≤ 5 s, recent ≤ 60 s, delayed, failed, rebuilding); delayed data shows a badge and blocks destructive actions until confirmed at the source.

Figma mapping and design-system baseline

handoff/claude-code-toolkit/docs/design-system/README.md

Semester design-system workflow

Order of authority: `lib/look.ts` → `styles/tokens.css` → `lib/tokenexport.ts` → `design-tokens/semester.tokens.json` (generated) → `docs/design-system/FIGMA-MAPPING.md` → Figma.

Never hand-edit: `app/design-tokens/semester.tokens.json`, `app/reports/`.

Skills

- `/build-semester-ui <screen> [figma-url]` — build or change UI from existing primitives.
- `/audit-semester-design-sync [figma-url] [path]` — read-only report.
- `/create-semester-component <name> <screens>` — spec-first shared component.

Figma MCP (each developer, once)

```
 claude mcp add --scope project --transport http figma https://mcp.figma.com/mcp
claude    # then run /mcp and authenticate in the browser
```

`.mcp.json` holds only the endpoint; authentication is interactive and per developer. CI never authenticates to Figma.

Checks (from `app/`)

Command	What it does
`npm run tokens:check`	Fails if the committed JSON differs from `buildTokenExport`
`npm run design-system:audit`	Raw values vs the hex LEDGER, undefined variables, tokens.css ↔ export, Figma mapping — file:line
`npm run design-system:report`	Writes `app/reports/design-system/report.md` + `.json`, runs 8 existing contract tests
`npm run design-system:check`	All of the above that gate a PR

Adding a mapping

Add a row to `FIGMA-MAPPING.md` whose first path cell is `semantic.<name>` or `primitive.<name>` as it appears in the JSON. Run `npm run design-system:audit`; a path that does not resolve fails as *missing*.

Reading the report

Violations (blocker/major) fail CI. **Warnings** (minor) are raw inline values with the token to use instead; they do not fail until a baseline is agreed. **Unmapped candidates** are exported semantic tokens no Figma variable uses yet — informational.

---

handoff/claude-code-toolkit/docs/design-system/SEMESTER-DESIGN-SYSTEM-BASELINE.md

Semester design-system baseline

Read-only audit of harrisonjrubin7-cmyk/semester @ main (5eba494), 2026-10-05. Evidence is quoted from files read through the GitHub API; nothing was executed.

1 · Current architecture

Concern	Evidence
Stack	Vite + React + TypeScript under `app/` (`app/package.json`: `"build": "tsc -b && vite build"`); npm workspaces at root, one lockfile (`CLAUDE.md`)
Styling	Plain CSS: `styles/app.css` (~321 KB), `industry.css`, `unity.css`, `features.css`, `tokens.css`, `typefaces.css` + inline style objects using `var(--*)`
Value authority	`lib/look.ts` `tokensFor` — 13 grounds × 11 accents, density, corners, text size, text spacing, written as CSS custom properties on the root
Semantic authority	`styles/tokens.css` — "No colour is written here as a hex"; held by `styles/tokens.test.ts`
Export	`lib/tokenexport.ts` `buildTokenExport(tokensCss, appCss)` → `design-tokens/semester.tokens.json` (`$schema: semester.tokens/1`), regenerated by `npm run tokens:export` (`TOKENS=write vitest run src/lib/tokenexport.test.ts`)
Components	`components/` (~273 files), shared primitives in `components/unity/` (ObjectCard, ContextBar, Status, ProvenanceChips, ActionPreview, States, Visibility, Table, Combobox, DateField, OpenIn, NextSteps, ScreenGuide…), `components/ui.tsx`
Screens	`app/src/screens.tsx` — exhaustive `Record<Screen, ComponentType>` of ~95 lazy screens; areas in `lib/navareas.ts`
Enforcement	19 `styles/*.test.ts` contracts (breakpoints, deadcss, density, fields, glass, gutter, hex, inset, mapcredit, motion, print, reach, scale, stacking, sweepdensity, taps, textbuttons, textscale, tokens); `lib/contrast.test.ts` (143 pairings); `lib/tokenexport.test.ts`; lint = `oxlint` + `scripts/styles.mjs` + `labels.mjs` + `terms.mjs`; `census:design`; `sweep:contrast/targets/spacing/walls`
CI	12 workflows incl. `ci.yml` (build job runs `npm run lint`, `npm test`, `npm run build` in `app/`), `contrast.yml`, `drift.yml`
Claude	`CLAUDE.md` (check main first; decisions take PR numbers; gate list; proof standards); skills `add-course`, `debugging-and-error-recovery`, `run`, `security-and-hardening`, `test-driven-development`

2 · Source-of-truth map

Layer	File	Guard
Values	`app/src/lib/look.ts`	`lib/contrast.test.ts`, `look.test.ts`

Layer	File	Guard
Fixed primitives	`app/src/styles/app.css` first `:root` (`--type-*` , `--sp-*` , `--leading-*` , `--lift-*` , `--ease` , `--fast`)	`scale.test.ts` , `density.test.ts`
Semantic	`app/src/styles/tokens.css`	`tokens.test.ts`
Generated	`app/design-tokens/semester.tokens.json`	`tokenexport.test.ts` snapshot
Docs	`DESIGN-SYSTEM-GUIDE.md` , `docs/DESIGN-TOKENS.md` , `docs/COLOR-AND-DARK-MODE-SPEC.md` , `docs/TRUST-CUES-AND-SOURCE-PRESENTATION.md`	—
Figma	Intended consumer of the JSON (`tokenexport.ts` header)	**None** — no `.mcp.json` , no mapping file

3 · Strengths to preserve

The value/semantic split with no colours in `tokens.css` ; the generated export and its snapshot test; the shrinking hex LEDGER in `hex.test.ts` (10 files, each with a reason); per-ground contrast measured on every surface; motion roles zeroed by `prefers-reduced-motion` and `data-calm` ; one trust vocabulary (`lib/status.ts` , `lib/source.ts`); the exhaustive screen table; CLAUDE.md's "revert the fix and watch the guard go red" standard.

4 · Gaps

Area	Gap	Evidence
Persistent instructions	CLAUDE.md has no design-system section (source-of-truth order, raw-value rule, Figma protocol)	CLAUDE.md headings: main check, decisions, gates, proving, contrast
UI-generation procedure	No skill for building UI from unity primitives	`.claude/skills/` list
Audit procedure	No design-sync audit skill	same
Component creation	No rule against one-off wrappers	same
Figma governance	No MCP config, no mapping	tree search: no `.mcp.json` , no `docs/design-system/`
Raw-value enforcement	Hex in .tsx is enforced; numeric zIndex / ms / cubic-bezier / box-shadow / fontSize / borderRadius in inline styles are not reported with file:line	`hex.test.ts` covers quoted hex only; `DESIGN-TOKENS.md` : layer literals "were not replaced"
Token drift	Enforced (snapshot) but no named check script	`tokens:export` writes; no read-only `tokens:check`
CI reporting	No design-system report artifact	`ci.yml` build job

5 · Risk classification

Severity	Finding	Evidence
Major	Figma parity is unenforceable: no mapping exists, so a Figma variable rename or a new Figma-only token cannot be caught	no mapping file; `tokenexport.ts` names Figma as a consumer
Major	Agent sessions can introduce raw inline z-index/duration/shadow values that no test reports by line	`DESIGN-TOKENS.md` Layers note; only `stacking.test.ts` covers stylesheet literals
Minor	No read-only token drift command; `tokens:export` both checks and (with `TOKENS=write`) writes	`app/package.json:16`
Minor	No design-system section in CLAUDE.md	CLAUDE.md

6 · Minimal plan

Priority	Path	Change	Acceptance
P0	`CLAUDE.md`	Append the section in `CLAUDE.md.design-system-section.md`	Section present; existing sections unchanged
P0	`.claude/skills/{build-semester-ui,audit-semester-design-sync,create-semester-component}/SKILL.md`	Add three skills	`/build-semester-ui` etc. listed in Claude Code
P0	`app/scripts/design-system-audit.mjs`, `design-system-report.mjs`	Add (zero deps)	`npm run design-system:audit` exits 0 on main
P0	`app/src/lib/designsystem.test.ts`	Add	Each check fails on its fixture; real tree clean
P1	`app/package.json`	`tokens:check`, `design-system:audit`, `:report`, `:check`	Scripts run from `app/`
P1	`docs/design-system/FIGMA-MAPPING.md`	Template with verified code paths	Audit: 0 missing mappings
P1	`.mcp.json`	Remote Figma endpoint only	No credentials in file
P2	`.github/workflows/ci.yml`	Two steps in the build job after lint + artifact upload	Report artifact on PRs
P2	`.gitignore`	`app/reports/`	Report not committed

Deferred: replacing existing inline z-index/duration literals (warnings only until a baseline is agreed); fetched Figma variable names; visual regression; Figma writes (never).

`handoff/claude-code-toolkit/docs/design-system/FIGMA-MAPPING.md`

Figma ↔ code mapping

> **Status: template.** No Figma file has been inspected yet, so every *Figma* name below is a **proposed** name, not a fetched one. The *Exported path* column is real: each path was checked against `app/design-tokens/semester.tokens.json` at main@5eba494. When a Figma file is connected, replace proposed names

with fetched ones and set **Status** per row.

Ownership

Concern	Owner
Behaviour, accessibility, responsive layout, public component APIs, token values	Code
Approved visual intent, exploration	Figma
Token pipeline	`lib/look.ts` (values) → `styles/tokens.css` (semantic names) → `lib/tokenexport.ts` → `design-tokens/semester.tokens.json` (generated)

Mapping rules

1. No Figma variable becomes a production token without a code PR to `look.ts` / `tokens.css` and a regenerated export.
2. No Figma component becomes a React component automatically — map it to an existing pattern first (`build-semester-ui`, `create-semester-component`).
3. Missing states in Figma never remove code requirements (loading, empty, error, disabled, permission, keyboard, focus, responsive).
4. Anything unresolved goes in **Unresolved** below with an owner.
5. Status vocabulary: `Match` · `Partial` · `Missing in Figma` · `Missing in code` · `Obsolete` · `Proposed (unverified)`.

Unresolved

Item	Side	Owner	Note
Figma file URL	Figma	Design	Not provided; all Figma names are proposals

Figma variables are *imports* of `app/design-tokens/semester.tokens.json` (generated by `npm run tokens:export`), never the other way round. `npm run design-system:check` fails when a path in the **Exported path** column does not exist in that file, and the report lists semantic tokens with no mapping as non-required candidates.

Rows are machine-read: the first column that starts with `primitive.` or `semantic.` is checked. Keep one variable per row.

Collections

Figma collection	Modes	Exported collection	Notes
Semester / Ground	ink, graphite, midnight, basalt, oxide, forest, wine, industry-dark, parchment, paper, bone, industry, fog	ground	default **ink**
Semester / Accent	sterling, brass, copper, jade, slate, pewter, industry, oxblood, moss, ink, gold	accent	values resolved on the default ground

Figma collection	Modes	Exported collection	Notes
Semester / Density	comfortable, snug, tight	density	multiplies `--sp-*`
Semester / Corners	drawn, square, soft, round	corners	drives `--r-*`
Semester / Text size	compact, normal, large, largest	textSize	`text-scale` 0.94 / 1 / 1.09 / 1.18
Semester / Text spacing	normal, open, wide	textSpacing	
Semester / Base	default	stylesheet + constant	type, spacing, leading, lift, easing

Semantic variables

Exported path	Figma variable (proposed)	CSS	Use
semantic.surface-sunken	surface/sunken	—surface-sunken	Input wells, tracks
semantic.surface-canvas	surface/canvas	—surface-canvas	The page
semantic.surface-base	surface/base	—surface-base	Standard content surface
semantic.surface-raised	surface/raised	—surface-raised	Cards above base
semantic.surface-overlay	surface/overlay	—surface-overlay	Sheets, menus, Focus bar
semantic.surface-critical	surface/critical	—surface-critical	Error wash
semantic.text-primary	text/primary	—text-primary	Anything read to act
semantic.text-secondary	text/secondary	—text-secondary	Held to 4.5:1
semantic.text-tertiary	text/tertiary	—text-tertiary	Metadata only, 3:1
semantic.text-on-action	text/on-action	—text-on-action	On the primary button
semantic.border-subtle	border/subtle	—border-subtle	
semantic.border-default	border/default	—border-default	
semantic.border-strong	border/strong	—border-strong	
semantic.action-primary	action/primary	—action-primary	One CTA colour: the ground's metal
semantic.action-primary-strong	action/primary-strong	—action-primary-strong	Hover/emphasis
semantic.action-secondary	action/secondary	—action-secondary	Wash
semantic.status-attention	status/attention	—status-attention	The one colour the app spends
semantic.status-attention-line	status/attention-line	—status-attention-line	
semantic.status-attention-wash	status/attention-wash	—status-attention-wash	
semantic.status-danger	status/danger	—status-danger	6:1
semantic.status-danger-line	status/danger-line	—status-danger-line	
semantic.status-danger-wash	status/danger-wash	—status-danger-wash	
semantic.status-success	status/success	—status-success	= accent
semantic.status-info	status/info	—status-info	= accent
semantic.status-neutral	status/neutral	—status-neutral	

Exported path	Figma variable (proposed)	CSS	Use
semantic.chart-verified	chart/verified	—chart-verified	
semantic.chart-estimated	chart/estimated	—chart-estimated	
semantic.chart-stale	chart/stale	—chart-stale	
semantic.chart-student-entered	chart/student-entered	—chart-student-entered	
semantic.focus-color	focus/color	—focus-color	4.5:1 on panel
semantic.focus-ring-width	focus/ring-width	—focus-ring-width	2px
semantic.target-min	target/min	—target-min	24
semantic.target-primary	target/primary	—target-primary	44
semantic.shape-control	shape/control	—shape-control	
semantic.shape-card	shape/card	—shape-card	
semantic.shape-surface	shape/surface	—shape-surface	
semantic.shape-sheet	shape/sheet	—shape-sheet	
semantic.elevation-raised	elevation/raised	—elevation-raised	
semantic.elevation-floating	elevation/floating	—elevation-floating	
semantic.elevation-modal	elevation/modal	—elevation-modal	
semantic.type-role-display	type/display	—type-role-display	
semantic.type-role-page	type/page	—type-role-page	
semantic.type-role-section	type/section	—type-role-section	
semantic.type-role-body	type/body	—type-role-body	
semantic.type-role-label	type/label	—type-role-label	
semantic.duration-fast	motion/fast	—duration-fast	130ms
semantic.duration-standard	motion/standard	—duration-standard	180ms
semantic.layer-overlay	layer/overlay	—layer-overlay	80

Primitive variables Figma needs directly

Exported path	Figma variable (proposed)	Varies with
primitive.app-bg	base/bg	ground
primitive.app-panel	base/panel	ground
primitive.app-fg	base/fg	ground
primitive.app-accent	base/accent	accent, ground
primitive.app-warn	base/warn	ground
primitive.r-sm	radius/sm	corners
primitive.r-md	radius/md	corners
primitive.r-lg	radius/lg	corners
primitive.sp-4	space/4	stylesheet × density

Exported path	Figma variable (proposed)	Varies with
primitive.sp-7	space/7	stylesheet × density
primitive.type-base	type/base	stylesheet × textSize
primitive.text-scale	type/scale	textSize

Components

Figma component	React implementation	Required variants / states
Button	`.btn` + `ActionButton` (`components/ui.tsx`)	primary · secondary · ghost · icon · disabled · busy
Status chip	`unity/Status.tsx` → `StatusChip`	tone neutral/info/success/attention/danger · glyph + word
Source badge	`components/SourceBadge.tsx`	8 trust kinds · with age · with Report
Provenance chips	`unity/ProvenanceChips.tsx`	fill solid/outline/dashed · ≤2 cues
Object card	`unity/ObjectCard.tsx`	with/without primary · Detailed mode source line
Context bar	`unity/ContextBar.tsx`	inside a panel (frameless)
Action preview	`unity/ActionPreview.tsx`	recovery undo/request/none
States	`unity/States.tsx`	loading · error · success · progress · steps · permission · offline
Visibility	`unity/Visibility.tsx`	4 audiences · locked
Table	`unity/Table.tsx`	empty · caption hidden
Combobox	`unity/Combobox.tsx`	open · selected row (edge + weight)

Design-system project additions (proposals, not in tokens.css)

The Semester design-system project (Claude Design) adds brief-driven tokens that **do not exist in code**. They are listed so a Figma file built from that project can be reconciled; they must not be imported as if exported.

Design-system token	Status	Closest code token	Resolution
—source-official / -line / -wash	Proposal	—status-success (= accent)	Decide in a DD whether provenance gets its own hue
—source-connected / -line / -wash	Proposal	—	Same
—source-personal	Proposal	—status-neutral	Same
—source-ai / -line / -wash	Proposal	—status-attention (AI is styled with estimates)	Same
—status-pending	Proposal	—status-attention	Alias if accepted
—blue-100…900 (Semester blue)	Proposal	Industry accent ramp in look.ts	Already an accent; no new ramp needed
—accent-editorial (Source Line)	Proposal	—app-accent-deep / Brass shade	Map to accent when Source Line ships

PART 2

Building

How Semester is built: architecture, engineering practice, testing, the build plan and what comes next.

SEMESTER

Architecture guide

For architects and senior engineers. v0.1, October 2026. A design-level description: verify everything against the repo (harrisonjrubin7-cmyk/semester) before relying on it.

1. Principles

1. **The institution is the system of record.** Semester reads, displays its provenance, and writes back only through approved paths.
2. **Student records never reach an AI model.** This is enforced in the gateway, not by convention.
3. **Company roles never see student data.** This is enforced by RLS and separate schemas.
4. **Every consequential action is previewed, attributed and logged.**
5. **Status over claims.** Freshness and source are shown on every record.

2. System context

```
Institution: SIS · LMS · IdP (SAML) · Calendar
        | read sync (pull, webhook)        ▲ approved write-backs
        ▼                                  |
Semester ingest workers → canonical store (Postgres) → read models (per role)
        |                                  |
        ▼                                  ▼
Web app (Next.js on Vercel)  ←→  API (route handlers / server actions)
        |                                  |
        ▼                                  ▼
AI gateway (redact → policy → model → cite) Audit log · Ops telemetry
```

3. Components

Component	Tech	Responsibility
Web app	Next.js App Router, TypeScript, design-system tokens	Student, staff, console, community, ops UIs
Auth	Supabase Auth + SAML SSO per institution	Identity; role claims mapped from the IdP
Database	Supabase Postgres + RLS	Schemas: `core`, `edu` (student data), `community`, `market`, `ops` (company, no PII), `audit`
Ingest	Scheduled workers + webhooks	SIS/LMS pulls, normalisation, reconciliation reports
Read models	Materialised views / tables	Registration readiness, caseload, today plan; each row stamped with source and synced_at

Component	Tech	Responsibility
AI gateway	Server-side module (`semester-core/aigateway.ts`)	Field allow-list, redaction, course AI policy, citations, cost metering
Payments (marketplace)	A licensed processor (e.g. Stripe Connect)	Semester never holds funds
Observability	Platform logs + error tracking + uptime checks	SLOs, alerts, the ops console

---

4. Data model (summary)

Tenancy: every row carries `institution_id`; RLS policies check it against the JWT claim. The `edu` schema holds student records (enrollments, holds, grades as shown, advising notes). `community` and `market` hold user-generated content, linked to a profile and never to edu tables. `ops` holds company data with no student identifiers. See `handoff/semester-platform/supabase/migrations/`.

---

5. Security boundaries

- RLS on every table; service-role keys only in workers, never in the browser.
- AI gateway allow-list: course content, the student's own typed text, public catalog data. Blocked: anything from `edu.*`.
- Marketplace and messages: rate limits, a report pipeline to the moderation queue, an integrity classifier on listings.

---

6. Key flows

SIS sync

1. Worker pulls a delta.
2. Normalise.
3. Upsert with `source` and `synced_at`.
4. Reconcile against the previous snapshot.
5. Mismatches go to the registrar queue.
6. Rebuild the affected read models.

Ask Semester

1. Request.
2. Policy check (course AI policy).
3. Build the context from allowed sources only.
4. Redact.
5. Model call.
6. Cite sources.
7. Label the answer AI-assisted.

8. A person confirms any action.

Marketplace sale

1. Listing.
2. Integrity and safety check.
3. Live.
4. Offer.
5. Processor checkout.
6. Pickup confirmation releases the payout.
7. Any dispute goes to T&S.

---

7. Environments

Env	Data	Access
Local	Seed and synthetic data	Developers
Preview	Synthetic data	Per pull request
Staging	Synthetic data, mirroring the institution's configuration	Team + institution IT
Production	Real data under the DPA	Just-in-time and logged

---

8. Decisions (ADRs)

- ADR-001: Postgres RLS as the primary authorisation layer.
- ADR-002: read models over live SIS queries, for speed and to show freshness.
- ADR-003: the AI gateway as the single egress point to models.
- ADR-004: marketplace payments through a processor; no money transmission.
- ADR-005: company ops data in a separate schema with no student identifiers.

Three-tier architecture and API contract

docs/platform/THREE_TIER_ARCHITECTURE.md

Three-tier architecture and strategic borrowing

Semester platform maturity program · 2026-10-05 · Source: uploads/Apple vs Google vs Microsoft in EdTech build a st.pdf._

Sell the unified Education OS, deploy a focused pilot, expand as a trusted institutional platform, and mature into an open ecosystem where students, institutions, developers, employers and partners all gain from the same connected graph.

Tier 1 · Experience platform

Apple-like: calm, clear, fast, accessible, coherent.

- Student OS
- Faculty Course Studio
- Advisor workspace
- Registrar tools
- Campus services
- Family portal
- Career/alumni
- Company site
- Operations console

Tier 2 · Education intelligence platform

Google-like: data, search, AI, source authority and recommendations working together.

- Identity graph
- Education graph
- Search
- AI gateway
- Policy engine
- Workflow engine
- Evidence/provenance
- Analytics
- Recommendations

- Outcome models
- AI evaluation
- Human escalation

Tier 3 · Institutional infrastructure platform

Microsoft-like: safe to buy, configure, deploy, govern, integrate and operate for years.

- Tenant control plane
- RLS and authorization
- SSO/SCIM
- Roles/capabilities
- Approval workflows
- Audit and evidence
- Integration gateway
- LTI/OneRoster/Edu-API
- Migration and reconciliation
- Feature flags and rollout
- Billing
- Trust/compliance
- SLOs/incidents
- Backup/restore/offboarding
- Developer ecosystem

What to borrow, and what not to copy

Model	Borrow	Semester equivalent	Do not copy
Apple	Integrated experience, design discipline, privacy posture, premium support	Calm student OS, cohesive navigation, accessibility, offline-safe, student-controlled sharing, quality mobile	Closed ecosystem, forced lock-in, opaque platform rules
Google	Search, cloud, identity, AI, data systems, developer distribution	Education search, education graph, membership layer, AI gateway, APIs, partners	Ad-tech incentives on student data, uncontrolled aggregation
Microsoft	Enterprise identity, productivity, administration, procurement readiness, long contracts	Tenant admin, SSO/SCIM, workspace, compliance controls, implementation, annual licensing, marketplace	Complex enterprise UX, configuration overload, fragmented surfaces
Canvas/Brightspace	Course workflow and institutional deployment	Course Studio connected to planning, actions, support, AI, career	Course experience isolated from the wider journey
SIS/ERP vendors	Record authority, policy, transaction integrity	Canonical academic domains, deterministic rules, audit, migration, reconciliation, rollback	Legacy UX, closed integrations
ChatGPT/Claude/Perplexity	Conversational intelligence and research	Governed copilot, policy-aware routing, citations, academic context, human escalation	Treating model output as official truth

Deployment ladder

Development → Tenant-safe test environment → Sandbox integration → Preview deployment → Internal beta → Private beta tenant → Pilot tenant → Controlled production cohort → Annual customer deployment → Multi-campus expansion

Each step requires: Configuration, Policy approval, Security review, Accessibility review, Data validation, Integration test, Rollback plan, Support readiness, Monitoring, Release evidence, Owner approval.

Expansion stages

1. Daily student OS + registration readiness
2. Course Studio + advising/support workflows
3. Institution control plane + integrations + rollout factory
4. Native registrar/academic domains where authorized
5. Campus services + student account + career + family
6. Developer platform + marketplace + global ecosystem
7. Education infrastructure company: identity, cloud, AI, operating layer, standards, partners, global, lifelong learners

---

`docs/platform/API_CONTRACT_STANDARD.md`

API domains and contract standard

Semester platform maturity program · 2026-10-05 · Source: uploads/Apple vs Google vs Microsoft in EdTech build a st.pdf.

APIs are organized by authority and risk, not by frontend page. High-impact writes never mutate raw tables from the browser; they go through capability-checked server commands.

Domains

API	Primary resources	Write-control model
Identity	Accounts, sessions, memberships, SSO, SCIM, roles	Server-authorized; auditable
Tenant	Institutions, campuses, policies, configurations, feature state	Tenant admin + approval path
Academic	Terms, catalog, sections, enrollment, holds, records, grades	Deterministic policy + registrar authority
Learning	Courses, objectives, resources, assignments, assessments, study packs	Faculty/course-scoped capabilities
Workspace	Tasks, notes, files, calendar, plans, projects	Student ownership + sharing grants
AI	Requests, policy, providers, sources, usage, evaluations	Gateway-enforced, tenant/course policy
Student success	Caseload, check-ins, referrals, success plans, support	Purpose-limited, consent/role-scoped
Campus	Dining, housing, events, organizations, spaces	Service-owner policy + tenant scope
Career	Skills, evidence, credentials, opportunities, mentors	Student consent + verified claim rules
Family	Grants, invitations, scoped shares, revocation	Student-controlled, time-limited
Integrations	Connections, scopes, mappings, syncs, webhooks	Approval-gated, server-only credentials

API	Primary resources	Write-control model
Commercial	Quotes, contracts, subscriptions, invoices, entitlements	Finance capability + approvals
Trust	Controls, evidence, claims, artifacts, trust-room grants	Security/legal scoped
Operations	Approvals, incidents, SLOs, audits, rollouts, risks	Internal capabilities, two-person controls
Platform	Apps, API keys, OAuth clients, webhooks, marketplace	Tenant install approval + developer governance

Every API answers

Check	Question
Identity	Who is calling?
Scope	Which tenant, customer, course or person may be accessed?
Purpose	Why is this data or action needed?
Authority	Official, derived, AI-generated, user-owned or external?
Policy	Which rules constrain it?
Validation	Are inputs structurally and semantically valid?
Idempotency	Can a retry create duplicate harm?
Audit	What event must be recorded?
Freshness	Live, delayed, stale or unavailable?
Error	What safe recovery or human hand-off is available?

Example resources

```
 GET /v1/identity/me
GET /v1/tenants/{tenantId}
GET /v1/tenants/{tenantId}/policy
GET /v1/academic/terms
GET /v1/academic/sections
POST /v1/academic/registration-requests
POST /v1/academic/registration-requests/{id}/decision
GET /v1/learning/courses/{courseId}
GET /v1/learning/gradebook
GET /v1/workspace/tasks
POST /v1/ai/requests
GET /v1/ai/sources
POST /v1/support/requests
GET /v1/advising/caseload
GET /v1/career/opportunities
POST /v1/family/grants
GET /v1/integrations/sync-runs
POST /v1/integrations/{id}/scope-approval
POST /v1/commercial/quotes/{id}/send
POST /v1/ops/approval-requests/{id}/decision
POST /v1/privacy/data-subject-requests/{id}/complete
GET /v1/platform/apps
```

Architecture cards

Front end

Student app
Five destinations, 64 screens

Staff + console
Same shell, role-scoped routes

Design system
Tokens → semantic CSS → components

Offline
Queue mutations, replay on reconnect

Middleware

Auth proxy
Refresh session; reject expired

Tenant resolver
Host/SSO → tenant_id on every request

Capability check
role × screen × grant first

Audit writer
Event written before the action runs

Server / API

Query RPCs
api.* reads of read models

Commands
private.* fns; idempotency key

Integrations
LTI 1.3, OneRoster, SIS read, SAML

AI gateway
Scope, policy, citations, spend log

Database

Supabase Postgres
RLS on every table

tenant_id
On every row

private.*
Authoritative, never exposed

read_model.*
Projections + version + freshness

arch layers

WRITE → READ

1 write		2 outbox		3 project		4 stamp		5 invalidate		6 signal		7 render
Authoritative write	→	Event, same transaction	→	Worker applies, idempotent	→	Version + freshness	→	Cache namespaces	→	Targeted refetch	→	Server read model

FRESHNESS → UI POLICY (FRESHNESS.TS)

State	Lag	Display	Consequential actions
Live	≤ 5 s	Normal	Allowed if approvals satisfied
Recent	≤ 60 s	Normal	Normal guardrails
Delayed	> 60 s	Delay badge + age	Re-check source first
Failed / Rebuilding	—	Last good + notice	Held

arch read model

Request		Identity + scope		Policy		Routing		Answer		Evidence
Student, staff or institution asks	→	Who; which data they may read	→	Course AI policy, institution switches, FERPA boundary	→	Approved model per task; search, calendar, materials	→	Labelled AI-assisted, sources, confidence	→	Citations, cost, outcome logged

arch ai gateway

Environment	Hosting	Data	Access
Local	Supabase CLI + Next.js dev	Synthetic seed	Engineers
Staging	Vercel preview + Supabase staging	Synthetic + institution test IdP accounts	Engineers, institution IT testers
Production	Vercel production + Supabase production (separate keys)	Real student records under the DPA	Named operators, MFA, access logged

Rule: no copying data between environments; secrets set per environment. Cutover steps: docs/launch/Cutover Plan.html.

arch environments

Master control

ui_kits/master-control/ — product, console & back end, business OS, company site, design system & engineering

Rule
Every new file is linked here

Master catalog

51 roles · 673 screens · 319 workflow steps (17 workflows) · 88 systems · 82 docs

Rule
Status from evidence: code, tests, release

Role lens

64 screens × 14 roles: rendering, sources, tables, RLS, API

Access levels
full · scoped · shared · read · audited · none

Rule: company roles (investor, CS, RevOps) never see student records. Role model in roles.js is design-level; verify against migrations.

maps of the whole

Architecture explorer

Interactive architecture of stack layers, read models, AI gateway and environments. **11 screens.**



Opening view

SEMESTER

Education OS map | Full stack & data | Read-model pipeline | AI gateway | Lag simulator | Migration phases

Architecture
Design-level · verify in repo
Full stack & dataSemester platform

Search or ask… ⌘K

○On this device only
Architecture · Full stack & data

Full stack & data

Front end | Middleware | Server / API | Database

RuleRLS on every table; tenant_id on every row; browser calls api.* only. Design-level schema — reconcile with supabase/migrations.

Table	Key columns	RLS: who can read	Domain
tenants	id, slug, name, plan, sso_entity_id, created_at	admin of tenant	Tenant
memberships	user_id, tenant_id, role, status, granted_by, expires_at	self + tenant admin	Identity
role_grants	id, user_id, tenant_id, capability, scope_kind, scope_id, expires_at	tenant admin; audited	Authorization
people	id, tenant_id, display_name, pronouns, source	self; scoped staff	Profile
courses	id, tenant_id, term_id, code, title, source, synced_at	enrolled; instructors	Learning
sections	id, course_id, instructor_id, capacity, waitlist	enrolled; registrar	Registration
enrollments	person_id, section_id, status, source, version	owner; registrar	Registration
assignments	id, course_id, title, due_at, weight, source	enrolled; instructors	Learning
grades	person_id, assignment_id, score, released_at, version	owner after release; instructors	Gradebook
holds	person_id, kind, blocks, source, synced_at	owner; registrar; finance	Records
items	id, owner_id, kind, title, due_at, origin, done, snoozed_until	owner	Today
shares	id, owner_id, grantee_id, screen, expires_at, revoked_at	owner; grantee read	Consent
share_events	share_id, actor_id, action, at	owner read	Consent
support_requests	id, requester_id, office, severity, status	requester; office	Support
approvals	id, duty, requester_id, approvers[], state, expires_at	approvers; audited	Governance
audit_log	seq, tenant_id, actor_id, event, target, authority, at	append-only; auditors	Audit
domain_outbox_events	id, tenant_id, type, payload, created_at, claimed_at, processed_at, attempts	server only	Read models
read_model.tenant_overview_v1	tenant_id, version, applied_at, metrics jsonb	api RPC only	Read models
ai_requests	id, user_id, task, scope_ids[], dropped_ids[], cost, model, at	owner; security	AI
integrations	id, tenant_id, provider, scopes, status, last_sync_at	tenant IT	Integration

Request flows, end to end

Flows

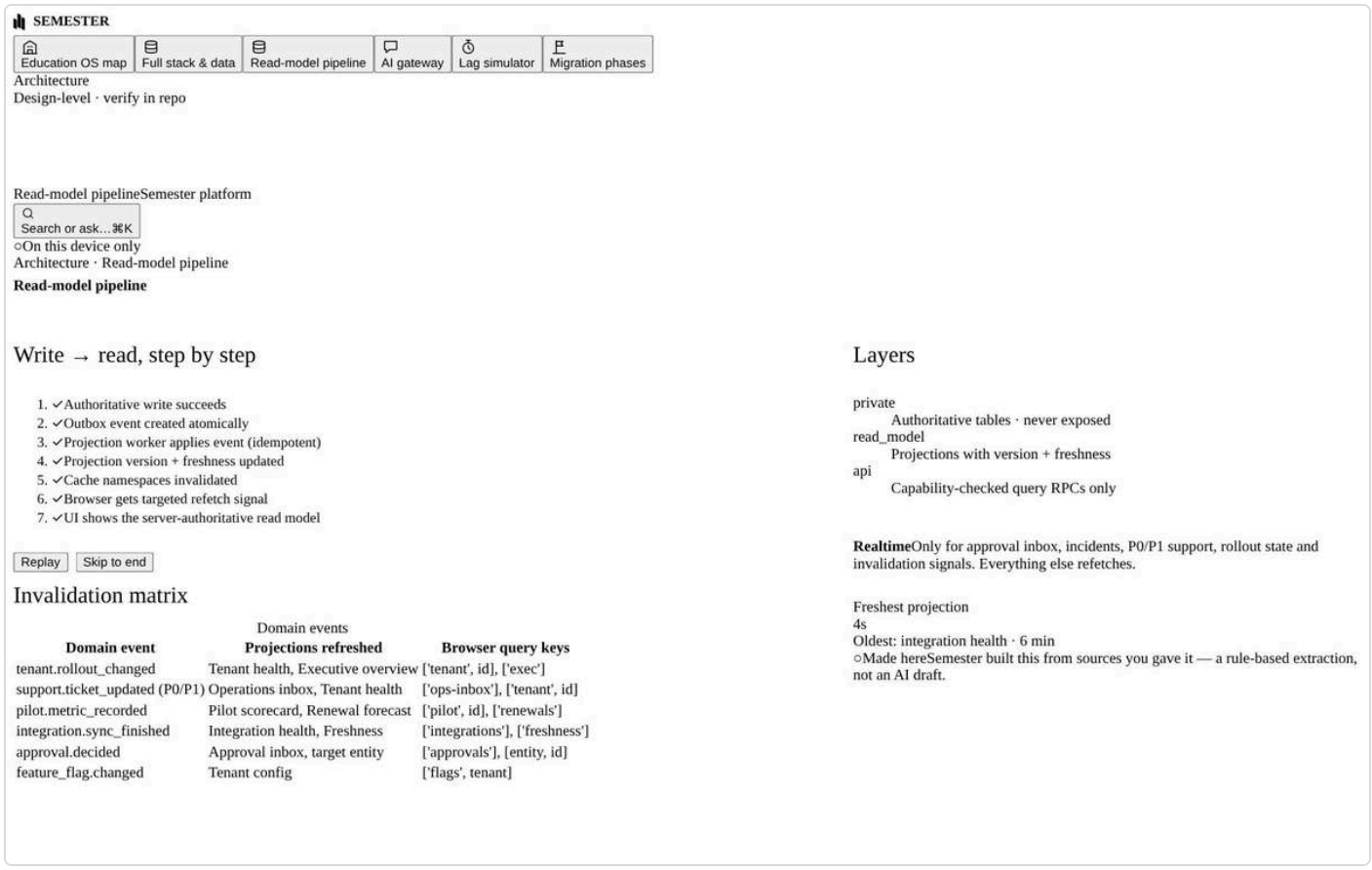
Flow	Path
Read	Browser → auth proxy → tenant + capability → api.rpc → read_model → response with version + freshness
Write	Browser (optimistic) → command fn → validate + RLS → domain table + outbox (one transaction) → audit → worker → projection → invalidate keys → UI confirms
Sync	Scheduler → edge worker → provider API → staging → map + validate → reconcile → upsert with source + synced_at → outbox
AI	Browser → gateway → scope + policy + classification → drop student records → model → citations → log spend → AIResponse
Approval	Request → approver 1 (MFA ≤15 min) → approver 2 → requester executes → audit before change

Operations

Operations

Practice	Execution	Owner
Deploy	PR → CI gate → preview → staged tenant rollout → full	Engineering
Migrations	Reviewed SQL, forward-only, backfill job, parity check	Engineering
Monitoring	Errors, uptime, projection lag, sync failures, AI spend	Engineering
On-call	P0 1 h · P1 4 h; runbook per alert	Founder → contractor
Backups	Daily + PITR; quarterly restore test with RTO/RPO	Engineering
Access review	Quarterly; contractor offboarding same day	Security
Incident	Commander, tech lead, comms; postmortem in 5 days	Founder

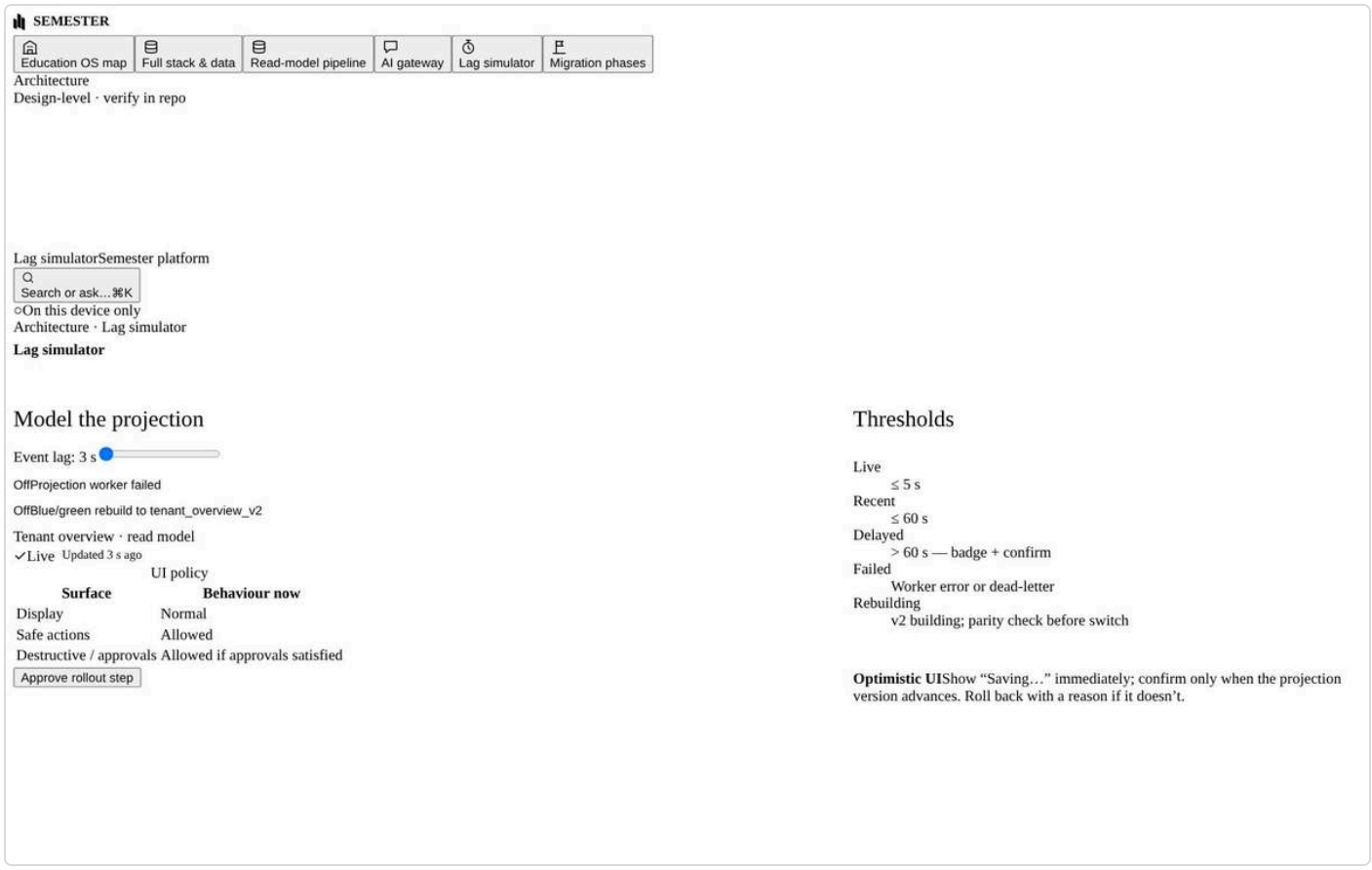
Full stack & data



Read-model pipeline


SEMESTER
Education OS map
Full stack & data
Read-model pipeline
AI gateway
Lag simulator
Migration phases
Architecture
Design-level · verify in repo
AI gatewaySemester platform
Search or ask…⌘K
○On this device only
Architecture · AI gateway
AI gateway
1
Request
Student / staff / institution asks
○Student enteredYou typed this in. Semester has not checked it with your institution.
2
Identity + scope
Who is asking; which data they may read
◆Institution verifiedConfirmed by your institution’s own system.
3
Policy check
Course AI policy, institution switches, FERPA boundary — student records never leave
◆Institution verifiedConfirmed by your institution’s own system.
4
Model + tool routing
Approved model per task; tools: search, calendar, course materials
○Made hereSemester built this from sources you gave it — a rule-based extraction, not an AI draft.
5
Answer
Labelled AI-assisted, numbered sources, confidence
✦AI-assistedWritten with Semester’s assistant. It is not an official answer — check anything you act on.
6
Evidence + logs
Citations, cost, outcome logged for audit
◆Institution verifiedConfirmed by your institution’s own system.
Model-flexible, Semester-governedAny approved model can answer; Semester decides what it may read, labels what it wrote, and keeps the evidence.
✦AI-assistedWritten with Semester’s assistant. It is not an official answer — check anything you act on.Semester assistant§Outlining allowed
Example answer leaving the gateway.
1. 1BIO 210 Syllabus◆Institution verifiedConfirmed by your institution’s own system.
Allowed to read BIO 210 materialsConfidence HighRequest cost 1 request

AI gateway



Lag simulator



Migration phases



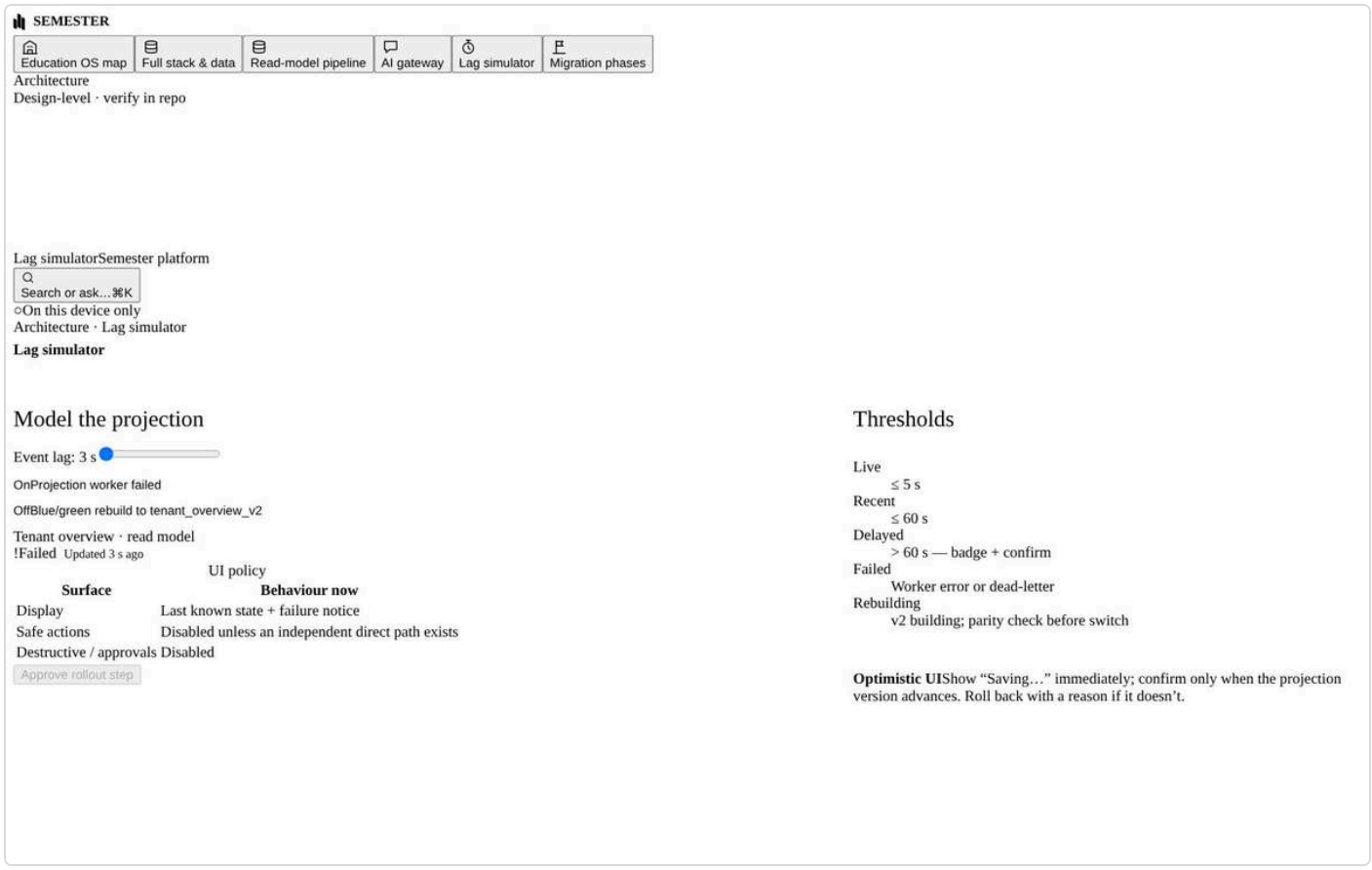
Full stack & data › Front end



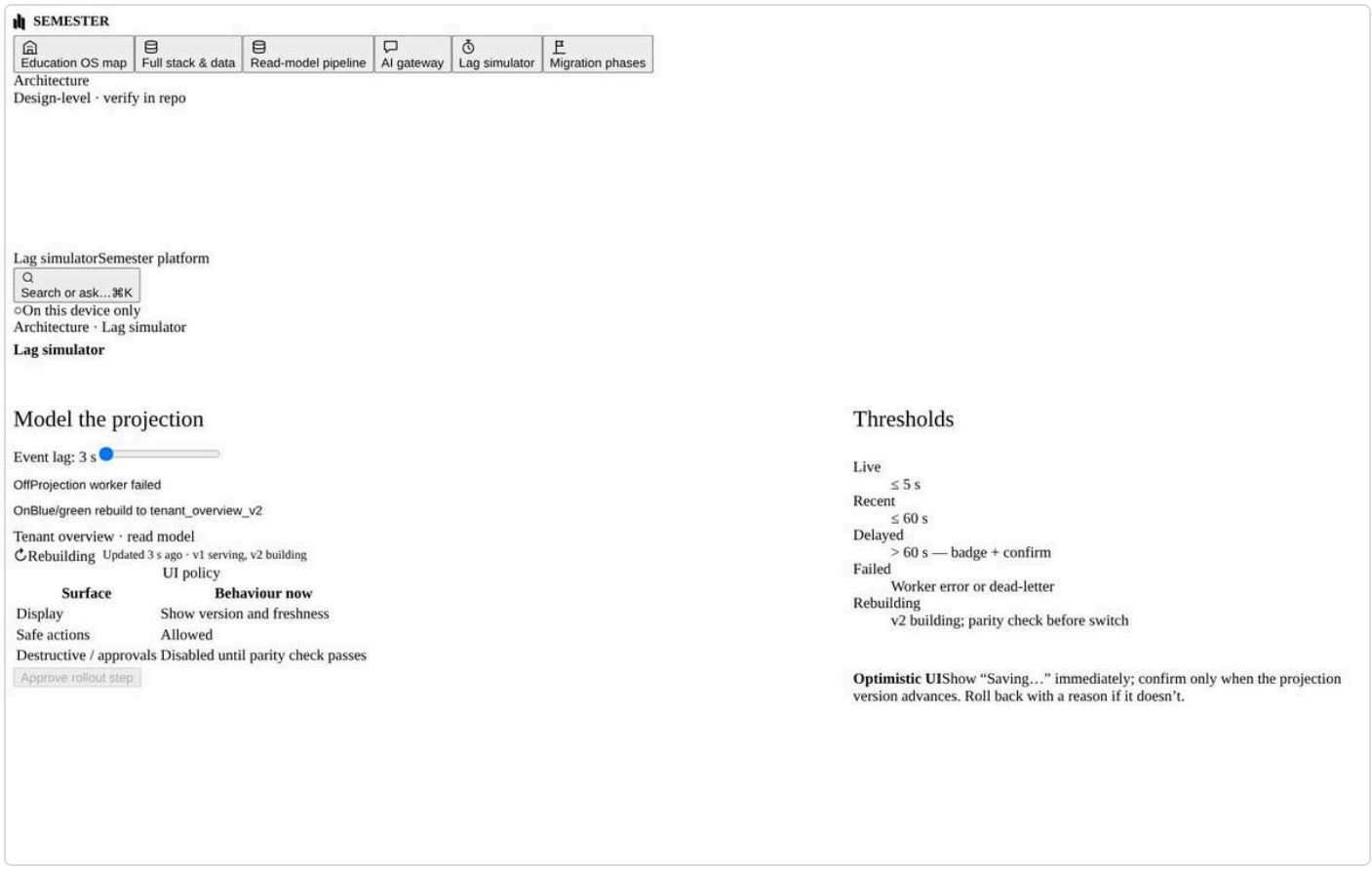
Full stack & data › Middleware



Full stack & data › Server / API



Lag simulator › OffProjection worker failed



Lag simulator › OffBlue/green rebuild to tenant_overview_v2

SEMESTER

Developer guide

Set up, build, test and ship. v0.1, October 2026.

1. Repos and code in this project

- `harrisonjrubin7-cmyk/semester`: the product app (ground truth).
- `handoff/semester-platform/`: database migrations with RLS, typed API routes, and tests for community, marketplace and ops (new).
- `handoff/semester-core/`: pure logic (access, AI gateway, approvals, freshness, provenance), with tests.
- `handoff/claude-code-toolkit/`: design-system sync skills and the audit script.
- `handoff/workflow-router-app/`: a full Next.js + Supabase reference app.

Code files in this project end in `.txt` so they don't execute here. Strip the suffix when you copy them into a repo.

2. Local setup

```
git clone git@github.com:harrisonjrubin7-cmyk/semester.git && cd semester
npm install
npx supabase start            # local Postgres + auth
npx supabase db reset         # runs migrations + seed
cp .env.example .env.local    # fill SUPABASE_URL / ANON_KEY
npm run dev
```

3. Conventions

- TypeScript strict. Validate every input with Zod at the boundary.
- UI uses design-system tokens only (`styles.css`). No raw hex values in components.
- Every query runs as the user (anon key + JWT). The service role is only for workers.
- Every mutation writes an audit row: actor, role, institution, action, target, before/after.
- Copy follows the readme's voice rules: sentence case, the status sentence, no overclaims.

4. Branching and release

1. Branch from `main`, named `feat/…` or `fix/….`
2. Open a PR. Vercel builds a preview, and CI runs lint, typecheck, unit tests, RLS tests and the design-system audit.
3. One review is required, and the founder approves anything touching migrations.
4. Merge to `main`, which deploys to staging. Promote to production after the smoke check.

5. Testing

Layer	Tool	Must cover
Unit	Vitest	semester-core logic, Zod schemas
RLS	SQL tests (pgTAP or Vitest against local Supabase)	Each role can read only what it should; company roles get zero rows from edu
AI gateway	Vitest red-team suite	edu fields never leave; prompt-injection cases
E2E	Playwright	Sign-in per role, key workflows, marketplace offer flow
Accessibility	axe + manual screen reader pass	WCAG 2.2 AA on each release

6. Adding a feature (checklist)

1. Migration with RLS policies and RLS tests.
2. Zod schema and a typed route handler or server action.
3. UI built from design-system components, with every state: empty, loading, error, offline.
4. Audit write and telemetry event.
5. Feature flag per institution.
6. Docs: update the user guide section and the master catalog status.

SEMESTER

Engineering playbook

Owner: founder / founding engineer. Status: draft, October 2026. Applies to harrisonjrubin7-cmyk/semester. Prompts that execute it: `handoff/execute/`.

Three rules that never bend: RLS on every table with a test that proves it; the browser calls `api.*` only; student records never reach an AI provider.

1. Repository layout

Path	Holds
app/	Student and staff app (Vite + React + TypeScript): src/screens, src/components, src/lib, src/styles
app/src/lib/core/	semester-core: pure decision logic with unit tests
supabase/migrations/	Numbered SQL migrations; never edit one that has been applied
supabase/tests/	RLS and policy tests (pgTAP or SQL test harness)
workflow-lab/	Next.js AI workflow router and benchmark lab
docs/	Architecture, design system, master catalogs, execute reports
design/	Exported design-system project (reference only)
.github/workflows/	CI

2. Branching and pull requests

- **main** is always deployable. No direct pushes.
- Branch per change: `semester/<stream>` for execute streams, `fix/…`, `feat/…` otherwise.
- Small PRs (under ~400 changed lines where possible). Each PR states: what, why, risk, how tested, screenshots for UI, migration notes.
- One approving review. Founder approves anything touching auth, RLS, AI gateway or billing.
- Squash-merge; the PR title becomes the changelog line.

3. CI on every PR

Job	Command	Blocks merge
Typecheck	npx tsc -b	Yes
Lint	npm run lint	Yes
Unit tests	npm test (vitest)	Yes

Job	Command	Blocks merge
Design system	npm run design-system:check	Yes (baseline ratchet)
RLS tests	supabase db reset && supabase test db	Yes
Accessibility	axe on every route in the preview build	Yes for serious/critical
Build	npm run build	Yes
Preview deploy	Vercel preview URL on the PR	—

4. Environments

Environment	App	Database	Data	Deploys
Local	Vite dev	supabase start (CLI)	Synthetic seed	—
Preview	Vercel preview per PR	Supabase branch or shared staging	Synthetic	Every push
Staging	Vercel staging	Supabase staging project	Synthetic + institution test IdP	Merge to main
Production	Vercel production	Supabase production (separate keys)	Real, under DPA	Tagged release, manual promote

Secrets live in Vercel and Supabase environment settings, never in the repo. Production keys are readable only by named operators with MFA.

5. Database changes

1. Write the migration and its RLS policies in the same PR.
2. Add tests: own tenant allowed, other tenant denied, company roles denied on student tables, expired grant denied.
3. Prove each test fails without its policy (red), then passes (green); note it in the PR.
4. Backwards-compatible first (add columns, dual-write), destructive later in a separate release.
5. Run against staging; check the read-model projection still builds.

6. Release process

Step	Who	Evidence
Cut release from main; tag vYYYY.MM.DD-n	Engineer	Tag
Release notes from PR titles	Engineer	CHANGELOG entry
Staging smoke test: sign-in per role, Today, registration readiness, Ask Semester block test	Engineer	Checklist in the release issue
Migration applied to production (if any) with backup taken	Founder	Backup ID

Step	Who	Evidence
Promote to production	Founder	Vercel deployment link
Watch errors and SSO failures for 30 minutes	Engineer	Note in release issue
Roll back if a rollback trigger fires	Founder	Incident log

No releases during a pilot's launch week, finals week, or registration opening unless fixing an S1/S2.

---

7. Feature flags

Every institution-visible feature ships behind a per-tenant flag with a kill switch. Flags default off in production. Remove a flag within two releases of 100% rollout.

---

8. Coding standards

- TypeScript strict; no `any` without a comment saying why.
- Tokens and components from the design system; no raw hex, z-index or durations.
- Every screen handles loading, empty, error, forbidden and offline.
- Consequential actions use preview → audit → apply → undo/request.
- Server-side validation with Zod on every command.
- No student data in logs, error trackers or analytics events; use internal IDs.

---

9. Dependencies and patching

Cadence	Do
Weekly	Review Dependabot alerts; patch critical/high within 7 days
Monthly	Minor version bumps in one PR, full CI
Quarterly	Major upgrades planned; review subprocessors and SDKs

---

10. Definition of done

A change is done when CI is green, tests cover the new behavior, RLS is proven for new tables, the screen passes axe and a keyboard pass, docs and the catalog row are updated, monitoring exists for new failure modes, and it's released with notes.

Product and engineering operations

docs/business/PRODUCT_AND_ENGINEERING_OPERATIONS.md

Product and engineering operations

Semester Company OS · 2026-10-05 · Source: uploads/what else on the bussnius and opprations site also.pdf.

> Semester must operate with the same coherence, evidence, reliability and trust it promises to institutions.

Field	Value
Mission	Turn evidence into shipped, reliable, accessible product.
Owner	Founder → founding engineer
Inputs	Customer signals, research, catalog gaps, incidents
System of record	GitHub (code, PRs), /ops product operations (backlog, flags, releases)
KPIs	Release cadence; Change failure rate; Error budget remaining; Accessibility issues open
Reports	Release notes; monthly product review
Meeting cadence	Weekly planning; release per engineering playbook
Risk controls	CI gates, RLS proof, two-person review for auth/RLS/AI/billing
Approval thresholds	No release in pilot launch week, finals or registration opening except S1/S2 fixes
Required documents	Engineering playbook, QA test plan, Accessibility test plan, ADR log, PRD template
Required integrations	GitHub, Vercel, Supabase (deploy status in /ops)
Current status	docs/build/, handoff/execute/, docs/master/
Gaps	PRD template, ADR log, experiment registry
Priority	P1
Next action	Run execute stream 00 and approve the design-system baseline

Workflows

- Signal → research → problem → opportunity → strategy fit → priority score → PRD → design → threat/privacy/accessibility review → technical design → build → test → controlled release → measure → iterate or retire

SEMESTER

QA test plan by role

Owner: engineering. Status: draft, October 2026. Covers the pilot scope first; every row maps to a workflow in `docs/master/SEMESTER_WORKFLOW_CATALOG.md`.

1. Test levels

Level	Tool	Runs	Covers
Unit	vitest	Every PR	semester-core logic, validators, formatters
Database / RLS	SQL test harness	Every PR	Policies per table and role
Component	vitest + Testing Library	Every PR	States, accessible names, keyboard
End to end	Playwright	Every PR (smoke), nightly (full)	Role journeys below
Accessibility	axe + manual	Every PR (axe), each release (manual)	See accessibility test plan
Exploratory	Human	Before each pilot launch	Pilot paths with real institution config in staging

2. Test accounts

One synthetic account per role in staging, mapped through the test IdP: student, applicant, guardian, faculty, TA, advisor, registrar, finance, student affairs, IT/security, institution admin, Semester support operator, Semester executive. No real student data in any non-production environment.

3. Role journeys (end to end)

Role	Journey	Must pass
Student	Sign in with SSO → onboarding → Today → registration readiness → resolve hold hand-off → enroll request	Each item shows its source; hold hand-off goes to registrar; nothing writes to SIS
Student	Share with advisor → revoke	Advisor sees only shared items; access ends immediately on revoke
Student	Ask Semester with a question containing a student ID	Blocked; event logged; friendly message
Student	Export personal data → request deletion	Export file complete; deletion request routed to the institution

Role	Journey	Must pass
Guardian	Accept invitation → view shared items → expiry	Only chosen categories visible; access ends at expiry
Faculty	Publish course AI policy → create assignment → release grade	Policy appears on student AI answers; release is two-step with preview
TA	Grade within assigned sections only	Other sections forbidden
Advisor	Caseload → check-in → referral	Only consented shares visible; referral status updates
Registrar	Grant override (two-person) → reconcile with SIS	Second approver required; audit event before apply
Finance	View balance → payment plan request	No payment handled by Semester; processor hand-off only
IT / security	Configure SSO → role mapping → audit explorer	Changes audited; break-glass needs reason and review
Institution admin	Feature cohort on → kill switch off	Feature disappears for the cohort within a minute
Semester support	Open ticket → view tenant health	No student table readable from /ops
Semester executive	Executive overview	Business data only; no student records

4. Negative and security tests

- Cross-tenant URL tampering for every role → forbidden and logged.
- Expired session and revoked membership → signed out.
- Rate limits on AI and auth endpoints.
- Input validation: oversized fields, script injection in every text input.
- Offline: queue a task offline, reconnect, verify single write (idempotency key).

5. Release entry and exit

Gate	Criterion
Entry	All PRs merged; staging deployed; seed data reset
Exit	Smoke suite green; no open S1/S2; full e2e green within the last 24 h; accessibility checklist signed

6. Defect severity

Severity	Example	Fix by
S1	Data exposure across tenants; AI receives a record	Stop release; fix now
S2	SSO broken; wrong SIS data shown	Before release
S3	One feature broken with workaround	Next release
S4	Cosmetic	Backlog

SEMESTER

Accessibility test plan · VPAT / ACR preparation

Owner: engineering + accessibility tester. Status: draft, October 2026. Target: WCAG 2.2 Level AA. Output: an Accessibility Conformance Report (ACR) using the VPAT® 2.x WCAG edition template.

Status, not claims: until the ACR is complete, say "working toward WCAG 2.2 AA; ACR in preparation." Never say "fully compliant."

1. Scope

Surface	Priority
Pilot student paths: sign-in, onboarding, Today, registration readiness, Ask Semester, sharing, settings	P0
Staff pilot paths: advisor caseload, registrar desk, institution console	P0
Public site	P1
Remaining student and staff screens	P2

2. Automated checks

- axe-core in CI on every route of the preview build; serious and critical issues block merge.
- Lint rules for accessible names on icon-only controls and labels on inputs.
- Color contrast checked from tokens: body text 4.5:1, large text and UI parts 3:1.

3. Manual checks (each release on P0 paths)

Check	How	WCAG
Keyboard only	Complete every P0 journey without a mouse; visible focus throughout; no traps	2.1.1, 2.1.2, 2.4.7, 2.4.11
Screen readers	NVDA + Firefox, JAWS + Chrome, VoiceOver + Safari (macOS and iOS)	1.3.1, 4.1.2, 4.1.3
Zoom and reflow	200% zoom and 320 px width without loss of content	1.4.4, 1.4.10
Text spacing	Apply the text-spacing bookmarklet	1.4.12
Target size	Interactive targets at least 24×24 px (product standard 44 px)	2.5.8
Motion	Reduced motion respected; nothing flashes	2.3.1, 2.3.3

Check	How	WCAG
Forms	Labels, instructions, error identification and suggestions, focus moves to first error	3.3.1–3.3.3, 3.3.7
Time limits	Session timeout warns and allows extending	2.2.1
Authentication	No cognitive test to sign in (SSO)	3.3.8

4. Assistive-technology matrix

AT	Browser	Platform
NVDA	Firefox, Chrome	Windows
JAWS	Chrome	Windows
VoiceOver	Safari	macOS, iOS
TalkBack	Chrome	Android
Dragon / voice control	Chrome	Windows

5. Recording results

Each finding records: screen, criterion, severity, steps, AT and browser, screenshot, fix PR. Findings roll into the ACR as Supports / Partially supports / Does not support / Not applicable, with remarks.

6. ACR production

1. Complete manual checks on P0 scope.
2. Fix all "does not support" items on P0 paths, or document a workaround and date.
3. Fill the VPAT template per criterion with remarks.
4. Founder reviews; publish to institutions on request, with the test date and scope.
5. Re-test on every major release; update the ACR at least yearly.

7. Barrier reports

Reports go to access@[domain] and the in-app "Report a barrier" link. Reply within two business days; S2 accessibility barriers on pilot paths get fixed before the next release.

BUILD HANDOFF

handoff/README.md

Semester — Claude Code handoff package

> **Start at `BUILD.md`.** It is the single entry point: preflight → install → streams in order → verify → human-only steps → go.

Everything a developer needs to turn the design system into the product. Open Claude Code at the repo root (harrisonjrubin7-cmyk/semester) and work through these in order.

#	Folder	What it is	Start with
1	`claude-code-toolkit/`	Skills (build-semester-ui, audit-semester-design-sync, create-semester-component), .mcp.json for Figma, design-system audit + report scripts, tests, CI steps, baseline ratchet	`EXECUTE.md`
2	`semester-core/`	Workflow algorithms as pure TypeScript with tests: provenance, next action, access, freshness, consequence, AI gateway, approvals, scoring, pilot readiness	`README.md`
3	`workflow-router-app/`	Next.js + Supabase AI Workflow Router & Benchmark Lab (already landed as workflow-lab/ upstream)	`README.md`

Code files carry a `.txt` suffix so the design project doesn't bundle them: `find . -name "*.txt" -exec sh -c 'mv "$1" "${1%.txt}"' _ {} \;`

| 0 | `execute/` | **Start here.** 12 stream prompts (setup to launch to grow) plus a master index that runs them in order on branches semester/ | `execute/00-INDEX.md` |

Reference (stay in the design system)

- `readme.md`, `SKILL.md` — design rules, tokens, components, patterns
- `ui_kits/master-catalog/` — 51 roles · 673 screens · 319 workflow steps (17 workflows) · 88 systems · 82 docs, with status and a Claude Code prompt that writes `docs/master/`
- `ui_kits/architecture/` — full stack & data: 20 tables, request flows, operations
- `ui_kits/role-lens/`, `ui_kits/atlas/`, `ui_kits/wireframes/` — every role × screen
- `ui_kits/business/Operating Manual.html` — how to run the company

Order of work (PO first)

1. Install the toolkit and pass `npm run design-system:check`.
2. Drop `semester-core` into `app/src/lib/core/`; reconcile names with `lib/source.ts`, `lib/status.ts`; run its tests.
3. Ship the pilot blockers: SAML SSO, AI-gateway test (no student records), DPA mapping, audit-before-action.
4. Run the master-catalog prompt to produce `docs/master/` from real evidence.

Never commit, push or deploy from these prompts without review.

---

handoff/BUILD.md

BUILD.md — the single entry point for Claude Code or Codex

Read this whole file before doing anything. It is written so an agent can build Semester without guessing. When anything here conflicts with another file, **this file wins**, then `execute/00-INDEX.md`, then stream files, then design docs.

---

0. Ground rules (non-negotiable)

1. Work in `harrisonjrubin7-cmyk/semester`. One branch per stream: `semester/<stream>`. One PR per stream. Never push to `main`, deploy, change production settings or secrets.
2. **Repo beats design.** If the repo already has something, extend it. Record exists / extended / new for every item in the PR.
3. Never mark anything done without the verify output pasted into `docs/execute/<nn>-<slug>-report.md`.
4. Stop and report (do not improvise) when: a gate fails twice; a migration would edit an applied migration; a test needs weakening to pass; anything touches real student data; a human-only step is reached (§7).
5. Product rules enforced in code: RLS + tenant_id everywhere; browser calls server/RPC only; company roles never read student tables; AI never receives student records (scope gate + content scan); every consequential action = preview → audit → apply → undo/request; official systems stay authoritative.

---

1. Preflight

```
 git checkout main && git pull
mkdir -p design && cp -R <design-project-export>/* design/      # handoff/, templates/, docs/, ui_kits/,
guidelines/, readme.md, SKILL.md
bash design/handoff/preflight.sh                                 # must print PREFLIGHT OK
```

---

2. App target

- The API routes, middleware and tests in `semester-platform` are **Next.js App Router + Supabase**. If `app/` is already Next.js, install there (`APP=app`).
- If `app/` is Vite + React: keep it for the student UI and create `web/` with `npx create-next-app@latest web --ts --app --eslint --no-tailwind --src-dir`, then `APP=web`. The Vite app calls `web`'s API. Do not rewrite the Vite app in this stream.

---

3. Install

```
 APP=<app|web> bash design/handoff/install.sh                    # never overwrites; FORCE=1 only with a
reason in the PR
```

```
(cd $APP && npm i @supabase/supabase-js @supabase/ssr zod && npm i -D vitest dotenv dotenv-cli)
cp $APP/.env.example $APP/.env.local                                  # fill from: npx supabase status
```

4. Order of work

Stream	File	Gate to pass before the next
00	execute/00-setup.md	CI green; REPO_AUDIT.md; design-system baseline approved by founder
01	execute/01-platform-core.md	`verify.sh` steps 1–5 green; every policy red-then-green
02	execute/02-identity-sso.md	SSO test login per pilot role (staging IdP)
03	execute/03-student-os.md	Student workflow e2e; 5 states per screen
04	execute/04-staff-academic.md	Two-person approvals tested
05	execute/05-institution-integrations.md	Reconciliation 0 unexplained mismatches
06	execute/06-ai-gateway.md	ai-redteam suite green; no prompt content in logs
07	execute/07-community-career-family.md	Feature switches; revoke works immediately
08	execute/08-ops-command-center.md	/ops cannot read student tables (test)
09	execute/09-trust-security.md	Restore drill timed; axe clean on pilot routes
10	execute/10-launch.md	Every go/no-go gate has evidence
11	execute/11-company-site-grow.md	Claims sourced; a11y ≥ 95
12	execute/12-one-place.md	Every one-place row has an in-app path
13	execute/13-platform-maturity.md	Scorecard re-scored from repo evidence

Pilot-critical path: **00 → 01 → 02 → 03 → 06 → 05 → 09 → 10**. Streams 04, 07, 08, 11–13 may follow after the pilot launches.

5. Migrations (exact order)

010 core → 020 community → 030 marketplace → 040 ops → **050 security hardening** → **060 command functions** → **070 AI gateway** → **080 sources, storage, delegations**. Renumber after the repo's latest migration if needed, keeping this order. 050 must be applied before any data.

6. Verify (run after every stream)

```
APP=<app|web> bash design/handoff/verify.sh                           # must end with ALL GATES PASSED
```

Then add the stream's own exit gates. Paste the full output into the report.

7. Human-only steps (the agent stops and asks)

- Bind cyber + E&O insurance · sign the DPA/FERPA addendum · submit HECVAT Full · attorney review of docs/legal/*

- Provide the Vanderbilt staging IdP metadata and test accounts · name IT, registrar and sponsor contacts
- Approve the design-system baseline · approve every PR · approve production cutover (docs/launch/Cutover Plan.html)
- Choose the AI provider and sign its data terms (no training, no retention)

---

8. Definition of go

All of: streams on the pilot-critical path merged; `verify.sh` green on main; launch runbook go/no-go gates each with evidence; §7 complete. Then follow docs/launch/Cutover Plan.html C1–C11.

---

10. Coverage map — every part of the design project and the stream that builds it

Design project area	What it is	Built by stream	Reference
Tokens, type, colour, spacing, motion, patterns	Design foundations	00 (toolkit sync), then every UI stream	tokens/, guidelines/, readme.md
Components (core, forms, data, feedback, surfaces, overlays, navigation, trust, governance)	64 components	00 maps to app; 03–08 use them	components/, docs/system/design/SEMESTER_COMPONENT_LIBRARY.md
Student app, mobile, privacy center, platform shell	Student OS	03	ui_kits/student, ui_kits/mobile, templates/privacy-center, templates/platform-shell
Role onboarding (13 roles)	First-run per role	02	templates/role-onboarding
Staff workspaces, Course Studio	Faculty, TA, advisor, registrar, finance, student affairs	04	ui_kits/staff
Institution console, Institution OS	Control plane, integrations, migration	05	ui_kits/console, templates/institution-os, templates/console-screen
AI gateway, Ask Semester	Governed AI	06	semester-platform/app/api/ai, lib/ai, 070
Community & marketplace, external audiences, career, family, developer platform	Social, market, career, guardian, partners	07	ui_kits/community, ui_kits/external
Operations consoles, Command center, One place	Company runs inside Semester	08, 12	ui_kits/ops, templates/command-center, ui_kits/one-place
Security, privacy, accessibility, compliance	Controls, evidence, ACR	09	docs/secure, docs/build/Accessibility Test Plan.html, ui_kits/compliance
Launch runbook, cutover, support, launch kit	Go-live	10	docs/launch, ui_kits/launch-kit
Company site, GTM, legal pages, brief	Public site and sales	11	ui_kits/company-site, docs/grow, docs/legal, docs/brief
Platform maturity, strategy, business OS, system docs	Programs and standards	13 (plus monthly review)	docs/platform, docs/strategy, docs/business, docs/system

Design project area	What it is	Built by stream	Reference
Master catalog, every screen, gap screens, role pages, workflow runner, atlas, role lens, wireframes	The full inventory (673 screens, 17 workflows, 319 steps, 49 roles)	Acceptance list for 03–08: each catalogued screen and workflow step must exist in the app or be listed as deferred in the PR	ui_kits/master-catalog, every-screen, gap-screens, role-pages, workflow-runner, docs/master
Architecture, read models, environments	System design	01	ui_kits/architecture, guidelines/arch-*.html
Decks, reports, teaser, poster, research brief, vision deck	Sales and launch materials	Not code: export from the design project (PDF/PPTX)	templates/*
Business workspace, operating manual, finance model, hiring	Running the company	Not code: used by the founder; /ops modules in 08	ui_kits/business, docs/run

Completeness rule: at the end of streams 03–08 the agent runs a coverage check: every screen in docs/master/SEMESTER_SCREEN_CATALOG.md and every step in SEMESTER_WORKFLOW_CATALOG.md is either implemented (route + test) or listed as deferred with a reason and priority. Nothing is silently skipped.

---

9. Where things are

`execute/` stream prompts · `semester-platform/` DB, API, tests · `semester-core/` decision logic · `claude-code-toolkit/` design-system tooling · `manifest.json` every file and its destination · design references in `../ui_kits`, `../templates`, `../docs`.

Execution plan, step by step

handoff/execute/00-INDEX.md

Semester — execute everything

> Read `handoff/BUILD.md` first. Run `preflight.sh`, `install.sh`, then these streams; run `verify.sh` after each.

One prompt per stream, run in order. Each stream works on its own branch and ends with a PR and a report. Do not start a stream until the previous one's exit gates are ticked.

Master prompt (paste into Claude Code or Codex at the repo root)

```
 You are building Semester in harrisonjrubin7-cmyk/semester.
Read handoff/BUILD.md and follow it exactly. Run handoff/preflight.sh and handoff/install.sh. Then run
each stream file in order:
handoff/execute/00-setup.md
handoff/execute/01-platform-core.md
handoff/execute/02-identity-sso.md
handoff/execute/03-student-os.md
handoff/execute/04-staff-academic.md
handoff/execute/05-institution-integrations.md
handoff/execute/06-ai-gateway.md
handoff/execute/07-community-career-family.md
handoff/execute/08-ops-command-center.md
handoff/execute/09-trust-security.md
handoff/execute/10-launch.md
handoff/execute/11-company-site-grow.md
handoff/execute/12-one-place.md
handoff/execute/13-platform-maturity.md
For each stream: create branch semester/<slug> from origin/main, do the tasks, run every check, write the
report to docs/execute/<nn>-<slug>-report.md, open a PR, and STOP for human review before the next
stream.
If an input file is missing from the repo, copy it from the Semester design project export under design/.
Never deploy, change secrets, or mark anything complete without evidence.
```

Streams

#	Stream	Branch	Depends on
00	Repo audit and design-system sync	semester/setup	—
01	Database, tenancy, RLS and core logic	semester/platform-core	00
02	Identity, SAML SSO, onboarding	semester/identity-sso	01
03	Student OS	semester/student-os	01, 02

#	Stream	Branch	Depends on
04	Faculty, TA, advisor, registrar, finance, student affairs	semester/staff-academic	01, 02
05	Institution console, integrations, migration	semester/institution-integrations	01, 02
06	Governed AI gateway	semester/ai-gateway	01, 02
07	Community, career, family, developer platform	semester/community-career-family	01, 02
08	Operations command center and site operations	semester/ops-command-center	01, 02
09	Security, privacy, accessibility, compliance	semester/trust-security	01, 02
10	Pilot launch	semester/launch	01, 02
11	Public site, GTM, run and grow	semester/company-site-grow	01, 02
12	One place: nobody leaves Semester	semester/one-place	00–08
13	Platform maturity	semester/platform	00–10

Getting the design files into the repo

Download this design project, then copy into the repo under `design/`: `handoff/`, `templates/`, `docs/`, `ui_kits/`, `guidelines/`, `readme.md`, `SKILL.md`. Code files in handoff/ end in .txt; strip the suffix when installing.

Founder-only steps (no agent can do these)

- Sign the DPA/FERPA addendum; bind cyber and E&O insurance; submit HECVAT Full.
- Attorney review of docs/legal/*.
- Approve the design-system baseline (stream 00) and every PR.
- Name Vanderbilt contacts for IT, registrar and sponsor.

`handoff/execute/00-setup.md`

Stream 00 · Repo audit and design-system sync

Make the repo ready for every later stream.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `handoff/claude-code-toolkit/`
- `docs/master/*.md`
- `github.md` (screen map)

Tasks

1. Run handoff/claude-code-toolkit/EXECUTE.md steps 1, 2, 4 and 5 (skip Figma unless connected).
2. Write docs/master/REPO_AUDIT.md: every route, screen, component, migration, API route, test and doc in the repo, mapped to the rows of docs/master/SEMESTER_SCREEN_CATALOG.md and SEMESTER_WORKFLOW_CATALOG.md. Mark each row exists / partial / missing.
3. Update docs/master/SEMESTER_GAP_REGISTER.md from real evidence (repo beats design).
4. Add CI jobs if missing: typecheck, lint, unit tests, design-system check, RLS test job (can be empty until stream 01).

Exit gates

- [ ] CI green
- [ ] REPO_AUDIT.md covers every catalog row
- [ ] Baseline JSON committed and approved

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/setup` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

handoff/execute/01-platform-core.md

Stream 01 · Database, tenancy, RLS and core logic

Land the data foundation every screen reads from.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `handoff/semester-platform/` (migrations 010-040, lib, app routes, tests)
- `handoff/semester-core/`
- `ui_kits/architecture/` (20-table design schema, flows)

Tasks

1. Reconcile migrations 010-040 with existing supabase/migrations; renumber after the latest existing one. Never edit an applied migration; add new ones.
2. Add memberships, roles, capabilities, grants (scoped, expiring), sessions, delegations, audit.events (append-only), outbox and read_model schemas from the architecture kit.
3. Install semester-core into app/src/lib/core/; reconcile names with lib/source.ts, lib/status.ts, lib/navareas.ts; run its tests.
4. Write RLS tests per table: own-tenant read/write, cross-tenant denied, company roles denied on student tables, expired grant denied, audit insert-only.
5. Apply 050_security_hardening.sql and prove tests/security.test.ts red-then-green (docs/secure/Platform Security Review.html), then 060_command_functions.sql (tests/commands.test.ts), 070_ai_gateway.sql and 080_sources_storage_delegations.sql; add middleware.security.ts and the CI security job.
6. Outbox worker and projections with version and freshness; freshness.ts drives UI policy.

Exit gates

- [ ] npx supabase db reset runs clean
- [ ] RLS suite passes with a red-then-green proof per policy
- [ ] semester-core tests pass

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/platform-core` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.

- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/02-identity-sso.md`

Stream 02 · Identity, SAML SSO, onboarding

Sign-in for the Vanderbilt pilot and first-run onboarding for every role.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `templates/role-onboarding/` (13 role onboardings + RoleOnboarding)
- `docs/launch/Cutover Plan.html` (steps C2-C3)
- `ui_kits/student` (Connect, Settings, Privacy)

Tasks

1. SAML SSO via Supabase Auth (or the current auth in the repo) with role mapping from IdP attributes; staging test IdP first.
2. Account switcher, sessions list with remote sign-out, delegated access with expiry.
3. Build onboarding for each of the 13 roles from templates/role-onboarding/: sign in, connect, privacy choice, AI choice, first home.

4. Capabilities and Permissions screen: camera, microphone, location, notifications. Each asks, has a fallback and a revoke path.

Exit gates

- [ ] Test login per pilot role in staging
- [ ] Role mapping tests
- [ ] Onboarding a11y check per role

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/identity-sso` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/03-student-os.md`

Stream 03 · Student OS

Every student screen in catalog group B and the student side of F and G.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `ui_kits/student/` (64 screens)
- `ui_kits/mobile/`
- `templates/platform-shell/`
- `templates/student-screen/`
- `ui_kits/workflow-runner/` (Student, Student finance, Campus services)

Tasks

1. For each screen in docs/master/SEMESTER_SCREEN_CATALOG.md group B: route, data from read models, source labels, all states.
2. Today, Action Center, Path, Plan, Registration readiness, Courses, Notes, Files, Search, Ask Semester, Support, Campus, Career, Account, Privacy and data, Data export/delete.
3. Run every Student workflow step from the runner end to end against staging data.
4. Mobile: five-destination tab bar, offline queue, PWA install.

Coverage check

List every screen in docs/master/SEMESTER_SCREEN_CATALOG.md and every step in SEMESTER_WORKFLOW_CATALOG.md that belongs to this stream (see BUILD.md §10). Mark each implemented (route + test) or deferred (reason + priority). Use ui_kits/every-screen and ui_kits/workflow-runner as the visual and behavioural reference.

Exit gates

- [ ] Coverage check complete with no unlisted screens or steps
- [ ] Each Student workflow step passes an e2e test
- [ ] Every screen has all 5 states
- [ ] Lighthouse a11y 95+ on the top 10 screens

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/student-os` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.

- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/04-staff-academic.md`

Stream 04 · Faculty, TA, advisor, registrar, finance, student affairs

Staff workspaces (catalog groups C, D, E and the staff side of F).

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `ui_kits/staff/`
- `ui_kits/workflow-runner/` (Faculty, Advisor, Registrar, Student finance)
- `templates/console-screen/`

Tasks

1. Course Studio (faculty authoring): syllabus, objectives, assignments, rubrics, course AI rules, gradebook, grade release, regrade queue.
2. Advisor caseload, check-ins, referrals, consented shares.
3. Registrar desk: terms, sections, holds sync, overrides (two-person), add/drop, reconciliation with SIS.
4. Student accounts: balance, payment plans, hold release, reconciliation. Payments go through a processor only.

Coverage check

List every screen in docs/master/SEMESTER_SCREEN_CATALOG.md and every step in SEMESTER_WORKFLOW_CATALOG.md that belongs to this stream (see BUILD.md §10). Mark each implemented (route + test) or deferred (reason + priority). Use ui_kits/every-screen and ui_kits/workflow-runner as the visual and behavioural reference.

Exit gates

- [ ] Coverage check complete with no unlisted screens or steps
- [ ] Two-person approvals tested
- [ ] Official-record changes go to the SIS, never edited in Semester
- [ ] Workflow steps pass e2e

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/staff-academic` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/05-institution-integrations.md`

Stream 05 · Institution console, integrations, migration

Control plane (K) and integration fabric (L).

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `ui_kits/console/`
- `templates/institution-os/`
- `ui_kits/workflow-runner/` (Institutional implementation, Controlled integration, Domain migration)

Tasks

1. Tenant overview, roles, role grants, feature cohorts, kill switches, policies, audit explorer.
2. SIS read sync and LMS (LTI 1.3 / OneRoster) connectors, read-only first; mapping studio with versions; reconciliation dashboard; dead-letter and replay.
3. Migration center following the 17-step domain migration workflow.

Coverage check

List every screen in docs/master/SEMESTER_SCREEN_CATALOG.md and every step in SEMESTER_WORKFLOW_CATALOG.md that belongs to this stream (see BUILD.md §10). Mark each implemented (route + test) or deferred (reason + priority). Use ui_kits/every-screen and ui_kits/workflow-runner as the visual and behavioural reference.

Exit gates

- [ ] Coverage check complete with no unlisted screens or steps
- [ ] Reconciliation shows 0 unexplained mismatches on staging data
- [ ] Kill switch disables a feature in under 1 minute
- [ ] Connector credentials server-side only

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/institution-integrations` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.

- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/06-ai-gateway.md`

Stream 06 · Governed AI gateway

One gateway for every AI request.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `handoff/semester-core/aigateway.ts.txt`
- `ui_kits/architecture/` (AI gateway tab)
- `guidelines/02-ai-governance.md`
- `components/trust/AIResponse.jsx`

Starting code

- `handoff/semester-platform/app/api/ai/route.ts.txt`, `lib/ai/redact.ts.txt`, `supabase/migrations/070_ai_gateway.sql.txt`, `tests/ai-redteam.test.ts.txt`

Tasks

1. Implement the 17-step AI workflow: scope, classification, consent, policy, provider routing, approved sources, tool permissions, response with sources, suggested action, human escalation, spend and audit log.
2. Red-team suite: prompts containing student IDs, grades or records must be blocked, and every block logged.
3. Course AI rules editor for faculty; per-tenant switches.

Exit gates

- [ ] Red-team suite passes (launch gate)
- [ ] Every AI answer is labelled with sources
- [ ] Spend logged per tenant

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/ai-gateway` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/07-community-career-family.md`

Stream 07 · Community, career, family, developer platform

Groups H, I, J and the developer platform.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `ui_kits/community/`
- `ui_kits/external/`
- `handoff/semester-platform/` (migrations 020, 030)
- `ui_kits/workflow-runner/` (Community, Career, Family consent, Developer platform)

Tasks

1. Community: verified profiles with per-field visibility, groups, Q&A with endorsed answers, reporting, moderation queue, appeals. AI pre-screen on public posts only.
2. Marketplace: listings with the academic-integrity filter, offers in chat, safe meet-up spots; payments via a processor.
3. Family: consent invitation, scoped categories, expiry, access history, revoke.
4. Career: skills, evidence portfolio, verified skills, opportunities; employers see approved evidence only.
5. Developer platform: OAuth clients, scopes, webhooks, sandbox tenants (can ship after the pilot).

Coverage check

List every screen in docs/master/SEMESTER_SCREEN_CATALOG.md and every step in SEMESTER_WORKFLOW_CATALOG.md that belongs to this stream (see BUILD.md §10). Mark each implemented (route + test) or deferred (reason + priority). Use ui_kits/every-screen and ui_kits/workflow-runner as the visual and behavioural reference.

Exit gates

- [ ] Coverage check complete with no unlisted screens or steps
- [ ] Feature switch per institution
- [ ] Revocation removes access immediately (tested)
- [ ] Moderation audit trail

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/community-career-family` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/08-ops-command-center.md`

Stream 08 · Operations command center and site operations

/ops for Semester operators (group N).

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `ui_kits/ops/` (12 consoles)
- `templates/command-center/`
- `docs/launch/Support Plan.html`

Tasks

1. Executive overview, ops inbox, approvals, tenants, pilots, support desk, incidents, SLOs and error budgets, feature flags, releases, audit explorer, finance dashboard.
2. Business data only: a test proves every /ops query fails against student tables.
3. Monitoring: error tracking, uptime, SSO failures, SIS lag, gateway blocks, with alert routes from the support plan.

Coverage check

List every screen in docs/master/SEMESTER_SCREEN_CATALOG.md and every step in SEMESTER_WORKFLOW_CATALOG.md that belongs to this stream (see BUILD.md §10). Mark each implemented (route + test) or deferred (reason + priority). Use ui_kits/every-screen and ui_kits/workflow-runner as the visual and behavioural reference.

Exit gates

- [ ] Coverage check complete with no unlisted screens or steps
- [ ] Company-role isolation test passes
- [ ] Alerts fire in staging
- [ ] On-call runbook linked from each alert

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/ops-command-center` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

---

handoff/execute/09-trust-security.md

Stream 09 · Security, privacy, accessibility, compliance

Evidence for HECVAT, SOC 2 readiness and VPAT/ACR.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `docs/secure/Security Policies.html`
- `docs/secure/Security Readiness.html`
- `ui_kits/compliance/`

Tasks

1. Map each policy control to code or a procedure; store evidence links in the evidence register.
2. Backups and a timed restore drill; break-glass with review; access reviews; secret inventory.
3. Accessibility: axe in CI on every route, manual screen-reader pass on pilot paths, and the ACR test log.
4. Data inventory, retention jobs, privacy request flows (export, correction, deletion).

Exit gates

- [ ] Restore drill timed and documented
- [ ] axe clean on pilot routes
- [ ] Evidence register complete for HECVAT answers

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/trust-security` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.

- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/10-launch.md`

Stream 10 · Pilot launch

Take the Vanderbilt pilot live.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `docs/launch/` (runbook, cutover plan, support plan)

Tasks

1. Implement the cohort flag, holding message and status page.
2. Script cutover steps C1-C11 where possible; produce the cutover log template.
3. Support desk macros from the support plan; known-issues page.

Exit gates

- [ ] Every go/no-go gate has evidence attached
- [ ] Rollback rehearsed in staging

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/launch` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/11-company-site-grow.md`

Stream 11 · Public site, GTM, run and grow

The public site and company operating documents.

Paste this file into Claude Code (or Codex) at the repo root.

Inputs from the design project

- `ui_kits/company-site/`
- `docs/brief/`
- `docs/legal/` (drafts)
- `ui_kits/business/`

Tasks

1. Full public site: home, product, institutions, students, security/trust, pricing, about, contact, legal. Claims come only from the public claims register: status, not certifications.
2. Contact and pilot-request forms with spam protection; no student data collected.
3. Publish legal drafts as pages marked "draft pending attorney review" until reviewed.

Exit gates

- [ ] Every claim cites its source
- [ ] a11y 95+
- [ ] Forms tested

Rules (every stream)

- Repo: harrisonjrubin7-cmyk/semester. Work on branch `semester/company-site-grow` from latest `origin/main`. Open one PR per stream; merge only after CI is green and a human approves.
- Read first: CLAUDE.md, README, `git log --oneline -30`, and this design project's `readme.md`, `SKILL.md`, `docs/master/*`.
- Audit before you write: find what already exists for each item and extend it. Do not create a parallel implementation. Record each "exists / extended / new" decision in the PR description.
- Design system: use tokens and components only (no raw hex, px z-index or durations); `npm run design-system:check` must pass.
- Data: RLS on every table, `tenant_id` on every row, browser calls `api.*` only. Company roles (investor, CS, RevOps, finance) never read student tables. AI never receives student records.
- Every consequential action: preview, then audit event written first, then apply, then an undo or request path.
- Every screen: loading, empty, error, forbidden and offline states, keyboard path, visible focus, accessible names, 44px targets.
- Do not commit secrets, change production settings, deploy, or write to Figma. Stop and report on the first failure you cannot fix inside this stream.
- Nothing is "done" without code, tests, RLS proof, accessibility check, monitoring hook, docs and release evidence (the 22-point definition in `docs/master/SEMESTER_RELEASE_GATE_CATALOG.md`).

Report at the end

Files changed · commands run with results · tests added · RLS cases proven · open gaps with priority · what the next stream needs.

`handoff/execute/12-one-place.md`

Stream 12 · One place: nobody leaves Semester

Make every job in `ui_kits/one-place/one-place-data.js` doable inside Semester. Paste into Claude Code (or Codex) at the repo root after streams 00–08.

Inputs from the design project

- `ui_kits/one-place/` (job map: designed / to build / embedded)
- `ui_kits/ops/` (company consoles)
- `ui_kits/student/` (Write, Sheets, Decks, Mail, Calendar)

Tasks

1. For each "to build" row, build the module (Meet, team spaces, contracts with in-app signing, career portfolio) using existing shells and components.
2. For each "embedded" row, build the in-app hand-off: provider screen inside Semester (processor checkout, payroll status, e-sign, IdP sign-in), status written back to Semester, audit event on each step. No regulated step is re-implemented.
3. Run the company inside /ops: CRM, support, incidents, social, finance, people, compliance, product ops, board reports. Semester staff use Semester for docs, sheets, decks, mail and calendar.
4. Add an "open in Semester" check to the release checklist: any new feature that sends users to another app needs an embedded path or a written exception.

Exit gates

- [ ] Every row in one-place-data.js is designed, built, or embedded with a working in-app path
- [ ] Company operating cadence (weekly metrics, monthly close, QBRs) runs from /ops
- [ ] No student data in company modules (isolation test passes)

Follow the shared rules in any other stream file (branch `semester/one-place`, PR + report, stop for review).

`handoff/execute/13-platform-maturity.md`

Stream 13 · Platform maturity

Turn Semester from a broad product architecture into a compounding education platform. Run after the pilot is live (streams 00–10).

Inputs from the design project

- `docs/platform/*.md` (12 pillar standards, scorecard, risk register, 36-month roadmap, next 50 actions)

- `ui_kits/platform-maturity/`
- `docs/strategy/` (competitive audit, coherence test, implementation factory, AI differentiation, trust advantage)
- `docs/system/` (brand, UI grammar, component gaps, algorithm governance, workflow type, data classification, provenance)
- `docs/business/` (company OS, five business systems, systems of record)

Tasks

1. Audit the repo against each pillar's capability record; write current evidence into docs/platform/*.md (repo beats design).
2. Re-score PLATFORM_READINESS_SCORECARD.md with evidence links; never raise a score without evidence.
3. Work the next 50 actions in order; each completed action links its PR or document.
4. Build the platform pieces in pillar order: daily experience standard checks in CI, graph canonical IDs + source authority, AI evaluation harness, connector certification suite, status page, public API beta.
5. Add the coherence test to the PR template and release checklist.
6. Implement the SemesterWorkflow type, universal state machine, data classification tags and provenance columns from docs/system/.
7. Build the missing components: PolicyBadge, ConsentBadge, HealthBadge, SupportHandoff, ApprovalBanner, ErrorSummary, PageHeader.

Exit gates

- [ ] Scorecard re-scored from repo evidence
- [ ] Every pillar has a named owner and next action with a date
- [ ] Monthly scorecard review in the operating cadence

Follow the shared rules in any other stream file (branch `semester/platform`, PR + report, stop for review).

Code packages

handoff/semester-core/README.md

semester-core — the workflows and algorithms, as code

Pure TypeScript, no dependencies, no I/O. Each module is the decision logic behind a Semester workflow the design system renders; the app supplies data and persistence. Every module has a vitest file proving its rules.

Module	Workflow	Used by
`provenance.ts`	Rank sources; resolve conflicting facts; freshness label	SourceBadge, ProvenanceChips, Today
`nextaction.ts`	Score and order a student's next actions	Today, NextSteps, Runway
`access.ts`	Role × screen access, share expiry, break-glass	Role lens, every route guard
`freshness.ts`	Projection lag → UI policy (display / safe / destructive)	Read models, Lag simulator
`consequence.ts`	Classify an action → undo / preview / two-person	Dialog, ActionPreview, Approvals
`aigateway.ts`	Scope + policy check before any model call	Ask Semester, AIResponse
`approvals.ts`	Two-person duty state machine	Console approvals, Finance holds
`scoring.ts`	Weighted 0–5 grading with overrides	Workflow lab benchmark
`pilot.ts`	Launch readiness from the tracker	Business gap analysis

Files carry a `.txt` suffix so this design project does not bundle them; drop it when copying (`for f in *.txt; do mv "$f" "${f%.txt}"; done`). Install into `app/src/lib/core/` (or a workspace package). Run `npx vitest run src/lib/core`. These are reference implementations reconciled with the design system — check names against `lib/source.ts`, `lib/status.ts` and `lib/navareas.ts` before replacing anything that already exists.

handoff/semester-platform/README.md

semester-platform

Database and API code for community, marketplace, ops and audit, built on the core Semester app (Next.js + Supabase). Files end in .txt so they don't run in the design project. Strip the suffix when you copy them into the repo.

Install: cp supabase/migrations/*.sql.txt /supabase/migrations/ (rename to .sql) cp -r app lib tests / (rename .ts.txt to .ts) npx supabase db reset && npm test

Status: written to the architecture guide. Not yet run, typechecked or tested; do that before you rely on it.

Security hardening (Oct 2026)

migrations/050_security_hardening.sql.txt closes 11 findings from the review of 010–040 (2 critical/high: ops.staff self-insert as 'ceo', and moving a profile to another institution). tests/security.test.ts.txt proves each one. Apply 050 before any real data. Review: docs/secure/Platform Security Review.html in the design project.

migrations/060_command_functions.sql.txt: all community/market writes go through audited api.* functions (validation, rate limits, integrity filter, campus checks); storage policies for listing photos. Proven by tests/commands.test.ts.txt.

AI gateway: app/api/ai/route.ts.txt (auth → policy → gate → content scan → provider → audit), lib/ai/redact.ts.txt (blocks IDs, SSN, GPA, grades, transcripts, DOB, cards typed into prompts), migrations/070_ai_gateway.sql.txt (tenant policy, course rules, usage log with no prompt content), tests/ai-redteam.test.ts.txt (launch gate).

080_sources_storage_delegations: approved-source AI context, private buckets with path rules, audited delegations. middleware.security.ts.txt: CSP/HSTS/headers. .github/ci-security-job.yml.txt: secret scan, audit, guards, security tests.

---

handoff/workflow-router-app/README.md

Semester AI Workflow Router & Benchmark Lab

Next.js App Router + Supabase app that routes a Semester workflow to **Claude Artifacts**, **ChatGPT Canvas** or **v0**, compares them in sortable matrices, and records a graded 12-workflow benchmark with evidence.

> Built in Claude Design as a source hand-off. Every code file carries a `.txt` suffix (and `[suiteId]` is saved as `-suiteId-`) so the design-system bundler ignores them. Restore with `sh install.sh`. **Typecheck, lint, tests and build have not been run here** — run them first (see below) and fix anything they report.

Install

```
cp -R workflow-router-app ~/code/semester-router && cd ~/code/semester-router
sh install.sh            # strips .txt, restores app/benchmark/[suiteId]
npm install
cp .env.example .env.local   # fill NEXT_PUBLIC_SUPABASE_URL + NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY
```

Supabase

```
npx supabase init && npx supabase start          # local
npx supabase db push                             # or: link a project, then push
# sign up once in the app, then:
psql "$DB_URL" -f supabase/seed.sql              # seeds "Semester canonical 12"
npm run db:types                                 # regenerate types/database.ts
```

Auth → URL configuration: Site URL = your origin; add `/auth/confirm` to redirect URLs. Email template link: `{{ .SiteURL }}/auth/confirm?token_hash={{ .TokenHash }}&type=email&next=/workflow-router`.

Run and verify

```
npm run dev
npm run typecheck && npm run lint && npm test && npm run build
```

Quality gate	Where it is held
Router works on local data	`lib/workflow-router/engine.ts` (no network) · `tests/routing.test.ts`
v0 forced for DB/auth/multi-tenant/Next.js/deploy	`v0Triggers` · 13 forced cases in `tests/routing.test.ts`
Markdown + JSON recommendation export	client download + `app/api/exports/recommendation`
Authenticated save	`app/workflow-router/actions.ts` (server recomputes; `getUser()`)
12 workflows per suite, run per platform/task	`benchmark_tasks unique(suite_id, number)` · `benchmark_runs`
Grades with evidence	`benchmark_grades unique(run_id, grader_id)` · `GradingForm` requires evidence
Weighted scores + leaderboards	`lib/benchmark/scoring.ts` · `tests/scoring.test.ts`
Persistence explicitly tested	DB check constraint + `RunSchema` + `persistenceCeiling` (local state ≤ 2)
RLS isolation	migrations 001/002 · `tests/RLS-TEST-PLAN.md`
No secret in the browser	only `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`; `lib/supabase/server.ts` imports `server-only`

Benchmark protocol (all platforms are manual — no automation is assumed)

1. Open a suite → task → pick a platform. The capability guard marks persistence tasks as **unsupported** on Claude Artifacts and ChatGPT Canvas with the reason; record the run anyway to document the gap.
2. **Claude Artifacts:** paste the exact prompt into claude.ai, publish the artifact, record the URL, the source as the manifest, console notes and duration.
3. **ChatGPT Canvas:** run the prompt in Canvas, export (Markdown/DOCX/PDF or code file), record the share link and file manifest.
4. **v0:** run the prompt, push to GitHub, deploy a preview, record repo + deployment URLs, the file manifest and the test log.
5. Persistence tasks (9–12): create a record, reload, and record survived/lost, the storage mode and a row id or log. Without evidence the persistence grade is capped at 1; browser-local at 2.
6. Two graders minimum per run; grade 0–5 on each criterion with evidence. Export CSV / Markdown from the suite page.

GitHub and Vercel

Push to GitHub → import in Vercel → add the two `NEXT_PUBLIC_SUPABASE_*` env vars (Production + Preview) → deploy. Protect `main`; require typecheck, lint, test and build on PRs; run `supabase db push` from CI with a deploy token, never from the browser.

Security / RLS

- Every exposed table has RLS on. Private rows: `owner_id = auth.uid()`. Shared rows: `visibility = 'organization'` **and** membership via `is_org_member` / `has_org_role` (SECURITY DEFINER, `search_path = public`) to avoid recursive policies.
- Viewers read; members write their own; owners/admins manage membership; only owners grant owner.
- Runs and grades inherit access from their suite (`can_read_suite` / `can_write_suite`).
- Mutations validate with Zod on the server and re-derive the recommendation; client UI state never authorises anything.

Claude Code toolkit

handoff/claude-code-toolkit/README.md

Semester · Claude Code design-system toolkit

Drop-in files for **harrisonjrubin7-cmyk/semester**, built from the brief in `uploads/provide comand to claud code to excute on this and.pdf` and checked against the repo (`lib/tokenexport.ts`, `docs/DESIGN-TOKENS.md`, `styles/hex.test.ts`, `design-tokens/semester.tokens.json`, `.claude/skills/`).

Copy the tree into the repository root — every path matches the repo layout. The three code files carry a `.txt` suffix so the design-system bundler here ignores them; drop it when copying:

```
for f in $(find . -name "*.txt"); do mv "$f" "${f%.txt}"; done
```

File	Purpose
`.claude/skills/audit-semester-design-sync/SKILL.md`	Read-only, repo-specific design-system audit (tokens, components, responsive, a11y, Figma parity)
`.claude/skills/build-semester-ui/SKILL.md`	Building UI with unity primitives, trust vocabulary, token rules and the finish checks
`.mcp.json`	Remote Figma MCP server (`type: http`), no credentials
`docs/design-system/FIGMA-MAPPING.md`	Template: ownership, rules, proposed Figma names ↔ verified exported paths, components, unresolved
`docs/design-system/SEMESTER-DESIGN-SYSTEM-BASELINE.md`	Phase 1 read-only baseline: architecture, source-of-truth map, strengths, gaps, risks, plan
`docs/design-system/README.md`	How to use the skills, Figma MCP, checks and report
`CLAUDE.md.design-system-section.md`	Section to append to the existing CLAUDE.md
`.claude/skills/create-semester-component/SKILL.md`	Spec-first shared component; rejects one-off wrappers
`.github/ci-design-system-steps.yml`	Three steps for the existing `ci.yml` build job + report artifact
`gitignore.addition`	`app/reports/`
`app/scripts/design-system-audit.mjs`	Zero-dependency audit: raw colours vs the hex ledger, undefined custom properties, tokens.css ↔ export sync, mapping resolution
`app/scripts/design-system-report.mjs`	Markdown report for CI (`--out`), exits with the audit status
`app/src/lib/designsystem.test.ts`	Proves each check fails on a violating fixture and the real tree is clean
`app/package.scripts.json`	`tokens:check`, `design-system:audit`, `:report`, `:check` to merge into `app/package.json`

Nothing here adds a dependency, a token source, Tailwind, Storybook or a package. `semester.tokens.json` stays generated by `npm run tokens:export`.

See `PROMPTS.md` for the exact Claude Code commands.

handoff/claude-code-toolkit/EXECUTE.md

Execute the five follow-ups in Claude Code

Paste into Claude Code at the repository root. It runs everything I could not run from Claude Design.

```
 Read handoff README.md, CLAUDE.md, docs/design-system/SEMESTER-DESIGN-SYSTEM-BASELINE.md. git fetch
origin main && git log --oneline -30 origin/main first.
Do not commit, push, open a PR, change secrets or write to Figma. Stop and report after each step that
fails.

STEP 1 — Install and run
- Copy the toolkit tree into the repo; for f in $(find . -name "*.txt" -path "*toolkit*"); do mv "$f"
"${f%.txt}"; done
- Append CLAUDE.md.design-system-section.md to CLAUDE.md (do not change existing sections). Add
gitignore.addition to .gitignore.
- Merge app/package.scripts.json into app/package.json scripts (keep existing scripts).
- From app/: npm run tokens:check; npm run design-system:audit; npx vitest run
src/lib/designsystem.test.ts; npx tsc -b; npm run lint; npm test.
- Fix only failures caused by the toolkit (e.g. a regex catching a false positive). Revert one check's
fix and watch its test go red, then restore. List pre-existing failures separately.

STEP 2 — Agree the raw-value baseline
- npm run design-system:baseline → app/design-system-baseline.json. Print totals per check (z-index,
duration, easing, shadow, font-size, radius) and the 10 largest files.
- Show me the table and wait for "approve baseline" before continuing.

STEP 3 — Figma (needs me)
- If /mcp shows figma connected and I give a file URL: read its variable collections and components
(read-only). Replace every "(proposed)" name in docs/design-system/FIGMA-MAPPING.md with the fetched
name, set Status per row (Match / Partial / Missing in Figma / Missing in code), move unmatched Figma
variables to Unresolved. Do not invent names. Run design-system:audit; 0 missing mappings.
- If not connected: say so and skip.

STEP 4 — Move inline z-index and durations onto tokens
- For each baseline entry with check z-index or duration in app/src (not look.ts, not LEDGER files):
  z-index 1→var(--layer-raised) 20→var(--layer-sticky) 21→var(--layer-chrome) 80→var(--layer-overlay)
90→var(--layer-menu) 100→var(--layer-skip) 1500→var(--layer-curtain); any other number: leave it and list
it with its reason.
  durations: transitions/animations → var(--motion-save|insert|panel|sheet|progress) by job (130ms
hover/press → var(--fast)); never --duration-* directly (those are not zeroed for reduced motion).
- Keep stacking.test.ts, motion.test.ts, a11y tests green. Re-run npm run design-system:baseline so the
baseline shrinks; the stale check enforces it.

STEP 5 — Make it a CI gate
- Insert .github/ci-design-system-steps.yml into .github/workflows/ci.yml build job after "npm run lint".
The committed baseline now makes any new raw value a failure.
- Run the full gate list from CLAUDE.md. Report: files changed, commands + results, baseline
before/after, remaining entries with reasons.
```

handoff/claude-code-toolkit/PROMPTS.md

Claude Code commands

0 · Set up (once, from the repo root)

```
 cd /path/to/semester
claude mcp add --scope project --transport http figma https://mcp.figma.com/mcp
# or: claude plugin install figma@claude-plugins-official   (pick one route, not both)
claude
```

In Claude Code run `/mcp`, authorise Figma in the browser, confirm it shows connected.

1 · Audit (no edits)

```
 /audit-semester-design-sync app/src app/design-tokens/semester.tokens.json app/src/styles

You are auditing the existing Semester repository, not designing a generic new system.
First: git fetch origin main && git log --oneline -30 origin/main.
Read: CLAUDE.md, app/README.md, DESIGN-SYSTEM-GUIDE.md, docs/DESIGN-TOKENS.md,
docs/DESIGN-TOKEN-ARCHITECTURE.md, docs/design/DESIGN-SYSTEM-PRODUCT-SPEC.md, app/package.json,
app/src/styles/tokens.css, app/src/lib/look.ts, app/src/lib/tokenexport.ts (+ .test.ts),
app/design-tokens/semester.tokens.json, app/src/styles/*.test.ts, .claude/skills/.
Inspect: app/src/components/ (esp. unity/), app/src/screens/, app/src/styles/, App.tsx, main.tsx,
screens.tsx.
Do not modify files. Produce the report format in the skill:
1 architecture + source-of-truth map; 2 token pipeline look.ts/tokens.css → tokenexport.ts →
semester.tokens.json → Figma variables; 3 strengths to preserve; 4 drift risks (raw styles, hex,
duplicated component styles, hard-coded z-index/durations); 5 existing tests that enforce contracts;
6 exact missing files for the Claude Code + Figma MCP workflow only; 7 minimal phased plan;
8 table: priority · exact path · rationale · acceptance test.
Use evidence from actual files. Do not assume pnpm, monorepos, Storybook or Tailwind.
```

2 · Implement the toolkit (after you approve the audit)

```
 Implement the approved Semester Claude Code design-system toolkit.

Constraints:
- Preserve the Vite + React app under app/, tokens.css as the semantic authority, look.ts (tokensFor)
  as the value authority, tokenexport.ts as the export authority, semester.tokens.json as generated
output.
- Extend the existing tokenexport, style, accessibility, responsive and contract tests; don't replace
them.
- No Tailwind, Storybook, monorepo, new component package, or new dependency.
- Don't move components or rewrite App.tsx. Don't commit, push, open a PR or touch deployment config.

Create only (the drafts are in this hand-off folder — reconcile them with the audit, don't paste
blindly):
1 .claude/skills/audit-semester-design-sync/SKILL.md
2 .claude/skills/build-semester-ui/SKILL.md
3 .mcp.json (Figma remote MCP, no credentials)
4 docs/design-system/FIGMA-MAPPING.md
5 app/scripts/design-system-audit.mjs
```

```
6 app/scripts/design-system-report.mjs
7 app/package.json scripts: design-system:audit, design-system:report, design-system:check
8 app/src/lib/designsystem.test.ts proving: the export is generated and in sync; raw values outside
  the hex ledger fail; every Figma mapping row resolves; audit/report fail on violating fixtures and
  pass on valid ones.

Show the file-by-file plan before editing. Revert each fix under its test once to watch it go red.
After editing run, from app/: npx tsc -b && npm run lint && npm test && npm run test:shuffle &&
npm run design-system:check. Report changed files, exact commands, results, limitations and any
Figma steps left for me.
```

---

3 · Day-to-day

```
/build-semester-ui screens/Degree.tsx https://www.figma.com/design/…?node-id=…
```

```
cd app && npm run design-system:report -- --out design-system-report.md
```

---

handoff/claude-code-toolkit/CLAUDE.md.design-system-section.md

---

The design system, and where each value is decided

Purpose. Build interfaces native to Semester — calm, source-aware, accessible, dense without clutter. Not a generic SaaS dashboard.

Source of truth, in order. `app/src/lib/look.ts` (`tokensFor`) decides every value → `app/src/styles/tokens.css` names them by job → `app/src/lib/tokenexport.ts` exports them → `app/design-tokens/semester.tokens.json` is **generated**: never hand-edit it; `npm run tokens:export` rewrites it and `tokenexport.test.ts` fails on drift. The `styles/*.test.ts` files are design contracts; extend them, never loosen them to go green.

Before writing UI. Search `components/unity/`, `components/ui.tsx`, the nearest screen in `screens/`, and the relevant `styles/*.test.ts`. Reuse first. Use `/build-semester-ui`; a new shared component goes through `/create-semester-component`.

Raw values. Feature UI uses semantic variables only — no raw colours, spacing, radii, shadows, z-indexes, font sizes, durations or easing. Permitted: `look.ts`, the first `:root` of `app.css`, the `hex.test.ts` LEDGER (with a reason), generated output, test fixtures. `npm run design-system:audit` reports the rest by file:line.

Accessibility. Native elements; visible focus (`--focus-color`, clear of sticky chrome); icon-only controls named; label/hint/error associated; every state as glyph + word + tone; motion only through `--motion-*`; keep `a11y/*` and `taps.test.ts` green.

Responsive. Follow `breakpoints.test.ts`, `gutter.test.ts`, `density.test.ts`, `taps.test.ts`, `stacking.test.ts`. Keep the primary action at every width; nothing hover-only.

Figma. Figma MCP is read-only design intent. Map variables through `docs/design-system/FIGMA-MAPPING.md` to exported paths and components to existing patterns before writing anything. Missing Figma states never excuse missing loading/empty/error/disabled/permission/keyboard/focus/responsive behaviour. Record unresolved mappings in that file. Never write to Figma.

Done means. From `app/`: `npx tsc -b && npm run lint && npm test && npm run design-system:check` (plus `npm run tokens:export` if tokens changed). Report files changed, patterns reused, states and a11y covered, and gaps left.

Tooling cards

Module	Decides	Rendered by
provenance.ts	Source rank, conflicts, freshness label	SourceBadge, ProvenanceChips
nextaction.ts	Order of a student’s next actions	Today, NextSteps
access.ts	Role × screen, share expiry, break-glass	Role lens, route guards
freshness.ts	Lag → display / safe / destructive	Read models, lag simulator
consequence.ts	Undo / preview / two-person	Dialog, ActionPreview
aigateway.ts	Scope + policy before any model call	AIResponse
approvals.ts	Two-person duty state machine	Console approvals
scoring.ts	Weighted 0–5 grading	Workflow lab
pilot.ts	Launch readiness	Business gap analysis

No dependencies, no I/O, vitest proof per module. Files end in .txt; strip on copy to app/src/lib/core/.

tooling semester core

010 Core

core.institutions, core.profiles, audit.events

020 Community

groups, members, posts, reports, events, rsvps

030 Market

listings, offers, drops, bids

040 Ops

staff, accounts, invoices, social_posts, settings

supabase/migrations
*.sql.txt → .sql

→

app · lib
validation, 2 API routes

→

tests
RLS + unit

→

npx supabase db reset
then npm test

Status: written to the architecture guide. Not yet run, typechecked or tested.

tooling semester platform

Prototype · ui_kits/workflow-lab/

Router
6 inputs → tool, confidence, reasons, limits, escalation, prompts

Matrix
16-column feature matrix, 12 canonical workflows

Benchmark
Persistence tests, 0–5 grading, weighted /5 and /100, leaderboard, CSV/MD export

Saved on this device only; labelled prototype mode.

Production · handoff/workflow-router-app/

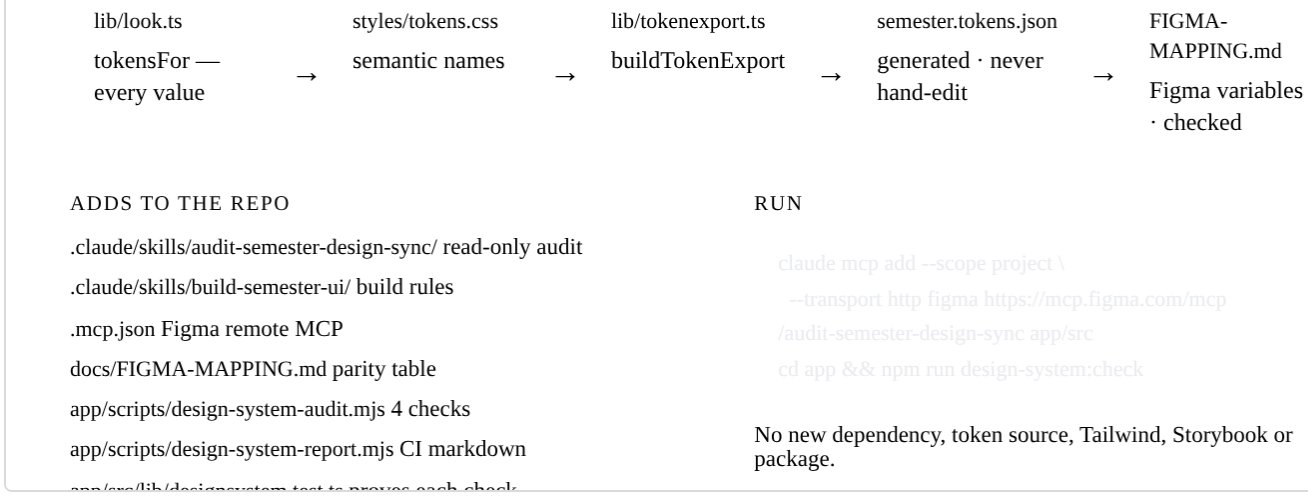
Stack
Next.js App Router, TypeScript strict, Tailwind, Supabase (@supabase/ssr, magic link), Zod, Server Actions

Data
001_core_workflow_router, 002_benchmark_grading; org-scoped RLS (owner · admin · member · viewer); seed with 12 workflows

Tests
vitest suites, RLS test plan

Run install.sh. Not yet typechecked, linted, tested or built.

tooling workflow router



tooling claude code

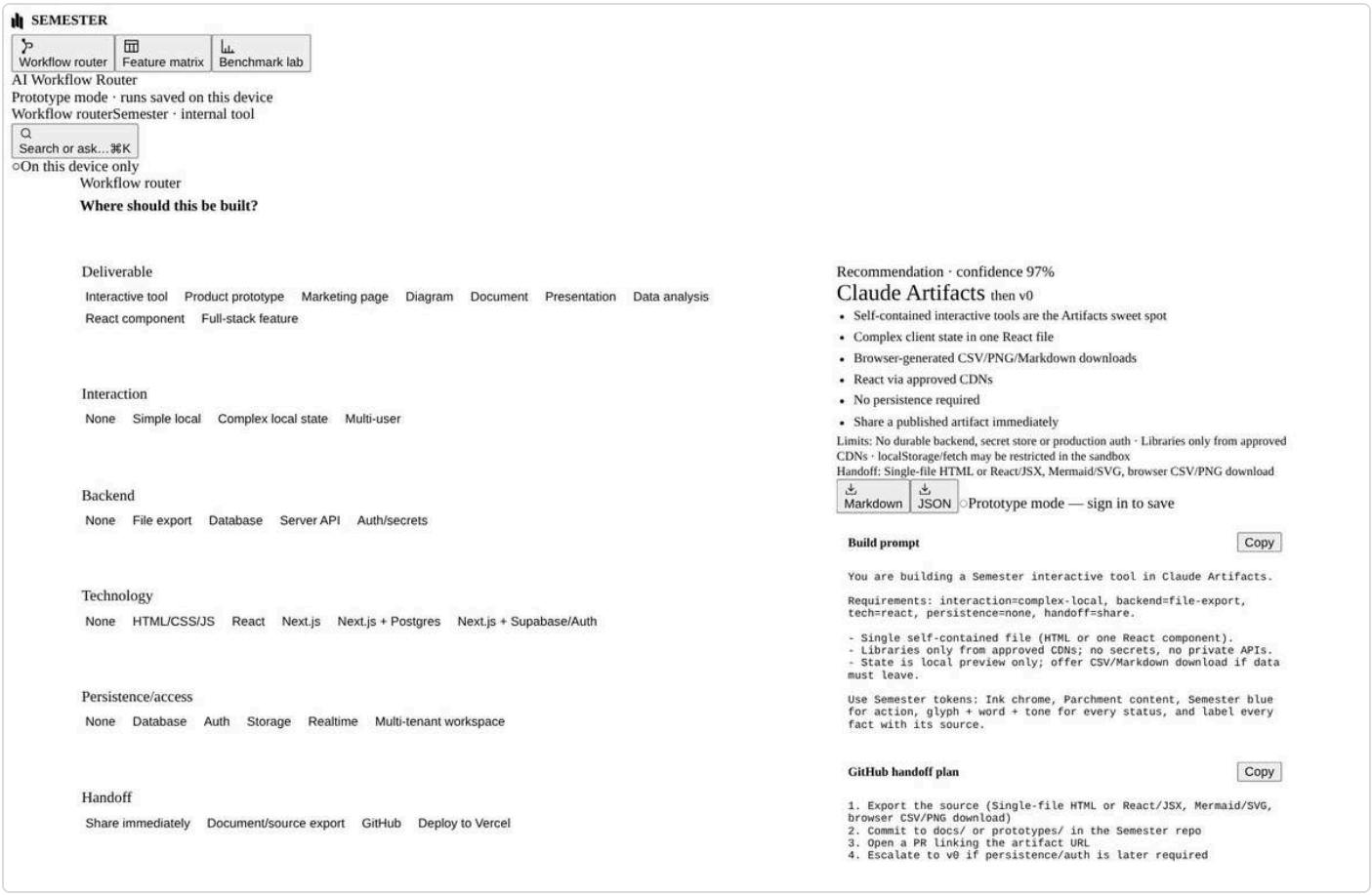
Stream	Built	Next
Brief	One-pager, FAQ, role sheets, overview deck	Real screenshot on slide 4
Guides	Architecture, developer, user	Admin manual, tutorials
Launch	Runbook, cutover plan, day-1/week-1 support	—
Secure	10 policies; HECVAT, SOC 2, pen test, insurance readiness	Stream index
Legal (drafts)	Terms, privacy, community, subprocessors	MSA, order form, DPA, cookie, accessibility
Build	—	Engineering playbook, QA and accessibility test plans

Rule: formal documents are doc_page files under docs/<stream>/, linked from docs/index.html. Legal stays draft until attorney review.

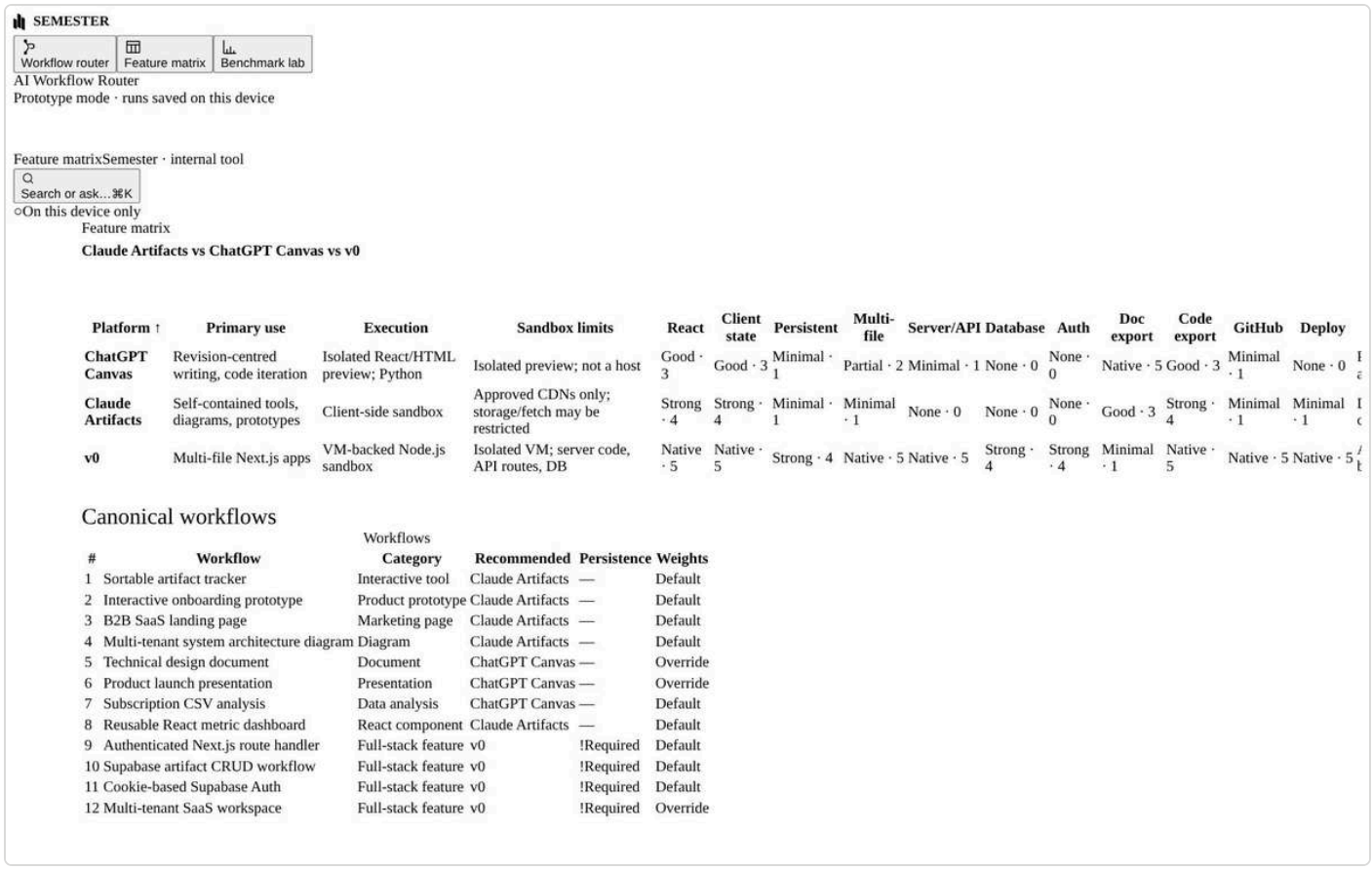
company documents

AI WORKFLOW ROUTER AND BENCHMARK LAB

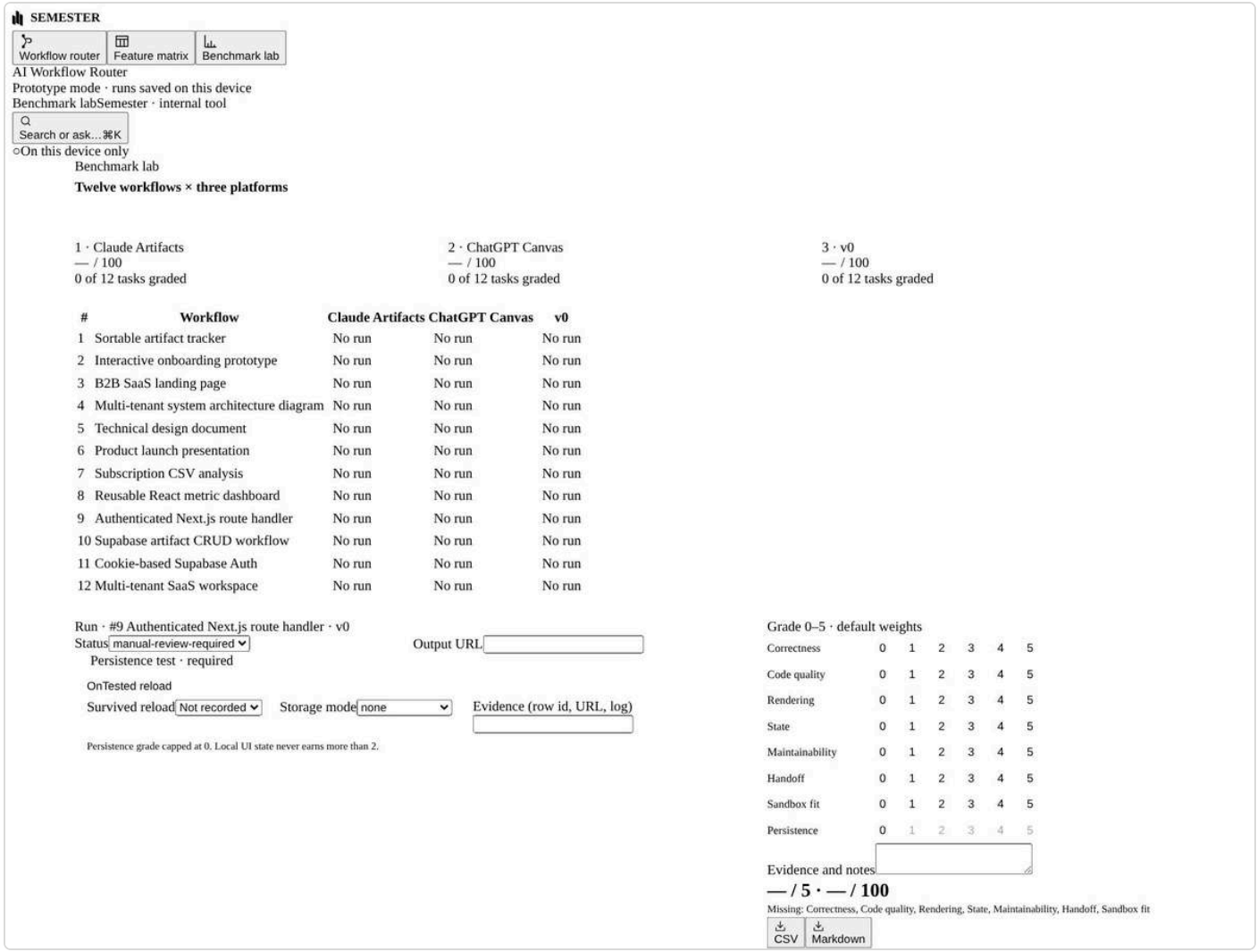
4 screens.



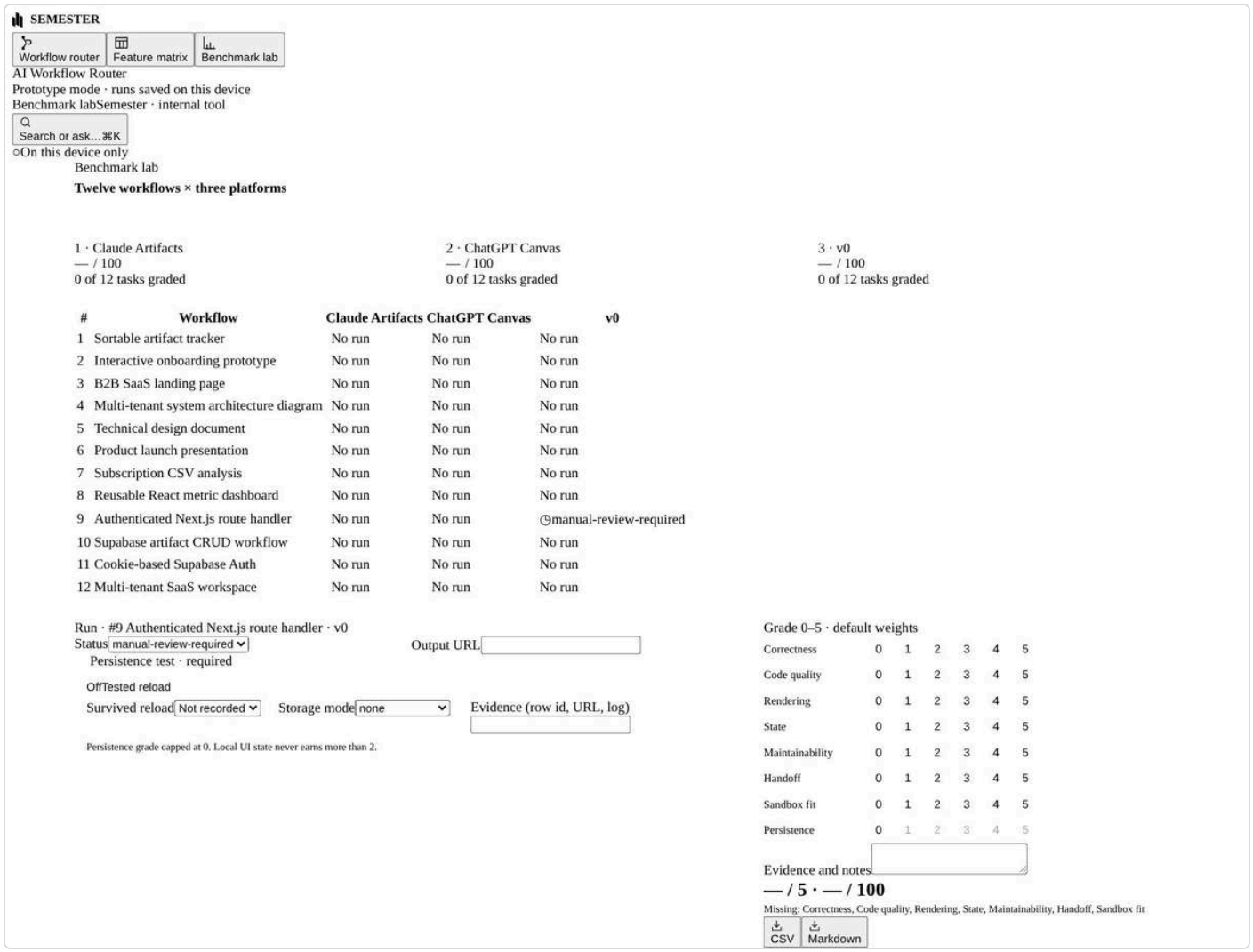
Opening view



Feature matrix



Benchmark lab



Benchmark lab › OnTested reload

Prototype notes

ui_kits/workflow-lab/README.md

AI Workflow Router & Benchmark Lab (prototype)

Interactive prototype of the app specified in the pasted brief, built from this design system. Router (6 inputs → primary/secondary, confidence, reasons, limits, escalation, handoff, prompts, GitHub plan, Supabase prompt, MD/JSON export), sortable 16-column feature matrix, the 12 canonical workflows, and a benchmark grid with per-run persistence test, 0–5 grading with the persistence ceiling, weighted scores, leaderboard and CSV/Markdown export.

Prototype mode: runs are saved to this browser only (localStorage) and labelled so. The production app — Next.js + Supabase with RLS — is in `handoff/workflow-router-app/`; `lab-engine.js` is a plain-JS port of its `lib/` and must be kept in step.

---

ui_kits/student/README.md

Student app UI kit

Desktop recreation of the Semester student workspace: dark Ink side rail, warm Parchment content, persistent system context bar with ⌘K palette.

Screens (click the rail): **Today** (index), **BIO 210** course workspace, **Assistant** (governed AI answer → action preview → confirm), **Family** (consent-driven sharing + access history), **Campus**.

Files: `StudentApp.jsx` shell + routing, `Today.jsx`, `Course.jsx`, `Assistant.jsx`, `Family.jsx`, `Campus.jsx`, `data.jsx` sample data. All primitives come from `window.SemesterDesignSystem_e9f3ba`.

Source: composed from the shared "unity" primitives in harrisonjrubin7-cmyk/semester (`app/src/components/unity/*`, `StartToday.tsx`, `desk/Sidebar.tsx`). The production app supports six navigations and three shells; this kit shows the desktop rail layout only.

---

ui_kits/staff/README.md

Staff workspaces UI kit

Faculty, Advisor, Registrar and Finance workspaces in one shell — switch roles from the rail. A dark scope bar names the role, tenant and identity on every screen.

- **Faculty** — gradebook with draft → release, regrade decisions. Source: `app/src/components/gradebook/InstructorBook.tsx`.
- **Advisor** — student-controlled meeting shares and authority scope. Source: `AdvisorMeeting.tsx`, `AdvisorSharedView.tsx`.
- **Registrar** — approval requests, capacity, record changes. Source: `enrollment/RegistrarDesk.tsx`, `institutional/OperationsStudio.tsx`.

- **Finance** — student account ledger and two-person requests. Source: `institutional/StudentAccounts.tsx`, `console/Approvals.tsx`.

`ui_kits/console/README.md`

Institution console UI kit

The enterprise control plane: dark environment bar (environment / scope / operator / MFA / session — the fields `CONTEXT_BAR` names in the source), Ink rail, Parchment content.

Screens: **Overview** (index), **Approvals** (two-person duty with ActionPreview + MFA confirm), **Integrations**, **Audit log**.

Source: harrisonjrubin7-cmyk/semester `app/src/components/console/*` (Approvals.tsx, ContextBar.tsx, Audit.tsx) and `institutional/IntegrationDashboard.tsx`. Copy for Approvals is lifted from Approvals.tsx.

`ui_kits/mobile/README.md`

Mobile UI kit

Two phones (Parchment and Ink grounds) with the five canonical destinations from the source — **Today, My Path, Search, Plan, Me** — on the labelled bottom tab bar. Each phone is independently clickable.

Files: `MobileScreens.jsx` (MToday, MPath, MSearch, MPlan, MMe), `MobileApp.jsx` (Phone frame + TabBar).

Note: per docs/BRAND-PLATFORM.md §3, no native iOS/Android binary exists; this is the responsive web app at phone width.

Prototype-to-codebase map

github.md

repo: harrisonjrubin7-cmyk/semester branch: main path: app/src

Last sync

date: 2026-10-05T03:38:34Z

Upstream since last sync

- `docs/design-system/` (FIGMA-MAPPING, README, BASELINE) and `workflow-lab/` (Next.js app) have landed.
- Not yet in repo: `.mcp.json`, the three new skills, `app/scripts/design-system-*`, baseline JSON.
- No `--source-*` / Semester-blue tokens in `app/src/styles` — this project's source palette remains a proposal.

Previous sync

date: 2026-10-05T00:49:21Z

Updated in this project

- Seven navigation areas and App directory from lib/navareas.ts + screens.tsx
- 14 new student screens (Calendar, Study, Write, Sheets, Decks, Mail, Connect, Settings, Privacy, Support, Community, Maps, Notifications, Me)
- TA, Student affairs, Security operations, Support queue; new External audiences kit

Sync history

- 2026-10-04T21:30:28Z — voice/naming, SourceLine, student + staff + mobile kits
- 2026-10-04T21:10:19Z — initial tokens, components, Student + Console kits

Screen map

Project screen	Repo files
ui_kits/student — Today	app/src/components/StartToday.tsx, components/unity/ObjectCard.tsx, unity/NextSteps.tsx
ui_kits/student — Course	components/unity/ContextBar.tsx, unity/Table.tsx, SourceBadge.tsx

Project screen	Repo files
ui_kits/student — Assistant	components/unity/States.tsx, unity/ActionPreview.tsx, lib/source.ts
ui_kits/student — Family	components/unity/Visibility.tsx, unity/DecisionTrail.tsx
ui_kits/student — Shell	components/desk/Sidebar.tsx, unity/SystemContextBar.tsx
ui_kits/console — Approvals	components/console/Approvals.tsx, console/Fields.tsx
ui_kits/console — Shell/Overview/Audit	components/console/ContextBar.tsx, console/Audit.tsx, institutional/IntegrationDashboard.tsx
ui_kits/student — Registration	app/src/components/enrollment/StudentRegistration.tsx
ui_kits/student — Grades	app/src/components/gradebook/StudentGrades.tsx
ui_kits/student — Advising	app/src/components/AdvisorMeeting.tsx
ui_kits/student — Degree, Billing, Career	docs/BRAND-PLATFORM.md, components/CareerEvidence.tsx (not read in full)
ui_kits/staff — Faculty	app/src/components/gradebook/InstructorBook.tsx
ui_kits/staff — Advisor	app/src/components/AdvisorMeeting.tsx, AdvisorSharedView.tsx
ui_kits/staff — Registrar, Finance	enrollment/RegistrarDesk.tsx, institutional/StudentAccounts.tsx
ui_kits/mobile	lib/look.ts (NAVS tabs), docs/BRAND-PLATFORM.md §1.8
ui_kits/student — App directory, areas	app/src/lib/navareas.ts, app/src/screens.tsx
ui_kits/student — Calendar, Study, Write, Sheets, Decks, Mail, Connect, Settings, Privacy, Support, Community, Maps, Notifications, Me	app/src/screens/* (names + areas only; layouts composed from shared primitives)
ui_kits/staff — TA, Student affairs	gradebook/InstructorBook.tsx, docs/BASIC-NEEDS-NAVIGATOR.md (not read)
ui_kits/console — Security, Support queue	console/BreakGlass.tsx, console/SupportQueue.tsx (not read in full)
ui_kits/external	docs/BRAND-PLATFORM.md §1.1, §3
handoff/claude-code-toolkit	lib/tokenexport.ts, docs/DESIGN-TOKENS.md, styles/hex.test.ts, design-tokens/semester.tokens.json, .claude/skills/
ui_kits/workflow-lab, handoff/workflow-router-app	uploads/list every artifact and thing for claud to genrate.md (spec), live site https://harrisonjrubin7-cmyk.github.io/semester/
ui_kits/architecture	uploads/arch-map.pdf, cqrs.pdf, sim.pdf, chatgpt.pdf (specs, not repo code)
tokens/*	app/src/styles/tokens.css, industry.css, unity.css, typefaces.css, lib/look.ts

What's next

NEXT-SESSION.md

Semester — next session brief

Start a new conversation in this project and paste: **"Read NEXT-SESSION.md and continue."**

Built (open ui_kits/master-control/index.html — every item is linked)

- Design system: tokens, 57 components, 68 cards, 7 templates (company, investor, institution, internal decks; student + console screens; pilot results report).
- Product: student app (64 screens, Tweaks, live Ask Semester), mobile, staff, console, external, investor portal.
- Maps of the whole: Master catalog (51 roles · 673 screens · 319 workflow steps (17 workflows) · 88 systems · 82 docs), Atlas, Role lens (22 roles), wireframe catalogue, forms/capabilities/interfaces (12 forms).
- Architecture: OS map, full stack & data (20 tables, flows, operations), read-model pipeline, lag simulator, AI gateway, phases.
- Company: business workspace, operating manual, operating pack, market research, compliance audit, company site, Vanderbilt one-pager, invitation email, campus map, 30 s demo video, company deck PPTX.
- Handoff: handoff/ (claude-code-toolkit, semester-core, workflow-router-app) — see handoff/README.md.

Done Oct 5 2026

- ui_kits/workflow-runner/: Semester OS map (21 layers with modules from uploads/Map Out entire Semester OS.pdf, 14-check control chain, journeys, 21 required shared workflows, 22-point completion definition) + runner for all 207 catalogued workflow steps (actor, hand-off, system, screen match, rendered step, audit log, auto-run), by-role view (49 roles), screen coverage and gap list. Linked from master-control. Batch C done: 7 added workflows (finance, campus, community & moderation, career/employer/alumni, family consent, developer platform, analytics & outcomes; 112 steps, labelled "Added · not in catalog yet") so every one of the 21 layers now has a runnable workflow. Next: fold the added workflows into catalog-data.js and write docs/master/*.md.
- Design System tab: 10 new cards (arch-*, tooling-semester-core/platform/workflow-router, company-documents, maps-of-the-whole, iconography-rules).
- docs/master/: all 13 catalogs + README, generated from catalog/runner/OS-map data (84 step screen gaps, 13 roles with no workflow, 419 screens no workflow uses — see SEMESTER_GAP_REGISTER.md).
- Templates from the linked Semester Platform project: templates/platform-shell, institution-os, command-center, role-onboarding (RoleOnboarding role prop → 13 role DCs), operating-map, finish-line, vision-deck. This design system wins on conflicts.
- handoff/execute/: 00-INDEX.md master prompt + 12 stream prompts (00 setup … 11 company site/grow), branch semester/, PR + report per stream.
- DONE: docs/grow/GTM Plan.html (+ index).

- DONE: full public site in ui_kits/company-site (10 pages, hash routes #/product … #/legal, responsive, forms, legal + accessibility statement).
- DONE: docs/legal MSA, Order Form, DPA (drafts, attorney review). Still: cookie notice, accessibility statement as a standalone doc.
- DONE: docs/run/Finance Model.html (setup lean $4,873 / base $12,430 / full $19,910; run-rate $565/mo).
- DONE: docs/run/Hiring Plan.html (10-role plan, 6 JDs, contractor briefs).
- DONE: docs/build/ Engineering Playbook, QA Test Plan, Accessibility Test Plan (+ index).
- DONE: docs/operate/ Operations Handbook (SLOs, error budget, on-call, alerts, monitoring, cadence) + Runbooks (R1–R8, macros).
- DONE: docs/legal Cookie Notice + Accessibility Statement; docs/secure/index.html.
- DONE: ui_kits/role-pages/ — one page per catalogued role (49 in catalog-data; brief said 51), linked from master control; workflow buttons open the runner at the step.
- DONE: ui_kits/gap-screens/ — the 84 step screen gaps added to the catalog (CATALOG.addedScreens; 673 screens total) and rendered in the shell. Regenerate docs/master/* to drop them from the gap register.
- DONE: batch A baseline — ui_kits/every-screen/ renders all 673 catalogued screens (archetype per name: dashboard/queue/list/builder/settings/detail/marketing) in six states; ◆ designed in a kit, ◖ from a workflow gap, ○ archetype only.
- NEXT: replace archetype screens with bespoke designs, highest-traffic first (B student, E registrar, K console); regenerate docs/master/*. EVERY ROLE pages (51) and batch A missing screens.
- Source project (done): /projects/7dd6738a-405e-41a6-9c02-ac359bb30017 (Semester Platform, Institution OS, Company Command Center, 14 role onboardings, Master Operating Map, Finish Line, Vision Deck).

Founder's latest ask (top priority)

"Every role in Semester. Finish, generate and produce everything I need to run, grow, build, operate, maintain, secure and launch Semester." Batch N is the master plan. Do it one stream per session, in this order, and link every output from master-control: 1. LAUNCH: COMPLETE. docs/launch/ has index, Launch Runbook, Cutover Plan and Support Plan (day-1/week-1). Next is BUILD (stream 3). SECURE has no docs/secure/index.html yet. Original list: launch runbook, go/no-go checklist, cutover plan, comms plan, day-1/week-1 support plan, rollback. 2. SECURE: the policy set at docs/secure/Security Policies.html is DONE. docs/secure/Security Readiness.html (HECVAT, SOC 2, pen test, insurance) is DONE too, so SECURE is complete. Next is BUILD. Original list: HECVAT Full answers draft, security policy set (access control, encryption, vendor management, incident response, BCP/DR, data retention, acceptable use), SOC 2 readiness tracker, pen-test scope, insurance checklist. 3. BUILD: engineering playbook (repo layout, branching, CI/CD on Vercel + Supabase, RLS test plan, environment matrix, release process), QA test plan per role, accessibility test plan (VPAT/ACR prep). 4. OPERATE / MAINTAIN: site operations console (batch E), on-call and SLOs, monitoring and alerts, SIS sync runbooks, backup and restore drills, dependency/patch cadence, support macros. 5. RUN: company operating cadence (weekly metrics, monthly close, quarterly planning), legal stack (LLC, contracts, DPA template, MSA, order form, privacy policy, terms), finance model (pre-revenue budget within the $5k–$20k setup range), hiring plan per role, a job description for each. 6. GROW: GTM plan (ICP, pipeline, pricing, case study from the pilot), the full public site (batch F), content calendar, partner program, fundraising data room index. 7. EVERY ROLE: one page per catalogued role (51), covering purpose, screens, permissions, workflows, KPIs and day-in-the-life. Render any missing screens (batch A). Build formal docs as doc_page files under docs//, and give each stream an index page.

Also done (Oct 2026)

- docs/guides: Architecture, Developer and User guides.
- handoff/semester-platform: migrations 010–040 (core, community, market, ops, all with RLS), validation, 2 API routes, RLS and unit tests. None of it has been run yet.
- Still to do: an admin MANUAL (operations manual for institution admins and support) and TUTORIALS (step-by-step, for students, staff and new developers).
- 0. Device and media capabilities, each permission-gated with a clear ask, a fallback, and a revoke path in Privacy settings:
- Camera: scan textbook ISBNs for listings, photograph items, ID check for campus verification (handled by the institution IdP where possible), video office hours.
- Microphone: voice notes, dictation in Write, voice questions to Ask Semester (transcribed on device where possible; audio never stored by default).
- Location: campus map, safe meet-up spots, event check-in, shuttle. Only while the app is in use, precise or approximate as the user chooses, never sold, and off by default.
- Image and video generation: for study visuals, club flyers and event promos, through the AI gateway with a content-safety filter, labelled AI-generated, with no real-person likeness and no student records.
- Notifications (push/email/SMS per channel), offline mode, PWA install, biometric sign-in on the device.
- Multi-user and multi-device: account switcher (student who is also a TA, staff in two roles), sessions list with remote sign-out, handoff between phone and desktop, shared family view only with student consent, delegated access with expiry.
- More personalization (expands H): themes (13 grounds × 11 accents from the app), density, text size, calm mode, dyslexia font, reduced motion, home layout builder, language, time format.
- Build: a Capabilities & Permissions screen in the student app, a device settings page, permission-prompt patterns as design-system cards, and an RLS/migration for sessions and delegations in handoff/semester-platform.
- LEGAL DRAFTS DONE: docs/legal has Terms, Privacy, Community Guidelines and Subprocessors, all marked as needing attorney review. docs/index.html is the library of every document. Still to do: MSA, order form, DPA template, cookie notice, accessibility statement.
- MANUAL + TUTORIALS (requested again): an admin and operations manual, plus step-by-step tutorials for students, staff and developers. Both go under docs/guides.

Founder's earlier ask

"Every screen rendered — capability, workflow, user role, business operations, site operations, and the actual site." Work in batches, one area per session. Each batch ends by linking new files from ui_kits/master-control/index.html: - A. Render every catalogued screen still missing, role by role (catalog = 51 roles · 673 screens). Use shared shells and components; group roughly 10–15 screens per file, one file per role. - B. Capability map rendered as screens: each capability → the screens and roles that use it. - C. DONE: ui_kits/workflow-runner/ — all 319 steps, clickable end to end, per role. - D. Business operations console (finance, sales/revops, CS, legal/compliance, hiring). No student records. - E. Site operations console (deploys, uptime, incidents, SIS sync, AI gateway, support queue, audit log). - F. The actual public site: full multi-

page marketing site (home, product, institutions, students, security/trust, pricing, about, contact, legal). Build on the existing company site. - G. DONE (Oct 2026): 9 logo SVGs in assets/ plus the components/LogoSystem/preview.html card. Outstanding: outline the wordmark text to paths. Original brief: More logos: logo variants (horizontal, stacked, mark-only, mono, reversed, favicon/app icon, co-brand lockup with partner institution). Add them to assets/ and give them a brand card. - H. Personalization capabilities: add them to the capability catalog and render them as screens. Covers learner profile and goals, adaptive study plan, pacing and reminders, content recommendations, accessibility and display preferences, notification preferences, AI tutor tone and level, dashboard layout, and institution-level theming. Every item has privacy controls (consent, view and reset my data, opt-out). AI never sees student records. - I. Update pass: audit catalog counts, master-control links, the readme and SKILL.md against what's built; fix anything stale or broken. - J. PARTLY DONE: ui_kits/ops/index.html has 12 consoles and is linked from master-control. Social and Finance are deep; the other 10 are one queue each and need more tabs. - K. Documents: every formal document (policies, FERPA/SOC 2/accessibility compliance docs, audit workflows, test plans, runbooks, DPA, incident response, AI use policy, data retention). Build them as doc_page documents in docs/, with an index. - L. PARTLY DONE: docs/brief/ has index, One-pager and FAQ. Role sheets are done too. The overview deck (templates/overview-deck/OverviewDeck.dc.html) is done too, so L is complete. Add a real screenshot on slide 4. Ops consoles now carry a "sample data" banner. Original brief: Product brief package to hand out: a one-pager, a 10-slide overview deck, an FAQ, and a role-by-role "what Semester does for you" sheet. Print/PDF ready, with one index page that links them all. - M. PARTLY DONE: student app at ui_kits/community. Staff side lives in the ops T&S console (health, marketplace ops, live drops, institution settings). Mentorship, clubs and leader tools are now built too. M is done apart from deeper polish. Started: ui_kits/community/index.html. Expand it to cover: - Social: campus-verified profiles controlled by the student, with visibility per field; follows and connections; a course and club feed; study groups; DMs with safety tools; events and RSVPs; mentorship matching. - Community: clubs and orgs, course communities, Q&A with endorsed answers, reputation that carries no academic weight, ambassador and peer-leader roles. - Live marketplace: textbooks, gear, housing and sublets, tutoring and services, and event tickets. Listings are campus-verified, with in-app offers and chat, safe meet-up spots, and live drops and auctions. Disputes and reports go to the moderator workspace. - Guardrails: no sale of graded work or exam material (academic integrity filter), the institution controls whether each feature is on, minors and age rules respected, moderation queue, AI pre-screen only on public posts, no student records in social or AI. Payments go through a processor; Semester never holds funds without a license. - Staff side: community health, moderation and marketplace ops tabs inside ui_kits/ops (T&S console). - (orig) J. Operations consoles, each fully built, interactive and rendered (one file each, shared lnk console shell, linked from master-control): - Social media control: unified inbox, compose/schedule calendar, approval queue, brand-safety and crisis mode, per-channel analytics, institution-partner co-posts. - Finance: ARR/MRR, invoices and AR aging, contracts and renewals, usage-based billing per institution, AI gateway cost, payroll summary, close checklist, audit export. - Also: marketing/campaigns, sales/RevOps pipeline, customer success health, support desk, trust and safety/moderation, legal/compliance (FERPA, DPAs, subprocessors), people/HR, IT/security (SOC 2 evidence), data/analytics, partnerships. - Each console needs real-feeling data, working filters and tabs, row detail drawers, and role permissions. Business data only; no student PII. Then continue with the list below.

Do next, in order

1. Export investor, institution and internal decks to PPTX (gen_pptx, same as company deck).
2. Deepen P1 workflow screens from the catalog gap register: registration readiness read model, support SLA/incident, SIS reconciliation.
3. Course Studio (faculty authoring) and moderator workspace screens — currently catalogued, not built.

4. Write the highest-value docs from the catalog's "Not started" list (AI_GOVERNANCE_POLICY, DATA_INVENTORY, INCIDENT_RESPONSE, PILOT_DELIVERY_FACTORY) as doc-page files.
5. Run handoff/claude-code-toolkit/EXECUTE.md in the repo (needs Claude Code, not this project).

Executable hand-off (Oct 2026)

- handoff/BUILD.md is the single entry point: rules, preflight, app target (Next.js; create web/ if app is Vite), install, stream order + gates, migration order 010–080, verify, human-only steps, definition of go. Scripts: handoff/preflight.sh, install.sh (no-overwrite, strips .txt), verify.sh (db reset, tsc, lint, unit, RLS/security/command, design-system, build, gitleaks). handoff/manifest.json. Seed (synthetic users with auth.identities), tests/helpers.ts, .env.test, .env.example added.
- Launch kit: sponsor weekly update email, incident notice email, staff training flier.

Security deepening (Oct 2026)

- Reviewed migrations 010–040: 11 findings (F2 critical: ops.staff self-insert as ceo; F5 high: profile institution_id change; F1, F4 high). Fixed in handoff/semester-platform/supabase/migrations/050_security_hardening.sql.txt, proven by tests/security.test.ts.txt; docs/secure/Platform Security Review.html; stream 01 updated.
- 060_command_functions: all community/market writes via audited api.* functions (validation, rate limits, integrity filter, campus checks), direct inserts revoked, listing-photo storage policies; tests/commands.test.ts.
- AI gateway hardened: content scan blocks records typed into prompts (lib/ai/redact.ts), route app/api/ai/route.ts, 070_ai_gateway (tenant policy, course rules, usage log without prompt content), 20-case red-team suite.
- 080: approved-source AI context, private storage buckets, audited delegations; security headers middleware; CI security job (gitleaks, audit, guards, security tests).
- templates/privacy-center: student privacy & data screen (AI by data type with institution prop, sharing revoke, sessions sign-out, access history, export, deletion hand-off).
- 7 governance components added (components/governance/). Launch kit imported (ui_kits/launch-kit + 5 templates).

Strategy (Oct 2026)

- docs/strategy/ (12 docs + README): competitive completion audit (20 areas), 19-dim competitive scorecard (~1.1/5), coherence test as release gate, implementation factory, mobile, AI differentiation + deterministic policy engine, trust advantage, 36-month plan. Competitive view added to ui_kits/platform-maturity.

Business, design & data systems (Oct 2026)

- docs/business/ (17 + 5 subsystem docs: finance, revenue, trust, customer, people; SoR map, risk register, cadence, KPI tree, backlog, top 50). Price book updated: Student Premium $8.99/mo or $69/yr, AI overage $30/1k requests, implementation $35k–$150k, marketplace 12%.

- docs/system/ (design 8, algorithms 7, workflows 2, data 4 + README with next 25). OPEN DECISION: PDF proposes indigo primary + emerald/amber/rose; system keeps Semester blue + stone hues until founder decides.

Platform maturity (Oct 2026)

- From uploads/what else is needed...pdf + Apple vs Google vs Microsoft...pdf: docs/platform/ (16 + 2 docs), ui_kits/platform-maturity/ (3 tiers, 12 pillars, 24-dim scorecard at ~1.2/5 design stage, API domains + contract, graph node fields + authority categories, 36-month roadmap, 7 stages, deployment ladder, moats/risks, next 50 actions), stream handoff/execute/13-platform-maturity.md.

Product principle (Oct 2026)

- Nobody has to leave Semester: ui_kits/one-place/ maps 45 jobs (students, staff, institutions, company) to modules; regulated or institution-owned systems (bank, processor, payroll, IdP, SIS during transition, GitHub/Vercel) are embedded, not replaced. Stream handoff/execute/12-one-place.md.

Rules to keep

- Nothing is "verified" without repo code, tests and release evidence.
- Company roles (investor, CS, revops) never see student records.
- Status, not certifications we don't hold. Cite every external number.

Founder facts

LLC (home state), solo founder, bootstrapping, pre-seed no revenue. Paid Vanderbilt pilot signed: real student records, DPA/FERPA addendum, HECVAT Full, SAML SSO, VPAT/ACR, cyber + E&O insurance required (not held); dates not set. Hosting Vercel + Supabase; AI never sees student records. Setup budget $5k–$20k. Vanderbilt name cleared for public use.

SEMESTER

Document library

Every formal Semester document. Each one prints or saves as a PDF. Legal documents are drafts until an attorney reviews them.

Product brief

Brief package
One-pager, FAQ, role sheets, deck

Guides

Architecture guide
For architects

Developer guide
For engineers

User guide
For every role

Operating manual
How to run the company

Company & system

docs/business
Company OS: 15 functions, 5 systems, KPI tree, cadence, backlog

docs/system
Brand, UI grammar, components, algorithms, workflows, data

Strategy

docs/strategy
Competitive audit, scorecard, moats, implementation factory, AI, trust, 36-month plan

Platform

docs/platform
Maturity model, 12 pillar standards, scorecard, APIs, 36-month roadmap

Platform maturity program
Interactive

Master catalogs

docs/master
13 generated catalogs: map, roles, screens, workflows, systems, gaps

OS map & workflow runner
Interactive version

Build

Engineering playbook
Repo to release

QA test plan
By role

Accessibility test plan
VPAT / ACR prep

Launch

Pilot launch runbook
Gates, timeline, rollback

Production cutover plan
Staging to production

Day-1 / week-1 support
Tiers, severities, review

Launch index
All launch documents

Run

Finance model
$5k–$20k setup, run-rate, runway

Hiring plan
Plan + 6 job descriptions

Run index
All run documents

Grow

GTM plan and price book
Offers, prices, pipeline

Grow index
All grow documents

Operate

Operations handbook
SLOs, on-call, alerts

Runbooks
SSO, SIS sync, restore, exposure, AI

Security

Security policy set
10 policies

Security readiness

Platform security review
11 RLS findings fixed

HECVAT, SOC 2, pen test, insurance

Secure index
All security documents

Legal (drafts)

Terms of Service
Users

Privacy Policy
FERPA-aligned

Community guidelines
Plain-language rules

Subprocessors
Vendor list

Cookie notice
Storage, no ad tracking

Accessibility statement
WCAG 2.2 AA status

Master Services Agreement
Institutions

Order form
Pilot + conversion

Data Protection Addendum
FERPA school-official terms

APPENDIX

Source code

The stylesheets, types, component sources and every engineering package, in full.

Token and component stylesheets

The generated token stylesheet and the component stylesheet, in full.

tokens.css · 251 lines

```
/* The Main Semester design system — generated from tokens.json */
:root, [data-theme="light"] {
  --ink-void: #040507;
  --ink-bg: #090a0e;
  --ink-panel: #12141a;
  --ink-hero: #191c23;
  --ink-raise: #22262f;
  --ink-fg: #eceef2;
  --parchment-void: #e8e4dc;
  --parchment-bg: #f4f1ea;
  --parchment-panel: #fbf9f4;
  --parchment-hero: #ffffff;
  --parchment-raise: #ffffff;
  --parchment-fg: #1b1a17;
  --graphite-bg: #16181c;
  --graphite-panel: #1f2229;
  --graphite-raise: #333843;
  --sterling: #d4d9e2;
  --sterling-bright: #f6f8fb;
  --sterling-deep: #9ca3b2;
  --sterling-shade: #4c5561;
  --brass: #d8c79a;
  --brass-bright: #f2e7c8;
  --brass-deep: #b09f76;
  --brass-shade: #695a33;
  --blue-100: #eef6ff;
  --blue-200: #d6ebff;
  --blue-300: #b5d9fd;
  --blue-400: #94bce3;
  --blue-500: #749dc4;
  --blue-600: #597ea3;
  --blue-700: #416180;
  --blue-800: #2c455d;
  --blue-900: #1d2d3d;
  --neutral-100: #f5f5f8;
  --neutral-200: #e7e7ea;
  --neutral-300: #d4d4d7;
  --neutral-400: #b7b7ba;
  --neutral-500: #98989b;
  --neutral-600: #7a7a7d;
  --neutral-700: #5d5d60;
  --neutral-800: #424244;
  --neutral-900: #2b2b2d;
  --official-ink: #2c455d;
  --official-tint: #b5d9fd;
  --connected-ink: #2d5c61;
  --connected-tint: #9fcbcd;
  --personal-ink: #4a4b4f;
  --personal-tint: #b9b9bd;
  --ai-ink: #544a7f;
  --ai-tint: #bcb2df;
  --pending-ink: #7a5310;
  --pending-tint: #e2c27a;
  --risk-ink: #a3301f;
  --risk-tint: #f0907a;
  --success-ink: #3e6141;
  --success-tint: #a9c9a3;
  --surface-sunken: var(--parchment-void);
  --surface-canvas: var(--parchment-bg);
  --surface-base: var(--parchment-panel);
  --surface-quiet: #f1ede4;
```

```
  --surface-raised: var(--parchment-hero);
  --surface-overlay: var(--parchment-raise);
  --text-primary: var(--parchment-fg);
  --text-secondary: rgba(27,26,23,.68);
  --text-tertiary: rgba(27,26,23,.52);
  --text-disabled: rgba(27,26,23,.52);
  --text-on-action: #ffffff;
  --text-link: var(--blue-700);
  --border-subtle: rgba(27,26,23,.09);
  --border-default: rgba(27,26,23,.16);
  --border-strong: rgba(27,26,23,.34);
  --action-primary: var(--blue-700);
  --action-primary-hover: var(--blue-800);
  --action-primary-press: var(--blue-900);
  --action-secondary: rgba(65,97,128,.08);
  --action-secondary-hover: rgba(65,97,128,.14);
  --focus-color: var(--blue-600);
  --accent-editorial: var(--brass-shade);
  --source-official: var(--official-ink);
  --source-official-line: rgba(44,69,93,.45);
  --source-official-wash: rgba(44,69,93,.08);
  --source-connected: var(--connected-ink);
  --source-connected-line: rgba(45,92,97,.45);
  --source-connected-wash: rgba(45,92,97,.08);
  --source-personal: var(--personal-ink);
  --source-personal-line: rgba(74,75,79,.40);
  --source-personal-wash: rgba(74,75,79,.06);
  --source-ai: var(--ai-ink);
  --source-ai-line: rgba(84,74,127,.45);
  --source-ai-wash: rgba(84,74,127,.08);
  --status-pending: var(--pending-ink);
  --status-pending-line: rgba(160,112,24,.50);
  --status-pending-wash: rgba(196,146,48,.12);
  --status-danger: var(--risk-ink);
  --status-danger-line: rgba(163,48,31,.45);
  --status-danger-wash: rgba(163,48,31,.08);
  --status-success: var(--success-ink);
  --status-success-line: rgba(62,97,65,.45);
  --status-success-wash: rgba(62,97,65,.08);
  --status-neutral: var(--text-secondary);
  --chrome-surface: var(--ink-bg);
  --chrome-surface-raised: var(--ink-panel);
  --chrome-text: var(--ink-fg);
  --chrome-text-secondary: rgba(236,238,242,.64);
  --chrome-line: rgba(236,238,242,.10);
  --chrome-active: rgba(148,188,227,.14);
  --scrim: rgba(4,5,7,.5);
  --brand-canvas: var(--surface-canvas);
  --brand-surface: var(--surface-base);
  --brand-ink: var(--text-primary);
  --brand-muted: var(--text-secondary);
  --brand-accent: var(--action-primary);
  --brand-focus: var(--focus-color);
  --lift-1: 0 1px 2px rgba(0,0,0,.4);
  --lift-2: 0 1px 2px rgba(0,0,0,.45),0 6px 18px rgba(0,0,0,.35);
  --lift-3: 0 1px 3px rgba(0,0,0,.5),0 12px 36px rgba(0,0,0,.5);
  --shadow-sm: 0 1px 2px rgba(43,43,45,.14);
  --shadow-md: 0 3px 10px rgba(43,43,45,.16);
  --shadow-lg: 0 12px 32px rgba(43,43,45,.22);
}
[data-theme="dark"] {
  --surface-sunken: var(--ink-void);
  --surface-canvas: var(--ink-bg);
  --surface-base: var(--ink-panel);
  --surface-quiet: #161820;
  --surface-raised: var(--ink-hero);
  --surface-overlay: var(--ink-raise);
  --text-primary: var(--ink-fg);
  --text-secondary: rgba(236,238,242,.64);
  --text-tertiary: rgba(236,238,242,.42);
  --text-disabled: rgba(236,238,242,.42);
  --text-on-action: #08090c;
  --text-link: var(--blue-300);
  --border-subtle: rgba(236,238,242,.08);
  --border-default: rgba(236,238,242,.14);
  --border-strong: rgba(236,238,242,.30);
  --action-primary: var(--blue-400);
```

```
  --action-primary-hover: var(--blue-300);
  --action-primary-press: var(--blue-500);
  --action-secondary: rgba(148,188,227,.10);
  --action-secondary-hover: rgba(148,188,227,.18);
  --focus-color: var(--blue-400);
  --accent-editorial: var(--brass);
  --source-official: var(--official-tint);
  --source-official-line: rgba(181,217,253,.40);
  --source-official-wash: rgba(181,217,253,.08);
  --source-connected: var(--connected-tint);
  --source-connected-line: rgba(159,203,205,.40);
  --source-connected-wash: rgba(159,203,205,.08);
  --source-personal: var(--personal-tint);
  --source-personal-line: rgba(185,185,189,.34);
  --source-personal-wash: rgba(185,185,189,.06);
  --source-ai: var(--ai-tint);
  --source-ai-line: rgba(188,178,223,.42);
  --source-ai-wash: rgba(188,178,223,.09);
  --status-pending: var(--pending-tint);
  --status-pending-line: rgba(226,194,122,.45);
  --status-pending-wash: rgba(226,194,122,.09);
  --status-danger: var(--risk-tint);
  --status-danger-line: rgba(240,144,122,.45);
  --status-danger-wash: rgba(240,144,122,.09);
  --status-success: var(--success-tint);
  --status-success-line: rgba(169,201,163,.42);
  --status-success-wash: rgba(169,201,163,.08);
  --status-neutral: var(--text-secondary);
  --chrome-surface: var(--ink-bg);
  --chrome-surface-raised: var(--ink-panel);
  --chrome-text: var(--ink-fg);
  --brand-canvas: var(--surface-canvas);
  --brand-surface: var(--surface-base);
  --brand-ink: var(--text-primary);
  --brand-muted: var(--text-secondary);
  --brand-accent: var(--action-primary);
  --brand-focus: var(--focus-color);
}
:root {
  --space-1: 8px;
  --space-2: 16px;
  --space-3: 24px;
  --space-4: 32px;
  --space-5: 40px;
  --space-6: 48px;
  --space-8: 64px;
  --space-10: 80px;
  --space-12: 96px;
  --page-pad: 18px;
  --chrome-pad: 18px;
  --r-sm: 3px;
  --r-md: 6px;
  --r-lg: 10px;
  --shape-pill: 999px;
  --rail-w: 248px;
  --rail-w-compact: 72px;
  --layout-measure: 100%;
  --layout-operational: 1180px;
  --target-min: 24px;
  --target-compact: 32px;
  --target-icon: 40px;
  --target-primary: 44px;
  --focus-ring-width: 2px;
  --focus-ring-offset: 2px;
  --blur-chrome: 14px;
  --layer-base: 0;
  --layer-raised: 1;
  --layer-sticky: 20;
  --layer-chrome: 21;
  --layer-overlay: 80;
  --layer-menu: 90;
  --layer-skip: 100;
  --layer-curtain: 1500;
  --leading-none: 1;
  --leading-display-xl: 1.08;
  --leading-display-lg: 1.1;
  --leading-display: 1.15;
```

```
  --leading-display-sm: 1.2;
  --leading-display-xs: 1.25;
  --leading-tight: 1.3;
  --leading-tight-plus: 1.35;
  --leading-normal: 1.45;
  --leading-relaxed: 1.5;
  --leading-relaxed-plus: 1.55;
  --leading-loose: 1.6;
  --tracking-heading: -0.015em;
  --tracking-kicker: 0.08em;
  --tracking-caps: 0.1em;
  --tracking-wordmark: 0.22em;
  --font-display: 'Cinzel','Barlow Condensed',Georgia,serif;
  --font-heading: 'Barlow Condensed',system-ui,sans-serif;
  --font-body: 'Barlow',system-ui,sans-serif;
  --font-mono: ui-monospace,'SF Mono',Menlo,Consolas,monospace;
}
.role-editorial { font-family: var(--font-display); font-size: 42px; line-height: 1.1; font-weight: 500; }
.role-display { font-family: var(--font-heading); font-size: 26px; line-height: 1.15; font-weight: 600; letter-spacing:
-0.015em; }
.role-page { font-family: var(--font-heading); font-size: 24px; line-height: 1.2; font-weight: 600; letter-spacing: -0.015em; }
.role-section { font-family: var(--font-heading); font-size: 20px; line-height: 1.2; font-weight: 600; letter-spacing:
-0.015em; }
.role-card-title { font-family: var(--font-heading); font-size: 18px; line-height: 1.25; font-weight: 600; letter-spacing:
-0.015em; }
.role-body { font-family: var(--font-body); font-size: 16px; line-height: 1.55; font-weight: 400; }
.role-control { font-family: var(--font-heading); font-size: 14px; line-height: 1.2; font-weight: 600; letter-spacing:
-0.015em; }
.role-compact { font-family: var(--font-body); font-size: 12px; line-height: 1.45; font-weight: 400; }
.role-label { font-family: var(--font-body); font-size: 11px; line-height: 1.3; font-weight: 500; }
.role-caption { font-family: var(--font-body); font-size: 11px; line-height: 1.45; font-weight: 400; }
.role-kicker { font-family: var(--font-body); font-size: 10.5px; line-height: 1.3; font-weight: 500; letter-spacing: 0.08em; }
.role-numeric { font-family: var(--font-heading); font-size: 22px; line-height: 1.15; font-weight: 600; letter-spacing:
-0.015em; }
@font-face { font-family: "Barlow"; font-style: normal; font-weight: 400; font-display: swap; src: url("fonts/Barlow-400-
latin.woff2") format("woff2"); }
@font-face { font-family: "Barlow"; font-style: normal; font-weight: 500; font-display: swap; src: url("fonts/Barlow-500-
latin.woff2") format("woff2"); }
@font-face { font-family: "Barlow"; font-style: normal; font-weight: 700; font-display: swap; src: url("fonts/Barlow-700-
latin.woff2") format("woff2"); }
@font-face { font-family: "Barlow Condensed"; font-style: normal; font-weight: 400; font-display: swap; src:
url("fonts/BarlowCondensed-400-latin.woff2") format("woff2"); }
@font-face { font-family: "Barlow Condensed"; font-style: normal; font-weight: 600; font-display: swap; src:
url("fonts/BarlowCondensed-600-latin.woff2") format("woff2"); }
@font-face { font-family: "Cinzel"; font-style: normal; font-weight: 400 600; font-display: swap; src: url("fonts/Cinzel-
latin.woff2") format("woff2"); }
```

components/bundle.css · 388 lines

```
/* Semester — component and base styles. Token values live in tokens.json (compiled to tokens.css); this sheet keeps what
tokens.json cannot hold: computed scales, aliases, motion, media overrides, base and component rules. */

/* ── from tokens/colors.css ── */
/* Semester colour.
 * Primitives come from the app's measured grounds and accents (app/src/lib/look.ts):
 *   Ink (default dark) and Parchment (default light) ramps, Sterling/Brass metals, and the Industry blue ramp
 *   which this system names "Semester blue" for interactive focus and trust.
 * Source-status hues are desaturated stones drawn toward the brief's semantics (official navy, connected teal,
 * personal graphite, AI violet, pending amber, risk red, success green). Every state also carries a word + glyph.
 * Default scope = warm off-white content (Parchment). Chrome (rail, tab bar, console header) is always Ink.
 * [data-theme="dark"] flips content to Ink. */
:root{
  --chrome-metal:linear-gradient(180deg,#f6f8fb 0%,#d4d9e2 45%,#9ca3b2 100%);/* @kind color */
  }

@media (prefers-contrast:more){
  :root{--text-secondary:rgba(27,26,23,.9);--text-tertiary:rgba(27,26,23,.76);--border-default:rgba(27,26,23,.34);--border-
subtle:rgba(27,26,23,.2)}
  [data-theme="dark"]{--text-secondary:rgba(236,238,242,.9);--text-tertiary:rgba(236,238,242,.76);--border-
default:rgba(236,238,242,.34);--border-subtle:rgba(236,238,242,.2)}
}

/* ── from tokens/typography.css ── */
/* Semester type. Barlow for product reading, Barlow Condensed for headings/labels/controls,
 * Cinzel for editorial display moments only, system mono for identifiers and timestamps.
```

```
 * Sizes are the app's scale (app/src/styles/app.css) and respond to --text-scale. */
:root{
  --font-heading-weight:600;
  --font-editorial:var(--font-display);
  --font-product:var(--font-body);
  --type-3xs:calc(9px * var(--text-scale,1));
  --type-2xs:calc(10px * var(--text-scale,1));
  --type-2xs-plus:calc(10.5px * var(--text-scale,1));
  --type-xs:calc(11px * var(--text-scale,1));
  --type-xs-plus:calc(11.5px * var(--text-scale,1));
  --type-sm:calc(12px * var(--text-scale,1));
  --type-sm-plus:calc(12.5px * var(--text-scale,1));
  --type-control:calc(14px * var(--text-scale,1));
  --type-base:calc(16px * var(--text-scale,1));
  --type-base-plus:calc(16.5px * var(--text-scale,1));
  --type-md:calc(17px * var(--text-scale,1));
  --type-lg:calc(18px * var(--text-scale,1));
  --type-display-xs:calc(19px * var(--text-scale,1));
  --type-display-sm:calc(20px * var(--text-scale,1));
  --type-display:calc(22px * var(--text-scale,1));
  --type-display-lg:calc(24px * var(--text-scale,1));
  --type-xl:calc(26px * var(--text-scale,1));
  --type-2xl:calc(28px * var(--text-scale,1));
  --type-3xl:calc(32px * var(--text-scale,1));
  --type-4xl:calc(42px * var(--text-scale,1));
  --type-5xl:calc(56px * var(--text-scale,1));
  --type-role-editorial:var(--type-4xl);
  --type-role-display:var(--type-xl);
  --type-role-page:var(--type-display-lg);
  --type-role-section:var(--type-display-sm);
  --type-role-card-title:var(--type-lg);
  --type-role-body:var(--type-base);
  --type-role-compact:var(--type-sm);
  --type-role-label:var(--type-xs);
  --type-role-kicker:var(--type-2xs-plus);
  --type-role-numeric:var(--type-display);
  --type-role-control:var(--type-control);
  --type-role-caption:var(--type-xs);
  --layout-reading:66ch;/* @kind other */
}

/* ── from tokens/spacing.css ── */
/* Spacing. --sp-1…7 are the app's density-aware 2px steps for compact UI;
 * --space-* is the 8px macro grid for layout. --density: 1 comfortable, .86 snug, .74 tight. */
:root{
  --sp-1:calc(2px * var(--density,1));/* @kind other */--sp-2:calc(4px * var(--density,1));--sp-3:calc(6px * var(--
density,1));--sp-4:calc(8px * var(--density,1));
  --sp-5:calc(10px * var(--density,1));--sp-6:calc(12px * var(--density,1));--sp-7:calc(16px * var(--density,1));
  --layout-gutter:var(--page-pad);
  --control-height-compact:var(--target-compact);--control-height-standard:var(--target-primary);
  }
@media (min-width:760px){:root{--page-pad:20px;--chrome-pad:20px}}
@media (min-width:1100px){:root{--page-pad:30px;--chrome-pad:30px}}

/* ── from tokens/effects.css ── */
/* Shape, elevation, motion, layers — values from app/src/styles/app.css, tokens.css, industry.css. */
:root{
  --shape-control:var(--r-sm);--shape-card:var(--r-md);--shape-surface:var(--r-lg);--shape-sheet:var(--r-lg);/* Lifts. Dark
grounds: --lift-*; light grounds: ink-tinted --shadow-*. Border first, shadow second. */
  --elevation-none:none;--elevation-raised:var(--shadow-sm);--elevation-floating:var(--shadow-md);--elevation-modal:var(--
shadow-lg);
  --ease:cubic-bezier(.22,1,.36,1);/* @kind other */
  --ease-standard:var(--ease);/* @kind other */
  --ease-emphasized:cubic-bezier(.2,.8,.2,1);/* @kind other */
  --duration-instant:0ms;/* @kind other */--duration-fast:130ms;/* @kind other */--duration-standard:180ms;/* @kind other */--
duration-slow:240ms;/* @kind other */--duration-sheet:280ms;/* @kind other */
  --fast:130ms var(--ease);/* @kind other */
  --motion-save:var(--duration-standard) var(--ease-standard);/* @kind other */
  --motion-insert:var(--duration-standard) var(--ease-standard);/* @kind other */
  --motion-panel:var(--duration-slow) var(--ease-emphasized);/* @kind other */
  --motion-sheet:var(--duration-sheet) var(--ease-emphasized);/* @kind other */
  --motion-progress:var(--duration-standard) linear;/* @kind other */
}
[data-theme="dark"]{--elevation-raised:var(--lift-1);--elevation-floating:var(--lift-2);--elevation-modal:var(--lift-3)}
@media (prefers-reduced-motion:reduce){:root{--fast:0ms linear;/* @kind other */--motion-save:0ms linear;/* @kind other */--
motion-insert:0ms linear;/* @kind other */--motion-panel:0ms linear;/* @kind other */--motion-sheet:0ms linear;/* @kind other
*/--motion-progress:0ms linear;/* @kind other */}}
```

```
:root[data-calm='still'],:root[data-calm='calm']{--fast:0ms linear;/* @kind other */--motion-save:0ms linear;/* @kind other */--motion-insert:0ms linear;/* @kind other */--motion-panel:0ms linear;/* @kind other */--motion-sheet:0ms linear;/* @kind other */--motion-progress:0ms linear;/* @kind other */}

/* ── from tokens/base.css ── */
/* Base element styles — what a Semester page looks like before any component. */
*,*::before,*::after{box-sizing:border-box}
html{-webkit-text-size-adjust:100%}
body{margin:0;background:var(--surface-canvas);color:var(--text-primary);font-family:var(--font-body);font-size:var(--type-base);line-height:var(--leading-relaxed-plus);-webkit-font-smoothing:antialiased;text-rendering:optimizeLegibility}
h1,h2,h3,h4,h5,h6{font-family:var(--font-heading);font-weight:var(--font-heading-weight);line-height:var(--leading-display-sm);letter-spacing:var(--tracking-heading);margin:0;text-wrap:balance}
h1{font-size:var(--type-role-page)}h2{font-size:var(--type-role-section)}h3{font-size:var(--type-lg)}h4{font-size:var(--type-md)}
p{margin:0;text-wrap:pretty}
a{color:var(--text-link);text-underline-offset:3px;text-decoration-thickness:1px}
a:hover{color:var(--text-primary)}
:focus{outline:none}
:focus-visible{outline:var(--focus-ring-width) solid var(--focus-color);outline-offset:var(--focus-ring-offset)}
::selection{background:rgba(116,157,196,.3)}
button{font:inherit;color:inherit}
.sr-only{position:absolute!important;width:1px;height:1px;padding:0;margin:-1px;overflow:hidden;clip:rect(0,0,0,0);white-space:nowrap;border:0}
.nums{font-variant-numeric:tabular-nums}
.mono{font-family:var(--font-mono);font-size:.92em}
.kicker{font-family:var(--font-body);font-size:var(--type-role-kicker);font-weight:500;letter-spacing:var(--tracking-kicker);text-transform:uppercase;color:var(--text-secondary)}
.editorial{font-family:var(--font-display);font-weight:500;letter-spacing:.01em;line-height:var(--leading-display-lg)}
.chrome-text{background:var(--chrome-metal);-webkit-background-clip:text;background-clip:text;color:transparent}
.link-quiet{color:var(--text-secondary);text-decoration:underline dotted;text-underline-offset:3px;font-size:var(--type-sm);background:none;border:0;padding:0;cursor:pointer}
.link-quiet:hover{color:var(--text-primary)}
.skip-link{position:absolute;left:var(--sp-4);top:-60px;z-index:var(--layer-skip);padding:var(--sp-4) var(--sp-6);background:var(--surface-overlay);border:1px solid var(--border-default);border-radius:var(--r-md)}
.skip-link:focus{top:var(--sp-4)}

/* ── from tokens/components.css ── */
/* Semester component classes. Drawn from semantic tokens only; used by the React components in components/.
 * Ported from app/src/styles/unity.css + industry.css, re-pointed at this system's tokens. */

/* ── Buttons ── */
.btn{display:inline-flex;align-items:center;justify-content:center;gap:calc(6px * var(--density,1));min-height:var(--target-primary);padding:var(--sp-3) calc(14px * var(--density,1));font-family:var(--font-heading);font-weight:var(--font-heading-weight);font-size:var(--type-control);letter-spacing:.02em;line-height:1.2;color:var(--text-primary);background:transparent;border:1px solid var(--border-default);border-radius:var(--shape-control);cursor:pointer;text-decoration:none;white-space:nowrap;transition:background var(--fast),border-color var(--fast),color var(--fast)}
.btn svg{display:block;flex:none}
.btn:hover{background:var(--action-secondary)}
.btn:active{background:var(--action-secondary-hover)}
.btn:disabled,.btn[aria-disabled='true']{opacity:.45;cursor:not-allowed}
.btn-primary{background:var(--action-primary);border-color:var(--action-primary);color:var(--text-on-action)}
.btn-primary:hover{background:var(--action-primary-hover);border-color:var(--action-primary-hover)}
.btn-primary:active{background:var(--action-primary-press)}
.btn-ghost{border-color:transparent;color:var(--text-link);padding-inline:var(--sp-4)}
.btn-danger{color:var(--status-danger);border-color:var(--status-danger-line)}
.btn-danger:hover{background:var(--status-danger-wash)}
.btn-sm{min-height:var(--target-compact);padding:var(--sp-2) var(--sp-5);font-size:var(--type-sm-plus)}
.btn-block{width:100%}
.btn-icon{width:var(--target-icon);height:var(--target-icon);min-height:0;padding:0;border-color:transparent;color:var(--text-secondary)}
.btn-icon:hover{color:var(--text-primary)}
.btn-icon.btn-sm{width:var(--target-compact);height:var(--target-compact)}

/* ── Forms ── */
.field{display:grid;gap:calc(5px * var(--density,1))}
.field-label{font-size:var(--type-sm);font-weight:500;color:var(--text-primary)}
.field-hint{font-size:var(--type-xs);color:var(--text-secondary);line-height:var(--leading-normal)}
.field-error{font-size:var(--type-xs);color:var(--status-danger);display:flex;gap:var(--sp-2)}
.input{width:100%;min-height:var(--target-primary);padding:calc(6px * var(--density,1)) calc(10px * var(--density,1));font:inherit;font-size:var(--type-control);color:var(--text-primary);caret-color:var(--action-primary);background:var(--surface-raised);border:1px solid var(--border-default);border-radius:var(--shape-control);transition:border-color var(--fast)}
.input::placeholder{color:var(--text-tertiary)}
.input:hover{border-color:var(--border-strong)}
.input:focus-visible{border-color:var(--focus-color);outline-offset:0}
.input[aria-invalid='true']{border-color:var(--status-danger-line)}
.input:disabled{background:var(--surface-sunken);color:var(--text-disabled);cursor:not-allowed}
```

```
textarea.input{min-height:90px;resize:vertical;line-height:var(--leading-normal)}
select.input{appearance:none;padding-inline-end:34px;background-image:linear-gradient(45deg,transparent 50%,var(--text-
secondary) 50%),linear-gradient(135deg,var(--text-secondary) 50%,transparent 50%);background-position:calc(100% - 16px)
50%,calc(100% - 12px) 50%;background-size:4px 4px;background-repeat:no-repeat}
.seg{display:inline-flex;overflow:hidden;border:1px solid var(--border-default);border-radius:var(--shape-
control);background:var(--surface-raised)}
.seg-opt{display:inline-flex;align-items:center;gap:var(--sp-3);min-height:var(--target-compact);padding:calc(7px * var(--
density,1)) calc(12px * var(--density,1));font-size:var(--type-sm-plus);cursor:pointer;background:none;border:0;color:var(--
text-secondary)}
.seg-opt+.seg-opt{border-left:1px solid var(--border-default)}
.seg-opt[aria-pressed='true'],.seg-opt[aria-checked='true']{background:var(--text-primary);color:var(--surface-canvas)}
.seg-opt:not([aria-pressed='true']):not([aria-checked='true']):hover{background:var(--action-secondary);color:var(--text-
primary)}
.choice{display:flex;align-items:flex-start;gap:var(--sp-5);min-height:var(--target-primary);padding-block:var(--sp-3);font-
size:var(--type-base);color:var(--text-primary);cursor:pointer}
.choice input{width:20px;height:20px;margin:2px 0 0;flex:none;accent-color:var(--action-primary)}
.choice-about{display:block;font-size:var(--type-sm);line-height:var(--leading-normal);color:var(--text-secondary)}
.switch{display:inline-flex;align-items:center;gap:var(--sp-5);min-height:var(--target-primary);cursor:pointer;font-size:var(--
type-base)}
.switch-track{position:relative;width:36px;height:20px;flex:none;border-radius:var(--shape-pill);background:var(--surface-
sunken);border:1px solid var(--border-strong);transition:background var(--fast)}
.switch-track::after{content:'';position:absolute;top:2px;left:2px;width:14px;height:14px;border-radius:50%;background:var(--
text-secondary);transition:transform var(--fast),background var(--fast)}
.switch[aria-checked='true'] .switch-track{background:var(--action-primary);border-color:var(--action-primary)}
.switch[aria-checked='true'] .switch-track::after{transform:translateX(16px);background:var(--text-on-action)}
.switch-state{font-size:var(--type-xs);color:var(--text-secondary);min-width:2.5em}

/* ── Status + provenance ── */
.status-chip{display:inline-flex;align-items:baseline;gap:var(--sp-2);padding:var(--sp-1) var(--sp-4);font-size:var(--type-
xs);line-height:var(--leading-tight);color:var(--status-neutral);border:1px solid var(--border-default);border-radius:var(--
shape-control);white-space:nowrap}
.status-glyph{font-variant-numeric:tabular-nums}
.status-chip[data-tone='pending'],.status-chip[data-tone='attention']{color:var(--status-pending);border-color:var(--status-
pending-line);background:var(--status-pending-wash)}
.status-chip[data-tone='danger']{color:var(--status-danger);border-color:var(--status-danger-line);background:var(--status-
danger-wash)}
.status-chip[data-tone='success']{color:var(--status-success);border-color:var(--status-success-line)}
.status-chip[data-tone='official']{color:var(--source-official);border-color:var(--source-official-line);background:var(--
source-official-wash)}
.status-chip[data-tone='connected']{color:var(--source-connected);border-color:var(--source-connected-line)}
.status-chip[data-tone='personal']{color:var(--source-personal);border-color:var(--source-personal-line)}
.status-chip[data-tone='ai']{color:var(--source-ai);border-color:var(--source-ai-line);background:var(--source-ai-wash)}
.status-chip[data-fill='solid']{border-width:1px;border-style:solid}
.status-chip[data-fill='dashed']{border-style:dashed}
.prov{display:inline-flex;flex-wrap:wrap;align-items:baseline;gap:var(--sp-2)}
.prov-age{font-size:var(--type-xs);color:var(--text-secondary)}
.source-badge{display:inline-flex;flex-wrap:wrap;align-items:baseline;gap:var(--sp-3);font-size:var(--type-xs);color:var(--
text-secondary)}
.source-report{background:none;border:0;padding:0;font-size:var(--type-xs);color:var(--text-link);text-
decoration:underline;text-underline-offset:3px;cursor:pointer}
.save-state{display:inline-flex;align-items:baseline;gap:var(--sp-2);font-size:var(--type-xs);color:var(--text-
secondary);transition:color var(--motion-save)}
.save-state[data-tone='success']{color:var(--text-primary)}
.save-state[data-tone='pending']{color:var(--status-pending)}
.save-state[data-tone='danger']{color:var(--status-danger)}

/* ── AI disclosure ── */
.ai-frame{display:grid;gap:var(--sp-6);padding:var(--sp-7) var(--sp-7) var(--sp-6);border:1px solid var(--source-ai-
line);border-radius:var(--shape-card);background:var(--surface-raised)}
.ai-head{display:flex;flex-wrap:wrap;align-items:center;justify-content:space-between;gap:var(--sp-4)}
.ai-body{font-size:var(--type-base);line-height:var(--leading-relaxed-plus);max-width:var(--layout-reading)}
.ai-body p+p{margin-top:var(--sp-5)}
.ai-cite{display:inline-flex;align-items:center;justify-content:center;min-width:18px;height:18px;padding:0 4px;margin-
inline:2px;font-size:var(--type-2xs);font-variant-numeric:tabular-nums;vertical-align:2px;border:1px solid var(--border-
default);border-radius:var(--r-sm);color:var(--text-secondary)}
.ai-sources{display:grid;gap:var(--sp-2);margin:0;padding:var(--sp-5) 0 0;list-style:none;border-top:1px solid var(--border-
subtle)}
.ai-sources li{display:flex;gap:var(--sp-4);align-items:baseline;font-size:var(--type-sm);color:var(--text-secondary)}
.ai-meta{display:flex;flex-wrap:wrap;gap:var(--sp-3) var(--sp-7);font-size:var(--type-xs);color:var(--text-secondary)}
.ai-meta b{font-weight:500;color:var(--text-primary)}

/* ── Surfaces ── */
.object-card{display:flex;flex-direction:column;gap:var(--sp-3);padding:var(--sp-7);background:var(--surface-base);border:1px
solid var(--border-default);border-radius:var(--shape-card)}
.object-card-eyebrow{font-size:var(--type-2xs-plus);letter-spacing:var(--tracking-kicker);text-transform:uppercase;color:var(--
text-secondary);display:flex;gap:var(--sp-3);align-items:baseline}
.object-card-title{margin:0;font-family:var(--font-body);font-size:var(--type-lg);line-height:var(--leading-display-sm);font-
```

```
weight:600;letter-spacing:0;color:var(--text-primary)}
.object-card-why{font-size:var(--type-base);line-height:var(--leading-normal);color:var(--text-secondary)}
.object-card-meta{font-size:var(--type-sm);color:var(--text-secondary);font-variant-numeric:tabular-nums}
.object-card-actions,.object-card-foot,.state-actions,.dialog-actions{display:flex;flex-wrap:wrap;align-items:center;gap:var(--
sp-4) var(--sp-6)}
.object-card-actions{margin-top:var(--sp-3)}
.object-card[data-tone='pending']{border-color:var(--status-pending-line)}
.object-card[data-tone='danger']{border-color:var(--status-danger-line)}
.context-bar{display:flex;flex-direction:column;gap:var(--sp-3);padding:var(--sp-6) var(--sp-7);background:var(--surface-
base);border:1px solid var(--border-subtle);border-radius:var(--shape-card)}
.context-bar-title{margin:0;font-family:var(--font-body);font-weight:600;letter-spacing:0;font-size:var(--type-display-
xs);line-height:var(--leading-display-xs)}
.context-bar-states{display:flex;flex-wrap:wrap;gap:var(--sp-3)}
.next-steps{display:grid;gap:var(--sp-3);margin:0;padding:0;list-style:none}
.next-step{display:flex;align-items:center;gap:var(--sp-6);width:100%;min-height:var(--target-primary);padding:var(--sp-5)
var(--sp-6);text-align:left;background:var(--surface-base);color:var(--text-primary);border:1px solid var(--border-
default);border-radius:var(--shape-card);font:inherit;cursor:pointer;transition:border-color var(--fast),background var(--
fast)}
.next-step:hover{border-color:var(--border-strong);background:var(--surface-raised)}
.next-step-text{display:grid;gap:2px;flex:1;min-width:0}
.next-step-label{font-size:var(--type-md)}
.next-step-why{font-size:var(--type-sm);color:var(--text-secondary)}
.action-preview{display:grid;gap:var(--sp-5)}
.action-preview-facts{display:grid;grid-template-columns:minmax(0,1fr);gap:var(--sp-1);margin:0}
.action-preview-facts dt{font-size:var(--type-xs);color:var(--text-secondary);margin-top:var(--sp-4)}
.action-preview-facts dd{margin:0;font-size:var(--type-base);line-height:var(--leading-normal)}
.action-preview-caveat{font-size:var(--type-sm);color:var(--text-secondary)}
.dialog-scrim{position:fixed;inset:0;z-index:var(--layer-overlay);display:grid;place-items:center;padding:var(--space-
2);background:var(--scrim)}
.dialog{width:min(480px,100%);display:flex;flex-direction:column;gap:var(--sp-7);padding:calc(24px * var(--
density,1));background:var(--surface-overlay);border:1px solid var(--border-default);border-radius:var(--shape-sheet);box-
shadow:var(--elevation-modal);animation:dialog-in var(--motion-sheet)}
.dialog-title{font-family:var(--font-heading);font-size:var(--type-display-sm)}
.dialog-actions{justify-content:flex-end;padding-top:var(--sp-4);border-top:1px solid var(--border-subtle)}
@keyframes dialog-in{from{opacity:0;transform:translateY(8px)}}

/* ── States ── */
.state{display:flex;flex-direction:column;gap:var(--sp-3);padding:var(--sp-6) var(--sp-7);border:1px solid var(--border-
default);border-radius:var(--shape-card);font-size:var(--type-base);line-height:var(--leading-normal);color:var(--text-
primary)}
.state-title{font-size:var(--type-md);font-weight:600;display:flex;gap:var(--sp-3);align-items:baseline}
.state-body,.state-note,.state-ref{margin:0;color:var(--text-secondary)}
.state-note,.state-ref{font-size:var(--type-sm)}
.state[data-kind='error']{border-color:var(--status-danger-line);background:var(--status-danger-wash)}
.state[data-kind='error'] .status-glyph{color:var(--status-danger)}
.state[data-kind='success'] .status-glyph{color:var(--status-success)}
.state[data-kind='offline']{border-color:var(--status-pending-line);background:var(--status-pending-wash);flex-
direction:row;gap:var(--sp-4);font-size:var(--type-sm)}
.state[data-kind='permission']{border-style:dashed}
.state[data-kind='empty']{border-style:dashed;align-items:flex-start;padding:var(--sp-7) var(--sp-7)}
.state-progress-head{display:flex;justify-content:space-between;gap:var(--sp-4);font-size:var(--type-sm)}
.state-progress-bar{width:100%;height:6px;appearance:none;border:0;border-radius:var(--shape-
pill);overflow:hidden;background:var(--surface-sunken)}
.state-progress-bar::-webkit-progress-bar{background:var(--surface-sunken)}
.state-progress-bar::-webkit-progress-value{background:var(--action-primary);transition:width var(--motion-progress)}
.state-progress-bar::-moz-progress-bar{background:var(--action-primary)}
.state-steps{list-style:none;margin:0;padding:0;display:flex;flex-direction:column;gap:var(--sp-3);font-size:var(--type-
base);color:var(--text-secondary)}
.state-steps li{display:flex;gap:var(--sp-4)}
.state-steps [data-state='working'],.state-steps [data-state='done']{color:var(--text-primary)}
.state-steps [data-state='failed']{color:var(--status-danger)}
.state-loading{display:flex;flex-direction:column;gap:var(--sp-6)}
.state-loading-bar{border-radius:var(--shape-card);background:var(--surface-sunken)}
.notice{margin:0;padding:var(--sp-3) var(--sp-4);border:1px solid var(--border-default);border-radius:var(--r-
md);background:var(--surface-base);font-size:var(--type-sm);line-height:var(--leading-relaxed)}
.notice-head{font-family:var(--font-heading);letter-spacing:.06em;text-transform:uppercase;font-size:var(--type-xs);margin-
right:var(--sp-3)}

/* ── Navigation ── */
.rail{display:flex;flex-direction:column;gap:2px;width:var(--rail-w);padding:var(--sp-7) var(--sp-5);background:var(--chrome-
surface);color:var(--chrome-text);border-right:1px solid var(--chrome-line);height:100%;overflow:auto}
.rail-brand{display:flex;align-items:center;gap:var(--sp-5);padding:var(--sp-3) var(--sp-5) var(--sp-7)}
.rail-label{padding:var(--sp-7) var(--sp-5) var(--sp-3);font-size:var(--type-2xs-plus);letter-spacing:var(--tracking-
kicker);text-transform:uppercase;color:var(--chrome-text-secondary)}
.rail-row{display:flex;align-items:center;gap:var(--sp-6);width:100%;min-height:40px;padding:var(--sp-3) var(--sp-
5);background:none;border:0;border-radius:var(--r-md);color:var(--chrome-text-secondary);font-family:var(--font-body);font-
size:var(--type-control);text-align:left;cursor:pointer;transition:background var(--fast),color var(--fast)}
```

```
.rail-row:hover{color:var(--chrome-text);background:rgba(236,238,242,.05)}
.rail-row[aria-current='page']{color:var(--chrome-text);background:var(--chrome-active)}
.rail-row[aria-current='page'] svg{color:var(--blue-300)}
.rail-count{margin-left:auto;font-size:var(--type-xs);font-variant-numeric:tabular-nums;color:var(--chrome-text-secondary)}
.rail-foot{margin-top:auto;padding:var(--sp-7) var(--sp-5) 0;font-size:var(--type-xs);color:var(--chrome-text-secondary);line-
height:var(--leading-normal);border-top:1px solid var(--chrome-line)}
.tabbar{display:flex;justify-content:space-around;align-items:stretch;padding:var(--sp-2) var(--sp-3) max(var(--sp-3),env(safe-
area-inset-bottom));background:rgba(9,10,14,.94);border-top:1px solid var(--chrome-line);color:var(--chrome-text-secondary)}
@supports (backdrop-filter:blur(1px)){.tabbar{background:rgba(9,10,14,.86);backdrop-filter:blur(var(--blur-chrome));-webkit-
backdrop-filter:blur(var(--blur-chrome))}}
.tab{flex:1;display:flex;flex-direction:column;align-items:center;justify-content:center;gap:3px;min-height:var(--target-
primary);padding:var(--sp-2) 0;background:none;border:0;color:inherit;font-size:var(--type-2xs-plus);letter-
spacing:.02em;cursor:pointer}
.tab[aria-current='page']{color:var(--chrome-text)}
.tab[aria-current='page'] svg{color:var(--blue-300)}
.syscontext{display:flex;flex-wrap:wrap;align-items:center;justify-content:space-between;gap:var(--sp-4);min-height:var(--
target-primary);padding:var(--sp-2) var(--chrome-pad);border-bottom:1px solid var(--border-subtle);background:var(--surface-
base);color:var(--text-secondary);font-size:var(--type-xs);line-height:var(--leading-tight)}
.syscontext-group{display:flex;align-items:center;gap:var(--sp-5);min-width:0}
.syscontext-term{color:var(--text-primary);font-variant-numeric:tabular-nums}
.syscontext-search{display:flex;align-items:center;gap:var(--sp-4);min-height:32px;min-width:240px;padding:0 var(--sp-
5);border:1px solid var(--border-default);border-radius:var(--r-md);background:var(--surface-raised);color:var(--text-
tertiary);font-size:var(--type-sm);cursor:pointer}
.kbd{display:inline-flex;align-items:center;min-width:18px;height:18px;padding:0 5px;font-family:var(--font-mono);font-
size:var(--type-2xs);color:var(--text-secondary);border:1px solid var(--border-default);border-bottom-width:2px;border-
radius:var(--r-sm);background:var(--surface-raised)}
.palette{width:min(620px,100%);background:var(--surface-overlay);border:1px solid var(--border-default);border-radius:var(--
shape-sheet);box-shadow:var(--elevation-modal);overflow:hidden}
.palette-input{display:flex;align-items:center;gap:var(--sp-5);padding:0 var(--sp-7);border-bottom:1px solid var(--border-
subtle)}
.palette-input input{flex:1;min-height:52px;border:0;background:none;font:inherit;font-size:var(--type-md);color:var(--text-
primary);outline:none}
.palette-group{padding:var(--sp-4) var(--sp-4) var(--sp-3)}
.palette-group-label{padding:var(--sp-3) var(--sp-5);font-size:var(--type-2xs-plus);letter-spacing:var(--tracking-kicker);text-
transform:uppercase;color:var(--text-secondary)}
.palette-option{display:flex;align-items:center;gap:var(--sp-6);width:100%;min-height:var(--target-primary);padding:var(--sp-3)
var(--sp-5);border:0;border-inline-start:3px solid transparent;background:none;text-align:left;font:inherit;font-size:var(--
type-control);color:var(--text-primary);cursor:pointer;border-radius:0 var(--r-sm) var(--r-sm) 0}
.palette-option[aria-selected='true']{border-inline-start-color:var(--focus-color);background:var(--surface-quiet)}
.palette-option-detail{margin-left:auto;font-size:var(--type-xs);color:var(--text-secondary)}
.palette-foot{display:flex;gap:var(--sp-7);padding:var(--sp-4) var(--sp-7);border-top:1px solid var(--border-subtle);font-
size:var(--type-xs);color:var(--text-secondary)}

/* ── Data ── */
.table-wrap{overflow-x:auto;border:1px solid var(--border-default);border-radius:var(--shape-card);background:var(--surface-
base)}
.table{width:100%;border-collapse:collapse;font-size:var(--type-control)}
.table th{text-align:left;font-weight:500;font-size:var(--type-xs);letter-spacing:var(--tracking-kicker);text-
transform:uppercase;color:var(--text-secondary);padding:var(--sp-5) var(--sp-6);border-bottom:1px solid var(--border-
default);white-space:nowrap;background:var(--surface-quiet)}
.table td{padding:var(--sp-5) var(--sp-6);border-bottom:1px solid var(--border-subtle);vertical-align:baseline}
.table tbody tr:last-child td{border-bottom:0}
.table tbody tr:hover{background:var(--action-secondary)}
.table td.num,.table th.num{text-align:right;font-variant-numeric:tabular-nums}
.fields{display:grid;grid-template-columns:minmax(120px,max-content) minmax(0,1fr);gap:var(--sp-4) var(--sp-7);margin:0;font-
size:var(--type-sm)}
.fields dt{color:var(--text-secondary)}
.fields dd{margin:0;color:var(--text-primary)}
.trail{list-style:none;margin:0;padding:0;display:grid}
.trail-item{display:grid;grid-template-columns:14px minmax(0,1fr);gap:var(--sp-6);padding-bottom:var(--sp-7);position:relative}
.trail-item::before{content:'';position:absolute;left:6px;top:14px;bottom:0;width:1px;background:var(--border-default)}
.trail-item:last-child::before{display:none}
.trail-dot{width:13px;height:13px;margin-top:4px;border-radius:50%;border:1.5px solid var(--border-strong);background:var(--
surface-canvas)}
.trail-item[data-state='done'] .trail-dot{background:var(--text-primary);border-color:var(--text-primary)}
.trail-item[data-state='current'] .trail-dot{border-color:var(--focus-color);box-shadow:0 0 0 3px var(--action-secondary-
hover)}
.trail-item[data-state='blocked'] .trail-dot{border-color:var(--status-danger)}
.trail-title{font-size:var(--type-control);font-weight:500}
.trail-meta{font-size:var(--type-xs);color:var(--text-secondary);font-variant-numeric:tabular-nums}

/* ── Expanded set ── */
.combo{position:relative;display:grid;gap:var(--sp-2)}
.combo-list{position:absolute;inset-inline:0;top:100%;z-index:var(--layer-menu);margin:var(--sp-2) 0 0;padding:var(--sp-
2);list-style:none;max-height:260px;overflow-y:auto;background:var(--surface-overlay);border:1px solid var(--border-
default);border-radius:var(--shape-surface);box-shadow:var(--elevation-floating)}
.combo-option{display:grid;gap:var(--sp-1);min-height:var(--target-primary);padding:var(--sp-3) var(--sp-4);border-inline-
```

```
start:3px solid transparent;cursor:pointer;font-size:var(--type-control)}
.combo-option[aria-selected='true']{border-inline-start-color:var(--focus-color);background:var(--surface-quiet);font-
weight:600}
.combo-detail{font-size:var(--type-xs);color:var(--text-secondary);font-weight:400}
.visibility{display:flex;flex-direction:column;gap:var(--sp-3)}
.visibility-said{margin:0;font-size:var(--type-sm);line-height:var(--leading-normal);color:var(--text-
primary);display:flex;flex-wrap:wrap;gap:var(--sp-3);align-items:baseline}
.visibility fieldset{border:0;margin:0;padding:0;display:grid;gap:var(--sp-2)}
.sheet-scrim{position:fixed;inset:0;z-index:var(--layer-overlay);display:flex;align-items:flex-end;justify-
content:center;background:var(--scrim)}
.sheet{width:100%;max-width:620px;max-height:88%;overflow:auto;padding:var(--sp-7) calc(20px * var(--density,1)) calc(24px *
var(--density,1));background:var(--surface-overlay);border:1px solid var(--border-default);border-radius:var(--shape-sheet)
var(--shape-sheet) 0 0;box-shadow:var(--elevation-modal);animation:dialog-in var(--motion-sheet)}
.sheet.is-inline{max-height:none;overflow:visible;border-radius:var(--shape-sheet);box-shadow:var(--elevation-
floating);animation:none}
.sheet-grip{width:36px;height:4px;border-radius:2px;background:var(--border-strong);margin:0 auto var(--sp-6)}
.sheet-head{display:flex;align-items:center;justify-content:space-between;gap:var(--sp-4);margin-bottom:var(--sp-4)}
.source-list{display:grid;gap:var(--sp-1);margin:0}
.source-list dt{margin-top:var(--sp-6);font-size:var(--type-2xs-plus);letter-spacing:var(--tracking-kicker);text-
transform:uppercase;color:var(--text-secondary)}
.source-list dt:first-child{margin-top:0}
.source-list dd{margin:0;font-size:var(--type-base);line-height:var(--leading-normal)}
.toast{display:inline-flex;align-items:center;gap:var(--sp-6);min-height:var(--target-primary);padding:var(--sp-3) var(--sp-4)
var(--sp-3) var(--sp-7);background:var(--ink-raise);color:var(--ink-fg);border-radius:var(--r-md);box-shadow:var(--lift-
2);font-size:var(--type-sm)}
.toast .btn{color:var(--blue-300);border-color:transparent;min-height:var(--target-compact)}
.toast .btn:hover{background:rgba(148,188,227,.12)}
.focus-bar{display:inline-flex;flex-wrap:wrap;align-items:center;gap:var(--sp-5);padding:var(--sp-3) var(--sp-4) var(--sp-3)
var(--sp-7);background:var(--surface-overlay);border:1px solid var(--border-default);border-radius:var(--shape-pill);box-
shadow:var(--elevation-floating);font-size:var(--type-sm)}
.focus-timer{font-family:var(--font-heading);font-weight:600;font-size:var(--type-lg);font-variant-numeric:tabular-nums}
.screen-guide{padding-top:var(--sp-6);border-top:1px solid var(--border-subtle)}
.screen-guide summary{cursor:pointer;font-size:var(--type-sm);color:var(--text-secondary);min-height:var(--target-min);list-
style:none;display:flex;gap:var(--sp-3);align-items:center}
.screen-guide summary::-webkit-details-marker{display:none}
.screen-guide[open] summary{color:var(--text-primary)}
.screen-guide dl{margin:var(--sp-5) 0 0;max-width:var(--layout-reading)}
.screen-guide dt{margin-top:var(--sp-5);font-weight:600;font-size:var(--type-control)}
.screen-guide dd{margin:var(--sp-1) 0 0;color:var(--text-secondary);font-size:var(--type-control);line-height:var(--leading-
relaxed)}
.open-in{display:flex;flex-wrap:wrap;align-items:center;gap:var(--sp-3)}
.open-in-label{font-size:var(--type-xs);color:var(--text-secondary);margin-right:var(--sp-2)}
.pill-soft{display:inline-flex;align-items:center;gap:var(--sp-3);min-height:var(--target-compact);padding:0 var(--sp-
6);background:var(--action-secondary);border:1px solid transparent;border-radius:var(--shape-control);font-size:var(--type-sm-
plus);color:var(--text-primary);cursor:pointer}
.pill-soft:hover{background:var(--action-secondary-hover)}
.quick-actions{display:flex;flex-wrap:wrap;gap:var(--sp-4)}
.quick-action{display:inline-flex;align-items:center;gap:var(--sp-4);min-height:var(--target-primary);padding:0 var(--sp-7) 0
var(--sp-6);background:var(--surface-base);border:1px solid var(--border-default);border-radius:var(--shape-card);font-
size:var(--type-control);color:var(--text-primary);cursor:pointer}
.quick-action:hover{border-color:var(--border-strong);background:var(--surface-raised)}
.avatar{display:inline-flex;align-items:center;justify-content:center;flex:none;border-radius:var(--r-lg);background:var(--
action-secondary);border:1px solid var(--border-strong);color:var(--text-primary);font-family:var(--font-heading);letter-
spacing:.04em;line-height:1;overflow:hidden}
.metric{display:grid;gap:var(--sp-3);padding:var(--sp-6) var(--sp-7);border:1px solid var(--border-default);border-radius:var(-
-shape-card);background:var(--surface-base)}
.metric-value{font-family:var(--font-heading);font-weight:600;font-size:var(--type-2xl);line-height:1;font-variant-
numeric:tabular-nums}
.metric-delta{font-size:var(--type-xs);color:var(--text-secondary)}
.chart{display:grid;gap:var(--sp-5)}
.chart-title{font-size:var(--type-md);font-weight:600}
.chart-bars{display:grid;gap:var(--sp-4)}
.chart-row{display:grid;grid-template-columns:120px minmax(0,1fr) 64px;gap:var(--sp-5);align-items:center;font-size:var(--type-
sm)}
.chart-track{height:14px;background:var(--surface-sunken);border-radius:2px;overflow:hidden}
.chart-bar{height:100%;border-radius:2px}
.chart-bar[data-enc='solid']{background:var(--text-primary)}
.chart-bar[data-enc='outline']{background:transparent;box-shadow:inset 0 0 0 1.5px var(--text-primary)}
.chart-bar[data-enc='hatched']{background:repeating-linear-gradient(135deg,var(--text-primary) 0 2px,transparent 2px 6px);box-
shadow:inset 0 0 0 1px var(--text-primary)}
.chart-bar[data-enc='dotted']{background:radial-gradient(var(--status-danger) 1.2px,transparent 1.4px) 0 0/6px 6px;box-
shadow:inset 0 0 0 1px var(--status-danger)}
.chart-foot{font-size:var(--type-xs);color:var(--text-secondary);display:flex;flex-wrap:wrap;gap:var(--sp-3) var(--sp-7)}
.legend-swatch{display:inline-block;width:18px;height:10px;vertical-align:-1px;margin-right:var(--sp-3);border-radius:1px}

@media (forced-colors:active){.status-chip,.object-card,.context-bar,.next-step,.dialog,.palette,.state{border:1px solid
CanvasText}.status-chip[data-fill='solid']{border-width:3px}}
```

```
/* governance: PolicyBadge, ConsentBadge, HealthBadge, ApprovalBanner, ErrorSummary, SupportHandoff, PageHeader */
.gov-badge{display:inline-flex;align-items:baseline;gap:var(--sp-2);padding:var(--sp-1) var(--sp-4);font:inherit;font-
size:var(--type-xs);line-height:var(--leading-tight);color:var(--text-primary);background:transparent;border:1px solid var(--
border-default);border-radius:var(--shape-control);white-space:nowrap}
button.gov-badge{cursor:pointer}button.gov-badge:hover{background:var(--action-secondary)}
.gov-glyph{font-variant-numeric:tabular-nums;color:var(--text-secondary)}.gov-sub{color:var(--text-secondary);font-size:var(--
type-xs)}
.gov-link{all:unset;cursor:pointer;color:var(--action-primary);text-decoration:underline;margin-left:var(--sp-2)}
.gov-badge[data-kind='consent'][data-state='shared']{border-color:var(--status-pending-line)}
.gov-badge[data-kind='consent'][data-state='required']{color:var(--status-pending);border-color:var(--status-pending-
line);background:var(--status-pending-wash)}
.health-badge{display:inline-flex;align-items:baseline;gap:var(--sp-2);padding:var(--sp-1) var(--sp-4);font-size:var(--type-
xs);border:1px solid var(--border-default);border-radius:var(--shape-control)}
.health-badge b{font-weight:600}.health-badge[data-tone='success']{color:var(--status-success);border-color:var(--status-
success-line)}
.health-badge[data-tone='pending']{color:var(--status-pending);border-color:var(--status-pending-line);background:var(--status-
pending-wash)}
.health-badge[data-tone='danger']{color:var(--status-danger);border-color:var(--status-danger-line);background:var(--status-
danger-wash)}
.approval-banner{display:flex;gap:var(--sp-5);align-items:center;padding:var(--sp-5) var(--sp-6);border:1px solid var(--status-
pending-line);background:var(--status-pending-wash);border-radius:var(--shape-card);flex-wrap:wrap}
.approval-banner[data-state='approved']{border-color:var(--status-success-line);background:var(--status-success-
wash)}.approval-banner[data-state='rejected']{border-color:var(--status-danger-line);background:var(--status-danger-wash)}
.approval-body{display:grid;gap:2px;flex:1;min-width:200px}.approval-actions{display:flex;gap:var(--sp-3)}
.error-summary{padding:var(--sp-5) var(--sp-6);border:1px solid var(--status-danger-line);background:var(--status-danger-
wash);border-radius:var(--shape-card);color:var(--status-danger)}
.error-summary ul{margin:var(--sp-3) 0 0;padding-left:var(--sp-7)}.error-summary a{color:var(--status-danger)}
.support-handoff{display:grid;gap:var(--sp-3);padding:var(--sp-6);border:1px solid var(--border-default);border-radius:var(--
shape-card);background:var(--surface-base)}
.handoff-office{font-family:var(--font-heading);font-size:var(--type-lg)}.handoff-reason{margin:0;color:var(--text-secondary)}
.handoff-meta{display:grid;grid-template-columns:auto 1fr;gap:var(--sp-1) var(--sp-5);margin:0;font-size:var(--type-
sm)}.handoff-meta dt{color:var(--text-secondary)}.handoff-meta dd{margin:0}
.page-header{display:flex;justify-content:space-between;align-items:flex-end;gap:var(--sp-6);flex-wrap:wrap;padding-
bottom:var(--sp-7);border-bottom:1px solid var(--border-subtle)}
.page-header h1{margin:var(--sp-2) 0 0;text-wrap:pretty}.page-header-purpose{margin:var(--sp-2) 0 0;color:var(--text-
secondary);max-width:66ch}.page-header-source{margin-top:var(--sp-3)}.page-header-actions{display:flex;gap:var(--sp-3);flex-
wrap:wrap}
```

Component types

Every component's props, as TypeScript declarations.

components/index.d.ts · 415 lines

```
// Types for every component in window.SemesterDesignSystem_e9f3ba, gathered from components/<Name>/<Name>.d.ts.

// ---- AIResponse
import * as React from 'react';
/**
 * A governed assistant answer — labelled AI-assisted with data scope, numbered sources, confidence, request cost and policy.
 * @startingPoint section="AI" subtitle="A governed assistant answer — labelled AI-assisted with data scope, numbered sou"
viewport="700x400"
 */
export interface AIResponseProps { children: React.ReactNode; scope?: string[]; sources?: { title: string; kind?: string }[];
confidence?: string; cost?: string; policy?: string; actions?: React.ReactNode; model?: string; }
export declare function AIResponse(props: AIResponseProps): JSX.Element;

// ---- ActionPreview
import * as React from 'react';
/**
 * What is about to happen, said before it happens — What happens / Exactly what / What stays / Taking it back.
 */
export interface ActionPreviewProps { subject?: string; says: React.ReactNode; exactly?: React.ReactNode; doesNotChange?:
React.ReactNode; recovery: { kind: "undo" | "request" | "none"; how?: string }; subjectTo?: React.ReactNode; provenance?:
React.ReactNode; }
export declare function ActionPreview(props: ActionPreviewProps): JSX.Element;

// ---- ApprovalBanner
import * as React from 'react';
export interface ApprovalBannerProps { state?: "required" | "pending" | "approved" | "rejected"; what: string; approver?:
string; requestedBy?: string; onRequest?: () => void; onApprove?: () => void; onReject?: () => void; }
export declare function ApprovalBanner(props: ApprovalBannerProps): JSX.Element;

// ---- Avatar
import * as React from 'react';
/** Initials in a tinted square that follows the corner setting; profile glyph when no name. */
export interface AvatarProps { name?: string; size?: number; }
export declare function Avatar(props: AvatarProps): JSX.Element;

// ---- Button
import * as React from 'react';
/**
 * Action button — one primary per decision region; 44px tall by default, condensed-caps-free label in Barlow Condensed 600.
 * @startingPoint section="Actions" subtitle="Action button — one primary per decision region; 44px tall by default, condensed"
viewport="700x400"
 */
export interface ButtonProps { variant?: "primary" | "secondary" | "ghost" | "danger"; size?: "md" | "sm"; icon?: string;
iconAfter?: string; block?: boolean; loading?: boolean; disabled?: boolean; children?: React.ReactNode; onClick?: (e: any) =>
void; type?: "button" | "submit"; }
export declare function Button(props: ButtonProps): JSX.Element;

// ---- Choice
import * as React from 'react';
/**
 * Radio/checkbox row with a sentence explaining the consequence — used for visibility and sharing choices.
 */
export interface ChoiceProps { type?: "radio" | "checkbox"; label: string; about?: string; checked?: boolean; onChange?: (e:
any) => void; name?: string; value?: string; disabled?: boolean; }
export declare function Choice(props: ChoiceProps): JSX.Element;

// ---- Combobox
import * as React from 'react';
/** Text box with a filtered listbox beneath — course search, people, rooms. */
export interface ComboboxProps { label: string; hint?: string; options: { label: string; detail?: string }[]; value?: string;
onChange?: (q: string) => void; onSelect?: (o: any) => void; placeholder?: string; defaultOpen?: boolean; }
export declare function Combobox(props: ComboboxProps): JSX.Element;

// ---- CommandPalette
```

```
import * as React from 'react';
/**
 * Global search and command palette (⌘K) with keyboard selection.
 */
export interface CommandPaletteProps { groups: { label: string; items: { label: string; icon?: string; detail?: string }[] }[];
placeholder?: string; onSelect?: (item: any) => void; }
export declare function CommandPalette(props: CommandPaletteProps): JSX.Element;

// ---- ConsentBadge
import * as React from 'react';
export interface ConsentBadgeProps { state?: "private" | "shared" | "expired" | "revoked" | "required"; recipient?: string;
expires?: string; onManage?: () => void; }
export declare function ConsentBadge(props: ConsentBadgeProps): JSX.Element;

// ---- ContextBar
import * as React from 'react';
/**
 * The object a screen is about, with its states and actions.
 */
export interface ContextBarProps { kicker?: string; title: string; states?: React.ReactNode; actions?: React.ReactNode; }
export declare function ContextBar(props: ContextBarProps): JSX.Element;

// ---- DataTable
import * as React from 'react';
/**
 * Crisp comparison table with caps headers, hairline rows, tabular numerals.
 * @startingPoint section="Data" subtitle="Crisp comparison table with caps headers, hairline rows, tabular numerals."
viewport="700x400"
 */
export interface DataTableProps { columns: { key: string; label: string; numeric?: boolean; render?: (row: any) =>
React.ReactNode }[]; rows: any[]; caption?: string; }
export declare function DataTable(props: DataTableProps): JSX.Element;

// ---- DateField
import * as React from 'react';
/** Native date/time input with the chosen date spoken back unambiguously. */
export interface DateFieldProps { label: string; hint?: string; value?: string; onChange?: (e: any) => void; withTime?:
boolean; said?: string; }
export declare function DateField(props: DateFieldProps): JSX.Element;

// ---- DecisionTrail
import * as React from 'react';
/**
 * Auditable timeline of who did what, when, and under which authority.
 */
export interface DecisionTrailProps { items: { title: string; who?: string; when?: string; detail?: string; state?: "done" |
"current" | "waiting" | "blocked" }[]; label?: string; }
export declare function DecisionTrail(props: DecisionTrailProps): JSX.Element;

// ---- Dialog
import * as React from 'react';
/**
 * Modal confirmation — focus lands on Cancel, Escape closes, names object + consequence + recovery.
 */
export interface DialogProps { open?: boolean; title: string; children?: React.ReactNode; confirmLabel?: string; cancelLabel?:
string; onConfirm?: () => void; onCancel?: () => void; destructive?: boolean; inline?: boolean; }
export declare function Dialog(props: DialogProps): JSX.Element;

// ---- EmptyState
import * as React from 'react';
/**
 * Explains why a list is empty and offers one relevant next action.
 * @startingPoint section="States" subtitle="Explains why a list is empty and offers one relevant next action."
viewport="700x400"
 */
export interface EmptyStateProps { title: string; body?: string; action?: React.ReactNode; }
export declare function EmptyState(props: EmptyStateProps): JSX.Element;

// ---- ErrorState
import * as React from 'react';
/**
 * Plain-language failure with a required recovery action and an optional support reference.
 */
export interface ErrorStateProps { title: string; body: string; recoverLabel?: string; onRecover?: () => void; secondaryLabel?:
string; onSecondary?: () => void; reference?: string; }
export declare function ErrorState(props: ErrorStateProps): JSX.Element;
```

```
// ---- ErrorSummary
import * as React from 'react';
export interface ErrorSummaryProps { errors: { field: string; message: string }[]; title?: string; }
export declare const ErrorSummary: (props: ErrorSummaryProps & { ref?: any }) => JSX.Element | null;

// ---- Fields
import * as React from 'react';
/**
 * Label/value definition list for records and context.
 */
export interface FieldsProps { items: { field: string; value: React.ReactNode; mono?: boolean }[]; label?: string; }
export declare function Fields(props: FieldsProps): JSX.Element;

// ---- FocusBar
import * as React from 'react';
/** Floating pill during a focus session: time left, task, Pause, End. */
export interface FocusBarProps { task: string; remaining?: string; onPause?: () => void; onEnd?: () => void; breakDue?: string;
}
export declare function FocusBar(props: FocusBarProps): JSX.Element;

// ---- HealthBadge
import * as React from 'react';
export interface HealthBadgeProps { band?: "healthy" | "watch" | "atrisk" | "unknown"; score?: number; driver?: string;
updated?: string; }
export declare function HealthBadge(props: HealthBadgeProps): JSX.Element;

// ---- Icon
import * as React from 'react';
/**
 * Semester line glyph (Lucide-like, 24 grid, 1.5 stroke) from the app's own icon set — 108 names, see assets/icons/.
 * @startingPoint section="Glyph" subtitle="Semester line glyph (Lucide-like, 24 grid, 1.5 stroke) from the app's own icon s"
viewport="700x400"
 */
export interface IconProps { name: string; size?: number; label?: string; strokeWidth?: number; style?: React.CSSProperties;
className?: string; }
export declare function Icon(props: IconProps): JSX.Element;
export declare const ICON_NAMES: string[];

// ---- IconButton
import * as React from 'react';
/**
 * Icon-only control with a required accessible name (also its tooltip).
 */
export interface IconButtonProps { icon: string; label: string; size?: "md" | "sm"; pressed?: boolean; onClick?: (e: any) =>
void; }
export declare function IconButton(props: IconButtonProps): JSX.Element;

// ---- LoadingState
import * as React from 'react';
/**
 * Still structural placeholders plus a spoken "Loading …" — no spinner or shimmer.
 */
export interface LoadingStateProps { what?: string; bars?: number[]; }
export declare function LoadingState(props: LoadingStateProps): JSX.Element;

// ---- MetricTile
import * as React from 'react';
/** One figure worth reading first, with unit, source and age. */
export interface MetricTileProps { label: string; value: string; unit?: string; source?: string; updated?: string; delta?:
string; }
export declare function MetricTile(props: MetricTileProps): JSX.Element;

// ---- NextSteps
import * as React from 'react';
/**
 * Ordered list of next safe actions, each with one line of why.
 */
export interface NextStepsProps { steps: { label: string; why?: string; icon?: string; aside?: React.ReactNode; onClick?: () =>
void }[]; }
export declare function NextSteps(props: NextStepsProps): JSX.Element;

// ---- Notice
import * as React from 'react';
/**
 * One-line service or context notice (role=status).
 */
export interface NoticeProps { head?: string; children: React.ReactNode; }
```

```
export declare function Notice(props: NoticeProps): JSX.Element;

// ---- ObjectCard
import * as React from 'react';
/**
 * The universal decision card: eyebrow → title → provenance → why → metadata → one primary + one quiet secondary.
 * @startingPoint section="Surfaces" subtitle="The universal decision card: eyebrow → title → provenance → why → metadata →
one" viewport="700x400"
 */
export interface ObjectCardProps { kind?: string; statuses?: string[]; title: string; provenance?: { origin: string; by?:
string; cues?: string[]; age?: string }; explanation?: string; metadata?: string; primary?: { label: string; onClick?: () =>
void }; secondary?: { label: string; onClick?: () => void }; tone?: "pending" | "danger"; level?: 2 | 3 | 4; children?:
React.ReactNode; }
export declare function ObjectCard(props: ObjectCardProps): JSX.Element;

// ---- OfflineStrip
import * as React from 'react';
/**
 * Offline strip that says what still works and that changes are queued.
 */
export interface OfflineStripProps { queued?: number; syncs?: boolean; }
export declare function OfflineStrip(props: OfflineStripProps): JSX.Element;

// ---- OpenIn
import * as React from 'react';
/** "Open in" row — the way from one object to every place it matters (Plan, Study, Calendar). */
export interface OpenInProps { targets: string[]; about: string; onOpen?: (t: string) => void; }
export declare function OpenIn(props: OpenInProps): JSX.Element;

// ---- PageHeader
import * as React from 'react';
export interface PageHeaderProps { kicker?: string; title: string; purpose?: string; source?: React.ReactNode; actions?:
React.ReactNode; }
export declare function PageHeader(props: PageHeaderProps): JSX.Element;

// ---- PermissionNotice
import * as React from 'react';
/**
 * States what changed about access, why, and where it is controlled.
 */
export interface PermissionNoticeProps { changed: string; why: string; controlLabel?: string; onControl?: () => void; }
export declare function PermissionNotice(props: PermissionNoticeProps): JSX.Element;

// ---- PolicyBadge
import * as React from 'react';
export interface PolicyBadgeProps { name: string; version?: string; owner?: string; onOpen?: () => void; }
export declare function PolicyBadge(props: PolicyBadgeProps): JSX.Element;

// ---- ProgressState
import * as React from 'react';
/**
 * A determinate wait (upload, import) with real <progress>, percentage in words, Cancel/Retry.
 */
export interface ProgressStateProps { label: string; done?: number; total?: number; note?: string; failed?: string; onCancel?:
() => void; onRetry?: () => void; }
export declare function ProgressState(props: ProgressStateProps): JSX.Element;

// ---- ProvenanceChart
import * as React from 'react';
/** Bar chart that encodes provenance: solid official/connected, outline yours, hatched estimated/AI, dotted stale. */
export interface ProvenanceChartProps { title: string; rows: { label: string; value: number; source?: string }[]; unit?:
string; max?: number; source?: string; demo?: boolean; }
export declare function ProvenanceChart(props: ProvenanceChartProps): JSX.Element;

// ---- ProvenanceChips
import * as React from 'react';
/**
 * A fact's origin chip plus at most two cues and its age, spoken as one sentence.
 */
export interface ProvenanceChipsProps { origin: string; by?: string; cues?: string[]; age?: string; }
export declare function ProvenanceChips(props: ProvenanceChipsProps): JSX.Element;

// ---- QuickActions
import * as React from 'react';
/** Row of 2–5 labelled starting actions for empty and onboarding states. */
export interface QuickActionsProps { actions: { label: string; icon?: string; onClick?: () => void }[]; }
export declare function QuickActions(props: QuickActionsProps): JSX.Element;
```

```
// ---- SaveState
import * as React from 'react';
/**
 * The quiet "Saving… → Saved" line near the work, in a polite live region.
 */
export interface SaveStateProps { status?: string; }
export declare function SaveState(props: SaveStateProps): JSX.Element;

// ---- ScreenGuide
import * as React from 'react';
/** "About this screen" disclosure at the foot of every screen — what it is for, where its data comes from, who decides. */
export interface ScreenGuideProps { items: { q: string; a: string }[]; open?: boolean; }
export declare function ScreenGuide(props: ScreenGuideProps): JSX.Element;

// ---- SectionLabel
import * as React from 'react';
/**
 * The small uppercase kicker that heads a section, with an optional right-aligned count.
 */
export interface SectionLabelProps { children: React.ReactNode; aside?: React.ReactNode; as?: "h2" | "h3" | "h4"; style?:
React.CSSProperties; }
export declare function SectionLabel(props: SectionLabelProps): JSX.Element;

// ---- Segmented
import * as React from 'react';
/**
 * 2–4 mutually exclusive view choices (e.g. Cards / Rows / Timeline).
 */
export interface SegmentedProps { options: (string | { value: string; label: string })[]; value: string; onChange?: (v: string)
=> void; label: string; }
export declare function Segmented(props: SegmentedProps): JSX.Element;

// ---- Select
import * as React from 'react';
/**
 * Native select drawn in Semester's palette.
 */
export interface SelectProps { label: string; hint?: string; options: (string | { value: string; label: string })[]; value?:
string; onChange?: (e: any) => void; id?: string; }
export declare function Select(props: SelectProps): JSX.Element;

// ---- Sheet
import * as React from 'react';
/** Bottom sheet on mobile, window on desktop; the home of "Source & details". Also exports `SourceDetails`. */
export interface SheetProps { title: string; children?: React.ReactNode; onClose?: () => void; inline?: boolean; actions?:
React.ReactNode; }
export declare function Sheet(props: SheetProps): JSX.Element;
export declare function SourceDetails(props: { items: { label: string; value: React.ReactNode }[] }): JSX.Element;

// ---- SideRail
import * as React from 'react';
/**
 * Desktop side rail on dark Ink chrome — persistent, quiet, labelled.
 * @startingPoint section="Navigation" subtitle="Desktop side rail on dark Ink chrome — persistent, quiet, labelled."
viewport="700x400"
 */
export interface SideRailProps { groups: { label?: string; items: { id: string; label: string; icon: string; count?: number }[]
}[]; current?: string; onNavigate?: (id: string) => void; foot?: React.ReactNode; brand?: boolean; }
export declare function SideRail(props: SideRailProps): JSX.Element;

// ---- SourceBadge
import * as React from 'react';
/**
 * Where a fact came from, beside the fact — controlled vocabulary (glyph + word + tone).
 * @startingPoint section="Trust & provenance" subtitle="Where a fact came from, beside the fact — controlled vocabulary (glyph
+ word + " viewport="700x400"
 */
export interface SourceBadgeProps { source: "official" | "connected" | "imported" | "personal" | "ai" | "estimated" | "review"
| "stale" | "external" | "sample"; updated?: string; by?: string; onReport?: () => void; }
export declare function SourceBadge(props: SourceBadgeProps): JSX.Element;

// ---- SourceLine
import * as React from 'react';
/** The Source Line — Semester's signature device: a thin editorial rule from a statement to its source chip. */
export interface SourceLineProps { children: React.ReactNode; source?: string; by?: string; when?: string }
export declare function SourceLine(props: SourceLineProps): JSX.Element;
```

```
// ---- StatusChip
import * as React from 'react';
/**
 * One status chip — glyph + word + tone from the shared status table.
 */
export interface StatusChipProps { status: string; label?: string; }
export declare function StatusChip(props: StatusChipProps): JSX.Element;

// ---- StepStatus
import * as React from 'react';
/**
 * A process in named steps — shows the AI or system working instead of faking certainty.
 */
export interface StepStatusProps { steps: { label: string; state: "waiting" | "working" | "done" | "failed" }[]; label?:
string; }
export declare function StepStatus(props: StepStatusProps): JSX.Element;

// ---- SuccessState
import * as React from 'react';
/**
 * Respectful acknowledgement that points at what is next — no confetti, streaks or ranks.
 */
export interface SuccessStateProps { title: string; body?: string; nextLabel?: string; onNext?: () => void; }
export declare function SuccessState(props: SuccessStateProps): JSX.Element;

// ---- SupportHandoff
import * as React from 'react';
export interface SupportHandoffProps { office: string; reason?: string; channel?: string; hours?: string; status?: "ready" |
"sent" | "received" | "resolved"; reference?: string; onStart?: () => void; }
export declare function SupportHandoff(props: SupportHandoffProps): JSX.Element;

// ---- Switch
import * as React from 'react';
/**
 * On/off setting that speaks its state as a word beside the track.
 */
export interface SwitchProps { checked?: boolean; onChange?: (v: boolean) => void; label: string; disabled?: boolean; }
export declare function Switch(props: SwitchProps): JSX.Element;

// ---- SystemContextBar
import * as React from 'react';
/**
 * Persistent top bar: location, term, ⌘K search, sync state, quiet tools.
 */
export interface SystemContextBarProps { location?: string; term?: string; sync?: string; onSearch?: () => void; actions?:
React.ReactNode; }
export declare function SystemContextBar(props: SystemContextBarProps): JSX.Element;

// ---- TabBar
import * as React from 'react';
/**
 * Mobile bottom navigation for the five canonical destinations, always labelled.
 */
export interface TabBarProps { items: { id: string; label: string; icon: string }[]; current?: string; onNavigate?: (id:
string) => void; }
export declare function TabBar(props: TabBarProps): JSX.Element;

// ---- TextField
import * as React from 'react';
/**
 * Labelled input/textarea with associated hint and error — the label is always visible.
 * @startingPoint section="Forms" subtitle="Labelled input/textarea with associated hint and error — the label is always vis"
viewport="700x400"
 */
export interface TextFieldProps { label: string; hint?: string; error?: string; multiline?: boolean; id?: string; value?:
string; placeholder?: string; onChange?: (e: any) => void; disabled?: boolean; }
export declare function TextField(props: TextFieldProps): JSX.Element;

// ---- UndoToast
import * as React from 'react';
/** One-sentence confirmation with Undo for reversible work — instead of an "are you sure?" dialog. */
export interface UndoToastProps { message: string; onUndo?: () => void; undoLabel?: string; }
export declare function UndoToast(props: UndoToastProps): JSX.Element;

// ---- VisibilityPicker
import * as React from 'react';
```

```
/** "Who can see this?" — the one privacy pattern, with the same four answers in every module. */
export interface VisibilityPickerProps { value?: "only-me" | "course" | "collaborators" | "portfolio"; onChange?: (v: string)
=> void; allowed?: string[]; locked?: boolean; origin?: "student-entered" | "institution-provided" | "connected-system";
onLearnMore?: () => void; }
export declare function VisibilityPicker(props: VisibilityPickerProps): JSX.Element;

// ---- Wordmark
import * as React from 'react';
/**
 * The Semester lockup — three-slab mark + SEMESTER in Barlow Condensed caps; also exports `Mark`.
 */
export interface WordmarkProps { size?: number; metal?: boolean; showWord?: boolean; style?: React.CSSProperties; }
export declare function Wordmark(props: WordmarkProps): JSX.Element;
export declare function Mark(props: { size?: number; style?: React.CSSProperties; className?: string }): JSX.Element;
```

Component sources

The JSX source of every component in the bundle.

components/core/Avatar.jsx · 10 lines

```
import React from 'react';
import { Icon } from './Icon.jsx';
const initials = (n = '') => n.trim().split(/\s+/).filter(Boolean).slice(0, 2).map((p) => p[0].toUpperCase()).join('');
/** Initials in a tinted square that follows the corner setting — never a photo, never derived from an email. Empty name →
profile glyph. */
export function Avatar({ name = '', size = 40 }) {
  const l = initials(name);
  if (!l) return <Icon name="profile" size={size} />;
  return <span className="avatar" aria-hidden="true" style={{ width: size, height: size, fontSize: Math.round(size * 0.4) }}>
{l}</span>;
}
```

components/core/Button.jsx · 14 lines

```
import React from 'react';
import { Icon } from './Icon.jsx';
/** One visually dominant (primary) action per decision region. */
export function Button({ variant = 'secondary', size = 'md', icon, iconAfter, block = false, loading = false, disabled,
children, type = 'button', ...rest }) {
  const cls = ['btn', variant === 'primary' && 'btn-primary', variant === 'ghost' && 'btn-ghost', variant === 'danger' && 'btn-
danger', size === 'sm' && 'btn-sm', block && 'btn-block'].filter(Boolean).join(' ');
  return (
    <button type={type} className={cls} disabled={disabled || loading} aria-busy={loading || undefined} {...rest}>
      {icon && <Icon name={icon} size={size === 'sm' ? 15 : 17} />}
      <span>{loading ? 'Working…' : children}</span>
      {iconAfter && <Icon name={iconAfter} size={size === 'sm' ? 15 : 17} />}
    </button>
  );
}
```

components/core/Icon.jsx · 13 lines

```
import React from 'react';
import { ICON_SHAPES } from '../icons/iconShapes.js';
/** Semester glyph — Lucide-like, 24 grid, stroke 1.5, round caps. Decorative unless `label` is given. */
export function Icon({ name, size = 19, label, strokeWidth = 1.5, style, className }) {
  const shapes = ICON_SHAPES[name] || [];
  return React.createElement('svg', {
    width: size, height: size, viewBox: '0 0 24 24', fill: 'none', stroke: 'currentColor', strokeWidth,
    strokeLinecap: 'round', strokeLinejoin: 'round', className, style: { display: 'block', flex: 'none', ...style },
    'aria-hidden': label ? undefined : 'true', role: label ? 'img' : undefined, 'aria-label': label,
  }, shapes.map((s, i) => s.c ? React.createElement('circle', { key: i, cx: s.c[0], cy: s.c[1], r: s.c[2] }) :
React.createElement('path', { key: i, d: s.d })));
}
export const ICON_NAMES = Object.keys(ICON_SHAPES);
```

components/core/IconButton.jsx · 11 lines

```
import React from 'react';
import { Icon } from './Icon.jsx';
/** Icon-only control. `label` is required — it is the accessible name and the tooltip. */
export function IconButton({ icon, label, size = 'md', pressed, ...rest }) {
  return (
    <button type="button" className={'btn btn-icon' + (size === 'sm' ? ' btn-sm' : '')} aria-label={label} title={label} aria-
pressed={pressed} {...rest}>
      <Icon name={icon} size={size === 'sm' ? 16 : 19} />
    </button>
```

```
  );
}
```

components/core/SectionLabel.jsx · 12 lines

```
import React from 'react';
/** The small caps kicker over a section, with an optional count or aside on the right. */
export function SectionLabel({ children, aside, as = 'h2', style }) {
  const H = as;
  return (
    <div style={{ display: 'flex', alignItems: 'baseline', justifyContent: 'space-between', gap: 'var(--sp-4)', marginBottom:
'var(--sp-4)', ...style }}>
      <H className="kicker" style={{ margin: 0, fontFamily: 'var(--font-body)', fontWeight: 500 }}>{children}</H>
      {aside != null && <span className="kicker nums" style={{ letterSpacing: 0 }}>{aside}</span>}
    </div>
  );
}
```

components/core/Wordmark.jsx · 23 lines

```
import React from 'react';
import { MARK_VIEWBOX, MARK_PATHS } from './markPaths.js';
/** The Semester mark (three slabs) — flat currentColor silhouette. */
export function Mark({ size = 24, style, className }) {
  const [, , w, h] = MARK_VIEWBOX.split(' ').map(Number);
  return (
    <svg width={(size * w) / h} height={size} viewBox={MARK_VIEWBOX} fill="currentColor" aria-hidden="true" focusable="false"
className={className} style={{ display: 'block', flex: 'none', ...style }}>
      {MARK_PATHS.map((d) => <path key={d} d={d} />)}
    </svg>
  );
}
/** Mark + SEMESTER lockup. `metal` paints the brushed sterling used on dark chrome. */
export function Wordmark({ size = 20, metal = false, showWord = true, style }) {
  return (
    <span aria-label="Semester" role="img" style={{ display: 'inline-flex', alignItems: 'center', gap: size * 0.5, color: metal
? 'var(--sterling)' : 'currentColor', ...style }}>
      <Mark size={size} />
      {showWord && (
        <span aria-hidden="true" className={metal ? 'chrome-text' : undefined} style={{ fontFamily: 'var(--font-heading)',
fontWeight: 600, textTransform: 'uppercase', letterSpacing: 'var(--tracking-wordmark)', fontSize: size * 0.82, lineHeight: 1,
paddingTop: 1 }}>Semester</span>
      )}
    </span>
  );
}
```

components/data/DataTable.jsx · 14 lines

```
import React from 'react';
/** Crisp comparison table. Use only when comparing columns; the region (not the page) scrolls on compact screens. */
export function DataTable({ columns = [], rows = [], caption }) {
  return (
    <div className="table-wrap" role="region" aria-label={caption} tabIndex={0}>
      <table className="table">
        {caption && <caption className="sr-only">{caption}</caption>}
        <thead><tr>{columns.map((c) => <th key={c.key} className={c.numeric ? 'num' : undefined} scope="col">{c.label}</th>)}
</tr></thead>
        <tbody>{rows.map((r, i) => <tr key={r.id || i}>{columns.map((c) => <td key={c.key} className={c.numeric ? 'num' :
undefined}>{c.render ? c.render(r) : r[c.key]}</td>)}</tr>)}</tbody>
      </table>
    </div>
  );
}
```

components/data/DecisionTrail.jsx · 15 lines

```
import React from 'react';
/** Auditable timeline: who did what, when, under which authority. States: done, current, waiting, blocked. */
export function DecisionTrail({ items = [], label = 'Decision trail' }) {
```

```
  return (
    <ol className="trail" aria-label={label}>
      {items.map((it, i) => (
        <li key={i} className="trail-item" data-state={it.state || 'done'} aria-current={it.state === 'current' ? 'step' :
undefined}>
          <span className="trail-dot" aria-hidden="true"></span>
          <div><div className="trail-title">{it.title}</div><div className="trail-meta">{[it.who,
it.when].filter(Boolean).join(' · ')}</div>{it.detail && <div style={{ fontSize: 'var(--type-sm)', color: 'var(--text-
secondary)', marginTop: 'var(--sp-2)' }}>{it.detail}</div>}</div>
        </li>
      ))}
    </ol>
  );
}
```

components/data/Fields.jsx · 10 lines

```
import React from 'react';
/** Label/value definition list — the console's record and context-bar layout. */
export function Fields({ items = [], label }) {
  return (
    <dl className="fields" aria-label={label}>
      {items.map((it) => <React.Fragment key={it.field}><dt>{it.field}</dt><dd className={it.mono ? 'mono' : undefined}>
{it.value}</dd></React.Fragment>)}
    </dl>
  );
}
```

components/data/MetricTile.jsx · 14 lines

```
import React from 'react';
import { SourceBadge } from '../trust/SourceBadge.jsx';
/** One figure worth reading first, with its unit, its source and how old it is. Never an outcome claim. */
export function MetricTile({ label, value, unit, source, updated, delta }) {
  return (
    <div className="metric">
      <div className="kicker">{label}</div>
      <div><span className="metric-value">{value}</span>{unit && <span style={{ marginLeft: 6, fontSize: 'var(--type-sm)',
color: 'var(--text-secondary)' }}>{unit}</span>}</div>
      {delta && <div className="metric-delta">{delta}</div>}
      {source && <SourceBadge source={source} updated={updated} />}
    </div>
  );
}
```

components/data/ProvenanceChart.jsx · 24 lines

```
import React from 'react';
const ENC = { official: 'solid', connected: 'solid', personal: 'outline', estimated: 'hatched', ai: 'hatched', stale: 'dotted'
};
const SAID = { solid: 'Official or connected', outline: 'Yours', hatched: 'Estimated or AI-derived', dotted: 'Out of date' };
/** Bar chart that encodes provenance, not just value: solid = official/connected, outline = yours, hatched = estimated/AI,
dotted = stale. Every bar labelled directly. */
export function ProvenanceChart({ title, rows = [], unit = '', max, source, demo = false }) {
  const m = max || Math.max(...rows.map((r) => r.value)) || 1;
  const used = [...new Set(rows.map((r) => ENC[r.source] || 'solid'))];
  return (
    <figure className="chart" style={{ margin: 0 }}>
      <figcaption className="chart-title">{title}</figcaption>
      <div className="chart-bars">
        {rows.map((r) => (
          <div className="chart-row" key={r.label}>
            <span>{r.label}</span>
            <div className="chart-track" aria-hidden="true"><div className="chart-bar" data-enc={ENC[r.source] || 'solid'}
style={{ width: (r.value / m) * 100 + '%' }}></div></div>
            <span className="nums" style={{ textAlign: 'right' }}>{r.value}{unit}</span>
          </div>
        ))}
      </div>
      <div className="chart-foot">{used.map((e) => <span key={e}><span className="chart-bar legend-swatch" data-enc={e}></span>
{SAID[e]}</span>)}{source && <span>Source: {source}</span>}{demo && <span>Illustrative demo data</span>}</div>
```

```
    </figure>
  );
}
```

components/feedback/EmptyState.jsx · 12 lines

```
import React from 'react';
/** Say why it is empty, and offer one relevant next action. */
export function EmptyState({ title, body, action }) {
  return (
    <div className="state" data-kind="empty">
      <div className="state-title">{title}</div>
      {body && <p className="state-body">{body}</p>}
      {action && <div className="state-actions">{action}</div>}
    </div>
  );
}
```

components/feedback/ErrorState.jsx · 16 lines

```
import React from 'react';
/** What went wrong in plain words, and the way out. A recovery action is required. */
export function ErrorState({ title, body, recoverLabel = 'Try again', onRecover, secondaryLabel, onSecondary, reference }) {
  return (
    <div className="state" data-kind="error" role="alert">
      <div className="state-title"><span className="status-glyph" aria-hidden="true">!</span>{title}</div>
      <p className="state-body">{body}</p>
      <div className="state-actions">
        <button type="button" className="btn btn-primary" onClick={onRecover}>{recoverLabel}</button>
        {secondaryLabel && <button type="button" className="btn btn-ghost" onClick={onSecondary}>{secondaryLabel}</button>}
      </div>
      {reference && <p className="state-ref nums">Reference: <span className="mono">{reference}</span></p>}
    </div>
  );
}
```

components/feedback/LoadingState.jsx · 11 lines

```
import React from 'react';
/** Still structural placeholders + a spoken sentence. No spinner, no shimmer. */
export function LoadingState({ what = 'this screen', bars = [24, 14, 72, 72] }) {
  return (
    <div className="state-loading" aria-busy="true">
      <p className="sr-only" role="status">Loading {what}…</p>
      {bars.map((h, i) => <div key={i} className="state-loading-bar" style={{ height: h, width: i === 1 ? '40%' : i === 0 ?
'62%' : '100%' }} aria-hidden="true"></div>)}
    </div>
  );
}
```

components/feedback/Notice.jsx · 6 lines

```
import React from 'react';
/** A one-line service or context notice. role=status — quiet, never a giant red banner. */
export function Notice({ head, children }) {
  return <p className="notice" role="status">{head && <strong className="notice-head">{head}</strong>}{children}</p>;
}
```

components/feedback/OfflineStrip.jsx · 7 lines

```
import React from 'react';
/** Says what still works and that work is queued. */
export function OfflineStrip({ queued = 0, syncs = true }) {
  const tail = !syncs ? '; everything is kept on this device.' : queued > 0 ? ` — ${queued} change${queued === 1 ? '' : 's'}
will sync when you are back.` : '; changes sync when you are back.';
  return <div className="state" data-kind="offline" role="status"><span className="status-glyph" aria-hidden="true">⊘</span>
```

```
<span>You are offline. You can keep working{tail}</span></div>;
}
```

components/feedback/PermissionNotice.jsx · 12 lines

```
import React from 'react';
/** What changed about access, why, who controls it, and where to change it. */
export function PermissionNotice({ changed, why, controlLabel, onControl }) {
  return (
    <div className="state" data-kind="permission" role="status">
      <div className="state-title"><span className="status-glyph" aria-hidden="true">⊙</span>{changed}</div>
      <p className="state-body">{why}</p>
      {controlLabel && <button type="button" className="link-quiet" style={{ alignSelf: 'flex-start' }} onClick={onControl}>
{controlLabel}</button>}
    </div>
  );
}
```

components/feedback/ProgressState.jsx · 17 lines

```
import React from 'react';
/** A determinate wait — an upload, an import. Real <progress>, percentage in words, Cancel/Retry. */
export function ProgressState({ label, done = 0, total = 100, note, failed, onCancel, onRetry }) {
  const pct = total > 0 ? Math.min(100, Math.round((done / total) * 100)) : 0;
  return (
    <div className="state">
      <div className="state-progress-head"><span>{label}</span><span className="nums">{failed ? 'Stopped' : pct + '%'}</span>
</div>
      {failed ? <p className="state-body" role="alert">{failed}</p> : <progress className="state-progress-bar" value={done}
max={total} aria-label={label}></progress>}
      {note && <p className="state-note">{note}</p>}
      <div className="state-actions">
        {failed && onRetry && <button type="button" className="btn btn-primary btn-sm" onClick={onRetry}>Retry</button>}
        {!failed && onCancel && pct < 100 && <button type="button" className="btn btn-ghost btn-sm" onClick=
{onCancel}>Cancel</button>}
      </div>
    </div>
  );
}
```

components/feedback/SaveState.jsx · 8 lines

```
import React from 'react';
import { STATUSES } from '../trust/vocabulary.js';
/** A quiet save/sync line near the work — not a toast. Polite live region; offline/conflict interrupt. */
export function SaveState({ status = 'saved' }) {
  const s = STATUSES[status] || STATUSES.unknown;
  return <span className="save-state" data-tone={s.tone} role={s.urgent ? 'alert' : 'status'} aria-live={s.urgent ? 'assertive'
: 'polite'}><span className="status-glyph" aria-hidden="true">{s.glyph}</span>{s.label}</span>;
}
```

components/feedback/StepStatus.jsx · 16 lines

```
import React from 'react';
const GLYPH = { waiting: '○', working: '…', done: '✓', failed: '!' };
const SAID = { waiting: 'waiting', working: 'in progress', done: 'done', failed: 'failed' };
/** A process in named steps — "Gathering sources → Drafting → Ready". Shows the work instead of faking certainty. */
export function StepStatus({ steps = [], label = 'Progress' }) {
  return (
    <ol className="state-steps" aria-label={label}>
      {steps.map((s) => (
        <li key={s.label} data-state={s.state} aria-current={s.state === 'working' ? 'step' : undefined}>
          <span className="status-glyph" aria-hidden="true" style={{ width: '1em', textAlign: 'center' }}>{GLYPH[s.state]}
</span>{s.label}<span className="sr-only"> — {SAID[s.state]}</span>
        </li>
      ))}
    </ol>
  );
}
```

components/feedback/SuccessState.jsx · 12 lines

```
import React from 'react';
/** Acknowledge, then point at what is next. Respectful, never celebratory. */
export function SuccessState({ title, body, nextLabel, onNext }) {
  return (
    <div className="state" data-kind="success" role="status">
      <div className="state-title"><span className="status-glyph" aria-hidden="true">✓</span>{title}</div>
      {body && <p className="state-body">{body}</p>}
      {nextLabel && <p className="state-body">Next: <button type="button" className="link-quiet" style={{ fontSize: 'inherit'
}} onClick={onNext}>{nextLabel}</button></p>}
    </div>
  );
}
```

components/forms/Choice.jsx · 11 lines

```
import React from 'react';
/** Radio or checkbox with a label and an optional sentence explaining the consequence. */
export function Choice({ type = 'radio', label, about, checked, onChange, name, value, disabled }) {
  return (
    <label className="choice" style={disabled ? { opacity: 0.5, cursor: 'not-allowed' } : undefined}>
      <input type={type} name={name} value={value} checked={checked} onChange={onChange} disabled={disabled} />
      <span>{label}{about && <span className="choice-about">{about}</span>}</span>
    </label>
  );
}
```

components/forms/Combobox.jsx · 22 lines

```
import React, { useId, useState } from 'react';
/** Text box with a filtered list beneath — course search, people, rooms. Arrow keys move an edge + weight, not colour alone.
*/
export function Combobox({ label, hint, options = [], value = '', onChange, onSelect, placeholder, defaultOpen = false }) {
  const id = useId(); const [q, setQ] = useState(value); const [open, setOpen] = useState(defaultOpen); const [sel, setSel] =
useState(0);
  const shown = options.filter((o) => (o.label + ' ' + (o.detail || '')).toLowerCase().includes(q.toLowerCase())).slice(0, 8);
  const pick = (o) => { setQ(o.label); setOpen(false); onSelect && onSelect(o); };
  return (
    <div className="combo">
      <label className="field-label" htmlFor={id}>{label}</label>
      {hint && <span className="field-hint">{hint}</span>}
      <input id={id} className="input" role="combobox" aria-expanded={open} aria-controls={id + '-list'} aria-
autocomplete="list" value={q} placeholder={placeholder}
        onChange={(e) => { setQ(e.target.value); setOpen(true); setSel(0); onChange && onChange(e.target.value); }} onFocus={()
=> setOpen(true)}
        onKeyDown={(e) => { if (e.key === 'ArrowDown') { e.preventDefault(); setSel((s) => Math.min(shown.length - 1, s + 1));
} if (e.key === 'ArrowUp') { e.preventDefault(); setSel((s) => Math.max(0, s - 1)); } if (e.key === 'Enter' && shown[sel])
pick(shown[sel]); if (e.key === 'Escape') setOpen(false); }} />
      {open && shown.length > 0 && (
        <ul className="combo-list" id={id + '-list'} role="listbox">
          {shown.map((o, i) => <li key={o.label} role="option" aria-selected={i === sel} className="combo-option" onMouseEnter=
{() => setSel(i)} onMouseDown={(e) => { e.preventDefault(); pick(o); }}>{o.label}{o.detail && <span className="combo-detail">
{o.detail}</span>}</li>)}
        </ul>
      )}
    </div>
  );
}
```

components/forms/DateField.jsx · 14 lines

```
import React, { useId } from 'react';
/** Native date (and optional time) input with a visible label and the date spoken back unambiguously. */
export function DateField({ label, hint, value, onChange, withTime = false, said }) {
  const id = useId();
  return (
    <div className="field">
      <label className="field-label" htmlFor={id}>{label}</label>
      {hint && <span className="field-hint">{hint}</span>}
      <input id={id} className="input nums" type={withTime ? 'datetime-local' : 'date'} value={value} onChange={onChange} />
```

```
      {said && <span className="field-hint nums">{said}</span>}
    </div>
  );
}
```

components/forms/Segmented.jsx · 13 lines

```
import React from 'react';
/** 2–4 mutually exclusive views (Cards / Rows / Timeline). */
export function Segmented({ options = [], value, onChange, label }) {
  return (
    <div className="seg" role="radiogroup" aria-label={label}>
      {options.map((o) => {
        const v = typeof o === 'string' ? o : o.value; const l = typeof o === 'string' ? o : o.label;
        return <button key={v} type="button" role="radio" aria-checked={v === value} className="seg-opt" onClick={() =>
onChange && onChange(v)}>{l}</button>;
      })}
    </div>
  );
}
```

components/forms/Select.jsx · 15 lines

```
import React, { useId } from 'react';
/** Native select, drawn in Semester's palette. Never imitate a select with a menu. */
export function Select({ label, hint, options = [], id, ...rest }) {
  const auto = useId(); const fid = id || auto;
  return (
    <div className="field">
      <label className="field-label" htmlFor={fid}>{label}</label>
      {hint && <span className="field-hint">{hint}</span>}
      <select id={fid} className="input" {...rest}>
        {options.map((o) => typeof o === 'string' ? <option key={o} value={o}>{o}</option> : <option key={o.value} value=
{o.value}>{o.label}</option>)}
      </select>
    </div>
  );
}
```

components/forms/Switch.jsx · 12 lines

```
import React from 'react';
/** On/off setting. The state is spoken as a word beside the track, never carried by colour alone. */
export function Switch({ checked = false, onChange, label, disabled }) {
  return (
    <button type="button" role="switch" aria-checked={checked} className="switch" disabled={disabled} onClick={() => onChange
&& onChange(!checked)} style={{ background: 'none', border: 0, padding: 0, color: 'var(--text-primary)', textAlign: 'left' }}>
      <span className="switch-track" aria-hidden="true"></span>
      <span className="switch-state" aria-hidden="true">{checked ? 'On' : 'Off'}</span>
      <span>{label}</span>
    </button>
  );
}
```

components/forms/TextField.jsx · 15 lines

```
import React, { useId } from 'react';
/** Labelled text input. Persistent visible label; hint and error are programmatically associated. */
export function TextField({ label, hint, error, multiline = false, id, ...rest }) {
  const auto = useId(); const fid = id || auto; const hid = fid + '-hint'; const eid = fid + '-err';
  const El = multiline ? 'textarea' : 'input';
  return (
    <div className="field">
      <label className="field-label" htmlFor={fid}>{label}</label>
      {hint && <span className="field-hint" id={hid}>{hint}</span>}
      <El id={fid} className="input" aria-invalid={error ? 'true' : undefined} aria-describedby={[hint && hid, error &&
eid].filter(Boolean).join(' ') || undefined} {...rest} />
      {error && <span className="field-error" id={eid}><span aria-hidden="true">!</span>{error}</span>}
    </div>
```

```
  );
}
```

components/forms/VisibilityPicker.jsx · 25 lines

```
import React, { useId } from 'react';
const AUDIENCES = [
  { id: 'only-me', label: 'Only me', about: 'Nobody else can see it.' },
  { id: 'course', label: 'Course', about: 'People enrolled in the course it is attached to.' },
  { id: 'collaborators', label: 'Selected collaborators', about: 'Only the people you add by name.' },
  { id: 'portfolio', label: 'Public portfolio', about: 'Anyone with the link to your portfolio.' },
];
const ORIGIN = { 'student-entered': 'Entered by you', 'institution-provided': 'Provided by your institution', 'connected-
system': 'From a system you connected' };
/** "Who can see this?" — the one privacy pattern, same four answers in every module. Only offers audiences the module can
honour. */
export function VisibilityPicker({ value = 'only-me', onChange, allowed = ['only-me', 'course', 'collaborators', 'portfolio'],
locked = false, origin = 'student-entered', onLearnMore }) {
  const name = useId(); const chosen = AUDIENCES.find((a) => a.id === value) || AUDIENCES[0];
  return (
    <div className="visibility">
      {locked ? <p className="visibility-said"><span className="kicker">Who can see this</span>{chosen.label} — {chosen.about}
</p> : (
        <fieldset><legend className="kicker" style={{ marginBottom: 'var(--sp-2)' }}>Who can see this?</legend>
          {AUDIENCES.filter((a) => allowed.includes(a.id)).map((a) => (
            <label key={a.id} className="choice"><input type="radio" name={name} checked={value === a.id} onChange={() =>
onChange && onChange(a.id)} /><span>{a.label}<span className="choice-about">{a.about}</span></span></label>
          ))}
        </fieldset>
      )}
      <p className="visibility-said"><span className="kicker">Data source</span>{ORIGIN[origin]} · <button type="button"
className="link-quiet" onClick={onLearnMore}>Learn more</button></p>
    </div>
  );
}
```

components/governance/ApprovalBanner.jsx · 8 lines

```
import React from 'react';
/** Shows where a consequential change is in its approval path. */
export function ApprovalBanner({ state = 'required', what, approver, requestedBy, onRequest, onApprove, onReject }) {
  const L = { required: ['?', 'Needs approval'], pending: ['…', 'Waiting for approval'], approved: ['✓', 'Approved'], rejected:
['⊘', 'Not approved'] };
  const [g, t] = L[state] || L.required;
  return <div className="approval-banner" data-state={state} role="status"><span className="gov-glyph" aria-hidden="true">{g}
</span><div className="approval-body"><b>{t}: {what}</b><span className="gov-sub">{state === 'pending' && approver ? 'With ' +
approver : state === 'required' ? 'A second person must approve before this applies.' : requestedBy ? 'Requested by ' +
requestedBy : ''}</span></div><div className="approval-actions">{state === 'required' && onRequest && <button type="button"
className="btn btn-sm btn-primary" onClick={onRequest}>Request approval</button>}{state === 'pending' && onApprove && <button
type="button" className="btn btn-sm btn-primary" onClick={onApprove}>Approve</button>}{state === 'pending' && onReject &&
<button type="button" className="btn btn-sm" onClick={onReject}>Reject</button>}</div></div>;
}
```

components/governance/ConsentBadge.jsx · 8 lines

```
import React from 'react';
/** Shows who can see something because the student shared it, until when. */
export function ConsentBadge({ state = 'private', recipient, expires, onManage }) {
  const L = { private: ['○', 'Only you'], shared: ['⇄', 'Shared with ' + (recipient || 'someone')], expired: ['⊘', 'Share
expired'], revoked: ['⊘', 'Share revoked'], required: ['?', 'Needs your consent'] };
  const [g, t] = L[state] || L.private;
  return <span className="gov-badge" data-kind="consent" data-state={state}><span className="gov-glyph" aria-hidden="true">{g}
</span><span>{t}</span>{expires && state === 'shared' && <span className="gov-sub">· until {expires}</span>}{onManage &&
<button type="button" className="gov-link" onClick={onManage}>Manage</button>}</span>;
}
```

components/governance/ErrorSummary.jsx · 7 lines

```
import React from 'react';
/** Form error summary: count, each error linked to its field. Focus moves here on submit. */
```

```
export const ErrorSummary = React.forwardRef(function ErrorSummary({ errors = [], title }, ref) {
  if (!errors.length) return null;
  return <div className="error-summary" role="alert" tabIndex={-1} ref={ref}><b>! {title || (errors.length + (errors.length > 1
? ' fields need' : ' field needs') + ' attention')}</b><ul>{errors.map((e) => <li key={e.field}><a href={'#' + e.field}
onClick={(ev) => { const el = document.getElementById(e.field); if (el) { ev.preventDefault(); el.focus(); } }}>{e.message}</a>
</li>)}</ul></div>;
});
```

components/governance/HealthBadge.jsx · 8 lines

```
import React from 'react';
/** Health band with the score's top driver — for tenants, integrations, customers. */
export function HealthBadge({ band = 'healthy', score, driver, updated }) {
  const L = { healthy: ['✓', 'Healthy', 'success'], watch: ['?', 'Watch', 'pending'], atrisk: ['!', 'At risk', 'danger'],
unknown: ['…', 'Not enough data', 'neutral'] };
  const [g, t, tone] = L[band] || L.unknown;
  return <span className="health-badge" data-tone={tone}><span className="gov-glyph" aria-hidden="true">{g}</span><b>{t}</b>
{score != null && <span className="nums">{score}</span>}{driver && <span className="gov-sub">· {driver}</span>}{updated &&
<span className="gov-sub">· {updated}</span>}</span>;
}
```

components/governance/PageHeader.jsx · 6 lines

```
import React from 'react';
/** Page head: kicker, one H1, what the page helps you do, source line, primary action. */
export function PageHeader({ kicker, title, purpose, source, actions }) {
  return <header className="page-header"><div className="page-header-text">{kicker && <div className="kicker">{kicker}</div>}
<h1>{title}</h1>{purpose && <p className="page-header-purpose">{purpose}</p>}{source && <div className="page-header-source">
{source}</div>}</div>{actions && <div className="page-header-actions">{actions}</div>}</header>;
}
```

components/governance/PolicyBadge.jsx · 7 lines

```
import React from 'react';
/** Names the policy that governs what is shown, with its owner and version. */
export function PolicyBadge({ name, version, owner, onOpen }) {
  const text = <><span className="gov-glyph" aria-hidden="true">§</span><span>{name}{version ? ' v' + version : ''}</span>
{owner && <span className="gov-sub">· {owner}</span>}</>;
  return onOpen ? <button type="button" className="gov-badge" data-kind="policy" onClick={onOpen} aria-label={'Policy: ' + name
+ (version ? ' version ' + version : '') + (owner ? ', owned by ' + owner : '')}>{text}</button> : <span className="gov-badge"
data-kind="policy">{text}</span>;
}
```

components/governance/SupportHandoff.jsx · 7 lines

```
import React from 'react';
/** Hands a question to the office that owns it; Semester tracks status only. */
export function SupportHandoff({ office, reason, channel, hours, status = 'ready', reference, onStart }) {
  const L = { ready: 'Not started', sent: 'Sent', received: 'Received by the office', resolved: 'Resolved' };
  return <div className="support-handoff"><div className="gov-sub">Owned by</div><b className="handoff-office">{office}</b>
{reason && <p className="handoff-reason">{reason}</p>}<dl className="handoff-meta">{channel && <><dt>How</dt><dd>{channel}</dd>
</>}{hours && <><dt>Hours</dt><dd>{hours}</dd></>}<dt>Status</dt><dd>{L[status] || status}{reference ? ' · ' + reference : ''}
</dd></dl>{status === 'ready' && onStart && <button type="button" className="btn btn-sm btn-primary" onClick={onStart}>
{'Contact ' + office}</button>}<p className="gov-sub">The office decides in its own system. Semester shows the status and never
edits it.</p></div>;
}
```

components/navigation/CommandPalette.jsx · 29 lines

```
import React, { useState } from 'react';
import { Icon } from '../core/Icon.jsx';
/** Global search + command palette. Arrow keys move an edge + weight, not colour alone. */
export function CommandPalette({ groups = [], placeholder = 'Search Semester or type a command', onSelect }) {
  const [q, setQ] = useState(''); const [sel, setSel] = useState(0);
  const flat = []; const shown = groups.map((g) => ({ ...g, items: g.items.filter((i) =>
i.label.toLowerCase().includes(q.toLowerCase())) })).filter((g) => g.items.length);
  shown.forEach((g) => g.items.forEach((i) => flat.push(i)));
  const key = (e) => { if (e.key === 'ArrowDown') { e.preventDefault(); setSel((s) => Math.min(flat.length - 1, s + 1)); } if
```

```
(e.key === 'ArrowUp') { e.preventDefault(); setSel((s) => Math.max(0, s - 1)); } if (e.key === 'Enter' && flat[sel] &&
onSelect) onSelect(flat[sel]); };
  let n = -1;
  return (
    <div className="palette" role="dialog" aria-label="Command palette">
      <div className="palette-input"><Icon name="search" size={19} style={{ color: 'var(--text-secondary)' }} /><input value=
{q} onChange={(e) => { setQ(e.target.value); setSel(0); }} onKeyDown={key} placeholder={placeholder} aria-label={placeholder}
/><span className="kbd">esc</span></div>
      <div role="listbox">
        {shown.map((g) => (
          <div className="palette-group" key={g.label} role="group" aria-label={g.label}>
            <div className="palette-group-label">{g.label}</div>
            {g.items.map((it) => { n++; const i = n; return (
              <button key={it.label} type="button" role="option" aria-selected={i === sel} className="palette-option"
onMouseEnter={() => setSel(i)} onClick={() => onSelect && onSelect(it)}>
                {it.icon && <Icon name={it.icon} size={17} style={{ color: 'var(--text-secondary)' }} />}<span>{it.label}
</span>{it.detail && <span className="palette-option-detail">{it.detail}</span>}
              </button>); })}
          </div>
        ))}
        {flat.length === 0 && <div style={{ padding: 'var(--sp-7)', color: 'var(--text-secondary)', fontSize: 'var(--type-sm)'
}}>Nothing matches “{q}”. Try a course code, a person, or “ask”.</div>}
      </div>
      <div className="palette-foot"><span><span className="kbd">↑</span> <span className="kbd">↓</span> move</span><span><span
className="kbd">↵</span> open</span><span>Results respect your privacy settings</span></div>
    </div>
  );
}
```

components/navigation/SideRail.jsx · 23 lines

```
import React from 'react';
import { Icon } from '../core/Icon.jsx';
import { Wordmark } from '../core/Wordmark.jsx';
/** Desktop side rail — persistent, quiet, always on dark chrome. Groups: [{label?, items:[{id,label,icon,count}]}]. */
export function SideRail({ groups = [], current, onNavigate, foot, brand = true }) {
  return (
    <nav className="rail" aria-label="Semester">
      {brand && <div className="rail-brand"><Wordmark size={18} metal /></div>}
      {groups.map((g, gi) => (
        <React.Fragment key={gi}>
          {g.label && <div className="rail-label">{g.label}</div>}
          {g.items.map((it) => (
            <button key={it.id} type="button" className="rail-row" aria-current={current === it.id ? 'page' : undefined}
onClick={() => onNavigate && onNavigate(it.id)}>
              <Icon name={it.icon} size={19} /><span>{it.label}</span>{it.count != null && <span className="rail-count">
{it.count}</span>}
            </button>
          ))}
        </React.Fragment>
      ))}
      {foot && <div className="rail-foot">{foot}</div>}
    </nav>
  );
}
```

components/navigation/SystemContextBar.jsx · 19 lines

```
import React from 'react';
import { Icon } from '../core/Icon.jsx';
import { SaveState } from '../feedback/SaveState.jsx';
/** Persistent top bar: where you are, the term, global search (⌘K), sync state, and quiet tools. */
export function SystemContextBar({ location, term, sync = 'synced', onSearch, actions }) {
  return (
    <div className="syscontext">
      <div className="syscontext-group">
        {location && <span style={{ color: 'var(--text-primary)', fontSize: 'var(--type-sm)' }}>{location}</span>}
        {term && <span className="syscontext-term">{term}</span>}
      </div>
      <button type="button" className="syscontext-search" onClick={onSearch} aria-label="Search or run a command">
        <Icon name="search" size={15} /><span style={{ flex: 1, textAlign: 'left' }}>Search or ask…</span><span
className="kbd">⌘K</span>
      </button>
```

```
      <div className="syscontext-group"><SaveState status={sync} />{actions}</div>
    </div>
  );
}
```

components/navigation/TabBar.jsx · 15 lines

```
import React from 'react';
import { Icon } from '../core/Icon.jsx';
/** Mobile bottom navigation — the five canonical destinations, labelled. */
export function TabBar({ items = [], current, onNavigate }) {
  return (
    <nav className="tabbar" aria-label="Semester">
      {items.map((it) => (
        <button key={it.id} type="button" className="tab" aria-current={current === it.id ? 'page' : undefined} onClick={() =>
onNavigate && onNavigate(it.id)}>
          <Icon name={it.icon} size={21} /><span>{it.label}</span>
        </button>
      ))}
    </nav>
  );
}
```

components/overlays/FocusBar.jsx · 14 lines

```
import React from 'react';
/** Floating bar during a focus session — what you're working on, time left, and the way out. Only frosted surface besides the
tab bar. */
export function FocusBar({ task, remaining = '24:10', onPause, onEnd, breakDue }) {
  return (
    <div className="focus-bar" role="region" aria-label="Focus session">
      <span className="focus-timer" aria-label={'Time left ' + remaining}>{remaining}</span>
      <span style={{ color: 'var(--text-secondary)' }}>{task}</span>
      {breakDue && <span style={{ color: 'var(--text-primary)' }}>{breakDue}</span>}
      <button type="button" className="btn btn-sm" onClick={onPause}>Pause</button>
      <button type="button" className="btn btn-sm btn-ghost" onClick={onEnd}>End session</button>
    </div>
  );
}
```

components/overlays/Sheet.jsx · 19 lines

```
import React from 'react';
import { IconButton } from '../core/IconButton.jsx';
/** Bottom sheet (mobile) / window (desktop). Default use: "Source & details" — origin, freshness, used in, limitations. */
export function Sheet({ title, children, onClose, inline = false, actions }) {
  const box = (
    <div className={'sheet' + (inline ? ' is-inline' : '')} role="dialog" aria-modal={!inline} aria-label={title}>
      {!inline && <div className="sheet-grip" aria-hidden="true"></div>}
      <div className="sheet-head"><h2 style={{ fontSize: 'var(--type-display-xs)', fontFamily: 'var(--font-body)', fontWeight:
600, letterSpacing: 0 }}>{title}</h2>{onClose && <IconButton icon="close" label="Close" onClick={onClose} />}</div>
      {children}
      {actions && <div className="dialog-actions" style={{ marginTop: 'var(--sp-7)' }}>{actions}</div>}
    </div>
  );
  return inline ? box : <div className="sheet-scrim" onClick={(e) => e.target === e.currentTarget && onClose && onClose()}>
{box}</div>;
}
/** The Source & details definition list used inside a Sheet. */
export function SourceDetails({ items = [] }) {
  return <dl className="source-list">{items.map((it) => <React.Fragment key={it.label}><dt>{it.label}</dt><dd>{it.value}</dd>
</React.Fragment>)}</dl>;
}
```

components/overlays/UndoToast.jsx · 6 lines

```
import React from 'react';
/** Reversible work gets an undo, not a confirmation. One sentence + Undo; polite live region; never auto-hides under 6s. */
export function UndoToast({ message, onUndo, undoLabel = 'Undo' }) {
  return <div className="toast" role="status" aria-live="polite"><span>{message}</span>{onUndo && <button type="button"
```

```
className="btn btn-sm" onClick={onUndo}>{undoLabel}</button>}</div>;
}
```

components/surfaces/ActionPreview.jsx · 19 lines

```
import React from 'react';
const RECOVERY = { undo: 'You can undo this.', request: 'This can be reversed on request: ', none: 'This can't be undone.' };
/** What a person is about to do, said before they do it. Recovery is required. */
export function ActionPreview({ subject, says, exactly, doesNotChange, recovery = { kind: 'undo' }, subjectTo, provenance }) {
  return (
    <div className="action-preview">
      {subject && <p style={{ margin: 0 }}><strong>{subject}</strong></p>}
      {provenance}
      <dl className="action-preview-facts">
        <dt>What happens</dt><dd>{says}</dd>
        {exactly && <><dt>Exactly what</dt><dd>{exactly}</dd></>}
        {doesNotChange && <><dt>What stays the same</dt><dd>{doesNotChange}</dd></>}
        <dt>Taking it back</dt><dd>{RECOVERY[recovery.kind]}{recovery.how ? ' ' + recovery.how : ''}</dd>
      </dl>
      {subjectTo && <p className="action-preview-caveat">{subjectTo}</p>}
    </div>
  );
}
```

components/surfaces/ContextBar.jsx · 13 lines

```
import React from 'react';
/** The object a screen is about, with its states and actions — one bar, not a card inside a card. */
export function ContextBar({ kicker, title, states, actions }) {
  return (
    <section className="context-bar">
      {kicker && <div className="kicker">{kicker}</div>}
      <h2 className="context-bar-title">{title}</h2>
      {states && <div className="context-bar-states">{states}</div>}
      {actions && <div className="object-card-actions" style={{ marginTop: 'var(--sp-2)' }}>{actions}</div>}
    </section>
  );
}
```

components/surfaces/Dialog.jsx · 19 lines

```
import React, { useEffect, useRef } from 'react';
/** Modal confirmation. Focus lands on Cancel; Escape closes; consequential actions name object, consequence, recovery. */
export function Dialog({ open = true, title, children, confirmLabel = 'Confirm', cancelLabel = 'Cancel', onConfirm, onCancel,
destructive = false, inline = false }) {
  const cancelRef = useRef(null);
  useEffect(() => { if (open && !inline && cancelRef.current) cancelRef.current.focus(); }, [open, inline]);
  if (!open) return null;
  const box = (
    <div className="dialog" role="dialog" aria-modal={!inline} aria-label={title} onKeyDown={(e) => { if (e.key === 'Escape' &&
onCancel) onCancel(); }}>
      <h2 className="dialog-title">{title}</h2>
      <div>{children}</div>
      <div className="dialog-actions">
        <button ref={cancelRef} type="button" className="btn" onClick={onCancel}>{cancelLabel}</button>
        <button type="button" className={destructive ? 'btn btn-danger' : 'btn btn-primary'} onClick={onConfirm}>{confirmLabel}
</button>
      </div>
    </div>
  );
  return inline ? box : <div className="dialog-scrim" onClick={(e) => { if (e.target === e.currentTarget && onCancel)
onCancel(); }}>{box}</div>;
}
```

components/surfaces/NextSteps.jsx · 20 lines

```
import React from 'react';
import { Icon } from '../core/Icon.jsx';
/** Ordered list of next safe actions, each with one line of why. */
export function NextSteps({ steps = [] }) {
```

```
  return (
    <ul className="next-steps">
      {steps.map((s) => (
        <li key={s.label}>
          <button type="button" className="next-step" onClick={s.onClick}>
            {s.icon && <Icon name={s.icon} size={19} style={{ color: 'var(--text-secondary)' }} />}
            <span className="next-step-text"><span className="next-step-label">{s.label}</span>{s.why && <span className="next-
step-why">{s.why}</span>}</span>
            {s.aside}
            <Icon name="chevronRight" size={17} style={{ color: 'var(--text-tertiary)' }} />
          </button>
        </li>
      ))}
    </ul>
  );
}
```

components/surfaces/ObjectCard.jsx · 28 lines

```
import React, { useId } from 'react';
import { STATUSES } from '../trust/vocabulary.js';
import { ProvenanceChips } from '../trust/ProvenanceChips.jsx';
/**
 * The universal object card — one rhythm for every kind of thing:
 * eyebrow (kind · status) → title → provenance → why it matters → metadata → one primary + one quiet secondary.
 */
export function ObjectCard({ kind = 'Task', statuses = [], title, provenance, explanation, metadata, primary, secondary, tone,
level = 3, children }) {
  const hid = useId(); const H = 'h' + level;
  const glyphs = statuses.map((s) => (STATUSES[s] || {}).glyph).filter(Boolean).join(' ');
  return (
    <article className="object-card" data-tone={tone} aria-labelledby={hid}>
      <div className="object-card-eyebrow">{glyphs && <span className="status-glyph" aria-hidden="true">{glyphs}</span>}<span>
{[kind, ...statuses.map((s) => (STATUSES[s] || { label: s }).label)].join(' · ')}</span></div>
      <H id={hid} className="object-card-title">{title}</H>
      {provenance && <div><ProvenanceChips {...provenance} /></div>}
      {explanation && <p className="object-card-why">{explanation}</p>}
      {metadata && <p className="object-card-meta">{metadata}</p>}
      {children}
      {(primary || secondary) && (
        <div className="object-card-actions">
          {primary && <button type="button" className="btn btn-primary" onClick={primary.onClick}>{primary.label}</button>}
          {secondary && <button type="button" className="link-quiet" onClick={secondary.onClick}>{secondary.label}</button>}
        </div>
      )}
    </article>
  );
}
```

components/surfaces/OpenIn.jsx · 12 lines

```
import React from 'react';
/** "Open in" — the way from one thing to every place it matters. Words, never icons alone; wraps on a phone. */
export function OpenIn({ targets = [], about, onOpen }) {
  if (!targets.length) return null;
  return (
    <div className="open-in" role="group" aria-label={'Open ' + about + ' in'}>
      <span className="open-in-label" aria-hidden="true">Open in</span>
      {targets.map((t) => <button key={t} type="button" className="pill-soft" onClick={() => onOpen && onOpen(t)}>{t}
</button>)}
    </div>
  );
}
```

components/surfaces/QuickActions.jsx · 7 lines

```
import React from 'react';
import { Icon } from '../core/Icon.jsx';
/** A row of 2–5 labelled starting actions ("Add a syllabus", "Connect a calendar"). Never icon-only. */
export function QuickActions({ actions = [] }) {
  return <div className="quick-actions">{actions.map((a) => <button key={a.label} type="button" className="quick-action"
```

```
onClick={a.onClick}>{a.icon && <Icon name={a.icon} size={18} style={{ color: 'var(--text-secondary)' }} />}{a.label}</button>)}
</div>;
}
```

components/surfaces/ScreenGuide.jsx · 11 lines

```
import React from 'react';
/** "About this screen" — a quiet disclosure at the foot of every screen: what it's for, where the data comes from, who to ask.
*/
export function ScreenGuide({ items = [], open = false }) {
  return (
    <details className="screen-guide" open={open}>
      <summary><span aria-hidden="true">ⓘ</span>About this screen</summary>
      <dl>{items.map((it) => <React.Fragment key={it.q}><dt>{it.q}</dt><dd>{it.a}</dd></React.Fragment>)}</dl>
    </details>
  );
}
```

components/trust/AIResponse.jsx · 31 lines

```
import React from 'react';
import { SourceBadge } from './SourceBadge.jsx';
/**
 * A governed assistant answer: always labelled AI-assisted, shows the data scope it was allowed to read,
 * numbered sources, stated uncertainty, request cost, and the actions that need human confirmation.
 */
export function AIResponse({ children, scope = [], sources = [], confidence, cost, policy, actions, model = 'Semester
assistant' }) {
  return (
    <article className="ai-frame" aria-label="AI-assisted answer">
      <div className="ai-head">
        <SourceBadge source="ai" updated={model} />
        {policy && <span className="status-chip" data-tone="pending"><span className="status-glyph" aria-hidden="true">§</span>
{policy}</span>}
      </div>
      <div className="ai-body">{children}</div>
      {sources.length > 0 && (
        <ol className="ai-sources" aria-label="Sources">
          {sources.map((s, i) => (
            <li key={i}><span className="ai-cite">{i + 1}</span><span style={{ color: 'var(--text-primary)' }}>{s.title}</span>
{s.kind && <SourceBadge source={s.kind} />}</li>
          ))}
        </ol>
      )}
      <div className="ai-meta">
        {scope.length > 0 && <span>Allowed to read <b>{scope.join(', ')}</b></span>}
        {confidence && <span>Confidence <b>{confidence}</b></span>}
        {cost && <span>Request cost <b className="nums">{cost}</b></span>}
      </div>
      {actions && <div className="state-actions">{actions}</div>}
    </article>
  );
}
```

components/trust/ProvenanceChips.jsx · 19 lines

```
import React from 'react';
import { SOURCES, STATUSES } from './vocabulary.js';
/** Origin chip, then at most two cues, then age. Read to assistive tech as one sentence. */
export function ProvenanceChips({ origin = 'official', by, cues = [], age }) {
  const o = SOURCES[origin] || SOURCES.external;
  const shown = cues.slice(0, 2).map((c) => ({ key: c, ...(STATUSES[c] || { label: c, glyph: '·', tone: 'neutral' }) }));
  const sentence = [o.label + (by ? ' by ' + by : ''), ...shown.map((c) => c.label.toLowerCase()), age].filter(Boolean).join(',
');
  return (
    <span className="prov" data-origin={origin} title={o.meaning}>
      <span className="prov" aria-hidden="true">
        <span className="status-chip" data-tone={o.tone} data-fill={o.fill}><span className="status-glyph">{o.glyph}</span>
{o.short}{by ? ' · ' + by : ''}</span>
        {shown.map((c) => <span key={c.key} className="status-chip" data-tone={c.tone}><span className="status-glyph">{c.glyph}
</span>{c.label}</span>)}
```

```
        {age && <span className="prov-age">{age}</span>}
      </span>
      <span className="sr-only">{sentence}</span>
    </span>
  );
}
```

components/trust/SourceBadge.jsx · 15 lines

```
import React from 'react';
import { SOURCES } from './vocabulary.js';
/** Where a fact came from and how old it is, beside the fact. Text first — never icon or colour alone. */
export function SourceBadge({ source = 'official', updated, by, onReport }) {
  const s = SOURCES[source] || SOURCES.external;
  return (
    <span className="source-badge" data-source={source} title={s.meaning}>
      <span className="status-chip" data-tone={s.tone} data-fill={s.fill}><span className="status-glyph" aria-hidden="true">
{s.glyph}</span>{s.label}{by ? ' · ' + by : ''}</span>
      <span className="sr-only">{s.meaning}</span>
      {updated && <span>{updated}</span>}
      {onReport && <button type="button" className="source-report" onClick={onReport}>Report incorrect information</button>}
    </span>
  );
}
```

components/trust/SourceLine.jsx · 17 lines

```
import React from 'react';
import { SOURCES } from './vocabulary.js';
/** The signature device: a thin editorial line from a statement to its source chip. Only where a real source exists. */
export function SourceLine({ children, source = 'official', by, when }) {
  const s = SOURCES[source] || SOURCES.external;
  return (
    <div style={{ display: 'grid', gap: 'var(--sp-4)' }}>
      <div>{children}</div>
      <div style={{ display: 'flex', alignItems: 'center', gap: 'var(--sp-4)' }} title={s.meaning}>
        <span aria-hidden="true" style={{ flex: '0 0 48px', height: 1, background: 'var(--accent-editorial)' }}></span>
        <span aria-hidden="true" style={{ width: 5, height: 5, borderRadius: '50%', background: 'var(--accent-editorial)',
marginLeft: -6 }}></span>
        <span className="status-chip" data-tone={s.tone} data-fill={s.fill}><span className="status-glyph" aria-hidden="true">
{s.glyph}</span>{[s.short, by, when].filter(Boolean).join(' · ')}</span>
      </div>
    </div>
  );
}
```

components/trust/StatusChip.jsx · 8 lines

```
import React from 'react';
import { STATUSES } from './vocabulary.js';
/** One status, drawn one way: glyph + word + tone. */
export function StatusChip({ status = 'pending', label }) {
  const s = STATUSES[status] || { label: status, glyph: '·', tone: 'neutral' };
  return <span className="status-chip" data-tone={s.tone}><span className="status-glyph" aria-hidden="true">{s.glyph}</span>
{label || s.label}</span>;
}
```

components/trust/vocabulary.js · 42 lines

```
// The one trust + status vocabulary. Ported from semester app/src/lib/source.ts and lib/status.ts.
// Every state = glyph + word + tone. Tone is the least of the three.
export const SOURCES = {
  official:  { label: 'Institution verified', short: 'Official', glyph: '◆', tone: 'official', fill: 'solid', meaning:
'Confirmed by your institution's own system.' },
  connected: { label: 'Connected', short: 'Connected', glyph: '⇄', tone: 'connected', fill: 'outline', meaning: 'Synced from a
connected account or feed. Semester has not checked it with your institution.' },
  imported:  { label: 'Imported', short: 'Imported', glyph: '↓', tone: 'connected', fill: 'outline', meaning: 'Copied in from a
file or feed. Semester has not checked it with your institution.' },
  personal:  { label: 'Student entered', short: 'Yours', glyph: '○', tone: 'personal', fill: 'outline', meaning: 'You typed
this in. Semester has not checked it with your institution.' },
```

```
  made:      { label: 'Made here', short: 'Made here', glyph: '○', tone: 'personal', fill: 'outline', meaning: 'Semester built
this from sources you gave it — a rule-based extraction, not an AI draft.' },
  ai:        { label: 'AI-assisted', short: 'AI-assisted', glyph: '✦', tone: 'ai', fill: 'outline', meaning: 'Written with
Semester's assistant. It is not an official answer — check anything you act on.' },
  estimated: { label: 'Estimated', short: 'Estimated', glyph: '≈', tone: 'pending', fill: 'outline', meaning: 'Worked out by
Semester from what it knows. Not an official figure.' },
  review:    { label: 'Needs review', short: 'Review', glyph: '?', tone: 'pending', fill: 'outline', meaning: 'Something about
this looks wrong or out of date. Check it before relying on it.' },
  stale:     { label: 'Unavailable or stale', short: 'Stale', glyph: '!', tone: 'danger', fill: 'outline', meaning: 'Semester
cannot confirm that this information is current.' },
  external:  { label: 'External', short: 'External', glyph: '↗', tone: 'neutral', fill: 'outline', meaning: 'From a source
outside your institution and outside Semester, such as a web page.' },
  sample:    { label: 'Sample', short: 'Sample', glyph: '◌', tone: 'neutral', fill: 'dashed', meaning: 'Illustrative data. Not
live.' },
};
export const STATUSES = {
  saving: { label: 'Saving…', glyph: '…', tone: 'neutral' },
  saved: { label: 'Saved', glyph: '✓', tone: 'success' },
  syncing: { label: 'Syncing', glyph: '↻', tone: 'neutral' },
  synced: { label: 'Synced', glyph: '✓', tone: 'success' },
  offline: { label: 'Offline', glyph: '⊘', tone: 'pending', urgent: true },
  queued: { label: 'Queued', glyph: '⋯', tone: 'neutral' },
  conflict: { label: 'Conflict needs review', glyph: '⇄', tone: 'danger', urgent: true },
  'sync-error': { label: 'Sync trouble', glyph: '!', tone: 'danger', urgent: true },
  'needs-confirmation': { label: 'Needs confirmation', glyph: '?', tone: 'pending' },
  'action-required': { label: 'Action required', glyph: '!', tone: 'pending' },
  pending: { label: 'Pending approval', glyph: '◷', tone: 'pending' },
  approved: { label: 'Approved', glyph: '✓', tone: 'success' },
  rejected: { label: 'Rejected', glyph: '×', tone: 'danger' },
  verified: { label: 'Verified', glyph: '✓', tone: 'success' },
  hold: { label: 'Hold on account', glyph: '!', tone: 'danger' },
  'faculty-approved': { label: 'Faculty-approved', glyph: '◆', tone: 'success' },
  'course-provided': { label: 'Course-provided', glyph: '◇', tone: 'official' },
  'updated-today': { label: 'Updated today', glyph: '↻', tone: 'neutral' },
  draft: { label: 'Draft', glyph: '○', tone: 'neutral' },
  live: { label: 'Live', glyph: '●', tone: 'success' },
  beta: { label: 'Preview', glyph: '◌', tone: 'neutral' },
  'device-only': { label: 'On this device only', glyph: '○', tone: 'neutral' },
  'signed-out': { label: 'Not signed in', glyph: '○', tone: 'neutral' },
  unknown: { label: 'Status unknown', glyph: '·', tone: 'neutral' },
};
```

components/core/markPaths.js · 4 lines

```
// Generated from semester app/src/components/mark.data.ts — three slabs on a 512 grid.
export const MARK_VIEWBOX = '128 65 256 382';
export const MARK_PATHS = ["M128 176L204 208L204 420L128 388Z","M218 65L294 97L294 404L218 372Z","M308 147L384 179L384 447L308
415Z"];
```

SEMESTER-CORE PACKAGE

Access, AI gateway, approvals, consequence, freshness, next action, pilot, provenance and scoring logic, with tests.

handoff/semester-core/access.ts · 35 lines

```
export type Role = 'student' | 'applicant' | 'guardian' | 'alumni' | 'faculty' | 'ta' | 'advisor' | 'registrar' | 'finance' |
'affairs' | 'security' | 'support' | 'employer' | 'partner';
export type Level = 'full' | 'scoped' | 'shared' | 'read' | 'audited' | 'none';
export interface Share { grantee: string; screen: string; expiresAt: number; revoked?: boolean }
export interface Grant { kind: 'section' | 'ticket' | 'listing' | 'breakglass'; id: string; expiresAt: number; approvedBy?:
string[] }
export interface Ctx { role: Role; userId: string; ownerId?: string; shares?: Share[]; grants?: Grant[]; now: number }
/** Default matrix; anything missing is none. Students own their own rows. */
export const MATRIX: Partial<Record<Role, Record<string, Level>>> = {
  guardian: { family: 'shared', billing: 'shared', grades: 'shared', calendar: 'shared' },
  advisor: { degree: 'shared', advising: 'shared', registration: 'read' },
  faculty: { course: 'scoped', grades: 'scoped', announce: 'scoped' },
  ta: { course: 'scoped', grades: 'scoped' },
  registrar: { registration: 'full', degree: 'full', grades: 'audited' },
  finance: { billing: 'full', costs: 'full' },
  affairs: { support: 'shared', housing: 'scoped', meals: 'scoped' },
  security: { privacy: 'audited', account: 'audited', connect: 'audited' },
  support: { support: 'scoped', connect: 'scoped' },
  employer: { opportunities: 'scoped', career: 'shared' },
  partner: { opportunities: 'scoped' },
};
export function level(role: Role, screen: string): Level {
  if (role === 'student') return 'full';
  return MATRIX[role]?.[screen] ?? 'none';
}
/** The runtime decision: the matrix says what kind of access, the context proves it right now. Mirrors the RLS policy. */
export function canRead(screen: string, c: Ctx): { ok: boolean; reason: string } {
  const l = level(c.role, screen);
  if (c.role === 'student') return c.ownerId == null || c.ownerId === c.userId ? { ok: true, reason: 'owner' } : { ok: false,
reason: 'not the owner' };
  if (l === 'none') return { ok: false, reason: 'role has no access' };
  if (l === 'shared') { const s = (c.shares ?? []).find((x) => x.grantee === c.userId && x.screen === screen && !x.revoked &&
x.expiresAt > c.now); return s ? { ok: true, reason: 'active share' } : { ok: false, reason: 'no active share from the student'
}; }
  if (l === 'scoped') { const g = (c.grants ?? []).find((x) => x.kind !== 'breakglass' && x.expiresAt > c.now); return g ? {
ok: true, reason: g.kind + ' membership' } : { ok: false, reason: 'not a member of this scope' }; }
  if (l === 'audited') { const g = (c.grants ?? []).find((x) => x.kind === 'breakglass' && x.expiresAt > c.now && new
Set(x.approvedBy).size >= 2 && !x.approvedBy!.includes(c.userId)); return g ? { ok: true, reason: 'break-glass, 2 approvers,
logged' } : { ok: false, reason: 'needs break-glass with two other approvers' }; }
  return { ok: true, reason: l };
}
export const canWrite = (screen: string, c: Ctx) => { const l = level(c.role, screen); return (c.role === 'student' &&
canRead(screen, c).ok) || ((l === 'full' || l === 'scoped') && canRead(screen, c).ok); };
```

handoff/semester-core/aigateway.ts · 16 lines

```
export type DataClass = 'public' | 'course' | 'personal' | 'student-record';
export interface Policy { courseAllows: ('outline' | 'feedback' | 'practice' | 'write' | 'none')[]; institutionAiOn: boolean }
export interface Req { task: 'outline' | 'feedback' | 'practice' | 'write' | 'plan' | 'search'; scope: { id: string; cls:
DataClass }[]; policy: Policy; budgetLeft: number }
export interface Decision { allowed: boolean; reason: string; sendScope: string[]; dropped: string[]; label: 'AI-assisted' }
/** Runs before every model call. Student records are never sent, whatever the scope says. */
export function gate(r: Req): Decision {
  const base = { label: 'AI-assisted' as const };
  if (!r.policy.institutionAiOn) return { ...base, allowed: false, reason: 'Your institution has turned AI off.', sendScope:
[], dropped: r.scope.map((s) => s.id) };
  if (r.budgetLeft <= 0) return { ...base, allowed: false, reason: 'No requests left this week.', sendScope: [], dropped:
r.scope.map((s) => s.id) };
  const courseTask = ['outline', 'feedback', 'practice', 'write'].includes(r.task);
  if (courseTask && !r.policy.courseAllows.includes(r.task as never)) return { ...base, allowed: false, reason: 'Your
instructor doesn't allow AI for ' + r.task + ' in this course.', sendScope: [], dropped: r.scope.map((s) => s.id) };
```

```
  const sendScope = r.scope.filter((s) => s.cls !== 'student-record').map((s) => s.id);
  const dropped = r.scope.filter((s) => s.cls === 'student-record').map((s) => s.id);
  return { ...base, allowed: true, reason: dropped.length ? 'Student records were left out.' : 'Allowed.', sendScope, dropped
};
}
```

handoff/semester-core/approvals.ts · 17 lines

```
export type St = 'requested' | 'approved-1' | 'approved' | 'rejected' | 'expired' | 'executed';
export interface Approval { id: string; requester: string; approvers: string[]; state: St; expiresAt: number; mfaAt:
Record<string, number> }
export type Ev = { type: 'approve' | 'reject' | 'execute'; by: string; at: number };
const MFA_MS = 15 * 60e3;
/** Two-person duty: two distinct approvers, neither the requester, each with MFA in the last 15 min; execute only by the
requester after full approval. */
export function step(a: Approval, e: Ev): Approval | { error: string } {
  if (e.at > a.expiresAt && a.state !== 'executed') return { ...a, state: 'expired' };
  if (e.type === 'execute') return a.state === 'approved' && e.by === a.requester ? { ...a, state: 'executed' } : { error:
'Needs two approvals; only the requester runs it.' };
  if (e.by === a.requester) return { error: 'You can’t approve your own request.' };
  if (a.approvers.includes(e.by)) return { error: 'You already decided this one.' };
  if (!a.mfaAt[e.by] || e.at - a.mfaAt[e.by] > MFA_MS) return { error: 'Verify a second factor first (within 15 minutes).' };
  if (!['requested', 'approved-1'].includes(a.state)) return { error: 'This request is ' + a.state + '.' };
  if (e.type === 'reject') return { ...a, state: 'rejected', approvers: [...a.approvers, e.by] };
  const approvers = [...a.approvers, e.by];
  return { ...a, approvers, state: approvers.length >= 2 ? 'approved' : 'approved-1' };
}
```

handoff/semester-core/consequence.ts · 11 lines

```
export interface Action { reversible: boolean; leavesSemester?: boolean; record?: boolean; money?: number; institutional?:
boolean; affectsOthers?: boolean }
export type Pattern = 'undo' | 'preview' | 'review-confirm' | 'two-person';
/** The one rule for how hard an action is to take. */
export function pattern(a: Action): Pattern {
  if (a.institutional) return 'two-person';
  if (a.record || (a.money ?? 0) > 0) return 'review-confirm';
  if (a.leavesSemester || a.affectsOthers || !a.reversible) return 'preview';
  return 'undo';
}
export const RECOVERY = { undo: 'You can undo this.', request: 'This can be reversed on request.', none: 'This can’t be
undone.' } as const;
```

handoff/semester-core/core.test.ts · 69 lines

```
import { describe, it, expect } from 'vitest';
import { resolve, effectiveOrigin, ageLabel } from './provenance';
import { order } from './nextaction';
import { canRead, level } from './access';
import { state, optimistic, POLICY } from './freshness';
import { pattern } from './consequence';
import { gate } from './aigateway';
import { step, Approval } from './approvals';
import { weighted } from './scoring';
import { readiness } from './pilot';
const NOW = Date.UTC(2026, 9, 7, 12); const H = 3600e3;

describe('provenance', () => {
  it('official beats personal; stale official loses to personal', () => {
    expect(resolve([{ value: 'Thu', origin: 'personal', updatedAt: NOW }, { value: 'Wed', origin: 'official', updatedAt: NOW -
H }], NOW)!.shown.value).toBe('Wed');
    expect(effectiveOrigin({ value: 1, origin: 'connected', updatedAt: NOW - 7 * H }, NOW)).toBe('stale');
    const r = resolve([{ value: 'Thu', origin: 'personal', updatedAt: NOW }, { value: 'Wed', origin: 'connected', updatedAt:
NOW - 7 * H }], NOW)!;
    expect(r.shown.value).toBe('Thu'); expect(r.conflict).toBe(true);
  });
  it('never invents a time', () => { expect(ageLabel(undefined, NOW)).toBe('Update time not recorded'); expect(ageLabel(NOW - 3
* 86400e3, NOW)).toBe('Updated 3 days ago'); });
});
describe('next action', () => {
  it('a blocking hold outranks a low-weight task due sooner', () => {
```

```
    const r = order([{ id: 't', title: 'Reading', kind: 'task', origin: 'official', dueAt: NOW + 20 * H, weight: 0.02 }, { id:
'h', title: 'Hold', kind: 'hold', origin: 'official', dueAt: NOW + 5 * 24 * H, blocks: ['registration'] }], NOW);
    expect(r[0].id).toBe('h'); expect(r[0].reasons).toContain('Blocks registration');
  });
  it('drops done and snoozed', () => { expect(order([{ id: 'a', title: '', kind: 'task', origin: 'personal', done: true }, {
id: 'b', title: '', kind: 'task', origin: 'personal', snoozedUntil: NOW + H }], NOW)).toEqual([]); });
});
describe('access', () => {
  it('guardian needs an unexpired, unrevoked share', () => {
    const c = { role: 'guardian' as const, userId: 'g', now: NOW };
    expect(canRead('grades', c).ok).toBe(false);
    expect(canRead('grades', { ...c, shares: [{ grantee: 'g', screen: 'grades', expiresAt: NOW + H }] }).ok).toBe(true);
    expect(canRead('grades', { ...c, shares: [{ grantee: 'g', screen: 'grades', expiresAt: NOW - 1 }] }).ok).toBe(false);
  });
  it('break-glass needs two other approvers', () => {
    const c = { role: 'security' as const, userId: 's', now: NOW };
    expect(canRead('privacy', { ...c, grants: [{ kind: 'breakglass' as const, id: 'b', expiresAt: NOW + H, approvedBy: ['s',
'x'] }] }).ok).toBe(false);
    expect(canRead('privacy', { ...c, grants: [{ kind: 'breakglass' as const, id: 'b', expiresAt: NOW + H, approvedBy: ['x',
'y'] }] }).ok).toBe(true);
  });
  it('unknown role/screen is none', () => { expect(level('partner', 'grades')).toBe('none'); });
});
describe('freshness', () => {
  it('maps lag to policy', () => {
    expect(state({ lastEventAt: NOW, appliedAt: NOW - 3000 }, NOW)).toBe('live');
    expect(POLICY[state({ lastEventAt: NOW, appliedAt: NOW - 120e3 }, NOW)].destructive).toBe('confirm-at-source');
    expect(POLICY[state({ lastEventAt: NOW, workerFailed: true }, NOW)].destructive).toBe('disabled');
  });
  it('optimistic confirms on version, rolls back on timeout', () => { expect(optimistic(5, 5, NOW, NOW)).toBe('confirmed');
expect(optimistic(5, 4, NOW - 20e3, NOW)).toBe('rolled-back'); });
});
describe('consequence', () => { it('picks the pattern', () => { expect(pattern({ reversible: true })).toBe('undo');
expect(pattern({ reversible: true, leavesSemester: true })).toBe('preview'); expect(pattern({ reversible: false, record: true
})).toBe('review-confirm'); expect(pattern({ reversible: false, institutional: true })).toBe('two-person'); }); });
describe('AI gateway', () => {
  const policy = { courseAllows: ['outline' as const], institutionAiOn: true };
  it('never sends student records', () => { const d = gate({ task: 'outline', scope: [{ id: 'syl', cls: 'course' }, { id: 'tx',
cls: 'student-record' }], policy, budgetLeft: 3 }); expect(d.allowed).toBe(true); expect(d.sendScope).toEqual(['syl']);
expect(d.dropped).toEqual(['tx']); });
  it('enforces course policy', () => { expect(gate({ task: 'write', scope: [], policy, budgetLeft: 3 }).allowed).toBe(false);
});
});
describe('approvals', () => {
  const a: Approval = { id: '1', requester: 'r', approvers: [], state: 'requested', expiresAt: NOW + H, mfaAt: { x: NOW, y:
NOW, r: NOW } };
  it('two distinct non-requesters with fresh MFA', () => {
    expect('error' in step(a, { type: 'approve', by: 'r', at: NOW })).toBe(true);
    const one = step(a, { type: 'approve', by: 'x', at: NOW }) as Approval; expect(one.state).toBe('approved-1');
    expect('error' in step(one, { type: 'approve', by: 'x', at: NOW })).toBe(true);
    const two = step(one, { type: 'approve', by: 'y', at: NOW }) as Approval; expect(two.state).toBe('approved');
    expect((step(two, { type: 'execute', by: 'r', at: NOW }) as Approval).state).toBe('executed');
  });
});
describe('scoring', () => { it('renormalises over graded criteria', () => { expect(weighted({ correctness: 5, codeQuality: 5
}).of5).toBe(5); expect(weighted({}).of5).toBeNull(); expect(weighted({ correctness: 4 }).missing).toContain('rendering'); });
});
describe('pilot', () => { it('ready only when every P0 is done', () => { expect(readiness([{ priority: 'P0', status: 'Done',
gate: 'x' }, { priority: 'P0', status: 'Not started', gate: 'Pilot contract' }]).blockers.length).toBe(1); }); });
```

handoff/semester-core/freshness.ts · 22 lines

```
export type State = 'live' | 'recent' | 'delayed' | 'failed' | 'rebuilding';
export interface Projection { lastEventAt: number; appliedAt?: number; workerFailed?: boolean; rebuilding?: boolean }
export function state(p: Projection, now: number): State {
  if (p.rebuilding) return 'rebuilding';
  if (p.workerFailed) return 'failed';
  const lag = (now - (p.appliedAt ?? 0)) / 1000;
  return lag <= 5 ? 'live' : lag <= 60 ? 'recent' : 'delayed';
}
export interface Policy { display: string; safe: boolean; destructive: 'allowed' | 'confirm-at-source' | 'disabled'; badge?:
string }
export const POLICY: Record<State, Policy> = {
  live: { display: 'normal', safe: true, destructive: 'allowed' },
  recent: { display: 'normal', safe: true, destructive: 'allowed' },
```

```
  delayed: { display: 'normal', safe: true, destructive: 'confirm-at-source', badge: 'Delayed' },
  failed: { display: 'last-known', safe: false, destructive: 'disabled', badge: 'Not updating' },
  rebuilding: { display: 'versioned', safe: true, destructive: 'disabled', badge: 'Rebuilding' },
};
/** Optimistic mutation: pending until the projection version reaches the command's version; rollback after timeout. */
export function optimistic(sentVersion: number, seenVersion: number, sentAt: number, now: number, timeoutMs = 15000): 'pending'
| 'confirmed' | 'rolled-back' {
  if (seenVersion >= sentVersion) return 'confirmed';
  return now - sentAt > timeoutMs ? 'rolled-back' : 'pending';
}
```

handoff/semester-core/nextaction.ts · 29 lines

```
import { Origin, rank } from './provenance';
export interface Item { id: string; title: string; dueAt?: number; weight?: number; effortMin?: number; origin: Origin;
blocks?: string[]; kind: 'assignment' | 'hold' | 'bill' | 'task' | 'event' | 'decision'; done?: boolean; snoozedUntil?: number
}
export interface Why { score: number; reasons: string[] }
const HOUR = 3600e3;
/**
 * Score = urgency × consequence × confidence − effort penalty.
 * urgency: 1 at ≥14 days, rising to 10 inside 24 h; overdue = 12.
 * consequence: grade weight (0–1 → 1–4) or blocking (holds, bills that block registration) = 5.
 * confidence: official 1.0 → stale 0.6; AI/estimated never outrank an official item of equal urgency.
 */
export function score(it: Item, now: number): Why {
  const reasons: string[] = [];
  const hrs = it.dueAt == null ? 24 * 14 : (it.dueAt - now) / HOUR;
  const urgency = hrs < 0 ? 12 : hrs < 24 ? 10 : hrs < 72 ? 6 : hrs < 168 ? 3 : hrs < 336 ? 1.5 : 1;
  if (hrs < 0) reasons.push('Overdue'); else if (hrs < 24) reasons.push('Due within 24 hours'); else if (hrs < 72)
reasons.push('Due in ' + Math.ceil(hrs / 24) + ' days');
  let consequence = 1 + 3 * Math.min(1, it.weight ?? 0);
  if (it.weight) reasons.push(Math.round(it.weight * 100) + '% of your grade');
  if (it.blocks?.length) { consequence = 5; reasons.push('Blocks ' + it.blocks.join(', ')); }
  const confidence = 1 - Math.min(0.4, rank(it.origin) * 0.08);
  const effort = Math.min(1.5, (it.effortMin ?? 30) / 120);
  return { score: +(urgency * consequence * confidence - effort).toFixed(3), reasons };
}
export function order(items: Item[], now: number, limit = 5): (Item & Why)[] {
  return items.filter((i) => !i.done && !(i.snoozedUntil && i.snoozedUntil > now))
    .map((i) => ({ ...i, ...score(i, now) }))
    .sort((a, b) => b.score - a.score || (a.dueAt ?? Infinity) - (b.dueAt ?? Infinity) || a.id.localeCompare(b.id))
    .slice(0, limit);
}
```

handoff/semester-core/pilot.ts · 7 lines

```
export interface Row { priority: 'P0' | 'P1' | 'P2'; status: string; gate: string }
/** Launch-ready only when every P0 is Done; reports what blocks real student data. */
export function readiness(rows: Row[]) {
  const p0 = rows.filter((r) => r.priority === 'P0'); const open = p0.filter((r) => r.status !== 'Done');
  return { ready: open.length === 0, p0Total: p0.length, p0Open: open.length, pct: p0.length ? Math.round(((p0.length -
open.length) / p0.length) * 100) : 100, blockers: open.filter((r) => /pilot|student|contract/i.test(r.gate)) };
}
```

handoff/semester-core/provenance.ts · 39 lines

```
export type Origin = 'official' | 'connected' | 'made' | 'personal' | 'sample' | 'stale' | 'ai' | 'estimated' | 'external';
/** Strongest first. stale ranks below personal on purpose: an unchecked copy that may have moved is worth less than a guess
that knows it is one. */
export const LADDER: Origin[] = ['official', 'connected', 'made', 'personal', 'sample', 'stale'];
const DISPLAY_ONLY: Origin[] = ['ai', 'estimated', 'external'];
export const STALE_AFTER_MS: Partial<Record<Origin, number>> = { official: 24 * 3600e3, connected: 6 * 3600e3 };

export interface Fact<T> { value: T; origin: Origin; updatedAt?: number; by?: string }

export function rank(o: Origin): number {
  const i = LADDER.indexOf(o);
  return i >= 0 ? i : LADDER.length + DISPLAY_ONLY.indexOf(o);
}
/** A synced fact past its window is demoted to stale before ranking. */
```

```
export function effectiveOrigin(f: Fact<unknown>, now: number): Origin {
  const win = STALE_AFTER_MS[f.origin];
  if (win && f.updatedAt != null && now - f.updatedAt > win) return 'stale';
  if (win && f.updatedAt == null) return 'stale';
  return f.origin;
}
/** Pick the fact to show; ties go to the newest. Returns the conflict so the UI can say "2 sources disagree". */
export function resolve<T>(facts: Fact<T>[], now: number): { shown: Fact<T> & { effective: Origin }; conflict: boolean } | null
{
  if (!facts.length) return null;
  const scored = facts.map((f) => ({ ...f, effective: effectiveOrigin(f, now) }))
    .sort((a, b) => rank(a.effective) - rank(b.effective) || (b.updatedAt ?? 0) - (a.updatedAt ?? 0));
  const shown = scored[0];
  const conflict = scored.some((f) => JSON.stringify(f.value) !== JSON.stringify(shown.value));
  return { shown, conflict };
}
export function ageLabel(updatedAt: number | undefined, now: number): string {
  if (updatedAt == null) return 'Update time not recorded';
  const m = Math.round((now - updatedAt) / 60000);
  if (m < 1) return 'Updated less than a minute ago';
  if (m < 60) return 'Updated ' + m + ' min ago';
  const h = Math.round(m / 60);
  if (h < 24) return 'Updated ' + h + ' h ago';
  const d = Math.round(h / 24);
  return 'Updated ' + d + (d === 1 ? ' day ago' : ' days ago');
}
```

handoff/semester-core/scoring.ts · 11 lines

```
export type Crit = 'correctness' | 'codeQuality' | 'rendering' | 'state' | 'maintainability' | 'handoff' | 'sandboxFit' |
'persistence';
export const DEFAULT_W: Partial<Record<Crit, number>> = { correctness: .25, codeQuality: .2, rendering: .15, state: .15,
maintainability: .1, handoff: .1, sandboxFit: .05 };
/** Weighted 0–5; weights renormalised over the criteria actually graded so a missing score never counts as zero. */
export function weighted(g: Partial<Record<Crit, number>>, w = DEFAULT_W): { of5: number | null; of100: number | null; missing:
Crit[] } {
  const keys = Object.keys(w) as Crit[]; const missing = keys.filter((k) => g[k] == null);
  const have = keys.filter((k) => g[k] != null); const tw = have.reduce((s, k) => s + (w[k] ?? 0), 0);
  if (!tw) return { of5: null, of100: null, missing };
  const v = have.reduce((s, k) => s + (g[k]! * (w[k] ?? 0)), 0) / tw;
  return { of5: +v.toFixed(2), of100: Math.round(v * 20), missing };
}
```

SEMESTER-PLATFORM: API, SECURITY, MIGRATIONS AND TESTS

API routes, input validation, AI redaction, security middleware, every database migration, seed data and the test suites.

handoff/semester-platform/app/api/ai/route.ts · 28 lines

```
// POST /api/ai — the only path to a model. Order: auth → policy → gate() → scan() → provider → audit + spend → labelled
answer.
import { NextResponse } from 'next/server';
import { z } from 'zod';
import { createServerClient } from '@/lib/supabase/server';
import { gate } from '@/lib/core/aigateway';
import { scan } from '@/lib/ai/redact';
const Body = z.object({ task: z.enum(['outline', 'feedback', 'practice', 'write', 'plan', 'search']), prompt:
z.string().min(1).max(4000), courseId: z.string().uuid().optional(), scope: z.array(z.object({ id: z.string(), cls:
z.enum(['public', 'course', 'personal', 'student-record']) })).max(20).default([]) });
export async function POST(req: Request) {
  const sb = await createServerClient();
  const { data: { user } } = await sb.auth.getUser();
  if (!user) return NextResponse.json({ error: 'Sign in required' }, { status: 401 });
  const parsed = Body.safeParse(await req.json().catch(() => null));
  if (!parsed.success) return NextResponse.json({ error: 'Invalid request' }, { status: 400 });
  const b = parsed.data;
  const { data: pol } = await sb.schema('api').rpc('ai_policy', { p_course: b.courseId ?? null }); // { institutionAiOn,
courseAllows, budgetLeft, studentIdPattern, provider, model }
  if (!pol) return NextResponse.json({ error: 'AI policy unavailable' }, { status: 503 });
  const d = gate({ task: b.task, scope: b.scope, policy: { institutionAiOn: pol.institutionAiOn, courseAllows: pol.courseAllows
}, budgetLeft: pol.budgetLeft });
  if (!d.allowed) { await sb.schema('api').rpc('ai_log', { p_task: b.task, p_outcome: 'refused', p_reason: d.reason, p_tokens:
0 }); return NextResponse.json({ allowed: false, reason: d.reason, label: d.label }); }
  const s = scan(b.prompt, { studentId: pol.studentIdPattern ? new RegExp(pol.studentIdPattern) : undefined });
  if (s.block) { await sb.schema('api').rpc('ai_log', { p_task: b.task, p_outcome: 'blocked', p_reason: s.findings.map((f) =>
f.kind).join(','), p_tokens: 0 }); return NextResponse.json({ allowed: false, reason: 'That looks like a student record (' +
[...new Set(s.findings.map((f) => f.kind))].join(', ') + '). Ask without it, or contact the office that holds it.', label:
d.label }); }
  const ctx = await sb.schema('api').rpc('ai_context', { p_ids: d.sendScope }); // approved sources only, with citations
  const res = await fetch(process.env.AI_PROVIDER_URL!, { method: 'POST', headers: { 'content-type': 'application/json',
authorization: 'Bearer ' + process.env.AI_PROVIDER_KEY }, body: JSON.stringify({ model: pol.model, max_tokens: 800, system:
'You help students using only the provided sources. Cite them as [n]. Say when you are unsure and suggest the official office.
Never state institutional rules as decided; the institution decides.', messages: [{ role: 'user', content: s.text +
'\n\nSources:\n' + JSON.stringify(ctx.data ?? []) }] }), signal: AbortSignal.timeout(20000) }).catch(() => null);
  if (!res || !res.ok) { await sb.schema('api').rpc('ai_log', { p_task: b.task, p_outcome: 'provider_error', p_reason:
String(res?.status ?? 'timeout'), p_tokens: 0 }); return NextResponse.json({ allowed: true, error: 'Ask Semester is unavailable
right now. The offices below can help.', label: d.label }, { status: 502 }); }
  const out = await res.json();
  await sb.schema('api').rpc('ai_log', { p_task: b.task, p_outcome: 'answered', p_reason: d.reason, p_tokens:
out.usage?.output_tokens ?? 0 });
  return NextResponse.json({ allowed: true, label: d.label, answer: out.content?.[0]?.text ?? '', sources: ctx.data ?? [],
dropped: d.dropped.length, redacted: s.findings.length, model: pol.model });
}
```

handoff/semester-platform/app/api/community/reports/route.ts · 12 lines

```
import { NextResponse } from 'next/server';
import { createClient } from '@/lib/supabase/server';
import { Report } from '@/lib/validate';
export async function POST(req: Request) {
  const parsed = Report.safeParse(await req.json());
  if (!parsed.success) return NextResponse.json({ error: 'Pick a reason' }, { status: 400 });
  const sb = await createClient();
  const { error } = await sb.schema('community').from('reports').insert({ target_type: parsed.data.targetType, target_id:
parsed.data.targetId, reason: parsed.data.reason, note: parsed.data.note });
  if (error) return NextResponse.json({ error: 'Could not send the report. Try again.' }, { status: 500 });
```

```
  return NextResponse.json({ message: 'Report sent to campus moderators.' }, { status: 201 });
}
```

handoff/semester-platform/app/api/market/listings/route.ts · 16 lines

```
import { NextResponse } from 'next/server';
import { createClient } from '@/lib/supabase/server';
import { Listing, integrityPrecheck } from '@/lib/validate';
export async function POST(req: Request) {
  const parsed = Listing.safeParse(await req.json());
  if (!parsed.success) return NextResponse.json({ error: 'Check the listing details', issues: parsed.error.flatten() }, {
status: 400 });
  const sb = await createClient();
  const { data: { user } } = await sb.auth.getUser();
  if (!user) return NextResponse.json({ error: 'Sign in to sell' }, { status: 401 });
  const pre = integrityPrecheck(parsed.data.title);
  const { data, error } = await sb.schema('market').from('listings').insert({ title: parsed.data.title, category:
parsed.data.category, price_cents: parsed.data.priceCents }).select().single();
  if (error) return NextResponse.json({ error: 'Your campus may not have the marketplace turned on.' }, { status: 403 });
  await sb.rpc('log', { a: 'listing.create', t: data.id, b: null, af: { ...data, precheck: pre } });
  return NextResponse.json({ listing: data, message: pre.hold ? 'Submitted. It will be reviewed before going live.' :
'Submitted. Live after the automatic check.' }, { status: 201 });
}
```

handoff/semester-platform/lib/ai/redact.ts · 22 lines

```
// Content scan that runs AFTER gate() and BEFORE any provider call. gate() trusts scope labels; this catches records typed or
pasted into the prompt.
// Patterns are deliberately conservative: a false positive costs a rephrase, a false negative costs a FERPA incident.
export interface Finding { kind: string; match: string }
export interface Scan { text: string; findings: Finding[]; block: boolean }
export interface TenantPatterns { studentId?: RegExp; extra?: { kind: string; re: RegExp }[] }
const BASE: { kind: string; re: RegExp; block?: boolean }[] = [
  { kind: 'ssn', re: /\b\d{3}-\d{2}-\d{4}\b/g, block: true },
  { kind: 'email', re: /\b[\w.+-]+@[\w-]+\.[\w.]+\b/g },
  { kind: 'phone', re: /\b(?:\+?1[ .-]?)?\(?\d{3}\)?[ .-]?\d{3}[ .-]?\d{4}\b/g },
  { kind: 'dob', re: /\b(?:born|dob|date of birth)[:\s]+\d{1,2}[\/.-]\d{1,2}[\/.-]\d{2,4}\b/gi, block: true },
  { kind: 'gpa', re: /\bgpa\s*(?:of|is|:)?\s*[0-4]\.\d{1,2}\b/gi, block: true },
  { kind: 'grade-record', re: /\b(?:got|received|earned|my grade (?:is|was))\s+(?:an?\s+)?[A-F][+-]?\s+(?:in|on)\b/gi, block:
true },
  { kind: 'transcript', re: /\b(?:transcript|academic record|disciplinary|accommodation (?:letter|plan)|financial aid
award)\b/gi, block: true },
  { kind: 'card', re: /\b(?:\d[ -]?){13,16}\b/g, block: true },
];
export function scan(input: string, t: TenantPatterns = {}): Scan {
  const rules = [...BASE, ...(t.studentId ? [{ kind: 'student-id', re: new RegExp(t.studentId.source, 'g'), block: true }] :
[]), ...(t.extra ?? []).map((e) => ({ ...e, block: true }))];
  const findings: Finding[] = []; let text = input; let block = false;
  for (const r of rules) { text = text.replace(r.re, (m) => { findings.push({ kind: r.kind, match: m }); if (r.block) block =
true; return '[' + r.kind + ' removed]'; }); }
  return { text, findings, block };
}
```

handoff/semester-platform/lib/validate.ts · 11 lines

```
import { z } from 'zod';
export const Listing = z.object({ title: z.string().min(3).max(120), category:
z.enum(['textbooks','gear','housing','services','tickets']), priceCents: z.number().int().min(0).max(1_000_000) });
export const Offer = z.object({ listingId: z.string().uuid(), amountCents: z.number().int().positive() });
export const Post = z.object({ groupId: z.string().uuid().nullable(), body: z.string().min(1).max(5000) });
export const Report = z.object({ targetType: z.enum(['post','message','listing','profile']), targetId: z.string().uuid(),
reason: z.enum(['harassment','spam','integrity','unsafe','other']), note: z.string().max(2000).optional() });
const BANNED = [/answer\s*key/i, /exam\s*\d*\s*(answers|solutions)/i, /graded\s*(essay|paper|work)/i, /write\s*your\s*
(essay|paper)/i];
export function integrityPrecheck(title: string): { hold: boolean; reason?: string } {
  const hit = BANNED.find(r => r.test(title));
  return hit ? { hold: true, reason: 'Possible academic integrity violation' } : { hold: false };
}
```

handoff/semester-platform/middleware.security.ts · 27 lines

```
// Security headers + session refresh for every request. Copy to the app root as middleware.ts (merge with existing auth
middleware).
import { NextResponse, type NextRequest } from 'next/server';
const CSP = [
  "default-src 'self'",
  "script-src 'self'",
  "style-src 'self' 'unsafe-inline'",
  "img-src 'self' data: blob: https://*.supabase.co",
  "connect-src 'self' https://*.supabase.co wss://*.supabase.co",
  "font-src 'self'",
  "frame-ancestors 'none'",
  "base-uri 'self'",
  "form-action 'self'",
  "upgrade-insecure-requests",
].join('; ');
export function middleware(req: NextRequest) {
  const res = NextResponse.next();
  res.headers.set('Content-Security-Policy', CSP);
  res.headers.set('Strict-Transport-Security', 'max-age=63072000; includeSubDomains; preload');
  res.headers.set('X-Content-Type-Options', 'nosniff');
  res.headers.set('Referrer-Policy', 'strict-origin-when-cross-origin');
  res.headers.set('Permissions-Policy', 'camera=(self), microphone=(self), geolocation=(self), payment=(), usb=()');
  res.headers.set('Cross-Origin-Opener-Policy', 'same-origin');
  if (req.nextUrl.pathname.startsWith('/api/')) res.headers.set('Cache-Control', 'no-store');
  return res;
}
export const config = { matcher: ['/((?!_next/static|_next/image|favicon.ico).*)'] };
```

handoff/semester-platform/supabase/migrations/010_platform_core.sql · 16 lines

```
-- Tenancy, profiles, roles, audit
create schema if not exists core; create schema if not exists audit;
create table core.institutions(id uuid primary key default gen_random_uuid(), name text not null, features jsonb not null
default '{}'::jsonb, created_at timestamptz default now());
create table core.profiles(id uuid primary key references auth.users on delete cascade, institution_id uuid not null references
core.institutions, display_name text not null, role text not null check (role in
('student','faculty','ta','advisor','registrar','student_affairs','it_admin','moderator','support')), visibility jsonb not null
default '{"major":true,"class_year":false}'::jsonb, created_at timestamptz default now());
create or replace function core.my_institution() returns uuid language sql stable as $$ select institution_id from
core.profiles where id = auth.uid() $$;
create or replace function core.my_role() returns text language sql stable as $$ select role from core.profiles where id =
auth.uid() $$;
create or replace function core.feature_on(f text) returns boolean language sql stable as $$ select coalesce((features-
>>f)::boolean,false) from core.institutions where id = core.my_institution() $$;
alter table core.profiles enable row level security;
create policy profiles_same_campus on core.profiles for select using (institution_id = core.my_institution());
create policy profiles_self_update on core.profiles for update using (id = auth.uid()) with check (id = auth.uid() and role =
core.my_role());
create table audit.events(id bigserial primary key, at timestamptz default now(), actor uuid, actor_role text, institution_id
uuid, action text not null, target text, before jsonb, after jsonb);
alter table audit.events enable row level security;
create policy audit_admin_read on audit.events for select using (institution_id = core.my_institution() and core.my_role() in
('it_admin','registrar'));
-- inserts only through security definer function
create or replace function audit.log(a text, t text, b jsonb, af jsonb) returns void language sql security definer set
search_path = '' as $$ insert into audit.events(actor,actor_role,institution_id,action,target,before,after) values (auth.uid(),
core.my_role(), core.my_institution(), a, t, b, af) $$;
```

handoff/semester-platform/supabase/migrations/020_community.sql · 22 lines

```
create schema if not exists community;
create table community.groups(id uuid primary key default gen_random_uuid(), institution_id uuid not null default
core.my_institution(), name text not null, kind text not null check (kind in ('course','club','community','rec')), created_by
uuid not null default auth.uid(), created_at timestamptz default now());
create table community.members(group_id uuid references community.groups on delete cascade, profile_id uuid references
core.profiles on delete cascade, role text not null default 'member' check (role in ('member','leader')), primary
key(group_id,profile_id));
create table community.posts(id uuid primary key default gen_random_uuid(), institution_id uuid not null default
core.my_institution(), group_id uuid references community.groups on delete cascade, author uuid not null default auth.uid(),
body text not null check (length(body) between 1 and 5000), status text not null default 'visible' check (status in
('visible','held','removed')), created_at timestamptz default now());
create table community.reports(id uuid primary key default gen_random_uuid(), institution_id uuid not null default
```

```
core.my_institution(), target_type text not null check (target_type in ('post','message','listing','profile')), target_id uuid
not null, reason text not null, note text, reporter uuid not null default auth.uid(), status text not null default 'open',
created_at timestamptz default now());
create table community.events(id uuid primary key default gen_random_uuid(), institution_id uuid not null default
core.my_institution(), group_id uuid references community.groups, title text not null, starts_at timestamptz not null, location
text);
create table community.rsvps(event_id uuid references community.events on delete cascade, profile_id uuid default auth.uid()
references core.profiles on delete cascade, primary key(event_id,profile_id));
alter table community.groups enable row level security; alter table community.members enable row level security; alter table
community.posts enable row level security; alter table community.reports enable row level security; alter table
community.events enable row level security; alter table community.rsvps enable row level security;
create policy g_read on community.groups for select using (institution_id = core.my_institution() and
core.feature_on('community'));
create policy g_create on community.groups for insert with check (institution_id = core.my_institution() and
core.feature_on('community'));
create policy m_read on community.members for select using (exists(select 1 from community.groups g where g.id = group_id and
g.institution_id = core.my_institution()));
create policy m_join on community.members for insert with check (profile_id = auth.uid() and role = 'member');
create policy p_read on community.posts for select using (institution_id = core.my_institution() and (status = 'visible' or
author = auth.uid() or core.my_role() in ('moderator','student_affairs')));
create policy p_write on community.posts for insert with check (author = auth.uid() and core.feature_on('community') and
(group_id is null or exists(select 1 from community.members m where m.group_id = posts.group_id and m.profile_id =
auth.uid())));
create policy p_moderate on community.posts for update using (core.my_role() in ('moderator','student_affairs') and
institution_id = core.my_institution());
create policy r_create on community.reports for insert with check (reporter = auth.uid());
create policy r_mod on community.reports for select using (institution_id = core.my_institution() and core.my_role() in
('moderator','student_affairs'));
create policy e_read on community.events for select using (institution_id = core.my_institution());
-- RSVPs visible to self and to group leaders only
create policy rs_self on community.rsvps for all using (profile_id = auth.uid()) with check (profile_id = auth.uid());
create policy rs_leader on community.rsvps for select using (exists(select 1 from community.events e join community.members m
on m.group_id = e.group_id where e.id = event_id and m.profile_id = auth.uid() and m.role = 'leader'));
```

handoff/semester-platform/supabase/migrations/030_marketplace.sql · 20 lines

```
create schema if not exists market;
create table market.listings(id uuid primary key default gen_random_uuid(), institution_id uuid not null default
core.my_institution(), seller uuid not null default auth.uid(), title text not null, category text not null check (category in
('textbooks','gear','housing','services','tickets')), price_cents int not null check (price_cents >= 0), status text not null
default 'pending_check' check (status in ('pending_check','live','held','sold','removed')), integrity_score real, created_at
timestamptz default now());
create table market.offers(id uuid primary key default gen_random_uuid(), listing_id uuid not null references market.listings
on delete cascade, buyer uuid not null default auth.uid(), amount_cents int not null check (amount_cents > 0), status text not
null default 'open' check (status in ('open','accepted','declined','paid','completed','disputed','refunded')), processor_ref
text, created_at timestamptz default now());
create table market.drops(id uuid primary key default gen_random_uuid(), institution_id uuid not null default
core.my_institution(), listing_id uuid references market.listings, seller_is_official boolean not null default false, ends_at
timestamptz not null, approved boolean not null default false);
create table market.bids(id bigserial primary key, drop_id uuid references market.drops on delete cascade, bidder uuid not null
default auth.uid(), amount_cents int not null, at timestamptz default now());
alter table market.listings enable row level security; alter table market.offers enable row level security; alter table
market.drops enable row level security; alter table market.bids enable row level security;
create policy l_read on market.listings for select using (institution_id = core.my_institution() and
core.feature_on('marketplace') and (status = 'live' or seller = auth.uid() or core.my_role() in
('moderator','student_affairs')));
-- new listings always start pending_check; a worker runs the integrity classifier and sets live/held
create policy l_create on market.listings for insert with check (seller = auth.uid() and status = 'pending_check' and
core.feature_on('marketplace'));
create policy l_seller_update on market.listings for update using (seller = auth.uid()) with check (seller = auth.uid() and
status in ('pending_check','sold','removed'));
create policy l_mod on market.listings for update using (core.my_role() in ('moderator','student_affairs') and institution_id =
core.my_institution());
create policy o_parties on market.offers for select using (buyer = auth.uid() or exists(select 1 from market.listings l where
l.id = listing_id and l.seller = auth.uid()));
create policy o_create on market.offers for insert with check (buyer = auth.uid() and status = 'open' and exists(select 1 from
market.listings l where l.id = listing_id and l.status = 'live' and l.seller <> auth.uid()));
create policy d_read on market.drops for select using (institution_id = core.my_institution() and approved);
create policy b_create on market.bids for insert with check (bidder = auth.uid() and exists(select 1 from market.drops d where
d.id = drop_id and d.approved and d.ends_at > now()));
create policy b_read on market.bids for select using (exists(select 1 from market.drops d where d.id = drop_id and
d.institution_id = core.my_institution()));
-- Bids must exceed current max
create or replace function market.check_bid() returns trigger language plpgsql as $$ begin if new.amount_cents <=
coalesce((select max(amount_cents) from market.bids where drop_id = new.drop_id),0) then raise exception 'Bid must be higher
```

```
than the current bid'; end if; return new; end $$;
create trigger bid_higher before insert on market.bids for each row execute function market.check_bid();
```

handoff/semester-platform/supabase/migrations/040_ops.sql · 18 lines

```
-- Company operations. No student identifiers in this schema, ever.
create schema if not exists ops;
create table ops.staff(id uuid primary key references auth.users, role text not null check (role in
('ceo','cfo','finance','comms','social','sales','revops','cs','support','tns','legal','people','security','data','partnerships'
)));
create or replace function ops.my_role() returns text language sql stable as $$ select role from ops.staff where id =
auth.uid() $$;
create table ops.accounts(id uuid primary key default gen_random_uuid(), institution_name text not null, acv_cents bigint,
term_ends date, health int check (health between 0 and 100), csm uuid references ops.staff);
create table ops.invoices(id text primary key, account_id uuid references ops.accounts, amount_cents bigint not null, due date
not null, status text not null default 'sent');
create table ops.social_posts(id uuid primary key default gen_random_uuid(), body text not null, channels text[] not null,
scheduled_at timestamptz, author uuid default auth.uid(), status text not null default 'draft' check (status in
('draft','needs_approval','approved','scheduled','published','paused','rejected')), legal_review boolean default false);
create table ops.settings(key text primary key, value jsonb not null);
alter table ops.accounts enable row level security; alter table ops.invoices enable row level security; alter table
ops.social_posts enable row level security; alter table ops.settings enable row level security;
create policy acc_read on ops.accounts for select using (ops.my_role() in
('ceo','cfo','finance','sales','revops','cs','partnerships'));
create policy inv_read on ops.invoices for select using (ops.my_role() in ('ceo','cfo','finance'));
create policy sp_read on ops.social_posts for select using (ops.my_role() in ('ceo','comms','social'));
create policy sp_draft on ops.social_posts for insert with check (ops.my_role() in ('comms','social') and status in
('draft','needs_approval'));
create policy sp_approve on ops.social_posts for update using (ops.my_role() = 'comms');
create policy st_read on ops.settings for select using (ops.my_role() is not null);
create policy st_crisis on ops.settings for update using (ops.my_role() in ('comms','ceo') and key = 'crisis_mode');
-- Guard: ops staff without a campus profile can never read edu/community/market (they have no core.profiles row, so
my_institution() is null).
```

handoff/semester-platform/supabase/migrations/050_security_hardening.sql · 80 lines

```
-- 050 · Security hardening from the Oct 2026 review of migrations 010–040.
-- Each block names the finding it closes (see docs/secure/Platform Security Review.html). Not yet run: apply in staging and
prove with tests/security.test.ts.

-- F1 core.institutions had no RLS: any signed-in user could read every tenant's feature flags.
alter table core.institutions enable row level security;
create policy inst_own_read on core.institutions for select using (id = core.my_institution());

-- F2 CRITICAL ops.staff had no RLS: a tenant user could read company staff or insert themselves as 'ceo' and gain every ops
policy.
alter table ops.staff enable row level security;
create policy staff_self_read on ops.staff for select using (id = auth.uid());
revoke insert, update, delete on ops.staff from anon, authenticated;
-- staff rows are created only by the service role from the company IdP.

-- F3 Helper functions read RLS-protected tables from inside RLS policies (recursion risk, planner surprises). Make them
definer functions with a fixed search_path.
create or replace function core.my_institution() returns uuid language sql stable security definer set search_path = '' as $$
select institution_id from core.profiles where id = auth.uid() $$;
create or replace function core.my_role() returns text language sql stable security definer set search_path = '' as $$ select
role from core.profiles where id = auth.uid() $$;
create or replace function core.feature_on(f text) returns boolean language sql stable security definer set search_path = '' as
$$ select coalesce((i.features->>f)::boolean, false) from core.institutions i where i.id = core.my_institution() $$;
create or replace function ops.my_role() returns text language sql stable security definer set search_path = '' as $$ select
role from ops.staff where id = auth.uid() $$;
revoke all on function core.my_institution(), core.my_role(), core.feature_on(text), ops.my_role() from anon;

-- F4 profiles_same_campus exposed every column (role, visibility settings) of every profile on campus, ignoring per-field
visibility.
drop policy if exists profiles_same_campus on core.profiles;
create policy profiles_self_read on core.profiles for select using (id = auth.uid());
create policy profiles_staff_read on core.profiles for select using (institution_id = core.my_institution() and core.my_role()
in ('advisor','registrar','student_affairs','it_admin','support','moderator'));
-- Directory for everyone else: only display name and fields the person made visible.
create or replace view core.directory with (security_barrier = true) as
  select p.id, p.display_name,
         case when (p.visibility->>'major')::boolean then p.visibility->>'major_value' end as major,
```

```
         case when (p.visibility->>'class_year')::boolean then p.visibility->>'class_year_value' end as class_year
  from core.profiles p
  where p.institution_id = core.my_institution() and coalesce((p.visibility->>'findable')::boolean, false);
grant select on core.directory to authenticated;

-- F5 HIGH profiles_self_update let a user move themselves to another institution_id (cross-tenant escalation). Freeze identity
columns.
create or replace function core.freeze_profile_identity() returns trigger language plpgsql security definer set search_path =
'' as $$
begin
  if new.id <> old.id or new.institution_id <> old.institution_id or new.role <> old.role then
    raise exception 'identity columns are managed by SSO provisioning';
  end if;
  return new;
end $$;
drop trigger if exists profiles_freeze on core.profiles;
create trigger profiles_freeze before update on core.profiles for each row execute function core.freeze_profile_identity();

-- F6 audit.log was executable by any user, so clients could write arbitrary audit rows.
revoke execute on function audit.log(text, text, jsonb, jsonb) from anon, authenticated;
-- server command functions (security definer) call audit.log; grant to the role that owns them:
-- grant execute on function audit.log(text, text, jsonb, jsonb) to service_role;

-- F7 Audit must be append-only.
revoke update, delete, truncate on audit.events from anon, authenticated;

-- F8 market.bids exposed every bidder's identity to the whole campus.
drop policy if exists b_read on market.bids;
create policy b_own on market.bids for select using (bidder = auth.uid());
create or replace view market.drop_bid_summary with (security_barrier = true) as
  select b.drop_id, count(*) as bids, max(b.amount_cents) as top_bid_cents
  from market.bids b join market.drops d on d.id = b.drop_id
  where d.institution_id = core.my_institution()
  group by b.drop_id;
grant select on market.drop_bid_summary to authenticated;

-- F9 community.members exposed membership of every group (including sensitive ones) campus-wide.
-- A policy on members cannot query members itself (infinite recursion), so membership is checked in a definer function.
create or replace function community.is_member(g uuid) returns boolean language sql stable security definer set search_path =
'' as $ select exists(select 1 from community.members m where m.group_id = g and m.profile_id = auth.uid()) $;
revoke all on function community.is_member(uuid) from anon;
drop policy if exists m_read on community.members;
create policy m_read_same_group on community.members for select using (
  profile_id = auth.uid() or community.is_member(group_id) or (core.my_role() in ('moderator','student_affairs') and exists
(select 1 from community.groups g where g.id = group_id and g.institution_id = core.my_institution())));

-- F10 rsvps 'all' policy checked only profile_id, so a user could RSVP to another institution's event by id.
drop policy if exists rs_self on community.rsvps;
create policy rs_self_read on community.rsvps for select using (profile_id = auth.uid());
create policy rs_self_write on community.rsvps for insert with check (profile_id = auth.uid() and exists (select 1 from
community.events e where e.id = event_id and e.institution_id = core.my_institution()));
create policy rs_self_delete on community.rsvps for delete using (profile_id = auth.uid());

-- F11 Default-deny for anonymous users on every schema.
revoke all on all tables in schema core, audit, community, market, ops from anon;
alter default privileges in schema core, audit, community, market, ops revoke all on tables from anon;
```

handoff/semester-platform/supabase/migrations/060_command_functions.sql · 101 lines

```
-- 060 · Write paths as audited command functions (closes "no insert policies" from the security review).
-- Clients never insert into these tables directly; every write goes through a function that validates, rate-limits, writes the
audit event first, then applies.
-- Not yet run. Prove with tests/commands.test.ts.

create schema if not exists api;

-- Simple per-user rate limit (window, max) backed by a counter table.
create table if not exists core.rate_counters(actor uuid not null, action text not null, window_start timestamptz not null, n
int not null default 0, primary key(actor, action, window_start));
alter table core.rate_counters enable row level security; -- no policies: definer functions only
create or replace function core.rate_ok(a text, per interval, max_n int) returns boolean language plpgsql security definer set
search_path = '' as $$
declare w timestamptz := date_trunc('minute', now()); c int;
begin
  insert into core.rate_counters(actor, action, window_start, n) values (auth.uid(), a, w, 1)
```

```
  on conflict (actor, action, window_start) do update set n = core.rate_counters.n + 1 returning n into c;
  select coalesce(sum(n),0) into c from core.rate_counters where actor = auth.uid() and action = a and window_start > now() -
per;
  return c <= max_n;
end $$;

-- Re-grant audit.log to the owner of the command functions (definer functions run as owner).
grant execute on function audit.log(text, text, jsonb, jsonb) to postgres;

-- Create a post. Public (non-group) posts start held for the AI pre-screen; group posts are visible.
create or replace function api.create_post(p_group uuid, p_body text) returns uuid language plpgsql security definer set
search_path = '' as $$
declare pid uuid; st text;
begin
  if auth.uid() is null then raise exception 'sign in required'; end if;
  if not core.feature_on('community') then raise exception 'community is off for this institution'; end if;
  if length(coalesce(p_body,'')) not between 1 and 5000 then raise exception 'post must be 1–5000 characters'; end if;
  if not core.rate_ok('create_post', interval '10 minutes', 20) then raise exception 'too many posts, try again shortly'; end
if;
  if p_group is not null and not community.is_member(p_group) then raise exception 'not a member of this group'; end if;
  st := case when p_group is null then 'held' else 'visible' end;
  pid := gen_random_uuid();
  perform audit.log('community.post.create', pid::text, null, jsonb_build_object('group', p_group, 'status', st, 'length',
length(p_body)));
  insert into community.posts(id, institution_id, group_id, author, body, status) values (pid, core.my_institution(), p_group,
auth.uid(), p_body, st);
  return pid;
end $$;

-- Create a listing. Always starts pending the academic-integrity check; never live on insert.
create or replace function api.create_listing(p_title text, p_category text, p_price_cents int) returns uuid language plpgsql
security definer set search_path = '' as $$
declare lid uuid;
begin
  if auth.uid() is null then raise exception 'sign in required'; end if;
  if not core.feature_on('marketplace') then raise exception 'marketplace is off for this institution'; end if;
  if core.my_role() <> 'student' then raise exception 'only students can list items'; end if;
  if length(coalesce(p_title,'')) not between 3 and 120 then raise exception 'title must be 3–120 characters'; end if;
  if p_category not in ('textbooks','gear','housing','services','tickets') then raise exception 'unknown category'; end if;
  if p_price_cents < 0 or p_price_cents > 1000000 then raise exception 'price out of range'; end if;
  if p_title ~* '(answer key|exam|test bank|solutions manual|graded essay|homework answers)' then raise exception 'listings
that break academic integrity rules are not allowed'; end if;
  if not core.rate_ok('create_listing', interval '1 day', 10) then raise exception 'daily listing limit reached'; end if;
  lid := gen_random_uuid();
  perform audit.log('market.listing.create', lid::text, null, jsonb_build_object('category', p_category, 'price_cents',
p_price_cents));
  insert into market.listings(id, institution_id, seller, title, category, price_cents, status) values (lid,
core.my_institution(), auth.uid(), p_title, p_category, p_price_cents, 'pending_check');
  return lid;
end $$;

-- Make an offer. Not on your own listing; listing must be live and on your campus.
create or replace function api.make_offer(p_listing uuid, p_amount_cents int) returns uuid language plpgsql security definer
set search_path = '' as $$
declare oid uuid; l record;
begin
  select * into l from market.listings where id = p_listing;
  if l is null or l.institution_id <> core.my_institution() or l.status <> 'live' then raise exception 'listing not available';
end if;
  if l.seller = auth.uid() then raise exception 'you cannot offer on your own listing'; end if;
  if p_amount_cents <= 0 or p_amount_cents > 1000000 then raise exception 'offer out of range'; end if;
  if not core.rate_ok('make_offer', interval '1 hour', 30) then raise exception 'too many offers, try again later'; end if;
  oid := gen_random_uuid();
  perform audit.log('market.offer.create', oid::text, null, jsonb_build_object('listing', p_listing, 'amount_cents',
p_amount_cents));
  insert into market.offers(id, listing_id, buyer, amount_cents) values (oid, p_listing, auth.uid(), p_amount_cents);
  return oid;
end $$;

-- Report content. The target must exist on the reporter's campus; reporter identity is visible to moderators only.
create or replace function api.report(p_type text, p_target uuid, p_reason text, p_note text) returns uuid language plpgsql
security definer set search_path = '' as $$
declare rid uuid; ok boolean;
begin
  if p_type not in ('post','message','listing','profile') then raise exception 'unknown report type'; end if;
  if p_reason not in ('harassment','hate','threat','privacy','integrity','scam','illegal','other') then raise exception
'unknown reason'; end if;
```

```
  ok := case p_type
    when 'post' then exists(select 1 from community.posts where id = p_target and institution_id = core.my_institution())
    when 'listing' then exists(select 1 from market.listings where id = p_target and institution_id = core.my_institution())
    when 'profile' then exists(select 1 from core.profiles where id = p_target and institution_id = core.my_institution())
    else true end;
  if not ok then raise exception 'target not found'; end if;
  if not core.rate_ok('report', interval '1 hour', 20) then raise exception 'too many reports, try again later'; end if;
  rid := gen_random_uuid();
  perform audit.log('community.report.create', rid::text, null, jsonb_build_object('type', p_type, 'reason', p_reason));
  insert into community.reports(id, institution_id, target_type, target_id, reason, note, reporter) values (rid,
core.my_institution(), p_type, p_target, p_reason, left(p_note, 2000), auth.uid());
  return rid;
end $$;

-- Only signed-in users may call commands; direct table writes stay closed.
revoke all on schema api from anon;
grant usage on schema api to authenticated;
grant execute on function api.create_post(uuid, text), api.create_listing(text, text, int), api.make_offer(uuid, int),
api.report(text, uuid, text, text) to authenticated;
revoke insert on community.posts, community.reports, market.listings, market.offers from authenticated;

-- Storage: listing photos and post attachments. Path convention: <institution_id>/<owner uid>/<file>.
-- insert into storage.buckets (id, name, public) values ('listing-photos', 'listing-photos', false) on conflict do nothing;
create policy photos_campus_read on storage.objects for select using (bucket_id = 'listing-photos' and
(storage.foldername(name))[1] = core.my_institution()::text);
create policy photos_owner_write on storage.objects for insert with check (bucket_id = 'listing-photos' and
(storage.foldername(name))[1] = core.my_institution()::text and (storage.foldername(name))[2] = auth.uid()::text);
create policy photos_owner_delete on storage.objects for delete using (bucket_id = 'listing-photos' and
(storage.foldername(name))[2] = auth.uid()::text);
```

handoff/semester-platform/supabase/migrations/070_ai_gateway.sql · 22 lines

```
-- 070 · AI gateway support: per-tenant policy, context of approved sources only, usage log. Not yet run.
create table if not exists core.ai_policy(institution_id uuid primary key references core.institutions, ai_on boolean not null
default false, weekly_budget int not null default 50, student_id_pattern text, provider text not null default 'approved-
default', model text not null);
alter table core.ai_policy enable row level security;
create policy aip_admin on core.ai_policy for select using (institution_id = core.my_institution() and core.my_role() =
'it_admin');
create table if not exists core.course_ai_rules(course_id uuid primary key, institution_id uuid not null references
core.institutions, allows text[] not null default '{}');
alter table core.course_ai_rules enable row level security;
create policy car_read on core.course_ai_rules for select using (institution_id = core.my_institution());
create table if not exists audit.ai_usage(id bigserial primary key, at timestamptz default now(), actor uuid, institution_id
uuid, task text, outcome text check (outcome in ('answered','refused','blocked','provider_error')), reason text, tokens int);
alter table audit.ai_usage enable row level security;
create policy aiu_admin on audit.ai_usage for select using (institution_id = core.my_institution() and core.my_role() =
'it_admin');
revoke update, delete, truncate on audit.ai_usage from anon, authenticated;
-- Note: ai_usage stores task, outcome and reason codes only. Never the prompt or the answer.
create or replace function api.ai_policy(p_course uuid) returns jsonb language sql stable security definer set search_path = ''
as $$
  select jsonb_build_object('institutionAiOn', p.ai_on, 'courseAllows', coalesce(r.allows, '{}'), 'model', p.model,
'studentIdPattern', p.student_id_pattern,
    'budgetLeft', p.weekly_budget - (select count(*) from audit.ai_usage u where u.actor = auth.uid() and u.outcome =
'answered' and u.at > now() - interval '7 days'))
  from core.ai_policy p left join core.course_ai_rules r on r.course_id = p_course and r.institution_id = p.institution_id
  where p.institution_id = core.my_institution() $$;
create or replace function api.ai_log(p_task text, p_outcome text, p_reason text, p_tokens int) returns void language sql
security definer set search_path = '' as $$
  insert into audit.ai_usage(actor, institution_id, task, outcome, reason, tokens) values (auth.uid(), core.my_institution(),
p_task, p_outcome, left(p_reason, 200), p_tokens) $$;
grant execute on function api.ai_policy(uuid), api.ai_log(text, text, text, int) to authenticated;
-- api.ai_context(p_ids) returns approved course/public sources by id with titles for citation; implement against the course
resource tables in stream 06.
```

handoff/semester-platform/supabase/migrations/080_sources_storage_delegations.sql · 43 lines

```
-- 080 · Remaining review items: approved-source context for AI, storage buckets, sessions and delegations. Not yet run.

-- Approved sources for AI answers: course resources and public institution pages only, never student records.
create table if not exists core.sources(id uuid primary key default gen_random_uuid(), institution_id uuid not null references
core.institutions, course_id uuid, kind text not null check (kind in ('policy','course','catalog','support','public')), title
```

```
text not null, url text, body text not null, version text, updated_at timestamptz default now(), approved boolean not null
default false);
alter table core.sources enable row level security;
create policy src_read on core.sources for select using (institution_id = core.my_institution() and approved);
create or replace function api.ai_context(p_ids text[]) returns jsonb language sql stable security definer set search_path = ''
as $$
  select coalesce(jsonb_agg(jsonb_build_object('n', rn, 'id', id, 'title', title, 'kind', kind, 'version', version, 'url', url,
'excerpt', left(body, 1500)) order by rn), '[]'::jsonb)
  from (select s.*, row_number() over (order by s.kind = 'policy' desc, s.updated_at desc) rn from core.sources s
        where s.institution_id = core.my_institution() and s.approved and s.id::text = any(p_ids) limit 8) t $$;
grant execute on function api.ai_context(text[]) to authenticated;

-- Storage: every bucket private; path = <institution_id>/<owner uid>/<file>.
-- buckets: 'student-files' (owner only), 'course-materials' (course members read, faculty write), 'evidence' (Semester trust
team only, service role).
create policy sf_owner on storage.objects for all using (bucket_id = 'student-files' and (storage.foldername(name))[1] =
core.my_institution()::text and (storage.foldername(name))[2] = auth.uid()::text) with check (bucket_id = 'student-files' and
(storage.foldername(name))[1] = core.my_institution()::text and (storage.foldername(name))[2] = auth.uid()::text);
create policy cm_read on storage.objects for select using (bucket_id = 'course-materials' and (storage.foldername(name))[1] =
core.my_institution()::text);
create policy cm_write on storage.objects for insert with check (bucket_id = 'course-materials' and (storage.foldername(name))
[1] = core.my_institution()::text and core.my_role() in ('faculty','ta'));
-- 'evidence' bucket: no policies for authenticated; service role only.

-- Sessions and delegations (account switcher, remote sign-out, delegated access with expiry).
create table if not exists core.delegations(id uuid primary key default gen_random_uuid(), institution_id uuid not null default
core.my_institution(), grantor uuid not null default auth.uid(), grantee uuid not null, scope text[] not null, purpose text not
null, expires_at timestamptz not null check (expires_at <= now() + interval '365 days'), revoked_at timestamptz, created_at
timestamptz default now());
alter table core.delegations enable row level security;
create policy dl_parties on core.delegations for select using (grantor = auth.uid() or grantee = auth.uid());
create or replace function api.grant_delegation(p_grantee uuid, p_scope text[], p_purpose text, p_expires timestamptz) returns
uuid language plpgsql security definer set search_path = '' as $$
declare did uuid;
begin
  if not exists(select 1 from core.profiles where id = p_grantee and institution_id = core.my_institution()) then raise
exception 'recipient not found'; end if;
  if p_expires <= now() or p_expires > now() + interval '365 days' then raise exception 'expiry must be within a year'; end if;
  did := gen_random_uuid();
  perform audit.log('core.delegation.grant', did::text, null, jsonb_build_object('grantee', p_grantee, 'scope', p_scope,
'expires', p_expires));
  insert into core.delegations(id, grantee, scope, purpose, expires_at) values (did, p_grantee, p_scope, p_purpose, p_expires);
  return did;
end $$;
create or replace function api.revoke_delegation(p_id uuid) returns void language plpgsql security definer set search_path = ''
as $$
begin
  perform audit.log('core.delegation.revoke', p_id::text, null, null);
  update core.delegations set revoked_at = now() where id = p_id and grantor = auth.uid() and revoked_at is null;
  if not found then raise exception 'not found'; end if;
end $$;
create or replace function core.has_delegation(p_grantor uuid, p_scope text) returns boolean language sql stable security
definer set search_path = '' as $$
  select exists(select 1 from core.delegations d where d.grantor = p_grantor and d.grantee = auth.uid() and p_scope =
any(d.scope) and d.revoked_at is null and d.expires_at > now()) $$;
grant execute on function api.grant_delegation(uuid, text[], text, timestamptz), api.revoke_delegation(uuid) to authenticated;
```

handoff/semester-platform/supabase/seed.sql · 41 lines

```
set search_path = public, extensions;
-- Seed runs as the postgres role, so RLS and defaults that call auth.uid() are bypassed or null: every row sets institution_id
and owner explicitly.
-- Test seed for local Supabase (supabase/seed.sql). Synthetic people only. IDs are fixed so tests can reference them via
.env.test.
-- Local test users. auth.identities rows are required for password sign-in on current Supabase versions.
insert into auth.users (instance_id, id, aud, role, email, encrypted_password, email_confirmed_at, raw_app_meta_data,
raw_user_meta_data, created_at, updated_at, confirmation_token, email_change, email_change_token_new, recovery_token) values
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-
00000000a001','authenticated','authenticated','student@campus-a.test',crypt('Test-pass-1!',
gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','',''),
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-
00000000a002','authenticated','authenticated','student2@campus-a.test',crypt('Test-pass-1!',
gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','',''),
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-00000000a003','authenticated','authenticated','seller@campus-
a.test',crypt('Test-pass-1!', gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','',''),
```

```
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-
00000000a004','authenticated','authenticated','advisor@campus-a.test',crypt('Test-pass-1!',
gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','',''),
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-00000000a005','authenticated','authenticated','it@campus-
a.test',crypt('Test-pass-1!', gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','',''),
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-
00000000b001','authenticated','authenticated','student@campus-b.test',crypt('Test-pass-1!',
gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','',''),
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-
00000000c001','authenticated','authenticated','cfo@semester.test',crypt('Test-pass-1!',
gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','',''),
 ('00000000-0000-0000-0000-000000000000','00000000-0000-0000-0000-
00000000c002','authenticated','authenticated','social@semester.test',crypt('Test-pass-1!',
gen_salt('bf')),now(),'{"provider":"email","providers":["email"]}','{}',now(),now(),'','','','');
insert into auth.identities (provider_id, user_id, identity_data, provider, last_sign_in_at, created_at, updated_at) values
 ('00000000-0000-0000-0000-00000000a001','00000000-0000-0000-0000-00000000a001','{"sub":"00000000-0000-0000-0000-
00000000a001","email":"student@campus-a.test"}','email',now(),now(),now()),
 ('00000000-0000-0000-0000-00000000a002','00000000-0000-0000-0000-00000000a002','{"sub":"00000000-0000-0000-0000-
00000000a002","email":"student2@campus-a.test"}','email',now(),now(),now()),
 ('00000000-0000-0000-0000-00000000a003','00000000-0000-0000-0000-00000000a003','{"sub":"00000000-0000-0000-0000-
00000000a003","email":"seller@campus-a.test"}','email',now(),now(),now()),
 ('00000000-0000-0000-0000-00000000a004','00000000-0000-0000-0000-00000000a004','{"sub":"00000000-0000-0000-0000-
00000000a004","email":"advisor@campus-a.test"}','email',now(),now(),now()),
 ('00000000-0000-0000-0000-00000000a005','00000000-0000-0000-0000-00000000a005','{"sub":"00000000-0000-0000-0000-
00000000a005","email":"it@campus-a.test"}','email',now(),now(),now()),
 ('00000000-0000-0000-0000-00000000b001','00000000-0000-0000-0000-00000000b001','{"sub":"00000000-0000-0000-0000-
00000000b001","email":"student@campus-b.test"}','email',now(),now(),now()),
 ('00000000-0000-0000-0000-00000000c001','00000000-0000-0000-0000-00000000c001','{"sub":"00000000-0000-0000-0000-
00000000c001","email":"cfo@semester.test"}','email',now(),now(),now()),
 ('00000000-0000-0000-0000-00000000c002','00000000-0000-0000-0000-00000000c002','{"sub":"00000000-0000-0000-0000-
00000000c002","email":"social@semester.test"}','email',now(),now(),now());
insert into core.institutions (id, name, features) values
 ('00000000-0000-0000-0000-0000000000aa','Campus A','{"community":true,"marketplace":true}'),
 ('00000000-0000-0000-0000-0000000000bb','Campus B','{"community":true,"marketplace":true}');
insert into core.profiles (id, institution_id, display_name, role, visibility) values
 ('00000000-0000-0000-0000-00000000a001','00000000-0000-0000-0000-0000000000aa','Student
A','student','{"findable":true,"major":true,"major_value":"Biology"}'),
 ('00000000-0000-0000-0000-00000000a002','00000000-0000-0000-0000-0000000000aa','Student Two','student','{"findable":false}'),
 ('00000000-0000-0000-0000-00000000a003','00000000-0000-0000-0000-0000000000aa','Seller','student','{"findable":true}'),
 ('00000000-0000-0000-0000-00000000a004','00000000-0000-0000-0000-0000000000aa','Advisor Lee','advisor','{}'),
 ('00000000-0000-0000-0000-00000000a005','00000000-0000-0000-0000-0000000000aa','IT Admin','it_admin','{}'),
 ('00000000-0000-0000-0000-00000000b001','00000000-0000-0000-0000-0000000000bb','Student B','student','{}');
insert into ops.staff (id, role) values ('00000000-0000-0000-0000-00000000c001','cfo'),('00000000-0000-0000-0000-
00000000c002','social');
insert into community.groups (id, institution_id, name, kind, created_by) values ('00000000-0000-0000-0000-
00000000d001','00000000-0000-0000-0000-0000000000aa','Private study group','community','00000000-0000-0000-0000-00000000a001');
insert into community.members (group_id, profile_id, role) values ('00000000-0000-0000-0000-00000000d001','00000000-0000-0000-
0000-00000000a001','leader');
insert into community.posts (id, institution_id, author, body) values ('00000000-0000-0000-0000-00000000e001','00000000-0000-
0000-0000-0000000000aa','00000000-0000-0000-0000-00000000a001','Seed post');
insert into community.events (id, institution_id, title, starts_at) values ('00000000-0000-0000-0000-00000000f0b1','00000000-
0000-0000-0000-0000000000bb','Campus B event', now() + interval '7 days');
insert into market.listings (id, institution_id, seller, title, category, price_cents, status) values ('00000000-0000-0000-
0000-00000000aa11','00000000-0000-0000-0000-0000000000aa','00000000-0000-0000-0000-00000000a003','Seed
lamp','gear',500,'live');
insert into core.ai_policy (institution_id, ai_on, weekly_budget, student_id_pattern, model) values ('00000000-0000-0000-0000-
0000000000aa', true, 50, '\\b\\d{9}\\b', 'approved-default');
-- If a column above does not exist in the repo's schema, fix the seed to match the migration (the migration is the source of
truth), never the reverse.
```

handoff/semester-platform/tests/ai-redteam.test.ts · 22 lines

```
// AI red-team suite (launch gate). Unit-level: no provider call needed. npx vitest run tests/ai-redteam.test.ts
import { describe, it, expect } from 'vitest';
import { gate } from '../lib/core/aigateway';
import { scan } from '../lib/ai/redact';
const SID = /\b\d{9}\b/; // tenant student-ID format, e.g. Vanderbilt-style 9 digits (set per tenant)
const policy = { institutionAiOn: true, courseAllows: ['outline', 'practice'] as any };
const blocked = [
  'My student ID is 004512398, what classes am I in?', 'Summarize my transcript', 'I got a C- in BIO 2100, explain my GPA
2.71',
  'My SSN is 123-45-6789', 'DOB: 04/12/2005 can you check my aid?', 'Here is my accommodation letter text', 'Read my financial
aid award and tell me what to do',
  'Card 4111 1111 1111 1111 for the bill', 'Write about my disciplinary record', 'Student 004512398 missed class, draft an
email',
```

```
];
const allowed = ['When is the add/drop deadline?', 'Explain enzyme kinetics simply', 'Make 5 practice questions on supply and
demand', 'Who do I contact about a registration hold?', 'How do I share my plan with my advisor?'];
describe('AI red-team', () => {
  for (const p of blocked) it('blocks: ' + p.slice(0, 40), () => { expect(scan(p, { studentId: SID }).block).toBe(true); });
  for (const p of allowed) it('allows: ' + p.slice(0, 40), () => { expect(scan(p, { studentId: SID }).block).toBe(false); });
  it('gate drops student-record scope even when allowed', () => { const d = gate({ task: 'plan', scope: [{ id: 'a', cls:
'course' }, { id: 'b', cls: 'student-record' }], policy, budgetLeft: 5 }); expect(d.sendScope).toEqual(['a']);
expect(d.dropped).toEqual(['b']); });
  it('institution switch off refuses everything', () => { expect(gate({ task: 'search', scope: [], policy: { ...policy,
institutionAiOn: false }, budgetLeft: 5 }).allowed).toBe(false); });
  it('course policy refuses disallowed task', () => { expect(gate({ task: 'write', scope: [], policy, budgetLeft: 5
}).allowed).toBe(false); });
  it('budget exhausted refuses', () => { expect(gate({ task: 'search', scope: [], policy, budgetLeft: 0
}).allowed).toBe(false); });
  it('redaction never leaks the match', () => { const s = scan('email me at a.student@vanderbilt.edu', {});
expect(s.text).not.toMatch(/vanderbilt\.edu/); });
});
```

handoff/semester-platform/tests/commands.test.ts · 19 lines

```
// Proves migration 060. npx supabase start && npx vitest run tests/commands.test.ts
import { describe, it, expect } from 'vitest';
import { asUser } from './helpers';
const rpc = (c, fn, args) => c.schema('api').rpc(fn, args);
describe('060 command functions', () => {
  it('direct inserts are closed', async () => { const c = await asUser('student@campus-a.test'); const { error } = await
c.schema('community').from('posts').insert({ body: 'hi' }); expect(error).not.toBeNull(); });
  it('public posts start held for pre-screen', async () => { const c = await asUser('student@campus-a.test'); const { data: id
} = await rpc(c, 'create_post', { p_group: null, p_body: 'Study group tonight?' }); const { data } = await
c.schema('community').from('posts').select('status').eq('id', id).single(); expect(data.status).toBe('held'); });
  it('non-members cannot post in a group', async () => { const c = await asUser('student2@campus-a.test'); const { error } =
await rpc(c, 'create_post', { p_group: process.env.PRIVATE_GROUP, p_body: 'x' }); expect(error?.message).toMatch(/not a
member/); });
  it('listings never go live on create', async () => { const c = await asUser('student@campus-a.test'); const { data: id } =
await rpc(c, 'create_listing', { p_title: 'BIO textbook', p_category: 'textbooks', p_price_cents: 4000 }); const { data } =
await c.schema('market').from('listings').select('status').eq('id', id).single(); expect(data.status).toBe('pending_check');
});
  it('integrity filter blocks exam material', async () => { const c = await asUser('student@campus-a.test'); const { error } =
await rpc(c, 'create_listing', { p_title: 'CHEM 101 answer key', p_category: 'textbooks', p_price_cents: 500 });
expect(error?.message).toMatch(/academic integrity/); });
  it('staff cannot list items', async () => { const c = await asUser('advisor@campus-a.test'); const { error } = await rpc(c,
'create_listing', { p_title: 'Desk', p_category: 'gear', p_price_cents: 100 }); expect(error).not.toBeNull(); });
  it('no offers on own listing', async () => { const c = await asUser('seller@campus-a.test'); const { error } = await rpc(c,
'make_offer', { p_listing: process.env.SELLER_LISTING, p_amount_cents: 100 }); expect(error?.message).toMatch(/own listing/);
});
  it('no offers across campuses', async () => { const c = await asUser('student@campus-b.test'); const { error } = await rpc(c,
'make_offer', { p_listing: process.env.SELLER_LISTING, p_amount_cents: 100 }); expect(error?.message).toMatch(/not available/);
});
  it('reports must target own campus', async () => { const c = await asUser('student@campus-b.test'); const { error } = await
rpc(c, 'report', { p_type: 'post', p_target: process.env.CAMPUS_A_POST, p_reason: 'harassment', p_note: null });
expect(error?.message).toMatch(/not found/); });
  it('rate limit holds', async () => { const c = await asUser('student@campus-a.test'); let last; for (let i = 0; i < 22; i++)
last = await rpc(c, 'create_post', { p_group: null, p_body: 'spam ' + i }); expect(last.error?.message).toMatch(/too many/);
});
  it('every command writes an audit event first', async () => { const it = await asUser('it@campus-a.test'); const { data } =
await it.schema('audit').from('events').select('action').like('action', 'market.listing.create'); expect((data ??
[]).length).toBeGreaterThan(0); });
  it('photos only under own campus and uid', async () => { const c = await asUser('student@campus-a.test'); const { error } =
await c.storage.from('listing-photos').upload(process.env.CAMPUS_B + '/' + process.env.STUDENT_A_ID + '/x.jpg', new
Blob(['x'])); expect(error).not.toBeNull(); });
});
```

handoff/semester-platform/tests/helpers.ts · 14 lines

```
// tests/helpers.ts — signed-in and anonymous clients against local Supabase.
import { createClient, type SupabaseClient } from '@supabase/supabase-js';
import 'dotenv/config';
const url = process.env.SUPABASE_URL!, key = process.env.SUPABASE_ANON_KEY!;
const cache = new Map<string, SupabaseClient>();
export async function asUser(email: string): Promise<SupabaseClient> {
  if (cache.has(email)) return cache.get(email)!;
  const c = createClient(url, key, { auth: { persistSession: false } });
```

```
  const { error } = await c.auth.signInWithPassword({ email, password: process.env.TEST_PASSWORD! });
  if (error) throw new Error('seed user missing: ' + email + ' — run npx supabase db reset');
  cache.set(email, c); return c;
}
export function anon(): SupabaseClient { return createClient(url, key, { auth: { persistSession: false } }); }
```

handoff/semester-platform/tests/rls.test.ts · 12 lines

```
// Run against local Supabase: npx supabase start && npx vitest run tests/rls.test.ts
import { describe, it, expect } from 'vitest';
import { asUser } from './helpers'; // signs in seeded users and returns a scoped client
describe('RLS boundaries', () => {
  it('ops staff read zero community posts', async () => { const c = await asUser('cfo@semester.test'); const { data } = await
c.schema('community').from('posts').select('id'); expect(data ?? []).toHaveLength(0); });
  it('students cannot read another campus', async () => { const c = await asUser('student@campus-a.test'); const { data } =
await c.schema('market').from('listings').select('institution_id'); expect((data ?? []).every(r => r.institution_id ===
process.env.CAMPUS_A)).toBe(true); });
  it('new listings cannot skip the check', async () => { const c = await asUser('student@campus-a.test'); const { error } =
await c.schema('market').from('listings').insert({ title: 'Lamp', category: 'gear', price_cents: 500, status: 'live' });
expect(error).not.toBeNull(); });
  it('sellers cannot offer on their own listing', async () => { const c = await asUser('seller@campus-a.test'); const { error }
= await c.schema('market').from('offers').insert({ listing_id: process.env.SELLER_LISTING, amount_cents: 100 });
expect(error).not.toBeNull(); });
  it('RSVPs hidden from non-leaders', async () => { const c = await asUser('student2@campus-a.test'); const { data } = await
c.schema('community').from('rsvps').select('*').neq('profile_id', process.env.STUDENT2_ID); expect(data ?? []).toHaveLength(0);
});
  it('social manager cannot approve posts', async () => { const c = await asUser('social@semester.test'); const { data } =
await c.schema('ops').from('social_posts').update({ status: 'approved' }).eq('status','needs_approval').select(); expect(data
?? []).toHaveLength(0); });
});
```

handoff/semester-platform/tests/security.test.ts · 19 lines

```
// Proves migration 050. Run against local Supabase: npx supabase start && npx vitest run tests/security.test.ts
// Each test should FAIL before 050 is applied and PASS after (record both runs in the PR).
import { describe, it, expect } from 'vitest';
import { asUser, anon } from './helpers';
describe('050 security hardening', () => {
  it('F1 students read only their own institution row', async () => { const c = await asUser('student@campus-a.test'); const {
data } = await c.schema('core').from('institutions').select('id'); expect((data ?? []).map(r =>
r.id)).toEqual([process.env.CAMPUS_A]); });
  it('F2 tenant users cannot read or insert ops.staff', async () => { const c = await asUser('student@campus-a.test'); const r
= await c.schema('ops').from('staff').select('id'); expect(r.data ?? []).toHaveLength(0); const w = await
c.schema('ops').from('staff').insert({ id: process.env.STUDENT_A_ID, role: 'ceo' }); expect(w.error).not.toBeNull(); });
  it('F4 students cannot read other profiles directly', async () => { const c = await asUser('student@campus-a.test'); const {
data } = await c.schema('core').from('profiles').select('id').neq('id', process.env.STUDENT_A_ID); expect(data ??
[]).toHaveLength(0); });
  it('F4 directory shows only findable people and visible fields', async () => { const c = await asUser('student@campus-
a.test'); const { data } = await c.schema('core').from('directory').select('*'); expect((data ?? []).every(r => !('role' in r)
&& !('visibility' in r))).toBe(true); });
  it('F5 users cannot move themselves to another institution', async () => { const c = await asUser('student@campus-a.test');
const { error } = await c.schema('core').from('profiles').update({ institution_id: process.env.CAMPUS_B }).eq('id',
process.env.STUDENT_A_ID); expect(error).not.toBeNull(); });
  it('F5 users cannot change their role', async () => { const c = await asUser('student@campus-a.test'); const { error } =
await c.schema('core').from('profiles').update({ role: 'registrar' }).eq('id', process.env.STUDENT_A_ID);
expect(error).not.toBeNull(); });
  it('F6 clients cannot call audit.log', async () => { const c = await asUser('student@campus-a.test'); const { error } = await
c.schema('audit').rpc('log', { a: 'fake', t: 'x', b: {}, af: {} }); expect(error).not.toBeNull(); });
  it('F7 audit rows cannot be deleted', async () => { const c = await asUser('it@campus-a.test'); const { data } = await
c.schema('audit').from('events').delete().neq('id', 0).select(); expect(data ?? []).toHaveLength(0); });
  it('F8 bidders are not visible to other students', async () => { const c = await asUser('student2@campus-a.test'); const {
data } = await c.schema('market').from('bids').select('bidder'); expect((data ?? []).every(r => r.bidder ===
process.env.STUDENT2_ID)).toBe(true); });
  it('F9 non-members cannot list group members', async () => { const c = await asUser('student2@campus-a.test'); const { data }
= await c.schema('community').from('members').select('profile_id').eq('group_id', process.env.PRIVATE_GROUP); expect(data ??
[]).toHaveLength(0); });
  it('F10 cannot RSVP to another campus event', async () => { const c = await asUser('student@campus-a.test'); const { error }
= await c.schema('community').from('rsvps').insert({ event_id: process.env.CAMPUS_B_EVENT, profile_id: process.env.STUDENT_A_ID
}); expect(error).not.toBeNull(); });
  it('F11 anonymous clients read nothing', async () => { const c = anon(); for (const [s, t] of [['core','profiles'],
['community','posts'],['market','listings'],['ops','accounts']]) { const { data } = await
```

```
c.schema(s).from(t).select('*').limit(1); expect(data ?? []).toHaveLength(0); } });
});
```

handoff/semester-platform/tests/validate.test.ts · 8 lines

```
import { describe, it, expect } from 'vitest';
import { integrityPrecheck, Listing } from '../lib/validate';
describe('integrity precheck', () => {
  it('holds answer keys', () => expect(integrityPrecheck('CHEM 210 exam 2 answers').hold).toBe(true));
  it('allows textbooks', () => expect(integrityPrecheck('Organic Chemistry 9th ed').hold).toBe(false));
});
describe('Listing schema', () => { it('rejects negative price', () => expect(Listing.safeParse({ title: 'Lamp', category:
'gear', priceCents: -1 }).success).toBe(false)); });
```

Workflow router and benchmark app

handoff/workflow-router-app/app/api/exports/benchmark/route.ts · 20 lines

```
import { NextResponse } from 'next/server';
import { requireUser } from '@/lib/supabase/server';
import { aggregateGrades } from '@/lib/benchmark/scoring';
import { toCsv } from '@/lib/exports/csv';
import { benchmarkMarkdown } from '@/lib/exports/markdown';
export async function GET(req: Request) {
  const url = new URL(req.url); const suiteId = url.searchParams.get('suite'); const format = url.searchParams.get('format')
=== 'md' ? 'md' : 'csv';
  const { supabase, user } = await requireUser(); if (!user) return NextResponse.json({ error: 'Sign in' }, { status: 401 });
  if (!suiteId) return NextResponse.json({ error: 'suite is required' }, { status: 422 });
  const { data: tasks, error } = await supabase.from('benchmark_tasks').select('id, number, weights, benchmark_runs(id,
platform, status, benchmark_grades(*))').eq('suite_id', suiteId).order('number'); // RLS scopes this
  if (error) return NextResponse.json({ error: error.message }, { status: 500 });
  const rows = (tasks ?? []).flatMap((t: any) => (t.benchmark_runs ?? []).map((r: any) => {
    const sheets = (r.benchmark_grades ?? []).map((g: any) => ({ correctness: g.correctness, codeQuality: g.code_quality,
rendering: g.rendering, stateManagement: g.state_management, maintainability: g.maintainability, handoff: g.handoff,
sandboxFit: g.sandbox_fit, persistence: g.persistence }));
    const s = aggregateGrades(sheets, t.weights ?? undefined);
    return { workflow: t.number, platform: r.platform, status: r.status, outOf5: s.outOf5, outOf100: s.outOf100, graders:
s.graders, missing: s.missing.join(' ') };
  }));
  const body = format === 'md' ? benchmarkMarkdown(rows) : toCsv(rows, ['workflow', 'platform', 'status', 'outOf5', 'outOf100',
'graders', 'missing']);
  return new NextResponse(body, { headers: { 'content-type': format === 'md' ? 'text/markdown; charset=utf-8' : 'text/csv;
charset=utf-8', 'content-disposition': 'attachment; filename="semester-benchmark.' + format + '"' } });
}
```

handoff/workflow-router-app/app/api/exports/recommendation/route.ts · 15 lines

```
import { NextResponse } from 'next/server';
import { RouterAnswersSchema } from '@/lib/validation/workflow-router';
import { recommend } from '@/lib/workflow-router/engine';
import { buildPrompt } from '@/lib/workflow-router/prompts';
import { recommendationMarkdown } from '@/lib/exports/markdown';
import { toJson } from '@/lib/exports/json';
export async function POST(req: Request) {
  const format = new URL(req.url).searchParams.get('format') === 'json' ? 'json' : 'md';
  const parsed = RouterAnswersSchema.safeParse(await req.json().catch(() => null));
  if (!parsed.success) return NextResponse.json({ error: parsed.error.flatten() }, { status: 422 });
  const r = recommend(parsed.data); const prompt = buildPrompt(parsed.data, r);
  const body = format === 'json' ? toJson({ answers: parsed.data, recommendation: r, prompt }) :
recommendationMarkdown(parsed.data, r, prompt);
  return new NextResponse(body, { headers: { 'content-type': format === 'json' ? 'application/json' : 'text/markdown;
charset=utf-8', 'content-disposition': 'attachment; filename="semester-recommendation.' + format + '"' } });
}
```

handoff/workflow-router-app/app/auth/confirm/route.ts · 11 lines

```
import { type EmailOtpType } from '@supabase/supabase-js';
import { NextResponse, type NextRequest } from 'next/server';
import { createClient } from '@/lib/supabase/server';
export async function GET(request: NextRequest) {
  const url = new URL(request.url);
  const token_hash = url.searchParams.get('token_hash'); const type = url.searchParams.get('type') as EmailOtpType | null;
  const next = url.searchParams.get('next')?.startsWith('/') ? url.searchParams.get('next')! : '/workflow-router';
  if (token_hash && type) { const supabase = await createClient(); const { error } = await supabase.auth.verifyOtp({ type,
token_hash }); if (!error) return NextResponse.redirect(new URL(next, url.origin)); }
  return NextResponse.redirect(new URL('/auth/sign-in?error=link', url.origin));
}
```

handoff/workflow-router-app/app/auth/sign-in/page.tsx · 17 lines

```
'use client';
import { useState } from 'react';
import { createClient } from '@/lib/supabase/client';
export default function SignIn() {
  const [email, setEmail] = useState(''); const [state, setState] = useState<'idle' | 'sending' | 'sent' | 'error'>('idle');
const [msg, setMsg] = useState('');
  async function send(e: React.FormEvent) {
    e.preventDefault(); setState('sending');
    const { error } = await createClient().auth.signInWithOtp({ email, options: { emailRedirectTo: location.origin +
'/auth/confirm?next=/workflow-router' } });
    if (error) { setState('error'); setMsg(error.message); } else setState('sent');
  }
  return (<main className="mx-auto max-w-sm p-8"><h1 className="text-2xl font-semibold">Sign in</h1>
    <p className="mt-2 text-sm text-neutral-600">We'll email you a sign-in link. Without signing in, the router works in
prototype mode and nothing is saved.</p>
    <form onSubmit={send} className="mt-6 grid gap-3"><label className="grid gap-1 text-sm">School or work email<input required
type="email" value={email} onChange={(e) => setEmail(e.target.value)} className="min-h-11 rounded border px-3" /></label>
      <button disabled={state === 'sending'} className="min-h-11 rounded bg-slate-800 px-4 font-medium text-white
disabled:opacity-50">{state === 'sending' ? 'Sending…' : 'Email me a link'}</button></form>
    <p role="status" className="mt-4 text-sm">{state === 'sent' ? 'Check your inbox — the link signs you in on this device.' :
state === 'error' ? msg : ''}</p></main>);
}
```

handoff/workflow-router-app/app/auth/sign-out/route.ts · 4 lines

```
import { NextResponse } from 'next/server';
import { createClient } from '@/lib/supabase/server';
export async function POST(request: Request) { const supabase = await createClient(); await supabase.auth.signOut(); return
NextResponse.redirect(new URL('/', request.url), { status: 303 }); }
```

handoff/workflow-router-app/app/benchmark/-suiteId-/page.tsx · 16 lines

```
import { notFound } from 'next/navigation';
import { requireUser } from '@/lib/supabase/server';
import { aggregateGrades, leaderboard } from '@/lib/benchmark/scoring';
import { PLATFORM_LABEL, PLATFORMS, type Platform } from '@/lib/benchmark/benchmark.config';
import { Leaderboard } from '@/components/benchmark/leaderboard';
export default async function Page({ params }: { params: Promise<{ suiteId: string }> }) {
  const { suiteId } = await params; const { supabase, user } = await requireUser(); if (!user) notFound();
  const { data: suite } = await supabase.from('benchmark_suites').select('id, name').eq('id', suiteId).single(); if (!suite)
notFound(); // RLS hides other users' suites
  const { data: tasks } = await supabase.from('benchmark_tasks').select('id, number, title, weights, benchmark_runs(id,
platform, status, benchmark_grades(*))').eq('suite_id', suiteId).order('number');
  const cells = (tasks ?? []).flatMap((t: any) => PLATFORMS.map((p) => { const run = (t.benchmark_runs ?? []).find((r: any) =>
r.platform === p); const sheets = (run?.benchmark_grades ?? []).map((g: any) => ({ correctness: g.correctness, codeQuality:
g.code_quality, rendering: g.rendering, stateManagement: g.state_management, maintainability: g.maintainability, handoff:
g.handoff, sandboxFit: g.sandbox_fit, persistence: g.persistence })); return { task: t, platform: p as Platform, run, s:
aggregateGrades(sheets, t.weights ?? undefined) }; }));
  return (<div className="grid gap-8"><div className="flex items-end justify-between"><h1 className="text-2xl font-semibold">
{suite.name}</h1><div className="flex gap-2 text-sm"><a className="rounded border px-3 py-2" href={'/api/exports/benchmark?
suite=' + suiteId + '&format=csv'}>CSV</a><a className="rounded border px-3 py-2" href={'/api/exports/benchmark?suite=' +
suiteId + '&format=md'}>Markdown report</a></div></div>
    <Leaderboard title="Aggregate platform leaderboard" rows={leaderboard(cells.map((c) => ({ key: PLATFORM_LABEL[c.platform],
score: c.s.outOf5 })))} />
    <div className="overflow-x-auto rounded border"><table className="w-full text-sm"><thead className="bg-neutral-100"><tr><th
className="px-3 py-2 text-left">#</th><th className="px-3 py-2 text-left">Workflow</th>{PLATFORMS.map((p) => <th key={p}
className="px-3 py-2 text-left">{PLATFORM_LABEL[p]}</th>)}</tr></thead>
      <tbody>{(tasks ?? []).map((t: any) => <tr key={t.id} className="border-t"><td className="px-3 py-2 tabular-nums">
{t.number}</td><td className="px-3 py-2">{t.title}</td>{PLATFORMS.map((p) => { const c = cells.find((x) => x.task.id === t.id
&& x.platform === p)!; return <td key={p} className="px-3 py-2 tabular-nums">{!c.run ? 'No run' : c.run.status !== 'completed'
? c.run.status : c.s.outOf100 === null ? 'Not graded' : c.s.outOf100 + ' · ' + c.s.graders + ' grader' + (c.s.graders === 1 ?
'' : 's')}</td>; })}</tr>)}</tbody></table></div></div>);
}
```

handoff/workflow-router-app/app/benchmark/actions.ts · 21 lines

```
'use server';
import { revalidatePath } from 'next/cache';
import { requireUser } from '@/lib/supabase/server';
```

```
import { GradeSchema, RunSchema } from '@/lib/validation/benchmark';
export async function recordRun(input: unknown) {
  const p = RunSchema.safeParse(input); if (!p.success) return { ok: false as const, error: p.error.flatten() };
  const { supabase, user } = await requireUser(); if (!user) return { ok: false as const, error: 'Sign in to record runs.' };
  const d = p.data;
  const { data, error } = await supabase.from('benchmark_runs').insert({ task_id: d.taskId, platform: d.platform, created_by:
user.id, status: d.status, status_reason: d.statusReason ?? null, executed_prompt: d.executedPrompt, model_name: d.modelName ??
null, model_version: d.modelVersion ?? null, output_url: d.outputUrl || null, source_manifest: d.sourceManifest, runtime_notes:
d.runtimeNotes ?? null, console_notes: d.consoleNotes ?? null, duration_ms: d.durationMs ?? null, persistence_tested:
d.persistenceTested, persistence_survived_reload: d.persistenceSurvivedReload, persistence_mode: d.persistenceMode,
persistence_evidence: d.persistenceEvidence ?? null, persistence_failure_notes: d.persistenceFailureNotes ?? null
}).select('id').single();
  if (error) return { ok: false as const, error: error.message };
  revalidatePath('/benchmark'); return { ok: true as const, id: data.id };
}
export async function gradeRun(input: unknown) {
  const p = GradeSchema.safeParse(input); if (!p.success) return { ok: false as const, error: p.error.flatten() };
  const { supabase, user } = await requireUser(); if (!user) return { ok: false as const, error: 'Sign in to grade.' };
  const g = p.data;
  const { error } = await supabase.from('benchmark_grades').upsert({ run_id: g.runId, grader_id: user.id, correctness:
g.correctness, code_quality: g.codeQuality, rendering: g.rendering, state_management: g.stateManagement, maintainability:
g.maintainability, handoff: g.handoff, sandbox_fit: g.sandboxFit, persistence: g.persistence, evidence: g.evidence ?? null,
notes: g.notes ?? null }, { onConflict: 'run_id,grader_id' });
  if (error) return { ok: false as const, error: error.message };
  revalidatePath('/benchmark'); return { ok: true as const };
}
```

handoff/workflow-router-app/app/benchmark/page.tsx · 10 lines

```
import Link from 'next/link';
import { requireUser } from '@/lib/supabase/server';
export default async function Page() {
  const { supabase, user } = await requireUser();
  if (!user) return (<div className="grid gap-3"><h1 className="text-2xl font-semibold">Benchmark lab</h1><p>Sign in to create
suites, record runs and grade them. Benchmark data is private to you or your organization.</p><Link className="w-fit rounded
bg-slate-800 px-4 py-2 text-white" href="/auth/sign-in">Sign in</Link></div>);
  const { data: suites, error } = await supabase.from('benchmark_suites').select('id, name, description,
created_at').order('created_at', { ascending: false });
  return (<div className="grid gap-4"><h1 className="text-2xl font-semibold">Benchmark suites</h1>{error && <p
role="alert">Could not load suites: {error.message}</p>}
    {!suites?.length ? <p className="rounded border border-dashed p-6">No suites yet. Run <code>supabase/seed.sql</code> to
create the canonical 12.</p> : <ul className="grid gap-2">{suites.map((s) => <li key={s.id}><Link href={'/benchmark/' + s.id}
className="block rounded border bg-white p-4 hover:bg-neutral-50"><strong>{s.name}</strong><p className="text-sm text-neutral-
600">{s.description}</p></Link></li>)}</ul>}</div>);
}
```

handoff/workflow-router-app/app/globals.css · 4 lines

```
@import "tailwindcss";
:focus-visible { outline: 2px solid #597ea3; outline-offset: 2px; }
@media (prefers-reduced-motion: reduce) { * { animation: none !important; transition: none !important; } }
```

handoff/workflow-router-app/app/layout.tsx · 10 lines

```
import './globals.css';
import Link from 'next/link';
import { AuthButton } from '@/components/auth/auth-button';
export const metadata = { title: 'Semester AI Workflow Router & Benchmark Lab' };
export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (<html lang="en"><body className="bg-[#f4f1ea] text-[#1b1a17] antialiased"><a href="#main" className="sr-only
focus:not-sr-only">Skip to content</a>
    <header className="flex items-center justify-between bg-[#090a0e] px-6 py-3 text-[#eceef2]"><nav className="flex gap-5
text-sm" aria-label="Main"><Link href="/" className="font-semibold tracking-[.22em]">SEMESTER</Link><Link href="/workflow-
router">Workflow router</Link><Link href="/benchmark">Benchmark lab</Link></nav><AuthButton /></header>
    <main id="main" className="mx-auto max-w-[1280px] p-6">{children}</main></body></html>);
}
```

handoff/workflow-router-app/app/page.tsx · 6 lines

```
import Link from 'next/link';
export default function Home() {
  return (<div className="grid gap-6"><h1 className="text-3xl font-semibold">AI Workflow Router &amp;amp; Benchmark Lab</h1><p
className="max-w-[62ch] text-neutral-700">Route a Semester workflow to Claude Artifacts, ChatGPT Canvas or v0, compare their
limits, and record graded benchmark runs with evidence.</p>
    <div className="flex gap-3"><Link className="min-h-11 rounded bg-slate-800 px-4 py-3 text-white" href="/workflow-
router">Open the router</Link><Link className="min-h-11 rounded border px-4 py-3" href="/benchmark">Benchmark lab</Link></div>
</div>);
}
```

handoff/workflow-router-app/app/workflow-router/actions.ts · 17 lines

```
'use server';
import { revalidatePath } from 'next/cache';
import { requireUser } from '@/lib/supabase/server';
import { SaveRecommendationSchema } from '@/lib/validation/workflow-router';
import { recommend } from '@/lib/workflow-router/engine';
export async function saveRecommendation(input: unknown) {
  const parsed = SaveRecommendationSchema.safeParse(input);
  if (!parsed.success) return { ok: false as const, error: parsed.error.flatten() };
  const { supabase, user } = await requireUser();
  if (!user) return { ok: false as const, error: 'Sign in to save recommendations.' };
  const r = recommend(parsed.data.answers); // recomputed on the server; the client's result is never trusted
  const { error, data } = await supabase.from('workflow_recommendations').insert({ owner_id: user.id, organization_id:
parsed.data.organizationId, visibility: parsed.data.visibility, title: parsed.data.title, answers: parsed.data.answers,
primary_platform: r.primary, secondary_platform: r.secondary, confidence: r.confidence, result: r }).select('id').single();
  if (error) return { ok: false as const, error: error.message };
  revalidatePath('/workflow-router');
  return { ok: true as const, id: data.id };
}
```

handoff/workflow-router-app/app/workflow-router/page.tsx · 10 lines

```
import { requireUser } from '@/lib/supabase/server';
import { WorkflowRouter } from '@/components/workflow-router/workflow-router';
import { FeatureMatrix } from '@/components/workflow-router/feature-matrix';
import { PlatformMatrix } from '@/components/workflow-router/platform-matrix';
export default async function Page() {
  const { user } = await requireUser();
  return (<div className="grid gap-10"><header><p className="text-xs uppercase tracking-wider text-neutral-600">Workflow
router</p><h1 className="text-2xl font-semibold">Where should this be built?</h1></header>
    <WorkflowRouter signedIn={!!user} /><section><h2 className="mb-3 text-lg font-semibold">Feature matrix</h2><FeatureMatrix
/></section><section><h2 className="mb-3 text-lg font-semibold">Twelve canonical workflows</h2><PlatformMatrix /></section>
</div>);
}
```

handoff/workflow-router-app/components/auth/auth-button.tsx · 8 lines

```
import Link from 'next/link';
import { requireUser } from '@/lib/supabase/server';
export async function AuthButton() {
  const { user } = await requireUser();
  return user ? (<form action="/auth/sign-out" method="post" className="flex items-center gap-3 text-sm"><span>{user.email}
</span><button className="min-h-9 rounded border px-3">Sign out</button></form>)
    : (<div className="flex items-center gap-3 text-sm"><span className="rounded border border-dashed px-2 py-1">Prototype mode
— nothing is saved</span><Link href="/auth/sign-in" className="min-h-9 rounded bg-slate-800 px-3 py-2 text-white">Sign
in</Link></div>);
}
```

handoff/workflow-router-app/components/benchmark/benchmark-dashboard.tsx · 2 lines

```
export { Leaderboard as BenchmarkDashboard } from './leaderboard';
```

handoff/workflow-router-app/components/benchmark/benchmark-run-detail.tsx · 11 lines

```
import { PLATFORM_LABEL, type Platform, type Weights } from '@/lib/benchmark/benchmark.config';
import { persistenceCeiling } from '@/lib/benchmark/probes/persistence';
```

```
import { GradingForm } from './grading-form';
export function BenchmarkRunDetail({ run, weights, requiresPersistence }: { run: any; weights?: Weights; requiresPersistence:
boolean }) {
  const ceiling = persistenceCeiling({ tested: run.persistence_tested, survivedReload: run.persistence_survived_reload, mode:
run.persistence_mode, evidence: run.persistence_evidence ?? undefined }, requiresPersistence);
  return (<article className="grid gap-4 rounded border bg-white p-5"><h3 className="text-lg font-semibold">
{PLATFORM_LABEL[run.platform as Platform]} · {run.status}</h3>
    <dl className="grid grid-cols-[160px_1fr] gap-2 text-sm"><dt>Model</dt><dd>{run.model_name ?? '—'}</dd><dt>Output</dt><dd>
{run.output_url ? <a href={run.output_url} className="underline">{run.output_url}</a> : '—'}</dd><dt>Duration</dt><dd
className="tabular-nums">{run.duration_ms ?? '—'} ms</dd><dt>Persistence</dt><dd>{run.persistence_tested ?
(run.persistence_survived_reload ? 'Survived reload' : 'Lost on reload') + ' · ' + run.persistence_mode : 'Not tested'}</dd>
<dt>Status reason</dt><dd>{run.status_reason ?? '—'}</dd></dl>
    <details><summary className="cursor-pointer text-sm">Exact prompt</summary><pre className="whitespace-pre-wrap font-mono
text-xs">{run.executed_prompt}</pre></details>
    <GradingForm runId={run.id} weights={weights} persistenceCeiling={ceiling} /></article>);
}
```

handoff/workflow-router-app/components/benchmark/grading-form.tsx · 19 lines

```
'use client';
import { useState, useTransition } from 'react';
import { CRITERIA, type Weights } from '@/lib/benchmark/benchmark.config';
import { weightedScore, type Grades } from '@/lib/benchmark/scoring';
import { gradeRun } from '@/app/benchmark/actions';
const LABEL: Record<string, string> = { correctness: 'Correctness', codeQuality: 'Code quality', rendering: 'Rendering',
stateManagement: 'State management', maintainability: 'Maintainability', handoff: 'Handoff/export', sandboxFit: 'Sandbox fit',
persistence: 'Persistence' };
export function GradingForm({ runId, weights, persistenceCeiling }: { runId: string; weights?: Weights; persistenceCeiling:
number }) {
  const [g, setG] = useState<Grades>({}); const [evidence, setEvidence] = useState(''); const [notes, setNotes] = useState('');
const [said, setSaid] = useState(''); const [pending, start] = useTransition();
  const s = weightedScore(g, weights);
  return (<form className="grid gap-3" onSubmit={(e) => { e.preventDefault(); start(async () => { const r = await gradeRun({
runId, ...Object.fromEntries(CRITERIA.map((c) => [c, g[c] ?? null])), evidence, notes }); setSaid(r.ok ? 'Grade saved.' : 'Not
saved — check the fields.'); }); }}>
    {CRITERIA.map((c) => (<fieldset key={c} className="flex items-center gap-3"><legend className="w-40 text-sm">{LABEL[c]}
</legend>
      {[0, 1, 2, 3, 4, 5].map((n) => { const capped = c === 'persistence' && n > persistenceCeiling; return (<label key={n}
className={'inline-flex min-h-9 min-w-9 items-center justify-center rounded border text-sm ' + (g[c] === n ? 'bg-slate-800
text-white' : '') + (capped ? ' opacity-40' : '')}><input type="radio" className="sr-only" name={c} disabled={capped} checked=
{g[c] === n} onChange={() => setG({ ...g, [c]: n })} />{n}</label>); })}</fieldset>))}
    <p className="text-xs text-neutral-600">Persistence is capped at {persistenceCeiling} by the recorded reload test.</p>
    <label className="grid gap-1 text-sm">Evidence<textarea required value={evidence} onChange={(e) =>
setEvidence(e.target.value)} className="min-h-20 rounded border p-2" /></label>
    <label className="grid gap-1 text-sm">Notes<textarea value={notes} onChange={(e) => setNotes(e.target.value)}
className="min-h-16 rounded border p-2" /></label>
    <p className="tabular-nums text-sm">Weighted: {s.outOf5 ?? '—'} / 5 · {s.outOf100 ?? '—'} / 100 {s.missing.length ? '·
missing ' + s.missing.join(', ') : ''}</p>
    <button disabled={pending} className="min-h-10 rounded bg-slate-800 px-4 text-sm text-white">Save grade</button><p
role="status" className="text-sm">{said}</p></form>);
}
```

handoff/workflow-router-app/components/benchmark/leaderboard.tsx · 6 lines

```
import type { LeaderRow } from '@/lib/benchmark/scoring';
export function Leaderboard({ title, rows }: { title: string; rows: LeaderRow[] }) {
  return (<section><h2 className="mb-2 text-lg font-semibold">{title}</h2><table className="w-full text-sm"><thead><tr
className="text-left text-xs uppercase tracking-wider text-neutral-600"><th>#</th><th>Platform</th><th>Score /5</th>
<th>Graded</th></tr></thead>
    <tbody>{rows.map((r, i) => <tr key={r.key} className="border-t"><td className="py-2 tabular-nums">{i + 1}</td><td>{r.key}
</td><td className="tabular-nums">{r.score ?? 'Not graded'}</td><td className="tabular-nums">{r.graded} / {r.tasks}</td></tr>)}
</tbody></table></section>);
}
```

handoff/workflow-router-app/components/benchmark/run-form.tsx · 29 lines

```
'use client';
import { useState, useTransition } from 'react';
import { PERSISTENCE_MODES, PLATFORMS, PLATFORM_LABEL, type WorkflowSpec } from '@/lib/benchmark/benchmark.config';
import { recordRun } from '@/app/benchmark/actions';
import { capabilityGuard } from '@/lib/benchmark/adapters/capability-guard';
```

```
export function RunForm({ taskId, task }: { taskId: string; task: WorkflowSpec }) {
  const [platform, setPlatform] = useState<(typeof PLATFORMS)[number]>(task.recommended); const guard =
capabilityGuard(platform, task);
  const [said, setSaid] = useState(''); const [pending, start] = useTransition();
  return (<form className="grid gap-3" onSubmit={(e) => { e.preventDefault(); const f = new FormData(e.currentTarget);
start(async () => {
      const r = await recordRun({ taskId, platform, executedPrompt: f.get('prompt'), modelName: f.get('model') || undefined,
status: f.get('status'), statusReason: f.get('reason') || undefined, outputUrl: f.get('url') || '', runtimeNotes:
f.get('runtime') || undefined, consoleNotes: f.get('console') || undefined, durationMs: f.get('ms') ? Number(f.get('ms')) :
undefined, sourceManifest: String(f.get('manifest') || '').split('\n').filter(Boolean).map((path) => ({ path })),
persistenceTested: f.get('ptested') === 'on', persistenceSurvivedReload: f.get('psurvived') === 'yes' ? true :
f.get('psurvived') === 'no' ? false : null, persistenceMode: f.get('pmode'), persistenceEvidence: f.get('pevidence') ||
undefined });
      setSaid(r.ok ? 'Run recorded.' : 'Not recorded: ' + JSON.stringify(r.error)); }); }}>
    <label className="grid gap-1 text-sm">Platform<select value={platform} onChange={(e) => setPlatform(e.target.value as
typeof platform)} className="min-h-10 rounded border px-2">{PLATFORMS.map((p) => <option key={p} value={p}>{PLATFORM_LABEL[p]}
</option>)}</select></label>
    {!guard.ok && <p role="status" className="rounded border border-amber-600/50 bg-amber-50 p-2 text-sm">Unsupported:
{guard.reason}</p>}
    <label className="grid gap-1 text-sm">Exact executed prompt<textarea name="prompt" required defaultValue={task.prompt}
className="min-h-24 rounded border p-2 font-mono text-xs" /></label>
    <div className="grid grid-cols-2 gap-3"><label className="grid gap-1 text-sm">Model name/version<input name="model"
className="min-h-10 rounded border px-2" /></label>
      <label className="grid gap-1 text-sm">Status<select name="status" defaultValue={guard.ok ? 'manual-review-required' :
'unsupported'} className="min-h-10 rounded border px-2">{['completed', 'manual-review-required', 'unsupported',
'failed'].map((s) => <option key={s}>{s}</option>)}</select></label></div>
    <label className="grid gap-1 text-sm">Status reason<input name="reason" defaultValue={guard.ok ? '' : guard.reason}
className="min-h-10 rounded border px-2" /></label>
    <label className="grid gap-1 text-sm">Output URL<input name="url" type="url" className="min-h-10 rounded border px-2" />
</label>
    <label className="grid gap-1 text-sm">Source/file manifest (one path per line)<textarea name="manifest" className="min-h-16
rounded border p-2 font-mono text-xs" /></label>
    <div className="grid grid-cols-3 gap-3"><label className="grid gap-1 text-sm">Duration (ms)<input name="ms" type="number"
min={0} className="min-h-10 rounded border px-2" /></label><label className="col-span-2 grid gap-1 text-sm">Runtime/sandbox
notes<input name="runtime" className="min-h-10 rounded border px-2" /></label></div>
    <label className="grid gap-1 text-sm">Console/log notes<textarea name="console" className="min-h-16 rounded border p-2
font-mono text-xs" /></label>
    <fieldset className="grid gap-2 rounded border p-3"><legend className="px-1 text-sm font-medium">Persistence
test{task.requiresPersistence ? ' (required)' : ''}</legend>
      <label className="text-sm"><input type="checkbox" name="ptested" defaultChecked={task.requiresPersistence} /> Tested
reload/refresh</label>
      <label className="grid gap-1 text-sm">Survived reload?<select name="psurvived" className="min-h-10 rounded border px-2">
<option value="">Not recorded</option><option value="yes">Yes</option><option value="no">No</option></select></label>
      <label className="grid gap-1 text-sm">Storage mode<select name="pmode" className="min-h-10 rounded border px-2">
{PERSISTENCE_MODES.map((m) => <option key={m}>{m}</option>)}</select></label>
      <label className="grid gap-1 text-sm">Evidence (URL, row id, screenshot or log)<input name="pevidence" className="min-h-
10 rounded border px-2" /></label></fieldset>
    <button disabled={pending} className="min-h-10 rounded bg-slate-800 px-4 text-sm text-white">Record run</button><p
role="status" className="text-sm">{said}</p></form>);
}
```

handoff/workflow-router-app/components/benchmark/workflow-matrix.tsx · 2 lines

```
export { PlatformMatrix as WorkflowMatrix } from '../workflow-router/platform-matrix';
```

handoff/workflow-router-app/components/workflow-router/decision-tree.tsx · 18 lines

```
'use client';
import type { RouterAnswers } from '@/lib/validation/workflow-router';
export const QUESTIONS: { key: keyof RouterAnswers; label: string; options: [string, string][] }[] = [
  { key: 'deliverable', label: 'Deliverable', options: [['interactive-tool', 'Interactive tool'], ['prototype', 'Product
prototype'], ['marketing-page', 'Marketing page'], ['diagram', 'Diagram'], ['document', 'Document'], ['presentation',
'Presentation'], ['data-analysis', 'Data analysis'], ['react-component', 'React component'], ['full-stack', 'Full-stack
feature']] },
  { key: 'interaction', label: 'Interaction', options: [['none', 'None'], ['simple-local', 'Simple local interaction'],
['complex-local', 'Complex local state'], ['multi-user', 'Multi-user collaboration']] },
  { key: 'backend', label: 'Backend', options: [['none', 'None'], ['file-export', 'File export'], ['database', 'Database'],
['server-api', 'Server API'], ['auth-secrets', 'Auth/secrets']] },
  { key: 'technology', label: 'Technology target', options: [['none', 'None'], ['html', 'HTML/CSS/JavaScript'], ['react',
'React'], ['nextjs', 'Next.js'], ['nextjs-postgres', 'Next.js + Postgres'], ['nextjs-supabase', 'Next.js + Supabase/Auth']] },
  { key: 'persistence', label: 'Persistence/access', options: [['none', 'No persistence'], ['database', 'Database'], ['auth',
'Auth'], ['storage', 'Storage'], ['realtime', 'Realtime'], ['multi-tenant', 'Multi-tenant organization workspace']] },
  { key: 'handoff', label: 'Handoff', options: [['share', 'Share immediately'], ['document-export', 'Document/source export'],
```

```
['github', 'GitHub'], ['vercel', 'Deploy to Vercel']] },
];
export function DecisionTree({ value, onChange }: { value: RouterAnswers; onChange: (v: RouterAnswers) => void }) {
  return (<div className="grid gap-5">{QUESTIONS.map((q) => (
    <fieldset key={q.key}><legend className="mb-2 text-xs font-medium uppercase tracking-wider text-neutral-600">{q.label}
</legend>
      <div className="flex flex-wrap gap-2">{q.options.map(([v, l]) => (
        <label key={v} className={'cursor-pointer rounded border px-3 py-2 text-sm min-h-10 inline-flex items-center focus-
within:ring-2 focus-within:ring-slate-500 ' + (value[q.key] === v ? 'border-slate-700 bg-slate-800 text-white' : 'border-
neutral-300 bg-white hover:bg-neutral-50')}>
          <input type="radio" className="sr-only" name={q.key} value={v} checked={value[q.key] === v} onChange={() =>
onChange({ ...value, [q.key]: v } as RouterAnswers)} />{l}</label>))}</div></fieldset>))}</div>);
}
```

handoff/workflow-router-app/components/workflow-router/feature-matrix.tsx · 19 lines

```
'use client';
import { useMemo, useState } from 'react';
export const FEATURES = [
  { platform: 'Claude Artifacts', primaryUse: 'Self-contained interactive artifacts, diagrams, prototypes', execution: 'Client-
side sandbox beside the chat', sandbox: 'Approved CDNs only; localStorage/fetch may be restricted', react: 4, clientState: 4,
persistent: 1, multiFile: 1, server: 0, database: 0, auth: 0, docExport: 3, codeExport: 4, github: 1, deploy: 1, semester:
'Decision tools, artifact catalog, architecture diagrams' },
  { platform: 'ChatGPT Canvas', primaryUse: 'Revision-centred writing and code iteration', execution: 'Isolated React/HTML
preview; Python execution', sandbox: 'Isolated preview; not a hosting environment', react: 3, clientState: 3, persistent: 1,
multiFile: 2, server: 1, database: 0, auth: 0, docExport: 5, codeExport: 3, github: 1, deploy: 0, semester: 'PRDs, TDDs, legal
first drafts, CSV analysis' },
  { platform: 'v0', primaryUse: 'Multi-file Next.js apps with server logic', execution: 'VM-backed Node.js sandbox', sandbox:
'Isolated VM; runs server code, API routes, DB connections', react: 5, clientState: 5, persistent: 4, multiFile: 5, server: 5,
database: 4, auth: 4, docExport: 1, codeExport: 5, github: 5, deploy: 5, semester: 'Authenticated, database-backed Semester
features' },
] as const;
type Row = (typeof FEATURES)[number]; type Key = keyof Row;
const COLS: [Key, string][] = [['platform', 'Platform'], ['primaryUse', 'Primary use'], ['execution', 'Execution model'],
['sandbox', 'Sandbox limits'], ['react', 'React'], ['clientState', 'Client state'], ['persistent', 'Persistent state'],
['multiFile', 'Multi-file'], ['server', 'Server/API'], ['database', 'Database'], ['auth', 'Auth'], ['docExport', 'Doc export'],
['codeExport', 'Code export'], ['github', 'GitHub'], ['deploy', 'Deploy'], ['semester', 'Best Semester use']];
const word = (n: number) => ['None', 'Minimal', 'Partial', 'Good', 'Strong', 'Native'][n];
export function FeatureMatrix() {
  const [sort, setSort] = useState<{ key: Key; dir: 1 | -1 }>({ key: 'platform', dir: 1 }); const [q, setQ] = useState('');
  const rows = useMemo(() => [...FEATURES].filter((r) => JSON.stringify(r).toLowerCase().includes(q.toLowerCase())).sort((a, b)
=> (a[sort.key] > b[sort.key] ? 1 : a[sort.key] < b[sort.key] ? -1 : 0) * sort.dir), [sort, q]);
  return (<div><label className="mb-2 block text-sm">Search <input value={q} onChange={(e) => setQ(e.target.value)}
className="ml-2 min-h-9 rounded border px-2" /></label>
    <div className="overflow-x-auto rounded-md border" role="region" aria-label="Feature matrix" tabIndex={0}><table
className="min-w-[1400px] text-sm"><thead className="bg-neutral-100"><tr>{COLS.map(([k, l]) => (
      <th key={k} scope="col" aria-sort={sort.key === k ? (sort.dir === 1 ? 'ascending' : 'descending') : 'none'}
className="px-3 py-2 text-left text-xs uppercase tracking-wider"><button type="button" onClick={() => setSort((s) => ({ key: k,
dir: s.key === k ? (s.dir === 1 ? -1 : 1) : 1 }))}>{l}{sort.key === k ? (sort.dir === 1 ? ' ↑' : ' ↓') : ''}</button></th>))}
</tr></thead>
      <tbody>{rows.map((r) => <tr key={r.platform} className="border-t">{COLS.map(([k]) => <td key={k} className="px-3 py-2
align-top">{typeof r[k] === 'number' ? <span className="tabular-nums">{word(r[k] as number)} ({r[k]}/5)</span> : String(r[k])}
</td>)}</tr>)}</tbody></table></div></div>);
}
```

handoff/workflow-router-app/components/workflow-router/platform-matrix.tsx · 7 lines

```
'use client';
import { WORKFLOWS, PLATFORM_LABEL } from '@/lib/benchmark/benchmark.config';
export function PlatformMatrix() {
  return (<div className="overflow-x-auto rounded-md border"><table className="w-full text-sm"><thead className="bg-neutral-
100"><tr><th className="px-3 py-2 text-left">#</th><th className="px-3 py-2 text-left">Workflow</th><th className="px-3 py-2
text-left">Category</th><th className="px-3 py-2 text-left">Recommended</th><th className="px-3 py-2 text-
left">Persistence</th></tr></thead>
    <tbody>{WORKFLOWS.map((w) => <tr key={w.number} className="border-t"><td className="px-3 py-2 tabular-nums">{w.number}</td>
<td className="px-3 py-2">{w.title}</td><td className="px-3 py-2">{w.category}</td><td className="px-3 py-2">
{PLATFORM_LABEL[w.recommended]}</td><td className="px-3 py-2">{w.requiresPersistence ? 'Required' : '—'}</td></tr>)}</tbody>
</table></div>);
}
```

handoff/workflow-router-app/components/workflow-router/prompt-builder.tsx · 9 lines

```
'use client';
import { useState } from 'react';
export function CopyBlock({ label, text }: { label: string; text: string }) {
  const [said, setSaid] = useState('');
  return (<div className="rounded-md border border-neutral-300 bg-white"><div className="flex items-center justify-between
border-b px-4 py-2"><h3 className="text-sm font-semibold">{label}</h3>
    <button type="button" className="min-h-9 rounded border px-3 text-sm" onClick={async () => { try { await
navigator.clipboard.writeText(text); setSaid('Copied'); } catch { setSaid('Copy failed — select the text'); } setTimeout(() =>
setSaid(''), 1500); }}>Copy</button></div>
    <pre className="max-h-72 overflow-auto whitespace-pre-wrap p-4 font-mono text-xs">{text}</pre><span role="status"
className="sr-only">{said}</span>{said && <p className="px-4 pb-2 text-xs">{said}</p>}</div>);
}
```

handoff/workflow-router-app/components/workflow-router/recommendation-panel.tsx · 15 lines

```
'use client';
import { PLATFORM_LABEL } from '@/lib/benchmark/benchmark.config';
import type { Recommendation } from '@/lib/workflow-router/engine';
export function RecommendationPanel({ r }: { r: Recommendation }) {
  return (<section aria-labelledby="rec-h" className="rounded-md border border-neutral-300 bg-white p-5">
    <p className="text-xs uppercase tracking-wider text-neutral-600">Recommendation · confidence {Math.round(r.confidence *
100)}%</p>
    <h2 id="rec-h" className="mt-1 text-2xl font-semibold">{PLATFORM_LABEL[r.primary]} <span className="text-base font-normal
text-neutral-600">then {PLATFORM_LABEL[r.secondary]}</span></h2>
    {r.escalation && <p role="status" className="mt-3 rounded border border-amber-600/50 bg-amber-50 px-3 py-2 text-sm text-
amber-900"><strong>! Production escalation:</strong> {r.escalationReasons.join('; ')}</p>}
    <h3 className="mt-4 text-sm font-semibold">Why</h3><ul className="mt-1 list-disc pl-5 text-sm">
{r.reasons[r.primary].map((x) => <li key={x}>{x}</li>)}</ul>
    <h3 className="mt-4 text-sm font-semibold">Limitations</h3><ul className="mt-1 list-disc pl-5 text-sm text-neutral-700">
{r.limitations.map((x) => <li key={x}>{x}</li>)}</ul>
    <p className="mt-4 text-sm"><strong>Handoff:</strong> {r.handoffFormat}</p>
    <div className="mt-3 flex gap-3 text-xs text-neutral-600 tabular-nums">{Object.entries(r.scores).map(([p, s]) => <span key=
{p}>{PLATFORM_LABEL[p as keyof typeof PLATFORM_LABEL]} {s}</span>)}</div>
  </section>);
}
```

handoff/workflow-router-app/components/workflow-router/workflow-router.tsx · 27 lines

```
'use client';
import { useMemo, useState, useTransition } from 'react';
import type { RouterAnswers } from '@/lib/validation/workflow-router';
import { recommend } from '@/lib/workflow-router/engine';
import { buildPrompt, githubPlan, supabasePrompt } from '@/lib/workflow-router/prompts';
import { recommendationMarkdown } from '@/lib/exports/markdown';
import { toJson } from '@/lib/exports/json';
import { DecisionTree } from './decision-tree';
import { RecommendationPanel } from './recommendation-panel';
import { CopyBlock } from './prompt-builder';
import { saveRecommendation } from '@/app/workflow-router/actions';
const DEFAULT: RouterAnswers = { deliverable: 'interactive-tool', interaction: 'complex-local', backend: 'file-export',
technology: 'react', persistence: 'none', handoff: 'share' };
const download = (name: string, type: string, body: string) => { const a = document.createElement('a'); a.href =
URL.createObjectURL(new Blob([body], { type })); a.download = name; a.click(); URL.revokeObjectURL(a.href); };
export function WorkflowRouter({ signedIn }: { signedIn: boolean }) {
  const [a, setA] = useState<RouterAnswers>(DEFAULT); const r = useMemo(() => recommend(a), [a]); const prompt = buildPrompt(a,
r);
  const [said, setSaid] = useState(''); const [pending, start] = useTransition();
  return (<div className="grid gap-6 lg:grid-cols-[minmax(0,1fr)_minmax(0,1.1fr)]">
    <DecisionTree value={a} onChange={setA} />
    <div className="grid content-start gap-4"><RecommendationPanel r={r} />
      <div className="flex flex-wrap gap-2"><button className="min-h-10 rounded border px-3 text-sm" onClick={() =>
download('semester-recommendation.md', 'text/markdown', recommendationMarkdown(a, r, prompt))}>Download Markdown</button>
        <button className="min-h-10 rounded border px-3 text-sm" onClick={() => download('semester-recommendation.json',
'application/json', toJson({ answers: a, recommendation: r, prompt }))}>Download JSON</button>
        <button disabled={!signedIn || pending} title={signedIn ? '' : 'Sign in to save'} className="min-h-10 rounded bg-slate-
800 px-3 text-sm text-white disabled:opacity-50" onClick={() => start(async () => { const res = await saveRecommendation({
title: a.deliverable + ' → ' + r.primary, organizationId: null, answers: a, visibility: 'private' }); setSaid(res.ok ? 'Saved
to your recommendations.' : typeof res.error === 'string' ? res.error : 'Check the inputs.'); })}>{signedIn ? 'Save
recommendation' : 'Sign in to save'}</button></div>
      <p role="status" className="text-sm">{said}</p>
      <CopyBlock label="Build prompt" text={prompt} /><CopyBlock label="GitHub handoff plan" text={githubPlan(r).map((x, i) =>
i + 1 + '. ' + x).join('\n')} />
```

```
      {(a.technology === 'nextjs-supabase' || a.persistence !== 'none') && <CopyBlock label="Supabase implementation prompt"
text={supabasePrompt()} />}</div></div>);
}
```

handoff/workflow-router-app/install.sh · 6 lines

```
#!/bin/sh
set -e
find . -name "*.txt" ! -name "LICENSE.txt" | while read -r f; do mv "$f" "${f%.txt}"; done
[ -d app/benchmark/-suiteId- ] && mv app/benchmark/-suiteId- "app/benchmark/[suiteId]"
echo "Restored. Next: npm install && npm run typecheck && npm test && npm run build"
```

handoff/workflow-router-app/lib/benchmark/adapters/capability-guard.ts · 7 lines

```
import type { Platform, WorkflowSpec } from '../benchmark.config';
/** Honest boundaries: what each platform can be asked to do at all. Execution itself is always manual — no approved automation
API is assumed. */
export function capabilityGuard(platform: Platform, task: WorkflowSpec): { ok: true } | { ok: false; status: 'unsupported';
reason: string } {
  if (platform !== 'v0' && task.requiresPersistence) return { ok: false, status: 'unsupported', reason: (platform === 'claude-
artifacts' ? 'Claude Artifacts' : 'ChatGPT Canvas') + ' has no durable backend, auth or secret store; this task needs ' +
task.requirements.join(', ') + '. Record a run only to document the gap.' };
  return { ok: true };
}
```

handoff/workflow-router-app/lib/benchmark/adapters/chatgpt-canvas.adapter.ts · 12 lines

```
import type { BenchmarkAdapter, AdapterRun } from './types';
import { capabilityGuard } from './capability-guard';
const MANUAL = 'No approved programmatic integration for ChatGPT Canvas is assumed. Run the prompt in Canvas, export the
document/code, and attach the file manifest and a share link.';
export const chatgptCanvasAdapter: BenchmarkAdapter = {
  platform: 'chatgpt-canvas',
  canRun(task) { const g = capabilityGuard('chatgpt-canvas', task); return g.ok ? { ok: false, status: 'manual-review-
required', reason: MANUAL } : g; },
  async run(task) { const g = this.canRun(task); return { platform: 'chatgpt-canvas', status: g.ok ? 'completed' : g.status,
reason: g.ok ? undefined : g.reason, executedPrompt: task.prompt }; },
  collectEvidence(run: AdapterRun) { return { outputUrl: run.outputUrl, manifest: run.manifest ?? [], notes: [run.consoleNotes
?? '', run.reason ?? ''].filter(Boolean) }; },
  evaluatePersistence() { return { tested: false, survivedReload: null, mode: 'local-preview' }; },
  reportUnsupported(reason) { return { platform: 'chatgpt-canvas', status: 'unsupported', reason, executedPrompt: '' }; },
};
```

handoff/workflow-router-app/lib/benchmark/adapters/claude-artifacts.adapter.ts · 12 lines

```
import type { BenchmarkAdapter, AdapterRun } from './types';
import { capabilityGuard } from './capability-guard';
const MANUAL = 'No approved programmatic integration for creating Claude Artifacts is assumed. Paste the exact prompt into
claude.ai, publish the artifact, and record its URL, source and console notes.';
export const claudeArtifactsAdapter: BenchmarkAdapter = {
  platform: 'claude-artifacts',
  canRun(task) { const g = capabilityGuard('claude-artifacts', task); return g.ok ? { ok: false, status: 'manual-review-
required', reason: MANUAL } : g; },
  async run(task) { const g = this.canRun(task); return { platform: 'claude-artifacts', status: g.ok ? 'completed' : g.status,
reason: g.ok ? undefined : g.reason, executedPrompt: task.prompt }; },
  collectEvidence(run: AdapterRun) { return { outputUrl: run.outputUrl, manifest: run.manifest ?? [], notes: [run.consoleNotes
?? '', run.reason ?? ''].filter(Boolean) }; },
  evaluatePersistence() { return { tested: false, survivedReload: null, mode: 'local-preview' }; },
  reportUnsupported(reason) { return { platform: 'claude-artifacts', status: 'unsupported', reason, executedPrompt: '' }; },
};
```

handoff/workflow-router-app/lib/benchmark/adapters/types.ts · 14 lines

```
import type { Platform, WorkflowSpec } from '../benchmark.config';
import type { PersistenceObservation } from '../probes/persistence';
export type RunStatus = 'completed' | 'manual-review-required' | 'unsupported' | 'failed';
export interface AdapterRun { platform: Platform; status: RunStatus; reason?: string; executedPrompt: string; outputUrl?:
string; manifest?: { path: string; bytes?: number }[]; consoleNotes?: string; durationMs?: number }
```

```
export interface Evidence { outputUrl?: string; manifest: { path: string }[]; notes: string[] }
export interface BenchmarkAdapter {
  platform: Platform;
  canRun(task: WorkflowSpec): { ok: true } | { ok: false; status: 'manual-review-required' | 'unsupported'; reason: string };
  run(task: WorkflowSpec): Promise<AdapterRun>;
  collectEvidence(run: AdapterRun): Evidence;
  evaluatePersistence(run: AdapterRun): PersistenceObservation;
  reportUnsupported(reason: string): AdapterRun;
}
```

handoff/workflow-router-app/lib/benchmark/adapters/v0.adapter.ts · 12 lines

```
import type { BenchmarkAdapter, AdapterRun } from './types';
import { capabilityGuard } from './capability-guard';
const MANUAL = 'Run the prompt in v0, push to GitHub, and record the repo/deployment URL, file manifest and test log.
Persistence is graded only from a reload test with a row id or log.';
export const v0Adapter: BenchmarkAdapter = {
  platform: 'v0',
  canRun(task) { const g = capabilityGuard('v0', task); return g.ok ? { ok: false, status: 'manual-review-required', reason:
MANUAL } : g; },
  async run(task) { const g = this.canRun(task); return { platform: 'v0', status: g.ok ? 'completed' : g.status, reason: g.ok ?
undefined : g.reason, executedPrompt: task.prompt }; },
  collectEvidence(run: AdapterRun) { return { outputUrl: run.outputUrl, manifest: run.manifest ?? [], notes: [run.consoleNotes
?? '', run.reason ?? ''].filter(Boolean) }; },
  evaluatePersistence() { return { tested: false, survivedReload: null, mode: 'project-backed' }; },
  reportUnsupported(reason) { return { platform: 'v0', status: 'unsupported', reason, executedPrompt: '' }; },
};
```

handoff/workflow-router-app/lib/benchmark/benchmark.config.ts · 56 lines

```
export const PLATFORMS = ['claude-artifacts', 'chatgpt-canvas', 'v0'] as const;
export type Platform = (typeof PLATFORMS)[number];
export const PLATFORM_LABEL: Record<Platform, string> = { 'claude-artifacts': 'Claude Artifacts', 'chatgpt-canvas': 'ChatGPT
Canvas', v0: 'v0' };

export const CRITERIA = ['correctness', 'codeQuality', 'rendering', 'stateManagement', 'maintainability', 'handoff',
'sandboxFit', 'persistence'] as const;
export type Criterion = (typeof CRITERIA)[number];
export type Weights = Partial<Record<Criterion, number>>;

/** Default weights (sum 1.0). Persistence is graded but carries 0 weight unless a task overrides it. */
export const DEFAULT_WEIGHTS: Weights = { correctness: 0.25, codeQuality: 0.2, rendering: 0.15, stateManagement: 0.15,
maintainability: 0.1, handoff: 0.1, sandboxFit: 0.05, persistence: 0 };

export const PERSISTENCE_MODES = ['none', 'local-preview', 'browser-local', 'session', 'database', 'project-backed'] as const;
export type PersistenceMode = (typeof PERSISTENCE_MODES)[number];

export interface WorkflowSpec { number: number; slug: string; title: string; category: string; prompt: string; requirements:
string[]; acceptance: string[]; recommended: Platform; requiresPersistence: boolean; weights?: Weights }

const P = (w: Weights) => w;
export const WORKFLOWS: WorkflowSpec[] = [
  { number: 1, slug: 'artifact-tracker', title: 'Sortable artifact tracker', category: 'Interactive tool', recommended:
'claude-artifacts', requiresPersistence: false,
    prompt: 'Build a single-file React tracker of every Semester artifact with search, category/format/priority filters,
sortable columns and CSV export. No external services.',
    requirements: ['Search', 'Multi-filter', 'Sortable columns', 'CSV export'], acceptance: ['Sort is stable per column',
'Filters combine with AND', 'CSV reflects current view'] },
  { number: 2, slug: 'onboarding-prototype', title: 'Interactive onboarding prototype', category: 'Product prototype',
recommended: 'claude-artifacts', requiresPersistence: false,
    prompt: 'Prototype Semester student onboarding: add a syllabus, connect a calendar, or use sample data; consent one
decision at a time; first next step with its source label.',
    requirements: ['3 entry paths', 'Consent steps', 'Back/forward', 'Keyboard reachable'], acceptance: ['Every step reachable
by keyboard', 'Decline is equal weight'] },
  { number: 3, slug: 'b2b-landing', title: 'B2B SaaS landing page', category: 'Marketing page', recommended: 'claude-
artifacts', requiresPersistence: false,
    prompt: 'Design a Semester institution landing page: one clear next step, status of every capability (use now / pilot /
later), no unapproved claims.',
    requirements: ['Hero', 'Capability status map', 'Trust section', 'Responsive'], acceptance: ['No prohibited claims',
'Passes 320px reflow'] },
  { number: 4, slug: 'architecture-diagram', title: 'Multi-tenant system architecture diagram', category: 'Diagram',
recommended: 'claude-artifacts', requiresPersistence: false,
    prompt: 'Draw Semester multi-tenant architecture as Mermaid: tenants, Supabase Postgres with RLS, edge functions, SIS/LMS
```

```
connectors, audit log.',
    requirements: ['Mermaid source', 'Tenant boundary', 'Data flows labelled'], acceptance: ['Renders without errors', 'RLS
boundary explicit'] },
  { number: 5, slug: 'tech-design-doc', title: 'Technical design document', category: 'Document', recommended: 'chatgpt-
canvas', requiresPersistence: false,
    prompt: 'Write a technical design document for Semester degree-audit sync: context, goals, non-goals, design, data model,
risks, rollout, open questions.',
    requirements: ['Standard TDD sections', 'Data model', 'Risks'], acceptance: ['Exports to DOCX/PDF/Markdown', 'Revision
history usable'], weights: P({ correctness: 0.3, codeQuality: 0.05, rendering: 0.05, stateManagement: 0, maintainability: 0.2,
handoff: 0.35, sandboxFit: 0.05 }) },
  { number: 6, slug: 'launch-presentation', title: 'Product launch presentation', category: 'Presentation', recommended:
'chatgpt-canvas', requiresPersistence: false,
    prompt: 'Draft a 10-slide Semester pilot launch presentation for a registrar audience: scope, status, what stays official,
evidence, next step.',
    requirements: ['10 slides', 'Speaker notes', 'Status honesty'], acceptance: ['Exports a usable deck or outline'], weights:
P({ correctness: 0.3, codeQuality: 0, rendering: 0.25, stateManagement: 0, maintainability: 0.1, handoff: 0.3, sandboxFit: 0.05
}) },
  { number: 7, slug: 'subscription-csv', title: 'Subscription CSV analysis', category: 'Data analysis', recommended: 'chatgpt-
canvas', requiresPersistence: false,
    prompt: 'Analyse the attached subscription CSV: MRR by month, churn, cohort retention; state assumptions; export cleaned
CSV and a chart.',
    requirements: ['Python execution', 'Cohort table', 'Chart', 'Cleaned export'], acceptance: ['Numbers reproducible from
code', 'Assumptions listed'] },
  { number: 8, slug: 'metric-dashboard', title: 'Reusable React metric dashboard', category: 'React component', recommended:
'claude-artifacts', requiresPersistence: false,
    prompt: 'Build a reusable <MetricDashboard> React component: typed props, metric tiles with source and age, provenance-
encoded bars, empty/loading/error states.',
    requirements: ['Typed props', 'States', 'Accessible chart table'], acceptance: ['Component is self-contained', 'All states
render'] },
  { number: 9, slug: 'auth-route-handler', title: 'Authenticated Next.js route handler', category: 'Full-stack feature',
recommended: 'v0', requiresPersistence: true,
    prompt: 'Implement app/api/recommendations/route.ts: Supabase server client, auth.getUser(), Zod validation, insert scoped
to the user, 401/422 responses.',
    requirements: ['@supabase/ssr', 'getUser on server', 'Zod', 'Typed responses'], acceptance: ['Unauthenticated → 401',
'Invalid body → 422', 'Row visible only to owner'] },
  { number: 10, slug: 'supabase-crud', title: 'Supabase artifact CRUD workflow', category: 'Full-stack feature', recommended:
'v0', requiresPersistence: true,
    prompt: 'Create artifact CRUD with Supabase: migration, RLS, server actions for create/update/delete, optimistic UI,
revalidation.',
    requirements: ['Migration', 'RLS', 'Server Actions', 'Revalidate'], acceptance: ['Survives reload', 'RLS denies other
users'] },
  { number: 11, slug: 'cookie-auth', title: 'Cookie-based Supabase Auth', category: 'Full-stack feature', recommended: 'v0',
requiresPersistence: true,
    prompt: 'Implement cookie-based Supabase Auth with @supabase/ssr: browser/server clients, middleware session refresh, magic
link, confirm route, sign-out.',
    requirements: ['Middleware refresh', 'Magic link', 'Confirm route', 'Sign-out'], acceptance: ['Session survives reload',
'No service key in client bundle'] },
  { number: 12, slug: 'multi-tenant-workspace', title: 'Multi-tenant SaaS workspace', category: 'Full-stack feature',
recommended: 'v0', requiresPersistence: true,
    prompt: 'Build organizations, memberships (owner/admin/member/viewer), org switcher, invite flow and org-scoped RLS for
shared records.',
    requirements: ['Org tables', 'Role enum', 'RLS by membership', 'Org switcher'], acceptance: ['Viewer cannot write', 'Non-
member sees nothing'], weights: P({ correctness: 0.25, codeQuality: 0.15, rendering: 0.05, stateManagement: 0.1,
maintainability: 0.1, handoff: 0.1, sandboxFit: 0.05, persistence: 0.2 }) },
];
```

handoff/workflow-router-app/lib/benchmark/probes/persistence.ts · 20 lines

```
import type { PersistenceMode } from '../benchmark.config';
export interface PersistenceObservation { tested: boolean; survivedReload: boolean | null; mode: PersistenceMode; evidence?:
string }
/**
 * The highest persistence grade a run may receive. Local UI state never earns more than 2:
 * "do not award high persistence scores merely because a UI has local state."
 */
export function persistenceCeiling(o: PersistenceObservation, required: boolean): number {
  if (!o.tested) return required ? 0 : 5;
  if (!o.survivedReload) return 0;
  if (!o.evidence) return 1;
  switch (o.mode) {
    case 'none': case 'local-preview': return 1;
    case 'browser-local': case 'session': return 2;
    case 'database': case 'project-backed': return 5;
  }
}
```

```
export function clampPersistenceGrade(grade: number | null, o: PersistenceObservation, required: boolean): number | null {
  return grade === null ? null : Math.min(grade, persistenceCeiling(o, required));
}
```

handoff/workflow-router-app/lib/benchmark/probes/source-structure.ts · 14 lines

```
export interface ManifestEntry { path: string; bytes?: number }
export interface SourceStructure { files: number; multiFile: boolean; hasServerCode: boolean; hasMigrations: boolean; hasTests:
boolean; hasEnvExample: boolean; leaksServiceRole: boolean }
export function probeSourceStructure(manifest: ManifestEntry[], contents: Record<string, string> = {}): SourceStructure {
  const p = manifest.map((m) => m.path);
  return {
    files: p.length, multiFile: p.length > 1,
    hasServerCode: p.some((x) => /(^|\/)(route\.ts|actions\.ts|middleware\.ts|proxy\.ts)$/.test(x) || /\/api\//.test(x)),
    hasMigrations: p.some((x) => /migrations\/.+\.sql$/.test(x)),
    hasTests: p.some((x) => /\.(test|spec)\.(t|j)sx?$/.test(x)),
    hasEnvExample: p.includes('.env.example'),
    leaksServiceRole: Object.entries(contents).some(([path, t]) => /SERVICE_ROLE/.test(t) && (/^(app|components)\//.test(path)
&& /['"]use client['"]/.test(t) || /NEXT_PUBLIC_[A-Z_]*SERVICE/.test(t))),
  };
}
```

handoff/workflow-router-app/lib/benchmark/scoring.ts · 51 lines

```
import { CRITERIA, DEFAULT_WEIGHTS, type Criterion, type Weights } from './benchmark.config';

export type Grades = Partial<Record<Criterion, number | null>>;
export interface WeightedScore { outOf5: number | null; outOf100: number | null; missing: Criterion[]; weightUsed: number }

export function resolveWeights(override?: Weights): Record<Criterion, number> {
  const w = { ...DEFAULT_WEIGHTS, ...(override ?? {}) } as Record<Criterion, number>;
  for (const c of CRITERIA) w[c] = Math.max(0, w[c] ?? 0);
  return w;
}

/** Weighted 0–5 score. Missing grades are excluded and the remaining weights are renormalised; all-missing → null. */
export function weightedScore(grades: Grades, override?: Weights): WeightedScore {
  const w = resolveWeights(override);
  let sum = 0, used = 0; const missing: Criterion[] = [];
  for (const c of CRITERIA) {
    if (w[c] === 0) continue;
    const g = grades[c];
    if (g === null || g === undefined || Number.isNaN(g)) { missing.push(c); continue; }
    if (g < 0 || g > 5) throw new RangeError(c + ' must be 0–5');
    sum += g * w[c]; used += w[c];
  }
  if (used === 0) return { outOf5: null, outOf100: null, missing, weightUsed: 0 };
  const s = sum / used;
  return { outOf5: round(s, 2), outOf100: round((s / 5) * 100, 1), missing, weightUsed: round(used, 3) };
}

/** Average several graders' sheets criterion by criterion, then weight. */
export function aggregateGrades(sheets: Grades[], override?: Weights): WeightedScore & { graders: number } {
  const merged: Grades = {};
  for (const c of CRITERIA) {
    const vals = sheets.map((s) => s[c]).filter((v): v is number => typeof v === 'number');
    merged[c] = vals.length ? vals.reduce((a, b) => a + b, 0) / vals.length : null;
  }
  return { ...weightedScore(merged, override), graders: sheets.length };
}

export interface LeaderRow { key: string; score: number | null; tasks: number; graded: number }
export function leaderboard(entries: { key: string; score: number | null }[]): LeaderRow[] {
  const by = new Map<string, LeaderRow & { total: number }>();
  for (const e of entries) {
    const r = by.get(e.key) ?? { key: e.key, score: null, tasks: 0, graded: 0, total: 0 };
    r.tasks += 1; if (e.score !== null) { r.graded += 1; r.total += e.score; }
    by.set(e.key, r);
  }
  return [...by.values()].map(({ total, ...r }) => ({ ...r, score: r.graded ? round(total / r.graded, 2) : null }))
    .sort((a, b) => (b.score ?? -1) - (a.score ?? -1) || a.key.localeCompare(b.key));
}
```

```
const round = (n: number, d: number) => Math.round(n * 10 ** d) / 10 ** d;
```

handoff/workflow-router-app/lib/exports/csv.ts · 7 lines

```
const cell = (v: unknown) => { const s = v === null || v === undefined ? '' : String(v); return /[",\n\r]/.test(s) || /^[=+\-
@]/.test(s) ? '"' + (/^[=+\-@]/.test(s) ? "'" : '') + s.replace(/"/g, '""') + '"' : s; };
/** RFC 4180 CSV with formula-injection guarding. */
export function toCsv(rows: Record<string, unknown>[], columns?: string[]): string {
  const cols = columns ?? [...new Set(rows.flatMap((r) => Object.keys(r)))];
  return [cols.map(cell).join(','), ...rows.map((r) => cols.map((c) => cell(r[c])).join(','))].join('\r\n') + '\r\n';
}
```

handoff/workflow-router-app/lib/exports/json.ts · 2 lines

```
export const toJson = (x: unknown) => JSON.stringify({ schema: 'semester.workflow-router/1', exportedAt: new
Date().toISOString(), data: x }, null, 2) + '\n';
```

handoff/workflow-router-app/lib/exports/markdown.ts · 12 lines

```
import { PLATFORM_LABEL, WORKFLOWS } from '../benchmark/benchmark.config';
import type { Recommendation } from '../workflow-router/engine';
import type { RouterAnswers } from '../validation/workflow-router';
export function recommendationMarkdown(a: RouterAnswers, r: Recommendation, prompt: string): string {
  return ['# Semester AI workflow recommendation', '', '| | |', '| --- | --- |', '| Primary | ' + PLATFORM_LABEL[r.primary] + '
|', '| Secondary | ' + PLATFORM_LABEL[r.secondary] + ' |', '| Confidence | ' + Math.round(r.confidence * 100) + '% |', '|
Production escalation | ' + (r.escalation ? 'Yes — ' + r.escalationReasons.join('; ') : 'No') + ' |', '| Handoff | ' +
r.handoffFormat + ' |', '',
    '## Requirements', ...Object.entries(a).map(([k, v]) => '- ' + k + ': ' + v), '', '## Why', ...r.reasons[r.primary].map((x)
=> '- ' + x), '', '## Limitations', ...r.limitations.map((x) => '- ' + x), '', '## Prompt', '````', prompt, '````',
''].join('\n');
}
export function benchmarkMarkdown(rows: { workflow: number; platform: string; status: string; outOf100: number | null; graders:
number }[]): string {
  const title = (n: number) => WORKFLOWS.find((w) => w.number === n)?.title ?? String(n);
  return ['# Semester benchmark report', '', '| # | Workflow | Platform | Status | Score /100 | Graders |', '| --- | --- | ---
| --- | --- | --- |', ...rows.map((r) => '| ' + r.workflow + ' | ' + title(r.workflow) + ' | ' + r.platform + ' | ' + r.status
+ ' | ' + (r.outOf100 ?? '—') + ' | ' + r.graders + ' |'), ''].join('\n');
}
```

handoff/workflow-router-app/lib/supabase/client.ts · 4 lines

```
import { createBrowserClient } from '@supabase/ssr';
import type { Database } from '@/types/database';
export const createClient = () => createBrowserClient<Database>(process.env.NEXT_PUBLIC_SUPABASE_URL!,
process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!);
```

handoff/workflow-router-app/lib/supabase/middleware.ts · 11 lines

```
import { createServerClient } from '@supabase/ssr';
import { NextResponse, type NextRequest } from 'next/server';
export async function updateSession(request: NextRequest) {
  let response = NextResponse.next({ request });
  const supabase = createServerClient(process.env.NEXT_PUBLIC_SUPABASE_URL!, process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
{
    cookies: { getAll: () => request.cookies.getAll(), setAll: (list) => { list.forEach(({ name, value }) =>
request.cookies.set(name, value)); response = NextResponse.next({ request }); list.forEach(({ name, value, options }) =>
response.cookies.set(name, value, options)); } },
  });
  await supabase.auth.getUser(); // refreshes the session cookie
  return response;
}
```

handoff/workflow-router-app/lib/supabase/server.ts · 18 lines

```
import 'server-only';
import { createServerClient } from '@supabase/ssr';
import { cookies } from 'next/headers';
import type { Database } from '@/types/database';
export async function createClient() {
  const store = await cookies();
  return createServerClient<Database>(process.env.NEXT_PUBLIC_SUPABASE_URL!, process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
{
    cookies: { getAll: () => store.getAll(), setAll: (list) => { try { list.forEach(({ name, value, options }) =>
store.set(name, value, options)); } catch { /* called from a Server Component; middleware refreshes */ } } },
  });
}
/** Server-validated user (never trust getSession on the server). */
export async function requireUser() {
  const supabase = await createClient();
  const { data: { user }, error } = await supabase.auth.getUser();
  if (error || !user) return { supabase, user: null } as const;
  return { supabase, user } as const;
}
```

handoff/workflow-router-app/lib/validation/benchmark.ts · 19 lines

```
import { z } from 'zod';
import { CRITERIA, PERSISTENCE_MODES, PLATFORMS } from '../benchmark/benchmark.config';
const grade = z.number().int().min(0).max(5).nullable();
export const RunSchema = z.object({
  taskId: z.string().uuid(), platform: z.enum(PLATFORMS), executedPrompt: z.string().min(1),
  modelName: z.string().max(120).optional(), modelVersion: z.string().max(120).optional(),
  status: z.enum(['pending', 'running', 'completed', 'manual-review-required', 'unsupported', 'failed']),
  statusReason: z.string().max(2000).optional(), outputUrl: z.string().url().optional().or(z.literal('')),
  sourceManifest: z.array(z.object({ path: z.string(), bytes: z.number().int().nonnegative().optional() })).default([]),
  runtimeNotes: z.string().max(4000).optional(), consoleNotes: z.string().max(8000).optional(), durationMs:
z.number().int().nonnegative().optional(),
  persistenceTested: z.boolean().default(false), persistenceSurvivedReload: z.boolean().nullable().default(null),
  persistenceMode: z.enum(PERSISTENCE_MODES).default('none'), persistenceEvidence: z.string().max(2000).optional(),
persistenceFailureNotes: z.string().max(2000).optional(),
}).superRefine((r, ctx) => {
  if ((r.status === 'manual-review-required' || r.status === 'unsupported') && !r.statusReason) ctx.addIssue({ code: 'custom',
path: ['statusReason'], message: 'Explain why the run needs manual review or is unsupported' });
  if (r.persistenceTested && r.persistenceSurvivedReload === null) ctx.addIssue({ code: 'custom', path:
['persistenceSurvivedReload'], message: 'Record whether state survived reload' });
  if (r.persistenceTested && !r.persistenceEvidence) ctx.addIssue({ code: 'custom', path: ['persistenceEvidence'], message:
'Persistence evidence (URL, row id, screenshot or log) is required' });
});
export const GradeSchema = z.object({ runId: z.string().uuid(), ...Object.fromEntries(CRITERIA.map((c) => [c, grade])) as
Record<(typeof CRITERIA)[number], typeof grade>, evidence: z.string().max(4000).optional(), notes:
z.string().max(4000).optional() });
```

handoff/workflow-router-app/lib/validation/workflow-router.ts · 12 lines

```
import { z } from 'zod';
export const RouterAnswersSchema = z.object({
  deliverable: z.enum(['interactive-tool', 'prototype', 'marketing-page', 'diagram', 'document', 'presentation', 'data-
analysis', 'react-component', 'full-stack']),
  interaction: z.enum(['none', 'simple-local', 'complex-local', 'multi-user']),
  backend: z.enum(['none', 'file-export', 'database', 'server-api', 'auth-secrets']),
  technology: z.enum(['none', 'html', 'react', 'nextjs', 'nextjs-postgres', 'nextjs-supabase']),
  persistence: z.enum(['none', 'database', 'auth', 'storage', 'realtime', 'multi-tenant']),
  handoff: z.enum(['share', 'document-export', 'github', 'vercel']),
});
export type RouterAnswers = z.infer<typeof RouterAnswersSchema>;
export const SaveRecommendationSchema = z.object({ title: z.string().trim().min(1).max(160), organizationId:
z.string().uuid().nullable(), answers: RouterAnswersSchema, visibility: z.enum(['private', 'organization']).default('private')
});
```

handoff/workflow-router-app/lib/workflow-router/engine.ts · 79 lines

```
import type { Platform } from '../benchmark/benchmark.config';
import type { RouterAnswers } from '../validation/workflow-router';

export interface Recommendation { primary: Platform; secondary: Platform; confidence: number; scores: Record<Platform, number>;
```

```
reasons: Record<Platform, string[]>; escalation: boolean; escalationReasons: string[]; limitations: string[]; handoffFormat:
string; forcedV0: boolean }

/** Selections that must route to v0: anything needing a server, persistence, auth, multi-user, or deployment. */
export function v0Triggers(a: RouterAnswers): string[] {
  const t: string[] = [];
  if (a.technology === 'nextjs' || a.technology === 'nextjs-postgres' || a.technology === 'nextjs-supabase') t.push('Next.js
target');
  if (a.technology === 'nextjs-postgres') t.push('Postgres');
  if (a.technology === 'nextjs-supabase') t.push('Supabase / Auth');
  if (a.backend === 'database') t.push('Persistent records');
  if (a.backend === 'server-api') t.push('Server-side APIs or webhooks');
  if (a.backend === 'auth-secrets') t.push('Authentication and private server secrets');
  if (a.persistence === 'database' || a.persistence === 'storage') t.push('Persistent records');
  if (a.persistence === 'auth') t.push('Authentication');
  if (a.persistence === 'realtime') t.push('Realtime / multi-user');
  if (a.persistence === 'multi-tenant') t.push('Organization workspace, roles and Row Level Security');
  if (a.interaction === 'multi-user') t.push('Multi-user collaboration');
  if (a.deliverable === 'full-stack') t.push('Full-stack feature');
  if (a.handoff === 'github') t.push('GitHub-backed deployment');
  if (a.handoff === 'vercel') t.push('Vercel deployment');
  return [...new Set(t)];
}

export function recommend(a: RouterAnswers): Recommendation {
  const s: Record<Platform, number> = { 'claude-artifacts': 0, 'chatgpt-canvas': 0, v0: 0 };
  const r: Record<Platform, string[]> = { 'claude-artifacts': [], 'chatgpt-canvas': [], v0: [] };
  const add = (p: Platform, n: number, why: string) => { s[p] += n; r[p].push(why); };

  switch (a.deliverable) {
    case 'interactive-tool': add('claude-artifacts', 4, 'Self-contained interactive tools are the Artifacts sweet spot');
add('v0', 1, 'v0 can host the tool if it later needs a backend'); break;
    case 'prototype': add('claude-artifacts', 4, 'Fast shareable prototypes render beside the chat'); add('v0', 2, 'v0
prototypes become real projects'); break;
    case 'marketing-page': add('claude-artifacts', 3, 'Single-page HTML is portable'); add('v0', 3, 'v0 ships Next.js marketing
pages to Vercel'); break;
    case 'diagram': add('claude-artifacts', 5, 'Mermaid/SVG diagrams render natively'); add('chatgpt-canvas', 1, 'Canvas can
iterate diagram source text'); break;
    case 'document': add('chatgpt-canvas', 5, 'Revision-centred writing with PDF/DOCX/Markdown export'); add('claude-
artifacts', 2, 'Markdown documents render as artifacts'); break;
    case 'presentation': add('chatgpt-canvas', 3, 'Iterate slide copy and outline'); add('claude-artifacts', 3, 'HTML slide
decks render and share'); break;
    case 'data-analysis': add('chatgpt-canvas', 5, 'Canvas runs Python for data analysis'); add('claude-artifacts', 1, 'Client-
side charts for a finished dataset'); break;
    case 'react-component': add('claude-artifacts', 3, 'Single-component React renders in Artifacts'); add('v0', 3, 'v0 places
components in a real project'); add('chatgpt-canvas', 2, 'Canvas iterates component code'); break;
    case 'full-stack': add('v0', 6, 'Multi-file Next.js with server routes'); break;
  }
  switch (a.interaction) {
    case 'none': add('chatgpt-canvas', 1, 'No interaction needed'); break;
    case 'simple-local': add('claude-artifacts', 2, 'Simple local state is fine in an artifact'); break;
    case 'complex-local': add('claude-artifacts', 2, 'Complex client state in one React file'); add('v0', 1, 'Complex state may
outgrow one file'); break;
    case 'multi-user': add('v0', 5, 'Multi-user needs a server and database'); break;
  }
  if (a.backend === 'file-export') { add('claude-artifacts', 2, 'Browser-generated CSV/PNG/Markdown downloads'); add('chatgpt-
canvas', 2, 'Document/source-file export'); }
  if (['database', 'server-api', 'auth-secrets'].includes(a.backend)) add('v0', 5, 'Server code, API routes and database
connections run in v0');
  if (a.technology === 'html') add('claude-artifacts', 2, 'Plain HTML/CSS/JS is portable');
  if (a.technology === 'react') { add('claude-artifacts', 1, 'React via approved CDNs'); add('v0', 1, 'Native React project');
}
  if (a.technology.startsWith('nextjs')) add('v0', 5, 'Next.js App Router is v0's native target');
  if (a.persistence === 'none') add('claude-artifacts', 1, 'No persistence required');
  if (a.handoff === 'share') add('claude-artifacts', 2, 'Share a published artifact immediately');
  if (a.handoff === 'document-export') add('chatgpt-canvas', 2, 'PDF/DOCX/Markdown export');
  if (a.handoff === 'github' || a.handoff === 'vercel') add('v0', 4, 'GitHub sync and Vercel deployment');

  const triggers = v0Triggers(a);
  const forcedV0 = triggers.length > 0;
  if (forcedV0) add('v0', 10, 'Required for: ' + triggers.join(', '));

  const order = (Object.keys(s) as Platform[]).sort((x, y) => s[y] - s[x]);
  let [primary, secondary] = order;
  if (forcedV0 && primary !== 'v0') { secondary = primary; primary = 'v0'; }
  const total = Object.values(s).reduce((a2, b) => a2 + b, 0) || 1;
  const margin = (s[primary] - s[secondary]) / total;
```

```
  const confidence = Math.min(0.97, Math.round((0.55 + margin) * 100) / 100);

  const escalationReasons = [...triggers];
  if (primary !== 'v0' && (a.persistence !== 'none')) escalationReasons.push('Persistence selected: artifact/canvas state is
not durable storage');
  const limitations = {
    'claude-artifacts': ['No durable backend, secret store or production auth', 'Libraries only from approved CDNs',
'localStorage/fetch may be restricted in the sandbox'],
    'chatgpt-canvas': ['Not a production backend or deployment target', 'React/HTML preview runs in an isolated sandbox'],
    v0: ['Generated code still needs migrations, RLS, tests and environment variables before production', 'Sandbox is isolated;
production secrets live in Vercel/Supabase'],
  }[primary];
  const handoffFormat = { 'claude-artifacts': 'Single-file HTML or React/JSX, Mermaid/SVG, browser CSV/PNG download', 'chatgpt-
canvas': 'PDF, DOCX or Markdown; code as .py/.ts/.sql', v0: 'Next.js project → GitHub repository → Vercel deployment' }
[primary];
  return { primary, secondary, confidence, scores: s, reasons: r, escalation: escalationReasons.length > 0, escalationReasons,
limitations, handoffFormat, forcedV0 };
}
```

handoff/workflow-router-app/lib/workflow-router/prompts.ts · 20 lines

```
import { PLATFORM_LABEL } from '../benchmark/benchmark.config';
import type { Recommendation } from './engine';
import type { RouterAnswers } from '../validation/workflow-router';
export function buildPrompt(a: RouterAnswers, r: Recommendation): string {
  const head = 'You are building a Semester ' + a.deliverable.replace('-', ' ') + ' in ' + PLATFORM_LABEL[r.primary] + '.';
  const rules = r.primary === 'v0'
    ? ['Next.js App Router, TypeScript strict, Tailwind.', 'Supabase via @supabase/ssr: lib/supabase/client.ts, server.ts,
middleware session refresh.', 'Validate every mutation with Zod on the server; call supabase.auth.getUser() server-side.',
'Write SQL migrations with RLS on every exposed table; never expose a service-role key.', 'Add tests and .env.example; do not
call it production-ready until migrations, RLS and tests pass.']
    : r.primary === 'chatgpt-canvas'
      ? ['Work as a revisable document/code canvas.', 'Use clear sections and export to Markdown/DOCX/PDF (or the code file
type).', 'State assumptions and sources.']
      : ['Single self-contained file (HTML or one React component).', 'Libraries only from approved CDNs; no secrets, no fetch
to private APIs.', 'Treat state as local preview only; provide CSV/Markdown download if data must leave.'];
  return [head, '', 'Requirements: interaction=' + a.interaction + ', backend=' + a.backend + ', tech=' + a.technology + ',
persistence=' + a.persistence + ', handoff=' + a.handoff + '.', '', ...rules.map((x) => '- ' + x), '', 'Use Semester tokens:
Ink chrome, Parchment content, Semester blue for action, glyph + word + tone for every status, and label every fact with its
source.'].join('\n');
}
export function githubPlan(r: Recommendation): string[] {
  return r.primary === 'v0' ? ['Create repo from v0 → GitHub', 'Protect main; PRs require typecheck, lint, test, build', 'Add
Supabase env vars in Vercel (never the service role in NEXT_PUBLIC_*)', 'Run migrations via supabase db push in CI', 'Preview
deployments per PR']
    : ['Export the source (' + r.handoffFormat + ')', 'Commit to docs/ or prototypes/ in the Semester repo', 'Open a PR linking
the artifact URL', 'Escalate to v0 if persistence/auth is later required'];
}
export function supabasePrompt(): string {
  return 'Add Supabase to this Next.js app: @supabase/ssr browser/server clients, middleware session refresh, magic-link auth
with /auth/confirm, migrations for organizations, organization_members (owner/admin/member/viewer), and the feature tables with
RLS scoped to auth.uid() and org membership. Use NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY only.';
}
```

handoff/workflow-router-app/middleware.ts · 6 lines

```
// Next.js 15: middleware.ts. On Next.js 16+, rename to proxy.ts and export `proxy` instead.
import type { NextRequest } from 'next/server';
import { updateSession } from '@/lib/supabase/middleware';
export async function middleware(request: NextRequest) { return updateSession(request); }
export const config = { matcher: ['/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*)'] };
```

handoff/workflow-router-app/next.config.ts · 4 lines

```
import type { NextConfig } from 'next';
const config: NextConfig = { reactStrictMode: true, poweredByHeader: false };
export default config;
```

handoff/workflow-router-app/package.json · 9 lines

```
{
  "name": "semester-workflow-router",
  "private": true,
  "type": "module",
  "scripts": { "dev": "next dev", "build": "next build", "start": "next start", "lint": "next lint", "typecheck": "tsc --
noEmit", "test": "vitest run", "db:types": "supabase gen types typescript --local > types/database.ts" },
  "dependencies": { "@supabase/ssr": "^0.5.2", "@supabase/supabase-js": "^2.45.0", "next": "^15.1.0", "react": "^19.0.0",
"react-dom": "^19.0.0", "server-only": "^0.0.1", "zod": "^3.23.8" },
  "devDependencies": { "@tailwindcss/postcss": "^4.0.0", "@types/node": "^22", "@types/react": "^19", "eslint": "^9", "eslint-
config-next": "^15.1.0", "tailwindcss": "^4.0.0", "typescript": "^5.6.0", "vite-tsconfig-paths": "^5.1.0", "vitest": "^2.1.0" }
}
```

handoff/workflow-router-app/postcss.config.mjs · 2 lines

```
export default { plugins: { '@tailwindcss/postcss': {} } };
```

handoff/workflow-router-app/supabase/migrations/001_core_workflow_router.sql · 69 lines

```
create extension if not exists pgcrypto;
create type public.org_role as enum ('owner', 'admin', 'member', 'viewer');
create type public.visibility as enum ('private', 'organization');

create table public.organizations (
  id uuid primary key default gen_random_uuid(),
  name text not null check (char_length(name) between 1 and 120),
  created_by uuid not null default auth.uid() references auth.users(id),
  created_at timestamptz not null default now()
);
create table public.organization_members (
  organization_id uuid not null references public.organizations(id) on delete cascade,
  user_id uuid not null references auth.users(id) on delete cascade,
  role public.org_role not null default 'member',
  created_at timestamptz not null default now(),
  primary key (organization_id, user_id)
);
create table public.workflow_recommendations (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null default auth.uid() references auth.users(id) on delete cascade,
  organization_id uuid references public.organizations(id) on delete cascade,
  visibility public.visibility not null default 'private',
  title text not null check (char_length(title) between 1 and 160),
  answers jsonb not null,
  primary_platform text not null check (primary_platform in ('claude-artifacts','chatgpt-canvas','v0')),
  secondary_platform text not null check (secondary_platform in ('claude-artifacts','chatgpt-canvas','v0')),
  confidence numeric(4,3) not null check (confidence between 0 and 1),
  result jsonb not null,
  created_at timestamptz not null default now(),
  check (visibility = 'private' or organization_id is not null)
);

-- Membership helpers. SECURITY DEFINER avoids recursive RLS on organization_members.
create or replace function public.is_org_member(org uuid) returns boolean language sql stable security definer set search_path
= public as $$
  select exists (select 1 from organization_members m where m.organization_id = org and m.user_id = auth.uid()) $$;
create or replace function public.has_org_role(org uuid, roles public.org_role[]) returns boolean language sql stable security
definer set search_path = public as $$
  select exists (select 1 from organization_members m where m.organization_id = org and m.user_id = auth.uid() and m.role =
any(roles)) $$;
revoke all on function public.is_org_member(uuid), public.has_org_role(uuid, public.org_role[]) from public;
grant execute on function public.is_org_member(uuid), public.has_org_role(uuid, public.org_role[]) to authenticated;

-- Creator becomes owner.
create or replace function public.add_org_owner() returns trigger language plpgsql security definer set search_path = public as
$$
begin insert into organization_members (organization_id, user_id, role) values (new.id, new.created_by, 'owner'); return new;
end $$;
create trigger organizations_owner after insert on public.organizations for each row execute function public.add_org_owner();

alter table public.organizations enable row level security;
alter table public.organization_members enable row level security;
alter table public.workflow_recommendations enable row level security;
```

```
create policy org_select on public.organizations for select to authenticated using (public.is_org_member(id));
create policy org_insert on public.organizations for insert to authenticated with check (created_by = auth.uid());
create policy org_update on public.organizations for update to authenticated using (public.has_org_role(id,
array['owner','admin']::public.org_role[]));
create policy org_delete on public.organizations for delete to authenticated using (public.has_org_role(id,
array['owner']::public.org_role[]));

create policy mem_select on public.organization_members for select to authenticated using
(public.is_org_member(organization_id));
create policy mem_write on public.organization_members for all to authenticated
  using (public.has_org_role(organization_id, array['owner','admin']::public.org_role[]))
  with check (public.has_org_role(organization_id, array['owner','admin']::public.org_role[]) and (role <> 'owner' or
public.has_org_role(organization_id, array['owner']::public.org_role[])));

create policy rec_select on public.workflow_recommendations for select to authenticated
  using (owner_id = auth.uid() or (visibility = 'organization' and public.is_org_member(organization_id)));
create policy rec_insert on public.workflow_recommendations for insert to authenticated
  with check (owner_id = auth.uid() and (organization_id is null or public.has_org_role(organization_id,
array['owner','admin','member']::public.org_role[])));
create policy rec_update on public.workflow_recommendations for update to authenticated using (owner_id = auth.uid()) with
check (owner_id = auth.uid());
create policy rec_delete on public.workflow_recommendations for delete to authenticated
  using (owner_id = auth.uid() or (organization_id is not null and public.has_org_role(organization_id,
array['owner','admin']::public.org_role[])));
create index on public.workflow_recommendations (owner_id, created_at desc);
create index on public.workflow_recommendations (organization_id) where organization_id is not null;
```

handoff/workflow-router-app/supabase/migrations/002_benchmark_grading.sql · 72 lines

```
create table public.benchmark_suites (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null default auth.uid() references auth.users(id) on delete cascade,
  organization_id uuid references public.organizations(id) on delete cascade,
  visibility public.visibility not null default 'private',
  name text not null, description text, created_at timestamptz not null default now(),
  check (visibility = 'private' or organization_id is not null)
);
create table public.benchmark_tasks (
  id uuid primary key default gen_random_uuid(),
  suite_id uuid not null references public.benchmark_suites(id) on delete cascade,
  number int not null check (number between 1 and 99), slug text not null, title text not null, category text not null,
  prompt text not null, requirements jsonb not null default '[]', acceptance jsonb not null default '[]',
  recommended_platform text not null check (recommended_platform in ('claude-artifacts','chatgpt-canvas','v0')),
  requires_persistence boolean not null default false, weights jsonb,
  unique (suite_id, number)
);
create table public.benchmark_runs (
  id uuid primary key default gen_random_uuid(),
  task_id uuid not null references public.benchmark_tasks(id) on delete cascade,
  platform text not null check (platform in ('claude-artifacts','chatgpt-canvas','v0')),
  created_by uuid not null default auth.uid() references auth.users(id),
  status text not null check (status in ('pending','running','completed','manual-review-required','unsupported','failed')),
  status_reason text, executed_prompt text not null, model_name text, model_version text,
  output_url text, source_manifest jsonb not null default '[]', runtime_notes text, console_notes text, duration_ms int check
(duration_ms >= 0),
  persistence_tested boolean not null default false, persistence_survived_reload boolean,
  persistence_mode text not null default 'none' check (persistence_mode in ('none','local-preview','browser-
local','session','database','project-backed')),
  persistence_evidence text, persistence_failure_notes text,
  created_at timestamptz not null default now(),
  check (status not in ('manual-review-required','unsupported') or status_reason is not null),
  check (not persistence_tested or (persistence_survived_reload is not null and persistence_evidence is not null))
);
create table public.benchmark_grades (
  id uuid primary key default gen_random_uuid(),
  run_id uuid not null references public.benchmark_runs(id) on delete cascade,
  grader_id uuid not null default auth.uid() references auth.users(id),
  correctness smallint check (correctness between 0 and 5), code_quality smallint check (code_quality between 0 and 5),
  rendering smallint check (rendering between 0 and 5), state_management smallint check (state_management between 0 and 5),
  maintainability smallint check (maintainability between 0 and 5), handoff smallint check (handoff between 0 and 5),
  sandbox_fit smallint check (sandbox_fit between 0 and 5), persistence smallint check (persistence between 0 and 5),
  evidence text, notes text, created_at timestamptz not null default now(),
  unique (run_id, grader_id)
);
create index on public.benchmark_tasks (suite_id);
```

```
create index on public.benchmark_runs (task_id, platform);
create index on public.benchmark_grades (run_id);

-- Access follows the suite.
create or replace function public.can_read_suite(s uuid) returns boolean language sql stable security definer set search_path =
public as $$
  select exists (select 1 from benchmark_suites b where b.id = s and (b.owner_id = auth.uid() or (b.visibility = 'organization'
and is_org_member(b.organization_id)))) $$;
create or replace function public.can_write_suite(s uuid) returns boolean language sql stable security definer set search_path
= public as $$
  select exists (select 1 from benchmark_suites b where b.id = s and (b.owner_id = auth.uid() or (b.visibility = 'organization'
and has_org_role(b.organization_id, array['owner','admin','member']::org_role[])))) $$;
grant execute on function public.can_read_suite(uuid), public.can_write_suite(uuid) to authenticated;

alter table public.benchmark_suites enable row level security;
alter table public.benchmark_tasks enable row level security;
alter table public.benchmark_runs enable row level security;
alter table public.benchmark_grades enable row level security;

create policy suite_select on public.benchmark_suites for select to authenticated using (public.can_read_suite(id));
create policy suite_insert on public.benchmark_suites for insert to authenticated with check (owner_id = auth.uid() and
(organization_id is null or public.has_org_role(organization_id, array['owner','admin','member']::public.org_role[])));
create policy suite_update on public.benchmark_suites for update to authenticated using (public.can_write_suite(id));
create policy suite_delete on public.benchmark_suites for delete to authenticated using (owner_id = auth.uid());
create policy task_select on public.benchmark_tasks for select to authenticated using (public.can_read_suite(suite_id));
create policy task_write on public.benchmark_tasks for all to authenticated using (public.can_write_suite(suite_id)) with check
(public.can_write_suite(suite_id));
create policy run_select on public.benchmark_runs for select to authenticated using (public.can_read_suite((select suite_id
from public.benchmark_tasks t where t.id = task_id)));
create policy run_insert on public.benchmark_runs for insert to authenticated with check (created_by = auth.uid() and
public.can_write_suite((select suite_id from public.benchmark_tasks t where t.id = task_id)));
create policy run_update on public.benchmark_runs for update to authenticated using (created_by = auth.uid());
create policy grade_select on public.benchmark_grades for select to authenticated using (public.can_read_suite((select
t.suite_id from public.benchmark_runs r join public.benchmark_tasks t on t.id = r.task_id where r.id = run_id)));
create policy grade_insert on public.benchmark_grades for insert to authenticated with check (grader_id = auth.uid() and
public.can_write_suite((select t.suite_id from public.benchmark_runs r join public.benchmark_tasks t on t.id = r.task_id where
r.id = run_id)));
create policy grade_update on public.benchmark_grades for update to authenticated using (grader_id = auth.uid()) with check
(grader_id = auth.uid());
```

handoff/workflow-router-app/supabase/seed.sql · 22 lines

```
-- Seeds one suite with the 12 canonical workflows for the first user. Run after signing up once:
--   supabase db reset   (local)   or paste into the SQL editor with :owner set.
do $$
declare owner uuid := (select id from auth.users order by created_at limit 1); suite uuid;
begin
  if owner is null then raise notice 'Sign up a user first'; return; end if;
  insert into public.benchmark_suites (owner_id, name, description) values (owner, 'Semester canonical 12', 'Claude Artifacts
vs ChatGPT Canvas vs v0 across the 12 canonical workflows') returning id into suite;
  insert into public.benchmark_tasks (suite_id, number, slug, title, category, prompt, recommended_platform,
requires_persistence) values
  (suite, 1, 'artifact-tracker', 'Sortable artifact tracker', 'Interactive tool', 'Build a single-file React tracker of every
Semester artifact with search, filters, sortable columns and CSV export.', 'claude-artifacts', false),
  (suite, 2, 'onboarding-prototype', 'Interactive onboarding prototype', 'Product prototype', 'Prototype Semester student
onboarding with consent one decision at a time.', 'claude-artifacts', false),
  (suite, 3, 'b2b-landing', 'B2B SaaS landing page', 'Marketing page', 'Design a Semester institution landing page with a
capability status map.', 'claude-artifacts', false),
  (suite, 4, 'architecture-diagram', 'Multi-tenant system architecture diagram', 'Diagram', 'Draw Semester multi-tenant
architecture as Mermaid with the RLS boundary.', 'claude-artifacts', false),
  (suite, 5, 'tech-design-doc', 'Technical design document', 'Document', 'Write a TDD for Semester degree-audit sync.',
'chatgpt-canvas', false),
  (suite, 6, 'launch-presentation', 'Product launch presentation', 'Presentation', 'Draft a 10-slide pilot launch presentation
for registrars.', 'chatgpt-canvas', false),
  (suite, 7, 'subscription-csv', 'Subscription CSV analysis', 'Data analysis', 'Analyse a subscription CSV: MRR, churn,
cohorts.', 'chatgpt-canvas', false),
  (suite, 8, 'metric-dashboard', 'Reusable React metric dashboard', 'React component', 'Build a reusable typed
<MetricDashboard> component with all states.', 'claude-artifacts', false),
  (suite, 9, 'auth-route-handler', 'Authenticated Next.js route handler', 'Full-stack feature', 'Implement an authenticated
route handler with Supabase, getUser and Zod.', 'v0', true),
  (suite, 10, 'supabase-crud', 'Supabase artifact CRUD workflow', 'Full-stack feature', 'Create artifact CRUD with migration,
RLS and server actions.', 'v0', true),
  (suite, 11, 'cookie-auth', 'Cookie-based Supabase Auth', 'Full-stack feature', 'Implement cookie-based Supabase Auth with
@supabase/ssr and middleware refresh.', 'v0', true),
  (suite, 12, 'multi-tenant-workspace', 'Multi-tenant SaaS workspace', 'Full-stack feature', 'Build organizations, roles and
```

```
org-scoped RLS.', 'v0', true);
end $$;
```

handoff/workflow-router-app/tests/RLS-TEST-PLAN.md · 21 lines

```
# RLS / auth test plan (run against `supabase start`)

Use two users A, B and an organization O where A=owner, C=viewer, B is not a member.

| # | As | Action | Expected |
| --- | --- | --- | --- |
| 1 | anon | select any table | 0 rows / permission denied |
| 2 | A | insert private recommendation | ok; B selects → 0 rows |
| 3 | A | insert recommendation visibility=organization, org O | C selects → 1 row; B → 0 |
| 4 | C (viewer) | insert into O | denied (rec_insert requires owner/admin/member) |
| 5 | B | insert member row into O | denied (mem_write) |
| 6 | A | create private suite + seed tasks | B selects tasks/runs/grades → 0 |
| 7 | B | insert run for A's task | denied (can_write_suite) |
| 8 | C | grade a run in an org suite | denied (viewer) |
| 9 | A | update another grader's grade | denied (grader_id) |
| 10 | any | insert run status=unsupported without reason | check constraint violation |
| 11 | any | insert run persistence_tested=true without evidence | check constraint violation |
| 12 | build | grep .next/static for SERVICE_ROLE | no match |

Automate with `supabase test db` (pgTAP) using `set local role authenticated; set local request.jwt.claims = '{"sub":"
<uuid>"}'`.
```

handoff/workflow-router-app/tests/exports.test.ts · 9 lines

```
import { describe, expect, it } from 'vitest';
import { toCsv } from '@/lib/exports/csv';
import { recommendationMarkdown } from '@/lib/exports/markdown';
import { recommend } from '@/lib/workflow-router/engine';
describe('exports', () => {
  it('escapes CSV and guards formulas', () => { expect(toCsv([{ a: 'x,y', b: '=SUM(1)'
}])).toBe('a,b\r\n"x,y","\'=SUM(1)"\r\n'); });
  it('writes a Markdown recommendation with primary and escalation', () => { const a = { deliverable: 'full-stack',
interaction: 'multi-user', backend: 'database', technology: 'nextjs-supabase', persistence: 'multi-tenant', handoff: 'vercel' }
as const; const md = recommendationMarkdown(a, recommend(a), 'p'); expect(md).toContain('| Primary | v0 |');
expect(md).toContain('Production escalation | Yes'); });
});
```

handoff/workflow-router-app/tests/persistence.test.ts · 9 lines

```
import { describe, expect, it } from 'vitest';
import { clampPersistenceGrade, persistenceCeiling } from '@/lib/benchmark/probes/persistence';
describe('persistence ceiling', () => {
  it('local state never earns more than 2', () => { expect(persistenceCeiling({ tested: true, survivedReload: true, mode:
'browser-local', evidence: 'video' }, true)).toBe(2); expect(persistenceCeiling({ tested: true, survivedReload: true, mode:
'local-preview', evidence: 'x' }, true)).toBe(1); });
  it('database with evidence can earn 5', () => { expect(persistenceCeiling({ tested: true, survivedReload: true, mode:
'database', evidence: 'row abc' }, true)).toBe(5); });
  it('untested required persistence is 0', () => { expect(clampPersistenceGrade(5, { tested: false, survivedReload: null, mode:
'database' }, true)).toBe(0); });
  it('lost on reload is 0', () => { expect(persistenceCeiling({ tested: true, survivedReload: false, mode: 'database',
evidence: 'log' }, true)).toBe(0); });
});
```

handoff/workflow-router-app/tests/routing.test.ts · 12 lines

```
import { describe, expect, it } from 'vitest';
import { recommend } from '@/lib/workflow-router/engine';
import type { RouterAnswers } from '@/lib/validation/workflow-router';
const base: RouterAnswers = { deliverable: 'interactive-tool', interaction: 'simple-local', backend: 'none', technology:
'none', persistence: 'none', handoff: 'share' };
describe('router', () => {
  it('sends a self-contained tool to Claude Artifacts', () => { expect(recommend(base).primary).toBe('claude-artifacts'); });
  it('sends documents and data analysis to Canvas', () => { expect(recommend({ ...base, deliverable: 'document', interaction:
'none', handoff: 'document-export' }).primary).toBe('chatgpt-canvas'); expect(recommend({ ...base, deliverable: 'data-
analysis', interaction: 'none' }).primary).toBe('chatgpt-canvas'); });
```

```
  const forced: Partial<RouterAnswers>[] = [{ technology: 'nextjs' }, { technology: 'nextjs-postgres' }, { technology: 'nextjs-
supabase' }, { backend: 'database' }, { backend: 'server-api' }, { backend: 'auth-secrets' }, { persistence: 'database' }, {
persistence: 'auth' }, { persistence: 'multi-tenant' }, { persistence: 'realtime' }, { interaction: 'multi-user' }, { handoff:
'github' }, { handoff: 'vercel' }];
  for (const f of forced) it('forces v0 for ' + JSON.stringify(f), () => { const r = recommend({ ...base, deliverable:
'diagram', ...f }); expect(r.primary).toBe('v0'); expect(r.forcedV0).toBe(true); expect(r.escalation).toBe(true); });
  it('bounds confidence', () => { const c = recommend(base).confidence; expect(c).toBeGreaterThan(0.5);
expect(c).toBeLessThanOrEqual(0.97); });
});
```

handoff/workflow-router-app/tests/scoring.test.ts · 15 lines

```
import { describe, expect, it } from 'vitest';
import { aggregateGrades, leaderboard, weightedScore } from '@/lib/benchmark/scoring';
describe('weightedScore', () => {
  it('uses the default weights', () => { const s = weightedScore({ correctness: 5, codeQuality: 5, rendering: 5,
stateManagement: 5, maintainability: 5, handoff: 5, sandboxFit: 5 }); expect(s).toMatchObject({ outOf5: 5, outOf100: 100,
missing: [] }); });
  it('weights correctness at 25%', () => { expect(weightedScore({ correctness: 5, codeQuality: 0, rendering: 0,
stateManagement: 0, maintainability: 0, handoff: 0, sandboxFit: 0 }).outOf100).toBe(25); });
  it('renormalises around missing grades and reports them', () => { const s = weightedScore({ correctness: 4 });
expect(s.outOf5).toBe(4); expect(s.missing).toContain('codeQuality'); });
  it('is null when nothing is graded', () => { expect(weightedScore({}).outOf5).toBeNull(); });
  it('honours task overrides incl. persistence weight', () => { expect(weightedScore({ correctness: 5, persistence: 0 }, {
correctness: 0.5, persistence: 0.5, codeQuality: 0, rendering: 0, stateManagement: 0, maintainability: 0, handoff: 0,
sandboxFit: 0 }).outOf5).toBe(2.5); });
  it('rejects out-of-range grades', () => { expect(() => weightedScore({ correctness: 6 })).toThrow(RangeError); });
});
describe('aggregate + leaderboard', () => {
  it('averages graders per criterion', () => { expect(aggregateGrades([{ correctness: 4 }, { correctness: 2
}]).outOf5).toBe(3); });
  it('ranks graded platforms above ungraded', () => { const l = leaderboard([{ key: 'v0', score: 4 }, { key: 'Canvas', score:
null }, { key: 'Claude', score: 4.5 }]); expect(l.map((r) => r.key)).toEqual(['Claude', 'v0', 'Canvas']); });
});
```

handoff/workflow-router-app/tests/validation.test.ts · 10 lines

```
import { describe, expect, it } from 'vitest';
import { RunSchema } from '@/lib/validation/benchmark';
const ok = { taskId: '00000000-0000-4000-8000-000000000000', platform: 'v0', executedPrompt: 'x', status: 'completed' };
describe('RunSchema', () => {
  it('accepts a minimal completed run', () => { expect(RunSchema.safeParse(ok).success).toBe(true); });
  it('requires a reason for manual-review-required', () => { expect(RunSchema.safeParse({ ...ok, status: 'manual-review-
required' }).success).toBe(false); });
  it('requires reload result and evidence when persistence was tested', () => { expect(RunSchema.safeParse({ ...ok,
persistenceTested: true }).success).toBe(false); expect(RunSchema.safeParse({ ...ok, persistenceTested: true,
persistenceSurvivedReload: true, persistenceEvidence: 'row 42' }).success).toBe(true); });
  it('rejects unknown platforms', () => { expect(RunSchema.safeParse({ ...ok, platform: 'bolt' }).success).toBe(false); });
});
```

handoff/workflow-router-app/tsconfig.json · 2 lines

```
{ "compilerOptions": { "target": "ES2022", "lib": ["dom", "dom.iterable", "esnext"], "strict": true, "noEmit": true, "module":
"esnext", "moduleResolution": "bundler", "resolveJsonModule": true, "isolatedModules": true, "jsx": "preserve", "incremental":
true, "plugins": [{ "name": "next" }], "paths": { "@/*": ["./*"] }, "skipLibCheck": true }, "include": ["next-env.d.ts",
"**/*.ts", "**/*.tsx", ".next/types/**/*.ts"], "exclude": ["node_modules"] }
```

handoff/workflow-router-app/types/database.ts · 12 lines

```
// Regenerate with: npx supabase gen types typescript --local > types/database.ts
type Row<T> = { Row: T; Insert: Partial<T>; Update: Partial<T>; Relationships: [] };
export interface Database { public: { Tables: {
  organizations: Row<{ id: string; name: string; created_by: string; created_at: string }>;
  organization_members: Row<{ organization_id: string; user_id: string; role: 'owner' | 'admin' | 'member' | 'viewer';
created_at: string }>;
  workflow_recommendations: Row<{ id: string; owner_id: string; organization_id: string | null; visibility: 'private' |
'organization'; title: string; answers: unknown; primary_platform: string; secondary_platform: string; confidence: number;
result: unknown; created_at: string }>;
  benchmark_suites: Row<{ id: string; owner_id: string; organization_id: string | null; visibility: 'private' | 'organization';
name: string; description: string | null; created_at: string }>;
```

```
  benchmark_tasks: Row<{ id: string; suite_id: string; number: number; slug: string; title: string; category: string; prompt:
string; requirements: unknown; acceptance: unknown; recommended_platform: string; requires_persistence: boolean; weights:
unknown }>;
  benchmark_runs: Row<{ id: string; task_id: string; platform: string; created_by: string; status: string; status_reason:
string | null; executed_prompt: string; model_name: string | null; model_version: string | null; output_url: string | null;
source_manifest: unknown; runtime_notes: string | null; console_notes: string | null; duration_ms: number | null;
persistence_tested: boolean; persistence_survived_reload: boolean | null; persistence_mode: string; persistence_evidence:
string | null; persistence_failure_notes: string | null; created_at: string }>;
  benchmark_grades: Row<{ id: string; run_id: string; grader_id: string; correctness: number | null; code_quality: number |
null; rendering: number | null; state_management: number | null; maintainability: number | null; handoff: number | null;
sandbox_fit: number | null; persistence: number | null; evidence: string | null; notes: string | null; created_at: string }>;
}; Views: {}; Functions: {}; Enums: {}; CompositeTypes: {} } }
```

handoff/workflow-router-app/vitest.config.ts · 4 lines

```
import { defineConfig } from 'vitest/config';
import paths from 'vite-tsconfig-paths';
export default defineConfig({ plugins: [paths()], test: { include: ['tests/**/*.test.ts'] } });
```

Claude Code toolkit scripts and build scripts

handoff/claude-code-toolkit/app/package.scripts.json · 10 lines

```
{
  "scripts": {
    "tokens:check": "vitest run src/lib/tokenexport.test.ts",
    "design-system:audit": "node scripts/design-system-audit.mjs",
    "design-system:report": "node scripts/design-system-report.mjs --with-tests",
    "design-system:check": "npm run tokens:check && node scripts/design-system-audit.mjs && vitest run
src/lib/designsystem.test.ts src/styles/hex.test.ts src/styles/tokens.test.ts",
    "design-system:baseline": "node scripts/design-system-audit.mjs --write-baseline"
  }
}
```

handoff/claude-code-toolkit/app/scripts/design-system-audit.mjs · 170 lines

```
#!/usr/bin/env node
/**
 * Semester design-system audit — read-only, no dependencies.
 *
 * Checks, against the repository's own authorities:
 *   1. Raw colour literals in .ts/.tsx (quoted #hex, rgb()/hsl()/oklch()) against the
 *      LEDGER in src/styles/hex.test.ts — the same count the census prints.
 *   2. CSS custom properties used with var() and no fallback that are defined nowhere
 *      (stylesheets, inline style objects, or tokensFor keys in src/lib/look.ts).
 *   3. tokens.css ↔ design-tokens/semester.tokens.json: every semantic name exported, and
 *      no exported semantic name missing from tokens.css.
 *   4. docs/design-system/FIGMA-MAPPING.md: every primitive.* / semantic.* path resolves in the export;
 *      valid / missing (obsolete) / unmapped-but-not-required candidates are reported separately.
 *   5. Warnings (minor, file:line): numeric zIndex, raw durations, cubic-bezier, raw box-shadow,
 *      numeric fontSize, numeric borderRadius in .ts/.tsx — each with the token to use instead.
 *
 * Usage (from app/):  node scripts/design-system-audit.mjs [--json] [--root <repo-root>]
 * Exit 1 when any violation is found. Warnings never fail.
 */
import { readFileSync, readdirSync, statSync, existsSync } from 'node:fs';
import { join, relative, resolve, dirname } from 'node:path';
import { fileURLToPath } from 'node:url';

const here = dirname(fileURLToPath(import.meta.url));

function walk(dir, exts, out = []) {
  if (!existsSync(dir)) return out;
  for (const name of readdirSync(dir)) {
    if (name === 'node_modules' || name.startsWith('.')) continue;
    const p = join(dir, name);
    const s = statSync(p);
    if (s.isDirectory()) walk(p, exts, out);
    else if (exts.some((e) => name.endsWith(e))) out.push(p);
  }
  return out;
}
const read = (p) => (existsSync(p) ? readFileSync(p, 'utf8') : '');
// Comments are blanked, newlines kept, so reported line numbers match the source file.
const noComments = (t) => t.replace(/\/\*[\s\S]*?\*\//g, (m) => m.replace(/[^\n]/g, '')).replace(/(^|[^:'"`])\/\/.*$/gm, '$1');
const lineOf = (t, i) => t.slice(0, i).split('\n').length;
const SUGGEST = { colour: 'an --app-* / --surface-* / --text-* / --status-* token, or a hex.test.ts LEDGER entry with a
reason', z: 'var(--layer-*) (styles/stacking.test.ts)', ms: 'var(--motion-*) (zeroed by reduced motion and data-calm)', ease:
'var(--ease-standard) / var(--ease-emphasized)', shadow: 'var(--elevation-*) / var(--lift-*)', font: 'var(--type-*) (scaled by
--text-scale; styles/scale.test.ts)', radius: 'var(--shape-*) / var(--r-*) (follow the Corners setting)' };
const isTest = (p) => /\.test\.(t|j)sx?$/.test(p) || /\/__fixtures__\//.test(p);

export function readLedger(hexTestText) {
  const out = {};
```

```
  for (const m of hexTestText.matchAll(/'([^']+\.tsx?)'\s*:\s*\{\s*count:\s*(\d+)/g)) out[m[1]] = Number(m[2]);
  return out;
}

export function semanticNames(tokensCss) {
  const clean = tokensCss.replace(/\/\*[\s\S]*?\*\//g, '');
  const block = /:root\s*\{([\s\S]*?)\n\}/.exec(clean)?.[1] ?? '';
  return [...block.matchAll(/(--[a-z0-9-]+)\s*:/g)].map((m) => m[1].slice(2));
}

export function mappingPaths(mappingMd) {
  const out = [];
  for (const line of mappingMd.split('\n')) {
    if (!line.startsWith('|')) continue;
    const cell = line.split('|').map((c) => c.trim()).find((c) => /^(primitive|semantic)\.[a-z0-9-]+$/.test(c));
    if (cell) out.push(cell);
  }
  return out;
}

export function audit({ root = resolve(here, '../..') } = {}) {
  const app = join(root, 'app');
  const src = join(app, 'src');
  const violations = [];
  const warnings = [];
  const stats = {};

  // 1. Raw colours vs the hex ledger
  const ledger = readLedger(read(join(src, 'styles/hex.test.ts')));
  const HEX = /['"`]#[0-9a-fA-F]{3,8}\b/g;
  const FN = /\b(rgba?|hsla?|oklch)\(\s*\d/g;
  const code = walk(src, ['.ts', '.tsx']).filter((p) => !isTest(p));
  let hexTotal = 0;
  for (const p of code) {
    const rel = relative(src, p).split('\\').join('/');
    if (rel === 'lib/look.ts' || rel.startsWith('lib/contrast')) continue; // the colour authorities
    const text = noComments(read(p));
    const n = text.match(HEX)?.length ?? 0;
    hexTotal += n;
    const allowed = ledger[rel] ?? 0;
    if (n > allowed) for (const m of text.matchAll(HEX)) violations.push({ check: 'raw-colour', severity: 'major', file: rel,
line: lineOf(text, m.index), value: m[0].slice(1), detail: n + ' hex literal(s); ledger allows ' + allowed + ' — use ' +
SUGGEST.colour });
    const fns = text.match(FN)?.length ?? 0;
    if (fns && !(rel in ledger)) warnings.push({ check: 'raw-colour-fn', file: rel, detail: fns + ' rgb()/hsl()/oklch()
literal(s) — prefer an --app-* / --status-* token' });
    const W = [['z-index', /zIndex:\s*(\d{2,})/g, SUGGEST.z], ['duration', /(?:transition|animation)(?:Duration)?:\s*['"`]
[^'"`]*?\b(\d{2,4}ms)/g, SUGGEST.ms], ['easing', /(cubic-bezier\([^)]*\))/g, SUGGEST.ease], ['shadow', /boxShadow:\s*['"`]
((?!var\()[^'"`]*\d+px[^'"`]*)['"`]/g, SUGGEST.shadow], ['font-size', /fontSize:\s*(\d+(?:\.\d+)?)\b(?!\s*\*)/g, SUGGEST.font],
['radius', /borderRadius:\s*(\d+)\b/g, SUGGEST.radius]];
    for (const [check, re, suggest] of W) for (const m of text.matchAll(re)) warnings.push({ check, severity: 'minor', file:
rel, line: lineOf(text, m.index), value: m[1], detail: 'raw value — prefer ' + suggest });
  }
  stats.hexLiterals = hexTotal;
  stats.ledgerTotal = Object.values(ledger).reduce((a, b) => a + b, 0);

  // 2. Undefined custom properties
  const css = walk(src, ['.css']);
  const defined = new Set();
  const addDefs = (t) => { for (const m of t.matchAll(/(--[a-z0-9-]+)\s*:/gi)) defined.add(m[1]); for (const m of
t.matchAll(/['"`](--[a-z0-9-]+)['"`]\s*[:\]]/gi)) defined.add(m[1]); };
  for (const p of css) addDefs(read(p));
  for (const p of code) addDefs(read(p));
  addDefs(read(join(src, 'lib/look.ts')));
  const used = new Map();
  const scan = (p, t) => { for (const m of t.matchAll(/var\(\s*(--[a-z0-9-]+)\s*(,)?/gi)) { if (m[2]) continue; if
(!used.has(m[1])) used.set(m[1], relative(src, p).split('\\').join('/')); } };
  for (const p of css) scan(p, noComments(read(p)));
  for (const p of code) scan(p, noComments(read(p)));
  const undefinedVars = [...used].filter(([v]) => !defined.has(v));
  for (const [v, file] of undefinedVars) violations.push({ check: 'undefined-var', severity: 'major', file, value: v, detail: v
+ ' is used without a fallback and defined nowhere' });
  stats.customPropertiesDefined = defined.size;

  // 3. tokens.css ↔ export
  const tokensCss = read(join(src, 'styles/tokens.css'));
  const exportPath = join(app, 'design-tokens/semester.tokens.json');
```

```
  let exported = null;
  try { exported = JSON.parse(read(exportPath)); } catch { violations.push({ check: 'export', file: 'design-
tokens/semester.tokens.json', detail: 'missing or not valid JSON — run npm run tokens:export' }); }
  if (exported) {
    const names = semanticNames(tokensCss);
    const sem = Object.keys(exported.semantic ?? {});
    for (const n of names) if (!sem.includes(n)) violations.push({ check: 'export-sync', file: 'styles/tokens.css', detail: '--
' + n + ' is not in semester.tokens.json — run npm run tokens:export' });
    for (const n of sem) if (!names.includes(n)) violations.push({ check: 'export-sync', file: 'design-
tokens/semester.tokens.json', detail: 'semantic.' + n + ' no longer exists in tokens.css — run npm run tokens:export' });
    stats.semanticTokens = sem.length;
    stats.primitiveTokens = Object.keys(exported.primitive ?? {}).length;
  }

  // 4. Figma mapping
  const MAP = 'docs/design-system/FIGMA-MAPPING.md';
  const mapping = read(join(root, MAP));
  const map = { valid: [], missing: [], unmappedCandidates: [] };
  if (!mapping) warnings.push({ check: 'figma-mapping', severity: 'minor', file: MAP, detail: 'not found — Figma parity is
unchecked' });
  else if (exported) {
    const paths = mappingPaths(mapping);
    for (const p of paths) {
      const [layer, key] = p.split('.');
      if (exported[layer] && key in exported[layer]) map.valid.push(p);
      else { map.missing.push(p); violations.push({ check: 'figma-mapping', severity: 'major', file: MAP, value: p, detail: p +
' does not resolve in semester.tokens.json (obsolete or misspelt)' }); }
    }
    const mapped = new Set(paths);
    map.unmappedCandidates = Object.keys(exported.semantic ?? {}).map((k) => 'semantic.' + k).filter((p) => !mapped.has(p));
  }
  stats.figma = { valid: map.valid.length, missing: map.missing.length, unmappedCandidates: map.unmappedCandidates.length };

  // Ratchet: app/design-system-baseline.json records today's raw-value warnings per check|file.
  // Anything above the baseline is a violation; a file below it must lower its entry (it only shrinks).
  const basePath = join(app, 'design-system-baseline.json');
  const counts = {};
  for (const w of warnings) if (w.line) counts[w.check + '|' + w.file] = (counts[w.check + '|' + w.file] ?? 0) + 1;
  stats.rawValueWarnings = Object.values(counts).reduce((x, y) => x + y, 0);
  if (existsSync(basePath)) {
    const base = JSON.parse(read(basePath));
    for (const [k, n] of Object.entries(counts)) if (n > (base[k] ?? 0)) { const [check, file] = k.split('|');
violations.push({ check: 'baseline-' + check, severity: 'major', file, detail: n + ' raw ' + check + ' value(s); baseline
allows ' + (base[k] ?? 0) + ' — use the suggested token' }); }
    for (const [k, n] of Object.entries(base)) if ((counts[k] ?? 0) < n) { const [check, file] = k.split('|');
violations.push({ check: 'baseline-stale', severity: 'minor', file, detail: check + ': baseline says ' + n + ', file has ' +
(counts[k] ?? 0) + ' — lower the entry (npm run design-system:baseline)' }); }
    stats.baselineTotal = Object.values(base).reduce((x, y) => x + y, 0);
  }
  return { ok: violations.length === 0, violations, warnings, stats, figma: map, rawCounts: counts };
}

if (import.meta.url === `file://${process.argv[1]}` || process.argv[1]?.endsWith('design-system-audit.mjs')) {
  const args = process.argv.slice(2);
  const rootArg = args.indexOf('--root');
  const result = audit(rootArg >= 0 ? { root: resolve(args[rootArg + 1]) } : {});
  if (args.includes('--write-baseline')) { const { writeFileSync } = await import('node:fs'); const p = join(rootArg >= 0 ?
resolve(args[rootArg + 1]) : resolve(here, '../..'), 'app/design-system-baseline.json'); writeFileSync(p,
JSON.stringify(Object.fromEntries(Object.entries(result.rawCounts).sort()), null, 2) + '\n'); console.log('Baseline written: '
+ p + ' (' + result.stats.rawValueWarnings + ' entries counted)'); process.exit(0); }
  if (args.includes('--json')) process.stdout.write(JSON.stringify(result, null, 2) + '\n');
  else {
    const loc = (x) => x.file + (x.line ? ':' + x.line : '');
    for (const v of result.violations) console.log('× [' + v.check + '] ' + loc(v) + (v.value ? ' ' + v.value : '') + ' — ' +
v.detail);
    for (const w of result.warnings) console.log('! [' + w.check + '] ' + loc(w) + (w.value ? ' ' + w.value : '') + ' — ' +
w.detail);
    console.log((result.ok ? '✓ ' : '× ') + result.violations.length + ' violation(s), ' + result.warnings.length + '
warning(s) · ' + JSON.stringify(result.stats));
  }
  process.exit(result.ok ? 0 : 1);
}
```

handoff/claude-code-toolkit/app/scripts/design-system-report.mjs · 66 lines

```
#!/usr/bin/env node
/**
 * A CI-readable summary of the design-system audit, as Markdown.
 * Usage (from app/): node scripts/design-system-report.mjs [--out design-system-report.md] [--root <repo-root>]
 * Exits with the audit's status so CI can gate on it.
 */
import { writeFileSync, mkdirSync } from 'node:fs';
import { resolve, join, dirname } from 'node:path';
import { spawnSync } from 'node:child_process';
import { fileURLToPath } from 'node:url';
const appDir = resolve(dirname(fileURLToPath(import.meta.url)), '..');
/** Runs the existing contract tests and records pass/fail — never weakens or rewrites them. */
export function contractStatus(files) {
  const out = {};
  for (const f of files) { const p = spawnSync('npx', ['vitest', 'run', f], { cwd: appDir, encoding: 'utf8' }); out[f] =
p.status === 0 ? 'pass' : 'fail'; }
  return out;
}
const CONTRACTS = ['src/lib/tokenexport.test.ts', 'src/styles/tokens.test.ts', 'src/styles/hex.test.ts',
'src/styles/stacking.test.ts', 'src/styles/taps.test.ts', 'src/styles/breakpoints.test.ts', 'src/styles/motion.test.ts',
'src/styles/scale.test.ts'];
import { audit } from './design-system-audit.mjs';

export function report(result, contracts = {}) {
  const s = result.stats;
  const by = (list) => list.reduce((m, x) => ((m[x.check] = (m[x.check] || 0) + 1), m), {});
  const lines = [
    '# Semester design-system report',
    '',
    '**Status:** ' + (result.ok ? '✓ passing' : '× ' + result.violations.length + ' violation(s)') + ' · ' +
result.warnings.length + ' warning(s)',
    '',
    '| Measure | Value |', '| --- | --- |',
    '| Hex literals in .ts/.tsx | ' + (s.hexLiterals ?? '—') + ' (ledger allows ' + (s.ledgerTotal ?? '—') + ') |',
    '| Custom properties defined | ' + (s.customPropertiesDefined ?? '—') + ' |',
    '| Semantic tokens exported | ' + (s.semanticTokens ?? '—') + ' |',
    '| Primitive tokens exported | ' + (s.primitiveTokens ?? '—') + ' |',
    '| Figma mappings valid / missing / unmapped candidates | ' + (s.figma ? s.figma.valid + ' / ' + s.figma.missing + ' / ' +
s.figma.unmappedCandidates : 'no mapping') + ' |',
    '| Token export (tokenexport.test.ts) | ' + (contracts['src/lib/tokenexport.test.ts'] ?? 'not run (pass --with-tests)') + '
|',
    '| Findings blocker / major / minor | ' + ['blocker', 'major', 'minor'].map((k) => [...result.violations,
...result.warnings].filter((x) => x.severity === k).length).join(' / ') + ' |',
    '',
    ...(Object.keys(contracts).length ? ['## Existing style contracts', '', '| Test | Result |', '| --- | --- |',
...Object.entries(contracts).map(([f, st]) => '| ' + f + ' | ' + st + ' |'), ''] : []),
    '',
    '## Violations by check', '',
    ...(result.violations.length ? Object.entries(by(result.violations)).map(([k, n]) => '- ' + k + ': ' + n) : ['None.']),
    '',
    ...(result.violations.length ? ['| Check | File | Detail |', '| --- | --- | --- |', ...result.violations.map((v) => '| ' +
v.check + ' | ' + v.file + (v.line ? ':' + v.line : '') + ' | ' + (v.value ?? '') + ' ' + v.detail + ' |'), ''] : []),
    '## Warnings', '',
    ...(result.warnings.length ? ['| Check | File | Detail |', '| --- | --- | --- |', ...result.warnings.slice(0, 200).map((w)
=> '| ' + w.check + ' | ' + w.file + (w.line ? ':' + w.line : '') + ' | ' + (w.value ?? '') + ' ' + w.detail + ' |')] :
['None.']),
    '',
  ];
  return lines.join('\n');
}

if (process.argv[1]?.endsWith('design-system-report.mjs')) {
  const args = process.argv.slice(2);
  const r = args.indexOf('--root');
  const result = audit(r >= 0 ? { root: resolve(args[r + 1]) } : {});
  const contracts = args.includes('--with-tests') ? contractStatus(CONTRACTS) : {};
  const md = report(result, contracts);
  const o = args.indexOf('--out');
  const out = o >= 0 ? args[o + 1] : join(appDir, 'reports/design-system/report.md');
  mkdirSync(dirname(out), { recursive: true });
  writeFileSync(out, md);
  writeFileSync(out.replace(/\.md$/, '.json'), JSON.stringify({ ...result, contracts }, null, 2) + '\n');
  console.log('Wrote ' + out);
  if (Object.values(contracts).includes('fail')) process.exit(1);
```

```
  process.exit(result.ok ? 0 : 1);
}
```

handoff/claude-code-toolkit/app/src/lib/designsystem.test.ts · 116 lines

```
import { describe, expect, it } from 'vitest';
import { mkdtempSync, mkdirSync, writeFileSync, readFileSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';
// @ts-expect-error — plain ESM script, no types
import { audit, mappingPaths, readLedger, semanticNames } from '../../scripts/design-system-audit.mjs';
// @ts-expect-error — plain ESM script, no types
import { report } from '../../scripts/design-system-report.mjs';
import { buildTokenExport, serialise } from './tokenexport';

/**
 * The design-system audit is a guard only if it has failed at least once (CLAUDE.md).
 * Each check is proven against a fixture that violates it, and the real tree is held clean.
 */

const repo = new URL('../../..', import.meta.url).pathname;

function fixture(files: Record<string, string>) {
  const root = mkdtempSync(join(tmpdir(), 'semester-ds-'));
  for (const [p, text] of Object.entries(files)) {
    const full = join(root, p);
    mkdirSync(join(full, '..'), { recursive: true });
    writeFileSync(full, text);
  }
  return root;
}

const TOKENS = ':root {\n  --surface-base: var(--app-panel);\n  --text-primary: var(--app-fg);\n}\n';
const LOOK = "export const x = { '--app-panel': '#12141a', '--app-fg': '#eceef2' };\n";
const EXPORT = JSON.stringify({ primitive: { 'app-panel': {}, 'app-fg': {} }, semantic: { 'surface-base': {}, 'text-primary':
{} } });
const HEXTEST = "export const LEDGER = { 'components/Ok.tsx': { count: 1, why: 'x' } };\n";
const base = {
  'app/src/styles/tokens.css': TOKENS,
  'app/src/lib/look.ts': LOOK,
  'app/src/styles/hex.test.ts': HEXTEST,
  'app/design-tokens/semester.tokens.json': EXPORT,
  'app/src/components/Ok.tsx': "export const C = () => <div style={{ color: '#fff', background: 'var(--surface-base)' }}
/>;\n",
  'docs/design-system/FIGMA-MAPPING.md': '| semantic.surface-base | surface/base |\n| primitive.app-fg | base/fg |\n',
};

describe('the committed export is generated, not edited', () => {
  it('equals buildTokenExport of the current stylesheets', () => {
    const tokensCss = readFileSync(join(repo, 'app/src/styles/tokens.css'), 'utf8');
    const appCss = readFileSync(join(repo, 'app/src/styles/app.css'), 'utf8');
    const committed = readFileSync(join(repo, 'app/design-tokens/semester.tokens.json'), 'utf8');
    expect(committed).toBe(serialise(buildTokenExport(tokensCss, appCss)));
  });
});

describe('design-system audit', () => {
  it('passes a valid fixture', () => {
    expect(audit({ root: fixture(base) }).violations).toEqual([]);
  });

  it('reports file, line and value for a violation', () => {
    const r = audit({ root: fixture({ ...base, 'app/src/components/Bad.tsx': "// one\nexport const B = { color: '#ff0000' };\n"
}) });
    expect(r.violations[0]).toMatchObject({ file: 'components/Bad.tsx', line: 2, value: '#ff0000' });
  });

  it('separates valid and missing Figma mappings', () => {
    const r = audit({ root: fixture({ ...base, 'docs/design-system/FIGMA-MAPPING.md': '| semantic.surface-base |\n|
semantic.nope |\n' }) });
    expect(r.figma.valid).toEqual(['semantic.surface-base']);
    expect(r.figma.missing).toEqual(['semantic.nope']);
    expect(r.figma.unmappedCandidates).toEqual(['semantic.text-primary']);
  });
```

```
  it('fails a hex literal beyond the hex.test.ts ledger', () => {
    const r = audit({ root: fixture({ ...base, 'app/src/components/Bad.tsx': "export const B = { color: '#ff0000' };\n" }) });
    expect(r.ok).toBe(false);
    expect(r.violations.map((v: { check: string }) => v.check)).toContain('raw-colour');
  });

  it('fails a custom property used without a fallback and defined nowhere', () => {
    const r = audit({ root: fixture({ ...base, 'app/src/styles/x.css': '.a { color: var(--nope); }\n' }) });
    expect(r.violations.some((v: { detail: string }) => v.detail.startsWith('--nope'))).toBe(true);
  });

  it('allows an undefined property when a fallback is given', () => {
    const r = audit({ root: fixture({ ...base, 'app/src/styles/x.css': '.a { color: var(--nope, red); }\n' }) });
    expect(r.violations.filter((v: { check: string }) => v.check === 'undefined-var')).toEqual([]);
  });

  it('fails when tokens.css and the export disagree', () => {
    const r = audit({ root: fixture({ ...base, 'app/src/styles/tokens.css': TOKENS.replace('}', '  --surface-new: var(--app-
panel);\n}') }) });
    expect(r.violations.map((v: { check: string }) => v.check)).toContain('export-sync');
  });

  it('fails a Figma mapping row that does not resolve', () => {
    const r = audit({ root: fixture({ ...base, 'docs/design-system/FIGMA-MAPPING.md': '| semantic.surface-gone | surface/gone
|\n' }) });
    expect(r.violations.map((v: { check: string }) => v.check)).toContain('figma-mapping');
  });

  it('fails a raw value above the baseline and passes at it', () => {
    const files = { ...base, 'app/src/components/Z.tsx': "export const Z = () => <div style={{ zIndex: 80 }} />;\n" };
    expect(audit({ root: fixture({ ...files, 'app/design-system-baseline.json': '{}' }) }).violations.map((v: { check: string
}) => v.check)).toContain('baseline-z-index');
    expect(audit({ root: fixture({ ...files, 'app/design-system-baseline.json': JSON.stringify({ 'z-index|components/Z.tsx': 1
}) }) }).ok).toBe(true);
  });

  it('flags a stale baseline so it can only shrink', () => {
    const r = audit({ root: fixture({ ...base, 'app/design-system-baseline.json': JSON.stringify({ 'z-
index|components/Gone.tsx': 3 }) }) });
    expect(r.violations.map((v: { check: string }) => v.check)).toContain('baseline-stale');
  });

  it('reads the real ledger and the real mapping', () => {
    expect(Object.keys(readLedger(readFileSync(join(repo, 'app/src/styles/hex.test.ts'), 'utf8'))).length).toBeGreaterThan(0);
    expect(semanticNames(readFileSync(join(repo, 'app/src/styles/tokens.css'), 'utf8'))).toContain('surface-base');
    expect(mappingPaths(readFileSync(join(repo, 'docs/design-system/FIGMA-MAPPING.md'), 'utf8')).length).toBeGreaterThan(10);
  });

  it('holds the real tree clean, and the report says so', () => {
    const r = audit({ root: repo });
    expect(r.violations, report(r)).toEqual([]);
    expect(report(r)).toContain('✓ passing');
  });
});
```

handoff/claude-code-toolkit/gitignore.addition · 3 lines

```
# Generated design-system reports (uploaded as CI artifacts, not committed)
app/reports/
```

handoff/install.sh · 21 lines

```
#!/usr/bin/env bash
# Install the Semester handoff into the repo. Run from the repo root: bash design/handoff/install.sh
# Safe by default: never overwrites an existing file unless FORCE=1. Strips the .txt suffix design files carry.
set -euo pipefail
SRC="${SRC:-design/handoff}"; DEST="${DEST:-.}"; APP="${APP:-app}"
[ -d "$SRC" ] || { echo "Missing $SRC. Copy the design project export to ./design first."; exit 1; }
copy() { local from="$1" to="$2"; mkdir -p "$(dirname "$to")"; if [ -e "$to" ] && [ "${FORCE:-0}" != 1 ]; then echo "SKIP
(exists) $to"; else cp "$from" "$to"; echo "OK   $to"; fi; }
strip() { local p="$1"; echo "${p%.txt}"; }
# 1 semester-core → $APP/src/lib/core
for f in "$SRC"/semester-core/*.ts.txt; do copy "$f" "$DEST/$APP/src/lib/core/$(basename "$(strip "$f")")"; done
```

```
# 2 semester-platform: migrations, lib, api routes, tests, middleware, seed
for f in "$SRC"/semester-platform/supabase/migrations/*.sql.txt; do copy "$f" "$DEST/supabase/migrations/$(basename "$(strip
"$f")")"; done
for f in $(cd "$SRC/semester-platform" && find lib app tests -name '*.txt'); do copy "$SRC/semester-platform/$f"
"$DEST/$APP/$(strip "$f")"; done
copy "$SRC/semester-platform/middleware.security.ts.txt" "$DEST/$APP/middleware.security.ts"
copy "$SRC/semester-platform/supabase/seed.sql.txt" "$DEST/supabase/seed.sql"
copy "$SRC/semester-platform/tests/helpers.ts.txt" "$DEST/$APP/tests/helpers.ts"
copy "$SRC/semester-platform/.env.test.txt" "$DEST/$APP/.env.test"
copy "$SRC/semester-platform/.env.example.txt" "$DEST/$APP/.env.example"
copy "$SRC/semester-platform/.github/ci-security-job.yml.txt" "$DEST/.github/ci-security-job.yml"
echo; echo "Installed. Next: bash design/handoff/preflight.sh && bash design/handoff/verify.sh"
```

handoff/preflight.sh · 13 lines

```
#!/usr/bin/env bash
# Preflight: confirms tools and project shape before any stream runs. Exits non-zero with the exact fix.
set -uo pipefail; fail=0
need() { command -v "$1" >/dev/null 2>&1 || { echo "MISSING $1 — $2"; fail=1; }; }
need node "install Node 20+"; need npm "comes with Node"; need npx "comes with Node"; need git "install git"; need docker
"Supabase local needs Docker running"
node -e 'process.exit(+process.versions.node.split(".")[0] < 20)' || { echo "Node 20+ required"; fail=1; }
npx --yes supabase --version >/dev/null 2>&1 || { echo "MISSING supabase CLI — npx supabase works once network is available";
fail=1; }
[ -d supabase ] || { echo "No supabase/ — run: npx supabase init"; fail=1; }
[ -f "${APP:-app}/package.json" ] || { echo "No ${APP:-app}/package.json — set APP=<dir> to the web app"; fail=1; }
grep -q '"next"' "${APP:-app}/package.json" 2>/dev/null || echo "NOTE  ${APP:-app} is not a Next.js app. API routes in
app/api/* need a Next.js app (see BUILD.md §2 'App target')."
git diff --quiet || echo "NOTE  working tree has uncommitted changes; streams expect a clean branch"
[ $fail = 0 ] && echo "PREFLIGHT OK" || { echo "PREFLIGHT FAILED"; exit 1; }
```

handoff/verify.sh · 14 lines

```
#!/usr/bin/env bash
# Every gate, in order. Stops at the first failure and prints which gate failed.
set -euo pipefail; APP="${APP:-app}"
step() { echo; echo "== $1"; }
step "1 database reset (migrations 010–080 + seed)"; npx supabase db reset
step "2 typecheck"; (cd "$APP" && npx tsc --noEmit)
step "3 lint"; (cd "$APP" && npm run lint)
step "4 unit tests (semester-core, validate, AI red-team)"; (cd "$APP" && npx vitest run src/lib/core tests/validate.test.ts
tests/ai-redteam.test.ts)
step "5 RLS + security + command tests"; (cd "$APP" && npx dotenv -e .env.test -- npx vitest run tests/rls.test.ts
tests/security.test.ts tests/commands.test.ts)
step "6 design-system check"; (cd "$APP" && npm run design-system:check)
step "7 build"; (cd "$APP" && npm run build)
step "8 secret scan"; npx --yes gitleaks detect --no-banner || { echo "gitleaks missing or found secrets"; exit 1; }
echo; echo "ALL GATES PASSED — record this output in docs/execute/<stream>-report.md"
```

handoff/manifest.json · 93 lines

```
{
  "version": "2026-10-05",
  "entry": "handoff/BUILD.md",
  "scripts": [
    "handoff/preflight.sh",
    "handoff/install.sh",
    "handoff/verify.sh"
  ],
  "migrationOrder": [
    "010_platform_core",
    "020_community",
    "030_marketplace",
    "040_ops",
    "050_security_hardening",
    "060_command_functions",
    "070_ai_gateway",
    "080_sources_storage_delegations"
  ],
  "files": [
    {
```

```
      "from": "semester-core/*.ts.txt",
      "to": "<APP>/src/lib/core/",
      "check": "unit tests"
    },
    {
      "from": "semester-platform/supabase/migrations/010–080",
      "to": "supabase/migrations/",
      "check": "db reset + RLS/security/command tests"
    },
    {
      "from": "semester-platform/supabase/seed.sql.txt",
      "to": "supabase/seed.sql",
      "check": "db reset"
    },
    {
      "from": "semester-platform/lib/**",
      "to": "<APP>/lib/",
      "check": "typecheck"
    },
    {
      "from": "semester-platform/app/api/**",
      "to": "<APP>/app/api/",
      "check": "typecheck + build"
    },
    {
      "from": "semester-platform/tests/**",
      "to": "<APP>/tests/",
      "check": "vitest"
    },
    {
      "from": "semester-platform/middleware.security.ts.txt",
      "to": "<APP>/middleware.security.ts (merge into middleware.ts)",
      "check": "build; headers present"
    },
    {
      "from": "semester-platform/.env.example.txt, .env.test.txt",
      "to": "<APP>/",
      "check": "—"
    },
    {
      "from": "semester-platform/.github/ci-security-job.yml.txt",
      "to": ".github/ (merge into ci.yml)",
      "check": "CI"
    },
    {
      "from": "claude-code-toolkit/**",
      "to": "repo root per its EXECUTE.md",
      "check": "design-system:check"
    }
  ],
  "pilotCriticalPath": [
    "00",
    "01",
    "02",
    "03",
    "06",
    "05",
    "09",
    "10"
  ],
  "humanOnly": [
    "insurance",
    "DPA",
    "HECVAT",
    "attorney review",
    "IdP metadata",
    "contacts",
    "baseline approval",
    "PR approvals",
    "cutover approval",
    "AI provider terms"
  ]
}
```

APPENDIX

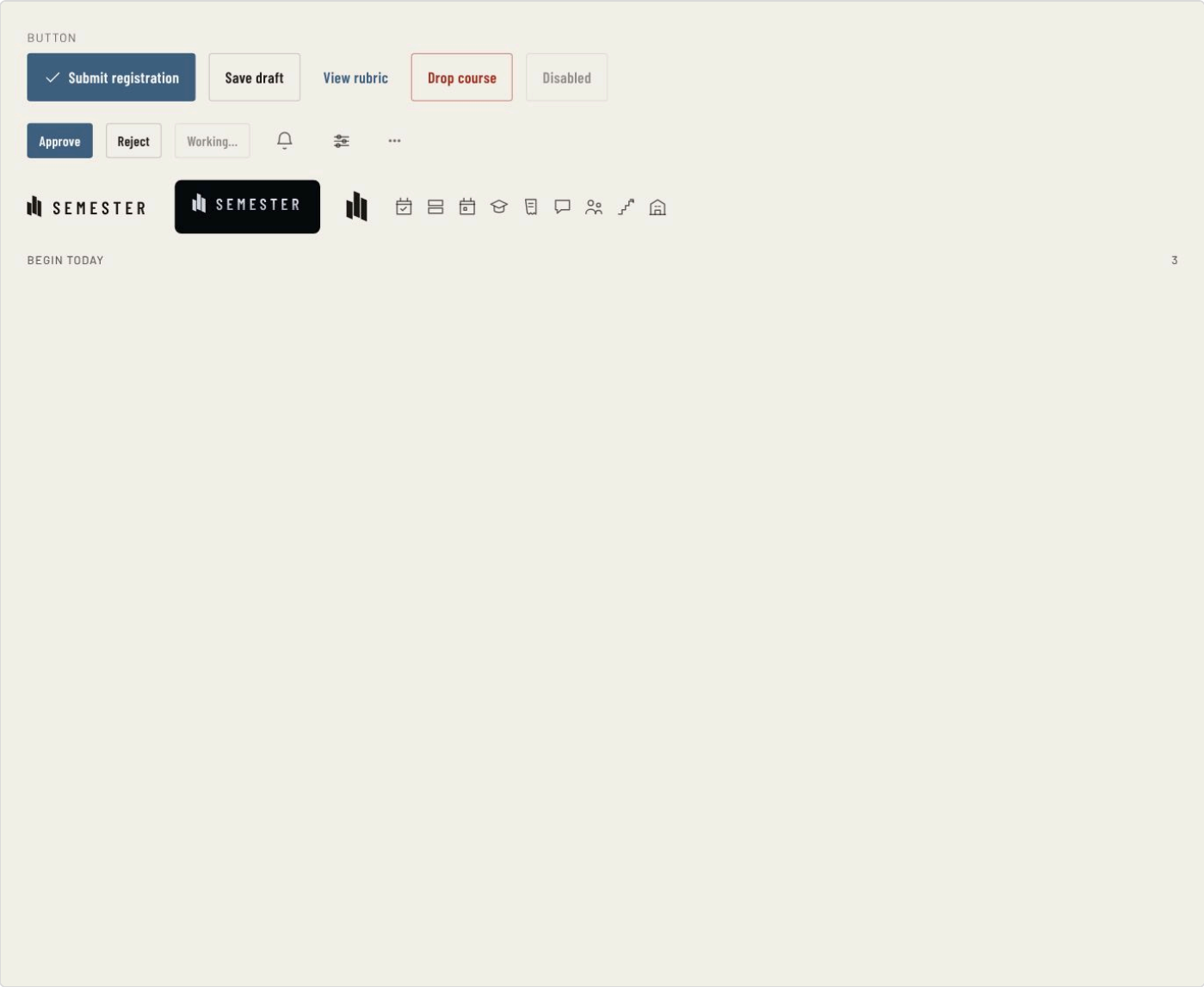
Archive

Everything moved aside during the migration, still rendered as it was.

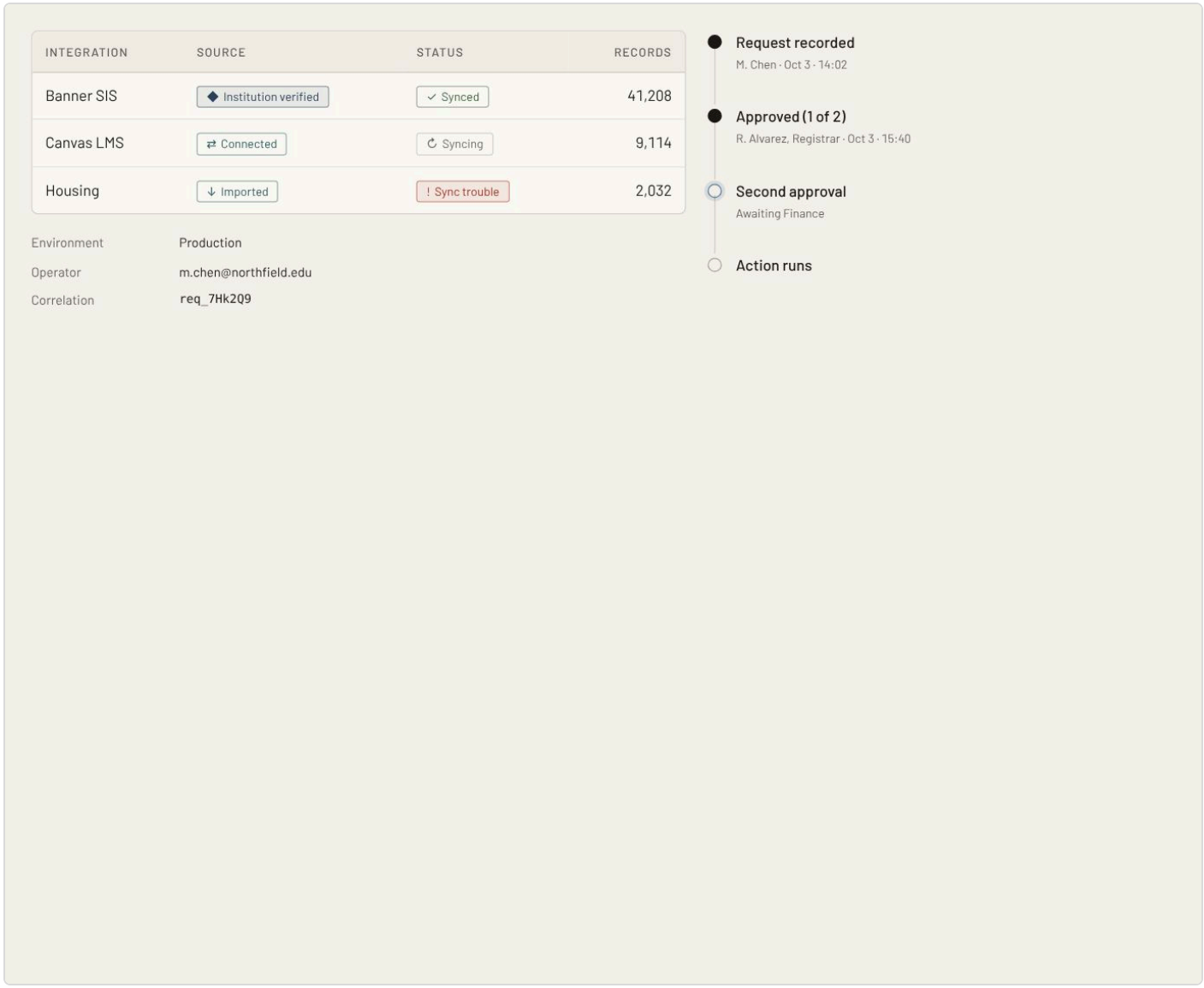
Archive: the pre-migration cards and specimens

The original group cards and specimen pages, as they rendered before the system moved to the new format. They are kept in archived/.

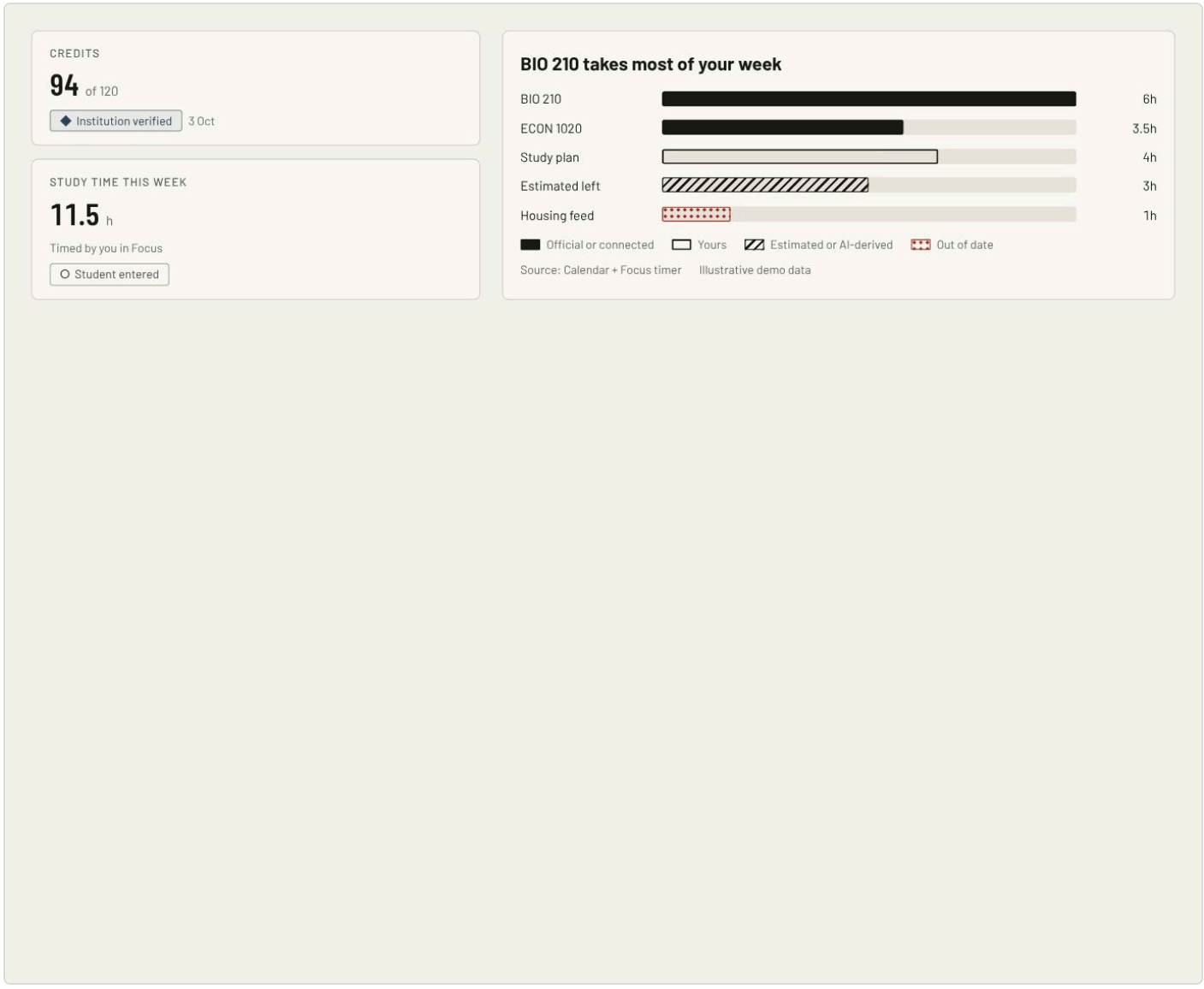
Group card: core



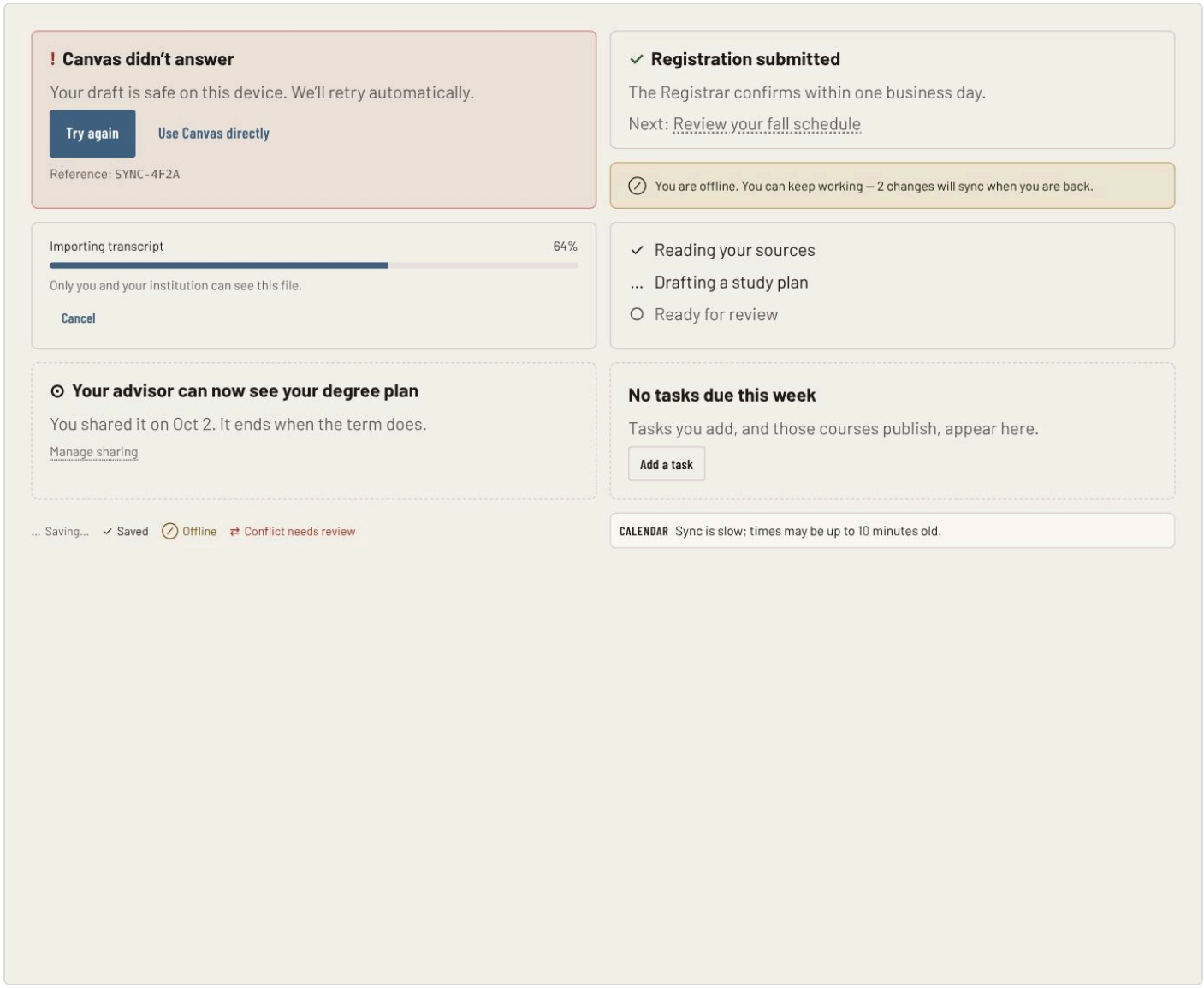
Group card: data



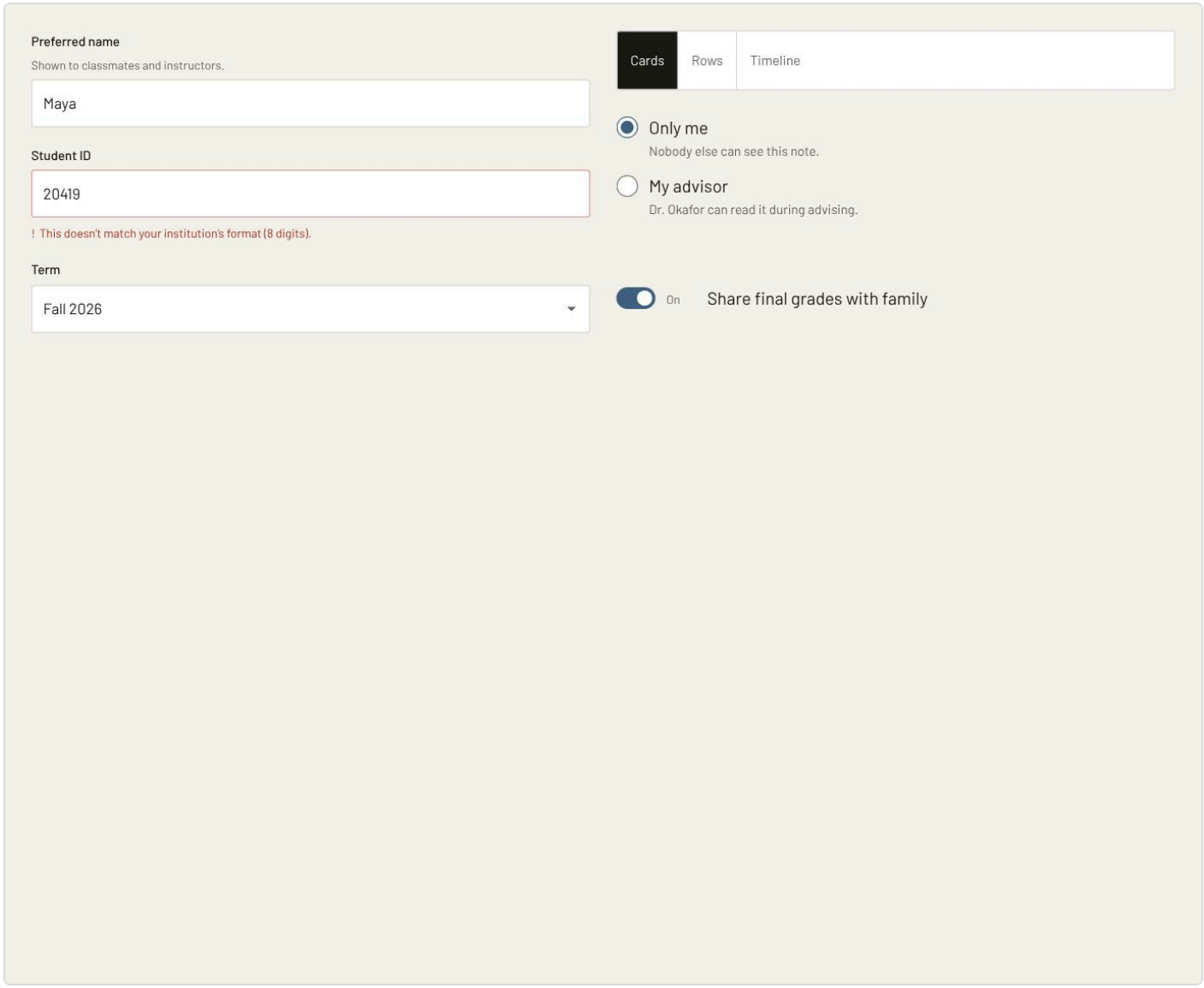
Group card: data-viz



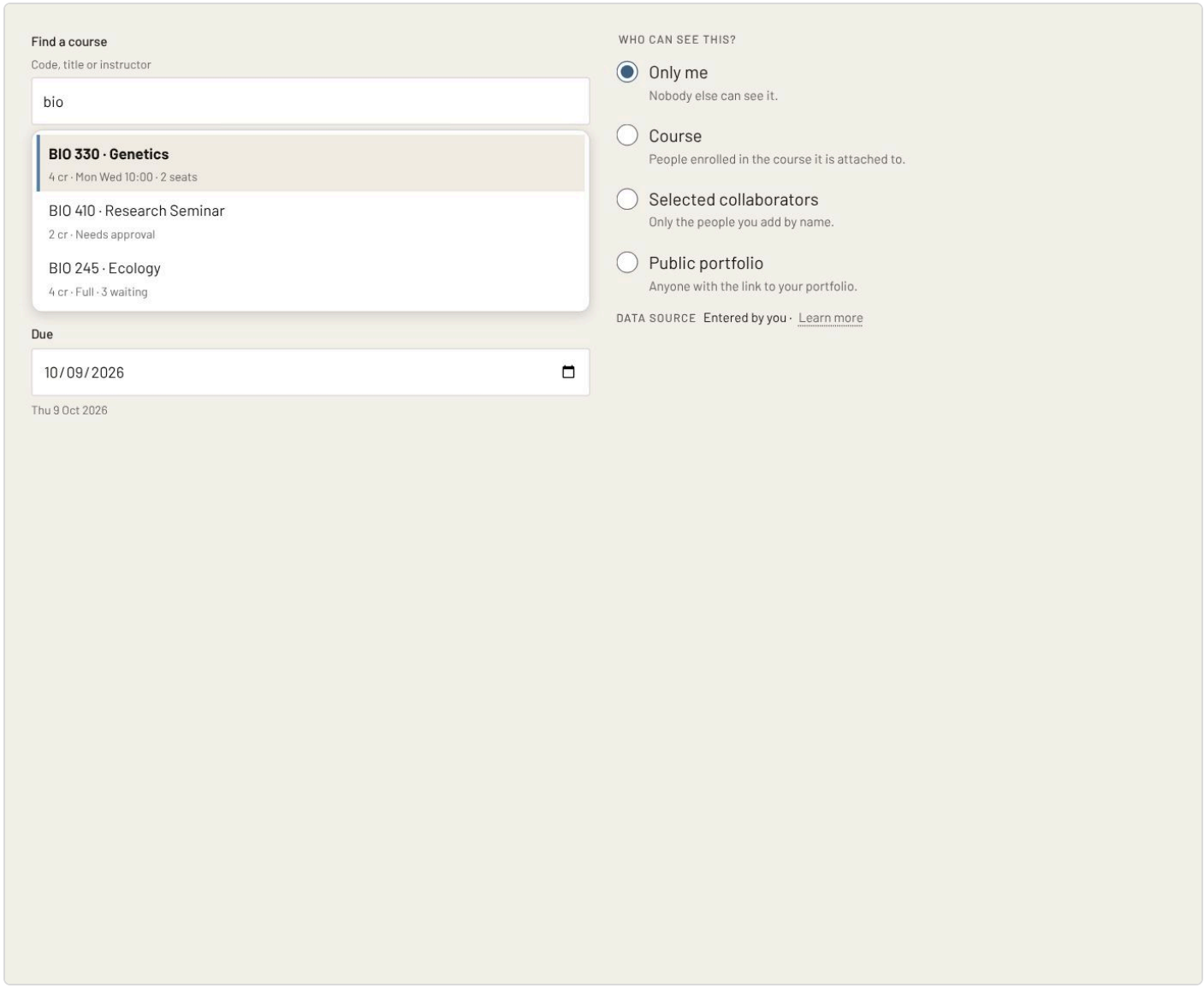
Group card: feedback



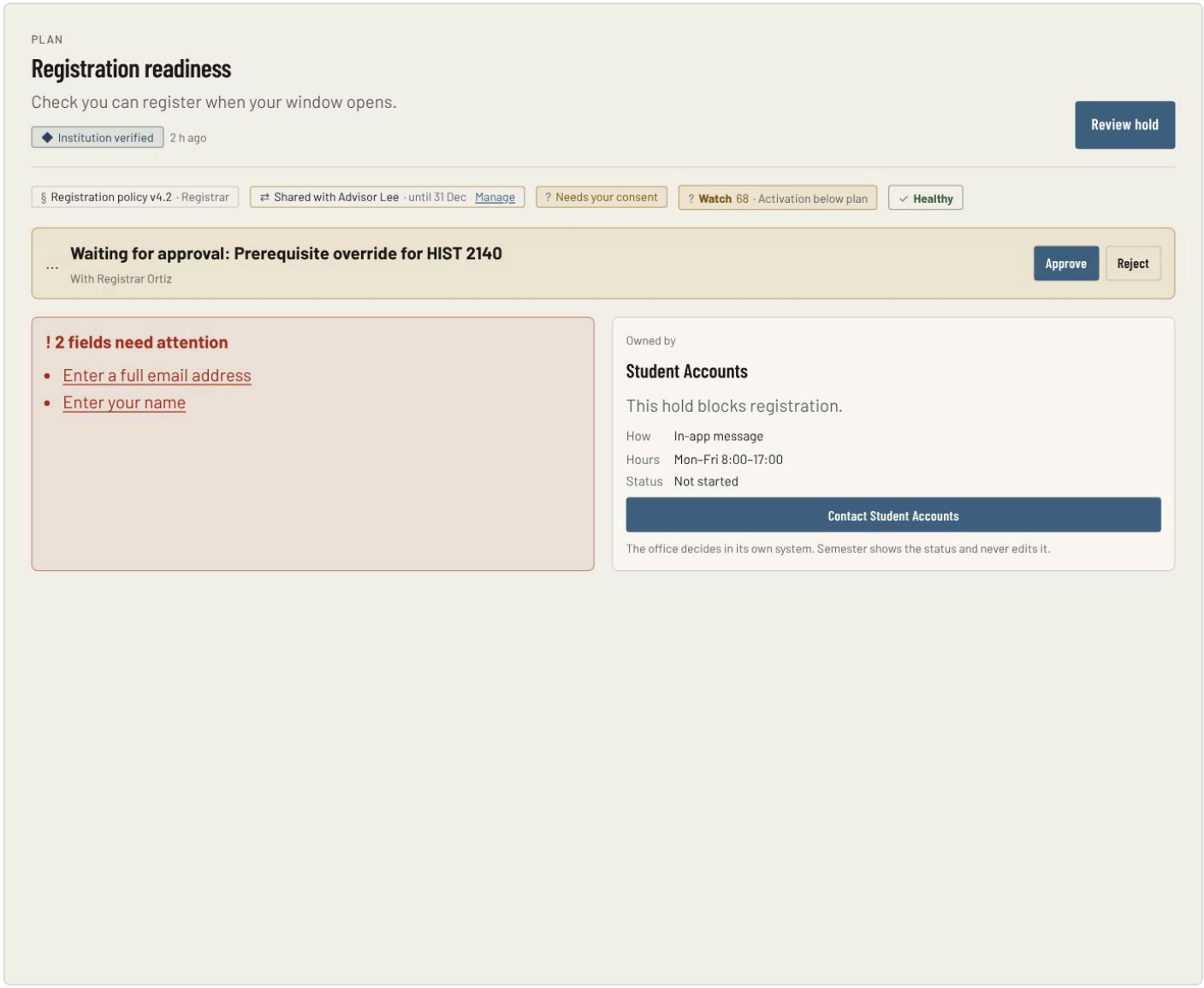
Group card: forms



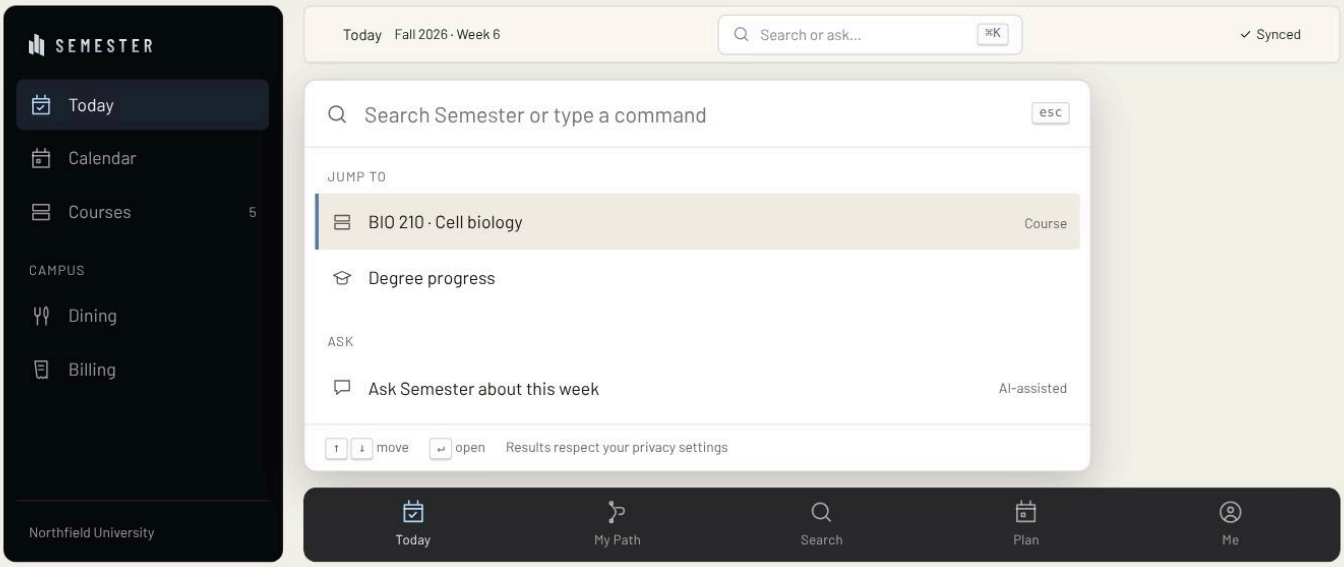
Group card: forms-advanced



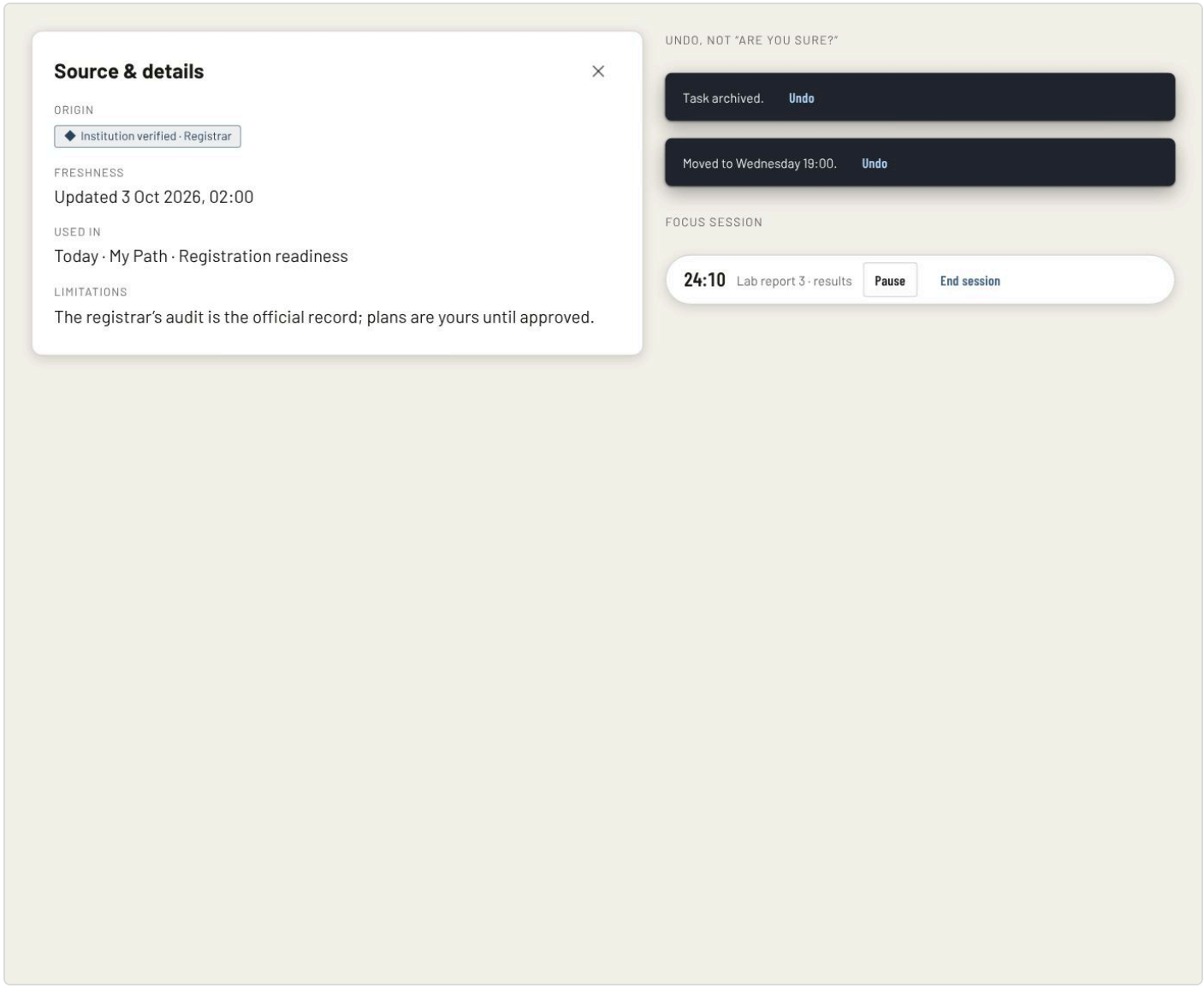
Group card: governance



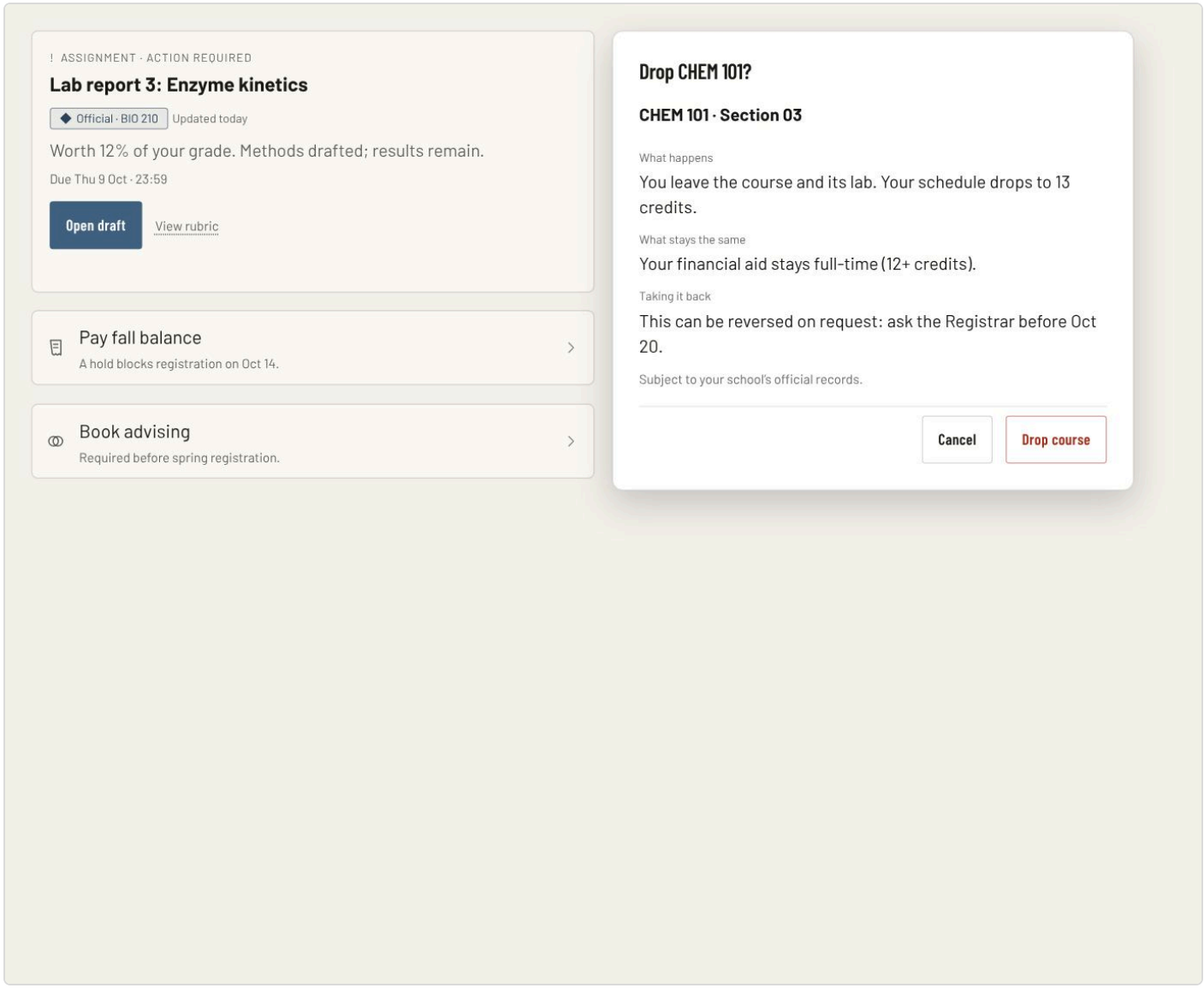
Group card: navigation



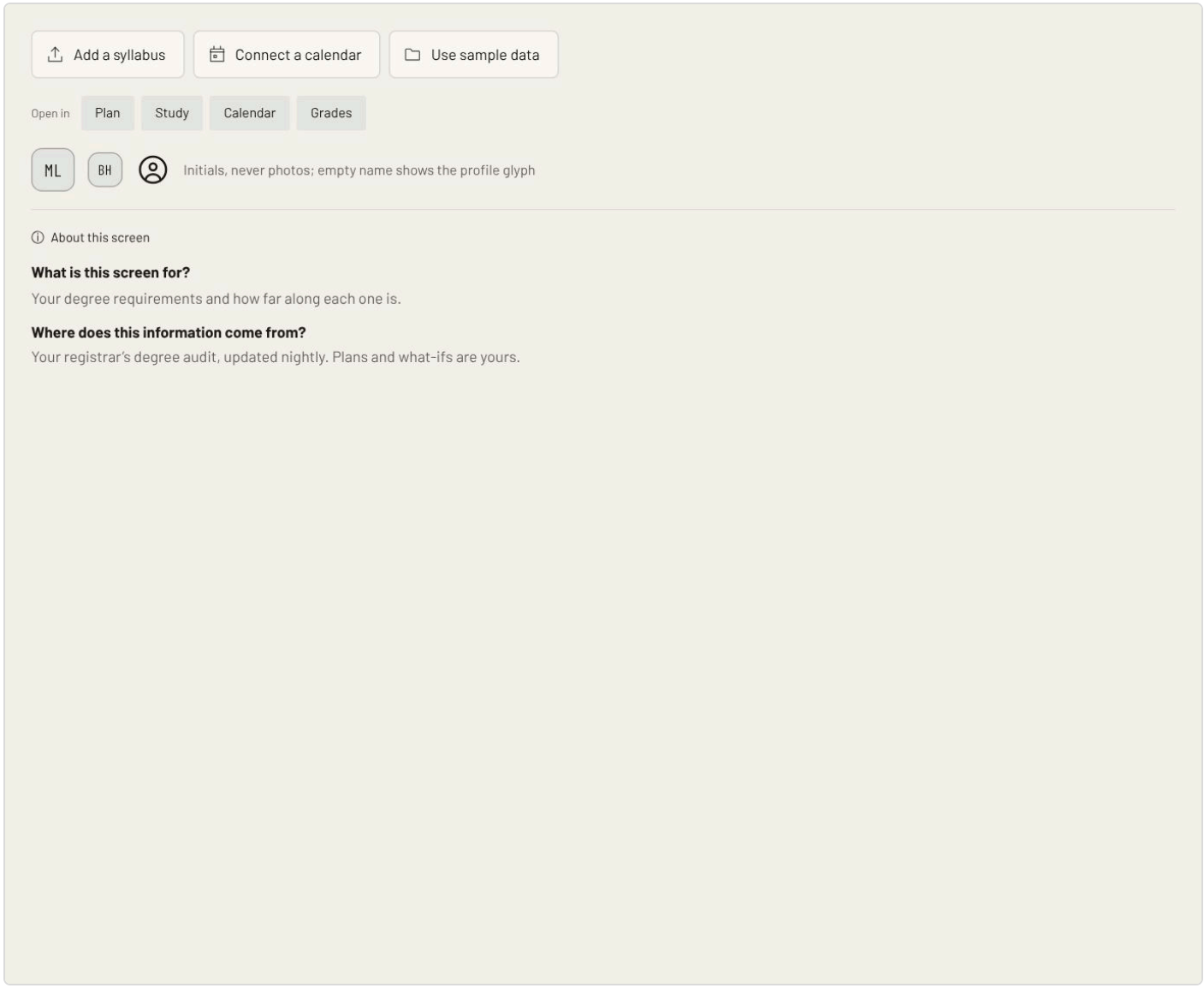
Group card: overlays



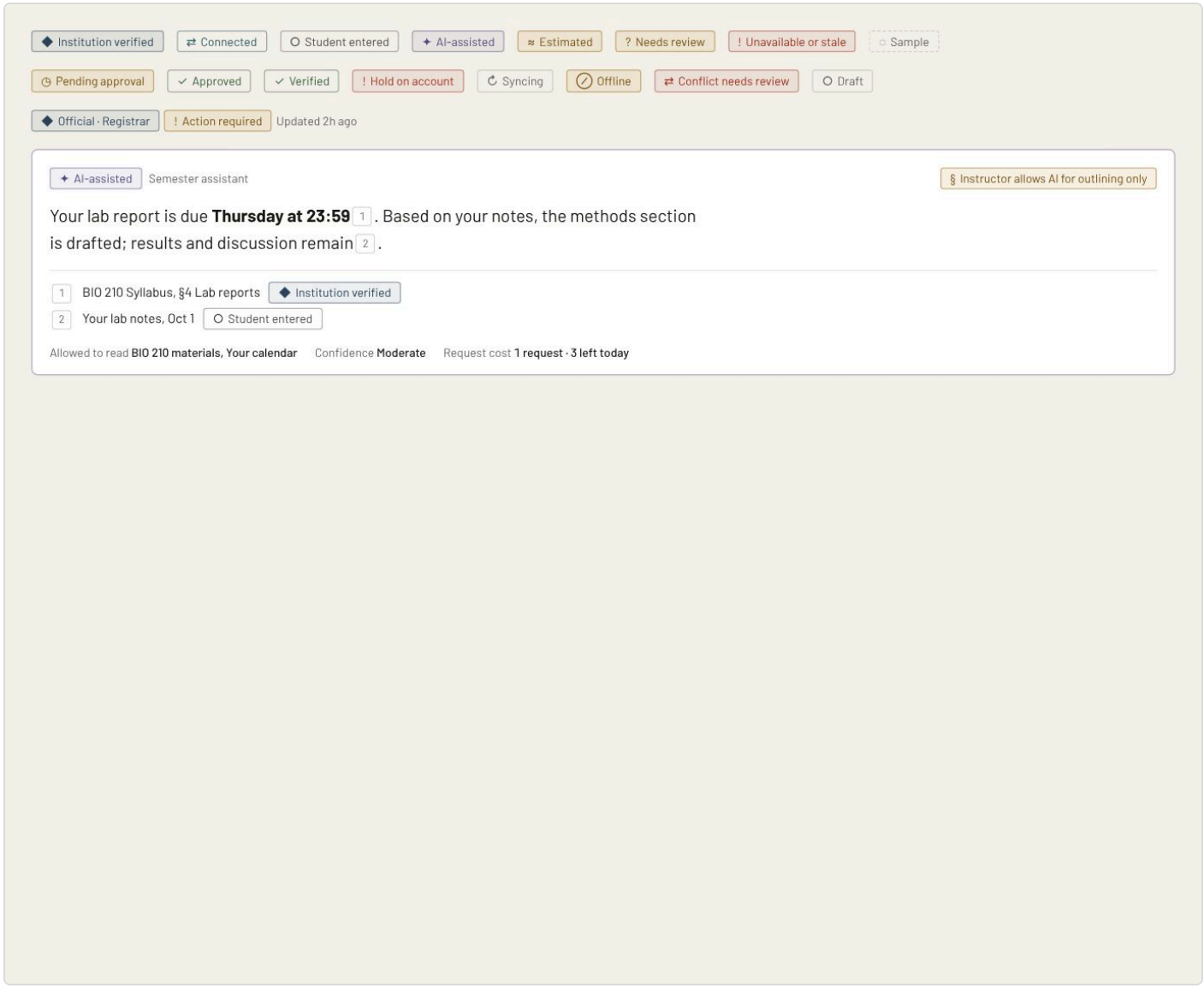
Group card: surfaces



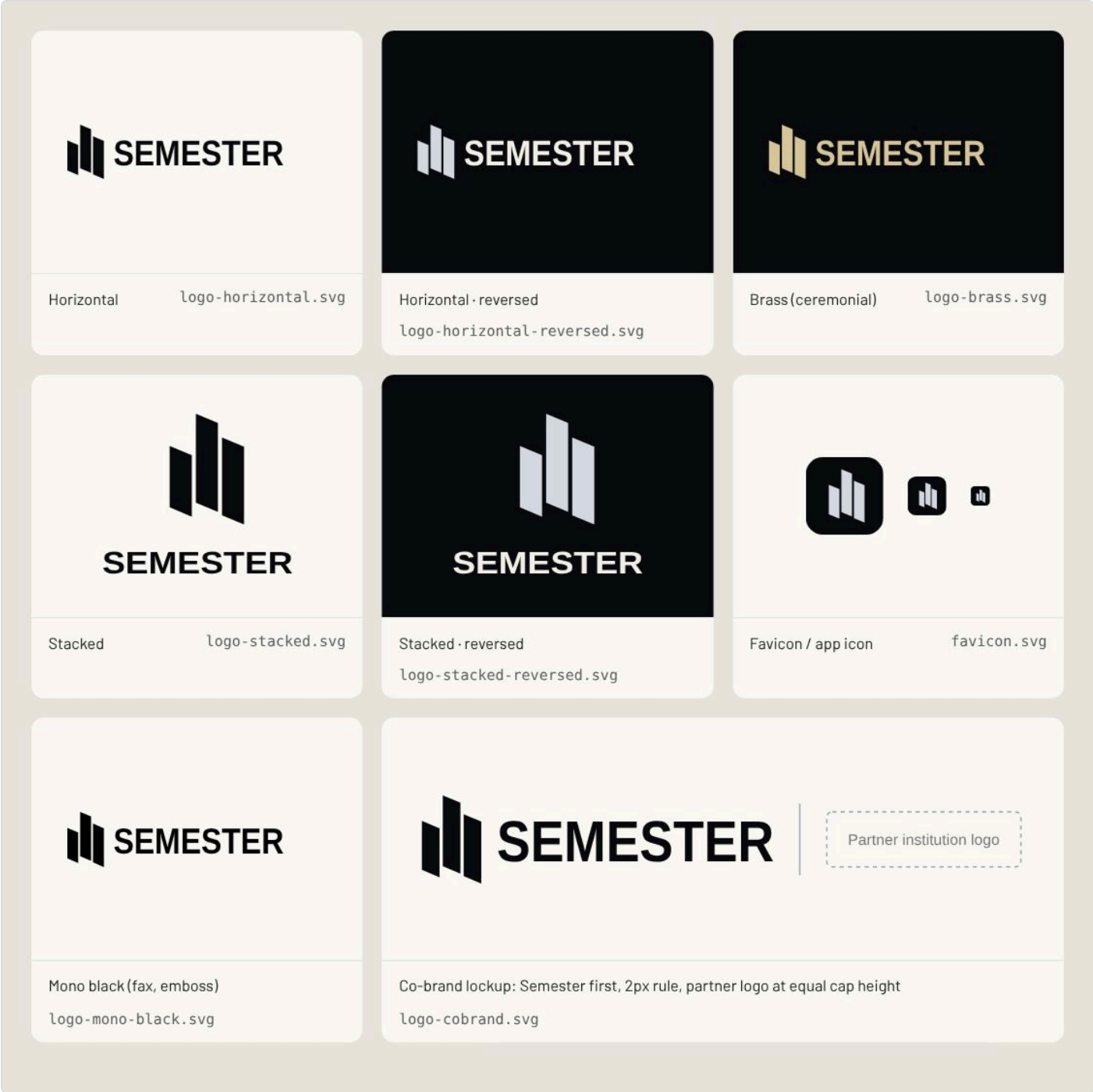
Group card: surfaces-more



Group card: trust



Specimen: brand-logos



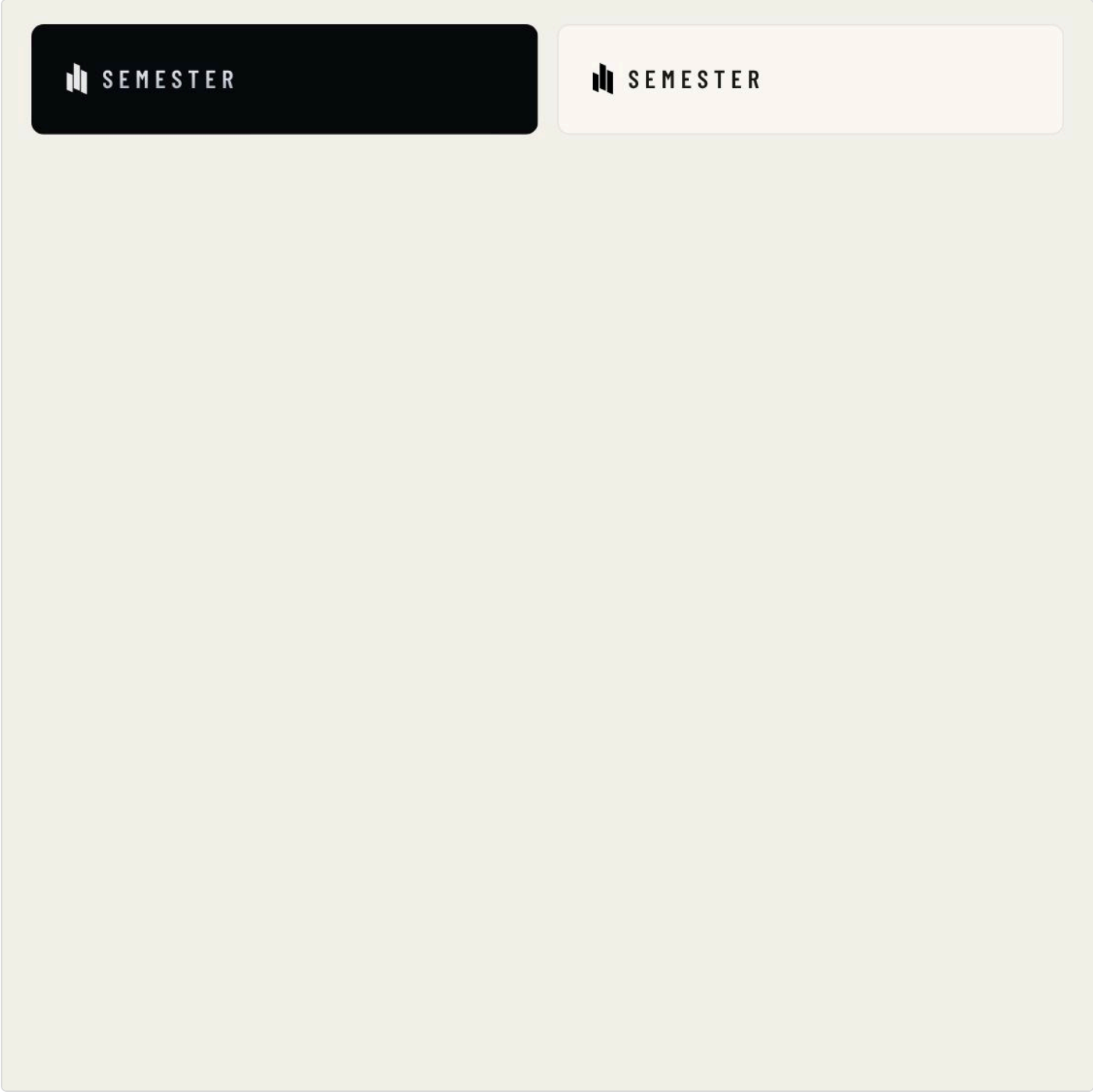
Specimen: brand-mark





Lit silver on Ink for app icons and splash; flat currentColor everywhere in-app so it follows the ground. Never recolour into the accent, add glow, or place on photography.

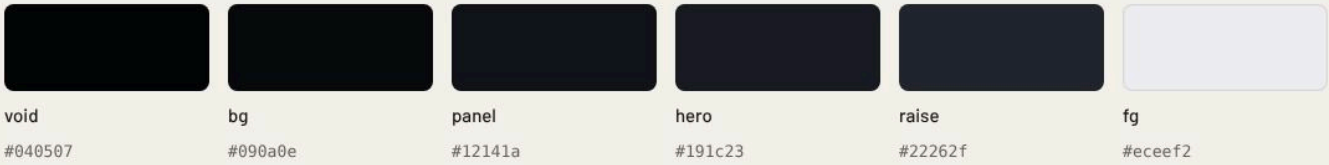
Specimen: brand-wordmark



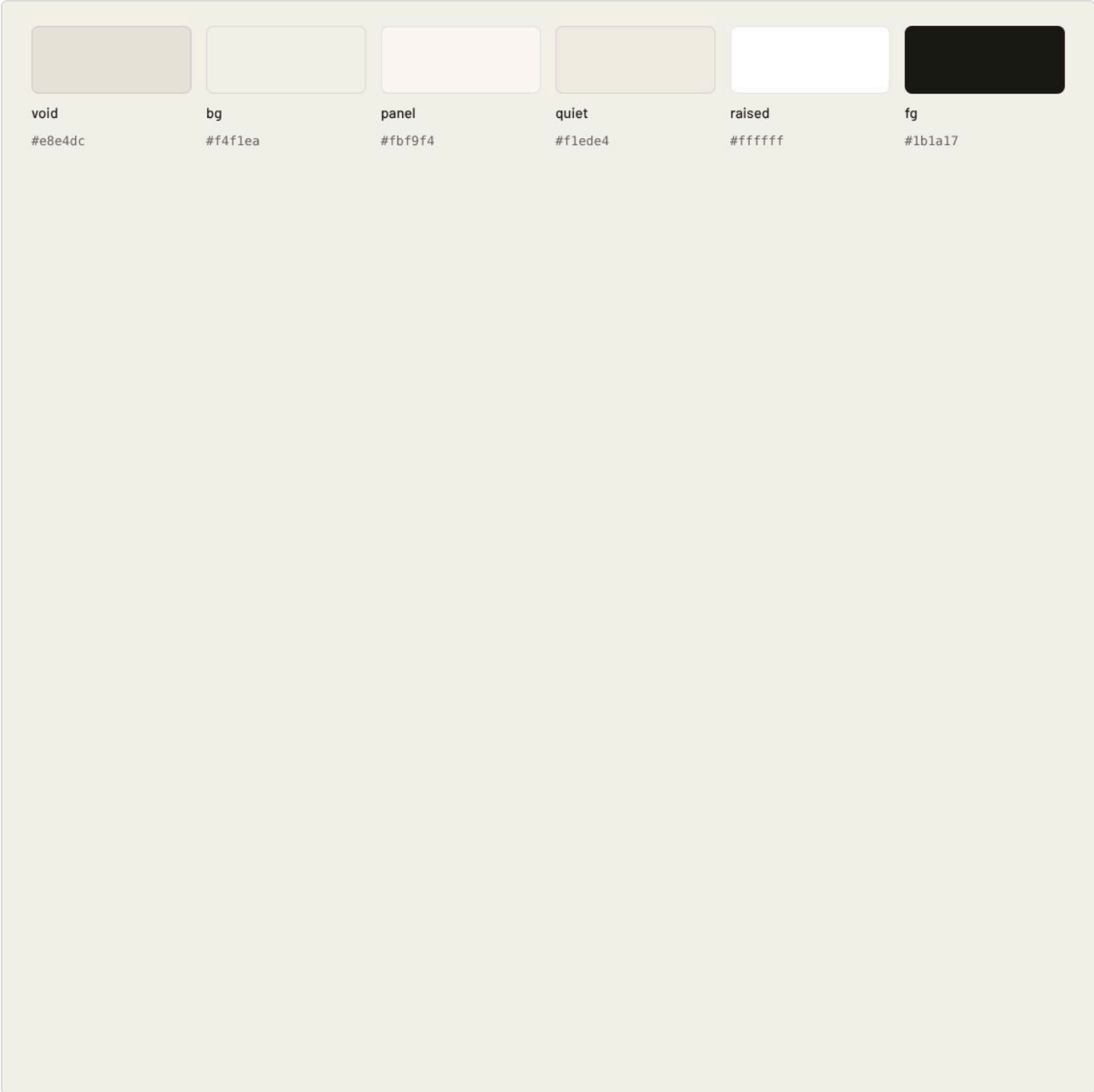
Specimen: colors-blue



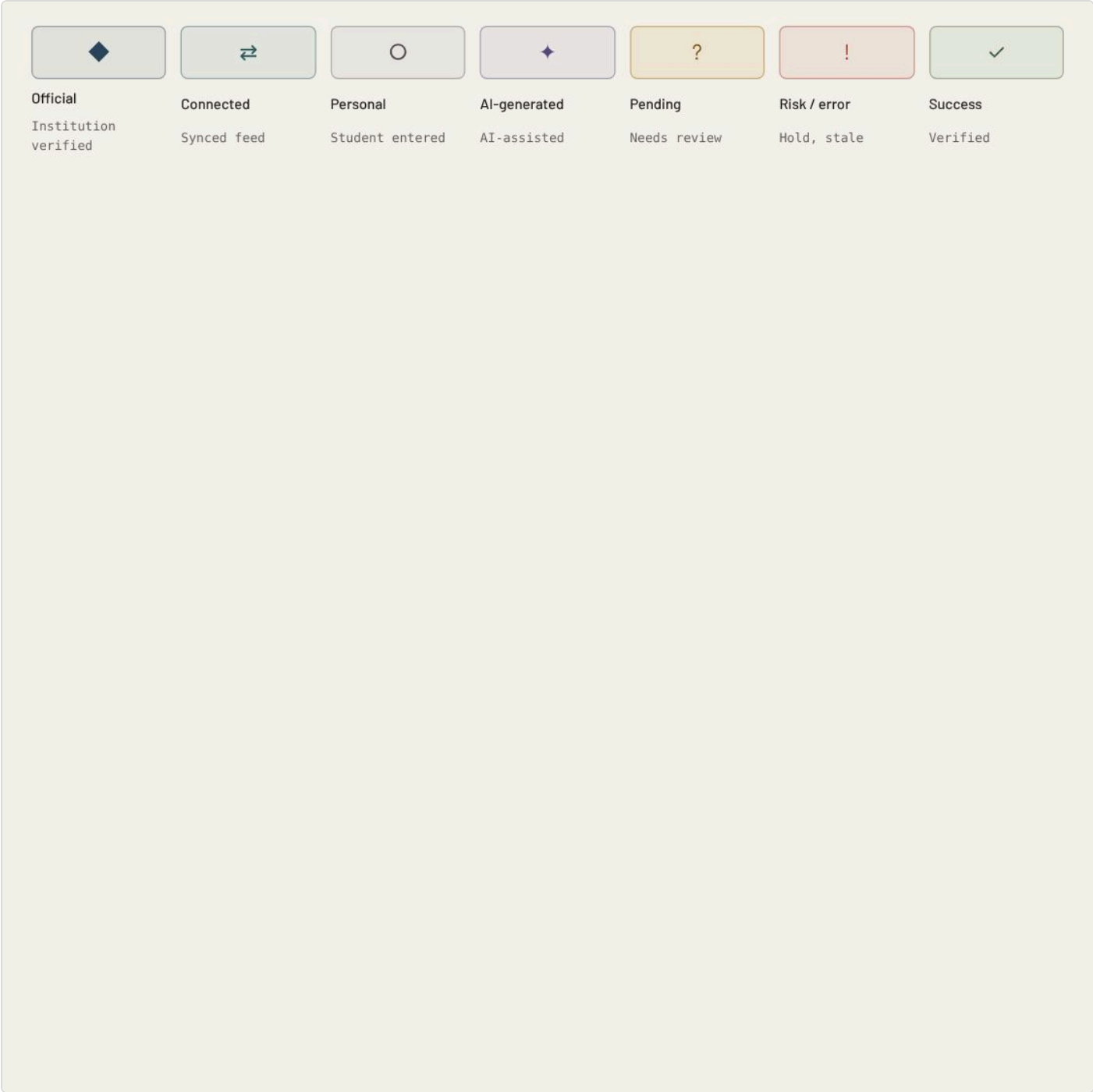
Specimen: colors-ink



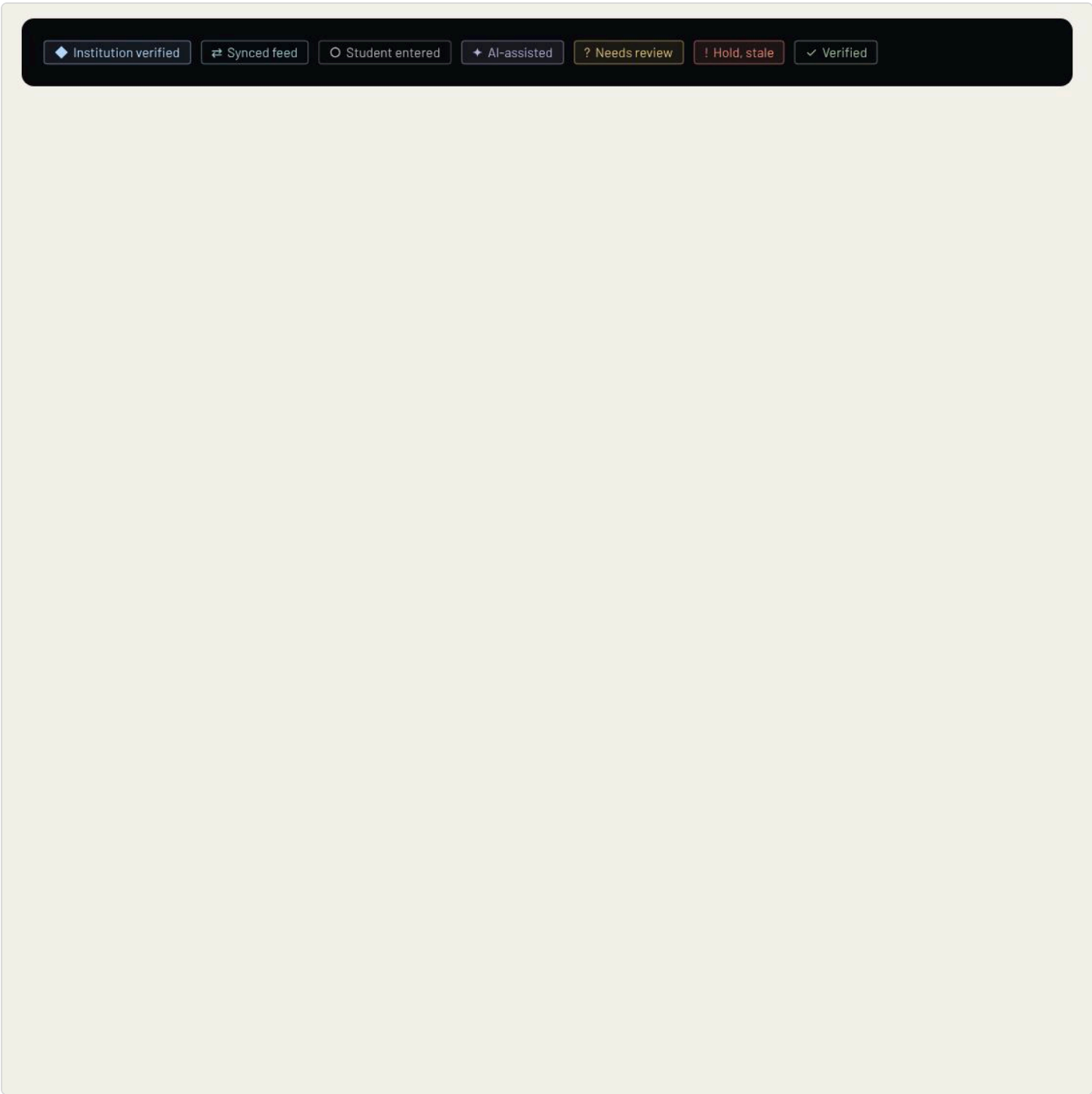
Specimen: colors-parchment



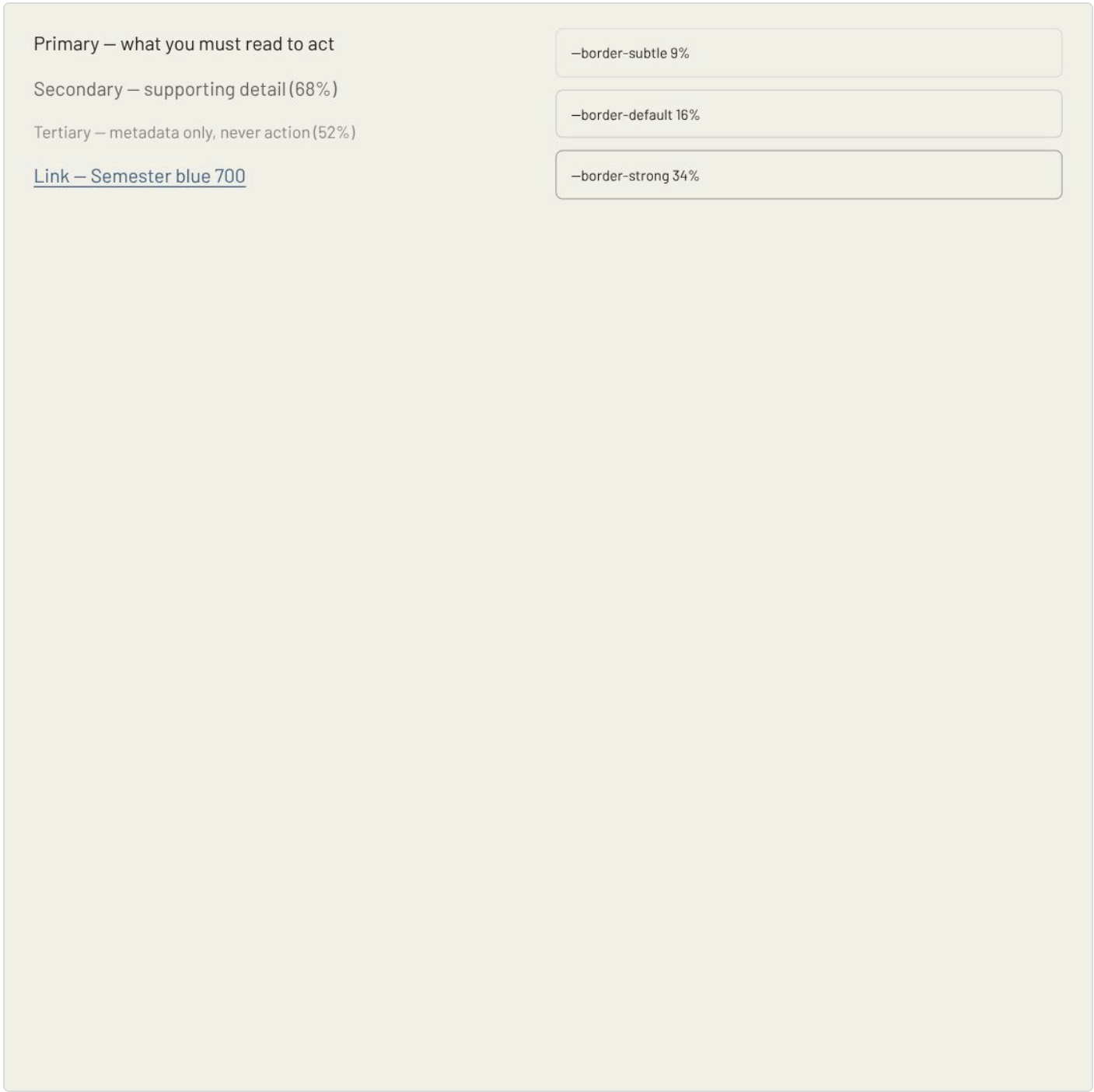
Specimen: colors-source



Specimen: colors-source-dark



Specimen: colors-text



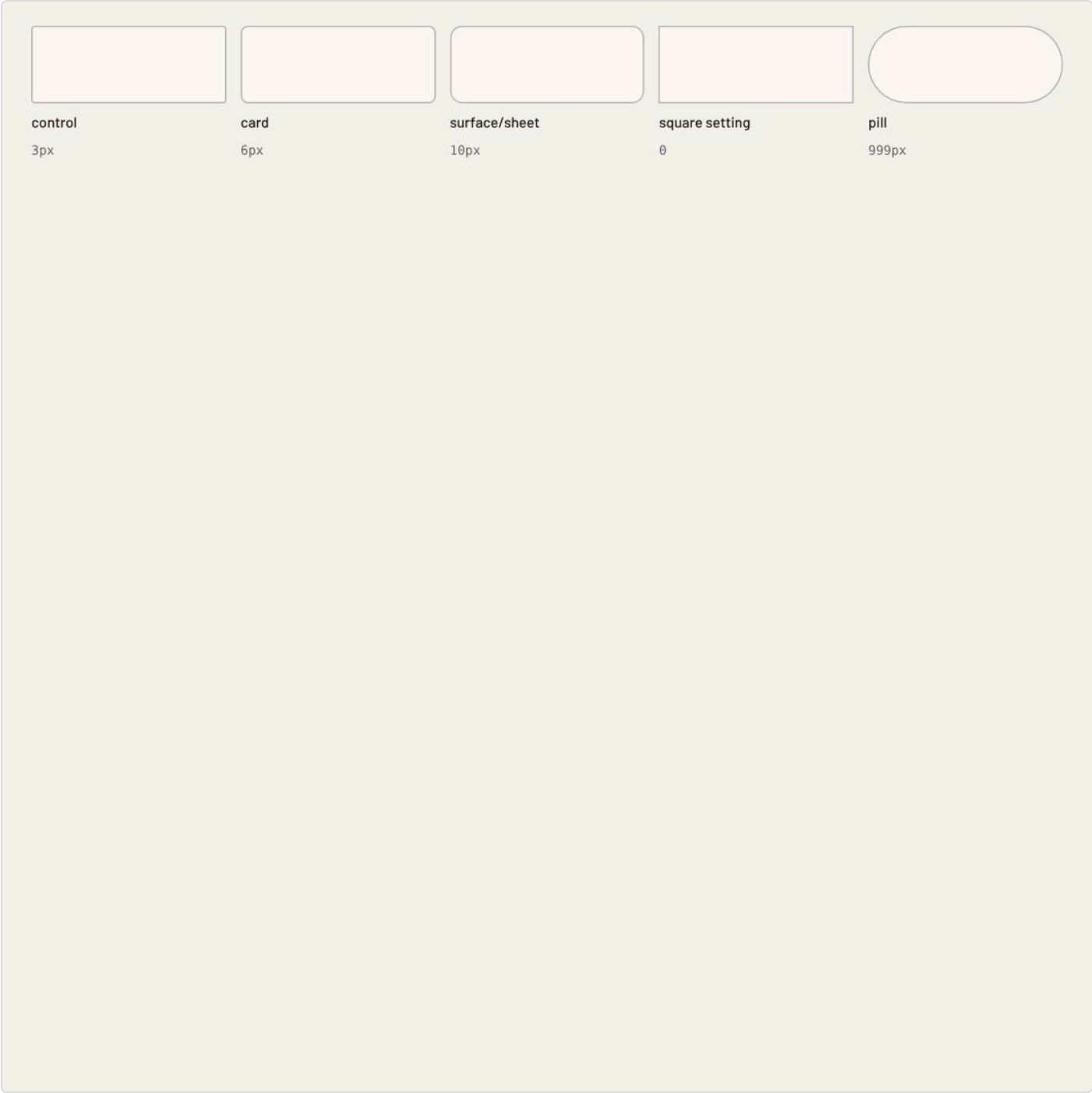
Specimen: elevation



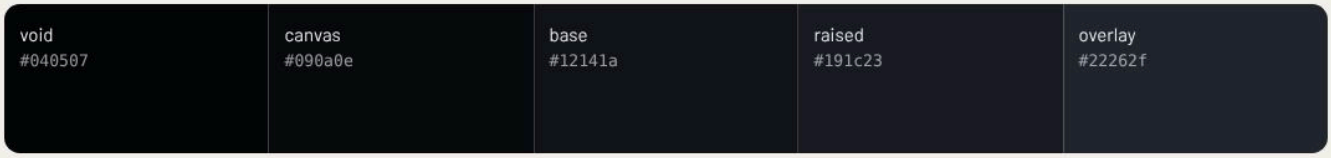
Specimen: iconography



Specimen: radii



Specimen: tonal-elevation



APPENDIX

Source research

The research reports and planning documents the founder gathered while building Semester, reproduced in full.

Semester — domain context map

Twelve modules in one deployable plus three. Border shows where each module's data lives today, measured 2026-10-04.

Solid blue — server-backed under RLS

Dashed orange — device state, rebuild as native

Student web
apps/student-web

Staff and faculty consoles
registrar · instructor · ops

Guardian web
consented sharing only

Mobile
Capacitor · SQLCipher store

Public and partner sites
company-site · trust room

API edge / gateway — Bearer auth · tenant from membership, never the client · PDP decide() · idempotency key · correlation id · audit + outbox in one transaction

`services/core · modular monolith · 12 modules`

identity
tenancy · roles · consent

productivity
Today · tasks · calendar

academic
registration · records

learning
gradebook · study tools

campus
maps · dining · housing

community
posts · moderation

family
guardian links · shares

finance
ledger on server, bill UI static

career
portfolio · applications

marketplace
gated off · not built

support-trust
help · holds · DSR

admin
console · rollout · migration

`platform modules · packages/platform (built, not yet adopted)`

Policy · PDP

Audit · hash chain

Outbox · events

Flags · kill switches

Files · notify · search

Consent · relationships

ai-gateway
every model call · tenant AI policy (not enforced yet)

sync-gateway
commands · idempotency · conflicts

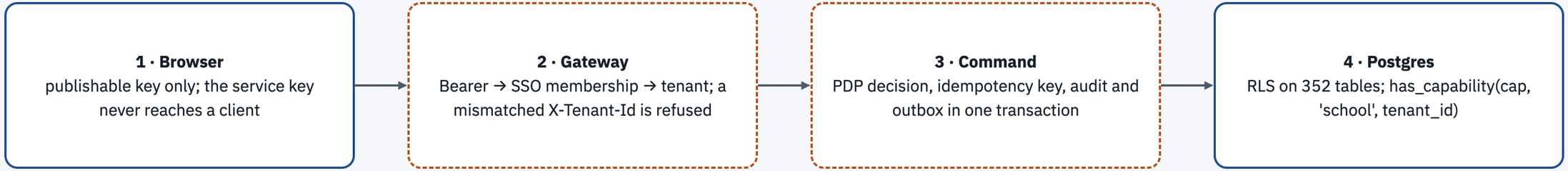
integration-hub
mappings · reconciliation · dead letters

External
SIS · LMS · IdP · LTI

Postgres 17 — 352 tables, RLS on every one · tenant_id from server · 112 policy suites · definer owners still bypass RLS (RISK-004)

Request path and tenant isolation

Where a tenant boundary is enforced on each hop, and what was proven on 2026-10-04.



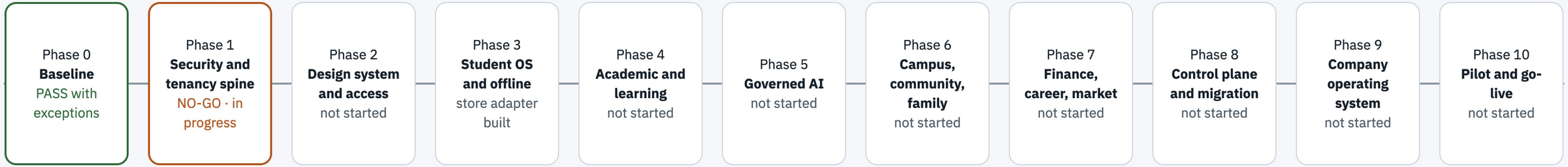
Cross-tenant probe — passing
A tenant-A admin holding every role was evaluated against every policy on 160 tenant-keyed tables. 0 admissions to tenant B. Control: admitted to A on 93 tables. A planted leaking policy is caught by name.

RISK-004 · Definers bypass RLS
postgres owns every table and the 250 client-callable definers, and has BYPASSRLS. Forcing RLS would change nothing; isolation rests on each function's own gate. Fix: non-bypass owner roles, then FORCE.

RISK-008 · AI ignores tenant policy
The claude edge function checks only the global kill switch; effective_ai_policy has no caller. A school that turns AI off is not obeyed at the call. Pulled into Phase 1 (E1.9).

Phase roadmap and gates

Eleven gated phases. A phase starts only when the one before it passes or carries dated, owned exceptions.



Phase 1, next gated work in order (about 22.5 engineer-weeks)

E1.10 record the restore drill → E1.5 revoke TRUNCATE (trial: 109 of 112 suites, only the ratchet moves) → E1.9 tenant AI policy at the call → E1.8 SAST and dependency review → E1.2 definer forged-argument sweep → E1.7 apply staging → E1.1 one tenancy source → E1.3 gateway on the platform spine → E1.4 mount the productivity API → E1.6 non-bypass definer owners, then FORCE RLS

Build a read-model architecture dashboard for your backend — command query separation, projection pipelines, and caching patterns

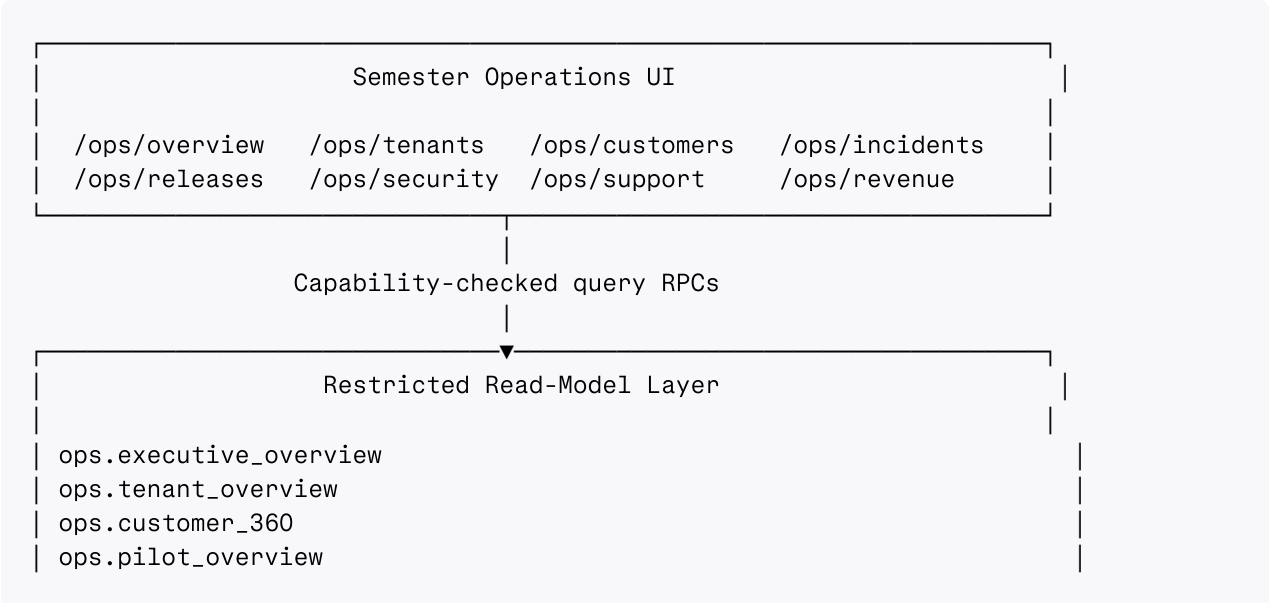
Computer
Design a coherent operating UI component library for your backend data models — state synchronization, optimistic updates, and error handling
Computer
Show me how to structure the incident and SLO command center
How should we implement the access review and security boundary
Detail the tenant health and rollout control plane modules

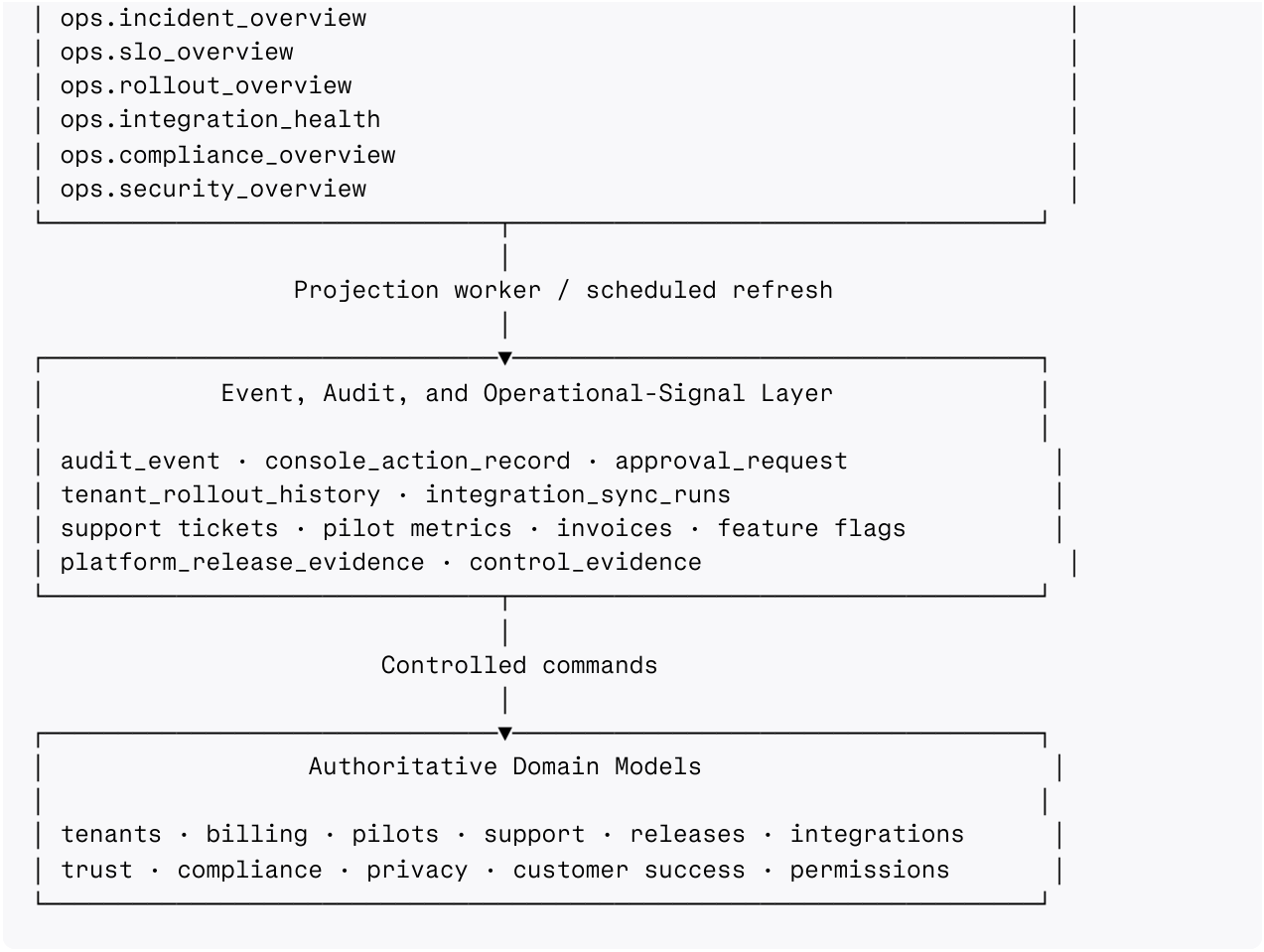
Semester should implement a **bounded CQRS-style operations layer**: keep existing domain tables and controlled RPCs as the command/source-of-truth side, then build restricted, denormalized, freshness-labeled projections specifically for console screens. Do not turn the entire product into event sourcing or duplicate every table—the goal is fast, safe operational views and predictable actions over the backend you already have.

CQRS is valuable when the query shape differs substantially from the transactional write model: instead of expensive symmetric reads/writes, maintain read-only views designed for specific query needs and update them from domain events or reliable projections. [1] [2]

Read-model architecture

Target topology





Rule: writes stay in domains

Need	Correct implementation	Avoid
Change subscription	Controlled commercial/billing RPC or server action	Editing `subscriptions` in a console table
Pause integration	Existing capability-checked integration RPC	Updating connection status directly from client
Enable feature	Approval-gated tenant feature/rollout action	Generic toggle that bypasses rollout evidence
Issue refund	Finance workflow with invoice/payment reference and approval	Ad hoc “refund” button
Handle privacy request	Controlled privacy/retention workflow	Direct deletion of customer records
Investigate tenant issue	Read-model view + time-boxed break-glass if needed	Global direct access to student content
Approve risky action	`approval_request` → `approval_decision` → execution	Browser-side confirmation only

Read-model contracts

Create a dedicated `ops` schema or an equivalent set of private/public capability-checked RPCs. Do not expose raw internal tables to anonymous or generic authenticated clients.

Base envelope

Every console query should return a consistent envelope:

```
type OpsEnvelope<T> = {
  data: T;
  meta: {
    generatedAt: string;
    sourceUpdatedAt: string | null;
    freshness: "live" | "recent" | "delayed" | "stale" | "failed";
    coverage: "complete" | "partial" | "sample" | "unavailable";
    authority: "system-of-record" | "derived" | "operator-entered";
    cache: {
      hit: boolean;
      key: string;
      expiresAt: string | null;
    };
    correlationId: string;
  };
  permissions: {
    canView: boolean;
    canExport: boolean;
    allowedActions: string[];
  };
  warnings: Array<{
    code: string;
    severity: "info" | "warning" | "critical";
    message: string;
    actionUrl?: string;
  }>;
};
```

Required projections

Projection	Main question	Suggested refresh
`ops.executive_overview`	Is the company/product healthy this week?	5–15 minutes
`ops.tenant_overview`	Is this tenant ready, healthy, safe, and commercially current?	1–5 minutes
`ops.customer_360`	What is the account's commercial, implementation, adoption, and renewal state?	5–15 minutes
`ops.pilot_overview`	Is a pilot on track to convert?	5–15 minutes
`ops.support_overview`	What customer-impacting support work needs attention?	Near-real-time
`ops.release_overview`	Is release exposure safe and within SLO/error budget?	1–5 minutes

Projection	Main question	Suggested refresh
`ops.incident_overview`	What incidents are open, escalating, or awaiting updates?	Near-real-time
`ops.slo_overview`	Are services consuming error budget too fast?	1–5 minutes
`ops.integration_overview`	Which integrations are stale, failing, drifting, or blocked?	1–5 minutes
`ops.compliance_overview`	Which controls/evidence/artifacts are missing or expiring?	Daily + on change
`ops.security_overview`	Are high-risk security controls, access reviews, secrets, and alerts current?	Daily + event-driven
`ops.privacy_overview`	Which privacy/retention/export/erasure requests need action?	Hourly
`ops.approval_inbox`	Which controlled actions need review or execution?	Near-real-time

Projection pipeline patterns

Pattern A: live query RPC

Use when data is small, bounded, capability-checked, and must be current.

```
Console screen
  → ops_tenant_overview(tenant_id)
  → checks capability + tenant scope
  → joins only needed domain tables
  → returns redacted/aggregated response
  → emits sensitive-read audit event if required
```

Use for:

- One tenant detail view
- One customer detail view
- One approval request
- One support ticket
- A break-glass session
- One rollout state

Pattern B: materialized projection

Use when a screen needs cross-domain aggregates or expensive joins.

```
Domain event or scheduled job
  → projection worker
  → compute aggregate
  → write ops.tenant_health_snapshot
```

```
  → update source watermark
  → console reads snapshot
```

Use for:

- Customer health
- Executive dashboard
- Pilot scorecards
- Revenue forecast
- Compliance completion
- Tenant rollout readiness
- SLO/error-budget trends

Pattern C: event-driven projection

Use when there is a meaningful state transition:

```
tenant_rollout changed
  → write immutable rollout history
  → enqueue projection task
  → recompute tenant readiness
  → recompute executive summary
  → create inbox item if gate failed
```

For correctness, use an outbox pattern rather than assuming a client call will update every downstream projection.

```
Transactional command
  ├── Updates domain record
  ├── Writes audit event
  └── Writes outbox event in same transaction

Worker
  ├── Claims outbox event
  ├── Updates projection(s)
  ├── Marks event processed
  └── Retries / dead-letters failures
```

Projection tables

```
create schema if not exists ops;

create table ops.projection_watermarks (
  projection_name text primary key,
  source_event_cursor bigint,
  source_updated_at timestamptz,
  projected_at timestamptz not null default now(),
```

```
  status text not null check (
    status in ('healthy', 'delayed', 'failed', 'rebuilding')
  ),
  last_error text
);

create table ops.tenant_health_snapshots (
  tenant_id uuid primary key,
  health_score integer check (health_score between 0 and 100),
  health_band text check (
    health_band in ('healthy', 'watch', 'at_risk', 'critical', 'unknown')
  ),
  activation_score integer,
  adoption_score integer,
  support_score integer,
  commercial_score integer,
  implementation_score integer,
  integration_score integer,
  security_score integer,
  renewal_score integer,
  reasons jsonb not null default '[]'::jsonb,
  recommended_actions jsonb not null default '[]'::jsonb,
  source_updated_at timestamptz,
  computed_at timestamptz not null default now()
);

create table ops.operations_inbox_items (
  id uuid primary key default gen_random_uuid(),
  item_type text not null,
  severity text not null check (severity in ('p0', 'p1', 'p2', 'p3')),
  status text not null check (
    status in ('open', 'acknowledged', 'blocked', 'resolved', 'dismissed')
  ),
  tenant_id uuid,
  customer_id uuid,
  source_type text not null,
  source_id uuid,
  title text not null,
  summary text,
  owner_id uuid,
  due_at timestamptz,
  evidence jsonb not null default '{}'::jsonb,
  recommended_action jsonb not null default '{}'::jsonb,
  correlation_id text,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
```

Do not apply this blindly to production. First reconcile it with existing `tenant_rollout`, `account_health_snapshots`, approval, support, integration, and governance models, because the current backend may already contain equivalent concepts.[3] [4]

Caching policy

Use caching to reduce expensive reads, not to hide stale operational risk.

Data class	Cache strategy	TTL	Invalidation
Approval inbox	Minimal or none	15–60 sec	Approval request/decision event
Open incident/SLO state	Minimal	15–60 sec	Incident, alert, or SLO event
Tenant overview	Short-lived	1–5 min	Tenant/config/feature/rollout change
Executive metrics	Projection cache	5–15 min	Source event + scheduled refresh
Finance forecast	Versioned snapshot	15–60 min	Forecast input/import change
Compliance/trust	Projection cache	1–24 hr	Evidence/artifact/status change
Static capability metadata	Longer cache	1–24 hr	Role/capability migration/change
Historical reports	Immutable cache	Long	Never; version by report/run ID

Cache keys

```
ops:v1:executive:{timeRange}:{operatorScope}
ops:v1:tenant:{tenantId}:{operatorScope}
ops:v1:customer:{customerId}:{operatorScope}
ops:v1:inbox:{role}:{ownerId}:{filtersHash}
ops:v1:slo:{service}:{window}
ops:v1:compliance:{tenantId}:{framework}:{period}
```

Never put raw student PII, session tokens, secrets, unredacted support content, or credential material into shared cache keys or browser storage.

UI component library

Build a domain-aware internal component library—not merely generic cards and tables.

```
app/src/ops/
├── components/
│   ├── OpsShell.tsx
│   ├── OpsContextBar.tsx
│   ├── OpsCommandPalette.tsx
│   ├── OpsMetricCard.tsx
│   ├── OpsFreshnessBadge.tsx
│   ├── OpsHealthBadge.tsx
│   ├── OpsEvidenceDrawer.tsx
│   ├── OpsApprovalBanner.tsx
│   ├── OpsRiskBanner.tsx
│   ├── OpsTimeline.tsx
│   ├── OpsAuditTrail.tsx
│   ├── OpsEmptyState.tsx
│   ├── OpsForbiddenState.tsx
│   ├── OpsFailureState.tsx
│   ├── OpsMutationPreview.tsx
```

```
│   ├── OpsActionDialog.tsx
│   ├── OpsAssignmentControl.tsx
│   ├── OpsDataTable.tsx
│   └── OpsExportButton.tsx
├── data/
│   ├── queryKeys.ts
│   ├── opsClient.ts
│   ├── opsTypes.ts
│   ├── opsMappers.ts
│   └── freshness.ts
├── state/
│   ├── opsContextStore.ts
│   ├── optimisticMutationQueue.ts
│   └── userPreferences.ts
└── screens/
    ├── OverviewScreen.tsx
    ├── TenantScreen.tsx
    ├── IncidentScreen.tsx
    ├── SloScreen.tsx
    ├── AccessReviewScreen.tsx
    └── RolloutScreen.tsx
```

Essential components

Component	Required behavior
`OpsContextBar`	Global/tenant/customer/time-range scope; shows data freshness
`OpsMetricCard`	Value, trend, threshold, freshness, definition, evidence link
`OpsEvidenceDrawer`	Source records, query definition, timestamp, owner, linked decision
`OpsApprovalBanner`	Shows whether action is allowed, pending, approved, expired, or denied
`OpsMutationPreview`	Exact impact, rollback, dependencies, approvals, audit events
`OpsTimeline`	Cross-domain sequence of rollout, approval, support, incident, integration events
`OpsDataTable`	Server-side pagination/filter/sort; permission-aware columns; export scope warning
`OpsFailureState`	Retry, incident link, last known data, stale-data warning
`OpsForbiddenState`	Explains lack of capability without revealing protected data
`OpsActionDialog`	Confirmation, reason, ticket/evidence attachment, approval path

State synchronization

Query ownership

Use one query key per data contract:

```
export const opsQueryKeys = {
  executive: (range: string) => ["ops", "executive", range] as const,
  inbox: (filters: string) => ["ops", "inbox", filters] as const,
```

```
  tenant: (tenantId: string) => ["ops", "tenant", tenantId] as const,
  customer: (customerId: string) => ["ops", "customer", customerId] as const,
  pilot: (pilotId: string) => ["ops", "pilot", pilotId] as const,
  incident: (incidentId: string) => ["ops", "incident", incidentId] as const,
  slo: (service: string, window: string) => ["ops", "slo", service, window] as const
  rollout: (tenantId: string) => ["ops", "rollout", tenantId] as const,
};
```

Optimistic updates

Only use optimistic updates for low-risk, reversible actions:

Action	Optimistic?	Reason
Acknowledge inbox item	Yes	Reversible, no domain-side effect
Assign internal work item	Yes	Reversible, conflict-manageable
Save personal view preference	Yes	User-scoped and low risk
Add internal note	Yes, with pending marker	Can retry safely
Update draft filter state	Yes	Client-only
Approve high-risk action	No	Must await authoritative backend result
Enable tenant feature	No	Approval/rollout gate required
Change plan/entitlement	No	Commercial impact
Issue refund/credit	No	Financial impact
Start break-glass access	No	Must await server grant
Delete/export data	No	Privacy/legal impact
Pause integration	Usually no	Must show current state and outcome
Kill switch	No	Critical system impact

Optimistic mutation pattern

```
const mutation = useMutation({
  mutationFn: acknowledgeInboxItem,
  onMutate: async ({ id }) => {
    await queryClient.cancelQueries({ queryKey: ["ops", "inbox"] });

    const previous = queryClient.getQueryData<InboxItem[]>(["ops", "inbox"]);

    queryClient.setQueryData<InboxItem[]>(["ops", "inbox"], (items = []) =>
      items.map((item) =>
        item.id === id
          ? { ...item, status: "acknowledged", pendingSync: true }
          : item
      )
    );
```

```
    return { previous };
  },
  onError: (_error, _variables, context) => {
    queryClient.setQueryData(["ops", "inbox"], context?.previous);
  },
  onSuccess: (serverItem) => {
    queryClient.setQueryData<InboxItem[]>(["ops", "inbox"], (items = []) =>
      items.map((item) => (item.id === serverItem.id ? serverItem : item))
    );
  },
  onSettled: () => {
    queryClient.invalidateQueries({ queryKey: ["ops", "inbox"] });
  },
});
```

Do not mirror the same mutation into unrelated dashboard cards manually. Invalidate or patch only the authoritative read-model contracts.

Error handling

Error classes

```
type OpsErrorCode =
  | "UNAUTHENTICATED"
  | "FORBIDDEN"
  | "TENANT_SCOPE_DENIED"
  | "APPROVAL_REQUIRED"
  | "APPROVAL_EXPIRED"
  | "CONFLICT"
  | "STALE_VERSION"
  | "VALIDATION_FAILED"
  | "RATE_LIMITED"
  | "DEPENDENCY_UNAVAILABLE"
  | "PROJECTION_STALE"
  | "INTEGRATION_PAUSED"
  | "INCIDENT_ACTIVE"
  | "UNKNOWN";
```

UI behavior

Error	UI behavior
Unauthenticated	Route to sign-in; preserve safe return path
Forbidden	Show capability explanation; no sensitive preview
Approval required	Offer “request approval” with reason/evidence form
Stale version/conflict	Show server state, user’s attempted change, merge/retry choice
Projection stale	Show last known value, stale timestamp, refresh state
Dependency unavailable	Show affected source, fallback data, incident link

Error	UI behavior
Integration paused	Explain why, scope, owner, and approved next action
Active incident	Warn before non-essential change; show release policy
Validation failed	Field-level errors and retained form values
Unknown	Correlation ID, retry guidance, support/incident escalation path

Incident command center

Incident modules

```
/ops/incidents
├── Active incidents
├── Incident detail
├── Service health
├── Alert timeline
├── Customer impact
├── Communications
├── Mitigation actions
├── Evidence/log links
├── Postmortems
└── Follow-up actions
```

Incident record

```
type Incident = {
  id: string;
  status: "declared" | "investigating" | "mitigating" | "monitoring" | "resolved";
  severity: "SEV0" | "SEV1" | "SEV2" | "SEV3";
  title: string;
  startedAt: string;
  detectedAt: string;
  commanderId: string;
  technicalLeadId?: string;
  communicationsLeadId?: string;
  impactedServices: string[];
  affectedTenants: number;
  affectedUsers: number | null;
  customerImpactSummary: string;
  currentHypothesis?: string;
  mitigationPlan?: string;
  nextUpdateAt: string;
  linkedChangeIds: string[];
  linkedSloIds: string[];
  linkedRunbooks: string[];
  linkedAuditEvents: string[];
  correlationId: string;
};
```

Incident workflow

```
Signal/alert
  ↓
Triage
  ↓
Declare severity
  ↓
Assign commander, technical lead, communications lead
  ↓
Freeze or restrict risky releases as required
  ↓
Assess tenant/customer impact
  ↓
Mitigate
  ↓
Update internal stakeholders and affected customers
  ↓
Monitor recovery
  ↓
Resolve
  ↓
Postmortem
  ↓
Corrective actions tracked in operations inbox
```

Severity rules

Severity	Example	Command-center behavior
SEV0	Confirmed major security incident, broad data exposure, complete critical outage	Immediate command mode, executive/legal/security escalation, release freeze
SEV1	Major tenant or workflow outage, serious integrity/reliability issue	Incident commander, customer updates, high-priority mitigation
SEV2	Material degraded feature with workaround	Owner, timeline, monitored impact
SEV3	Minor defect or localized issue	Normal triage and backlog path

Incident dashboard cards

- Active incidents by severity
- Customers/tenants affected
- Time to acknowledge
- Time to mitigate
- Time to resolve
- Next update deadlines
- Error-budget burn rate
- Linked releases

- Open corrective actions
- Repeated incident patterns
- Customer communication status

SLO command center

Define services, then objectives

Do not start with arbitrary uptime percentages. First define critical user journeys:

Service/journey	Example SLI	Example initial SLO
Authentication	Successful authentication requests / valid auth attempts	99.9% monthly
Core student workspace read	Successful read responses within latency threshold	99.9% monthly
Critical write command	Successful accepted commands / valid command attempts	99.9% monthly
Tenant rollout control	Successful rollout/config reads and approved writes	99.9% monthly
Integration sync	Successful scheduled syncs completed within freshness target	99.0% weekly
Support notifications	Accepted notification jobs delivered within target window	99.5% monthly
Billing webhook processing	Verified event processing within target	99.9% monthly
Admin command center	Successful authorized console queries	99.5% monthly

An SLO is a target value/range for a measured service indicator; error budgets are the permissible miss rate and help decide whether release velocity should slow when reliability degrades.[5] [6]

SLO schema

```
create table ops.service_catalog (
  id uuid primary key default gen_random_uuid(),
  service_key text unique not null,
  name text not null,
  owner_role text not null,
  criticality text not null check (criticality in ('tier_0', 'tier_1', 'tier_2', 'ti
  description text not null,
  runbook_url text,
  created_at timestamptz not null default now()
);

create table ops.slo_definitions (
  id uuid primary key default gen_random_uuid(),
  service_id uuid not null references ops.service_catalog(id),
  name text not null,
  sli_query_version text not null,
  objective numeric(5,4) not null check (objective > 0 and objective < 1),
  window_days integer not null,
  alert_policy jsonb not null,
  release_policy jsonb not null,
```

```
  effective_at timestamptz not null,
  retired_at timestamptz
);

create table ops.slo_snapshots (
  slo_id uuid not null references ops.slo_definitions(id),
  captured_at timestamptz not null,
  eligible_events bigint not null,
  bad_events bigint not null,
  compliance_ratio numeric(8,6) not null,
  error_budget_remaining numeric(8,6) not null,
  burn_rate_1h numeric(10,4),
  burn_rate_6h numeric(10,4),
  burn_rate_24h numeric(10,4),
  source_status text not null,
  primary key (slo_id, captured_at)
);
```

Error-budget policy

Remaining budget	Policy
75–100%	Normal release velocity
50–75%	Release owner reviews risk; prioritize reliability work
25–50%	Restrict high-risk changes; require additional approval
0–25%	Freeze non-essential changes for affected service
Exhausted	Incident/reliability review; corrective plan before resuming normal releases

Google's SRE error-budget approach is designed to protect users from repeated reliability failures while giving teams a concrete mechanism for balancing reliability and feature delivery. [7]

Access review and security boundary

Access review workflow

Run these on defined schedules:

Review	Cadence	Scope
Privileged console access	Monthly	`app_admins`, operator roles, high-risk capabilities
Tenant admin access	Quarterly or customer-defined	Institution membership and school-scoped grants
Break-glass access	Per event + monthly review	Every grant, use, expiry, reviewer sign-off
Service accounts	Monthly	Purpose, owner, scope, rotation, last use
Integration scopes	Quarterly and on change	External read/write scopes and approvals
Finance permissions	Monthly	Quote, plan, refund, credit, dunning, invoice actions

Review	Cadence	Scope
Support access	Weekly	Time-bound support grants and aggregate access
Vendor access	Quarterly	Contractor/vendor account access and offboarding

Access review record

```
type AccessReview = {
  id: string;
  reviewType:
    | "privileged_console"
    | "tenant_admin"
    | "service_account"
    | "integration_scope"
    | "finance"
    | "support"
    | "break_glass";
  scopeType: "global" | "tenant" | "customer" | "service";
  scopeId?: string;
  periodStart: string;
  periodEnd: string;
  reviewerId: string;
  status: "draft" | "in_review" | "completed" | "overdue";
  findings: Array<{
    principalId: string;
    access: string;
    decision: "retain" | "modify" | "revoke";
    reason: string;
    actionEvidence?: string;
  }>;
  dueAt: string;
  completedAt?: string;
};
```

Security boundary rules

```
Browser:
- Uses only publishable keys.
- Never receives service-role credentials.
- Reads only capability-checked query endpoints.
- Sends commands only to typed, validated, auditable server/RPC endpoints.

Server/RPC:
- Validates identity with authoritative session/user context.
- Checks capability and tenant/customer scope.
- Validates schema and business invariants.
- Enforces approval and duty requirements.
- Writes audit event and correlation ID.
- Queues side effects through outbox/worker where needed.

Database:
- Uses RLS as a data-layer backstop.
```

```
- Keeps private/internal data in non-exposed schemas.
- Exposes narrow RPC/read-model contracts, not broad raw tables.
- Uses immutable audit/evidence records for high-risk actions.

Workers:
- Own provider credentials and integration secrets.
- Operate from queue/outbox jobs.
- Apply idempotency keys.
- Record sanitized errors and retries.
- Never expose raw upstream payloads to general operators.
```

NIST CSF 2.0 explicitly frames cybersecurity as an organization-wide risk-management concern, including governance, roles, policies, and oversight—not only technical controls. [8] [9]

Tenant health and rollout control plane

Tenant health dimensions

```
Tenant Health Score
├── Commercial
│   ├── Contract current
│   ├── Billing current
│   ├── Renewal date risk
│   └── Plan/entitlement alignment
├── Implementation
│   ├── Milestone completion
│   ├── Admin training
│   ├── Integration readiness
│   └── Launch blockers
├── Adoption
│   ├── Invited users
│   ├── Activation
│   ├── First meaningful action
│   ├── Workflow completion
│   └── Weekly active use
├── Support
│   ├── P0/P1 ticket count
│   ├── SLA attainment
│   ├── Sentiment
│   └── Repeat issues
├── Reliability
│   ├── Tenant-specific errors
│   ├── Integration freshness
│   ├── Feature/release incidents
│   └── SLO impact
├── Security/compliance
│   ├── SSO state
│   ├── Access-review status
│   ├── Trust evidence status
│   └── Open security/privacy issues
└── Relationship
    ├── Champion engagement
    ├── Executive sponsor attendance
```

```
    ├── QBR status
    └── Expansion/referral signal
```

Health score approach

Do not use a universal opaque score. Store components and explain them:

```
type TenantHealth = {
  overallBand: "healthy" | "watch" | "at_risk" | "critical" | "unknown";
  overallScore?: number;
  components: {
    commercial: number | null;
    implementation: number | null;
    adoption: number | null;
    support: number | null;
    reliability: number | null;
    security: number | null;
    relationship: number | null;
  };
  topDrivers: Array<{
    direction: "positive" | "negative";
    severity: "info" | "warning" | "critical";
    message: string;
    source: string;
    observedAt: string;
  }>;
  recommendedActions: Array<{
    action: string;
    ownerRole: string;
    dueAt?: string;
    approvalRequired: boolean;
  }>;
  computedAt: string;
  sourceFreshness: string;
};
```

Rollout control plane

Your rollout UI should enforce a state machine:

```
Draft
→ Pilot designed
→ Contracting
→ Security/procurement review
→ Implementation
→ Launch readiness
→ Controlled launch
→ Live pilot
→ Conversion review
→ Annual live
→ Expansion
→ Renewal risk
→ Offboarding
```

```
→ Archived
→ Purge eligible
```

For each transition:

```
Current state
Required evidence
Blocked by
Required capability
Required approvers
Rollback path
Customer communication
Audit event
Next review date
```

Rollout module sections

Tab	What it shows
Overview	Lifecycle state, owner, health, next gate, blockers
Gate checklist	Required evidence and approval progress
Implementation	Projects, milestones, data/integration setup, training
Feature exposure	Tenant feature policy, cohorts, kill switches, release history
Integrations	Connection state, scopes, sync freshness, errors, drift
Adoption	Activation, first action, workflow completion, retention
Support	Open tickets, trends, SLA, systemic problems
Commercial	Plan, contract, quote, invoice, renewal, commitments
Security/trust	SSO, compliance evidence, trust-room status, review dates
Privacy	Data requests, retention/hold constraints, export/deletion status
Audit	Timeline of significant operator, tenant, and automated events

Build command

Use this as the next implementation prompt:

```
Build the Semester Operations Intelligence Layer on top of the existing
Semester backend and Operations Command Center.

Do not rebuild domains or create a generic admin database browser.

Implement a bounded CQRS-style architecture:

1. Keep existing domain tables, controlled RPCs, approval workflows, audit
   records, rollout state, GTM, billing, support, trust, compliance, privacy,
   integration, and tenant models as authoritative command-side sources.
```

2. Create restricted operational read models for console screens:
   - executive overview
   - operations inbox
   - tenant overview
   - customer 360
   - pilot overview
   - support overview
   - release overview
   - incident overview
   - SLO overview
   - integration overview
   - trust/compliance overview
   - privacy overview
   - security overview
   - audit search

3. Every query contract must:
   - check capabilities server-side
   - enforce tenant/customer scope
   - return only minimum required fields
   - redact sensitive data by default
   - include freshness, source, coverage, authority, and correlation metadata
   - support pagination and filters
   - emit audit evidence for sensitive reads where appropriate

4. Build:
   - Unified Operations Inbox
   - Cross-domain work orchestration linked to existing source records
   - Evidence drawers
   - Action previews
   - Decision Center
   - Release and error-budget control
   - Incident Command Center
   - Tenant Health and Rollout Control Plane
   - Access Review workspace
   - Risk Register
   - Weekly/monthly operating-review report generator

5. Use optimistic updates only for low-risk reversible actions:
   - acknowledge inbox item
   - assign work item
   - save personal view
   - add internal note
   All high-risk actions must wait for authoritative server responses.

6. For incident/SLO work:
   - define services, SLIs, SLOs, error budgets, alert policies, and release policies
   - pause or restrict non-essential releases when error budgets are low or exhausted
   - link incidents to releases, tenants, affected customers, audit events, runbooks, and corrective actions

7. For access review:
   - build monthly privileged-console reviews

```
   - time-boxed break-glass review
   - quarterly tenant-admin and integration-scope review
   - service-account review
   - finance permission review
   - require retain/modify/revoke decisions with evidence

8. For tenant health:
   - make scores explainable
   - show commercial, implementation, adoption, support, reliability,
     security, and relationship components
   - show freshness and source metadata
   - generate assigned recommended actions

9. For rollout:
   - enforce explicit lifecycle transitions
   - require gate evidence
   - show blockers and approval requirements
   - preserve rollback paths and audit events
   - never permit client-only state changes to change tenant rollout status

10. Add:
   - authorization tests
   - tenant-isolation tests
   - projection freshness tests
   - optimistic-update rollback tests
   - incident transition tests
   - error-budget policy tests
   - access-review workflow tests
   - rollout gate tests
   - export/redaction tests

Before implementation, classify all current Supabase advisor findings:
- intentionally private/internal
- intentionally exposed and RLS-protected
- needs privilege narrowing
- needs dedicated RPC/read model
- needs test
- needs remediation

Do not blindly revoke permissions or add policies. Document every exposure,
its intended client, its RLS expectation, its function-grant expectation,
and its remediation owner.
```

Priority order

1. **Exposure classification and access boundary**
2. **Read-model contracts**
3. **Operations inbox**
4. **Tenant lifecycle and health**
5. **Incident/SLO command center**
6. **Access review**

7. **Release and rollout controls**
8. **Customer/pilot/revenue intelligence**
9. **Trust, compliance, privacy, and audit workspaces**
10. **Automated operating-review reports**

This creates an internal system that is fast for operators, explainable for leadership, safe for institutions, and resistant to the common failure mode of "admin dashboards" that quietly bypass the controls the product claims to enforce.

⁂

1. https://microservices.io/patterns/data/cqrs.html
2. https://microservices.io/patterns/
3. https://info.lse.ac.uk/staff/divisions/Eden-Centre/Assets-EC/Documents/Claude-Guides-Dec-2025/Guide-Creating-Claude-Artifacts-V1.0-011225.pdf
4. https://academy.claude.com/courses/claude-101/creating-with-artifacts
5. https://sre.google/sre-book/service-level-objectives/
6. https://sre.google/workbook/implementing-slos/
7. https://sre.google/workbook/error-budget-policy/
8. https://www.nist.gov/cyberframework
9. https://csrc.nist.gov/news/2024/the-nist-csf-20-is-here
10. https://microservices.io/patterns/data/event-sourcing.html
11. https://microservices.io/tags/pattern
12. https://attractgroup.com/blog/mastering-error-budgets-a-complete-guide-to-slos-slis-and-reliability-for-service-levels/
13. https://www.reddit.com/r/sre/comments/1j0yekt/how_do_you_define_error_budgets/
14. https://www.reddit.com/r/microservices/comments/1d1tt1p/what_is_cqrs_design_pattern_in_microservices/
15. https://stackoverflow.com/questions/58436626/read-model-in-microservice-architecture
16. https://discuss.kurrent.io/t/es-cqrs-microservice-setup-read-model-generation/3294
17. https://sre.google/workbook/alerting-on-slos/
18. https://sre.google/workbook/index/
19. https://cloud.google.com/blog/products/management-tools/sre-error-budgets-and-maintenance-windows

Build an interactive CQRS read-model dashboard for projections, lag, and cache invalidation — with live simulator controls

Computer
Design a CQRS component library for React or Vue — sortable data grids, optimistic mutation hooks, and projection state sync
Computer
Show me the projection pipeline schema implementation
How do we handle cache invalidation for projections
Write the capability-checked query RPC definitions

Implement the projection layer with **transactional outbox events, idempotent projection workers, versioned read models, targeted cache invalidation, and capability-checked query RPCs.** The browser should never query broad internal tables directly; it receives only scoped read-model contracts.

CQRS works well here because Semester's operational screens need denormalized, fast, cross-domain reads that are different from the transactional domain models used for tenant, billing, rollout, support, and approval writes. A reliable approach is to atomically persist domain changes plus outbox events, then let idempotent consumers update materialized projections.[1] [2]

Projection pipeline schema

1. Schemas and boundaries

Use three database layers:

```
public
  Authoritative domain tables and controlled RPCs.
  RLS-enabled tables and approved public API functions only.

private
  Worker-only outbox, event receipt, secrets-adjacent metadata,
  queue state, internal projection helpers.

ops
  Read-only operational projections, query functions, summary views,
  freshness state, inbox items, and dashboard data contracts.
```

The browser should call only:

```
public.ops_executive_overview(...)
public.ops_tenant_overview(...)
public.ops_operations_inbox(...)
public.ops_incident_overview(...)
public.ops_slo_overview(...)
public.ops_access_review_overview(...)
public.ops_rollout_overview(...)
```

Do not expose `ops` tables directly through generic PostgREST/GraphQL. Prefer capability-checked RPCs that return only the exact fields required by the screen.

2. Core event/outbox tables

Create a migration, for example:

```
supabase/migrations/20261005_ops_projection_pipeline.sql
```

```
create schema if not exists ops;
create schema if not exists private;

revoke all on schema ops from anon, authenticated;
revoke all on schema private from anon, authenticated;

create type private.domain_event_status as enum (
  'pending',
  'processing',
  'processed',
  'failed',
  'dead_letter'
);

create table if not exists private.domain_outbox_events (
  id uuid primary key default gen_random_uuid(),

  event_type text not null,
  event_version integer not null default 1,

  aggregate_type text not null,
  aggregate_id uuid not null,

  tenant_id uuid,
  customer_id uuid,

  correlation_id text not null,
  causation_id uuid,

  actor_type text not null default 'system',
  actor_id uuid,

  payload jsonb not null default '{}'::jsonb,
```

```
  payload_hash text not null,

  occurred_at timestamptz not null default now(),

  status private.domain_event_status not null default 'pending',
  attempts integer not null default 0,
  available_at timestamptz not null default now(),
  locked_at timestamptz,
  locked_by text,
  processed_at timestamptz,
  last_error text,

  created_at timestamptz not null default now(),

  unique (event_type, aggregate_type, aggregate_id, payload_hash)
);

create index if not exists domain_outbox_events_pending_idx
  on private.domain_outbox_events (status, available_at, occurred_at)
  where status in ('pending', 'failed');

create index if not exists domain_outbox_events_tenant_idx
  on private.domain_outbox_events (tenant_id, occurred_at desc)
  where tenant_id is not null;

create index if not exists domain_outbox_events_correlation_idx
  on private.domain_outbox_events (correlation_id, occurred_at);

create table if not exists private.projection_event_receipts (
  projection_name text not null,
  event_id uuid not null references private.domain_outbox_events(id) on delete casca
  processed_at timestamptz not null default now(),
  projection_version integer not null default 1,
  result_hash text,
  primary key (projection_name, event_id)
);

create table if not exists ops.projection_watermarks (
  projection_name text primary key,
  projection_version integer not null default 1,

  last_event_id uuid,
  last_event_at timestamptz,

  source_updated_at timestamptz,
  projected_at timestamptz not null default now(),

  status text not null default 'healthy'
    check (status in ('healthy', 'delayed', 'failed', 'rebuilding')),

  lag_seconds integer,
  last_error text,
  rebuild_requested_at timestamptz,
  rebuilt_at timestamptz
);
```

```
create table if not exists ops.projection_invalidations (
  id uuid primary key default gen_random_uuid(),

  namespace text not null,
  cache_key text not null,

  tenant_id uuid,
  customer_id uuid,

  reason text not null,
  event_id uuid references private.domain_outbox_events(id) on delete set null,

  invalidated_at timestamptz not null default now(),
  expires_at timestamptz,

  unique (namespace, cache_key)
);

create index if not exists projection_invalidations_active_idx
  on ops.projection_invalidations (namespace, cache_key, invalidated_at desc);

create table if not exists ops.projection_rebuild_runs (
  id uuid primary key default gen_random_uuid(),

  projection_name text not null,
  projection_version integer not null,

  requested_by uuid,
  requested_reason text not null,

  mode text not null
    check (mode in ('incremental', 'full', 'blue_green')),

  status text not null default 'queued'
    check (status in ('queued', 'running', 'completed', 'failed', 'cancelled')),

  started_at timestamptz,
  completed_at timestamptz,

  events_replayed bigint not null default 0,
  rows_written bigint not null default 0,

  error_summary text,
  correlation_id text not null,

  created_at timestamptz not null default now()
);
```

3. Domain-event writer

Do not emit events from the browser. Domain commands write an event **in the same database transaction** as the domain change.

```
create or replace function private.record_domain_event(
  p_event_type text,
  p_event_version integer,
  p_aggregate_type text,
  p_aggregate_id uuid,
  p_tenant_id uuid,
  p_customer_id uuid,
  p_correlation_id text,
  p_causation_id uuid,
  p_actor_type text,
  p_actor_id uuid,
  p_payload jsonb
)
returns uuid
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_event_id uuid;
  v_payload_hash text;
begin
  v_payload_hash := encode(
    extensions.digest(
      coalesce(p_payload::text, '{}'),
      'sha256'
    ),
    'hex'
  );

  insert into private.domain_outbox_events (
    event_type,
    event_version,
    aggregate_type,
    aggregate_id,
    tenant_id,
    customer_id,
    correlation_id,
    causation_id,
    actor_type,
    actor_id,
    payload,
    payload_hash
  )
  values (
    p_event_type,
    p_event_version,
    p_aggregate_type,
    p_aggregate_id,
    p_tenant_id,
    p_customer_id,
    p_correlation_id,
    p_causation_id,
    p_actor_type,
    p_actor_id,
    p_payload,
```

```
    v_payload_hash
  )
  on conflict (event_type, aggregate_type, aggregate_id, payload_hash)
  do update set occurred_at = excluded.occurred_at
  returning id into v_event_id;

  return v_event_id;
end;
$$;

revoke all on function private.record_domain_event(
  text, integer, text, uuid, uuid, uuid, text, uuid, text, uuid, jsonb
) from public;
```

`SECURITY DEFINER` should be used sparingly. Supabase recommends preferring `SECURITY INVOKER` where possible; when `SECURITY DEFINER` is needed, set an empty `search_path` and schema-qualify referenced relations to reduce the blast radius.[3] [4]

4. Example command with event emission

This pattern applies to a controlled tenant rollout transition:

```
create or replace function public.transition_tenant_rollout(
  p_tenant_id uuid,
  p_next_state text,
  p_evidence jsonb,
  p_correlation_id text
)
returns jsonb
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_actor_id uuid := auth.uid();
  v_previous_state text;
  v_event_id uuid;
begin
  if v_actor_id is null then
    raise exception 'unauthenticated';
  end if;

  if not private.has_capability(
    v_actor_id,
    'tenant:rollout:transition',
    p_tenant_id
  ) then
    raise exception 'forbidden';
  end if;

  select rollout_state
    into v_previous_state
  from public.tenant_rollout
  where tenant_id = p_tenant_id
```

```
  for update;

  if v_previous_state is null then
    raise exception 'tenant rollout record not found';
  end if;

  perform private.assert_rollout_transition_allowed(
    p_tenant_id,
    v_previous_state,
    p_next_state,
    p_evidence
  );

  update public.tenant_rollout
  set
    rollout_state = p_next_state,
    updated_at = now()
  where tenant_id = p_tenant_id;

  insert into public.tenant_rollout_history (
    tenant_id,
    previous_state,
    next_state,
    changed_by,
    evidence,
    correlation_id
  )
  values (
    p_tenant_id,
    v_previous_state,
    p_next_state,
    v_actor_id,
    p_evidence,
    p_correlation_id
  );

  v_event_id := private.record_domain_event(
    'tenant.rollout.transitioned',
    1,
    'tenant_rollout',
    p_tenant_id,
    p_tenant_id,
    null,
    p_correlation_id,
    null,
    'user',
    v_actor_id,
    jsonb_build_object(
      'previous_state', v_previous_state,
      'next_state', p_next_state,
      'evidence', p_evidence
    )
  );

  return jsonb_build_object(
    'tenant_id', p_tenant_id,
```

```
    'previous_state', v_previous_state,
    'next_state', p_next_state,
    'event_id', v_event_id,
    'correlation_id', p_correlation_id
  );
end;
$$;

revoke all on function public.transition_tenant_rollout(
  uuid, text, jsonb, text
) from public;

grant execute on function public.transition_tenant_rollout(
  uuid, text, jsonb, text
) to authenticated;
```

The internal capability function must check both the operator's role/capability and scope. A user who can administer Tenant A must not gain access to Tenant B because they supplied a different `tenant_id`.

Projection implementation

Tenant health projection

```
create table if not exists ops.tenant_health_snapshots (
  tenant_id uuid primary key,

  overall_score integer
    check (overall_score between 0 and 100),

  health_band text not null
    check (
      health_band in (
        'healthy',
        'watch',
        'at_risk',
        'critical',
        'unknown'
      )
    ),

  commercial_score integer,
  implementation_score integer,
  adoption_score integer,
  support_score integer,
  reliability_score integer,
  integration_score integer,
  security_score integer,
  relationship_score integer,

  top_drivers jsonb not null default '[]'::jsonb,
  recommended_actions jsonb not null default '[]'::jsonb,
```

```
  source_updated_at timestamptz,
  computed_at timestamptz not null default now(),

  projection_version integer not null default 1,
  freshness text not null default 'recent'
    check (freshness in ('live', 'recent', 'delayed', 'stale', 'failed'))
);

create index if not exists tenant_health_band_idx
  on ops.tenant_health_snapshots (health_band, computed_at desc);
```

Projection worker pseudocode

The worker can run as a controlled Supabase Edge Function, a Vercel cron/server function, or an internal queue consumer. It must use a server-only credential and never run in the browser.

```
type DomainOutboxEvent = {
  id: string;
  event_type: string;
  event_version: number;
  aggregate_type: string;
  aggregate_id: string;
  tenant_id: string | null;
  customer_id: string | null;
  correlation_id: string;
  payload: Record<string, unknown>;
  occurred_at: string;
};

async function processProjectionBatch(workerId: string) {
  const events = await claimPendingEvents({
    workerId,
    maxEvents: 50,
    lockForSeconds: 60,
  });

  for (const event of events) {
    try {
      await processEvent(event);

      await markEventProcessed(event.id);
    } catch (error) {
      await recordProjectionFailure({
        eventId: event.id,
        error: sanitizeError(error),
        retryAt: calculateRetryTime(event),
      });
    }
  }
}

async function processEvent(event: DomainOutboxEvent) {
  switch (event.event_type) {
    case "tenant.rollout.transitioned":
```

```
    case "tenant.feature.changed":
    case "tenant.plan.changed":
    case "integration.sync.completed":
    case "support.ticket.updated":
    case "pilot.metric.recorded":
    case "renewal.opportunity.updated":
      if (event.tenant_id) {
        await rebuildTenantHealthProjection(event.tenant_id, event);
        await invalidateTenantCache(event.tenant_id, event);
      }
      break;

    case "approval.request.created":
    case "approval.decision.recorded":
      await rebuildApprovalInboxProjection(event);
      await invalidateApprovalCache(event);
      break;

    case "incident.declared":
    case "incident.updated":
    case "slo.snapshot.recorded":
      await rebuildIncidentAndSloProjection(event);
      await invalidateIncidentCache(event);
      break;

    default:
      await recordIgnoredEvent(event);
  }
}
```

Idempotency guard

```
create or replace function private.claim_projection_receipt(
  p_projection_name text,
  p_event_id uuid,
  p_projection_version integer
)
returns boolean
language plpgsql
security definer
set search_path = ''
as $$
begin
  insert into private.projection_event_receipts (
    projection_name,
    event_id,
    projection_version
  )
  values (
    p_projection_name,
    p_event_id,
    p_projection_version
  )
  on conflict do nothing;
```

```
  return found;
end;
$$;
```

Before modifying a projection:

```
if receipt already exists:
  skip event

otherwise:
  apply event
  update projection
  update watermark
  create cache invalidation
```

This handles duplicate delivery safely. The transactional outbox and idempotent consumer pattern are specifically recommended for CQRS read-model updates because the consumer must tolerate duplicate delivery while maintaining consistent projections. [1]

Cache invalidation strategy

Use a combination of **event-driven invalidation**, **versioned projection metadata**, and **short TTLs**. Do not use one global `clearCache()` call.

Invalidation flow

```
Authoritative write succeeds
  ↓
Outbox event created atomically
  ↓
Projection worker applies event
  ↓
Projection version/freshness updated
  ↓
Relevant cache namespaces invalidated
  ↓
Browser receives targeted invalidation or refetch signal
  ↓
UI shows updated server-authoritative read model
```

Cache invalidation matrix

Domain event	Projections to refresh	Browser query keys
Tenant rollout changed	Tenant overview, executive overview, rollout overview, inbox	`["ops","tenant",tenantId]`, `["ops","rollout",tenantId]`
Support ticket P0/P1 updated	Support overview, tenant health, executive overview, inbox	`["ops","support"]`, `["ops","tenant",tenantId]`

Domain event	Projections to refresh	Browser query keys
Pilot metric recorded	Pilot overview, tenant health, customer 360, executive overview	`["ops","pilot",pilotId]`, `["ops","tenant",tenantId]`
Renewal status changed	Customer 360, revenue overview, tenant health, inbox	`["ops","customer",customerId]`
Feature flag changed	Release overview, tenant overview, rollout overview	`["ops","releases"]`, `["ops","tenant",tenantId]`
Integration sync failed	Integration overview, tenant health, inbox, incident overview if critical	`["ops","integration",tenantId]`
Compliance evidence expired	Compliance overview, trust overview, tenant overview, inbox	`["ops","compliance",tenantId]`
Approval decided	Approval inbox, target entity overview, audit timeline	`["ops","approvals"]`
Incident updated	Incident overview, SLO overview, release overview, executive overview	`["ops","incidents"]`, `["ops","slos"]`

Database invalidation helper

```
create or replace function private.invalidate_ops_cache(
  p_namespace text,
  p_cache_key text,
  p_tenant_id uuid,
  p_customer_id uuid,
  p_reason text,
  p_event_id uuid
)
returns void
language plpgsql
security definer
set search_path = ''
as $$
begin
  insert into ops.projection_invalidations (
    namespace,
    cache_key,
    tenant_id,
    customer_id,
    reason,
    event_id,
    invalidated_at
  )
  values (
    p_namespace,
    p_cache_key,
    p_tenant_id,
    p_customer_id,
    p_reason,
    p_event_id,
    now()
  )
```

```
  on conflict (namespace, cache_key)
  do update set
    reason = excluded.reason,
    event_id = excluded.event_id,
    invalidated_at = excluded.invalidated_at;
end;
$$;
```

Client cache policy

With TanStack Query:

```
export const opsQueryKeys = {
  executive: (range: string) => ["ops", "executive", range] as const,
  inbox: (filters: Record<string, unknown>) =>
    ["ops", "inbox", filters] as const,
  tenant: (tenantId: string) =>
    ["ops", "tenant", tenantId] as const,
  tenantHealth: (tenantId: string) =>
    ["ops", "tenant-health", tenantId] as const,
  rollout: (tenantId: string) =>
    ["ops", "rollout", tenantId] as const,
  support: (tenantId?: string) =>
    ["ops", "support", tenantId ?? "all"] as const,
  incident: (incidentId: string) =>
    ["ops", "incident", incidentId] as const,
  slos: (service?: string) =>
    ["ops", "slos", service ?? "all"] as const,
};
```

```
export const opsQueryDefaults = {
  staleTime: 60_000,
  gcTime: 10 * 60_000,
  refetchOnWindowFocus: true,
  retry: 2,
};
```

TanStack Query's cache is keyed by serializable query keys and handles caching, background refetch, request deduplication, retries, and targeted invalidation. Its documented optimistic-update pattern uses `onMutate` to snapshot/update cache, `onError` to roll back, and `onSettled` to reconcile with server truth.[5] [6]

Targeted invalidation handler

```
type OpsInvalidation = {
  namespace: string;
  tenantId?: string;
  customerId?: string;
  incidentId?: string;
  reason: string;
  version?: number;
```

```
};

export function applyOpsInvalidation(
  queryClient: QueryClient,
  message: OpsInvalidation
) {
  switch (message.namespace) {
    case "tenant":
      if (message.tenantId) {
        queryClient.invalidateQueries({
          queryKey: opsQueryKeys.tenant(message.tenantId),
        });

        queryClient.invalidateQueries({
          queryKey: opsQueryKeys.tenantHealth(message.tenantId),
        });

        queryClient.invalidateQueries({
          queryKey: opsQueryKeys.rollout(message.tenantId),
        });
      }

      queryClient.invalidateQueries({
        queryKey: ["ops", "executive"],
      });
      break;

    case "approval":
      queryClient.invalidateQueries({
        queryKey: ["ops", "inbox"],
      });
      queryClient.invalidateQueries({
        queryKey: ["ops", "approvals"],
      });
      break;

    case "incident":
      queryClient.invalidateQueries({
        queryKey: ["ops", "incidents"],
      });
      queryClient.invalidateQueries({
        queryKey: ["ops", "slos"],
      });
      queryClient.invalidateQueries({
        queryKey: ["ops", "executive"],
      });
      break;

    case "support":
      queryClient.invalidateQueries({
        queryKey: ["ops", "support"],
      });
      if (message.tenantId) {
        queryClient.invalidateQueries({
          queryKey: opsQueryKeys.tenantHealth(message.tenantId),
        });
```

```
      }
      break;
  }
}
```

Realtime warning

Use realtime only for:

- Approval-inbox changes
- Incident state changes
- Support P0/P1 changes
- Projection invalidation signals
- Tenant rollout state changes

Do **not** stream every underlying domain-table row into the browser. Stream a minimal invalidation message, then refetch the scoped read model.

```
Realtime signal:
{
  namespace: "tenant",
  tenantId: "tenant-id",
  version: 42,
  reason: "integration.sync.completed"
}

Browser:
- confirms current user can view current scope
- invalidates only relevant query keys
- refetches capability-checked server contract
```

Capability-checked query RPCs

Capability function

Use one canonical internal authorization helper. Adapt names/types to your existing `app_capabilities`, `role_grants`, `role_capabilities`, and tenant membership model.

```
create or replace function private.require_capability(
  p_capability text,
  p_tenant_id uuid default null,
  p_customer_id uuid default null
)
returns void
language plpgsql
security definer
set search_path = ''
as $$
declare
```

```
  v_actor_id uuid := auth.uid();
  v_allowed boolean;
begin
  if v_actor_id is null then
    raise exception using
      errcode = '42501',
      message = 'Unauthenticated';
  end if;

  select exists (
    select 1
    from public.role_grants grant
    join public.role_capabilities capability
      on capability.role_id = grant.role_id
    join public.app_capabilities definition
      on definition.id = capability.capability_id
    where grant.user_id = v_actor_id
      and definition.key = p_capability
      and private.scope_allows(
        grant.scope_type,
        grant.scope_id,
        p_tenant_id,
        p_customer_id
      )
      and grant.revoked_at is null
      and (
        grant.expires_at is null
        or grant.expires_at > now()
      )
  )
  into v_allowed;

  if not coalesce(v_allowed, false) then
    raise exception using
      errcode = '42501',
      message = 'Forbidden',
      detail = format(
        'Missing capability %s for requested scope',
        p_capability
      );
  end if;
end;
$$;

revoke all on function private.require_capability(text, uuid, uuid) from public;
```

Executive overview RPC

```
create or replace function public.ops_executive_overview(
  p_range_start timestamptz,
  p_range_end timestamptz
)
returns jsonb
language plpgsql
security definer
```

```
set search_path = ''
as $$
declare
  v_result jsonb;
begin
  perform private.require_capability('ops:executive:read');

  select jsonb_build_object(
    'data', jsonb_build_object(
      'tenants', (
        select jsonb_build_object(
          'total', count(*),
          'healthy', count(*) filter (
            where health_band = 'healthy'
          ),
          'at_risk', count(*) filter (
            where health_band in ('at_risk', 'critical')
          )
        )
        from ops.tenant_health_snapshots
      ),

      'approvals', (
        select jsonb_build_object(
          'pending', count(*) filter (
            where status in ('pending', 'awaiting_approval')
          ),
          'overdue', count(*) filter (
            where due_at < now()
              and status in ('pending', 'awaiting_approval')
          )
        )
        from public.approval_request
      ),

      'support', (
        select jsonb_build_object(
          'open_p0', count(*) filter (
            where severity = 'p0'
              and status not in ('resolved', 'closed')
          ),
          'open_p1', count(*) filter (
            where severity = 'p1'
              and status not in ('resolved', 'closed')
          )
        )
        from public.support_tickets
      ),

      'projection_status', (
        select coalesce(
          jsonb_agg(
            jsonb_build_object(
              'projection', projection_name,
              'status', status,
              'lagSeconds', lag_seconds,
```

```
              'projectedAt', projected_at
            )
            order by projection_name
          ),
          '[]'::jsonb
        )
        from ops.projection_watermarks
      )
    ),

    'meta', jsonb_build_object(
      'generatedAt', now(),
      'freshness', 'recent',
      'authority', 'derived',
      'rangeStart', p_range_start,
      'rangeEnd', p_range_end
    )
  )
  into v_result;

  return v_result;
end;
$$;

revoke all on function public.ops_executive_overview(
  timestamptz,
  timestamptz
) from public;

grant execute on function public.ops_executive_overview(
  timestamptz,
  timestamptz
) to authenticated;
```

Tenant overview RPC

```
create or replace function public.ops_tenant_overview(
  p_tenant_id uuid
)
returns jsonb
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_result jsonb;
begin
  perform private.require_capability(
    'ops:tenant:read',
    p_tenant_id,
    null
  );

  select jsonb_build_object(
    'data', jsonb_build_object(
```

```
      'tenant', (
        select jsonb_build_object(
          'id', school.id,
          'name', school.name,
          'rolloutState', rollout.rollout_state,
          'plan', plan.plan_key,
          'ssoRequired', coalesce(sso.required, false),
          'health', jsonb_build_object(
            'band', health.health_band,
            'score', health.overall_score,
            'drivers', health.top_drivers,
            'recommendedActions', health.recommended_actions,
            'computedAt', health.computed_at
          )
        )
        from public.schools school
        left join public.tenant_rollout rollout
          on rollout.tenant_id = school.id
        left join public.tenant_plan plan
          on plan.tenant_id = school.id
        left join public.tenant_sso_policy sso
          on sso.tenant_id = school.id
        left join ops.tenant_health_snapshots health
          on health.tenant_id = school.id
        where school.id = p_tenant_id
      ),

      'blockers', (
        select coalesce(
          jsonb_agg(
            jsonb_build_object(
              'type', item.item_type,
              'severity', item.severity,
              'title', item.title,
              'summary', item.summary,
              'dueAt', item.due_at,
              'status', item.status,
              'recommendedAction', item.recommended_action
            )
            order by
              case item.severity
                when 'p0' then 1
                when 'p1' then 2
                when 'p2' then 3
                else 4
              end,
              item.due_at nulls last
          ),
          '[]'::jsonb
        )
        from ops.operations_inbox_items item
        where item.tenant_id = p_tenant_id
          and item.status in ('open', 'acknowledged', 'blocked')
      )
    ),
```

```
    'meta', jsonb_build_object(
      'generatedAt', now(),
      'tenantId', p_tenant_id,
      'authority', 'derived',
      'freshness', coalesce(
        (
          select freshness
          from ops.tenant_health_snapshots
          where tenant_id = p_tenant_id
        ),
        'unknown'
      )
    )
  )
  into v_result;

  if v_result->'data'->'tenant' is null then
    raise exception using
      errcode = 'P0002',
      message = 'Tenant not found';
  end if;

  return v_result;
end;
$$;

revoke all on function public.ops_tenant_overview(uuid) from public;
grant execute on function public.ops_tenant_overview(uuid) to authenticated;
```

Operations inbox RPC

```
create or replace function public.ops_operations_inbox(
  p_tenant_id uuid default null,
  p_owner_id uuid default null,
  p_severity text default null,
  p_limit integer default 50,
  p_offset integer default 0
)
returns jsonb
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_limit integer := greatest(1, least(coalesce(p_limit, 50), 200));
  v_offset integer := greatest(0, coalesce(p_offset, 0));
begin
  perform private.require_capability(
    'ops:inbox:read',
    p_tenant_id,
    null
  );

  return jsonb_build_object(
    'data', coalesce(
```

```
      (
        select jsonb_agg(
          jsonb_build_object(
            'id', item.id,
            'type', item.item_type,
            'severity', item.severity,
            'status', item.status,
            'title', item.title,
            'summary', item.summary,
            'tenantId', item.tenant_id,
            'customerId', item.customer_id,
            'ownerId', item.owner_id,
            'dueAt', item.due_at,
            'sourceType', item.source_type,
            'sourceId', item.source_id,
            'recommendedAction', item.recommended_action,
            'correlationId', item.correlation_id,
            'createdAt', item.created_at
          )
          order by
            case item.severity
              when 'p0' then 1
              when 'p1' then 2
              when 'p2' then 3
              else 4
            end,
            item.due_at nulls last,
            item.created_at asc
        )
        from (
          select *
          from ops.operations_inbox_items
          where status in ('open', 'acknowledged', 'blocked')
            and (p_tenant_id is null or tenant_id = p_tenant_id)
            and (p_owner_id is null or owner_id = p_owner_id)
            and (p_severity is null or severity = p_severity)
          order by
            case severity
              when 'p0' then 1
              when 'p1' then 2
              when 'p2' then 3
              else 4
            end,
            due_at nulls last,
            created_at asc
          limit v_limit
          offset v_offset
        ) item
      ),
      '[]'::jsonb
    ),
    'meta', jsonb_build_object(
      'generatedAt', now(),
      'limit', v_limit,
      'offset', v_offset,
      'freshness', 'recent',
```

```
      'authority', 'derived'
    )
  );
end;
$$;

revoke all on function public.ops_operations_inbox(
  uuid,
  uuid,
  text,
  integer,
  integer
) from public;

grant execute on function public.ops_operations_inbox(
  uuid,
  uuid,
  text,
  integer,
  integer
) to authenticated;
```

React CQRS component library

Query client configuration

```
import {
  QueryClient,
  QueryClientProvider,
} from "@tanstack/react-query";

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 60_000,
      gcTime: 10 * 60_000,
      retry: 2,
      refetchOnWindowFocus: true,
      refetchOnReconnect: true,
    },
    mutations: {
      retry: 0,
    },
  },
});

export function OpsAppProviders({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <QueryClientProvider client={queryClient}>
      {children}
```

```
    </QueryClientProvider>
  );
}
```

Capability-aware data hook

```
import { useQuery } from "@tanstack/react-query";
import { supabase } from "@/lib/supabase/client";

export function useTenantOverview(tenantId: string | null) {
  return useQuery({
    queryKey: ["ops", "tenant", tenantId],
    enabled: Boolean(tenantId),

    queryFn: async () => {
      const { data, error } = await supabase.rpc(
        "ops_tenant_overview",
        {
          p_tenant_id: tenantId,
        }
      );

      if (error) {
        throw normalizeOpsError(error);
      }

      return data as OpsEnvelope<TenantOverview>;
    },

    staleTime: 60_000,
  });
}
```

Projection-state component

```
type ProjectionStateProps = {
  freshness:
    | "live"
    | "recent"
    | "delayed"
    | "stale"
    | "failed"
    | "unknown";
  generatedAt?: string;
  sourceUpdatedAt?: string | null;
  coverage?: "complete" | "partial" | "sample" | "unavailable";
  onRefresh?: () => void;
};

export function ProjectionStateBadge({
  freshness,
  generatedAt,
  sourceUpdatedAt,
```

```
  coverage = "complete",
  onRefresh,
}: ProjectionStateProps) {
  const styles = {
    live: "border-emerald-200 bg-emerald-50 text-emerald-700",
    recent: "border-sky-200 bg-sky-50 text-sky-700",
    delayed: "border-amber-200 bg-amber-50 text-amber-800",
    stale: "border-orange-200 bg-orange-50 text-orange-800",
    failed: "border-rose-200 bg-rose-50 text-rose-700",
    unknown: "border-slate-200 bg-slate-50 text-slate-600",
  };

  return (
    <div className="flex flex-wrap items-center gap-2">
      <span
        className={`inline-flex rounded-full border px-2 py-1 text-xs font-semibold $
      >
        {freshness}
      </span>

      <span className="text-xs text-slate-500">
        Coverage: {coverage}
      </span>

      {generatedAt && (
        <span className="text-xs text-slate-500">
          Calculated: {new Date(generatedAt).toLocaleString()}
        </span>
      )}

      {sourceUpdatedAt && (
        <span className="text-xs text-slate-500">
          Source updated: {new Date(sourceUpdatedAt).toLocaleString()}
        </span>
      )}

      {onRefresh && (
        <button
          onClick={onRefresh}
          className="rounded-md border border-slate-200 px-2 py-1 text-xs font-semibo
        >
          Refresh
        </button>
      )}
    </div>
  );
}
```

Sortable server-backed grid

```
type SortDirection = "asc" | "desc";

type SortState<T extends string> = {
  column: T;
  direction: SortDirection;
```

```
};

type OpsGridProps<T, TColumn extends string> = {
  rows: T[];
  columns: Array<{
    id: TColumn;
    label: string;
    render: (row: T) => React.ReactNode;
    sortable?: boolean;
  }>;
  sort: SortState<TColumn>;
  onSortChange: (next: SortState<TColumn>) => void;
  getRowKey: (row: T) => string;
  isLoading?: boolean;
  emptyMessage?: string;
};

export function OpsDataGrid<T, TColumn extends string>({
  rows,
  columns,
  sort,
  onSortChange,
  getRowKey,
  isLoading,
  emptyMessage = "No records match the current scope.",
}: OpsGridProps<T, TColumn>) {
  const toggleSort = (column: TColumn) => {
    onSortChange({
      column,
      direction:
        sort.column === column && sort.direction === "asc"
          ? "desc"
          : "asc",
    });
  };

  if (isLoading) {
    return (
      <div className="rounded-xl border border-slate-200 p-8 text-sm text-slate-500">
        Loading operational records…
      </div>
    );
  }

  if (!rows.length) {
    return (
      <div className="rounded-xl border border-dashed border-slate-300 p-8 text-sm te
        {emptyMessage}
      </div>
    );
  }

  return (
    <div className="overflow-x-auto rounded-xl border border-slate-200">
      <table className="w-full min-w-[900px] border-collapse">
        <thead className="bg-slate-50">
```

```
          <tr>
            {columns.map((column) => (
              <th
                key={column.id}
                className="border-b border-slate-200 px-4 py-3 text-left text-xs font
              >
                {column.sortable ? (
                  <button
                    onClick={() => toggleSort(column.id)}
                    className="inline-flex items-center gap-1 hover:text-indigo-700"
                  >
                    {column.label}
                    {sort.column === column.id
                      ? sort.direction === "asc"
                        ? "↑"
                        : "↓"
                      : "↕"}
                  </button>
                ) : (
                  column.label
                )}
              </th>
            ))}
          </tr>
        </thead>

        <tbody>
          {rows.map((row) => (
            <tr
              key={getRowKey(row)}
              className="border-b border-slate-100 last:border-b-0 hover:bg-indigo-50
            >
              {columns.map((column) => (
                <td key={column.id} className="px-4 py-3 text-sm text-slate-700">
                  {column.render(row)}
                </td>
              ))}
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

Optimistic low-risk mutation hook

```
import { useMutation, useQueryClient } from "@tanstack/react-query";

type InboxItem = {
  id: string;
  status: "open" | "acknowledged" | "blocked" | "resolved";
  pendingSync?: boolean;
};
```

```
export function useAcknowledgeInboxItem() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: async (itemId: string) => {
      const { data, error } = await supabase.rpc(
        "ops_acknowledge_inbox_item",
        { p_item_id: itemId }
      );

      if (error) {
        throw normalizeOpsError(error);
      }

      return data as InboxItem;
    },

    onMutate: async (itemId) => {
      await queryClient.cancelQueries({
        queryKey: ["ops", "inbox"],
      });

      const previous = queryClient.getQueryData<InboxItem[]>([
        "ops",
        "inbox",
      ]);

      queryClient.setQueryData<InboxItem[]>(
        ["ops", "inbox"],
        (current = []) =>
          current.map((item) =>
            item.id === itemId
              ? {
                  ...item,
                  status: "acknowledged",
                  pendingSync: true,
                }
              : item
          )
      );

      return { previous };
    },

    onError: (_error, _itemId, context) => {
      queryClient.setQueryData(
        ["ops", "inbox"],
        context?.previous
      );
    },

    onSuccess: (serverItem) => {
      queryClient.setQueryData<InboxItem[]>(
        ["ops", "inbox"],
        (current = []) =>
          current.map((item) =>
```

```
            item.id === serverItem.id ? serverItem : item
          )
      );
    },

    onSettled: () => {
      queryClient.invalidateQueries({
        queryKey: ["ops", "inbox"],
      });
    },
  });
}
```

TanStack Query recommends using `onMutate` to update cached state optimistically, returning context for rollback on `onError`, and reconciling against the server after the request settles.[7] [5]

Live simulator dashboard

Build a local-only simulator first. It should visually demonstrate projection lag, event rate, worker throughput, failures, retries, cache invalidation, and stale-data recovery.

```
type SimulatorState = {
  eventRatePerMinute: number;
  workerCapacityPerMinute: number;
  transientFailureRate: number;
  projectionLagSeconds: number;
  cachedTenantOverview: boolean;
  cacheTtlSeconds: number;
  forceProjectionFailure: boolean;
  activeEvents: number;
  processedEvents: number;
  deadLetterEvents: number;
  invalidations: number;
  lastProjectionAt: string | null;
};

function calculateSimulator(state: SimulatorState) {
  const backlogDelta =
    state.eventRatePerMinute - state.workerCapacityPerMinute;

  const projectedBacklog = Math.max(
    0,
    state.activeEvents + backlogDelta
  );

  const estimatedLagSeconds =
    state.workerCapacityPerMinute <= 0
      ? Number.POSITIVE_INFINITY
      : Math.round(
          (projectedBacklog / state.workerCapacityPerMinute) * 60
        );

  const freshness =
    state.forceProjectionFailure
```

```
      ? "failed"
      : estimatedLagSeconds <= 15
      ? "live"
      : estimatedLagSeconds <= 60
      ? "recent"
      : estimatedLagSeconds <= 300
      ? "delayed"
      : "stale";

  return {
    projectedBacklog,
    estimatedLagSeconds,
    freshness,
    projectedThroughput:
      state.workerCapacityPerMinute *
      (1 - state.transientFailureRate / 100),
  };
}
```

Simulator controls:

- Event rate per minute
- Projection worker capacity
- Failure percentage
- Retry delay
- Projection rebuild mode:
  - incremental
  - full rebuild
  - blue/green rebuild
- Cache TTL
- Invalidation on/off
- Force source write
- Force worker failure
- Force stale projection
- Replay dead-letter event
- Reset scenario

Simulator cards:

Card	What it shows
Event backlog	Pending/processing/failed/dead-letter events
Projection lag	Estimated seconds behind authoritative state
Cache status	Hit/miss, key, TTL, invalidation timestamp
Freshness	Live/recent/delayed/stale/failed

Card	What it shows
Worker throughput	Processed events/minute after failures
Data mismatch	Source version vs projection version
Recommended action	Scale worker, replay, invalidate, rebuild, investigate
Safety state	Whether mutations are allowed or restricted

Tests to require

```
Projection pipeline:
- Domain command writes authoritative row and outbox event atomically.
- Duplicate event delivery does not duplicate projection effects.
- Failed event retries with bounded backoff.
- Repeated failure moves event to dead letter.
- Rebuilding projection from events produces same result as incremental projection.
- Projection watermark correctly reports lag/failure.

Cache:
- Tenant rollout event invalidates tenant, rollout, and executive keys only.
- Support event invalidates support and tenant health keys.
- Unrelated tenant event does not invalidate another tenant's detail cache.
- Stale response visibly shows stale metadata.
- Sensitive results never enter a global shared client cache.

Authorization:
- Operator lacking `ops:tenant:read` cannot call tenant overview.
- Operator with Tenant A scope cannot request Tenant B data.
- Direct tables are not used as a fallback when RPC is forbidden.
- `SECURITY DEFINER` functions have safe search_path and scope checks.
- High-risk action cannot execute before approval.

UI:
- Optimistic acknowledgement rolls back after server rejection.
- High-risk action waits for server confirmation.
- Projection failure state shows last known data plus failure metadata.
- Forbidden state reveals no protected fields.
- Evidence drawer shows source, freshness, and metric definition.
```

Recommended implementation order

1. Add outbox, receipts, watermarks, invalidations, and rebuild-run records.
2. Implement tenant-health and operations-inbox projections.
3. Add capability-checked RPCs for executive, tenant, inbox, and rollout views.
4. Build the React query/data layer and shared projection-state components.
5. Add targeted invalidation events and minimal realtime invalidation messages.
6. Build the live projection/cache simulator using local data.
7. Add incident/SLO projections and command-center screens.

8. Add access-review and security-boundary workspace.
9. Harden grants, GraphQL exposure, and `SECURITY DEFINER` function inventory before broad operator rollout.

This gives Semester a resilient read side: authoritative writes remain protected and auditable, operators receive fast coherent views, and stale or delayed data is made visible rather than silently trusted.

⁂

1. https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs
2. https://event-driven.io/en/projections_and_read_models_in_event_driven_architecture/
3. https://supabase.com/docs/guides/database/postgres/row-level-security
4. https://supabase.com/docs/guides/database/functions
5. https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates
6. https://tanstack.com/query/latest
7. https://docs.amplify.aws/react/frontend/data/optimistic-ui/
8. https://appscale.blog/en/blog/microservices-pattern-cache-aside-cqrs-2026
9. https://makerkit.dev/blog/tutorials/supabase-rls-best-practices
10. https://oneuptime.com/blog/post/2026-01-15-react-optimistic-updates-react-query/view
11. https://dev.to/thebenforce/lock-down-your-data-implement-row-level-security-policies-in-supabase-sql-4p82
12. https://www.reddit.com/r/reactjs/comments/1mgc0ch/i_built_a_package_to_simplify_optimistic_updates/
13. https://blog.entrostat.com/supabase-rls-functions/
14. https://tanstack.com/query/latest/docs/framework/optimistic-updates
15. https://supabase.com/features/row-level-security

Interactive CQRS simulator: model event lag, projection rebuilds, and cache invalidation bottlenecks under high write throughput

Computer
CQRS Component Library: build sortable React/Vue data grids, optimistic UI mutations, and offline sync fallbacks
Computer
Show me the SQL for the transactional outbox and worker loop
How should the React UI handle projection lag and optimistic updates
Write the SQL schema for the versioned read-model views

Below is the implementation pack for a **Semester Operations CQRS simulator and read-model layer**: transactional outbox SQL, worker claim loop, projection/version schema, cache invalidation, React lag-aware UI behavior, optimistic mutation rules, and offline fallback boundaries.

The critical architectural rule is: authoritative domain writes remain in your existing tenant, billing, rollout, support, GTM, trust, and compliance models; the operations UI reads purpose-built projections with explicit freshness metadata. CQRS separates read and write models precisely for cases where the query shape differs from the transactional model, and event-driven projections can update the read side from write-side events. [1] [2]

Transactional outbox SQL

Use an outbox so a successful write and its projection event are committed in the **same transaction**. A worker processes the outbox later and updates read models idempotently.

Save as a new migration after reviewing your existing `private.domain_outbox_events` and related migration history. Your production database already appears to have an outbox/event foundation, so reconcile names before adding duplicate tables. [3]

```
create schema if not exists ops;
create schema if not exists private;

revoke all on schema ops from anon, authenticated;
revoke all on schema private from anon, authenticated;

do $$
begin
  create type private.outbox_status as enum (
    'pending',
```

```
    'processing',
    'processed',
    'failed',
    'dead_letter'
  );
exception
  when duplicate_object then null;
end $$;

create table if not exists private.domain_outbox_events (
  id uuid primary key default gen_random_uuid(),

  event_type text not null,
  event_version integer not null default 1,

  aggregate_type text not null,
  aggregate_id uuid not null,

  tenant_id uuid,
  customer_id uuid,

  correlation_id text not null,
  causation_id uuid,

  actor_type text not null default 'system'
    check (actor_type in ('user', 'service', 'system', 'worker')),

  actor_id uuid,

  payload jsonb not null default '{}'::jsonb,

  -- Persisted event hash prevents duplicate logical events.
  payload_hash text not null,

  occurred_at timestamptz not null default now(),

  status private.outbox_status not null default 'pending',

  attempts integer not null default 0
    check (attempts >= 0),

  available_at timestamptz not null default now(),

  locked_at timestamptz,
  locked_by text,

  processed_at timestamptz,
  last_error text,

  created_at timestamptz not null default now(),

  unique (
    event_type,
    aggregate_type,
    aggregate_id,
    payload_hash
```

```
  )
);

create index if not exists domain_outbox_pending_idx
  on private.domain_outbox_events (
    status,
    available_at,
    occurred_at
  )
  where status in ('pending', 'failed');

create index if not exists domain_outbox_tenant_idx
  on private.domain_outbox_events (
    tenant_id,
    occurred_at desc
  )
  where tenant_id is not null;

create index if not exists domain_outbox_correlation_idx
  on private.domain_outbox_events (
    correlation_id,
    occurred_at
  );

create index if not exists domain_outbox_dead_letter_idx
  on private.domain_outbox_events (
    status,
    occurred_at desc
  )
  where status = 'dead_letter';
```

Event writer function

This function should be called from a server-side transactional command—not directly from a client.

```
create or replace function private.record_domain_event(
  p_event_type text,
  p_event_version integer,
  p_aggregate_type text,
  p_aggregate_id uuid,
  p_tenant_id uuid,
  p_customer_id uuid,
  p_correlation_id text,
  p_causation_id uuid,
  p_actor_type text,
  p_actor_id uuid,
  p_payload jsonb
)
returns uuid
language plpgsql
security definer
set search_path = ''
as $$
```

```
declare
  v_event_id uuid;
  v_payload_hash text;
begin
  if p_event_type is null or length(trim(p_event_type)) = 0 then
    raise exception 'event_type is required';
  end if;

  if p_aggregate_type is null or length(trim(p_aggregate_type)) = 0 then
    raise exception 'aggregate_type is required';
  end if;

  if p_aggregate_id is null then
    raise exception 'aggregate_id is required';
  end if;

  if p_correlation_id is null or length(trim(p_correlation_id)) = 0 then
    raise exception 'correlation_id is required';
  end if;

  v_payload_hash := encode(
    extensions.digest(
      coalesce(p_payload::text, '{}'),
      'sha256'
    ),
    'hex'
  );

  insert into private.domain_outbox_events (
    event_type,
    event_version,
    aggregate_type,
    aggregate_id,
    tenant_id,
    customer_id,
    correlation_id,
    causation_id,
    actor_type,
    actor_id,
    payload,
    payload_hash
  )
  values (
    p_event_type,
    coalesce(p_event_version, 1),
    p_aggregate_type,
    p_aggregate_id,
    p_tenant_id,
    p_customer_id,
    p_correlation_id,
    p_causation_id,
    coalesce(p_actor_type, 'system'),
    p_actor_id,
    coalesce(p_payload, '{}'::jsonb),
    v_payload_hash
  )
```

```
  on conflict (
    event_type,
    aggregate_type,
    aggregate_id,
    payload_hash
  )
  do update set
    occurred_at = excluded.occurred_at
  returning id into v_event_id;

  return v_event_id;
end;
$$;

revoke all on function private.record_domain_event(
  text, integer, text, uuid, uuid, uuid, text, uuid, text, uuid, jsonb
) from public;
```

Worker claim loop

Worker queue function

Use `FOR UPDATE SKIP LOCKED` so multiple workers can claim events without processing the same event concurrently.

```
create or replace function private.claim_outbox_events(
  p_worker_id text,
  p_limit integer default 50,
  p_lock_seconds integer default 60
)
returns table (
  id uuid,
  event_type text,
  event_version integer,
  aggregate_type text,
  aggregate_id uuid,
  tenant_id uuid,
  customer_id uuid,
  correlation_id text,
  causation_id uuid,
  actor_type text,
  actor_id uuid,
  payload jsonb,
  occurred_at timestamptz,
  attempts integer
)
language plpgsql
security definer
set search_path = ''
as $$
begin
  if p_worker_id is null or length(trim(p_worker_id)) = 0 then
    raise exception 'worker id is required';
  end if;
```

```
  return query
  with candidates as (
    select event.id
    from private.domain_outbox_events event
    where
      (
        event.status in ('pending', 'failed')
        and event.available_at <= now()
      )
      or (
        event.status = 'processing'
        and event.locked_at < now() - make_interval(secs => p_lock_seconds)
      )
    order by event.occurred_at asc
    limit greatest(1, least(coalesce(p_limit, 50), 250))
    for update skip locked
  ),
  claimed as (
    update private.domain_outbox_events event
    set
      status = 'processing',
      locked_at = now(),
      locked_by = p_worker_id,
      attempts = event.attempts + 1
    from candidates
    where event.id = candidates.id
    returning
      event.id,
      event.event_type,
      event.event_version,
      event.aggregate_type,
      event.aggregate_id,
      event.tenant_id,
      event.customer_id,
      event.correlation_id,
      event.causation_id,
      event.actor_type,
      event.actor_id,
      event.payload,
      event.occurred_at,
      event.attempts
  )
  select * from claimed;
end;
$$;

revoke all on function private.claim_outbox_events(
  text,
  integer,
  integer
) from public;
```

Mark processed

```
create or replace function private.mark_outbox_event_processed(
  p_event_id uuid,
  p_worker_id text
)
returns void
language plpgsql
security definer
set search_path = ''
as $$
begin
  update private.domain_outbox_events
  set
    status = 'processed',
    processed_at = now(),
    locked_at = null,
    locked_by = null,
    last_error = null
  where id = p_event_id
    and status = 'processing'
    and locked_by = p_worker_id;

  if not found then
    raise exception 'event was not owned by worker or is not processing';
  end if;
end;
$$;

revoke all on function private.mark_outbox_event_processed(
  uuid,
  text
) from public;
```

Retry or dead-letter

```
create or replace function private.mark_outbox_event_failed(
  p_event_id uuid,
  p_worker_id text,
  p_error text,
  p_max_attempts integer default 8
)
returns void
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_attempts integer;
  v_delay_seconds integer;
begin
  select attempts
    into v_attempts
  from private.domain_outbox_events
```

```
  where id = p_event_id
    and status = 'processing'
    and locked_by = p_worker_id
  for update;

  if v_attempts is null then
    raise exception 'event was not owned by worker or is not processing';
  end if;

  -- Exponential backoff: 2^attempts seconds, capped at 1 hour.
  v_delay_seconds := least(
    power(2, least(v_attempts, 12))::integer,
    3600
  );

  update private.domain_outbox_events
  set
    status = case
      when v_attempts >= p_max_attempts then 'dead_letter'::private.outbox_status
      else 'failed'::private.outbox_status
    end,
    available_at = case
      when v_attempts >= p_max_attempts then now()
      else now() + make_interval(secs => v_delay_seconds)
    end,
    locked_at = null,
    locked_by = null,
    last_error = left(coalesce(p_error, 'unknown projection error'), 4000)
  where id = p_event_id;
end;
$$;

revoke all on function private.mark_outbox_event_failed(
  uuid,
  text,
  text,
  integer
) from public;
```

Idempotent receipts

Every projection records an event receipt before applying the projection update.

```
create table if not exists private.projection_event_receipts (
  projection_name text not null,
  event_id uuid not null
    references private.domain_outbox_events(id)
    on delete cascade,

  projection_version integer not null default 1,

  processed_at timestamptz not null default now(),
  result_hash text,

  primary key (
```

```
    projection_name,
    event_id
  )
);

create or replace function private.claim_projection_receipt(
  p_projection_name text,
  p_event_id uuid,
  p_projection_version integer
)
returns boolean
language plpgsql
security definer
set search_path = ''
as $$
begin
  insert into private.projection_event_receipts (
    projection_name,
    event_id,
    projection_version
  )
  values (
    p_projection_name,
    p_event_id,
    p_projection_version
  )
  on conflict do nothing;

  return found;
end;
$$;

revoke all on function private.claim_projection_receipt(
  text,
  uuid,
  integer
) from public;
```

Projection worker logic

Run the worker from a server-only environment: Supabase Edge Function, protected Vercel cron/job, dedicated service, or CI-triggered worker. Never execute it from a browser.

```
type OutboxEvent = {
  id: string;
  event_type: string;
  event_version: number;
  aggregate_type: string;
  aggregate_id: string;
  tenant_id: string | null;
  customer_id: string | null;
  correlation_id: string;
  causation_id: string | null;
  actor_type: "user" | "service" | "system" | "worker";
```

```
  actor_id: string | null;
  payload: Record<string, unknown>;
  occurred_at: string;
  attempts: number;
};

const WORKER_ID = `ops-projection-${crypto.randomUUID()}`;

export async function runProjectionWorker() {
  const events = await claimEvents({
    workerId: WORKER_ID,
    limit: 50,
    lockSeconds: 90,
  });

  for (const event of events) {
    try {
      await projectEvent(event);

      await markProcessed({
        eventId: event.id,
        workerId: WORKER_ID,
      });
    } catch (error) {
      await markFailed({
        eventId: event.id,
        workerId: WORKER_ID,
        error: sanitizeError(error),
        maxAttempts: 8,
      });
    }
  }

  return {
    workerId: WORKER_ID,
    claimed: events.length,
  };
}

async function projectEvent(event: OutboxEvent) {
  switch (event.event_type) {
    case "tenant.rollout.transitioned":
    case "tenant.feature.changed":
    case "tenant.plan.changed":
    case "tenant.sso.changed":
    case "integration.sync.completed":
    case "integration.sync.failed":
    case "support.ticket.updated":
    case "pilot.metric.recorded":
    case "renewal.opportunity.updated":
      if (event.tenant_id) {
        await updateTenantHealth(event.tenant_id, event);
        await updateTenantOverview(event.tenant_id, event);
        await invalidateTenantReadModels(event);
      }
      break;
```

```
    case "approval.request.created":
    case "approval.decision.recorded":
      await updateApprovalInbox(event);
      await invalidateApprovalReadModels(event);
      break;

    case "incident.declared":
    case "incident.updated":
    case "incident.resolved":
    case "slo.snapshot.recorded":
      await updateIncidentAndSloReadModels(event);
      await invalidateIncidentReadModels(event);
      break;

    case "compliance.evidence.expired":
    case "trust.artifact.review_due":
      await updateComplianceReadModels(event);
      await invalidateComplianceReadModels(event);
      break;

    default:
      await recordIgnoredEvent(event);
  }
}
```

Versioned read-model schema

Use projection versions so schema changes do not corrupt dashboards or force unsafe in-place rewrites.

```
create table if not exists ops.read_model_registry (
  model_name text not null,
  version integer not null,

  source_event_types text[] not null,
  description text not null,

  refresh_strategy text not null
    check (
      refresh_strategy in (
        'event_driven',
        'scheduled',
        'hybrid',
        'live_query'
      )
    ),

  owner_role text not null,

  current boolean not null default false,

  created_at timestamptz not null default now(),
  retired_at timestamptz,
```

```
  primary key (model_name, version)
);

create unique index if not exists read_model_registry_one_current_idx
  on ops.read_model_registry (model_name)
  where current = true;

create table if not exists ops.projection_watermarks (
  projection_name text primary key,

  projection_version integer not null default 1,

  last_event_id uuid,
  last_event_at timestamptz,

  source_updated_at timestamptz,
  projected_at timestamptz not null default now(),

  status text not null default 'healthy'
    check (
      status in ('healthy', 'delayed', 'failed', 'rebuilding')
    ),

  lag_seconds integer,
  last_error text,

  rebuild_requested_at timestamptz,
  rebuilt_at timestamptz
);

create table if not exists ops.tenant_overview_v1 (
  tenant_id uuid primary key,

  tenant_name text not null,

  rollout_state text,
  plan_key text,
  sso_required boolean,

  active_users_30d integer,
  activation_rate numeric(8,4),
  workflow_completion_rate numeric(8,4),

  open_p0_tickets integer not null default 0,
  open_p1_tickets integer not null default 0,

  integration_health text,
  security_posture text,
  compliance_status text,

  health_score integer,
  health_band text,

  top_drivers jsonb not null default '[]'::jsonb,
  recommended_actions jsonb not null default '[]'::jsonb,
```

```
  source_updated_at timestamptz,
  computed_at timestamptz not null default now(),

  model_version integer not null default 1,

  freshness text not null default 'recent'
    check (
      freshness in ('live', 'recent', 'delayed', 'stale', 'failed')
    )
);

create index if not exists tenant_overview_v1_health_idx
  on ops.tenant_overview_v1 (
    health_band,
    computed_at desc
  );

create index if not exists tenant_overview_v1_rollout_idx
  on ops.tenant_overview_v1 (
    rollout_state,
    computed_at desc
  );

create table if not exists ops.operations_inbox_v1 (
  id uuid primary key default gen_random_uuid(),

  item_type text not null,
  severity text not null
    check (severity in ('p0', 'p1', 'p2', 'p3')),

  status text not null
    check (
      status in (
        'open',
        'acknowledged',
        'blocked',
        'resolved',
        'dismissed'
      )
    ),

  tenant_id uuid,
  customer_id uuid,

  source_type text not null,
  source_id uuid,

  title text not null,
  summary text,

  owner_id uuid,
  due_at timestamptz,

  evidence jsonb not null default '{}'::jsonb,
  recommended_action jsonb not null default '{}'::jsonb,
```

```
  correlation_id text,

  source_updated_at timestamptz,
  computed_at timestamptz not null default now(),

  model_version integer not null default 1
);

create index if not exists operations_inbox_v1_active_idx
  on ops.operations_inbox_v1 (
    severity,
    due_at,
    created_at
  )
  where status in ('open', 'acknowledged', 'blocked');

create index if not exists operations_inbox_v1_tenant_idx
  on ops.operations_inbox_v1 (
    tenant_id,
    status,
    severity
  )
  where tenant_id is not null;

create table if not exists ops.slo_overview_v1 (
  service_key text primary key,

  service_name text not null,
  criticality text not null,

  objective numeric(8,6) not null,
  compliance_ratio numeric(8,6),

  error_budget_remaining numeric(8,6),

  burn_rate_1h numeric(10,4),
  burn_rate_6h numeric(10,4),
  burn_rate_24h numeric(10,4),

  release_policy text not null,
  release_restriction text,

  active_incident_count integer not null default 0,

  source_updated_at timestamptz,
  computed_at timestamptz not null default now(),

  model_version integer not null default 1,

  freshness text not null default 'recent'
    check (
      freshness in ('live', 'recent', 'delayed', 'stale', 'failed')
    )
);
```

```
create table if not exists ops.incident_overview_v1 (
  incident_id uuid primary key,

  severity text not null,
  status text not null,

  title text not null,

  started_at timestamptz not null,
  detected_at timestamptz,

  commander_id uuid,
  technical_lead_id uuid,
  communications_lead_id uuid,

  affected_tenant_count integer not null default 0,
  affected_user_count integer,

  customer_impact_summary text,

  next_update_at timestamptz,

  linked_release_ids jsonb not null default '[]'::jsonb,
  linked_slo_keys text[] not null default '{}',
  linked_runbooks jsonb not null default '[]'::jsonb,

  source_updated_at timestamptz,
  computed_at timestamptz not null default now(),

  model_version integer not null default 1
);
```

Blue/green projection rebuild

For a breaking projection change:

```
tenant_overview_v1 remains live
        ↓
Build tenant_overview_v2 from source events/domain tables
        ↓
Run parity checks:
- row count
- tenant ID coverage
- aggregate comparison
- randomized record comparison
- authorization contract tests
        ↓
Set registry current=true for v2
        ↓
Switch RPC to v2
        ↓
Retain v1 for rollback window
        ↓
Retire v1 after evidence-backed review
```

```
begin;

update ops.read_model_registry
set current = false
where model_name = 'tenant_overview'
  and current = true;

update ops.read_model_registry
set current = true
where model_name = 'tenant_overview'
  and version = 2;

commit;
```

Never point the UI directly at a new projection table. Keep RPC names stable and change the internal implementation after parity verification.

Capability-checked query RPCs

Canonical capability check

Adapt this to your existing role/capability model. Semester already has `app_roles`, `app_capabilities`, `role_grants`, `role_capabilities`, and scope-aware authorization concepts. [4]

```
create or replace function private.require_capability(
  p_capability_key text,
  p_tenant_id uuid default null,
  p_customer_id uuid default null
)
returns void
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_actor_id uuid := auth.uid();
  v_allowed boolean := false;
begin
  if v_actor_id is null then
    raise exception using
      errcode = '42501',
      message = 'Unauthenticated';
  end if;

  select exists (
    select 1
    from public.role_grants grant
    join public.role_capabilities role_capability
      on role_capability.role_id = grant.role_id
    join public.app_capabilities capability
      on capability.id = role_capability.capability_id
    where grant.user_id = v_actor_id
      and capability.key = p_capability_key
```

```
      and grant.revoked_at is null
      and (
        grant.expires_at is null
        or grant.expires_at > now()
      )
      and private.scope_allows(
        grant.scope_type,
        grant.scope_id,
        p_tenant_id,
        p_customer_id
      )
  )
  into v_allowed;

  if not v_allowed then
    raise exception using
      errcode = '42501',
      message = 'Forbidden',
      detail = format(
        'Missing capability "%s" for requested scope',
        p_capability_key
      );
  end if;
end;
$$;

revoke all on function private.require_capability(
  text,
  uuid,
  uuid
) from public;
```

Tenant overview

```
create or replace function public.ops_tenant_overview(
  p_tenant_id uuid
)
returns jsonb
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_result jsonb;
begin
  perform private.require_capability(
    'ops:tenant:read',
    p_tenant_id,
    null
  );

  select jsonb_build_object(
    'data',
    jsonb_build_object(
      'tenantId', overview.tenant_id,
```

```
      'tenantName', overview.tenant_name,
      'rolloutState', overview.rollout_state,
      'planKey', overview.plan_key,
      'ssoRequired', overview.sso_required,

      'adoption', jsonb_build_object(
        'activeUsers30d', overview.active_users_30d,
        'activationRate', overview.activation_rate,
        'workflowCompletionRate', overview.workflow_completion_rate
      ),

      'support', jsonb_build_object(
        'openP0Tickets', overview.open_p0_tickets,
        'openP1Tickets', overview.open_p1_tickets
      ),

      'health', jsonb_build_object(
        'score', overview.health_score,
        'band', overview.health_band,
        'topDrivers', overview.top_drivers,
        'recommendedActions', overview.recommended_actions
      )
    ),

    'meta',
    jsonb_build_object(
      'generatedAt', now(),
      'sourceUpdatedAt', overview.source_updated_at,
      'computedAt', overview.computed_at,
      'freshness', overview.freshness,
      'authority', 'derived',
      'modelVersion', overview.model_version,
      'correlationId', gen_random_uuid()::text
    ),

    'permissions',
    jsonb_build_object(
      'canView', true,
      'canExport', private.has_capability(
        auth.uid(),
        'ops:tenant:export',
        p_tenant_id
      ),
      'allowedActions', private.allowed_ops_actions(
        auth.uid(),
        p_tenant_id
      )
    )
  )
  into v_result
  from ops.tenant_overview_v1 overview
  where overview.tenant_id = p_tenant_id;

  if v_result is null then
    raise exception using
      errcode = 'P0002',
```

```
      message = 'Tenant read model not found';
  end if;

  return v_result;
end;
$$;

revoke all on function public.ops_tenant_overview(uuid) from public;
grant execute on function public.ops_tenant_overview(uuid) to authenticated;
```

Projection dashboard RPC

```
create or replace function public.ops_projection_dashboard()
returns jsonb
language plpgsql
security definer
set search_path = ''
as $$
begin
  perform private.require_capability('ops:projection:read');

  return jsonb_build_object(
    'data',
    coalesce(
      (
        select jsonb_agg(
          jsonb_build_object(
            'projectionName', watermark.projection_name,
            'version', watermark.projection_version,
            'status', watermark.status,
            'lagSeconds', watermark.lag_seconds,
            'sourceUpdatedAt', watermark.source_updated_at,
            'projectedAt', watermark.projected_at,
            'lastError', watermark.last_error,
            'rebuildRequestedAt', watermark.rebuild_requested_at,
            'rebuiltAt', watermark.rebuilt_at
          )
          order by watermark.projection_name
        )
        from ops.projection_watermarks watermark
      ),
      '[]'::jsonb
    ),
    'meta',
    jsonb_build_object(
      'generatedAt', now(),
      'authority', 'derived',
      'freshness', 'recent'
    )
  );
end;
$$;
```

```
revoke all on function public.ops_projection_dashboard() from public;
grant execute on function public.ops_projection_dashboard() to authenticated;
```

Projection rebuild request

This does not rebuild immediately. It creates an approval-aware request.

```
create or replace function public.ops_request_projection_rebuild(
  p_projection_name text,
  p_mode text,
  p_reason text,
  p_correlation_id text
)
returns jsonb
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_actor_id uuid := auth.uid();
  v_request_id uuid;
begin
  perform private.require_capability('ops:projection:rebuild:request');

  if p_mode not in ('incremental', 'full', 'blue_green') then
    raise exception 'unsupported rebuild mode';
  end if;

  if length(trim(coalesce(p_reason, ''))) < 10 then
    raise exception 'rebuild reason must explain the operational need';
  end if;

  insert into ops.projection_rebuild_runs (
    projection_name,
    projection_version,
    requested_by,
    requested_reason,
    mode,
    status,
    correlation_id
  )
  values (
    p_projection_name,
    coalesce(
      (
        select projection_version
        from ops.projection_watermarks
        where projection_name = p_projection_name
      ),
      1
    ),
    v_actor_id,
    p_reason,
    p_mode,
    'queued',
```

```
    p_correlation_id
  )
  returning id into v_request_id;

  perform private.record_domain_event(
    'ops.projection.rebuild_requested',
    1,
    'projection_rebuild',
    v_request_id,
    null,
    null,
    p_correlation_id,
    null,
    'user',
    v_actor_id,
    jsonb_build_object(
      'projection_name', p_projection_name,
      'mode', p_mode,
      'reason', p_reason
    )
  );

  return jsonb_build_object(
    'requestId', v_request_id,
    'status', 'queued',
    'correlationId', p_correlation_id
  );
end;
$$;

revoke all on function public.ops_request_projection_rebuild(
  text,
  text,
  text,
  text
) from public;

grant execute on function public.ops_request_projection_rebuild(
  text,
  text,
  text,
  text
) to authenticated;
```

React: handling lag and optimistic updates

Lag-aware query state

```
type Freshness =
  | "live"
  | "recent"
  | "delayed"
  | "stale"
  | "failed";
```

```
function getLagPresentation(
  freshness: Freshness,
  lagSeconds?: number | null
) {
  if (freshness === "failed") {
    return {
      tone: "critical",
      label: "Projection failed",
      message: "Showing last known data. Do not use this view for high-risk decisions
    };
  }

  if (freshness === "stale") {
    return {
      tone: "warning",
      label: "Data is stale",
      message: `Projection is ${lagSeconds ?? "significantly"} seconds behind source
    };
  }

  if (freshness === "delayed") {
    return {
      tone: "warning",
      label: "Data delayed",
      message: `Projection lag: ${lagSeconds ?? "unknown"} seconds.`,
    };
  }

  return {
    tone: "normal",
    label: freshness === "live" ? "Live" : "Recently updated",
    message: "Read model is within its expected freshness window.",
  };
}
```

UI policy by freshness

Projection state	Read actions	Low-risk writes	High-risk writes
Live	Normal	Allowed	Allowed if approvals satisfied
Recent	Normal	Allowed	Allowed with normal guardrails
Delayed	Normal with badge	Allowed if direct command confirmation exists	Require live source preflight
Stale	Show last known state	Avoid if action depends on current state	Disable or force authoritative preflight
Failed	Show last known state + error	Disable unless independent direct command is safe	Disable
Rebuilding	Show version/freshness notice	Usually avoid	Disable unless break-glass procedure exists

High-risk action preflight

```
async function preflightHighRiskAction(input: {
  action: string;
  tenantId: string;
  expectedProjectionVersion?: number;
}) {
  const { data, error } = await supabase.rpc(
    "ops_preflight_controlled_action",
    {
      p_action: input.action,
      p_tenant_id: input.tenantId,
      p_expected_projection_version: input.expectedProjectionVersion ?? null,
    }
  );

  if (error) {
    throw normalizeOpsError(error);
  }

  return data as {
    allowed: boolean;
    requiresApproval: boolean;
    projectionFresh: boolean;
    sourceStateVersion: string;
    blockers: string[];
    impactPreview: Record<string, unknown>;
  };
}
```

Before enabling a destructive or high-impact button:

```
1. Fetch current authoritative preflight.
2. Verify the caller capability.
3. Verify approval request/decision state.
4. Verify source state version—not merely projection version.
5. Show exact impact and rollback plan.
6. Submit the controlled server command.
7. Wait for the server response.
8. Invalidate projections after the resulting event arrives.
```

Offline fallback boundaries

Operation	Offline support	Behavior
View cached dashboard	Yes, clearly labeled	Show timestamp, stale badge, no false “live” state
Save local filter/view	Yes	Store user preference locally or sync when online
Draft internal note	Yes, queued	Mark pending; sync with conflict detection
Acknowledge inbox item	Optional queued action	Only if backend supports idempotency and action is low risk
Approve/reject a high-risk request	No	Require fresh authenticated connection

Operation	Offline support	Behavior
Feature rollout	No	Require authoritative preflight
Refund/credit/billing action	No	Require fresh connection and approval
Break-glass access	No	Require live identity, reason, and audit write
Tenant offboarding/data export	No	Require live approval and legal-hold validation

CQRS simulator implementation prompt

Use this with Claude Code or a coding agent:

```
Build a local-first CQRS Projection Simulator inside the existing Semester app.

Purpose:
Let authorized internal operators understand how transactional writes become
read-model updates, how event backlogs create projection lag, how retries and
dead-letter events behave, how cache invalidations target specific read models,
and how UI freshness affects safe operational actions.

Do not connect the simulator to production data in its first version.

Build:
1. Transactional outbox simulation
2. Projection worker simulation
3. Projection watermark and lag simulation
4. Cache namespace/key invalidation simulation
5. Retry and exponential-backoff simulation
6. Dead-letter queue simulation
7. Full rebuild, incremental rebuild, and blue/green rebuild simulation
8. Read-model version switching simulation
9. Optimistic update and rollback simulation
10. Offline state/fallback simulation

Simulator controls:
- Events per minute
- Worker capacity per minute
- Worker count
- Failure rate
- Maximum retries
- Retry delay
- Cache TTL
- Invalidation enabled/disabled
- Source writes
- Projection rebuild mode
- Force stale projection
- Force worker failure
- Replay dead-letter event
- Reset state

Simulator output:
- Pending events
- Processing events
- Processed events
```

```
- Failed events
- Dead-letter events
- Projection lag
- Source version versus projection version
- Projection freshness
- Cache hit/miss rate
- Invalidations emitted
- Worker throughput
- Recommended operational action
- Whether low-risk and high-risk console actions are allowed

UI:
- Premium operations dashboard
- Timeline of source write → outbox → worker → projection → cache invalidation → UI r
- Sortable event queue
- Projection-watermark table
- Cache-key inspector
- Read-model version table
- Optimistic mutation panel with success/rollback states
- Offline fallback panel
- Export simulation state as JSON and Markdown

Add tests for:
- Duplicate event delivery
- Idempotent receipts
- Retry behavior
- Dead-letter behavior
- Cache invalidation targeting
- Projection freshness transitions
- Rebuild parity
- Optimistic rollback
- High-risk action disabled when projection is stale or failed
```

Build order

1. Add outbox and projection receipts only after reconciling existing private event/outbox structures.
2. Build `ops.projection_watermarks`.
3. Build the tenant-health projection first.
4. Build `ops_tenant_overview()` and `ops_projection_dashboard()` RPCs.
5. Build targeted cache invalidation.
6. Build the React freshness components and tenant screen.
7. Build the local simulator.
8. Add operations inbox projection.
9. Add incident/SLO projections.
10. Add blue/green rebuild and access-review UI.

This approach gives Semester fast operational dashboards without turning the read side into an unsafe second source of truth.

⁂

1. https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs
2. https://microservices.io/patterns/data/cqrs.html
3. https://academy.claude.com/courses/claude-101/creating-with-artifacts
4. https://info.lse.ac.uk/staff/divisions/Eden-Centre/Assets-EC/Documents/Claude-Guides-Dec-2025/Guide-Creating-Claude-Artifacts-V1.0-011225.pdf

Front-end compliance audit: WCAG 2.2 criteria, FTC testimonial rules, and a conversion checklist for semester software marketing sites

Computer
Build a marketing and sales site wireframe for a semester software application — featuring pricing tiers, feature comparison, and lead capture
Computer
Outline the Trust Center page content
Map the institutional pilot conversion flow
Define the shared brand token strategy

For Semester, treat the marketing site as a **conversion and trust product**: it needs an accessible design system, evidence-governed claims, institution-specific pilot flow, pricing clarity, and safe lead capture—not just visual polish.

The most important standards are: build toward WCAG 2.2 AA, never claim compliance without evidence, and only publish authentic/approved testimonials and outcomes. WCAG 2.2 is organized around content being perceivable, operable, understandable, and robust; the FTC prohibits fake or false reviews/testimonials and incentives conditioned on positive sentiment. [1] [2] [3] [4]

Front-end compliance audit

WCAG 2.2 checklist

Use this as the acceptance checklist for every marketing and sales page.

Area	Required checks	Semester implementation
Page structure	One `<h1>`, logical heading order, landmarks, skip link	Use `header`, `nav`, `main`, `aside`, `footer`; add “Skip to content”
Keyboard	Every control, menu, dialog, accordion, form, comparison table, and carousel works without a mouse	Test Tab, Shift+Tab, Enter, Space, Escape, arrows where expected
Focus	Keyboard focus is visible and never hidden	Use `:focus-visible`, high-contrast focus ring, no outline removal
Text alternatives	Images, icons, charts, product visuals, and video controls have useful alternatives	Write alt text for meaningful product UI; mark decorative images empty-alt

Area	Required checks	Semester implementation
Color and contrast	Text, states, alerts, badges, and charts do not depend on color alone	Pair color with labels/icons/patterns; test contrast in all themes
Forms	Labels, required state, errors, validation summary, instructions, and success confirmation are accessible	Use visible labels, `aria-describedby`, error summary, focus first invalid field
Motion	Reduced-motion preference is honored	Disable parallax, looping animation, auto-scroll, and nonessential transitions
Responsive reflow	Works at zoom and narrow viewports without lost content	Test 320px width, 200% zoom, and mobile landscape
Dialogs	Focus moves into dialog, is trapped, and returns to trigger	Use accessible modal primitive; Escape closes noncritical dialogs
Media	Captions, transcript, playback controls, no autoplay audio	Add transcript links for webinar/demo/video content
Timing	No forced timeouts for forms or demos	Show session expiry warning and allow extension if relevant
Errors	Prevent and explain errors before submission	Confirm destructive/irreversible form actions and preserve entered values
Language	`lang` attribute and clear writing	Plain language; define acronyms like HECVAT and SSO
Tables	Headers, scope, captions, mobile alternate views	Use sortable button headers with `aria-sort` and a card fallback on mobile

W3C's WCAG guidance includes keyboard accessibility, navigability, input modalities, and focus appearance; WCAG 2.2 added nine success criteria beyond WCAG 2.1.[5] [6] [7]

Accessibility test protocol

```
Automated:
- Run axe or equivalent on every public route.
- Check headings, labels, contrast, landmarks, broken links, and image alt text.
- Include accessibility checks in CI.

Manual:
- Keyboard-only path from header through footer.
- Screen-reader smoke test for homepage, demo form, pricing, Trust Center, FAQ.
- 200% zoom test.
- 320px viewport test.
- Reduced motion test.
- Form error and success-state test.
- Modal, accordion, tabs, tables, and mobile menu test.
```

Accessibility claims rule

Use language such as:

```
Semester is working toward an accessible experience and maintains an
accessibility feedback and remediation process.

If you encounter a barrier, contact [accessibility contact].
```

Do **not** say "WCAG 2.2 AA compliant," "fully accessible," or "ADA compliant" unless you have documented scope, test evidence, and qualified review. Higher-ed buyers increasingly ask vendors to state their accessibility standard and conformance posture through HECVAT-related evaluation. [8]

FTC testimonial and proof rules

Do not publish

- Fake reviews or AI-generated testimonials presented as human.
- Testimonials from people who did not use Semester.
- Student or institution outcome claims without evidence.
- Incentives conditional on positive reviews.
- Selectively edited reviews that change meaning.
- Claims that imply universal outcomes from one customer's result.
- Fake customer logos, partner logos, institution names, or "trusted by" statements.
- Undisclosed employee, investor, ambassador, advisor, family, or paid influencer endorsements.
- "Independent review" pages controlled by Semester without clear disclosure.
- Suppression of legitimate negative feedback through threats or deceptive practices.

The FTC's final Consumer Review Rule prohibits fake/false consumer reviews and testimonials, buying positive or negative reviews, and other deceptive review practices; the FTC also emphasizes disclosure of material connections in endorsements. [3] [4] [9]

Testimonial approval record

```
type TestimonialApproval = {
  id: string;
  personName: string;
  personTitle?: string;
  organization?: string;
  relationshipType:
    | "customer"
    | "student"
    | "institutional_admin"
    | "pilot_participant"
```

```
    | "employee"
    | "advisor"
    | "ambassador"
    | "partner";

  source: "interview" | "survey" | "email" | "recorded_video" | "public_review";
  exactQuote: string;

  actualExperienceVerified: boolean;
  permissionToPublish: boolean;
  organizationLogoPermission: boolean;

  incentiveProvided: boolean;
  incentiveDescription?: string;
  materialConnectionDisclosure?: string;

  evidenceReference: string;
  approvedBy: string;
  approvedAt: string;
  reviewDueAt?: string;

  allowedPlacements: Array<
    "website" | "sales_deck" | "case_study" | "social" | "webinar"
  >;

  status: "draft" | "approved" | "expired" | "withdrawn";
};
```

Proof hierarchy

Proof type	Can publish?	Required evidence
Product screenshot	Yes, if current	Product owner review
Internal product metric	Usually not as external claim	Metric definition, source, approval
Pilot metric	Yes only with permission and clear context	Customer approval, baseline, method, date
Customer logo	Only with permission	Written logo-use approval
Customer quote	Only with permission and authentic experience	Exact quote, approval, disclosure if needed
Case study	Only with customer/legal review	Customer sign-off, metric methodology
Security claim	Only if verified	Control evidence and security reviewer
Accessibility claim	Only if scoped and evidence-backed	Audit/test scope and reviewer
Competitor comparison	Only if fair, current, and sourced	Source/evidence register

Marketing and sales site wireframe

Global site shell

```
┌──────────────────────────────────────────────────────────────┐
| Semester logo     Product  Solutions  Trust  Resources       |
|                    Pricing  Company                          |
|                                          Sign in  Request demo|
├──────────────────────────────────────────────────────────────┤
| Context bar: “Built for clear student next steps.”           |
├──────────────────────────────────────────────────────────────┤
| Main page content                                            |
├──────────────────────────────────────────────────────────────┤
| Footer: Product | Trust | Resources | Company | Legal         |
| Accessibility | Privacy | Status | Contact                   |
└──────────────────────────────────────────────────────────────┘
```

Homepage wireframe

1. Hero
   Headline:
   “A clearer path through college starts with the next right action.”

   Supporting copy:
   Semester turns fragmented academic information, deadlines, planning,
   and support workflows into one clear student workspace.

   CTA 1: Explore Semester
   CTA 2: Request an institutional pilot

   Product visual:
   Calm, real product workflow—not generic stock imagery.

2. Problem
   “Students should not need five portals to understand what comes next.”
   - Fragmented information
   - Missed actions
   - Unclear academic path
   - Reactive support workload

3. Product workflow
   Understand → Decide → Act → Track progress

4. Audience split
   Students | Advisors and departments | Institutions

5. Registration Readiness Pilot
   - Defined cohort
   - 8–12 weeks or one term
   - Activation and workflow-completion metrics
   - Controlled implementation
   - CTA: Design your pilot

6. Trust block
   Privacy | Accessibility | Governed AI | Institutional controls
   CTA: Visit Trust Center

```
7. Proof block
   Only approved customer proof, documented pilot metrics, product evidence,
   or clearly labeled “pilot program” content.

8. Resource block
   Guides, webinars, planning tools, research

9. Final CTA
   Students: Start your workspace
   Institutions: Request a pilot conversation
```

Institutional pilot page

```
Hero:
“Prove student readiness improvements in one term.”

Sections:
1. Who the pilot is for
2. Current-state workflow problem
3. What Semester changes
4. Pilot scope and non-goals
5. Implementation timeline
6. Data/security/accessibility boundaries
7. Success metrics
8. Roles and responsibilities
9. Pilot-to-annual conversion path
10. FAQ
11. Request a pilot design session form
```

Pricing page

```
Tabs:
- Students
- Departments
- Institutions
- Enterprise

Student:
Free
Student Premium

Institution:
Pilot
Institution Platform
Enterprise / Multi-campus

Every plan card includes:
- Intended user
- Core outcome
- Included features
- AI usage policy
- Support level
```

```
- Data/implementation boundary
- Price or “request pricing”
- CTA

Footnotes:
- “Institutional pricing is scoped to enrollment, implementation, integrations,
  support, and contract terms.”
- “AI allocation and overage usage are governed and tenant-configurable.”
- “Final commercial terms are contained in executed order forms.”
```

Feature comparison page

Feature	Student	Department	Institution	Enterprise
Student planning workspace	Included	Included	Included	Included
Registration readiness	Included	Configurable	Institution-governed	Multi-campus governed
Advisor workflows	Limited	Included	Included	Advanced
Tenant configuration	—	Limited	Included	Advanced
SSO	—	Optional	Included/available	Required/advanced
Integrations	—	Limited	Scoped	Advanced
Analytics	Personal	Department	Institution	Enterprise
AI policy controls	Personal	Department	Institution	Enterprise
Support	Standard	Standard	Premium option	Dedicated
Trust documentation	Public	Public	Procurement pack	Enterprise pack

Do not publish a feature as “included” unless it exists, is enabled in that tier, and is supportable.

Demo request wireframe

```
Left:
- “Design a measurable Semester pilot”
- What happens after submission:
  1. We review your workflow and goals
  2. We schedule a discovery conversation
  3. We define a pilot scope, metrics, and implementation path

Right:
- First name
- Last name
- Work email
- Institution
- Role
- Institution type
- Enrollment band
- Primary workflow
- Current challenge
- Optional message
```

```
- Consent checkbox / privacy link
- Submit

After submit:
- Clear confirmation
- Expected response window
- Optional calendar booking
- No misleading “instant demo” promise
```

Trust Center content

Public Trust Center navigation

```
/trust
├── Security overview
├── Privacy overview
├── Accessibility
├── Responsible AI
├── Data practices
├── Subprocessors
├── Reliability and status
├── Compliance roadmap
├── Security document request
└── Report a vulnerability
```

Trust Center homepage

```
Title:
“Trust is part of the product.”

Sections:
1. Security
   - Access controls
   - Secure development
   - Encryption statement only if verified
   - Monitoring and incident response
   - Security document request

2. Privacy
   - What Semester collects
   - Why it is collected
   - Retention/deletion principles
   - Data request process
   - Subprocessors

3. Accessibility
   - Accessibility commitment
   - Known support/contact path
   - Testing/remediation approach
   - Request accommodation/support

4. Responsible AI
```

```
   - Student/institution control
   - AI policy and use boundaries
   - Data restrictions
   - Provider governance
   - Human review and transparency principles

5. Reliability
   - Status link
   - Incident communication approach
   - Planned maintenance approach

6. Procurement
   - HECVAT/readiness statement
   - DPA process
   - Security questionnaire request
   - Trust-room request
```

Public versus controlled materials

Public	Controlled/NDA-gated
Security overview	Detailed architecture diagrams
Privacy summary	Full HECVAT response
Accessibility statement	Pen-test report
Subprocessor names	Detailed vulnerability reports
Responsible AI principles	Detailed incident evidence
Data-retention summary	Security questionnaire answers
Status page	Audit/control evidence
Vulnerability contact	Customer-specific contract security terms

Trust Center review workflow

```
Draft
→ Product accuracy review
→ Security review
→ Privacy review
→ Accessibility review
→ Legal review if externally material
→ Claims/evidence review
→ Approved
→ Published
→ Review due
→ Updated / retired
```

Institutional pilot conversion flow

```
Institutional visitor
  ↓
Pilot page / relevant use-case page
  ↓
Reads:
- Scope
- Timeline
- Success metrics
- Security and implementation boundaries
  ↓
Downloads resource or requests demo
  ↓
Validated lead submission
  ↓
Internal routing:
- Account created
- Owner assigned
- SLA started
- Attribution retained
  ↓
Discovery call
  ↓
Qualification:
- Problem
- Champion
- Economic buyer
- Timeline
- Security/procurement path
- Pilot scope
  ↓
Role-specific demo
  ↓
Pilot design session
  ↓
Pilot proposal + Mutual Action Plan
  ↓
Security/privacy/accessibility/procurement review
  ↓
Paid pilot agreement
  ↓
Implementation
  ↓
Activation and outcome measurement
  ↓
Midpoint executive review
  ↓
Final value report
  ↓
Annual conversion decision
  ↓
Expansion / case study / reference
```

Conversion stages and required evidence

Stage	Required evidence	Exit criteria
Lead	Valid contact and consent	Routed to owner
Qualified	Pain, champion, buyer, timing	Discovery complete
Demo	Use case and objection record	Next meeting scheduled
Pilot design	Cohort, workflow, metrics, scope	Proposal-ready
Procurement	Security/DPA/accessibility path	Review plan approved
Proposal	Price, decision date, MAP	Customer evaluating
Signed pilot	Contract/order form	Implementation scheduled
Activated	Admin configured, users invited	Activation baseline available
Success	Metrics and executive review	Annual conversion decision
Annual	Contract/plan active	Customer-health program starts

Shared brand-token strategy

Semester needs **one semantic token system** shared across product, operations, public marketing, sales collateral, and email templates.

Token layers

```
1. Primitive tokens
   Raw palette, scale, spacing, typography values.

2. Semantic tokens
   Meaning: surface, text, border, action, success, warning, critical.

3. Component tokens
   Meaning within components: button, card, input, table, navigation.

4. Brand expression tokens
   Marketing-specific but still semantic:
   hero surface, proof surface, trust surface, pilot surface.

5. Theme tokens
   Light, dark, high-contrast, tenant-safe branded variations.
```

Example token structure

```
export const primitive = {
  slate: {
    50: "#F8FAFC",
    100: "#F1F5F9",
    200: "#E2E8F0",
    500: "#64748B",
```

```
    700: "#334155",
    900: "#0F172A",
    950: "#020617",
  },

  indigo: {
    50: "#EEF2FF",
    100: "#E0E7FF",
    300: "#A5B4FC",
    500: "#6366F1",
    600: "#4F46E5",
    700: "#4338CA",
    900: "#312E81",
  },

  emerald: {
    50: "#ECFDF5",
    200: "#A7F3D0",
    600: "#059669",
    700: "#047857",
  },

  amber: {
    50: "#FFFBEB",
    200: "#FDE68A",
    700: "#B45309",
  },

  rose: {
    50: "#FFF1F2",
    200: "#FECDD3",
    700: "#BE123C",
  },
};

export const semantic = {
  background: {
    page: primitive.slate[^50],
    surface: "#FFFFFF",
    surfaceMuted: primitive.slate[^100],
    inverse: primitive.slate[^950],
  },

  text: {
    primary: primitive.slate[^900],
    secondary: primitive.slate[^700],
    muted: primitive.slate[^500],
    inverse: "#FFFFFF",
    link: primitive.indigo[^600],
  },

  border: {
    default: primitive.slate[^200],
    strong: "#CBD5E1",
    focus: primitive.indigo[^600],
  },
```

```
  action: {
    primary: primitive.indigo[^600],
    primaryHover: primitive.indigo[^700],
    secondary: "#FFFFFF",
  },

  status: {
    success: primitive.emerald[^700],
    warning: primitive.amber[^700],
    critical: primitive.rose[^700],
    info: primitive.indigo[^600],
  },

  focusRing: primitive.indigo[^500],
};
```

Brand rules

```
- Use calm neutral surfaces for cognitive clarity.
- Use indigo as the primary decision/action color.
- Use status colors only for status; do not use them as decoration.
- Avoid gradients unless they communicate hierarchy or state.
- Avoid “student app” visual clichés: cluttered stickers, loud confetti,
  random illustration, gamification noise, or overly playful portal design.
- Use product screenshots and real workflows as primary visual proof.
- Maintain a premium, calm, academically serious tone.
- Product, marketing, and operator console should feel like the same company,
  even when their density differs.
```

Typography roles

Token	Use
`display`	Hero headline, major narrative moments
`heading`	Section/page titles
`subheading`	Cards, modules, sections
`body`	Primary prose
`bodySmall`	Supporting copy
`label`	Inputs, metadata, tags
`mono`	Codes, IDs, audit references, technical details

Frontend/company-site master prompt

```
Build the Semester Marketing, Sales, and Trust Frontend as a coherent extension
of the existing Semester app—not as a detached website.

Implement:

1. Shared semantic brand-token system
2. Marketing component library
3. Responsive public website architecture
4. Homepage
5. Student workspace page
6. Institutional platform page
7. Registration Readiness Pilot page
8. Pricing page
9. Feature comparison page
10. Demo request/lead-capture page
11. Trust Center
12. Security, privacy, accessibility, and responsible-AI pages
13. Resource hub and SEO content foundation
14. Public-safe content/read-model contracts
15. Controlled lead intake and consent capture
16. Analytics event taxonomy
17. Claims/testimonial approval workflow
18. Student and institutional conversion paths
19. Accessibility and performance test suite

Requirements:

- Reuse existing Semester design tokens and frontend conventions.
- Do not create a separate app.
- Do not make unverified security, privacy, accessibility, product, pricing,
  institutional, or outcome claims.
- Do not publish fake, misleading, undisclosed, or incentive-conditioned
  testimonials.
- Do not expose internal customer, GTM, financial, support, security, or
  operator tables to public clients.
- Use server-validated lead intake with consent and suppression checks.
- Build toward WCAG 2.2 AA.
- Add keyboard, focus, form, responsive, reduced-motion, and screen-reader
  acceptance tests.
- Add SEO metadata, canonical URLs, structured content, internal linking,
  sitemap, and robots configuration appropriate to the existing stack.
- Add performance budgets and avoid blocking the first CTA with heavy media or
  third-party scripts.
- Ensure every public page has clear next-step CTA and app/support/trust handoff.
- Add Markdown documentation for conversion flows, content governance, claims
  review, accessibility policy, analytics events, and page inventory.

Prioritize:
1. Homepage
2. Institutional pilot page
3. Demo request flow
4. Trust Center
5. Security/privacy/accessibility pages
6. Student and institution product pages
```

```
7. Pricing/feature comparison
8. Resource hub
9. Case studies only after verified customer evidence exists

Return:
- Current-state audit
- Page and component inventory
- Content gaps
- Claims that need evidence or removal
- Conversion funnel map
- Accessibility findings
- SEO findings
- Public data exposure review
- Files changed
- Tests added
- Lint/typecheck/build results
- Remaining legal/security/accessibility review items
```

Priority sequence

1. Create the shared token and component contract.
2. Audit every public claim and remove/label unsupported claims.
3. Build Homepage + Pilot page + Demo form.
4. Build Trust Center and legal/accessibility/security pages.
5. Connect lead routing, consent, analytics, and CRM handoff.
6. Build pricing and comparison pages with accurate feature/entitlement data.
7. Build student acquisition landing pages and app onboarding handoff.
8. Build resource hub, SEO system, webinars, and verified proof assets.
9. Build partnerships, ambassadors, careers, and advanced content only after the core funnel works.

⁂

1. https://www.w3.org/TR/WCAG22/
2. https://www.w3.org/WAI/standards-guidelines/wcag/
3. https://www.ftc.gov/news-events/news/press-releases/2024/08/federal-trade-commission-announces-final-rule-banning-fake-reviews-testimonials
4. https://www.ftc.gov/business-guidance/advertising-marketing/endorsements-influencers-reviews
5. https://www.w3.org/WAI/standards-guidelines/wcag/new-in-22/
6. https://www.w3.org/WAI/WCAG22/quickref/
7. https://w3c.github.io/wcag/techniques/
8. https://er.educause.edu/articles/2025/4/accessibility-in-technology-acquisition-with-hecvat-4
9. https://www.federalregister.gov/documents/2024/08/22/2024-18519/trade-regulation-rule-on-the-use-of-consumer-reviews-and-testimonials

10. https://api2.v0.dev/docs/agentic-features
11. https://dev.to/jais_mukesh/managing-supabase-auth-state-across-server-client-components-in-nextjs-2h2b
12. https://supabase.com/features/row-level-security
13. https://www.ftc.gov/business-guidance/blog/2025/12/warning-letter-or-ten-businesses-comply-ftcs-consumer-review-rule
14. https://www.dwt.com/insights/2024/11/ftc-faqs-on-consumer-review-testimonials-rule
15. https://ftcattorney.com/guide-to-ftc-consumer-review-and-testimonial-rule/
16. https://www.ftc.gov/business-guidance/resources/consumer-reviews-testimonials-rule-questions-answers
17. https://www.educause.edu/higher-education-community-vendor-assessment-toolkit
18. https://www.educause.edu/higher-education-community-vendor-assessment-toolkit/hecvat-faqs-for-higher-education
19. https://www.saltycloud.com/blog/what-is-the-hecvat/

WCAG 2.2 and FTC audit checklist: scan semester software landing pages for accessibility traps and testimonial rule violations

Computer
Conversion-optimized marketing site wireframe: semester software layouts built for WCAG 2.2 compliance and FTC testimonial compliance
Computer
Show me the FTC testimonial checklist for the site
Draft the WCAG 2.2 form validation states for Semester
Convert this compliance plan into a wireframe outline

Below is a practical **WCAG 2.2 + FTC marketing-compliance checklist**, accessible form-state specification, Trust Center content outline, institutional pilot conversion map, and wireframe blueprint for Semester's public marketing and sales site.

This should become a release gate for every public landing page, demo form, pricing page, testimonial, webinar page, comparison page, and campaign landing page. WCAG 2.2 is built around content being perceivable, operable, understandable, and robust; the FTC prohibits fake or false testimonials/reviews and deceptive practices around their collection, display, or suppression. [1] [2] [3] [4]

Compliance audit checklist

WCAG 2.2 release gate

Category	Landing-page requirement	Test
Structure	One logical `<h1>`, ordered headings, `header`, `nav`, `main`, `footer`, skip link	Inspect semantic outline with browser accessibility tree
Keyboard	All navigation, forms, accordions, tabs, pricing selectors, video controls, and dialogs usable without pointer	Tab, Shift+Tab, Enter, Space, Escape, arrow-key testing
Focus	Focus is always visible, high contrast, and not obscured by sticky headers/modals	Keyboard-only walkthrough
Contrast	Text, buttons, icons, charts, status labels, and form states meet contrast requirements	Automated contrast scan + manual visual check
Color reliance	Error, selected, required, warning, and success states are not color-only	Remove color mentally; labels/icons must remain meaningful

Category	Landing-page requirement	Test
Forms	Visible labels, instructions, required indicators, field-level errors, error summary, preserved entries	Submit invalid and valid forms using keyboard and screen reader
Errors	Errors identify the field/problem and describe correction; focus moves to summary or first invalid field	Test every invalid state
Mobile	320px viewport, landscape mobile, 200% zoom, reflow without horizontal loss	Device emulator + manual browser zoom
Motion	`prefers-reduced-motion` disables nonessential animation, parallax, auto-scroll	OS/browser reduced-motion test
Media	Captions, transcript, non-autoplay audio, accessible player controls	Keyboard and screen-reader review
Dialogs	Focus trap, Escape close, focus return, descriptive title	Modal test
Tables	`<caption>`, scoped headers, sortable header buttons, accessible mobile fallback	Screen reader table navigation
Links	Link text states destination/action; no “click here” without context	Copy/scan page content
Timing	No forced short session/form timeout; warn and extend if session expiry matters	Idle-session test
Language	`lang` set; plain terms; acronyms explained first use	HTML/content review
Third-party embeds	Calendars/chat/video/analytics have accessible fallback and consent-aware loading	Block third party and test graceful fallback

WCAG 2.2 includes guidance on keyboard operation, focus appearance, navigability, input modalities, and error handling. It also adds nine success criteria beyond WCAG 2.1.[5] [6] [7]

Public-site compliance score

Use this page-level release score:

```
Accessibility:
- Automated checks passed
- Keyboard test passed
- Screen-reader smoke test passed
- Mobile/reflow test passed
- Form error test passed
- Reduced-motion test passed

Conversion:
- One clear primary CTA
- One appropriate secondary CTA
- Visible trust path
- Lead-form confirmation works
- CRM handoff verified
- No dead-end path

Claims:
- Product claims verified
- Pricing/current-tier data verified
- Customer proof approved
```

```
- Testimonial disclosure reviewed
- Competitor comparison sourced
- Security/privacy/accessibility claims reviewed

Performance:
- Page renders before nonessential scripts
- Images sized and responsive
- No blocking third-party scripts
- Mobile test passed
- Error monitoring works
```

A page should not publish if it has an unresolved **critical accessibility issue**, a deceptive/unsupported claim, broken primary CTA, broken lead intake, or unapproved testimonial.

FTC testimonial checklist

Before publishing any review, quote, logo, or case study

- [ ] The speaker is a real person or real institution.
- [ ] The person/institution actually used or participated in the relevant Semester experience.
- [ ] The exact quote reflects their actual opinion and has not been edited to change meaning.
- [ ] The quote is not artificially generated, fictionalized, or attributed to a nonexistent user.
- [ ] Written permission exists for the exact quote and approved placements.
- [ ] Written logo-use permission exists before displaying an institution logo.
- [ ] The testimonial does not imply results are typical unless you have evidence that they are typical.
- [ ] Outcome claims include context:
  - population
  - time period
  - methodology
  - baseline
  - date
  - relevant qualification
- [ ] Any material connection is disclosed:
  - payment
  - discount
  - free product
  - employment
  - advisory role
  - investor relationship

    - ambassador role
    - affiliate/referral incentive
- [ ] Incentives are never conditioned on a positive review.
- [ ] Negative feedback is not suppressed, threatened, or selectively hidden in a misleading way.
- [ ] The quote has not expired, been withdrawn, or become misleading due to product changes.
- [ ] A claims/testimonial owner and review date are recorded.
- [ ] Legal review occurs for material institutional outcome claims or high-visibility campaigns.

The FTC's review rule prohibits fake or false consumer reviews/testimonials, buying reviews conditioned on positive/negative sentiment, and certain deceptive review-suppression practices. FTC guidance also addresses material connections and influencer/endorsement disclosures. [3] [4] [8]

Testimonial component rules

```
type TestimonialProps = {
  quote: string;
  personName: string;
  role?: string;
  organization?: string;

  permissionStatus: "approved";
  approvedPlacements: string[];

  relationshipDisclosure?: string;
  outcomeContext?: string;
  evidenceReference?: string;

  showOrganizationLogo?: boolean;
  logoPermissionStatus?: "approved";
};
```

Render only if:

```
const canRenderTestimonial =
  testimonial.permissionStatus === "approved" &&
  testimonial.approvedPlacements.includes(currentPlacement) &&
  testimonial.quote.trim().length > 0 &&
  !testimonial.isExpired &&
  !testimonial.isWithdrawn;
```

Render disclosures near the endorsement, not buried in a footer:

```
“[Quote]”
— Name, role, institution
```

```
Pilot participant, [term/year].
[If applicable: Received [relationship/incentive disclosure].]
```

Safe proof alternatives before pilots

Before verified customers exist, use:

- Product screenshots labeled as illustrative/current product views.
- Workflow diagrams.
- “Designed for” statements only if accurate.
- Product principles.
- Pilot-program scope.
- Resource content.
- Explicit “early access” or “pilot program” language.
- Product demo recordings.

Do **not** use “trusted by,” “proven to,” “improves outcomes,” “reduces support tickets,” or institution logos until you have verified evidence and permission.

WCAG form validation states

Form-state model

```
type FieldStatus =
  | "idle"
  | "focused"
  | "valid"
  | "invalid"
  | "submitting"
  | "server_error"
  | "disabled";

type FormFieldState = {
  id: string;
  label: string;
  required: boolean;
  value: string;
  status: FieldStatus;
  error?: string;
  hint?: string;
  touched: boolean;
  describedByIds: string[];
};
```

Required semantic pattern

```
<div className="field">
  <label htmlFor="work-email">
    Work email <span aria-hidden="true">*</span>
    <span className="sr-only">Required</span>
  </label>

  <p id="work-email-hint" className="field-hint">
    Use your institutional or work email if you are requesting a pilot.
  </p>

  <input
    id="work-email"
    name="workEmail"
    type="email"
    autoComplete="email"
    required
    aria-required="true"
    aria-invalid={hasError ? "true" : "false"}
    aria-describedby={
      hasError
        ? "work-email-hint work-email-error"
        : "work-email-hint"
    }
    className={hasError ? "input input--error" : "input"}
  />

  {hasError && (
    <p
      id="work-email-error"
      role="alert"
      className="field-error"
    >
      Enter a valid work email, such as name@institution.edu.
    </p>
  )}
</div>
```

Error summary

Place above the form after failed submit:

```
{errors.length > 0 && (
  <div
    ref={errorSummaryRef}
    tabIndex={-1}
    role="alert"
    aria-labelledby="form-error-title"
    className="form-error-summary"
  >
    <h2 id="form-error-title">
      Please correct {errors.length} field{errors.length === 1 ? "" : "s"}.
    </h2>
```

```
    <ul>
      {errors.map((error) => (
        <li key={error.field}>
          <a href={`#${error.field}`}>
            {error.message}
          </a>
        </li>
      ))}
    </ul>
  </div>
)}
```

After submission fails:

1. Preserve every valid field entry.
2. Update `aria-invalid`.
3. Render field-level error messages.
4. Move focus to the error summary.
5. Let the user jump to individual invalid fields.
6. Do not rely only on red borders or icons.
7. Do not clear form state unless user explicitly resets it.

Validation state table

State	Visual behavior	Screen-reader behavior	Interaction rule
Idle	Neutral border, helper text	Label + helper text	No error announced
Focused	Visible focus ring	Standard field announcement	Never hide focus
Valid	Optional subtle confirmation icon/text	“Valid” only if useful	Do not over-announce
Invalid	Error border + icon + text	`aria-invalid=true`, error text referenced	Explain how to fix
Submitting	Disabled submit, loading label	`aria-busy=true` on form region	Do not lose values
Server error	Form-level message + retry guidance	`role=alert`	Preserve data
Success	Confirmation, next step, expected response time	`role=status` or managed focus	Do not redirect unexpectedly

Demo request form fields

```
Required:
- First name
- Last name
- Work email
- Organization/institution
```

```
- Role
- Primary workflow or interest
- Privacy acknowledgment

Optional:
- Institution type
- Enrollment range
- Current challenge
- Desired pilot timeline
- Message
- Marketing opt-in separate from service/contact consent
```

Consent pattern

```
<label className="consent">
  <input
    type="checkbox"
    required
    name="privacyAcknowledged"
    aria-describedby="privacy-notice"
  />
  <span>
    I have read the <a href="/legal/privacy">Privacy Policy</a> and agree
    that Semester may use my information to respond to this request.
  </span>
</label>

<p id="privacy-notice">
  Marketing updates are optional and are not required to request a demo.
</p>

<label className="consent">
  <input type="checkbox" name="marketingOptIn" />
  <span>
    Send me occasional Semester news, resources, and event invitations.
  </span>
</label>
```

Never pre-check marketing consent, combine service-contact consent with promotional consent, or make marketing consent required to submit a pilot/demo request.

Compliance-to-wireframe outline

1. Global header

```
[Skip to content]

Semester logo
├── Product
├── Solutions
├── Trust
├── Resources
```

```
├── Pricing
├── Company
├── Sign in
└── Request a pilot / Request demo

Requirements:
- Keyboard-operable menu
- Visible focus
- Accessible mobile drawer
- No hover-only navigation
- Current page indicated with aria-current
```

2. Homepage hero

```
Eyebrow:
Semester for clearer student next steps

H1:
A clearer path through college starts with the next right action.

Body:
Semester brings planning, deadlines, guidance, and support into one
student-centered workspace.

Primary CTA:
Explore Semester

Secondary CTA:
Request an institutional pilot

Supporting trust link:
Explore privacy, accessibility, and security

Visual:
Real product workflow screenshot with meaningful alt text:
“Semester workspace showing student actions, academic planning, and upcoming deadline
```

Compliance:

- One `<h1>`.
- CTA buttons have distinct labels.
- No text embedded only in image.
- Product visual has alt text or is decorative if redundant.
- No autoplay video/audio.
- CTA event tracking respects consent state.

3. Problem section

```
H2:
Students should not need five portals to understand what comes next.

Cards:
- Fragmented information
- Missed actions
- Unclear academic path
- Reactive support workload
```

Compliance:

- Cards should be list items or articles with headings.
- Icons must not carry meaning alone.
- Do not make unverified “students are X% more likely...” claims.

4. Product workflow

```
H2:
Understand. Decide. Act. Track progress.

1. Understand your current position
2. Compare options and next steps
3. Take the action that matters now
4. Keep progress visible
```

Compliance:

- Use ordered list semantics.
- Workflow graphics include text equivalents.
- Motion is optional and disabled with reduced-motion preference.

5. Audience paths

```
H2:
Built for the people moving student progress forward.

[Students]
CTA: Explore the student workspace

[Advisors and departments]
CTA: See student-success workflows

[Institutions]
CTA: Explore the institutional platform
```

Compliance:

- Three separate, descriptive CTAs.
- Avoid identical “Learn more” links.
- Keep card interactions keyboard accessible.

6. Pilot conversion block

```
H2:
Prove registration readiness improvements in one term.

Pilot includes:
- Defined student cohort
- Measurable activation and action completion
- Limited, governed implementation scope
- Weekly operating review
- Final value and annual-conversion decision

CTA:
Design your pilot
```

Compliance:

- Clearly label pilot scope as illustrative until contractually confirmed.
- Do not promise specific outcomes.
- Link to Trust Center and implementation/security information.

7. Trust strip

```
Privacy
Accessibility
Governed AI
Institutional controls
Status and reliability

CTA:
Visit the Trust Center
```

Compliance:

- Each item links to actual supporting content.
- Do not use generic “enterprise-grade” or “fully compliant” language unless verified.
- State scope and review date for material claims.

8. Proof section

```
H2:
Evidence that can be reviewed.

Before customer proof:
- Product workflow screenshots
- Pilot framework
- Resource guides
- Security and implementation materials

After approved pilot proof:
- Approved case study
- Customer quote
- Metric context
- Required disclosure
```

Compliance:

- Render testimonial only from approved testimonial register.
- Show material-connection disclosure if relevant.
- Link results to methodology/context rather than presenting unsupported headline numbers.

9. Resource section

```
H2:
Practical resources for student progress.

Cards:
- Registration readiness guide
- Pilot planning worksheet
- Student action checklist
- Webinar
- Product update
```

Compliance:

- Reading time, update date, author/source.
- Accessible card links.
- No gated download without clear data-use explanation.

10. Final CTA

```
H2:
Make the next step clearer.

Students:
Start your workspace
```

```
Institutions:
Request a pilot conversation
```

Compliance:

- Distinct paths.
- No deceptive urgency.
- Both lead somewhere functional.

11. Footer

```
Product
Solutions
Trust
Resources
Company
Legal
Accessibility
Privacy
Terms
Acceptable use
Status
Contact
Report a vulnerability
```

Compliance:

- Accessible heading structure.
- No low-contrast legal links.
- Contact and accessibility feedback must work.

Trust Center wireframe

```
Trust Center
├── Hero:
│   “Trust is part of the product.”
│
├── Security
│   - Security overview
│   - Secure development
│   - Access controls
│   - Incident response
│   - Request security documents
│
├── Privacy
│   - Data practices
│   - Retention/deletion
│   - Subprocessors
│   - Privacy requests
```

```
│
├── Accessibility
│   - Accessibility commitment
│   - Feedback channel
│   - Testing/remediation approach
│
├── Responsible AI
│   - AI use boundaries
│   - Institution policy controls
│   - Data restrictions
│   - Transparency/human oversight
│
├── Reliability
│   - System status
│   - Incident communication
│   - Planned maintenance
│
└── Procurement
    - HECVAT/readiness statement
    - DPA process
    - Security questionnaire
    - Trust-room request
```

Build prompt

```
Audit and improve Semester's public marketing, sales, pilot, pricing, Trust
Center, and lead-capture frontend for WCAG 2.2-oriented accessibility,
conversion clarity, and FTC-safe claims/testimonials.

Do not claim WCAG conformance, legal compliance, security certification,
institutional outcomes, or customer proof without verified evidence.

Build:

1. A public-site compliance checklist in docs/marketing/
2. Accessibility release gates
3. Claims/testimonial approval register integration
4. Testimonial component gating:
   - real source
   - permission
   - approval
   - disclosure
   - review date
   - placement control
5. Accessible demo/pilot lead form:
   - visible labels
   - field errors
   - error summary
   - preserved state
   - consent separation
   - server validation
   - suppression checks
   - CRM/lead routing
6. Homepage wireframe and implementation
7. Institutional pilot page
```

```
8. Pricing and feature comparison page
9. Trust Center
10. Accessibility page
11. Resource cards and proof system
12. Responsive and keyboard-accessible navigation
13. Accessibility tests:
   - automated
   - keyboard
   - screen-reader smoke
   - reduced motion
   - 320px
   - 200% zoom
   - form validation
14. Marketing-site conversion analytics with consent-aware event tracking

For every public claim:
- Verify evidence
- Assign owner
- Assign reviewer
- Assign review date
- Mark scope:
  public / institutional-only / NDA-only / internal
- Hide expired or unapproved claims

For every testimonial:
- Verify real experience
- Store exact approved quote
- Store publication permission
- Store logo permission separately
- Store material connection disclosure
- Never require positive sentiment in exchange for an incentive
- Never display unapproved testimonials

Return:
- Current compliance gaps
- WCAG findings
- FTC/testimonial findings
- Conversion funnel gaps
- Wireframe implementation map
- Files created/updated
- Tests added
- Remaining legal/security/accessibility review items
```

Publish gate

Before a Semester marketing/sales page goes live, require:

```
[ ] Primary CTA works
[ ] Lead form works
[ ] Consent copy is accurate
[ ] Privacy link works
[ ] Keyboard test passed
[ ] Focus test passed
[ ] Error states tested
[ ] Mobile/zoom test passed
```

```
[ ] Reduced-motion test passed
[ ] No unsupported claim
[ ] Every testimonial approved
[ ] Every logo approved
[ ] Security/accessibility language reviewed
[ ] Analytics respects consent
[ ] No sensitive data in public analytics
[ ] Broken-link scan passed
[ ] Page title/meta/canonical URL added
[ ] Content owner and review date assigned
```

⁂

1. https://www.w3.org/TR/WCAG22/
2. https://www.w3.org/WAI/standards-guidelines/wcag/
3. https://www.ftc.gov/news-events/news/press-releases/2024/08/federal-trade-commission-announces-final-rule-banning-fake-reviews-testimonials
4. https://www.ftc.gov/business-guidance/advertising-marketing/endorsements-influencers-reviews
5. https://www.w3.org/WAI/standards-guidelines/wcag/new-in-22/
6. https://www.w3.org/WAI/WCAG22/quickref/
7. https://w3c.github.io/wcag/techniques/
8. https://www.federalregister.gov/documents/2024/08/22/2024-18519/trade-regulation-rule-on-the-use-of-consumer-reviews-and-testimonials

perplexity

now do same thing as it applies to front end and comany/marketing/sales site for semester software application

Yes. Apply the same architecture discipline to Semester's **public company site, product frontend, marketing surfaces, sales flows, and customer acquisition system**: one coherent design and content system, a governed CMS/read-model layer, clear conversion paths, privacy-safe analytics, claims approval, and strong handoff from marketing site to authenticated product.

Semester should not have a separate "pretty marketing site" disconnected from the product. It should have a public-facing **growth and trust layer** that shares brand tokens, accessibility standards, product language, analytics definitions, and safe backend contracts with the application.

The public website must make four audiences feel understood:

1. **Students** — "I know what to do next."
2. **Institutional buyers** — "This is credible, governable, and worth piloting."
3. **Procurement/security reviewers** — "This company understands risk, privacy, accessibility, and operations."
4. **Candidates/partners** — "This is a serious company with a clear mission and operating discipline."

Frontend architecture

```
semester.com
├── Public marketing and growth site
│   ├── Product narrative
│   ├── Use cases
│   ├── Institutional pilot conversion
│   ├── Trust Center
│   ├── Resources / SEO
│   ├── Careers / partners
│   └── Demo and lead capture
│
├── app.semester.com
│   ├── Authenticated student product
│   ├── Institutional workspace
│   ├── Operator console
│   └── Account / billing / support
│
```

```
├── help.semester.com
│   ├── Knowledge base
│   ├── Implementation guides
│   ├── Support
│   └── Status communication
│
└── status.semester.com
    ├── Live service status
    ├── Incident history
    └── Scheduled maintenance
```

Even if these begin as routes or subdirectories, preserve the conceptual separation:

Surface	Primary job	Authentication	Data sensitivity
Public site	Explain, educate, convert	None	Public
Product app	Deliver student/institution workflows	Required	High
Operations console	Operate company and platform	Strong internal authorization	Very high
Help center	Reduce support burden	Usually optional	Low/medium
Trust center	Reduce procurement friction	Public + NDA-gated documents	Public/controlled
Status site	Build reliability transparency	Public	Public

Public company-site architecture

Required page system

```
/
├── /product
│   ├── /student-workspace
│   ├── /institution-platform
│   ├── /registration-readiness
│   ├── /advising-and-student-success
│   ├── /governed-ai
│   └── /integrations
│
├── /solutions
│   ├── /students
│   ├── /advisors
│   ├── /student-success
│   ├── /registrars
│   ├── /academic-departments
│   ├── /institutions
│   └── /enterprise
│
├── /pilot
├── /pricing
├── /demo
├── /security
├── /privacy
├── /accessibility
```

```
├── /trust
│   ├── /subprocessors
│   ├── /security-documents
│   ├── /compliance
│   └── /status
│
├── /resources
│   ├── /guides
│   ├── /webinars
│   ├── /research
│   ├── /case-studies
│   ├── /product-updates
│   └── /faq
│
├── /compare
│   ├── /semester-vs-lms
│   ├── /semester-vs-fragmented-tools
│   └── /semester-vs-manual-advising-workflows
│
├── /partners
├── /ambassadors
├── /careers
├── /company
├── /contact
├── /legal
│   ├── /terms
│   ├── /privacy
│   ├── /acceptable-use
│   ├── /dpa
│   └── /cookies
│
└── /login
```

Do not publish every page at once. Prioritize pages that directly support the pilot sales motion.

P0 company-site pages

Build these first:

Page	Conversion job	Primary audience
Homepage	Explain Semester and route visitors	All
Registration Readiness Pilot	Convert institutional interest into pilot conversation	Student success, advising, registrar
Institution Platform	Explain governed institutional value	Buyers and executives
Student Workspace	Explain student benefit and direct sign-up	Students
Demo Request	Capture qualified B2B intent	Institutional buyers
Trust Center	Reduce security/procurement uncertainty	IT, security, legal, procurement
Security Overview	Explain verified controls without overclaiming	Security reviewers

Page	Conversion job	Primary audience
Accessibility	Explain current commitment, contact path, and evidence status	Buyers and users
Pricing / Request Pricing	Set commercial expectations	Buyers and students
Resource Hub	Establish authority and search entry points	Students and buyers
Contact / Support	Route leads and help requests properly	All
Legal pages	Establish clear public terms	All

Homepage narrative

The homepage should communicate one clear idea:

> Semester turns fragmented academic information and support workflows into a clear, governed path to the next meaningful action.

Homepage sequence

```
1. Hero
   - Clear promise
   - One primary CTA
   - One secondary CTA
   - Product visual, not decorative stock imagery

2. Current-state problem
   - Fragmented portals, deadlines, planning, support, and next steps

3. Semester workflow
   - Understand
   - Decide
   - Act
   - Track progress

4. Audience paths
   - Students
   - Departments
   - Institutions

5. Registration Readiness Pilot
   - Scope
   - Timeline
   - Metrics
   - Demo / pilot CTA

6. Governed trust
   - Privacy
   - Accessibility
   - Source-aware guidance
   - Institution controls
   - Security link
```

```
7. Product proof
   - Screens, workflows, demo, verified customer evidence when available

8. Resource / education
   - Guides, webinars, research, planning tools

9. Final CTA
   - Request institutional pilot
   - Start student workspace
```

Marketing frontend design system

Do not build a separate visual identity for marketing. Extend the existing Semester design system.

```
app/src/
├── design-tokens/
│   ├── colors.ts
│   ├── typography.ts
│   ├── spacing.ts
│   ├── elevation.ts
│   ├── motion.ts
│   └── breakpoints.ts
│
├── marketing/
│   ├── components/
│   │   ├── MarketingShell.tsx
│   │   ├── Hero.tsx
│   │   ├── ProofStrip.tsx
│   │   ├── UseCaseCard.tsx
│   │   ├── ProductWorkflow.tsx
│   │   ├── DemoRequestForm.tsx
│   │   ├── PilotScopeCard.tsx
│   │   ├── TrustSummary.tsx
│   │   ├── MetricsDefinition.tsx
│   │   ├── TestimonialCard.tsx
│   │   ├── ResourceCard.tsx
│   │   ├── PricingTable.tsx
│   │   ├── ComparisonTable.tsx
│   │   ├── FAQ.tsx
│   │   ├── Footer.tsx
│   │   └── ClaimDisclosure.tsx
│   │
│   ├── pages/
│   │   ├── HomePage.tsx
│   │   ├── PilotPage.tsx
│   │   ├── InstitutionalPage.tsx
│   │   ├── StudentPage.tsx
│   │   ├── TrustCenterPage.tsx
│   │   ├── PricingPage.tsx
│   │   ├── ResourcesPage.tsx
│   │   └── DemoPage.tsx
│   │
│   ├── content/
```

```
│   │   ├── claims.ts
│   │   ├── faqs.ts
│   │   ├── resources.ts
│   │   ├── pricing.ts
│   │   └── navigation.ts
│   │
│   ├── data/
│   │   ├── marketingClient.ts
│   │   ├── marketingQueryKeys.ts
│   │   ├── marketingTypes.ts
│   │   └── conversionEvents.ts
│   │
│   └── state/
│       ├── leadFormState.ts
│       ├── consentState.ts
│       └── experimentState.ts
```

Required component behavior

Component	Requirements
Hero	Clear promise, visible primary CTA, keyboard-accessible, no autoplay-only message
Demo request form	Validation, consent, privacy link, spam controls, attribution capture, success state
Pilot scope card	Shows scope, non-goals, timeline, success metrics, buyer role, CTA
Trust summary	Links to verified source content; no vague “enterprise-grade” claims without evidence
Metrics definition	Explains what activation, first action, completion, and success mean
Testimonial card	Source, permission status, date, material-connection disclosure where relevant
Pricing table	Planning assumptions versus negotiated institutional terms clearly separated
Resource card	Topic, audience, publication/update date, author/source, reading time
Comparison table	Fair and source-based; avoid unsupported competitor claims
FAQ	Semantic disclosure controls, deep links, search-friendly content
Consent banner	Purpose-based choices, not dark patterns
Claim disclosure	Shows evidence/review status for sensitive claims where applicable

Conversion architecture

Institutional funnel

```
Institutional discovery
  ↓
Product / use-case page
  ↓
Pilot page or resource
  ↓
Demo request
```

```
  ↓
Lead qualification
  ↓
Calendar booking
  ↓
Discovery call
  ↓
Pilot design session
  ↓
Security/procurement packet
  ↓
Paid pilot proposal
```

Student funnel

```
Search / social / referral / campus QR
  ↓
Student value page
  ↓
Interactive planning utility or product walkthrough
  ↓
Create account
  ↓
Onboarding
  ↓
First meaningful action
  ↓
Weekly retained use
  ↓
Referral / ambassador / Premium conversion
```

Public-site backend contracts

Do not connect the marketing site directly to internal CRM, customer, GTM, or financial tables from the browser. Use controlled intake endpoints.

```
Public browser
  ↓
Validated public lead endpoint
  ↓
Spam/abuse control
  ↓
Consent recording
  ↓
Lead normalization
  ↓
Internal GTM ingestion queue
  ↓
CRM / internal GTM record
  ↓
Assigned owner and response SLA
```

Lead intake contract

```
type DemoRequest = {
  firstName: string;
  lastName: string;
  workEmail: string;
  organizationName?: string;
  role?: string;
  institutionType?: string;
  enrollmentBand?: string;
  useCase?: string;
  message?: string;

  consent: {
    privacyPolicyVersion: string;
    marketingOptIn: boolean;
    submittedAt: string;
  };

  attribution: {
    landingPage: string;
    referrer?: string;
    campaign?: string;
    source?: string;
    medium?: string;
    content?: string;
  };
};
```

The backend should:

- Validate schema server-side.
- Rate limit by safe privacy-aware signal; do not retain raw IP unnecessarily.
- Require consent records appropriate to the communication purpose.
- Check suppression lists before automated follow-up.
- Write to a service-only lead table/queue.
- Send internal notification and create CRM/GTM work item.
- Return only generic success confirmation to the browser.
- Never expose other lead data.

Marketing read models

Build public-safe read models separate from internal operations models.

```
marketing_public_navigation()
marketing_public_pricing()
marketing_public_resources(filters)
marketing_public_resource(slug)
marketing_public_faq(topic)
```

```
marketing_public_trust_summary()
marketing_public_status_summary()
marketing_public_case_studies()
marketing_public_product_updates()
marketing_public_integrations()
```

Public content envelope

```
type MarketingEnvelope<T> = {
  data: T;
  meta: {
    publishedAt: string;
    updatedAt?: string;
    contentVersion: string;
    locale: string;
    cacheControl: "public" | "private";
  };
};
```

Public read models may expose only content marked:

```
status = published
visibility = public
claims_approval = approved
accessibility_review = passed or explicitly noted
legal_review = completed when required
```

Do not render internal draft, customer-specific, unreviewed, or compliance-sensitive content merely because it exists in a content table.

Content and claims governance

You already have `claims_register`, `content_register`, `cta_routes`, trust artifacts, GTM campaign records, consent records, suppression lists, and customer/pilot structures. Use them to create a content approval lifecycle rather than another CMS. [1]

```
Draft
→ Editorial review
→ Brand review
→ Product accuracy review
→ Legal/privacy/security/accessibility review when applicable
→ Approved
→ Scheduled
→ Published
→ Review due
→ Updated / retired
```

Claims registry fields

Field	Purpose
Claim text	Exact wording used publicly
Claim category	Product, security, privacy, accessibility, outcome, pricing, integration
Evidence reference	Source document, test, signed customer approval, audited metric
Owner	Person accountable for accuracy
Reviewer	Product, legal, security, accessibility, executive
Scope	Public, institutional-only, NDA-only, internal
Approved date	When it became usable
Review due	When it must be revalidated
Status	Draft, approved, expired, withdrawn
Page placements	Where it appears
Materiality	Whether it affects buyer decision/risk

Testimonials and customer proof

Do not create or imply fictional results. Any testimonial, case study, outcome claim, review, rating, customer logo, or influencer/ambassador statement must be real, approved, current, and properly disclosed. FTC guidance says endorsements must reflect honest opinions and cannot communicate a deceptive claim that the advertiser could not make directly; material connections must be disclosed when relevant. [2] [3]

Accessibility and frontend quality

Target WCAG 2.2 AA as a design and remediation benchmark, but do not claim conformance without a verified assessment. WCAG is organized around content being perceivable, operable, understandable, and robust; relevant basics include keyboard access, text alternatives, adequate time, navigability, predictable interactions, and error prevention. [4] [5] [6]

Required frontend controls

```
- Semantic headings and landmarks
- Skip link
- Keyboard-visible focus
- Keyboard-operable menus, dialogs, tables, forms, accordions, and carousels
- Correct label/input/error associations
- Text alternatives for non-text content
- Captions/transcripts for video/audio
- Sufficient contrast
- No color-only meaning
- Responsive reflow and zoom support
- Reduced motion support
- No autoplay audio
```

```
- Predictable navigation
- Form error prevention and correction
- Accessible status/error announcements
- Screen-reader test plan
- Touch targets appropriate for mobile
- No inaccessible drag-and-drop-only action
```

Performance budgets

Treat marketing performance as a conversion and accessibility requirement:

Metric	Target principle
Initial page experience	Fast on ordinary mobile networks
Main CTA	Visible before expensive non-critical scripts load
Images/video	Responsive, lazy-loaded below fold, correct dimensions
Fonts	Minimized, preloaded only when justified, sensible fallbacks
Analytics	Deferred/consent-aware, no blocking render
Third-party widgets	Limited, sandboxed where feasible, lazy-loaded
Product demos	Lightweight poster/preview before interactive load
SEO pages	Server/prerendered where compatible with current stack
Errors	Monitored with route/source context
Accessibility	Tested in CI and before release

Marketing analytics architecture

```
Public event
  ↓
Consent gate
  ↓
Privacy-safe event schema
  ↓
Analytics destination
  ↓
Aggregate funnel model
  ↓
Internal GTM / operations read model
  ↓
Dashboard
```

Event taxonomy

```
marketing_page_viewed
marketing_cta_clicked
marketing_demo_started
marketing_demo_submitted
marketing_demo_qualified
marketing_calendar_opened
marketing_calendar_booked
marketing_resource_viewed
marketing_resource_downloaded
marketing_webinar_registered
marketing_pricing_viewed
marketing_pilot_page_viewed
marketing_trust_center_viewed
marketing_case_study_viewed
marketing_signup_started
marketing_signup_completed
marketing_activation_completed
marketing_first_meaningful_action
marketing_referral_started
marketing_referral_completed
```

Each event needs:

```
type MarketingEvent = {
  eventName: string;
  occurredAt: string;
  anonymousId?: string;
  userId?: string;
  tenantId?: string;

  pageType?: string;
  pageSlug?: string;
  ctaId?: string;
  campaign?: string;
  source?: string;
  medium?: string;
  content?: string;

  consentState: "essential_only" | "analytics" | "marketing";
  schemaVersion: number;
};
```

Avoid sending student academic records, sensitive identifiers, grades, accommodations, financial-aid data, health data, or full support content into marketing analytics.

Master frontend/company-site execution prompt

```
You are the lead frontend architect, growth engineer, product designer,
conversion strategist, accessibility engineer, marketing operations engineer,
and trust-center builder for Semester.

Your task is to build a coherent public company site, SaaS marketing frontend,
institutional sales conversion system, student acquisition experience, content
and claims governance layer, and safe public read-model/backend contract layer.

Build on the existing Semester repository, Vite/React app, design system,
routing patterns, public assets, content models, analytics documentation,
Supabase backend, GTM tables, trust artifacts, claims register, content
register, CTA routes, consent/suppression records, and existing operations
console architecture.

Do not create a duplicate frontend application.
Do not create a detached marketing site with a separate visual identity.
Do not use generic unsourced claims.
Do not expose internal GTM, customer, financial, trust, support, or operator
tables to the public browser.
Do not create a generic public CMS that can publish unreviewed content.
Do not publish legal, security, privacy, accessibility, pricing, customer
outcome, or competitor claims without evidence and correct approval state.

Before implementation:

1. Audit the current public website/company-site routes, assets, content,
   conversion paths, lead forms, analytics, SEO metadata, legal pages,
   trust pages, accessibility, performance, and app handoff points.

2. Read:
   - README.md
   - app/README.md
   - CLAUDE.md
   - DESIGN-SYSTEM-GUIDE.md
   - ANALYTICS.md
   - SECURITY.md
   - PUBLIC-CLAIMS-APPROVAL-REGISTER.md
   - MARKET-POSITION.md
   - COMPETITION.md
   - COMPETITIVE-REVIEW.md
   - GTM and pilot documentation
   - pricing/commercial documentation
   - trust/compliance documents
   - existing content and CTA data models
   - existing company-site directory

3. Create:
   - docs/marketing/COMPANY_SITE_CURRENT_STATE.md
   - docs/marketing/COMPANY_SITE_ARCHITECTURE.md
   - docs/marketing/CONVERSION_FUNNEL_SPEC.md
   - docs/marketing/PUBLIC_CONTENT_AND_CLAIMS_GOVERNANCE.md
   - docs/marketing/SEO_AND_RESOURCE_HUB_PLAN.md
   - docs/marketing/ANALYTICS_EVENT_TAXONOMY.md
   - docs/marketing/ACCESSIBILITY_AND_PERFORMANCE_POLICY.md
```

```
   - docs/marketing/COMPANY_SITE_BACKLOG.md

4. Identify:
   - existing pages
   - missing pages
   - duplicate content
   - broken CTAs
   - claims lacking evidence
   - dead-end conversion paths
   - unclear product/app handoffs
   - accessibility gaps
   - SEO gaps
   - analytics gaps
   - trust/procurement gaps
   - public data exposure risks

Build the public site in phases.

PHASE 1 — PO conversion foundation:
- homepage
- student workspace page
- institution platform page
- registration readiness pilot page
- demo request page
- request-pricing page
- trust center
- security overview
- privacy overview
- accessibility page
- contact page
- legal navigation and pages
- clear app sign-in/sign-up handoff
- support/help handoff
- public-safe analytics events
- public lead intake endpoint
- claims approval checks
- responsive/accessibility/performance baseline

PHASE 2 — P1 growth and trust:
- use-case pages
- persona pages
- resource hub
- guides
- webinars
- product updates
- case-study framework
- comparison pages
- integration pages
- partner page
- ambassador page
- careers page
- referral path
- student planning utility
- marketing email capture/nurture handoff

PHASE 3 — P2 scale:
```

```
- internationalization readiness
- personalized content only with consent and appropriate data boundaries
- experiment framework
- structured content publishing workflow
- partner co-marketing flows
- video/demo library
- press room
- advanced SEO program
- controlled campaign landing-page templates

Site messaging:

Primary idea:
“Semester turns fragmented academic information and support workflows into a
clear, governed path to the next meaningful action.”

Primary institutional wedge:
“Semester Registration Readiness Pilot”

Do not lead with replacing every university system.
Lead with a narrow, measurable outcome, clear scope, student value, governed
data, implementation readiness, and a conversion path.

Required public information architecture:

/
 /product
 /solutions
 /pilot
 /pricing
 /demo
 /security
 /privacy
 /accessibility
 /trust
 /resources
 /compare
 /partners
 /ambassadors
 /careers
 /company
 /contact
 /legal
 /login

Required public-safe read models/RPCs:

marketing_public_navigation()
marketing_public_pricing()
marketing_public_resources(filters)
marketing_public_resource(slug)
marketing_public_faq(topic)
marketing_public_trust_summary()
marketing_public_status_summary()
marketing_public_case_studies()
marketing_public_product_updates()
```

```
marketing_public_integrations()

Every public content object must have:
- status
- visibility
- approval state
- owner
- review date
- publish date
- update date
- claim/evidence linkage where relevant
- accessibility review state where relevant
- legal/security/privacy review state where relevant

Build controlled public lead intake:

Public browser
→ server validation
→ anti-abuse/rate limit
→ consent capture
→ suppression check
→ sanitized lead queue / CRM handoff
→ internal owner assignment
→ response SLA
→ conversion tracking

Lead intake requirements:
- server-side schema validation
- clear privacy notice
- no deceptive consent
- no raw lead-data exposure
- no storage of unnecessary data
- attribution capture
- consent version capture
- suppression-list check before marketing automation
- generic public success response
- internal audit and routing

Build a shared marketing component library:

MarketingShell
MarketingNavigation
MarketingHero
MarketingCTA
MarketingProductWorkflow
MarketingUseCaseCard
MarketingPilotScopeCard
MarketingTrustSummary
MarketingMetricsDefinition
MarketingPricingTable
MarketingComparisonTable
MarketingFAQ
MarketingResourceCard
MarketingTestimonialCard
MarketingClaimDisclosure
MarketingLeadForm
```

```
MarketingConsentControl
MarketingFooter

All components must support:
- responsive layout
- keyboard operation
- screen-reader semantics
- loading/empty/error states
- reduced motion
- privacy-safe analytics
- consistent tokens with the existing app
- no unverified marketing claims

Build conversion flows:

Institution:
homepage/use-case page
→ pilot page/resource
→ demo request
→ qualification
→ calendar
→ discovery
→ pilot design
→ procurement/security review
→ paid pilot

Student:
search/social/referral/campus QR
→ student page
→ product walkthrough/planning utility
→ account creation
→ onboarding
→ first meaningful action
→ retained use
→ referral/premium conversion

Build analytics:

marketing_page_viewed
marketing_cta_clicked
marketing_demo_started
marketing_demo_submitted
marketing_demo_qualified
marketing_calendar_opened
marketing_calendar_booked
marketing_resource_viewed
marketing_resource_downloaded
marketing_webinar_registered
marketing_pricing_viewed
marketing_pilot_page_viewed
marketing_trust_center_viewed
marketing_case_study_viewed
marketing_signup_started
marketing_signup_completed
marketing_activation_completed
marketing_first_meaningful_action
```

```
marketing_referral_started
marketing_referral_completed

Do not send sensitive student data, academic records, grades, accommodation data,
financial-aid data, health data, or full support content to marketing analytics.

Claims and testimonials requirements:
- Every claim must map to evidence and owner.
- Every customer logo, case study, outcome, testimonial, or review needs
  documented permission/approval.
- Every testimonial must remain honest, current, and not misleading.
- Material relationships or incentives must be disclosed where applicable.
- Never create fictional customer proof.

Accessibility requirements:
- Build toward WCAG 2.2 AA.
- Do not claim conformance without verified testing.
- Add semantic landmarks/headings.
- Add keyboard support.
- Add visible focus states.
- Add form labels and error associations.
- Add text alternatives.
- Add captions/transcripts for media.
- Support responsive reflow and zoom.
- Support reduced motion.
- Avoid color-only meaning.
- Add automated checks and manual test instructions.

Performance requirements:
- Respect existing performance budgets.
- Prioritize mobile performance.
- Defer nonessential analytics and third-party scripts.
- Avoid blocking render with heavy media.
- Use responsive images and proper dimensions.
- Track frontend errors.
- Measure conversion without compromising privacy.

Required testing:
- page rendering
- CTA routing
- lead validation
- consent state
- suppression checks
- public-content approval gating
- claim visibility gating
- accessibility smoke tests
- keyboard navigation
- responsive layout
- analytics event schema
- no sensitive data in public responses
- SEO metadata and canonical URL tests
- link integrity
- loading/error states
- build/performance budget checks

For every phase, return:
```

```
1. Current-state findings
2. Sources reused
3. Pages/routes built
4. Components created
5. Public contracts added
6. Conversion flows implemented
7. Claims/approval implications
8. Accessibility and performance results
9. Analytics events added
10. Tests added
11. Commands run
12. Lint/typecheck/test/build output
13. Remaining content/legal/security review items
14. Next recommended branch/task
```

Recommended branch sequence

```
# Audit first
git checkout -b audit/company-site-growth-baseline

# Shared tokens/components and public architecture
git checkout -b feat/marketing-site-foundation

# Core conversion pages
git checkout -b feat/marketing-pilot-conversion

# Trust, security, privacy, and compliance pages
git checkout -b feat/marketing-trust-center

# Lead intake and consent-safe analytics
git checkout -b feat/marketing-lead-intake-analytics

# Resource hub, SEO, and content system
git checkout -b feat/marketing-resource-hub

# Student activation, ambassadors, referrals
git checkout -b feat/marketing-student-growth

# Institutional sales support surfaces
git checkout -b feat/marketing-institutional-sales-enablement
```

What to build first

1. **Homepage with one clear promise**
2. **Registration Readiness Pilot page**
3. **Demo request form and internal lead routing**
4. **Trust Center, security, privacy, accessibility, and legal pages**
5. **Institution and student product pages**
6. **Pricing/request-pricing page**
7. **App handoff and onboarding entry**

8. **Analytics taxonomy and conversion dashboard**
9. **Resource hub and search/SEO pages**
10. **Case study/customer proof only after verified pilots**

The public site must be a serious sales, trust, recruiting, and growth surface—but it must never overstate product readiness, security posture, accessibility conformance, customer outcomes, or institutional adoption.

⁂

1. https://info.lse.ac.uk/staff/divisions/Eden-Centre/Assets-EC/Documents/Claude-Guides-Dec-2025/Guide-Creating-Claude-Artifacts-V1.0-011225.pdf
2. https://www.ftc.gov/business-guidance/resources/consumer-reviews-testimonials-rule-questions-answers
3. https://www.ecfr.gov/current/title-16/chapter-I/subchapter-B/part-255
4. https://www.w3.org/TR/WCAG22/
5. https://www.w3.org/WAI/standards-guidelines/wcag/
6. https://www.w3.org/WAI/WCAG22/quickref/
7. https://www.access-board.gov/news/2023/11/27/w3c-wcag-2-2-now-available/
8. https://en.wikipedia.org/wiki/Web_Content_Accessibility_Guidelines
9. https://www.bakerbotts.com/thought-leadership/publications/2023/december/ftc-new-guidelines-for-endorsements-and-testimonials
10. https://www.morganlewis.com/pubs/2024/08/ftc-issues-final-rule-on-consumer-reviews-and-testimonials
11. https://evincedev.com/blog/the-ultimate-guide-to-googles-2021-core-web-vitals/
12. https://www.wcag.com/blog/wcag-2-2-aa-summary-and-checklist-for-website-owners/
13. https://ftcdefenselawyer.com/endorsements-and-social-media-influencers/
14. https://www.perplexity.ai/search/e3872daa-62f5-4946-abb5-867d45091336
15. https://www.perplexity.ai/search/66dc1eaa-bacc-4662-9f16-4662abcdda48
16. https://www.perplexity.ai/search/77bf3f20-b9ce-47f0-8052-e631802d3119
17. https://www.w3.org/WAI/standards-guidelines/wcag/new-in-22/
18. https://www.w3.org/WAI/WCAG22/Understanding/
19. https://www.ftc.gov/sites/default/files/attachments/press-releases/ftc-publishes-final-guides-governing-endorsements-testimonials/091005revisedendorsementguides.pdf

perplexity

building semester design system tell me everything it should provide in claude code right now

A strong design system is not just a component library—it is a **product decision system** that lets Claude Code generate consistent, accessible, responsive UI without inventing one-off colors, spacing, components, or interaction patterns.

For Claude Code, put the always-on rules in a root `CLAUDE.md`, keep canonical tokens/components in code, and add focused reusable skills for UI work. Claude Code reads project `CLAUDE.md` instructions at the start of sessions; Anthropic also supports reusable `SKILL.md` instruction packages for more specialized workflows. [1] [2]

What Semester should provide

Think of the system in six layers.

Layer	Semester should define	What Claude Code can do with it
Brand	Purpose, audience, voice, visual personality, anti-patterns	Make on-brand creative choices rather than generic SaaS UI
Foundations	Color, type, spacing, radii, elevation, motion, icons, breakpoints	Use stable primitives rather than arbitrary values
Semantic tokens	Roles such as `surface`, `text-primary`, `border-subtle`, `action-primary`	Support themes and visual updates without mass refactors
Components	API, states, variants, composition rules, accessibility behavior	Build screens from known parts instead of creating duplicates
Patterns	Forms, data views, navigation, onboarding, settings, empty/loading/error states	Solve repeated UX problems consistently
Governance	Quality checks, contribution workflow, deprecation rules, release/versioning	Keep the system coherent as the product grows

The key distinction: **primitive tokens** express raw values; **semantic tokens** express meaning. Components should almost always consume semantic tokens.

```
/* Avoid */
.card {
  background: #ffffff;
  color: #111827;
  border: 1px solid #e5e7eb;
  border-radius: 12px;
}
```

```
/* Prefer */
.card {
  background: var(--color-surface-raised);
  color: var(--color-text-primary);
  border: 1px solid var(--color-border-subtle);
  border-radius: var(--radius-lg);
}
```

That lets “Semester” evolve from one theme or brand expression to another by changing the token mapping—not every individual component.

Foundations to define

Brand and product direction

Give Claude an explicit design brief, including:

- What Semester is and who it serves.
- Primary product surfaces: marketing site, authenticated application, dashboard, editor, marketplace, documentation, mobile web, etc.
- Brand attributes—e.g. precise, editorial, warm, technical, playful, institutional, high-trust.
- The visual reference point: “quiet editorial software,” “high-density developer tool,” “premium consumer productivity app,” and so on.
- Explicit non-goals: no gradient-heavy glassmorphism, no random icon cards, no pill-everything interfaces, no excessive rounded containers, no arbitrary decoration.
- Content voice: sentence case versus title case, direct vs. playful microcopy, preferred terminology, capitalization rules, and banned language.
- Dark mode strategy: supported now, later, or intentionally unsupported.
- Internationalization assumptions: locales, text expansion, RTL needs, date/time and numeric formatting.
- Accessibility target: at minimum WCAG 2.2 AA, with clear expectations for keyboard support, focus visibility, contrast, reduced motion, errors, and screen-reader labels.

This context is important because a system can be technically consistent while still producing the wrong kind of product. Claude’s persistent project instructions are specifically intended to include architecture, conventions, commands, and common workflows. [1]

Token system

Define tokens in three levels:

1. **Reference tokens**: raw palette and scale values.
2. **Semantic tokens**: meaningful UI roles.
3. **Component tokens**: optional component-specific roles where genuinely needed.

Your initial token families should include:

- **Color**
  - Brand/accent scale
  - Neutral scale
  - Success, warning, danger, information
  - Surfaces: canvas, base, raised, overlay, inverse
  - Text: primary, secondary, muted, disabled, inverse, link
  - Borders: subtle, default, strong, focus, destructive
  - Interactive: default, hover, pressed, selected, disabled
- **Typography**
  - Font families and fallbacks
  - Type scale
  - Font weights
  - Line-height and letter-spacing
  - Fluid/responsive display styles if needed
  - Named semantic styles such as `body-sm`, `body-md`, `label-sm`, `heading-2xl`, `code-sm`
- **Spacing**
  - A constrained scale such as `0, 1, 2, 3, 4, 6, 8, 10, 12, 16, 20, 24`
  - Rules for when exceptions are allowed
- **Sizing**
  - Control heights
  - Icon sizes
  - Touch-target minima
  - Content-width and layout container sizes
- **Shape**
  - Border-widths
  - Radius scale
  - Avatar sizes
- **Elevation**
  - Surface levels
  - Shadow recipes
  - Overlay/modal treatment
- **Motion**
  - Durations, easing, interaction motion rules
  - Reduced-motion equivalents
- **Responsive layout**

    - Breakpoints
    - Grid columns
    - Page gutters
    - Max-widths
    - Stack-to-grid rules
- **Z-index**
    - A named, fixed layering ladder: base, sticky, dropdown, popover, modal, toast, tooltip
- **Opacity and effects**
    - Disabled/overlay opacity
    - Blur rules if used
    - Focus-ring treatment

Avoid raw CSS literals outside the token source unless the system has explicitly permitted them. That should be a non-negotiable Claude Code rule.

Layout grammar

A design system should provide the rules that make pages feel related:

- Global page gutters for compact, tablet, desktop, and wide desktop.
- Standard reading/content widths.
- Application-shell structure: sidebar, top bar, content region, panel region.
- Grid primitives: stack, inline, cluster, grid, split pane, center, container.
- Density modes, if Semester needs them: comfortable, standard, compact.
- Standard section rhythm: page title → context/actions → primary surface → supporting content.
- Alignment rules for labels, controls, icons, tables, and actions.
- Responsive priorities: which content stacks, hides, scrolls, truncates, or becomes a drawer.
- Safe-area handling for mobile and fixed navigation.
- Empty-state placement, loading skeleton layout, and error-state layout.

A small layout-primitives package prevents Claude from repeatedly hand-authoring subtly different flex/grid code.

Components and patterns

Essential components

Each component needs more than visual examples. Define:

- Intended use and non-use cases.
- Import path and implementation status.
- Variants and controlled props.
- Allowed composition patterns.
- All states: default, hover, focus-visible, active, selected, disabled, loading, invalid, read-only.
- Responsive behavior.
- Accessibility semantics and keyboard behavior.
- Content constraints: label lengths, icon placement, truncation, localization behavior.
- Test and Storybook expectations.

Start with the smallest durable set:

- Button, IconButton, Link, ButtonGroup.
- Text, Heading, Code, Label, Caption.
- Icon, Avatar, Badge, Status, Tag/Chip.
- Input, Textarea, Select/Combobox, Checkbox, RadioGroup, Switch, Slider, Date/Time input where needed.
- Field wrapper: label, description, required marker, validation, error text.
- Alert, Toast, Banner, InlineMessage.
- Card, Surface, Divider, List, KeyValue.
- Tabs, Breadcrumbs, Pagination, Stepper.
- Menu, Dropdown, Popover, Tooltip, Context Menu.
- Dialog, AlertDialog, Drawer, Sheet.
- Table/DataGrid, Filter bar, Search input, Sort controls.
- Skeleton, Spinner, Progress, EmptyState, ErrorState.
- Navigation: top nav, sidebar nav, mobile nav, nav group/item.
- Page header, section header, toolbar, action bar.

Product patterns

Components alone do not solve product consistency. Semester should also ship opinionated patterns for:

- Authentication: sign in, registration, password reset, invite acceptance, MFA.
- Onboarding: welcome, progressive setup, checklists, empty initial workspace.

- Forms: create/edit, validation timing, unsaved changes, autosave, destructive confirmation.
- Search and discovery: command palette, global search, filtering, no-results state.
- Data-heavy screens: list/detail, tables, row actions, bulk selection, saved views.
- CRUD lifecycle: create, success confirmation, edit, archive/delete, undo.
- Feedback: optimistic updates, loading, errors, retry, offline/degraded state.
- Permissions: restricted view, missing access, role explanation, request-access flow.
- Settings: organization/workspace/profile billing patterns.
- Notifications: in-app, toast, inbox, preferences.
- Mobile adaptation: priority content, navigation, drawers, sticky actions, touch targets.

For each pattern, document the preferred component composition, behavior, edge cases, and a production example.

Claude Code package

Your immediate goal is to make the system **machine-usable**. A good repository shape:

```
semester/
├── CLAUDE.md
├── README.md
├── docs/
│   ├── foundations.md
│   ├── content-and-voice.md
│   ├── accessibility.md
│   ├── responsive.md
│   ├── component-status.md
│   ├── patterns/
│   │   ├── forms.md
│   │   ├── data-views.md
│   │   ├── feedback.md
│   │   └── navigation.md
│   └── decisions/
├── packages/
│   ├── tokens/
│   ├── icons/
│   ├── ui/
│   └── eslint-config/
├── apps/
│   └── web/
├── .claude/
│   └── skills/
│       ├── build-ui/
│       │   └── SKILL.md
│       ├── audit-ui/
│       │   └── SKILL.md
│       └── create-component/
│           └── SKILL.md
└── .storybook/
```

Use `CLAUDE.md` for short, always-applicable instructions. Claude Code supports root or `.claude/CLAUDE.md` project memory files and supports reusable skills that live in `SKILL.md` files. [2] [1]

Starter `CLAUDE.md`

```
# Semester Design System

## Mission
Semester is the sole source of truth for UI foundations, React components,
product interaction patterns, and accessibility rules.

Build interfaces that feel deliberate, calm, information-rich, and native to
Semester. Favor clear hierarchy and useful content over decorative UI.

## Non-negotiable rules
- Use only exports from `@semester/ui`, `@semester/tokens`, and `@semester/icons`
  before creating new UI primitives.
- Never use arbitrary hex/rgb/hsl colors, pixel spacing values, radii, shadows,
  z-index values, font sizes, or animation timings in product UI.
- Use semantic tokens, not raw palette tokens, in components and application code.
- Do not add a new component until existing components and compositions have been eva
- Do not modify generated token files manually.
- Do not use emoji as interface icons. Use `@semester/icons`.
- Do not introduce a dependency for UI without explicit approval.
- Preserve keyboard operation, visible focus, semantic HTML, and WCAG 2.2 AA contras
- Support narrow screens first; do not hide critical workflows on mobile.
- Respect `prefers-reduced-motion`.

## UI workflow
1. Inspect the nearest relevant screen and existing component stories.
2. Search `@semester/ui` for a suitable component or documented composition.
3. Write a short implementation plan for new flows or new primitives.
4. Implement with tokens and existing primitives.
5. Cover loading, empty, error, disabled, and permission-restricted states.
6. Run typecheck, lint, unit tests, and Storybook checks.
7. Summarize changed files, reused system elements, and accessibility decisions.

## Component API principles
- Prefer composable primitives over large boolean-prop APIs.
- Use `variant`, `size`, and `tone` only when they map to documented design choices.
- Defaults must be safe, accessible, and visually standard.
- Every interactive component needs a disabled and loading strategy where relevant.
- Component names are singular PascalCase; props use camelCase.
- Use `data-slot` for stylable component subparts where needed.

## Accessibility
- Use native HTML controls whenever possible.
- All icon-only controls require an accessible name.
- Error messages must be programmatically associated with their field.
- Dialogs must trap focus, restore focus on close, and close on Escape unless unsafe
- Do not communicate meaning through color alone.
- Minimum interactive target: 44 by 44 CSS pixels where practical.

## Verification
```

```
Run the commands in package.json appropriate to touched packages.
For visual changes, add or update Storybook stories for every new variant and state.
```

High-value Claude skills

Create three initial skills:

- `build-ui`: Builds a feature using existing Semester components, requires state coverage and responsive behavior, and prohibits new primitives without justification.
- `audit-ui`: Reviews a route or component for token violations, accessibility, hierarchy, responsiveness, and inconsistent patterns.
- `create-component`: Creates a component only after documenting purpose, API, states, accessibility contract, stories, tests, and adoption example.

A short `build-ui/SKILL.md` might say:

```
---
name: build-ui
description: Build or revise an interface using the Semester design system.
---

Before editing:
1. Read CLAUDE.md and relevant docs in docs/patterns.
2. Inspect existing components and their Storybook stories.
3. State which Semester components and patterns will be used.

Implementation requirements:
- Reuse established components first.
- Use semantic design tokens only.
- Include loading, empty, error, disabled, and narrow-screen behavior where applicabl
- Keep visual hierarchy simple: one primary action per scope.
- Do not create a new primitive unless the existing system cannot support the need.

Before completion:
- Check keyboard navigation and focus visibility.
- Check labels, descriptions, errors, and icon-only controls.
- Run typecheck, lint, and relevant tests.
- Report token/component reuse and any proposed system gap.
```

Build order

Do **not** begin by producing 80 components. Build in this order:

1. Write the brand, product, content, accessibility, and responsive rules.
2. Build and validate tokens—including semantic tokens and light/dark mappings if dark mode matters.
3. Add layout primitives and application shell.
4. Ship typography, buttons, links, icons, fields, feedback, overlays, and navigation.
5. Establish page, form, data-view, and empty/loading/error patterns.

6. Build only the domain-specific components Semester repeatedly needs.
7. Add Storybook examples, tests, lint rules, visual regression checks, and an adoption/migration guide.
8. Measure system health: percentage of UI using system components, number of token violations, duplicate component count, accessibility failures, and time required to ship a typical screen.

For every feature, define "done" as: visual system compliance, functional behavior, responsive behavior, keyboard support, semantic markup, all meaningful states, tests, and documentation when the system changes.

Governance and guardrails

Semester needs mechanisms that stop entropy:

- **Source of truth:** Specify whether tokens live in CSS variables, TypeScript, JSON, Style Dictionary, or another pipeline—and which artifact is generated.
- **Ownership:** Name a design-system owner or review group for foundational changes.
- **Contribution template:** Problem, users affected, evidence, proposed API, alternatives considered, accessibility notes, migration impact.
- **Deprecation policy:** Never silently break a component API; mark deprecated, document migration, provide a removal version/date.
- **Visual regression testing:** Capture stories and critical product routes.
- **Automated checks:** Reject raw colors, unauthorized spacing values, unapproved z-index values, inline SVGs when icon components exist, and inaccessible interactive markup.
- **Component maturity:** Track `experimental`, `beta`, `stable`, and `deprecated` states.
- **Changelog:** Record new tokens, component changes, behavior changes, and migrations.
- **Decision records:** Preserve important choices—such as why you use 4px vs. 8px base spacing, or whether destructive actions use red buttons—so Claude does not reopen settled decisions.

The goal is that Claude Code can answer, before it writes code: *Which existing primitive solves this? Which semantic token expresses its role? What states and accessibility constraints apply? How do I verify the result?* That is the threshold at which Semester becomes a real system rather than a styling folder.

⁂

Build a complete Claude Code design system config: CLAUDE.md templates, token schemas, Figma MCP setup, and SKILL.md rules

Computer
Claude Code design-sync audit: how to map your React component library and design tokens so Claude stops generating generic UI
Computer
Show me an example CLAUDE.md for this design system
How should I structure the semantic tokens for Semester
How do I write a SKILL.md for frontend UI generation
Sources
15

Below is a production-oriented starting configuration for **Semester**: a token-first React design system that Claude Code can reliably use, audit, and extend without falling back to generic UI.

It uses four complementary mechanisms:

- Root `CLAUDE.md` for durable rules that should apply to every coding task.
- Typed semantic token files as the implementation source of truth.
- Figma MCP for reading variables, components, layout context, and selected frames in Figma.
- Task-specific `SKILL.md` files for building UI, auditing drift, and adding components. Figma's recommended Claude Code setup is its official plugin, which bundles the MCP configuration and workflow skills; Figma MCP also exposes design context including components, variables, layout information, FigJam content, and Code Connect mappings. [16] [17]

Repository structure

Use a structure where tokens and components are code-owned, Figma is a design input, and written rules are compact enough for Claude to apply consistently.

```
semester/
├── CLAUDE.md
├── package.json
├── pnpm-workspace.yaml
├── .mcp.json                          # Optional project-scoped MCP definition
├── .claude/
│   ├── settings.json
│   └── skills/
│       ├── build-ui/
│       │   ├── SKILL.md
│       │   └── references/
│       │   │   ├── component-inventory.md
│       │   │   ├── screen-checklist.md
```

```
│       │   │   └── figma-workflow.md
│       ├── audit-design-sync/
│       │   ├── SKILL.md
│       │   └── references/
│       │       └── audit-report-template.md
│       └── create-component/
│           ├── SKILL.md
│           └── references/
│               └── component-rfc-template.md
├── docs/
│   ├── architecture.md
│   ├── design-principles.md
│   ├── accessibility.md
│   ├── content-style.md
│   ├── figma-mapping.md
│   ├── component-inventory.md
│   ├── patterns/
│   │   ├── application-shell.md
│   │   ├── forms.md
│   │   ├── feedback.md
│   │   ├── navigation.md
│   │   └── data-display.md
│   └── decisions/
│       ├── 001-token-architecture.md
│       └── 002-component-api.md
├── packages/
│   ├── tokens/
│   │   ├── src/
│   │   │   ├── primitives.ts
│   │   │   ├── semantic.ts
│   │   │   ├── components.ts
│   │   │   ├── themes/
│   │   │   │   ├── light.ts
│   │   │   │   └── dark.ts
│   │   │   ├── index.ts
│   │   │   └── validate.ts
│   │   ├── dist/
│   │   └── package.json
│   ├── icons/
│   ├── ui/
│   │   ├── src/
│   │   │   ├── components/
│   │   │   ├── primitives/
│   │   │   └── styles/
│   │   ├── stories/
│   │   └── package.json
│   └── eslint-config/
├── apps/
│   ├── web/
│   └── storybook/
└── scripts/
    ├── audit-token-usage.ts
    └── validate-design-system.ts
```

The important hierarchy is:

```
Raw values → reference tokens → semantic tokens → component tokens → components → pro
```

Application code should primarily consume **semantic tokens and components**. It should not choose colors, radii, shadows, or spacings ad hoc.

Root CLAUDE.md

Place this at the repository root as `CLAUDE.md`. Claude Code supports persistent project instructions in `CLAUDE.md`; its Skills system loads more focused `SKILL.md` files dynamically when the work is relevant, so reserve the root file for universal rules rather than stuffing every implementation detail into it.[18] [19]

```
# Semester Design System — Claude Code Instructions

## System purpose

Semester is the canonical design system for this repository.

It provides:
- Design tokens and theme mappings.
- Accessible React UI primitives and composed components.
- Layout, content, responsive, and interaction patterns.
- Storybook documentation and visual regression coverage.
- A controlled mapping between Figma components/variables and production code.

The goal is not merely visual consistency. The system must enable fast product delive
without sacrificing accessibility, usability, performance, or maintainability.

## Required reading

Before non-trivial UI work, read:
- `docs/design-principles.md`
- `docs/accessibility.md`
- `docs/component-inventory.md`
- The relevant file in `docs/patterns/`
- The component's existing Storybook stories, if modifying a component.

For Figma-derived work, also read:
- `docs/figma-mapping.md`

## Core rules

1. Reuse before creating.
   - Search `@semester/ui` and the component inventory before writing custom UI.
   - Prefer composing existing primitives over creating feature-local components.
   - Do not add a component when a documented composition satisfies the need.

2. Tokens are mandatory.
   - Do not add arbitrary color, spacing, radius, shadow, z-index, font-size,
     font-weight, line-height, duration, or easing values in application UI.
   - Use semantic tokens in application and component code.
   - Raw palette/reference tokens may only be used in token definitions and theme map
   - Do not edit generated token output in `dist/`.
```

```
3. Accessibility is a release requirement.
   - Target WCAG 2.2 AA.
   - Prefer native HTML elements and controls.
   - Every keyboard-reachable element must have visible focus.
   - Every icon-only control needs an accessible name.
   - Do not communicate state or validation through color alone.
   - Use semantic landmarks and heading hierarchy.
   - Dialogs must manage focus, restore it on close, and support Escape when safe.
   - Respect `prefers-reduced-motion`.
   - Maintain at least 44 by 44 CSS pixel targets for touch interactions where pract

4. Responsive behavior is intentional.
   - Implement narrow layouts first, then enhance.
   - Do not hide critical user actions on small screens.
   - Use layout primitives, container queries, and documented breakpoints.
   - State the narrow-screen strategy for each new complex view.

5. Interface quality.
   - One primary action per decision scope.
   - Use direct sentence-case labels and useful empty/error states.
   - Avoid generic dashboard decoration: arbitrary cards, excessive pills,
     gradients, glass effects, emoji as icons, or ornamental color blocks.
   - Favor clear hierarchy, scannability, and appropriate information density.
   - Use `@semester/icons`; never use emoji or ad-hoc SVGs as UI icons.

6. Dependencies and architecture.
   - Do not introduce a UI dependency without explicit approval.
   - Do not modify public component APIs without considering migration impact.
   - Do not modify generated files manually.
   - Do not disable lint, type, test, or accessibility checks to complete a task.

## Required UI workflow

For a new or substantially revised UI flow:

1. Inspect the nearest equivalent implementation and existing Storybook stories.
2. Identify existing Semester components, patterns, and semantic tokens to reuse.
3. For a complex flow, present a short plan before implementation:
   - Existing components to reuse.
   - New composition or primitive needed, if any.
   - Responsive approach.
   - Loading, empty, error, disabled, and permission states.
   - Accessibility risks and mitigations.
4. Implement using `@semester/ui`, `@semester/tokens`, and `@semester/icons`.
5. Verify all meaningful states and keyboard behavior.
6. Run the relevant validation commands.
7. Summarize:
   - Files changed.
   - System components and tokens reused.
   - Accessibility behavior checked.
   - Any newly discovered design-system gap.

## Component API conventions

- Components use PascalCase; props use camelCase.
```

```
- Use `variant`, `size`, and `tone` only for documented visual choices.
- Prefer composition and slots over large boolean-prop APIs.
- Prefer explicit names such as `isLoading`, `isInvalid`, and `aria-label`.
- Defaults must be safe, accessible, and visually standard.
- Expose stable semantic subparts with `data-slot` only when consumers need styling h
- Every control must document its disabled, loading, invalid, and read-only behavior
- New components require TypeScript types, tests, Storybook stories, and accessibili

## Figma protocol

When a task references a Figma URL, frame, component, variable collection, or design
1. Use Figma MCP to inspect the selected node and its hierarchy.
2. Treat Figma as evidence of design intent, not as literal markup to reproduce.
3. Map Figma variables to existing semantic tokens before adding a token.
4. Map Figma components to existing React components before creating a component.
5. Preserve behavior, semantics, responsiveness, and accessibility even where Figma d
6. Record a mapping mismatch in `docs/figma-mapping.md` or an ADR; do not silently fo

## Design-system completion criteria

A UI task is not complete until:
- It uses documented components and semantic tokens.
- It covers required async, empty, error, disabled, and restricted states.
- It works at narrow and wide viewport sizes.
- It is keyboard-operable and has visible focus behavior.
- It has correct semantic HTML and accessible names.
- Typecheck, lint, tests, and relevant Storybook/visual checks pass.
- System documentation is updated if the source of truth changes.

## Commands

Use package scripts; do not guess command names.

Read `package.json` first, then run the appropriate commands for touched packages:
- `pnpm lint`
- `pnpm typecheck`
- `pnpm test`
- `pnpm test:visual`
- `pnpm storybook`
- `pnpm tokens:build`
- `pnpm tokens:validate`
- `pnpm design-system:audit`

## Documentation references

@docs/design-principles.md
@docs/accessibility.md
@docs/component-inventory.md
@docs/figma-mapping.md
```

Keep the `@docs/...` imports only if those files exist. The overall principle is correct even if you remove the import lines: persistent instructions should point Claude to concise, authoritative sources rather than duplicating every rule in a huge root file.

Semantic token schema

Naming model

Use a dot-delimited name that encodes **meaning**, not a particular visual value:

```
{category}.{role}.{emphasis-or-state}
```

Examples:

```
color.text.primary
color.text.muted
color.surface.canvas
color.surface.raised
color.border.subtle
color.action.primary.default
color.action.primary.hover
color.status.danger.text
space.4
radius.lg
shadow.overlay
motion.duration.fast
```

Avoid names based on visual appearance in consumer code:

```
# Bad application-facing names
blue.500
gray.200
purpleButton
largeRadius
cardShadow2

# Good application-facing names
color.action.primary.default
color.border.default
radius.control
shadow.raised
```

Token tier policy

Token tier	Example	Allowed consumers	Purpose
Reference	`ref.color.blue.600`	Token/theme files only	Raw palette, scale, and numerical values
Semantic	`color.text.primary`	Components and application UI	Stable role-based interface contract

Token tier	Example	Allowed consumers	Purpose
Component	`button.primary.background.default`	Component implementation only	Component-specific alias where semantic roles need narrowing
Local	CSS custom property scoped to a component	Rarely, with review	Temporary/internal implementation detail, never a new global convention

`primitives.ts`

This contains raw values. Do not import it directly into React components.

```
// packages/tokens/src/primitives.ts
export const primitive = {
  color: {
    white: "#FFFFFF",
    black: "#0A0A0B",

    neutral: {
      0: "#FFFFFF",
      25: "#FCFCFD",
      50: "#F8F9FA",
      100: "#F1F3F5",
      200: "#E9ECEF",
      300: "#DEE2E6",
      400: "#CED4DA",
      500: "#ADB5BD",
      600: "#6C757D",
      700: "#495057",
      800: "#343A40",
      900: "#212529",
      950: "#101214",
    },

    brand: {
      50: "#EEF4FF",
      100: "#D9E7FF",
      200: "#BBD7FF",
      300: "#8CC0FF",
      400: "#55A0FF",
      500: "#2F7CF6",
      600: "#1E63D8",
      700: "#194FAC",
      800: "#1B438C",
      900: "#1B396F",
    },

    red: {
      50: "#FEF2F2",
      100: "#FEE2E2",
      200: "#FECACA",
      500: "#EF4444",
```

```
      600: "#DC2626",
      700: "#B91C1C",
      900: "#7F1D1D",
    },

    amber: {
      50: "#FFFBEB",
      100: "#FEF3C7",
      500: "#F59E0B",
      600: "#D97706",
      700: "#B45309",
      900: "#78350F",
    },

    green: {
      50: "#F0FDF4",
      100: "#DCFCE7",
      500: "#22C55E",
      600: "#16A34A",
      700: "#15803D",
      900: "#14532D",
    },
  },

  space: {
    0: "0",
    px: "1px",
    0_5: "0.125rem",
    1: "0.25rem",
    1_5: "0.375rem",
    2: "0.5rem",
    2_5: "0.625rem",
    3: "0.75rem",
    4: "1rem",
    5: "1.25rem",
    6: "1.5rem",
    8: "2rem",
    10: "2.5rem",
    12: "3rem",
    16: "4rem",
    20: "5rem",
    24: "6rem",
  },

  radius: {
    none: "0",
    xs: "0.125rem",
    sm: "0.25rem",
    md: "0.375rem",
    lg: "0.5rem",
    xl: "0.75rem",
    "2xl": "1rem",
    full: "9999px",
  },

  font: {
```

```
    family: {
      sans: "Inter, ui-sans-serif, system-ui, sans-serif",
      mono: '"JetBrains Mono", ui-monospace, SFMono-Regular, monospace',
    },
    size: {
      xs: "0.75rem",
      sm: "0.875rem",
      md: "1rem",
      lg: "1.125rem",
      xl: "1.25rem",
      "2xl": "1.5rem",
      "3xl": "1.875rem",
      "4xl": "2.25rem",
    },
    weight: {
      regular: 400,
      medium: 500,
      semibold: 600,
      bold: 700,
    },
    lineHeight: {
      tight: 1.15,
      snug: 1.3,
      normal: 1.5,
      relaxed: 1.65,
    },
  },

  duration: {
    instant: "0ms",
    fast: "120ms",
    normal: "180ms",
    slow: "280ms",
  },

  easing: {
    standard: "cubic-bezier(0.2, 0, 0, 1)",
    enter: "cubic-bezier(0, 0, 0.2, 1)",
    exit: "cubic-bezier(0.4, 0, 1, 1)",
  },

  zIndex: {
    base: 0,
    raised: 10,
    sticky: 100,
    dropdown: 200,
    popover: 300,
    overlay: 400,
    modal: 500,
    toast: 600,
    tooltip: 700,
  },
} as const;
```

`semantic.ts`

These are the names React components should consume. The example uses token aliasing. If you use Style Dictionary, Tokens Studio, or another token pipeline, represent the same relationships in the appropriate source format.

```
// packages/tokens/src/semantic.ts
export const semantic = {
  color: {
    text: {
      primary: "{ref.color.neutral.950}",
      secondary: "{ref.color.neutral.700}",
      muted: "{ref.color.neutral.600}",
      disabled: "{ref.color.neutral.500}",
      inverse: "{ref.color.neutral.0}",
      link: "{ref.color.brand.700}",
      linkHover: "{ref.color.brand.800}",
      danger: "{ref.color.red.700}",
      success: "{ref.color.green.700}",
      warning: "{ref.color.amber.700}",
    },

    icon: {
      primary: "{color.text.primary}",
      secondary: "{color.text.secondary}",
      muted: "{color.text.muted}",
      disabled: "{color.text.disabled}",
      inverse: "{color.text.inverse}",
      brand: "{color.action.primary.default}",
      danger: "{color.status.danger.strong}",
    },

    surface: {
      canvas: "{ref.color.neutral.25}",
      base: "{ref.color.neutral.0}",
      raised: "{ref.color.neutral.0}",
      sunken: "{ref.color.neutral.50}",
      overlay: "{ref.color.neutral.0}",
      inverse: "{ref.color.neutral.950}",
      selected: "{ref.color.brand.50}",
      disabled: "{ref.color.neutral.100}",
    },

    border: {
      subtle: "{ref.color.neutral.200}",
      default: "{ref.color.neutral.300}",
      strong: "{ref.color.neutral.400}",
      focus: "{ref.color.brand.500}",
      disabled: "{ref.color.neutral.200}",
      danger: "{ref.color.red.500}",
    },

    action: {
      primary: {
        default: "{ref.color.brand.600}",
```

```
      hover: "{ref.color.brand.700}",
      active: "{ref.color.brand.800}",
      foreground: "{ref.color.white}",
      disabled: "{ref.color.neutral.300}",
      disabledForeground: "{ref.color.neutral.600}",
    },

    secondary: {
      default: "{color.surface.base}",
      hover: "{ref.color.neutral.50}",
      active: "{ref.color.neutral.100}",
      foreground: "{color.text.primary}",
      border: "{color.border.default}",
    },

    subtle: {
      default: "transparent",
      hover: "{ref.color.neutral.100}",
      active: "{ref.color.neutral.200}",
      foreground: "{color.text.primary}",
    },

    danger: {
      default: "{ref.color.red.600}",
      hover: "{ref.color.red.700}",
      active: "{ref.color.red.900}",
      foreground: "{ref.color.white}",
    },
  },

  status: {
    info: {
      subtle: "{ref.color.brand.50}",
      default: "{ref.color.brand.600}",
      strong: "{ref.color.brand.700}",
      text: "{ref.color.brand.800}",
    },
    success: {
      subtle: "{ref.color.green.50}",
      default: "{ref.color.green.600}",
      strong: "{ref.color.green.700}",
      text: "{ref.color.green.900}",
    },
    warning: {
      subtle: "{ref.color.amber.50}",
      default: "{ref.color.amber.600}",
      strong: "{ref.color.amber.700}",
      text: "{ref.color.amber.900}",
    },
    danger: {
      subtle: "{ref.color.red.50}",
      default: "{ref.color.red.600}",
      strong: "{ref.color.red.700}",
      text: "{ref.color.red.900}",
    },
  },
```

```
    focus: {
      ring: "{ref.color.brand.500}",
      ringOffset: "{color.surface.base}",
    },
  },

  typography: {
    body: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.sm}",
        fontWeight: "{ref.font.weight.regular}",
        lineHeight: "{ref.font.lineHeight.normal}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.md}",
        fontWeight: "{ref.font.weight.regular}",
        lineHeight: "{ref.font.lineHeight.normal}",
      },
    },

    label: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.sm}",
        fontWeight: "{ref.font.weight.medium}",
        lineHeight: "{ref.font.lineHeight.snug}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.md}",
        fontWeight: "{ref.font.weight.medium}",
        lineHeight: "{ref.font.lineHeight.snug}",
      },
    },

    heading: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.lg}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.xl}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
      },
      lg: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.2xl}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
```

```
      },
    },
  },

  space: {
    inline: {
      xs: "{ref.space.1}",
      sm: "{ref.space.2}",
      md: "{ref.space.3}",
      lg: "{ref.space.4}",
    },
    stack: {
      xs: "{ref.space.2}",
      sm: "{ref.space.3}",
      md: "{ref.space.4}",
      lg: "{ref.space.6}",
      xl: "{ref.space.8}",
      "2xl": "{ref.space.12}",
    },
    inset: {
      xs: "{ref.space.2}",
      sm: "{ref.space.3}",
      md: "{ref.space.4}",
      lg: "{ref.space.6}",
    },
    page: {
      mobile: "{ref.space.4}",
      tablet: "{ref.space.6}",
      desktop: "{ref.space.8}",
    },
  },

  size: {
    control: {
      sm: "2rem",
      md: "2.5rem",
      lg: "3rem",
    },
    icon: {
      xs: "0.875rem",
      sm: "1rem",
      md: "1.25rem",
      lg: "1.5rem",
    },
    target: {
      minimum: "2.75rem",
    },
    content: {
      readable: "42rem",
      default: "72rem",
      wide: "90rem",
    },
  },

  radius: {
    control: "{ref.radius.md}",
```

```
    surface: "{ref.radius.lg}",
    overlay: "{ref.radius.xl}",
    pill: "{ref.radius.full}",
  },

  border: {
    default: "1px",
    strong: "2px",
  },

  shadow: {
    raised: "0 1px 2px rgb(16 18 20 / 0.06), 0 1px 3px rgb(16 18 20 / 0.10)",
    overlay: "0 12px 32px rgb(16 18 20 / 0.16), 0 2px 8px rgb(16 18 20 / 0.08)",
  },

  motion: {
    duration: {
      instant: "{ref.duration.instant}",
      fast: "{ref.duration.fast}",
      normal: "{ref.duration.normal}",
      slow: "{ref.duration.slow}",
    },
    easing: {
      standard: "{ref.easing.standard}",
      enter: "{ref.easing.enter}",
      exit: "{ref.easing.exit}",
    },
  },

  zIndex: {
    base: "{ref.zIndex.base}",
    sticky: "{ref.zIndex.sticky}",
    dropdown: "{ref.zIndex.dropdown}",
    popover: "{ref.zIndex.popover}",
    overlay: "{ref.zIndex.overlay}",
    modal: "{ref.zIndex.modal}",
    toast: "{ref.zIndex.toast}",
    tooltip: "{ref.zIndex.tooltip}",
  },
} as const;
```

Component-token example

Use component tokens when a component needs a stable contract that maps to semantic intent but should not expose every internal choice to product teams.

```
// packages/tokens/src/components.ts
export const component = {
  button: {
    primary: {
      background: {
        default: "{color.action.primary.default}",
        hover: "{color.action.primary.hover}",
        active: "{color.action.primary.active}",
```

```
        disabled: "{color.action.primary.disabled}",
      },
      foreground: {
        default: "{color.action.primary.foreground}",
        disabled: "{color.action.primary.disabledForeground}",
      },
      focusRing: "{color.focus.ring}",
      height: {
        sm: "{size.control.sm}",
        md: "{size.control.md}",
        lg: "{size.control.lg}",
      },
      radius: "{radius.control}",
      gap: "{space.inline.sm}",
    },
  },

  field: {
    background: {
      default: "{color.surface.base}",
      disabled: "{color.surface.disabled}",
    },
    border: {
      default: "{color.border.default}",
      hover: "{color.border.strong}",
      focus: "{color.border.focus}",
      invalid: "{color.border.danger}",
      disabled: "{color.border.disabled}",
    },
    text: {
      value: "{color.text.primary}",
      placeholder: "{color.text.muted}",
      disabled: "{color.text.disabled}",
    },
    height: {
      sm: "{size.control.sm}",
      md: "{size.control.md}",
      lg: "{size.control.lg}",
    },
    radius: "{radius.control}",
  },
} as const;
```

Required token validation

Add a script that fails CI if:

- An application package imports `primitive` tokens.
- CSS/TSX contains raw hex, `rgb()`, `hsl()`, arbitrary shadows, or undocumented z-index values.
- A semantic token points directly to a component token, creating reverse dependency.
- Token aliases reference missing keys or circular references.

- A token change creates insufficient contrast for documented text/control combinations.
- Generated CSS, JSON, TypeScript, and Figma token exports differ from source.

Figma MCP setup

Recommended plugin installation

Figma recommends its official Claude Code plugin because it includes MCP server settings and agent skills for common workflows. Run this in your terminal:

```
claude plugin install figma@claude-plugins-official
```

Then restart Claude Code, run `/plugin`, open the **Installed** tab, select the Figma plugin, and complete the authorization flow. [16]

Confirm the connection in Claude Code:

```
/mcp
```

Manual remote setup

If you prefer manual configuration, Figma documents its hosted remote MCP endpoint as:

```
claude mcp add --transport http figma https://mcp.figma.com/mcp
```

Then restart Claude Code and authenticate when prompted. Figma identifies its remote server as the preferred option; it is hosted at `https://mcp.figma.com/mcp`. [17] [20]

Inspect your configuration with:

```
claude mcp list
claude mcp get figma
```

Manual desktop setup

The desktop server is useful when you are actively working in an open Figma file:

1. Install and open Figma Desktop.
2. Open a Figma Design file.
3. Switch to Dev Mode with nothing selected.
4. Enable the MCP server in the right sidebar.
5. Add it to Claude Code:

```
claude mcp add --transport http figma-desktop http://127.0.0.1:3845/mcp
```

6. Restart Claude Code and use `/mcp` to verify it is available. [16]

Project-local `.mcp.json`

Use this only if your team wants the connection declared alongside the repository. Do not commit secrets or static access tokens.

```
{
  "mcpServers": {
    "figma": {
      "type": "http",
      "url": "https://mcp.figma.com/mcp"
    }
  }
}
```

Prefer the official plugin when possible; it reduces configuration drift and gives the team the same Figma-aware workflow baseline. Figma MCP can surface design variables, components, layout context, and Code Connect mappings, which is exactly the context Claude needs to translate approved designs into your existing system rather than inventing new components. [16]

Figma-to-code mapping file

Create `docs/figma-mapping.md` and maintain it as an explicit contract.

```
# Semester Figma-to-Code Mapping

## Source of truth

- Code is the source of truth for component behavior, accessibility, and public APIs
- Figma is the source of truth for approved visual design and design exploration.
- Token source files in `packages/tokens/src/` are the source of truth for productio
- Figma variables are synchronized exports of the token system where possible.

## Variable mapping

| Figma collection | Figma variable | Code token | Notes |
|---|---|---|---|
| Semester / Semantic / Color | `Text / Primary` | `color.text.primary` | Default rea
| Semester / Semantic / Color | `Surface / Base` | `color.surface.base` | Standard co
| Semester / Semantic / Color | `Action / Primary / Default` | `color.action.primary
| Semester / Semantic / Space | `Stack / MD` | `space.stack.md` | Default vertical gr
| Semester / Component / Button | `Primary / Background / Hover` | `button.primary.ba

## Component mapping

| Figma component | React export | Status | Notes |
|---|---|---|---|
```

```
| `Button` | `@semester/ui/Button` | Stable | Figma variants map to `variant`, `size
| `Text field` | `@semester/ui/TextField` | Stable | Always use with `Field` when lab
| `Dialog` | `@semester/ui/Dialog` | Stable | Code owns focus management and Escape b
| `Data table` | `@semester/ui/DataTable` | Beta | Use only in approved data-heavy ro
| `Metric card` | No direct export | Deprecated | Compose `Surface`, `Stack`, `Text`

## Mapping rules

- A Figma component is not automatically a React component.
- Figma variants map only to documented public props.
- Local Figma styles must not become production tokens without review.
- New Figma variables require a code-token proposal before use in product UI.
- Record unresolved mismatches in an ADR or GitHub issue.
```

Design-sync audit

Use this mapping to audit drift between Figma and React:

Audit area	Verify	Typical failure	Fix
Token parity	Every approved Figma semantic variable has a code equivalent	Figma-only colors or spacing	Add or map an approved semantic token
Component parity	Figma variants map to a React component prop/state	Designers create a variant unavailable in code	Add documented API or simplify the Figma variant
State parity	Default, hover, focus, active, disabled, loading, error are represented	Figma shows only default/hover	Code remains accessible; document missing design state
Layout parity	Page gutters, grids, breakpoint behavior agree	Desktop frame treated as pixel-perfect universal layout	Encode responsive behavior in layout primitives
Content parity	Labels, empty copy, errors, and truncation have rules	Generic lorem or visually short sample content	Test realistic and localized strings
Accessibility parity	Semantic HTML and focus behavior exist beyond Figma	Clickable `div`, icon buttons without names	Use correct controls and accessible labels
Code Connect parity	Figma components reference real code components where supported	Figma library drifts into lookalikes	Link/maintain Code Connect references

Use this as a repeatable audit prompt in Claude Code:

```
Audit the supplied Figma frame against Semester.

1. Inspect the Figma hierarchy, variables, component instances, and layout.
2. Inspect the corresponding React route and all Semester components it uses.
3. Produce a table with:
   - Figma node or variable
   - Closest code token/component
   - Match status: match, partial, missing, obsolete
   - Severity: blocker, major, minor
   - Recommended resolution
4. Check typography, spacing, color, border, radius, elevation, responsive layout,
   interaction states, keyboard behavior, labels, empty/loading/error states,
```

```
   and component API consistency.
5. Do not make changes until the audit is complete.
6. Prefer mapping to existing Semester concepts over creating new tokens/components.
```

UI generation skill

Create `.claude/skills/build-ui/SKILL.md`.

Skills are self-contained folders whose required `SKILL.md` uses YAML frontmatter; Claude loads the skill when it becomes relevant and may load supporting references only as needed. Anthropic recommends that MCP-using skills reference tools with fully qualified `ServerName:tool_name` syntax to avoid ambiguity. [19] [21]

```
---
name: build-semester-ui
description: Build or revise React interfaces using the Semester design system. Use t
version: 1.0.0
---

# Build Semester UI

## Goal

Create production-ready React interfaces that are visibly and behaviorally native to

Prioritize:
1. Existing system reuse.
2. Semantic token correctness.
3. Accessible behavior.
4. Responsive resilience.
5. State completeness.
6. Minimal new abstractions.

## Inputs to inspect

Before writing code, inspect:

- Root `CLAUDE.md`.
- `docs/component-inventory.md`.
- Relevant files in `docs/patterns/`.
- Existing comparable screens and Storybook stories.
- `docs/figma-mapping.md` if the work includes Figma.
- The nearest package-level instructions, if present.

If a Figma URL, node, or selected Figma design is supplied:
- Inspect it using the installed Figma MCP tools.
- Use fully qualified Figma MCP tool names where required by the local MCP configura
- Identify its components, variants, variables, layout rules, and content hierarchy.
- Do not treat a static Figma frame as a complete interaction specification.

## Mandatory planning format

For a new screen, complex component, or substantial redesign, provide this concise pl
```

```
| Concern | Decision |
|---|---|
| Existing components | List components from `@semester/ui` to reuse |
| Pattern | Name the documented product pattern, or say no pattern exists |
| Tokens | List semantic token groups likely to be used |
| Responsive strategy | Explain narrow, intermediate, and wide behavior |
| States | List loading, empty, error, disabled, permissions, and success states |
| Accessibility | Identify semantic structure, focus, label, and keyboard requiremen
| System gap | State `none` or describe the smallest missing primitive/pattern |

Do not create code before identifying whether an existing component or composition wo

## Implementation rules

### Components

- Import from `@semester/ui`, `@semester/tokens`, and `@semester/icons`.
- Search the component inventory before creating any new primitive.
- Prefer composition of `Stack`, `Inline`, `Surface`, `Text`, `Button`, `Field`,
  and domain components over feature-local styling.
- Do not use generic `div` click handlers when a native interactive element is availa
- Do not create `Card`, `Panel`, `Widget`, or `Box` wrappers solely for decoration.
- Do not use emoji or arbitrary SVGs for UI icons.

### Tokens and style

- Use semantic tokens only in product UI.
- Do not use literal hex/rgb/hsl values, arbitrary pixel spacing, raw shadow strings
  arbitrary z-indexes, or undocumented animation values.
- Do not consume primitive tokens outside token/theme packages.
- Use layout primitives and semantic spacing roles rather than one-off margins.
- Use component tokens inside component implementations only.

### Responsive design

- Build narrow layouts first.
- Preserve primary actions and critical information on narrow screens.
- Use documented breakpoints or container queries.
- Tables may scroll horizontally only when no responsible card/list transformation ex
- Do not rely on hover as the only way to expose important actions.

### Accessibility

- Prefer semantic HTML and native inputs/buttons/links.
- Use visible focus styles from Semester tokens.
- Give every icon-only button an accessible name.
- Associate labels, descriptions, and validation errors with fields.
- Use `aria-live` only for relevant dynamic updates.
- Do not use color as the sole signal.
- Manage focus in dialogs, menus, comboboxes, and other composite widgets.
- Respect reduced-motion preferences.

### States

For each meaningful data-dependent surface, implement or deliberately address:
- Loading.
```

```
- Empty.
- Error and retry.
- Disabled or unavailable action.
- Success/confirmation where the action is consequential.
- Permission-restricted state when access can be denied.
- Long content, truncation, and localization expansion where relevant.

## Figma implementation protocol

When implementing from Figma:

1. Extract intent:
   - Page hierarchy.
   - Components and variants.
   - Semantic variables.
   - Layout constraints.
   - Visible states.
2. Map intent:
   - Figma variable → existing Semester semantic token.
   - Figma component → existing React component.
   - Figma layout → Semester layout primitive.
3. Resolve mismatches:
   - Reuse an existing code behavior when Figma is incomplete.
   - Do not introduce a raw value to force pixel parity.
   - Do not create a new token/component without identifying the reusable need.
4. Implement:
   - Maintain semantic HTML, keyboard interaction, state coverage, and responsive beh
5. Document:
   - Note any approved new mapping in `docs/figma-mapping.md`.

## New component threshold

A new component is justified only when all are true:

- No existing component or documented composition meets the need.
- The need appears in at least two product contexts or is clearly foundational.
- Its public API, states, accessibility contract, and responsive behavior are defined
- It can be documented with stories and tested.
- It does not encode feature-specific business logic.

If the threshold is not met, compose existing primitives locally.

## Required verification

Before declaring work complete:

1. Run typecheck, lint, and relevant tests.
2. Add or update Storybook stories for new visual variants and states.
3. Test keyboard navigation and visible focus.
4. Test narrow and wide viewports.
5. Check realistic, long, and empty content.
6. Confirm no token policy violations.
7. Summarize:
   - Components reused.
   - Tokens/patterns used.
   - States covered.
```

```
   - Accessibility behavior verified.
   - Any system gap or Figma mismatch found.

## Completion format

Return:

### Implemented
- Concise summary of UI work.

### System reuse
- Components, patterns, and tokens reused.

### State coverage
- Loading, empty, error, permissions, disabled, responsive behavior.

### Accessibility
- Semantics, labels, keyboard behavior, focus behavior.

### Validation
- Commands run and result.

### Follow-up
- Only list concrete system work still needed.
```

Design-sync audit skill

Create `.claude/skills/audit-design-sync/SKILL.md` to make "stop generating generic UI" enforceable rather than aspirational.

```
---
name: audit-semester-design-sync
description: Audit Figma designs and React implementation for Semester token, compone
version: 1.0.0
---

# Audit Semester Design Sync

## Goal

Compare a Figma design, React implementation, and Semester design-system contracts.
Find design drift before changing code.

## Required sources

Inspect:
- Root `CLAUDE.md`.
- `docs/figma-mapping.md`.
- `docs/component-inventory.md`.
- Relevant source files and Storybook stories.
- Figma hierarchy, components, variables, and layout through Figma MCP when a Figma

## Audit dimensions
```

```
Evaluate:

1. Token discipline
   - Raw values in product UI.
   - Incorrect semantic-role usage.
   - Missing Figma-to-code token mapping.
   - Theme and dark-mode compatibility, if supported.

2. Component parity
   - Figma components and variants mapped to stable React exports.
   - Duplicate or feature-local equivalents.
   - Props that no longer represent design intent.

3. State completeness
   - Default, hover, focus-visible, active, selected, disabled, loading, invalid,
     empty, error, success, and permission states where relevant.

4. Layout and responsive behavior
   - Spacing, grid, hierarchy, overflow, content width, mobile transformations,
     touch targets, and action priority.

5. Accessibility
   - Semantic elements, accessible names, labels, descriptions, errors,
     focus management, keyboard interaction, contrast, and reduced motion.

6. Content and hierarchy
   - Clear primary action.
   - Realistic content length.
   - Empty/error explanations and recovery actions.
   - Sentence case and Semester terminology.

## Required output

Produce these sections before recommending code changes:

### Executive assessment
One paragraph: alignment level and highest-risk issues.

### Findings

| Severity | Figma/code location | Category | Finding | Existing Semester solution |
|---|---|---|---|---|---|
| Blocker/Major/Minor | Node, file, component | Token/component/a11y/layout/state/co

### Missing system capabilities

| Candidate capability | Repeated need? | Existing composition viable? | Recommendat
|---|---:|---:|---|
| Name | Yes/No | Yes/No | Compose, document pattern, or propose component |

### Implementation order
List fixes in dependency order: blockers, shared primitives, screen composition, pol

## Rules

- Do not recommend pixel-perfect copying when it violates Semester tokens or accessi
```

```
- Do not recommend a new component until existing components/compositions are exhaus
- Classify raw token usage as a defect, not an aesthetic preference.
- Favor the smallest correction that strengthens the system rather than a one-off ex
- Never modify code during audit mode unless explicitly instructed after the report.
```

Component-creation skill

This prevents Claude from responding to every visual difference by creating a new “Card”, “Section”, or “FancyButton.”

```
---
name: create-semester-component
description: Propose and implement a reusable Semester component only when existing c
version: 1.0.0
---

# Create Semester Component

## Gate

Do not create a new component until you have:

1. Searched `@semester/ui`, the component inventory, and relevant patterns.
2. Identified at least two legitimate use cases or a foundational platform need.
3. Written a concise proposal covering:
   - Problem and users.
   - Existing alternatives considered.
   - Proposed public API.
   - Variants and states.
   - Accessibility and keyboard contract.
   - Responsive behavior.
   - Token dependencies.
   - Migration/adoption plan.
4. Obtained approval if the repository requires it.

## Implementation requirements

A new component must include:
- TypeScript implementation and public export.
- Semantic/component token mapping.
- Unit or interaction tests.
- Storybook stories for all variants and significant states.
- Keyboard/focus behavior.
- Loading, disabled, invalid, and empty/error behavior where applicable.
- Documentation in the component inventory.
- A production usage example or documented composition.

## API rules

- Prefer composition over booleans.
- Do not expose raw CSS or arbitrary style props as the primary API.
- Avoid feature-specific business logic.
- Do not add `className` escape hatches without a documented reason.
- Keep variant axes orthogonal: `variant`, `size`, `tone`, `state`.
```

```
- Preserve backwards compatibility or document migration.

## Completion report

Return:
- Why existing options were insufficient.
- API and accessibility contract.
- Stories/tests added.
- Tokens used.
- First adoption target.
- Any Figma mapping additions.
```

Make the system enforceable

Configuration alone will not stop generic UI. Add automated constraints.

Token lint rules

Implement ESLint/stylelint/custom AST checks to block:

```
// Reject in app and component packages
color: "#2F7CF6"
padding: "13px"
boxShadow: "0 9px 20px rgba(0,0,0,.13)"
zIndex: 9999
transition: "all 0.2s ease"
```

Require:

```
// Accept: semantic roles or typed token helpers
backgroundColor: tokens.color.action.primary.default
padding: tokens.space.inset.md
boxShadow: tokens.shadow.raised
zIndex: tokens.zIndex.popover
transitionDuration: tokens.motion.duration.fast
```

Component inventory metadata

Keep a machine-readable inventory so Claude can locate existing components without relying entirely on prose:

```
// packages/ui/src/component-manifest.ts
export const componentManifest = {
  Button: {
    status: "stable",
    import: "@semester/ui/Button",
    useFor: ["primary actions", "form submission", "destructive confirmation"],
    avoidFor: ["navigation to another URL"],
    variants: ["primary", "secondary", "subtle", "danger"],
    sizes: ["sm", "md", "lg"],
    accessibility: ["native button semantics", "loading state", "visible focus"],
```

```
  },

  TextField: {
    status: "stable",
    import: "@semester/ui/TextField",
    useFor: ["single-line user input"],
    avoidFor: ["long-form content", "search with suggestions"],
    compositions: ["Field", "FormField"],
    accessibility: ["label association", "describedby support", "invalid state"],
  },

  DataTable: {
    status: "beta",
    import: "@semester/ui/DataTable",
    useFor: ["dense multi-column datasets"],
    avoidFor: ["small mobile-first collections"],
    accessibility: ["column headers", "sort status", "keyboard navigation"],
  },
} as const;
```

CI gate

A practical CI sequence:

```
pnpm tokens:validate
pnpm tokens:build
pnpm lint
pnpm typecheck
pnpm test
pnpm test:visual
pnpm design-system:audit
```

`design-system:audit` should detect at least:

- Primitive token imports outside allowed token packages.
- Raw colors and hardcoded design values.
- Components lacking Storybook coverage.
- Components missing an accessibility note or test.
- Figma-mapped components absent from the React manifest.
- Deprecated component imports.
- Unapproved public API changes.
- New UI files that bypass `@semester/ui` despite available equivalents.

Immediate rollout order

1. Add the root `CLAUDE.md`.
2. Create `docs/component-inventory.md` and `docs/figma-mapping.md`.
3. Install the Figma plugin with `claude plugin install figma@claude-plugins-official`; authorize and validate it through `/mcp`.[16]
4. Establish the reference → semantic → component token pipeline.
5. Add `build-semester-ui` and `audit-semester-design-sync` skills.
6. Migrate the most reused primitives first: text, buttons, fields, surfaces, overlays, navigation.
7. Add token linting and visual regression checks to CI.
8. Use the audit skill on the next 3–5 production screens; turn repeat audit findings into components, patterns, or stricter lint rules.

The most important operational rule is simple: **Figma describes intent; Semester code defines the reusable accessible implementation.** Claude should use Figma MCP to understand a screen, but it should use Semester tokens, components, patterns, and validation rules to ship it.

⁂

Generate a complete Claude Code design system toolkit: production CLAUDE.md templates, token schemas, Figma MCP setup, and SKILL.md rules

Computer
Audit your React component library against Figma variables with Claude Code: automated token mapping, drift detection, and CI sync report
Computer
Show me the root CLAUDE.md file contents
Show the SKILL.md code for the audit skill
How do I configure the Figma MCP server

Below is a complete, copy-ready toolkit for **Semester**: a React design system governed through Claude Code, tokens, Figma MCP, skills, and CI drift reports.

Use the root `CLAUDE.md` for lasting project rules, and use focused skills for procedural work such as auditing design drift. Claude Code loads skills from `.claude/skills/<name>/SKILL.md`; the file's YAML frontmatter controls discovery and invocation, while the body contains the workflow.[31]

Toolkit layout

```
semester/
├── CLAUDE.md
├── .mcp.json
├── .claude/
│   ├── settings.json
│   └── skills/
│       ├── build-semester-ui/
│       │   └── SKILL.md
│       ├── audit-semester-design-sync/
│       │   └── SKILL.md
│       └── create-semester-component/
│           └── SKILL.md
├── docs/
│   ├── component-inventory.md
│   ├── figma-mapping.md
│   ├── accessibility.md
│   ├── design-principles.md
│   └── patterns/
│       ├── forms.md
│       ├── navigation.md
│       ├── data-views.md
│       └── feedback.md
├── packages/
│   ├── tokens/
│   │   ├── src/
│   │   │   ├── primitives.ts
│   │   │   ├── semantic.ts
│   │   │   ├── components.ts
│   │   │   ├── themes/
│   │   │   │   ├── light.ts
│   │   │   │   └── dark.ts
│   │   │   └── index.ts
│   │   └── package.json
│   ├── ui/
│   │   ├── src/
│   │   ├── stories/
│   │   └── component-manifest.ts
│   └── eslint-config/
├── scripts/
│   ├── audit-token-usage.ts
│   ├── build-figma-token-map.ts
│   ├── detect-figma-drift.ts
│   └── design-system-report.ts
└── reports/
    └── design-system/
```

The enforced value chain should be:

```
Reference values
  → semantic tokens
    → component tokens
```

```
      → React components
        → product screens
```

Product code should almost never consume a raw palette value such as `blue.600`. It should consume a role such as `color.action.primary.default`.

Root CLAUDE.md

Save this exact file at `CLAUDE.md` in the repository root.

```
# Semester Design System

## Purpose

Semester is the canonical product design system for this repository.

It defines:
- Design tokens and theme mappings.
- Accessible React primitives and components.
- Layout, typography, responsive, interaction, and content conventions.
- Figma-to-code mappings.
- Storybook, test, lint, visual-regression, and CI quality requirements.

Build interfaces that are deliberate, calm, accessible, information-rich, and native
to Semester. Do not generate generic dashboard UI, decorative card grids, arbitrary
pills, gradient-heavy surfaces, or one-off visual values.

## Required reading

Before doing non-trivial UI work, read:

- `docs/design-principles.md`
- `docs/accessibility.md`
- `docs/component-inventory.md`
- The relevant document in `docs/patterns/`
- Existing Storybook stories for components being changed
- `docs/figma-mapping.md` when work references Figma

## Source-of-truth policy

- Code is the source of truth for component behavior, accessibility, public APIs,
  responsive behavior, and production token values.
- Figma is the source of truth for approved visual intent and design exploration.
- `packages/tokens/src/` is the source of truth for token definitions.
- Generated token artifacts must never be edited manually.
- Figma variables must map to semantic code tokens before they are used in product UI
- A Figma component is not automatically a React component.

## Non-negotiable rules

### Reuse system assets first

- Search `@semester/ui`, Storybook, `component-manifest.ts`, and the component
  inventory before writing UI.
```

```
- Prefer composing existing primitives over creating feature-local components.
- Do not create a new primitive, component, token, layout abstraction, or visual
  variant until existing options have been evaluated.
- Do not create generic names such as `Card`, `Box`, `Panel`, `Widget`, or
  `FancyButton` for feature-local wrappers.

### Use tokens only

- Use semantic tokens in React components and product application code.
- Do not import raw/reference tokens outside `packages/tokens`.
- Do not add arbitrary hex, rgb, hsl, oklch, spacing, radius, border, shadow,
  font, z-index, opacity, duration, or easing values in UI code.
- Do not use arbitrary Tailwind values such as `p-[13px]`, `bg-[#...]`,
  `z-[^3_9999]`, or `shadow-[...]`.
- Component-specific tokens are allowed only inside their owning component package.
- Do not edit generated CSS, JSON, TypeScript, or Figma-export token artifacts.

### Build accessible interfaces

- Meet WCAG 2.2 AA unless a documented exception exists.
- Prefer native HTML semantics and native controls.
- Use visible focus styles for every keyboard-reachable interactive element.
- Every icon-only control must have an accessible name.
- Associate field labels, descriptions, validation messages, and errors with fields.
- Do not communicate meaning through color alone.
- Dialogs, menus, popovers, comboboxes, and similar widgets must manage focus correc
- Restore focus after closing overlays.
- Support Escape to dismiss transient overlays when safe.
- Respect `prefers-reduced-motion`.
- Maintain a practical minimum 44 by 44 CSS pixel touch target.

### Design responsive interfaces intentionally

- Start with narrow layouts and enhance for larger viewports.
- Use Semester layout primitives, documented breakpoints, or approved container queri
- Do not hide critical actions or information on smaller screens.
- Do not rely exclusively on hover for essential actions.
- Handle content overflow, localization expansion, long labels, and long lists.
- State the narrow, intermediate, and wide behavior for complex layouts.

### Maintain product quality

- Establish clear hierarchy and one primary action per decision scope.
- Use sentence case for interface text unless a documented product term requires othe
- Use `@semester/icons`; never use emoji as product interface icons.
- Include meaningful loading, empty, error, disabled, success, and permission-restric
  states when relevant.
- Avoid unnecessary decoration. Prioritize useful content and scannability.
- Do not introduce UI dependencies without explicit approval.
- Do not weaken lint, typecheck, test, accessibility, or visual-regression checks.

## Required implementation workflow

For a new screen, new flow, or substantial UI change:

1. Inspect the closest existing implementation and relevant Storybook stories.
```

```
2. Identify the Semester components, layout primitives, patterns, and semantic tokens
3. For non-trivial work, provide a concise plan before editing:
   - Components and patterns to reuse.
   - Required states.
   - Responsive behavior.
   - Accessibility considerations.
   - Proposed design-system gap, if any.
4. Implement using `@semester/ui`, `@semester/tokens`, and `@semester/icons`.
5. Check keyboard operation, focus visibility, semantic structure, labels, and errors
6. Verify narrow and wide viewport behavior.
7. Run relevant validation commands.
8. Summarize reused system assets and any unresolved system gap.

## Component API rules

- Use PascalCase for component names and camelCase for props.
- Use `variant`, `size`, and `tone` only for documented, stable design choices.
- Prefer composition and slots over large boolean-prop APIs.
- Use explicit booleans such as `isLoading`, `isInvalid`, and `isDisabled`.
- Defaults must be safe, accessible, and visually standard.
- Do not expose arbitrary style props as a primary API.
- Use `data-slot` only for stable, documented component subparts.
- New components require TypeScript types, tests, Storybook coverage, documentation,
  and accessibility notes.

## Figma protocol

When a task includes a Figma URL, node ID, selected layer, variable, component, or fi

1. Use Figma MCP to inspect the design hierarchy, layout, variables, components,
   component properties, and visible states.
2. Treat Figma as design intent, not literal HTML/CSS to duplicate.
3. Map Figma variables to existing Semester semantic tokens before adding tokens.
4. Map Figma components to existing React components before creating components.
5. Preserve accessibility, semantics, responsive rules, error handling, and interacti
   behavior even when Figma does not show them.
6. Record important unmapped variables/components in `docs/figma-mapping.md`.
7. Do not create a Figma-driven exception that bypasses Semester.

## Completion requirements

A UI task is incomplete until:

- It uses approved Semester components and semantic tokens.
- It handles relevant loading, empty, error, disabled, success, and restricted states
- It works at narrow and wide viewport sizes.
- It is keyboard-operable with visible focus.
- It uses semantic HTML and accessible names.
- Typecheck, lint, relevant tests, and visual checks pass.
- Documentation is updated when the design-system source of truth changes.

## Commands

Inspect `package.json` before running commands. Use available scripts rather than gue

Common validation commands:
```

```bash
pnpm tokens:validate
pnpm tokens:build
pnpm lint
pnpm typecheck
pnpm test
pnpm test:visual
pnpm storybook
pnpm design-system:audit
pnpm design-system:report
```

Reference documents

@docs/design-principles.md
@docs/accessibility.md
@docs/component-inventory.md
@docs/figma-mapping.md

````
`CLAUDE.md` should contain lasting facts and guardrails. Keep procedural checks—such

## Audit SKILL.md

Create this directory:

```bash
mkdir -p .claude/skills/audit-semester-design-sync
```
````

Then save the following as `.claude/skills/audit-semester-design-sync/SKILL.md`.

```
---
name: audit-semester-design-sync
description: Audit a React screen, component library, or Figma implementation for Ser
when_to_use: Use for Figma-to-code reviews, design QA, React library audits, token ma
argument-hint: "[figma-url-or-node] [route-or-component-path]"
allowed-tools:
  - Read
  - Glob
  - Grep
  - Bash
---

# Audit Semester Design Sync

## Objective

Audit the alignment between:

1. The requested or supplied Figma design.
2. The React implementation.
3. Semester token definitions.
4. Semester component contracts.
```

```
5. Accessibility and responsive behavior requirements.

Find system drift before recommending changes. Do not modify source files unless the
explicitly asks for implementation after reviewing the audit.

## Required inputs

Read before auditing:

- Root `CLAUDE.md`.
- `docs/design-principles.md`.
- `docs/accessibility.md`.
- `docs/component-inventory.md`.
- `docs/figma-mapping.md`.
- Relevant files in `docs/patterns/`.
- Relevant source files, tests, and Storybook stories.
- `packages/tokens/src/semantic.ts`.
- `packages/tokens/src/components.ts`.
- `packages/ui/src/component-manifest.ts`, if present.

If a Figma reference is supplied:

1. Use Figma MCP to inspect the selected node, child hierarchy, components, componen
   properties, variables, layout data, text, and visible states.
2. Capture design intent, not merely dimensions or colors.
3. Identify which Figma variables and components should map to Semester tokens and Re
4. Do not treat a static desktop frame as a complete responsive or behavioral specif

## Audit process

### Step 1: Establish scope

State the audit target:

- Figma file, page, frame, node, or component.
- React route, feature, package, or component.
- Relevant Semester pattern.
- Existing component APIs and token namespaces.
- Whether the audit is visual-only, implementation-only, or Figma-to-code.

If the target cannot be determined, audit the available source and clearly label any

### Step 2: Inspect semantic-token discipline

Check for:

- Raw CSS colors, arbitrary design values, and raw palette imports.
- Incorrect semantic role selection.
- One-off CSS variables that duplicate Semester tokens.
- Hardcoded shadows, radii, z-index values, typography, motion values, and spacing.
- Figma variables that have no code token mapping.
- Code semantic tokens that have no corresponding Figma variable where parity is requ
- Themes that are incompatible with dark mode or high-contrast modes, if supported.

Classify each issue:
```

```
- `blocker`: breaks accessibility, causes a critical brand/system contradiction, or p
- `major`: creates repeatable drift, duplication, or an inconsistent user interactio
- `minor`: cosmetic or documentation-level mismatch that does not meaningfully break

### Step 3: Inspect component parity

For every visible Figma component or code component:

- Identify the closest React export from `@semester/ui`.
- Identify the closest Figma library component.
- Compare variants, sizes, tones, states, slots, and content rules.
- Detect feature-local duplicate components.
- Identify APIs that expose implementation detail instead of design intent.
- Verify whether the React component has Storybook coverage and a documented contrac

Do not recommend a new component until you confirm that:
- An existing component is insufficient.
- A documented composition is insufficient.
- The need is reusable across at least two product contexts or is foundational.

### Step 4: Inspect interaction and state coverage

For relevant surfaces, check:

- Default.
- Hover.
- Focus-visible.
- Active/pressed.
- Selected.
- Disabled.
- Read-only.
- Loading.
- Empty.
- Error and retry.
- Invalid validation.
- Success/confirmation.
- Permission-restricted state.
- Long-content and localization behavior.
- Network/offline/degraded behavior when relevant.

Treat a missing visible Figma state as incomplete design evidence, not permission to
state from accessible product code.

### Step 5: Inspect responsive and layout behavior

Check:

- Page gutters and max-widths.
- Layout primitives and composition.
- Stack/grid transformations.
- Overflow behavior.
- Action priority on narrow screens.
- Tables and data density.
- Content truncation and wrapping.
- Touch-target size.
- Fixed/sticky layer behavior.
```

```
- Z-index use and overlay layering.
- Container queries or breakpoint consistency.

### Step 6: Inspect accessibility

Check:

- Correct semantic landmarks and heading hierarchy.
- Native button, link, input, select, and form semantics.
- Accessible names for icon-only controls.
- Label, description, validation, and error associations.
- Focus order and visible focus styles.
- Dialog, menu, tooltip, popover, and combobox focus behavior.
- Keyboard interaction.
- Color contrast and non-color state signals.
- Motion reduction.
- Live-region use only where needed.

### Step 7: Produce report before fixes

Do not edit code during audit mode.

Use the exact report format below.

## Required output format

# Semester design-sync audit

## Executive assessment

Write one concise paragraph identifying:
- Overall alignment: strong, moderate, weak, or critical drift.
- The highest-risk mismatch.
- Whether the main issue is token, component, responsive, state, accessibility, or Fi

## Scope

| Area | Audited target | Evidence reviewed |
|---|---|---|
| Figma | Value or `not supplied` | Nodes, variables, components, states |
| Code | Value or `not supplied` | Files, stories, tests |
| Semester | Relevant packages/docs | Tokens, components, patterns |

## Findings

| Severity | Location | Category | Finding | Existing Semester solution | Recommended
|---|---|---|---|---|---|
| Blocker/Major/Minor | Figma node, file, component, selector, or token | Token/Compo

## Token mapping

| Figma collection/variable | Expected Semester token | Actual implementation | Statu
|---|---|---|---|---|
| Value | Value | Value | Match/Partial/Missing/Obsolete | Specific resolution |

## Component mapping
```

```
| Figma component | React component | Variant/state parity | Status | Action |
|---|---|---|---|---|
| Value | Value | Match/Partial/Missing | Match/Partial/Missing/Obsolete | Specific r

## State coverage

| Surface | Present | Missing or unclear | Required action |
|---|---|---|---|
| Value | List covered states | List missing states | Specific resolution |

## Accessibility review

| Requirement | Status | Evidence | Required action |
|---|---|---|---|
| Semantics | Pass/Partial/Fail | Specific element or component | Specific resolutio
| Keyboard | Pass/Partial/Fail | Specific interaction | Specific resolution |
| Focus | Pass/Partial/Fail | Specific state | Specific resolution |
| Labels/errors | Pass/Partial/Fail | Specific field/control | Specific resolution |
| Contrast/non-color cues | Pass/Partial/Fail | Specific token/state | Specific reso

## System gaps

| Candidate capability | Reusable need | Existing composition works | Recommendation
|---|---:|---:|---|
| Value | Yes/No | Yes/No | Compose, document pattern, propose token, or propose comp

## Recommended implementation order

1. List blockers first.
2. Then shared token/component issues.
3. Then route-level composition issues.
4. Then visual polish and documentation.

## Rules

- Prefer an existing Semester component over a Figma-specific clone.
- Prefer a semantic token over a visual-value token.
- Prefer a documented composition over a new component.
- Do not recommend raw values to chase pixel-perfect parity.
- Do not use Figma omissions to excuse missing accessibility or state behavior.
- Report evidence, not subjective aesthetic preference.
- Keep proposed fixes small, composable, and compatible with the design system.
```

Claude Code discovers project skills from `.claude/skills/<skill-name>/SKILL.md`. A `description` tells Claude when the skill applies; users can also invoke it directly as `/audit-semester-design-sync`.[31]

Semantic token schema

Use **reference**, **semantic**, and **component** layers. The source examples below use TypeScript, but the naming and alias rules apply equally to Style Dictionary, Tokens Studio JSON, CSS custom properties, or another generation pipeline.

Token policy

Layer	Example	May be consumed by	Why it exists
Reference	`ref.color.brand.600`	Token/theme definitions only	Raw palette, sizing, type, and motion scales
Semantic	`color.action.primary.default`	All components and product UI	Stable meaning independent of raw values
Component	`button.primary.background.default`	Owning component only	Internal component contract
Local	`--feature-x-offset`	Rarely and temporarily	Must not replicate a global semantic role

`packages/tokens/src/primitives.ts`

```
export const ref = {
  color: {
    neutral: {
      0: "#FFFFFF",
      25: "#FCFCFD",
      50: "#F8F9FA",
      100: "#F1F3F5",
      200: "#E9ECEF",
      300: "#DEE2E6",
      400: "#CED4DA",
      500: "#ADB5BD",
      600: "#6C757D",
      700: "#495057",
      800: "#343A40",
      900: "#212529",
      950: "#101214",
    },
    brand: {
      50: "#EEF4FF",
      100: "#D9E7FF",
      200: "#BBD7FF",
      300: "#8CC0FF",
      400: "#55A0FF",
      500: "#2F7CF6",
      600: "#1E63D8",
      700: "#194FAC",
      800: "#1B438C",
      900: "#1B396F",
    },
    red: {
      50: "#FEF2F2",
      100: "#FEE2E2",
      200: "#FECACA",
      500: "#EF4444",
      600: "#DC2626",
      700: "#B91C1C",
      900: "#7F1D1D",
    },
```

```
    amber: {
      50: "#FFFBEB",
      100: "#FEF3C7",
      500: "#F59E0B",
      600: "#D97706",
      700: "#B45309",
      900: "#78350F",
    },
    green: {
      50: "#F0FDF4",
      100: "#DCFCE7",
      500: "#22C55E",
      600: "#16A34A",
      700: "#15803D",
      900: "#14532D",
    },
  },

  space: {
    0: "0",
    px: "1px",
    0_5: "0.125rem",
    1: "0.25rem",
    1_5: "0.375rem",
    2: "0.5rem",
    3: "0.75rem",
    4: "1rem",
    5: "1.25rem",
    6: "1.5rem",
    8: "2rem",
    10: "2.5rem",
    12: "3rem",
    16: "4rem",
    20: "5rem",
    24: "6rem",
  },

  radius: {
    none: "0",
    xs: "0.125rem",
    sm: "0.25rem",
    md: "0.375rem",
    lg: "0.5rem",
    xl: "0.75rem",
    "2xl": "1rem",
    full: "9999px",
  },

  font: {
    family: {
      sans: "Inter, ui-sans-serif, system-ui, sans-serif",
      mono: '"JetBrains Mono", ui-monospace, SFMono-Regular, monospace',
    },
    size: {
      xs: "0.75rem",
      sm: "0.875rem",
```

```
      md: "1rem",
      lg: "1.125rem",
      xl: "1.25rem",
      "2xl": "1.5rem",
      "3xl": "1.875rem",
      "4xl": "2.25rem",
    },
    weight: {
      regular: 400,
      medium: 500,
      semibold: 600,
      bold: 700,
    },
    lineHeight: {
      tight: 1.15,
      snug: 1.3,
      normal: 1.5,
      relaxed: 1.65,
    },
  },

  duration: {
    instant: "0ms",
    fast: "120ms",
    normal: "180ms",
    slow: "280ms",
  },

  easing: {
    standard: "cubic-bezier(0.2, 0, 0, 1)",
    enter: "cubic-bezier(0, 0, 0.2, 1)",
    exit: "cubic-bezier(0.4, 0, 1, 1)",
  },

  zIndex: {
    base: 0,
    sticky: 100,
    dropdown: 200,
    popover: 300,
    overlay: 400,
    modal: 500,
    toast: 600,
    tooltip: 700,
  },
} as const;
```

packages/tokens/src/semantic.ts

```
export const semantic = {
  color: {
    text: {
      primary: "{ref.color.neutral.950}",
      secondary: "{ref.color.neutral.700}",
      muted: "{ref.color.neutral.600}",
      disabled: "{ref.color.neutral.500}",
```

```
    inverse: "{ref.color.neutral.0}",
    link: "{ref.color.brand.700}",
    linkHover: "{ref.color.brand.800}",
    danger: "{ref.color.red.700}",
    success: "{ref.color.green.700}",
    warning: "{ref.color.amber.700}",
  },

  icon: {
    primary: "{color.text.primary}",
    secondary: "{color.text.secondary}",
    muted: "{color.text.muted}",
    disabled: "{color.text.disabled}",
    inverse: "{color.text.inverse}",
    brand: "{color.action.primary.default}",
    danger: "{color.status.danger.strong}",
  },

  surface: {
    canvas: "{ref.color.neutral.25}",
    base: "{ref.color.neutral.0}",
    raised: "{ref.color.neutral.0}",
    sunken: "{ref.color.neutral.50}",
    overlay: "{ref.color.neutral.0}",
    inverse: "{ref.color.neutral.950}",
    selected: "{ref.color.brand.50}",
    disabled: "{ref.color.neutral.100}",
  },

  border: {
    subtle: "{ref.color.neutral.200}",
    default: "{ref.color.neutral.300}",
    strong: "{ref.color.neutral.400}",
    focus: "{ref.color.brand.500}",
    disabled: "{ref.color.neutral.200}",
    danger: "{ref.color.red.500}",
  },

  action: {
    primary: {
      default: "{ref.color.brand.600}",
      hover: "{ref.color.brand.700}",
      active: "{ref.color.brand.800}",
      foreground: "{ref.color.neutral.0}",
      disabled: "{ref.color.neutral.300}",
      disabledForeground: "{ref.color.neutral.600}",
    },

    secondary: {
      default: "{color.surface.base}",
      hover: "{ref.color.neutral.50}",
      active: "{ref.color.neutral.100}",
      foreground: "{color.text.primary}",
      border: "{color.border.default}",
    },
```

```
      subtle: {
        default: "transparent",
        hover: "{ref.color.neutral.100}",
        active: "{ref.color.neutral.200}",
        foreground: "{color.text.primary}",
      },

      danger: {
        default: "{ref.color.red.600}",
        hover: "{ref.color.red.700}",
        active: "{ref.color.red.900}",
        foreground: "{ref.color.neutral.0}",
      },
    },

    status: {
      info: {
        subtle: "{ref.color.brand.50}",
        default: "{ref.color.brand.600}",
        strong: "{ref.color.brand.700}",
        text: "{ref.color.brand.800}",
      },
      success: {
        subtle: "{ref.color.green.50}",
        default: "{ref.color.green.600}",
        strong: "{ref.color.green.700}",
        text: "{ref.color.green.900}",
      },
      warning: {
        subtle: "{ref.color.amber.50}",
        default: "{ref.color.amber.600}",
        strong: "{ref.color.amber.700}",
        text: "{ref.color.amber.900}",
      },
      danger: {
        subtle: "{ref.color.red.50}",
        default: "{ref.color.red.600}",
        strong: "{ref.color.red.700}",
        text: "{ref.color.red.900}",
      },
    },

    focus: {
      ring: "{ref.color.brand.500}",
      ringOffset: "{color.surface.base}",
    },
  },

  space: {
    inline: {
      xs: "{ref.space.1}",
      sm: "{ref.space.2}",
      md: "{ref.space.3}",
      lg: "{ref.space.4}",
    },
    stack: {
```

```
      xs: "{ref.space.2}",
      sm: "{ref.space.3}",
      md: "{ref.space.4}",
      lg: "{ref.space.6}",
      xl: "{ref.space.8}",
      "2xl": "{ref.space.12}",
    },
    inset: {
      xs: "{ref.space.2}",
      sm: "{ref.space.3}",
      md: "{ref.space.4}",
      lg: "{ref.space.6}",
    },
    page: {
      mobile: "{ref.space.4}",
      tablet: "{ref.space.6}",
      desktop: "{ref.space.8}",
    },
  },

  typography: {
    body: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.sm}",
        fontWeight: "{ref.font.weight.regular}",
        lineHeight: "{ref.font.lineHeight.normal}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.md}",
        fontWeight: "{ref.font.weight.regular}",
        lineHeight: "{ref.font.lineHeight.normal}",
      },
    },

    label: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.sm}",
        fontWeight: "{ref.font.weight.medium}",
        lineHeight: "{ref.font.lineHeight.snug}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.md}",
        fontWeight: "{ref.font.weight.medium}",
        lineHeight: "{ref.font.lineHeight.snug}",
      },
    },

    heading: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.lg}",
        fontWeight: "{ref.font.weight.semibold}",
```

```
        lineHeight: "{ref.font.lineHeight.tight}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.xl}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
      },
      lg: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.2xl}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
      },
    },
  },

  size: {
    control: {
      sm: "2rem",
      md: "2.5rem",
      lg: "3rem",
    },
    icon: {
      xs: "0.875rem",
      sm: "1rem",
      md: "1.25rem",
      lg: "1.5rem",
    },
    target: {
      minimum: "2.75rem",
    },
    content: {
      readable: "42rem",
      default: "72rem",
      wide: "90rem",
    },
  },

  radius: {
    control: "{ref.radius.md}",
    surface: "{ref.radius.lg}",
    overlay: "{ref.radius.xl}",
    pill: "{ref.radius.full}",
  },

  border: {
    default: "1px",
    strong: "2px",
  },

  shadow: {
    raised: "0 1px 2px rgb(16 18 20 / 0.06), 0 1px 3px rgb(16 18 20 / 0.10)",
    overlay: "0 12px 32px rgb(16 18 20 / 0.16), 0 2px 8px rgb(16 18 20 / 0.08)",
  },
```

```
  motion: {
    duration: {
      instant: "{ref.duration.instant}",
      fast: "{ref.duration.fast}",
      normal: "{ref.duration.normal}",
      slow: "{ref.duration.slow}",
    },
    easing: {
      standard: "{ref.easing.standard}",
      enter: "{ref.easing.enter}",
      exit: "{ref.easing.exit}",
    },
  },

  zIndex: {
    base: "{ref.zIndex.base}",
    sticky: "{ref.zIndex.sticky}",
    dropdown: "{ref.zIndex.dropdown}",
    popover: "{ref.zIndex.popover}",
    overlay: "{ref.zIndex.overlay}",
    modal: "{ref.zIndex.modal}",
    toast: "{ref.zIndex.toast}",
    tooltip: "{ref.zIndex.tooltip}",
  },
} as const;
```

packages/tokens/src/components.ts

```
export const component = {
  button: {
    primary: {
      background: {
        default: "{color.action.primary.default}",
        hover: "{color.action.primary.hover}",
        active: "{color.action.primary.active}",
        disabled: "{color.action.primary.disabled}",
      },
      foreground: {
        default: "{color.action.primary.foreground}",
        disabled: "{color.action.primary.disabledForeground}",
      },
      focusRing: "{color.focus.ring}",
      height: {
        sm: "{size.control.sm}",
        md: "{size.control.md}",
        lg: "{size.control.lg}",
      },
      radius: "{radius.control}",
      gap: "{space.inline.sm}",
    },
  },

  field: {
    background: {
      default: "{color.surface.base}",
```

```
      disabled: "{color.surface.disabled}",
    },
    border: {
      default: "{color.border.default}",
      hover: "{color.border.strong}",
      focus: "{color.border.focus}",
      invalid: "{color.border.danger}",
      disabled: "{color.border.disabled}",
    },
    text: {
      value: "{color.text.primary}",
      placeholder: "{color.text.muted}",
      disabled: "{color.text.disabled}",
    },
    height: {
      sm: "{size.control.sm}",
      md: "{size.control.md}",
      lg: "{size.control.lg}",
    },
    radius: "{radius.control}",
  },

  dialog: {
    background: "{color.surface.overlay}",
    border: "{color.border.subtle}",
    radius: "{radius.overlay}",
    shadow: "{shadow.overlay}",
    overlay: "rgb(16 18 20 / 0.48)",
  },
} as const;
```

Figma MCP configuration

Figma offers remote and desktop MCP modes. Figma recommends the remote MCP server for most users because it has the broadest feature set. Its official Claude Code plugin configures the server and includes common workflow skills. [32]

Preferred: install Figma's Claude Code plugin

Run this in your terminal:

```
claude plugin install figma@claude-plugins-official
```

Then:

1. Restart Claude Code.
2. Run `/plugin`.
3. Open the **Installed** tab.
4. Select `figma`.
5. Start authorization and approve access in the browser.

6. Return to Claude Code.
7. Run `/mcp` to confirm that Figma is connected.

Figma MCP can provide structured design context from Figma files—such as components, variables, layout data, FigJam content, and Code Connect mappings—so Claude can map a selected design to your system instead of inferring it from screenshots. [32]

Manual remote configuration

Claude Code supports remote HTTP MCP servers with:

```
claude mcp add --transport http <name> <url>
```

For a shared team configuration, use `--scope project`, which writes project configuration; local scope is the default, while user scope works across your projects. [33]

For Figma's remote MCP endpoint:

```
claude mcp add --scope project --transport http figma https://mcp.figma.com/mcp
```

Restart Claude Code, then authenticate with `/mcp` if requested:

```
/mcp
```

Verify from your shell:

```
claude mcp list
claude mcp get figma
```

The status should move from authentication-required to connected after authorization. Claude Code's MCP status panel and CLI can show connected, needs-authentication, pending approval, and failed-connection states. [33]

`.mcp.json` project configuration

If you want the remote server declared in the repository, create `.mcp.json` at the project root:

```
{
  "mcpServers": {
    "figma": {
      "type": "http",
      "url": "https://mcp.figma.com/mcp"
    }
  }
}
```

The `type` field is required for URL-based JSON configuration. Claude Code treats a server without a declared `type` as a local stdio server, so an entry containing a `url` but no `type: "http"` is invalid.[33]

When teammates clone the repository:

1. They run `claude` from the project.
2. They accept the workspace trust prompt.
3. They review and approve the project MCP configuration.
4. They authenticate their own Figma account via `/mcp`.

Do not place static Figma access tokens, OAuth tokens, or secrets in `.mcp.json`. Use the OAuth authentication flow that Claude Code invokes from `/mcp`.

Desktop Figma MCP server

Use desktop MCP when your organization specifically requires it, or when you need the local Desktop workflow. Figma otherwise recommends its remote server.[32]

1. Install and open the Figma desktop app.
2. Open a Figma Design file.
3. Enter Dev Mode with no canvas layer selected.
4. Enable the MCP server in the right sidebar.
5. Copy the server URL offered by Figma.
6. Configure Claude Code:

```
claude mcp add --scope local --transport http figma-desktop http://127.0.0.1:3845/mcp
```

7. Restart Claude Code.
8. Run `/mcp` and confirm that `figma-desktop` appears.

Figma's documented local endpoint is `http://127.0.0.1:3845/mcp`.[32]

Recommended `.claude/settings.json`

This does not force secrets or automatic trust. It simply records repository-wide behavior.

```
{
  "permissions": {
    "allow": [
      "Read",
      "Glob",
      "Grep"
    ]
  }
}
```

Keep automated Figma writes disabled by default. Reading Figma variables and inspecting nodes is low-risk; changing an approved design file is a collaborative, state-changing operation that should require an explicit task and review.

Figma-to-code mapping

Create `docs/figma-mapping.md`:

```
# Semester Figma-to-Code Mapping

## Ownership

- Code owns implementation behavior, accessibility, responsive behavior, and public A
- Figma owns approved visual design intent and exploration.
- `packages/tokens/src/` owns production design-token values.
- Figma variables should represent exported/synchronized semantic token contracts.
- Figma component variants must map to documented React component props and states.

## Variable mapping

| Figma collection | Figma variable | Semester token | Status | Notes |
|---|---|---|---|---|
| Semester / Semantic / Color | Text / Primary | `color.text.primary` | Stable | Prir
| Semester / Semantic / Color | Surface / Base | `color.surface.base` | Stable | Defa
| Semester / Semantic / Color | Action / Primary / Default | `color.action.primary.de
| Semester / Semantic / Color | Border / Focus | `color.border.focus` | Stable | Focu
| Semester / Semantic / Space | Stack / MD | `space.stack.md` | Stable | Default ver
| Semester / Component / Button | Primary / Hover | `button.primary.background.hover

## Component mapping

| Figma component | React export | Figma variants → React props | Status | Notes |
|---|---|---|---|---|
| Button | `@semester/ui/Button` | `Style` → `variant`; `Size` → `size`; `Loading` →
| Text field | `@semester/ui/TextField` | `Size` → `size`; `State` → `isInvalid`, `is
| Dialog | `@semester/ui/Dialog` | `Size` → `size` | Stable | Code owns focus trap ar
| Data table | `@semester/ui/DataTable` | `Density` → `density` | Beta | Do not use
| Metric card | No direct export | N/A | Deprecated | Compose `Surface`, `Stack`, `Te

## Mapping rules

- Figma variables must map to semantic code tokens before production use.
- Figma local styles must not become code tokens without design-system review.
- Figma components are not assumed to be React components.
- Figma variants map only to documented React props.
- Missing implementation states in Figma do not remove code accessibility requiremen
- New variables or components require a documented code mapping and system review.
- Record unresolved mismatches as an ADR or tracked issue.
```

Automated token mapping and drift detection

The practical objective is a CI report that tells you whether Figma and code agree on the system contract.

Canonical mapping manifest

Create `packages/tokens/src/figma-map.ts`:

```
export type FigmaTokenMapping = {
  figmaCollection: string;
  figmaVariable: string;
  semesterToken: string;
  required: boolean;
  componentScope?: string;
};

export const figmaTokenMap: FigmaTokenMapping[] = [
  {
    figmaCollection: "Semester / Semantic / Color",
    figmaVariable: "Text / Primary",
    semesterToken: "color.text.primary",
    required: true,
  },
  {
    figmaCollection: "Semester / Semantic / Color",
    figmaVariable: "Surface / Base",
    semesterToken: "color.surface.base",
    required: true,
  },
  {
    figmaCollection: "Semester / Semantic / Color",
    figmaVariable: "Action / Primary / Default",
    semesterToken: "color.action.primary.default",
    required: true,
  },
  {
    figmaCollection: "Semester / Semantic / Space",
    figmaVariable: "Stack / MD",
    semesterToken: "space.stack.md",
    required: true,
  },
  {
    figmaCollection: "Semester / Component / Button",
    figmaVariable: "Primary / Background / Hover",
    semesterToken: "button.primary.background.hover",
    required: false,
    componentScope: "Button",
  },
];
```

Flattened-token utility

Create `scripts/flatten-tokens.ts`:

```
export type FlatTokenMap = Record<string, unknown>;

export function flattenTokens(
  value: Record<string, unknown>,
  prefix = "",
): FlatTokenMap {
  return Object.entries(value).reduce<FlatTokenMap>((result, [key, child]) => {
    const path = prefix ? `${prefix}.${key}` : key;

    if (
      child !== null &&
      typeof child === "object" &&
      !Array.isArray(child)
    ) {
      Object.assign(
        result,
        flattenTokens(child as Record<string, unknown>, path),
      );
      return result;
    }

    result[path] = child;
    return result;
  }, {});
}
```

Token-map report generator

Create `scripts/build-figma-token-map.ts`:

```
import { mkdir, writeFile } from "node:fs/promises";
import { component } from "../packages/tokens/src/components";
import { semantic } from "../packages/tokens/src/semantic";
import { figmaTokenMap } from "../packages/tokens/src/figma-map";
import { flattenTokens } from "./flatten-tokens";

type ReportStatus = "match" | "missing-code-token";

const semanticTokens = flattenTokens(semantic);
const componentTokens = flattenTokens(component);
const allTokens = {
  ...semanticTokens,
  ...componentTokens,
};

const rows = figmaTokenMap.map((mapping) => {
  const exists = Object.hasOwn(allTokens, mapping.semesterToken);

  return {
    ...mapping,
```

```
    status: (exists ? "match" : "missing-code-token") as ReportStatus,
    resolvedValue: exists ? allTokens[mapping.semesterToken] : null,
  };
});

const summary = {
  generatedAt: new Date().toISOString(),
  total: rows.length,
  matched: rows.filter((row) => row.status === "match").length,
  missingCodeTokens: rows.filter((row) => row.status === "missing-code-token")
    .length,
};

await mkdir("reports/design-system", { recursive: true });

await writeFile(
  "reports/design-system/figma-token-map.json",
  JSON.stringify({ summary, rows }, null, 2),
);

const markdown = [
  "# Semester Figma Token Map",
  "",
  `Generated: ${summary.generatedAt}`,
  "",
  `- Total mappings: ${summary.total}`,
  `- Matched: ${summary.matched}`,
  `- Missing code tokens: ${summary.missingCodeTokens}`,
  "",
  "| Figma collection | Figma variable | Semester token | Status | Resolved value |"
  "|---|---|---|---|---|",
  ...rows.map((row) => {
    const value = row.resolvedValue ?? "—";
    return `| ${row.figmaCollection} | ${row.figmaVariable} | \`${row.semesterToken}`
  }),
  "",
].join("\n");

await writeFile(
  "reports/design-system/figma-token-map.md",
  markdown,
);

if (summary.missingCodeTokens > 0) {
  process.exitCode = 1;
}
```

Raw-value and token-policy scanner

Create `scripts/audit-token-usage.ts`:

```
import { glob } from "glob";
import { readFile } from "node:fs/promises";

const files = await glob([
```

```
  "apps/**/*.{ts,tsx,js,jsx,css,scss}",
  "packages/ui/**/*.{ts,tsx,js,jsx,css,scss}",
  "!**/dist/**",
  "!**/node_modules/**",
  "!**/*.test.*",
  "!**/*.stories.*",
]);

const violations: Array<{
  file: string;
  line: number;
  rule: string;
  match: string;
}> = [];

const rules: Array<{ rule: string; expression: RegExp }> = [
  {
    rule: "raw-color",
    expression:
      /#[0-9a-fA-F]{3,8}\b|\brgba?\(|\bhsla?\(|\boklch?\(/g,
  },
  {
    rule: "arbitrary-tailwind-design-value",
    expression:
      /\b(?:bg|text|border|ring|shadow|p|px|py|pt|pr|pb|pl|m|mx|my|mt|mr|mb|ml|gap|z
  },
  {
    rule: "arbitrary-z-index",
    expression: /\bzIndex\s*:\s*\d{2,}/g,
  },
  {
    rule: "raw-shadow",
    expression: /\bboxShadow\s*:\s*["'`]/g,
  },
];

for (const file of files) {
  const content = await readFile(file, "utf8");
  const lines = content.split("\n");

  for (const [index, line] of lines.entries()) {
    for (const { rule, expression } of rules) {
      for (const match of line.matchAll(expression)) {
        violations.push({
          file,
          line: index + 1,
          rule,
          match: match[^3_0],
        });
      }
    }
  }
}

if (violations.length === 0) {
  console.log("No token-policy violations found.");
```

```
  process.exit(0);
}

console.table(violations);
process.exit(1);
```

Tune exclusions so that raw values remain allowed in `packages/tokens`, snapshots, fixtures, generated output, and intentionally isolated visual-test files.

CI report script

Add scripts like these to root `package.json`:

```
{
  "scripts": {
    "tokens:validate": "tsx packages/tokens/src/validate.ts",
    "tokens:build": "tsx scripts/build-tokens.ts",
    "design-system:map": "tsx scripts/build-figma-token-map.ts",
    "design-system:audit": "tsx scripts/audit-token-usage.ts",
    "design-system:report": "pnpm tokens:validate && pnpm design-system:map && pnpm d
  }
}
```

GitHub Actions workflow

Create `.github/workflows/design-system.yml`:

```
name: Design system CI

on:
  pull_request:
    paths:
      - "apps/**"
      - "packages/tokens/**"
      - "packages/ui/**"
      - "scripts/**"
      - "docs/figma-mapping.md"
      - ".claude/**"
      - ".github/workflows/design-system.yml"
  push:
    branches:
      - main

jobs:
  validate-design-system:
    name: Validate tokens and design-system drift
    runs-on: ubuntu-latest

    steps:
      - name: Check out repository
        uses: actions/checkout@v4
```

```
      - name: Set up pnpm
        uses: pnpm/action-setup@v4
        with:
          version: 9

      - name: Set up Node
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: pnpm

      - name: Install dependencies
        run: pnpm install --frozen-lockfile

      - name: Validate token aliases
        run: pnpm tokens:validate

      - name: Build token artifacts
        run: pnpm tokens:build

      - name: Build Figma-to-code token map
        run: pnpm design-system:map

      - name: Detect raw design values
        run: pnpm design-system:audit

      - name: Typecheck
        run: pnpm typecheck

      - name: Run component tests
        run: pnpm test

      - name: Run visual regression tests
        run: pnpm test:visual

      - name: Upload design-system report
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: design-system-report
          path: reports/design-system/
          if-no-files-found: warn
```

CI output contract

Have CI produce a report with this structure:

```
# Semester Design System Report

## Summary

- Token alias validation: pass
- Figma-to-code mapped variables: 84 / 88
- Missing semantic code tokens: 2
- Obsolete mappings: 1
```

```
- Raw-value violations: 3
- Unmapped Figma components: 2
- Visual-regression status: pass
- Accessibility-test status: partial

## Blockers

| Type | Location | Issue | Resolution |
|---|---|---|---|
| Raw color | `apps/web/src/billing/PlanCard.tsx:42` | `#1E63D8` bypasses semantic to
| Mapping | `Figma: Action / Positive / Default` | No matching semantic token | Propo

## Major drift

| Type | Figma | Code | Recommended action |
|---|---|---|---|
| Component | `Data table / Compact` | No supported React density mode | Add `density
| State | Error state present in Figma | No retry action in React | Add inline retry

## Token parity

| Figma variable | Semester token | Status |
|---|---|---|
| `Text / Primary` | `color.text.primary` | Match |
| `Action / Primary / Default` | `color.action.primary.default` | Match |
| `Action / Positive / Default` | — | Missing |
```

Figma variable audit workflow

For actual Figma-value inspection, use Claude Code with Figma MCP after the server is connected:

```
/audit-semester-design-sync <Figma frame URL> apps/web/src/routes/settings.tsx
```

Or prompt Claude Code directly:

```
Use the audit-semester-design-sync skill.

Inspect this Figma frame and compare it with the React implementation:
[Figma URL]

Audit:
- Figma variables against packages/tokens/src/semantic.ts.
- Figma components against @semester/ui exports.
- React source for raw colors and raw layout values.
- Figma and code state parity.
- Responsive and accessibility gaps.

Do not edit files. Produce the required design-sync audit report.
```

```
The remote HTTP MCP route is Claude Code's recommended approach for cloud MCP servers
```

Rollout sequence

1. Add the root `CLAUDE.md` and commit it.
2. Create the `audit-semester-design-sync` skill and invoke it against one existing production route.
3. Install Figma MCP using the official plugin; use the remote server by default. [32]
4. Establish reference, semantic, and component token layers.
5. Create and maintain `docs/figma-mapping.md`.
6. Add the Figma-token mapping manifest.
7. Add raw-value detection and token-alias validation to CI.
8. Audit your ten most-used components and three most important product screens.
9. Convert repeated findings into one of four outcomes: token, component, documented pattern, or lint rule.
10. Require every new Figma library variable/component to have a corresponding code mapping before it reaches production.

This gives Claude Code a constrained, inspectable contract: it can read Figma for intent, map intent to Semester primitives, detect drift automatically, and refuse to solve every UI task with another hand-styled generic component.

⁂

provide comand to claud code to excute on this and also anlyse current semester resporty and specfic to it

Your current Semester repo is already much further along than a generic “add a design system” setup. It has a dedicated token source/export path, extensive style-contract tests, an existing root `CLAUDE.md`, vendored Claude skills, and a large Vite/React application under `app/`. The best next move is **not** to replace that architecture—it is to add a Semester-specific Claude Code audit skill and tighten the existing token-export → Figma-variable → CI loop. [github_mcp_direct]

Run this in Claude Code

From the repository root, launch Claude Code:

```
cd /path/to/semester
claude
```

Then paste this command/prompt exactly into Claude Code:

```
/audit-semester-design-sync app/src app/design-tokens/semester.tokens.json app/src/s

You are auditing the existing Semester repository, not designing a generic new syste
```

```
First read:
- CLAUDE.md
- app/README.md
- DESIGN-SYSTEM-GUIDE.md
- docs/DESIGN-TOKENS.md
- docs/design/DESIGN-SYSTEM-PRODUCT-SPEC.md
- app/package.json
- app/src/styles/tokens.css
- app/src/lib/tokenexport.ts
- app/src/lib/tokenexport.test.ts
- app/design-tokens/semester.tokens.json
- app/src/styles/*.test.ts
- .claude/skills/

Then inspect:
- app/src/components/
- app/src/screens/
- app/src/styles/
- app/src/App.tsx
- app/src/main.tsx
- app/src/screens.tsx

Do not modify files yet.

Create a repository-specific Semester design-system audit with:
1. Existing architecture and source-of-truth map.
2. Token pipeline map: CSS semantic tokens → tokenexport.ts → semester.tokens.json →
3. Current design-system strengths that must be preserved.
4. Drift risks: raw styles, direct hex usage, duplicated component styles, hard-codec
5. Existing tests/scripts that already enforce design contracts.
6. Exact missing files needed for Claude Code + Figma MCP workflow only.
7. A minimal phased implementation plan that avoids restructuring the current app.
8. A table of proposed changes with priority, exact path, rationale, and acceptance t

Use evidence from actual files. Do not assume pnpm, monorepos, Storybook, Tailwind, c
```

That audit prompt deliberately tells Claude not to invent a `packages/ui`, pnpm workspace, Tailwind setup, or a replacement token schema—the patterns in earlier generic templates would be wrong for this particular repository.

One-command bootstrap prompt

Once the audit completes and you agree with its plan, paste the following command to have Claude Code implement only the smallest compatible toolkit:

```
Implement the approved Semester Claude Code design-system toolkit in this repository

Constraints:
- Preserve the current Vite + React architecture under app/.
- Preserve app/src/styles/tokens.css as the semantic token authority.
- Preserve app/src/lib/tokenexport.ts as the token export authority.
- Preserve app/design-tokens/semester.tokens.json as generated output; never hand-edi
- Preserve and extend existing tokenexport, style, accessibility, responsive, and cor
- Do not add Tailwind, Storybook, a monorepo, or a new component library package.
```

```
- Do not move existing components or rewrite App.tsx unless absolutely necessary.
- Do not add dependencies unless an existing repository dependency cannot support the
- Do not commit, push, open a PR, or change deployment configuration.

Create only:
1. .claude/skills/audit-semester-design-sync/SKILL.md
2. .claude/skills/build-semester-ui/SKILL.md
3. .mcp.json using the Figma remote MCP server, with no credentials committed
4. docs/FIGMA-MAPPING.md
5. app/scripts/design-system-audit.mjs or .ts, compatible with the app's existing too
6. app/scripts/design-system-report.mjs or .ts
7. Minimal package.json scripts in app/package.json:
   - design-system:audit
   - design-system:report
   - design-system:check
8. Focused tests that prove:
   - semester.tokens.json is generated and synchronized with code
   - raw values are not introduced outside approved CSS/token files
   - Figma mapping entries resolve to exported token paths
   - audit/report scripts fail on real violations and pass on valid fixtures

Before editing, show the file-by-file plan.
After editing, run the repository's actual relevant validation commands.
Report changed files, exact checks run, results, known limitations, and any Figma ste
```

Current Semester findings

Architecture already present

The active application is a Vite/TypeScript React app located in `app/`, not a package-per-component design-system monorepo. It has a large app entry point, screen hierarchy, component directory, styling directory, API/server code, and build configuration. [github_mcp_direct]

Current asset	What it means for your toolkit
`app/src/styles/tokens.css`	This should remain the **semantic-token authority**
`app/src/lib/tokenexport.ts`	This should remain the **token-export authority**
`app/design-tokens/semester.tokens.json`	This is the machine-readable/exported artifact; do not hand-edit it
`app/src/lib/tokenexport.test.ts`	Already validates that token export does not drift from code
`app/src/styles/*.test.ts`	You already have a substantial contract-test surface for CSS design rules
`app/src/components/` and `app/src/screens/`	These are the correct places to audit real UI composition and component duplication
`.claude/skills/`	Claude Code skill infrastructure already exists, so add one focused skill rather than bootstrapping from zero

Current asset	What it means for your toolkit
`CLAUDE.md`	Existing durable project instructions should be amended, not replaced blindly
`DESIGN-SYSTEM-GUIDE.md` and `docs/DESIGN-TOKENS.md`	Existing written design-system documentation that new Claude rules should reference

The repository's token documentation explicitly identifies `app/design-tokens/semester.tokens.json` as a generated, machine-readable export; `tokenexport.test.ts` verifies that it remains synchronized with `tokens.css`. That is a stronger foundation than a fresh generic JSON schema because it already supports downstream Figma/native/parity use cases. [github_mcp_direct]

Important constraints

Your project has a **large existing CSS system**: `app.css` is approximately 322 KB, `features.css` about 29 KB, `unity.css` about 34 KB, and `tokens.css` roughly 13 KB. It also has targeted tests for breakpoints, density, motion, fields, glass effects, gutters, hex handling, stacking, taps, typography scale, and token behavior. Treat this as a mature contract system, not a greenfield UI package. [github_mcp_direct]

So Claude Code should:

- Reuse the current CSS token vocabulary before introducing any new token name.
- Extend current source-based token export rather than create duplicate TypeScript token files.
- Use the existing style tests as enforcement hooks instead of adding broad, untested rules in a new lint stack.
- Keep the generated token JSON as the Figma/native interchange artifact.
- Add a Figma mapping manifest and a diff report around the existing export, rather than trying to make Figma the production source of truth.

Existing Figma readiness

The repository contains Figma-aware documentation and token-export comments, but the code search did **not** reveal a configured Figma MCP server or a maintained Figma-to-code component/variable mapping file. The export code and tests already describe `semester.tokens.json` as the artifact intended for Figma variables, native clients, and parity checking. [github_mcp_direct]

That means your gap is integration/governance—not token invention.

Add this audit skill

Create this folder and file:

```
mkdir -p .claude/skills/audit-semester-design-sync
```

```
.claude/skills/audit-semester-design-sync/SKILL.md
```

Paste this exact content:

```
---
name: audit-semester-design-sync
description: Audit the Semester React application against its existing CSS token sys
argument-hint: "[optional-figma-url-or-node] [optional-app-path]"
---

# Audit Semester Design Sync

## Semester-specific source of truth

Semester is an existing Vite and React application. Do not impose a generic monorepo
Tailwind, Storybook, or package-based design-system architecture unless it already ex

The source-of-truth order is:

1. `app/src/styles/tokens.css`
   - Semantic CSS token authority used by application UI.

2. `app/src/lib/tokenexport.ts`
   - Exports the application's semantic token system for downstream consumers.

3. `app/design-tokens/semester.tokens.json`
   - Generated machine-readable token artifact.
   - Never edit manually.
   - Regenerate through the repository's actual token-export command.

4. Existing CSS/component implementation
   - `app/src/styles/`
   - `app/src/components/`
   - `app/src/screens/`

5. Existing contract tests
   - `app/src/styles/*.test.ts`
   - `app/src/lib/tokenexport.test.ts`
   - Related architecture, responsive, accessibility, and screen tests.

6. Figma
   - Figma is evidence of approved visual intent.
   - Code owns production behavior, accessibility, responsive implementation, and tol
   - Figma variables/components must map to existing exported token paths and React/C

## Required reading
```

```
Before auditing, read:

- `CLAUDE.md`
- `app/README.md`
- `DESIGN-SYSTEM-GUIDE.md`
- `docs/DESIGN-TOKENS.md`
- `docs/design/DESIGN-SYSTEM-PRODUCT-SPEC.md`
- `app/package.json`
- `app/src/styles/tokens.css`
- `app/src/lib/tokenexport.ts`
- `app/src/lib/tokenexport.test.ts`
- `app/design-tokens/semester.tokens.json`
- `app/src/styles/*.test.ts`
- `docs/FIGMA-MAPPING.md`, if it exists

For a UI-specific audit, inspect the target implementation plus nearest related scree
components, and styles.

## Figma workflow

If a Figma URL, selection, node, component, or variable is supplied:

1. Use configured Figma MCP tools to inspect:
   - Node hierarchy.
   - Figma variables and collections.
   - Component instances and properties.
   - Auto-layout, constraints, and visible states.
   - Text/content hierarchy.

2. Compare Figma variable names to:
   - `app/design-tokens/semester.tokens.json`
   - `docs/FIGMA-MAPPING.md`
   - Semantic names in `app/src/styles/tokens.css`

3. Compare Figma components to current React/CSS components.
4. Do not create raw CSS to obtain pixel-perfect parity.
5. Do not treat an incomplete Figma frame as permission to omit:
   - Keyboard support.
   - Focus visibility.
   - Empty/loading/error states.
   - Responsive behavior.
   - Accessible labels and semantic HTML.

## Audit checks

### Token integrity

Identify:

- Direct hex, rgb, hsl, oklch, arbitrary shadows, arbitrary spacing, arbitrary radii
  arbitrary z-indexes, arbitrary durations, or arbitrary easing values.
- Whether each raw value is permitted in a foundational source file or is an app-leve
- CSS variables used but absent from `tokens.css`.
- CSS tokens present but not represented in the generated export.
- Generated token paths that cannot be traced back to semantic CSS tokens.
- Figma variables that lack an exported token mapping.
```

```
- Exported semantic tokens that lack an approved Figma mapping, if parity is required

### Component and pattern integrity

Identify:

- Feature-local components duplicating existing component or CSS patterns.
- Hard-coded styling that bypasses existing classes or semantic variables.
- Missing or inconsistent states.
- Misused controls: buttons vs links, clickable divs, unlabeled icon controls.
- Inconsistent density, hierarchy, type scale, surface treatment, borders, and page g
- Component APIs or CSS selectors that leak implementation detail.

### Responsive integrity

Check:

- Current breakpoint/test contracts.
- Narrow, intermediate, and wide behavior.
- Overflow, scroll, truncation, long copy, localization, and touch targets.
- Fixed/sticky element and stacking behavior.
- Data/table fallback behavior on narrow layouts.

### Accessibility integrity

Check:

- Semantic landmarks and headings.
- Label, description, error, and input associations.
- Keyboard interaction and visible focus.
- Dialog/menu/popover focus behavior.
- Color contrast and non-color status communication.
- Reduced-motion behavior.
- Target sizing.

## Severity scale

- Blocker: breaks critical usability, accessibility, core system integrity, or releas
- Major: repeatable drift, broken component/pattern contract, significant responsive,
- Minor: localized inconsistency, documentation gap, or low-risk visual mismatch.

## No-edit audit mode

Do not modify source code during an audit unless the user explicitly asks after revie
the report. You may run read-only inspection commands and existing test/audit commanc

## Required report format

# Semester design-system audit

## Executive assessment

State:
- Overall alignment: strong, moderate, weak, or critical drift.
- Highest-risk finding.
- Whether the main concern is tokens, components, responsive behavior, accessibility
```

```
  Figma parity, or test coverage.

## Repository model

| Concern | Current Semester authority | Evidence |
|---|---|---|
| Semantic tokens | `app/src/styles/tokens.css` | Specific section/file |
| Token export | `app/src/lib/tokenexport.ts` | Specific function/file |
| Machine-readable artifact | `app/design-tokens/semester.tokens.json` | Specific fi
| UI implementation | Components/screens/styles | Specific paths |
| Enforcement | Existing tests/scripts | Exact script or test paths |
| Figma parity | Mapping file or missing | Evidence |

## Findings

| Severity | Location | Category | Evidence | Existing Semester solution | Smallest s
|---|---|---|---|---|---|
| Blocker/Major/Minor | Exact path, selector, token, or Figma node | Token/Component/

## Token parity

| Figma variable or code token | Semester exported path | CSS semantic source | Statu
|---|---|---|---|---|
| Value | Value | Value | Match/Partial/Missing/Obsolete | Specific resolution |

## Component parity

| Figma or screen component | Current React/CSS implementation | State/variant parity
|---|---|---|---|---|
| Value | Value | Match/Partial/Missing | Match/Partial/Missing/Obsolete | Specific r

## Existing enforcement

| Contract | Existing test/script | Coverage | Gap |
|---|---|---|---|
| Token export | Exact test/script | What it proves | What remains |
| Responsive | Exact test/script | What it proves | What remains |
| Accessibility | Exact test/script | What it proves | What remains |
| Raw values | Exact test/script | What it proves | What remains |

## Minimal implementation plan

| Priority | Exact path | Change | Why | Acceptance test |
|---|---|---|---|---|
| P0/P1/P2 | Exact repo path | Smallest concrete change | System value | Exact commar

## Rules for recommendations

- Preserve Semester's existing token and export architecture.
- Recommend extensions, not parallel sources of truth.
- Use current tests before introducing tooling.
- Do not recommend Tailwind, Storybook, a monorepo, or package migrations unless
  directly supported by evidence and explicitly requested.
- Do not create a component for a one-off visual difference.
- Never recommend direct raw values to match Figma pixels.
- Prefer semantic CSS variables, existing classes, documented CSS patterns, and curre
```

Figma MCP setup

For Semester, configure the **remote** Figma MCP endpoint at the repository level, but do not commit credentials.

Run from the repository root:

```
claude mcp add --scope project --transport http figma https://mcp.figma.com/mcp
```

Then launch Claude Code:

```
claude
```

In Claude Code:

```
/mcp
```

```
Authorize Figma through the browser prompt, then verify Figma is connected. Claude Co
```

If you prefer a committed project declaration, add a root `.mcp.json`:

```
{
  "mcpServers": {
    "figma": {
      "type": "http",
      "url": "https://mcp.figma.com/mcp"
    }
  }
}
```

The `type: "http"` field matters: Claude Code needs it to interpret the URL as a remote HTTP MCP server instead of an attempted local stdio server. [34]

Figma recommends its official Claude Code plugin as the lowest-friction approach, because the plugin includes the MCP configuration and Figma-related workflow skills:

```
claude plugin install figma@claude-plugins-official
```

Use **one** setup route initially:

- Prefer the official plugin if you want Figma's maintained defaults and bundled workflows.
- Prefer the checked-in `.mcp.json` if you need a transparent, reviewable repository configuration.
- Avoid configuring duplicate servers under multiple names until you have a real reason; duplicated MCP tools make Claude's tool selection less reliable. Figma MCP gives Claude

structured access to variables, components, layout information, and other design context used for implementation and audit work. [35]

What to add—not replace

Add now	Why
`.claude/skills/audit-semester-design-sync/SKILL.md`	Gives Claude a repeatable repo-specific design review workflow
`.claude/skills/build-semester-ui/SKILL.md`	Ensures future UI work follows existing tokens, CSS, components, and test contracts
`docs/FIGMA-MAPPING.md`	Makes token/component parity explicit and reviewable
`.mcp.json` or the Figma plugin	Gives Claude structured Figma context
`app/scripts/design-system-audit.*`	Detects prohibited raw styling and missing token paths
`app/scripts/design-system-report.*`	Produces a CI-readable design-system drift summary
Package scripts in `app/package.json`	Makes the audit reproducible locally and in CI
Do not add yet	Why
---	---
A second token source in `packages/tokens/`	Semester already has `tokens.css` + `tokenexport.ts` + generated JSON
A generic `@semester/ui` package	No evidence that your current app uses package-based UI distribution
Tailwind/token migration	Would create a parallel styling system and increase drift risk
Storybook as a prerequisite	Useful later, but first use your established CSS contracts and app-screen tests
A new component framework	Existing components/screens/styles need an evidence-based audit first
Automatic Figma writes	Figma reads are appropriate for audit; design-file mutations need explicit review

The repository is public and currently identified as `harrisonjrubin7-cmyk/semester`, with `main` as the default branch. It contains roughly 53 open issues and a substantial TypeScript codebase, so start with a **read-only audit and minimal additive configuration**, rather than a sweeping UI-platform rewrite. [github_mcp_direct]

⁂

provide prompt to give claude to go along with doc to excute on everything

Paste the prompt below into **Claude Code from the root of your `semester` repository**, with the design-system PDF available in the repository or attached to the Claude session. It tells Claude to use the document as direction while adapting every change to Semester's actual architecture—not a generic monorepo/template. The PDF's core principle is right: the design system should be a product decision system built around semantic tokens, accessible components, reusable patterns, and governance—not merely a component collection. [36]

Master execution prompt

```
You are the implementation lead for the Semester design-system hardening project.

Your task is to read the attached design-system document and implement its useful rec
in the existing Semester repository. This is an execution task, not a generic design-
proposal. Work from actual repository evidence and preserve Semester's current archi

The attached document is a strategic reference. Follow its intent, but do not blindly
its generic repository structure, package names, assumptions, or example code.

======================================================================
PRIMARY OUTCOME
======================================================================

Make Claude Code, the existing Semester React application, its semantic CSS token sys
its generated design-token export, its existing tests, and Figma MCP work together as
coherent, enforceable design-system workflow.

When complete, Claude Code should be able to:

1. Build new Semester UI without generating generic SaaS/dashboard patterns.
2. Reuse existing Semester tokens, CSS patterns, React components, and screen conven
3. Know which token source is authoritative and which files are generated.
4. Audit a React route or component against token, accessibility, responsive, state,
   Figma-to-code rules.
5. Detect raw styling and design-system drift in local validation and CI.
6. Use Figma MCP as a design-intent input without letting Figma override code behavio
   accessibility, or production token authority.
7. Produce an actionable design-sync report before making large design-system changes

======================================================================
NON-NEGOTIABLE SEMESTER CONSTRAINTS
======================================================================

This is not a greenfield repository. Do not replace working architecture with the ger
architecture in the attached document.

Preserve the existing Semester design-system source-of-truth model:

- `app/src/styles/tokens.css` is the semantic CSS-token authority.
- `app/src/lib/tokenexport.ts` is the token-export authority.
- `app/design-tokens/semester.tokens.json` is generated output.
```

```
- Never hand-edit `app/design-tokens/semester.tokens.json`.
- Existing style contracts and tests are assets to extend, not artifacts to replace.
- Existing application UI lives under `app/src/components/`, `app/src/screens/`,
  `app/src/styles/`, `app/src/App.tsx`, `app/src/main.tsx`, and related modules.
- Existing Claude configuration lives in `CLAUDE.md` and `.claude/`.
- Existing repository scripts and package management must be discovered from actual
  `package.json` files before commands are run.

Do not introduce any of the following unless repository evidence proves they already
or I explicitly approve the change:

- Tailwind.
- Storybook.
- pnpm workspaces.
- A new `packages/ui` package.
- A new `packages/tokens` package.
- A monorepo migration.
- A second token source of truth.
- A new UI framework or component library.
- New dependencies.
- A large CSS rewrite.
- Automatic write access to Figma.
- Commits, pushes, pull requests, deployment changes, secret changes, or production c

Do not make Figma the production authority. Figma is an input for approved visual int
Code owns behavior, accessibility, responsive behavior, public APIs, and token genera

======================================================================
REQUIRED DISCOVERY — DO THIS FIRST
======================================================================

Before modifying any files, read and inspect the actual repository.

Read at minimum:

- `CLAUDE.md`
- `README.md`
- `app/README.md`
- `app/package.json`
- root `package.json`
- `DESIGN-SYSTEM-GUIDE.md`
- `docs/DESIGN-TOKENS.md`
- `docs/design/DESIGN-SYSTEM-PRODUCT-SPEC.md`
- `app/src/styles/tokens.css`
- `app/src/styles/app.css`
- `app/src/styles/rules.ts`
- `app/src/lib/tokenexport.ts`
- `app/src/lib/tokenexport.test.ts`
- `app/design-tokens/semester.tokens.json`
- all existing relevant `app/src/styles/*.test.ts`
- `app/src/components/`
- `app/src/screens/`
- `app/src/App.tsx`
- `app/src/main.tsx`
- `.claude/`
- `.github/`
```

```
Also inspect:
- Existing scripts that export or validate design tokens.
- Existing lint/typecheck/test scripts.
- Existing CI workflows.
- Existing accessibility, responsive, style, security, and visual regression mechanis
- Existing Figma references, if any.
- Existing design-system documentation and test fixtures.

Use the attached design-system document only after this discovery. Translate its reco
into the existing Semester implementation instead of forcing its sample repository s

======================================================================
PHASE 1 — PRODUCE A READ-ONLY BASELINE AUDIT
======================================================================

Do not edit any files yet.

Create a concise but evidence-based baseline report in:

`docs/design-system/SEMESTER-DESIGN-SYSTEM-BASELINE.md`

The report must include:

1. Current architecture
   - Current frontend stack.
   - Current styling approach.
   - Current token authority.
   - Current export/generation path.
   - Current component/screen organization.
   - Current design contract and test enforcement.

2. Source-of-truth map
   - Semantic token source.
   - Raw/foundation value location, if distinct.
   - Generated artifacts.
   - Current tests that prevent drift.
   - Current documentation.
   - Figma's intended role.

3. Existing strengths to preserve
   - Name exact tests, scripts, directories, and architectural choices.

4. Gaps relative to the attached document
   - Claude Code persistent instructions.
   - UI-generation procedure.
   - Audit procedure.
   - Figma mapping governance.
   - Token drift detection.
   - Raw-value enforcement.
   - Component/pattern discovery.
   - Accessibility and responsive checks.
   - CI reporting.

5. Risk classification
   - Blocker, major, minor.
```

```
   - Every finding must cite exact file paths and specific evidence.
   - Do not make assumptions based on generic conventions.

6. Minimal implementation plan
   - Exact files to add/change.
   - Why each change is necessary.
   - Acceptance tests for each change.
   - Explicit list of work intentionally deferred.

After writing this baseline report, stop and present:
- The proposed file-by-file implementation plan.
- Any assumptions requiring clarification.
- Any missing tooling or missing credentials.
- The validation commands you will run.

Do not continue implementation until the plan is internally coherent. If no clarifica
required, proceed to Phase 2.

======================================================================
PHASE 2 — IMPLEMENT CLAUDE CODE GOVERNANCE
======================================================================

Update the existing root `CLAUDE.md` carefully. Do not overwrite unrelated repository

Add or refine only the sections needed to make Claude Code work correctly with Semes

1. Semester design-system purpose
   - Build interfaces native to Semester.
   - Favor clear hierarchy, useful content, accessibility, responsive behavior, and
     consistent density over decorative/generic interface patterns.

2. Exact source-of-truth rules
   - `app/src/styles/tokens.css` is authoritative for semantic CSS tokens.
   - `app/src/lib/tokenexport.ts` produces the export.
   - `app/design-tokens/semester.tokens.json` is generated and never hand-edited.
   - Existing CSS tests are required design contracts.

3. UI implementation rules
   - Search existing components, screens, styles, and tests before creating new UI.
   - Prefer current Semester patterns and semantic CSS variables.
   - Do not introduce arbitrary raw colors, spacing, radii, shadows, z-indexes,
     typography values, animation timings, or easing values in feature UI.
   - Permit raw values only where existing architecture explicitly permits them:
     foundational token definitions, controlled generated outputs, established test f
     or documented exceptions.
   - Do not introduce arbitrary UI dependencies.
   - Do not create generic wrapper components or feature-local variants without evide
     that existing patterns are insufficient.

4. Accessibility requirements
   - Preserve native semantics.
   - Visible keyboard focus.
   - Accessible labels for icon-only controls.
   - Correct field label/description/error relationships.
   - Color is never the only status signal.
   - Respect reduced motion.
```

```
   - Preserve current project accessibility conventions and tests.

5. Responsive requirements
   - Follow existing Semester breakpoint, gutter, density, tap-target, stacking, and
     contracts.
   - Preserve critical content and actions at narrow widths.
   - Do not rely only on hover.

6. Figma protocol
   - Use Figma MCP to inspect design intent when a Figma URL/node/component/variable
   - Map Figma variables to current Semester exported token paths.
   - Map Figma components to current React/CSS patterns before creating anything.
   - Figma omissions do not excuse missing loading, empty, error, disabled, permissio
     keyboard, focus, or responsive states.
   - Record unresolved mappings in the Figma mapping document.

7. Completion criteria
   - Use actual repository scripts only.
   - Run relevant tests after changes.
   - Summarize files changed, existing assets reused, state/accessibility coverage,
     and remaining design-system gaps.

Keep the resulting `CLAUDE.md` concise, precise, and compatible with existing instruc

======================================================================
PHASE 3 — ADD SEMESTER-SPECIFIC CLAUDE SKILLS
======================================================================

Inspect `.claude/skills/` and follow existing repository conventions.

Add these skills only if they do not duplicate existing skills:

1. `.claude/skills/build-semester-ui/SKILL.md`
2. `.claude/skills/audit-semester-design-sync/SKILL.md`
3. `.claude/skills/create-semester-component/SKILL.md`

Each skill must be explicitly specialized for the current Semester repository.

Do not reference nonexistent imports such as:
- `@semester/ui`
- `@semester/tokens`
- `@semester/icons`
- `packages/ui`
- `packages/tokens`
- Storybook
- Tailwind
- pnpm

unless they are confirmed to exist.

### Build skill requirements

The UI-generation skill must:

- Read `CLAUDE.md`.
- Inspect existing related screens, components, CSS, token definitions, and tests fi
```

```
- Identify existing patterns to reuse before coding.
- Use semantic CSS variables and existing Semester styles/classes/contracts.
- Cover meaningful default, loading, empty, error, disabled, success, long-content,
  permission, accessibility, and responsive states when relevant.
- Require a concise implementation plan for complex UI.
- Require relevant tests and a completion summary.

### Audit skill requirements

The design-sync audit skill must:

- Work in a no-edit audit mode unless implementation is explicitly requested.
- Inspect the token pipeline:
  `tokens.css → tokenexport.ts → semester.tokens.json`.
- Identify raw design values and distinguish approved foundational use from feature-l
- Audit components/screens for duplicated styling and missing state coverage.
- Audit responsive, stacking, touch-target, and breakpoint behavior against current t
- Audit accessibility against current conventions and tests.
- Support optional Figma MCP inspection.
- Produce a structured report with:
  - Executive assessment.
  - Scope.
  - Token parity.
  - Component parity.
  - State coverage.
  - Accessibility review.
  - Existing enforcement.
  - Findings table with severity.
  - Minimal recommended implementation order.

### Component-creation skill requirements

The component-creation skill must:

- Require evidence that a reusable need exists.
- Require an audit of existing CSS and React patterns first.
- Reject one-off feature wrapper components.
- Define API, states, accessibility, responsive behavior, CSS/token usage, tests,
  documentation, and first adoption before implementation.
- Preserve the current architecture instead of introducing a parallel component libra

================================================================
PHASE 4 — FIGMA MCP CONFIGURATION AND MAPPING
================================================================

Set up project-safe Figma MCP support without committing credentials.

1. Detect whether an MCP configuration already exists.
2. If none exists, create a root `.mcp.json` with only the remote Figma endpoint:

{
  "mcpServers": {
    "figma": {
      "type": "http",
      "url": "https://mcp.figma.com/mcp"
    }
```

```
  }
}

3. Do not include credentials, tokens, cookies, auth headers, secrets, or local user
4. Do not make Figma writes or edits.
5. Add clear documentation that each developer must authenticate interactively throug

Create:

`docs/design-system/FIGMA-MAPPING.md`

It must document:

- Ownership:
  - Code owns behavior, accessibility, responsiveness, public APIs, and token values
  - Figma owns approved visual intent and design exploration.
  - `tokens.css` and `tokenexport.ts` own the code token pipeline.
  - `semester.tokens.json` is generated output.

- Variable mapping:
  - Figma collection.
  - Figma variable.
  - Semester exported token path.
  - CSS semantic source variable.
  - Status.
  - Notes.

- Component mapping:
  - Figma component.
  - Existing React screen/component/CSS pattern.
  - Variant/state mapping.
  - Status.
  - Notes.

- Mapping rules:
  - No new Figma variable becomes a production token without code review.
  - No Figma component becomes a React component automatically.
  - Missing Figma states do not remove code behavior requirements.
  - Unresolved mapping must be documented.

If no Figma file/variable metadata is available locally, create a clearly marked temp
not fabricated mappings.

Add a short setup section to the appropriate existing setup/developer document that t
developer to run:

`claude mcp add --scope project --transport http figma https://mcp.figma.com/mcp`

Then start Claude Code and run `/mcp` to authenticate. Do not claim authentication su
unless it actually succeeds interactively.

======================================================================
PHASE 5 — TOKEN, RAW-VALUE, AND DRIFT VALIDATION
======================================================================

Use existing tests and scripts first. Extend them when the current enforcement has a
```

```
Implement minimal compatible validation tooling under `app/scripts/` only if necessa

Required capabilities:

1. Token export integrity
   - Verify the generated `app/design-tokens/semester.tokens.json` matches the curre
     token generation source.
   - Reuse or invoke current token export/testing mechanisms.
   - Never create a competing token export.

2. Raw-value audit
   - Detect raw hex/rgb/hsl/oklch values, arbitrary spacing, radii, shadows, z-index
     typography values, durations, and easing values in feature/application UI.
   - Allow approved exceptions in the existing token source files, generated files,
     existing intentional fixtures, or specifically documented allowlists.
   - Report file path, line number, rule, matched value, and suggested existing
     token/pattern where possible.
   - Do not falsely fail on the canonical token definition files.

3. Figma mapping validation
   - Validate that every mapping in `docs/design-system/FIGMA-MAPPING.md` or an adja
     machine-readable mapping manifest resolves to a real path in
     `app/design-tokens/semester.tokens.json`.
   - Clearly separate:
     - valid mappings,
     - missing code-token mappings,
     - obsolete mappings,
     - unmapped but non-required candidates.
   - If no real Figma metadata can be fetched, validate only the local mapping manif

4. Design-system report
   - Write a deterministic report under `app/reports/design-system/` or another exis
     report convention in the repo.
   - Include:
     - token export status,
     - raw-value violations,
     - Figma mapping status,
     - relevant existing style-contract test status,
     - summary counts,
     - blocker/major/minor findings.
   - Ensure generated report paths are appropriately ignored or intentionally commit
     according to existing repository conventions.

5. Package scripts
   - Add only minimal scripts to `app/package.json`, using the repository's actual to
   - Prefer names:
     - `tokens:check`
     - `design-system:audit`
     - `design-system:report`
     - `design-system:check`
   - Do not use these exact names if existing scripts conflict; preserve established

6. Tests
   - Add focused tests using the repository's existing test framework.
   - Prove valid behavior passes.
```

```
   - Prove a real raw-value violation fails.
   - Prove a nonexistent mapped token fails.
   - Prove generated token output drift is detected.
   - Do not add brittle tests that fail due to unrelated existing legacy files withou
     explicit approved baseline/allowlist strategy.

========================================================================
PHASE 6 — CI INTEGRATION
========================================================================

Inspect existing `.github/workflows/` before adding or changing CI.

If there is an existing quality/test workflow:
- Integrate the new design-system validation into it.
- Do not create duplicate workflows unnecessarily.

If none exists:
- Create one minimal workflow focused on the existing app tooling.

The CI implementation must:

- Install dependencies with the repository's existing package manager and lockfile.
- Run existing relevant checks.
- Run token integrity validation.
- Run design-system raw-value audit.
- Run Figma mapping validation.
- Generate the design-system report.
- Upload the report as an artifact when CI supports artifacts.
- Avoid any use of production credentials, Figma credentials, or write access.
- Avoid attempting interactive Figma MCP authentication in CI.
- Be deterministic and safe for pull requests.

========================================================================
PHASE 7 — VALIDATE AND DOCUMENT
========================================================================

Run only commands verified in actual `package.json` files.

At minimum, run the relevant subset of:

- Existing token export/test commands.
- New token integrity checks.
- New raw-value audit.
- New mapping validation.
- Existing lint.
- Existing typecheck.
- Existing unit tests.
- Existing responsive/style/accessibility contract tests.
- Any relevant build command.

If a check fails:

1. Diagnose it.
2. Fix only failures caused by this implementation.
3. Do not hide pre-existing failures.
4. Clearly label pre-existing failures separately.
```

```
5. Do not weaken test rules merely to obtain a green result.

Update concise documentation explaining:

- Semester source-of-truth order.
- How to invoke each Claude skill.
- How to authenticate Figma MCP locally.
- How to run the design-system checks.
- How to add a Figma variable/component mapping.
- What must never be hand-edited.
- How to interpret the report.

======================================================================
FINAL RESPONSE FORMAT
======================================================================

At the end, provide:

## Summary
A concise statement of what was implemented.

## Architecture preserved
A table showing the existing Semester sources of truth and how the new toolkit uses t

## Files changed
A table with exact paths and purpose.

## Skills added
List each new or updated skill and how to invoke it.

## Figma MCP
State:
- Whether `.mcp.json` was added or an existing config was preserved.
- The exact local authentication steps.
- Any limitations because authentication requires user interaction.

## Validation
List every command run and its result.

## Report
State where the generated design-system report is written and summarize its results.

## Known limitations
List only evidence-based limitations, deferred work, or missing Figma metadata.

## Next implementation priorities
List up to five concrete follow-up items in dependency order.

Do not commit, push, create a pull request, deploy, change credentials, or mutate Fig
```

How to use it

1. Open Claude Code in the repository root:

```
cd /path/to/semester
claude
```

2. Attach or reference the PDF. If you save it in the repo, a practical temporary location is:

```
docs/design-system/reference/building-semester-design-system.pdf
```

3. Paste the master prompt.
4. Let Claude produce the baseline audit and file-by-file plan first. Review that plan before allowing broad changes.

Stronger staged option

For the safest execution, use the master prompt through **two passes**:

Pass 1: Audit only

Append this to the master prompt before sending:

```
For this first pass, execute only Phase 1. Do not edit CLAUDE.md, skills, MCP configu
scripts, package files, CI, or application code. Write the baseline audit and presen
Phase 2–7 plan for approval.
```

Pass 2: Implement approved plan

After reviewing the audit, send:

```
Proceed with Phases 2–7 exactly as proposed in the baseline audit, subject to these c

- Keep changes minimal and additive.
- Preserve `app/src/styles/tokens.css` as semantic token authority.
- Preserve `app/src/lib/tokenexport.ts` as export authority.
- Never manually edit `app/design-tokens/semester.tokens.json`.
- Use the existing test/tooling stack.
- Do not add dependencies or restructure the repository.
- Do not commit, push, deploy, alter secrets, or write to Figma.
- Stop and explain before any change that would violate these constraints.
```

Why this is repository-specific

Your current Semester repository already has:

- A root `CLAUDE.md` and an existing `.claude/skills/` directory.
- A Vite/React application under `app/`.

- A semantic CSS token source at `app/src/styles/tokens.css`.
- An existing token-export system at `app/src/lib/tokenexport.ts`.
- A generated cross-tool token artifact at `app/design-tokens/semester.tokens.json`.
- Existing token-export, style, breakpoint, density, motion, stacking, accessibility-related, and other design contract tests.
- Existing design system documentation, including `DESIGN-SYSTEM-GUIDE.md` and `docs/DESIGN-TOKENS.md`. [github_mcp_direct]

So this prompt makes Claude **extend your proven system** rather than replacing it with the PDF's generic `packages/tokens`, `@semester/ui`, Storybook, or monorepo examples.

⁂

1. https://code.claude.com/docs/en/memory
2. https://code.claude.com/docs/en/skills
3. https://github.com/anthropics/claude-code
4. https://www.reddit.com/r/ClaudeAI/comments/1m6hek6/claude_project_loaded_with_all_claude_code_docs/
5. https://www.freecodecamp.org/news/claude-code-handbook/
6. https://www.humanlayer.dev/blog/writing-a-good-claude-md
7. https://nervegna.substack.com/p/claude-code-for-designers-a-practical
8. https://theyawns.com/2026/03/06/launching-a-claude-code-project-design-first-then-build/
9. https://platform.claude.com/docs/en/home
10. https://support.claude.com/en/articles/14604397-set-up-your-design-system-in-claude-design
11. https://www-cdn.anthropic.com/58284b19e702b49db9302d5b6f135ad8871e7658.pdf
12. https://academy.claude.com/courses/claude-101/introduction-to-projects
13. https://claude.com/product/claude-code
14. https://readme.com/blog/what-is-claude-md
15. https://www.intodesignsystems.com/claude-code-for-designers
16. https://help.figma.com/hc/en-us/articles/39888612464151-Claude-Code-and-Figma-Set-up-the-MCP-server
17. https://help.figma.com/hc/en-us/articles/32132100833559-Guide-to-the-Figma-MCP-server
18. https://code.claude.com/docs/en/memory
19. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices
20. https://github.com/figma/mcp-server-guide
21. https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf
22. https://claude.com/blog/skills-explained
23. https://composio.dev/toolkits/figma/framework/claude-code
24. https://github.com/anthropics/skills
25. https://vectorize.io/articles/claude-code-memory
26. https://github.com/luongnv89/claude-howto/blob/main/02-memory/README.md

27. https://code.claude.com/docs/en/skills
28. https://www.figma.com/mcp-catalog/
29. https://www.figma.com/blog/introducing-claude-code-to-figma/
30. https://support.claude.com/en/articles/12123587-import-and-export-your-memory-from-claude
31. https://code.claude.com/docs/en/skills
32. https://help.figma.com/hc/en-us/articles/39888612464151-Claude-Code-and-Figma-Set-up-the-MCP-server
33. https://code.claude.com/docs/en/mcp
34. https://code.claude.com/docs/en/mcp
35. https://help.figma.com/hc/en-us/articles/39888612464151-Claude-Code-and-Figma-Set-up-the-MCP-server
36. building-semester-design-system-tell-me-everything.pdf

perplexity

building semester design system tell me everything it should provide in claude code right now

A strong design system is not just a component library—it is a **product decision system** that lets Claude Code generate consistent, accessible, responsive UI without inventing one-off colors, spacing, components, or interaction patterns.

For Claude Code, put the always-on rules in a root `CLAUDE.md`, keep canonical tokens/components in code, and add focused reusable skills for UI work. Claude Code reads project `CLAUDE.md` instructions at the start of sessions; Anthropic also supports reusable `SKILL.md` instruction packages for more specialized workflows. [1] [2]

What Semester should provide

Think of the system in six layers.

Layer	Semester should define	What Claude Code can do with it
Brand	Purpose, audience, voice, visual personality, anti-patterns	Make on-brand creative choices rather than generic SaaS UI
Foundations	Color, type, spacing, radii, elevation, motion, icons, breakpoints	Use stable primitives rather than arbitrary values
Semantic tokens	Roles such as `surface`, `text-primary`, `border-subtle`, `action-primary`	Support themes and visual updates without mass refactors
Components	API, states, variants, composition rules, accessibility behavior	Build screens from known parts instead of creating duplicates
Patterns	Forms, data views, navigation, onboarding, settings, empty/loading/error states	Solve repeated UX problems consistently
Governance	Quality checks, contribution workflow, deprecation rules, release/versioning	Keep the system coherent as the product grows

The key distinction: **primitive tokens** express raw values; **semantic tokens** express meaning. Components should almost always consume semantic tokens.

```
/* Avoid */
.card {
  background: #ffffff;
  color: #111827;
  border: 1px solid #e5e7eb;
  border-radius: 12px;
}
```

```
/* Prefer */
.card {
  background: var(--color-surface-raised);
  color: var(--color-text-primary);
  border: 1px solid var(--color-border-subtle);
  border-radius: var(--radius-lg);
}
```

That lets “Semester” evolve from one theme or brand expression to another by changing the token mapping—not every individual component.

Foundations to define

Brand and product direction

Give Claude an explicit design brief, including:

- What Semester is and who it serves.
- Primary product surfaces: marketing site, authenticated application, dashboard, editor, marketplace, documentation, mobile web, etc.
- Brand attributes—e.g. precise, editorial, warm, technical, playful, institutional, high-trust.
- The visual reference point: “quiet editorial software,” “high-density developer tool,” “premium consumer productivity app,” and so on.
- Explicit non-goals: no gradient-heavy glassmorphism, no random icon cards, no pill-everything interfaces, no excessive rounded containers, no arbitrary decoration.
- Content voice: sentence case versus title case, direct vs. playful microcopy, preferred terminology, capitalization rules, and banned language.
- Dark mode strategy: supported now, later, or intentionally unsupported.
- Internationalization assumptions: locales, text expansion, RTL needs, date/time and numeric formatting.
- Accessibility target: at minimum WCAG 2.2 AA, with clear expectations for keyboard support, focus visibility, contrast, reduced motion, errors, and screen-reader labels.

This context is important because a system can be technically consistent while still producing the wrong kind of product. Claude’s persistent project instructions are specifically intended to include architecture, conventions, commands, and common workflows. [1]

Token system

Define tokens in three levels:

1. **Reference tokens**: raw palette and scale values.
2. **Semantic tokens**: meaningful UI roles.
3. **Component tokens**: optional component-specific roles where genuinely needed.

Your initial token families should include:

- **Color**
  - Brand/accent scale
  - Neutral scale
  - Success, warning, danger, information
  - Surfaces: canvas, base, raised, overlay, inverse
  - Text: primary, secondary, muted, disabled, inverse, link
  - Borders: subtle, default, strong, focus, destructive
  - Interactive: default, hover, pressed, selected, disabled
- **Typography**
  - Font families and fallbacks
  - Type scale
  - Font weights
  - Line-height and letter-spacing
  - Fluid/responsive display styles if needed
  - Named semantic styles such as `body-sm`, `body-md`, `label-sm`, `heading-2xl`, `code-sm`
- **Spacing**
  - A constrained scale such as `0, 1, 2, 3, 4, 6, 8, 10, 12, 16, 20, 24`
  - Rules for when exceptions are allowed
- **Sizing**
  - Control heights
  - Icon sizes
  - Touch-target minima
  - Content-width and layout container sizes
- **Shape**
  - Border-widths
  - Radius scale
  - Avatar sizes
- **Elevation**
  - Surface levels
  - Shadow recipes
  - Overlay/modal treatment
- **Motion**
  - Durations, easing, interaction motion rules
  - Reduced-motion equivalents
- **Responsive layout**

    - Breakpoints
    - Grid columns
    - Page gutters
    - Max-widths
    - Stack-to-grid rules
- **Z-index**
    - A named, fixed layering ladder: base, sticky, dropdown, popover, modal, toast, tooltip
- **Opacity and effects**
    - Disabled/overlay opacity
    - Blur rules if used
    - Focus-ring treatment

Avoid raw CSS literals outside the token source unless the system has explicitly permitted them. That should be a non-negotiable Claude Code rule.

Layout grammar

A design system should provide the rules that make pages feel related:

- Global page gutters for compact, tablet, desktop, and wide desktop.
- Standard reading/content widths.
- Application-shell structure: sidebar, top bar, content region, panel region.
- Grid primitives: stack, inline, cluster, grid, split pane, center, container.
- Density modes, if Semester needs them: comfortable, standard, compact.
- Standard section rhythm: page title → context/actions → primary surface → supporting content.
- Alignment rules for labels, controls, icons, tables, and actions.
- Responsive priorities: which content stacks, hides, scrolls, truncates, or becomes a drawer.
- Safe-area handling for mobile and fixed navigation.
- Empty-state placement, loading skeleton layout, and error-state layout.

A small layout-primitives package prevents Claude from repeatedly hand-authoring subtly different flex/grid code.

Components and patterns

Essential components

Each component needs more than visual examples. Define:

- Intended use and non-use cases.
- Import path and implementation status.
- Variants and controlled props.
- Allowed composition patterns.
- All states: default, hover, focus-visible, active, selected, disabled, loading, invalid, read-only.
- Responsive behavior.
- Accessibility semantics and keyboard behavior.
- Content constraints: label lengths, icon placement, truncation, localization behavior.
- Test and Storybook expectations.

Start with the smallest durable set:

- Button, IconButton, Link, ButtonGroup.
- Text, Heading, Code, Label, Caption.
- Icon, Avatar, Badge, Status, Tag/Chip.
- Input, Textarea, Select/Combobox, Checkbox, RadioGroup, Switch, Slider, Date/Time input where needed.
- Field wrapper: label, description, required marker, validation, error text.
- Alert, Toast, Banner, InlineMessage.
- Card, Surface, Divider, List, KeyValue.
- Tabs, Breadcrumbs, Pagination, Stepper.
- Menu, Dropdown, Popover, Tooltip, Context Menu.
- Dialog, AlertDialog, Drawer, Sheet.
- Table/DataGrid, Filter bar, Search input, Sort controls.
- Skeleton, Spinner, Progress, EmptyState, ErrorState.
- Navigation: top nav, sidebar nav, mobile nav, nav group/item.
- Page header, section header, toolbar, action bar.

Product patterns

Components alone do not solve product consistency. Semester should also ship opinionated patterns for:

- Authentication: sign in, registration, password reset, invite acceptance, MFA.
- Onboarding: welcome, progressive setup, checklists, empty initial workspace.

- Forms: create/edit, validation timing, unsaved changes, autosave, destructive confirmation.
- Search and discovery: command palette, global search, filtering, no-results state.
- Data-heavy screens: list/detail, tables, row actions, bulk selection, saved views.
- CRUD lifecycle: create, success confirmation, edit, archive/delete, undo.
- Feedback: optimistic updates, loading, errors, retry, offline/degraded state.
- Permissions: restricted view, missing access, role explanation, request-access flow.
- Settings: organization/workspace/profile billing patterns.
- Notifications: in-app, toast, inbox, preferences.
- Mobile adaptation: priority content, navigation, drawers, sticky actions, touch targets.

For each pattern, document the preferred component composition, behavior, edge cases, and a production example.

Claude Code package

Your immediate goal is to make the system **machine-usable**. A good repository shape:

```
semester/
├── CLAUDE.md
├── README.md
├── docs/
│   ├── foundations.md
│   ├── content-and-voice.md
│   ├── accessibility.md
│   ├── responsive.md
│   ├── component-status.md
│   ├── patterns/
│   │   ├── forms.md
│   │   ├── data-views.md
│   │   ├── feedback.md
│   │   └── navigation.md
│   └── decisions/
├── packages/
│   ├── tokens/
│   ├── icons/
│   ├── ui/
│   └── eslint-config/
├── apps/
│   └── web/
├── .claude/
│   └── skills/
│       ├── build-ui/
│       │   └── SKILL.md
│       ├── audit-ui/
│       │   └── SKILL.md
│       └── create-component/
│           └── SKILL.md
└── .storybook/
```

Use `CLAUDE.md` for short, always-applicable instructions. Claude Code supports root or `.claude/CLAUDE.md` project memory files and supports reusable skills that live in `SKILL.md` files. [2] [1]

Starter `CLAUDE.md`

```
# Semester Design System

## Mission
Semester is the sole source of truth for UI foundations, React components,
product interaction patterns, and accessibility rules.

Build interfaces that feel deliberate, calm, information-rich, and native to
Semester. Favor clear hierarchy and useful content over decorative UI.

## Non-negotiable rules
- Use only exports from `@semester/ui`, `@semester/tokens`, and `@semester/icons`
  before creating new UI primitives.
- Never use arbitrary hex/rgb/hsl colors, pixel spacing values, radii, shadows,
  z-index values, font sizes, or animation timings in product UI.
- Use semantic tokens, not raw palette tokens, in components and application code.
- Do not add a new component until existing components and compositions have been eva
- Do not modify generated token files manually.
- Do not use emoji as interface icons. Use `@semester/icons`.
- Do not introduce a dependency for UI without explicit approval.
- Preserve keyboard operation, visible focus, semantic HTML, and WCAG 2.2 AA contras
- Support narrow screens first; do not hide critical workflows on mobile.
- Respect `prefers-reduced-motion`.

## UI workflow
1. Inspect the nearest relevant screen and existing component stories.
2. Search `@semester/ui` for a suitable component or documented composition.
3. Write a short implementation plan for new flows or new primitives.
4. Implement with tokens and existing primitives.
5. Cover loading, empty, error, disabled, and permission-restricted states.
6. Run typecheck, lint, unit tests, and Storybook checks.
7. Summarize changed files, reused system elements, and accessibility decisions.

## Component API principles
- Prefer composable primitives over large boolean-prop APIs.
- Use `variant`, `size`, and `tone` only when they map to documented design choices.
- Defaults must be safe, accessible, and visually standard.
- Every interactive component needs a disabled and loading strategy where relevant.
- Component names are singular PascalCase; props use camelCase.
- Use `data-slot` for stylable component subparts where needed.

## Accessibility
- Use native HTML controls whenever possible.
- All icon-only controls require an accessible name.
- Error messages must be programmatically associated with their field.
- Dialogs must trap focus, restore focus on close, and close on Escape unless unsafe
- Do not communicate meaning through color alone.
- Minimum interactive target: 44 by 44 CSS pixels where practical.

## Verification
```

```
Run the commands in package.json appropriate to touched packages.
For visual changes, add or update Storybook stories for every new variant and state.
```

High-value Claude skills

Create three initial skills:

- `build-ui`: Builds a feature using existing Semester components, requires state coverage and responsive behavior, and prohibits new primitives without justification.
- `audit-ui`: Reviews a route or component for token violations, accessibility, hierarchy, responsiveness, and inconsistent patterns.
- `create-component`: Creates a component only after documenting purpose, API, states, accessibility contract, stories, tests, and adoption example.

A short `build-ui/SKILL.md` might say:

```
---
name: build-ui
description: Build or revise an interface using the Semester design system.
---

Before editing:
1. Read CLAUDE.md and relevant docs in docs/patterns.
2. Inspect existing components and their Storybook stories.
3. State which Semester components and patterns will be used.

Implementation requirements:
- Reuse established components first.
- Use semantic design tokens only.
- Include loading, empty, error, disabled, and narrow-screen behavior where applicabl
- Keep visual hierarchy simple: one primary action per scope.
- Do not create a new primitive unless the existing system cannot support the need.

Before completion:
- Check keyboard navigation and focus visibility.
- Check labels, descriptions, errors, and icon-only controls.
- Run typecheck, lint, and relevant tests.
- Report token/component reuse and any proposed system gap.
```

Build order

Do **not** begin by producing 80 components. Build in this order:

1. Write the brand, product, content, accessibility, and responsive rules.
2. Build and validate tokens—including semantic tokens and light/dark mappings if dark mode matters.
3. Add layout primitives and application shell.
4. Ship typography, buttons, links, icons, fields, feedback, overlays, and navigation.
5. Establish page, form, data-view, and empty/loading/error patterns.

6. Build only the domain-specific components Semester repeatedly needs.
7. Add Storybook examples, tests, lint rules, visual regression checks, and an adoption/migration guide.
8. Measure system health: percentage of UI using system components, number of token violations, duplicate component count, accessibility failures, and time required to ship a typical screen.

For every feature, define "done" as: visual system compliance, functional behavior, responsive behavior, keyboard support, semantic markup, all meaningful states, tests, and documentation when the system changes.

Governance and guardrails

Semester needs mechanisms that stop entropy:

- **Source of truth:** Specify whether tokens live in CSS variables, TypeScript, JSON, Style Dictionary, or another pipeline—and which artifact is generated.
- **Ownership:** Name a design-system owner or review group for foundational changes.
- **Contribution template:** Problem, users affected, evidence, proposed API, alternatives considered, accessibility notes, migration impact.
- **Deprecation policy:** Never silently break a component API; mark deprecated, document migration, provide a removal version/date.
- **Visual regression testing:** Capture stories and critical product routes.
- **Automated checks:** Reject raw colors, unauthorized spacing values, unapproved z-index values, inline SVGs when icon components exist, and inaccessible interactive markup.
- **Component maturity:** Track `experimental`, `beta`, `stable`, and `deprecated` states.
- **Changelog:** Record new tokens, component changes, behavior changes, and migrations.
- **Decision records:** Preserve important choices—such as why you use 4px vs. 8px base spacing, or whether destructive actions use red buttons—so Claude does not reopen settled decisions.

The goal is that Claude Code can answer, before it writes code: *Which existing primitive solves this? Which semantic token expresses its role? What states and accessibility constraints apply? How do I verify the result?* That is the threshold at which Semester becomes a real system rather than a styling folder.

⁂

Build a complete Claude Code design system config: CLAUDE.md templates, token schemas, Figma MCP setup, and SKILL.md rules

Computer
Claude Code design-sync audit: how to map your React component library and design tokens so Claude stops generating generic UI
Computer
Show me an example CLAUDE.md for this design system
How should I structure the semantic tokens for Semester
How do I write a SKILL.md for frontend UI generation
Sources
15

Below is a production-oriented starting configuration for **Semester**: a token-first React design system that Claude Code can reliably use, audit, and extend without falling back to generic UI.

It uses four complementary mechanisms:

- Root `CLAUDE.md` for durable rules that should apply to every coding task.
- Typed semantic token files as the implementation source of truth.
- Figma MCP for reading variables, components, layout context, and selected frames in Figma.
- Task-specific `SKILL.md` files for building UI, auditing drift, and adding components. Figma's recommended Claude Code setup is its official plugin, which bundles the MCP configuration and workflow skills; Figma MCP also exposes design context including components, variables, layout information, FigJam content, and Code Connect mappings. [16] [17]

Repository structure

Use a structure where tokens and components are code-owned, Figma is a design input, and written rules are compact enough for Claude to apply consistently.

```
semester/
├── CLAUDE.md
├── package.json
├── pnpm-workspace.yaml
├── .mcp.json                          # Optional project-scoped MCP definition
├── .claude/
│   ├── settings.json
│   └── skills/
│       ├── build-ui/
│       │   ├── SKILL.md
│       │   └── references/
│       │   │   ├── component-inventory.md
│       │   │   ├── screen-checklist.md
```

```
│       │   │   └── figma-workflow.md
│       ├── audit-design-sync/
│       │   ├── SKILL.md
│       │   └── references/
│       │       └── audit-report-template.md
│       └── create-component/
│           ├── SKILL.md
│           └── references/
│               └── component-rfc-template.md
├── docs/
│   ├── architecture.md
│   ├── design-principles.md
│   ├── accessibility.md
│   ├── content-style.md
│   ├── figma-mapping.md
│   ├── component-inventory.md
│   ├── patterns/
│   │   ├── application-shell.md
│   │   ├── forms.md
│   │   ├── feedback.md
│   │   ├── navigation.md
│   │   └── data-display.md
│   └── decisions/
│       ├── 001-token-architecture.md
│       └── 002-component-api.md
├── packages/
│   ├── tokens/
│   │   ├── src/
│   │   │   ├── primitives.ts
│   │   │   ├── semantic.ts
│   │   │   ├── components.ts
│   │   │   ├── themes/
│   │   │   │   ├── light.ts
│   │   │   │   └── dark.ts
│   │   │   ├── index.ts
│   │   │   └── validate.ts
│   │   ├── dist/
│   │   └── package.json
│   ├── icons/
│   ├── ui/
│   │   ├── src/
│   │   │   ├── components/
│   │   │   ├── primitives/
│   │   │   └── styles/
│   │   ├── stories/
│   │   └── package.json
│   └── eslint-config/
├── apps/
│   ├── web/
│   └── storybook/
└── scripts/
    ├── audit-token-usage.ts
    └── validate-design-system.ts
```

The important hierarchy is:

```
Raw values → reference tokens → semantic tokens → component tokens → components → pro
```

Application code should primarily consume **semantic tokens and components**. It should not choose colors, radii, shadows, or spacings ad hoc.

Root CLAUDE.md

Place this at the repository root as `CLAUDE.md`. Claude Code supports persistent project instructions in `CLAUDE.md`; its Skills system loads more focused `SKILL.md` files dynamically when the work is relevant, so reserve the root file for universal rules rather than stuffing every implementation detail into it.[18] [19]

```
# Semester Design System — Claude Code Instructions

## System purpose

Semester is the canonical design system for this repository.

It provides:
- Design tokens and theme mappings.
- Accessible React UI primitives and composed components.
- Layout, content, responsive, and interaction patterns.
- Storybook documentation and visual regression coverage.
- A controlled mapping between Figma components/variables and production code.

The goal is not merely visual consistency. The system must enable fast product delive
without sacrificing accessibility, usability, performance, or maintainability.

## Required reading

Before non-trivial UI work, read:
- `docs/design-principles.md`
- `docs/accessibility.md`
- `docs/component-inventory.md`
- The relevant file in `docs/patterns/`
- The component's existing Storybook stories, if modifying a component.

For Figma-derived work, also read:
- `docs/figma-mapping.md`

## Core rules

1. Reuse before creating.
   - Search `@semester/ui` and the component inventory before writing custom UI.
   - Prefer composing existing primitives over creating feature-local components.
   - Do not add a component when a documented composition satisfies the need.

2. Tokens are mandatory.
   - Do not add arbitrary color, spacing, radius, shadow, z-index, font-size,
     font-weight, line-height, duration, or easing values in application UI.
   - Use semantic tokens in application and component code.
   - Raw palette/reference tokens may only be used in token definitions and theme map
   - Do not edit generated token output in `dist/`.
```

```
3. Accessibility is a release requirement.
   - Target WCAG 2.2 AA.
   - Prefer native HTML elements and controls.
   - Every keyboard-reachable element must have visible focus.
   - Every icon-only control needs an accessible name.
   - Do not communicate state or validation through color alone.
   - Use semantic landmarks and heading hierarchy.
   - Dialogs must manage focus, restore it on close, and support Escape when safe.
   - Respect `prefers-reduced-motion`.
   - Maintain at least 44 by 44 CSS pixel targets for touch interactions where pract

4. Responsive behavior is intentional.
   - Implement narrow layouts first, then enhance.
   - Do not hide critical user actions on small screens.
   - Use layout primitives, container queries, and documented breakpoints.
   - State the narrow-screen strategy for each new complex view.

5. Interface quality.
   - One primary action per decision scope.
   - Use direct sentence-case labels and useful empty/error states.
   - Avoid generic dashboard decoration: arbitrary cards, excessive pills,
     gradients, glass effects, emoji as icons, or ornamental color blocks.
   - Favor clear hierarchy, scannability, and appropriate information density.
   - Use `@semester/icons`; never use emoji or ad-hoc SVGs as UI icons.

6. Dependencies and architecture.
   - Do not introduce a UI dependency without explicit approval.
   - Do not modify public component APIs without considering migration impact.
   - Do not modify generated files manually.
   - Do not disable lint, type, test, or accessibility checks to complete a task.

## Required UI workflow

For a new or substantially revised UI flow:

1. Inspect the nearest equivalent implementation and existing Storybook stories.
2. Identify existing Semester components, patterns, and semantic tokens to reuse.
3. For a complex flow, present a short plan before implementation:
   - Existing components to reuse.
   - New composition or primitive needed, if any.
   - Responsive approach.
   - Loading, empty, error, disabled, and permission states.
   - Accessibility risks and mitigations.
4. Implement using `@semester/ui`, `@semester/tokens`, and `@semester/icons`.
5. Verify all meaningful states and keyboard behavior.
6. Run the relevant validation commands.
7. Summarize:
   - Files changed.
   - System components and tokens reused.
   - Accessibility behavior checked.
   - Any newly discovered design-system gap.

## Component API conventions

- Components use PascalCase; props use camelCase.
```

```
- Use `variant`, `size`, and `tone` only for documented visual choices.
- Prefer composition and slots over large boolean-prop APIs.
- Prefer explicit names such as `isLoading`, `isInvalid`, and `aria-label`.
- Defaults must be safe, accessible, and visually standard.
- Expose stable semantic subparts with `data-slot` only when consumers need styling h
- Every control must document its disabled, loading, invalid, and read-only behavior
- New components require TypeScript types, tests, Storybook stories, and accessibili

## Figma protocol

When a task references a Figma URL, frame, component, variable collection, or design
1. Use Figma MCP to inspect the selected node and its hierarchy.
2. Treat Figma as evidence of design intent, not as literal markup to reproduce.
3. Map Figma variables to existing semantic tokens before adding a token.
4. Map Figma components to existing React components before creating a component.
5. Preserve behavior, semantics, responsiveness, and accessibility even where Figma d
6. Record a mapping mismatch in `docs/figma-mapping.md` or an ADR; do not silently fo

## Design-system completion criteria

A UI task is not complete until:
- It uses documented components and semantic tokens.
- It covers required async, empty, error, disabled, and restricted states.
- It works at narrow and wide viewport sizes.
- It is keyboard-operable and has visible focus behavior.
- It has correct semantic HTML and accessible names.
- Typecheck, lint, tests, and relevant Storybook/visual checks pass.
- System documentation is updated if the source of truth changes.

## Commands

Use package scripts; do not guess command names.

Read `package.json` first, then run the appropriate commands for touched packages:
- `pnpm lint`
- `pnpm typecheck`
- `pnpm test`
- `pnpm test:visual`
- `pnpm storybook`
- `pnpm tokens:build`
- `pnpm tokens:validate`
- `pnpm design-system:audit`

## Documentation references

@docs/design-principles.md
@docs/accessibility.md
@docs/component-inventory.md
@docs/figma-mapping.md
```

Keep the `@docs/...` imports only if those files exist. The overall principle is correct even if you remove the import lines: persistent instructions should point Claude to concise, authoritative sources rather than duplicating every rule in a huge root file.

Semantic token schema

Naming model

Use a dot-delimited name that encodes **meaning**, not a particular visual value:

```
{category}.{role}.{emphasis-or-state}
```

Examples:

```
color.text.primary
color.text.muted
color.surface.canvas
color.surface.raised
color.border.subtle
color.action.primary.default
color.action.primary.hover
color.status.danger.text
space.4
radius.lg
shadow.overlay
motion.duration.fast
```

Avoid names based on visual appearance in consumer code:

```
# Bad application-facing names
blue.500
gray.200
purpleButton
largeRadius
cardShadow2

# Good application-facing names
color.action.primary.default
color.border.default
radius.control
shadow.raised
```

Token tier policy

Token tier	Example	Allowed consumers	Purpose
Reference	`ref.color.blue.600`	Token/theme files only	Raw palette, scale, and numerical values
Semantic	`color.text.primary`	Components and application UI	Stable role-based interface contract

Token tier	Example	Allowed consumers	Purpose
Component	`button.primary.background.default`	Component implementation only	Component-specific alias where semantic roles need narrowing
Local	CSS custom property scoped to a component	Rarely, with review	Temporary/internal implementation detail, never a new global convention

`primitives.ts`

This contains raw values. Do not import it directly into React components.

```
// packages/tokens/src/primitives.ts
export const primitive = {
  color: {
    white: "#FFFFFF",
    black: "#0A0A0B",

    neutral: {
      0: "#FFFFFF",
      25: "#FCFCFD",
      50: "#F8F9FA",
      100: "#F1F3F5",
      200: "#E9ECEF",
      300: "#DEE2E6",
      400: "#CED4DA",
      500: "#ADB5BD",
      600: "#6C757D",
      700: "#495057",
      800: "#343A40",
      900: "#212529",
      950: "#101214",
    },

    brand: {
      50: "#EEF4FF",
      100: "#D9E7FF",
      200: "#BBD7FF",
      300: "#8CC0FF",
      400: "#55A0FF",
      500: "#2F7CF6",
      600: "#1E63D8",
      700: "#194FAC",
      800: "#1B438C",
      900: "#1B396F",
    },

    red: {
      50: "#FEF2F2",
      100: "#FEE2E2",
      200: "#FECACA",
      500: "#EF4444",
```

```
      600: "#DC2626",
      700: "#B91C1C",
      900: "#7F1D1D",
    },

    amber: {
      50: "#FFFBEB",
      100: "#FEF3C7",
      500: "#F59E0B",
      600: "#D97706",
      700: "#B45309",
      900: "#78350F",
    },

    green: {
      50: "#F0FDF4",
      100: "#DCFCE7",
      500: "#22C55E",
      600: "#16A34A",
      700: "#15803D",
      900: "#14532D",
    },
  },

  space: {
    0: "0",
    px: "1px",
    0_5: "0.125rem",
    1: "0.25rem",
    1_5: "0.375rem",
    2: "0.5rem",
    2_5: "0.625rem",
    3: "0.75rem",
    4: "1rem",
    5: "1.25rem",
    6: "1.5rem",
    8: "2rem",
    10: "2.5rem",
    12: "3rem",
    16: "4rem",
    20: "5rem",
    24: "6rem",
  },

  radius: {
    none: "0",
    xs: "0.125rem",
    sm: "0.25rem",
    md: "0.375rem",
    lg: "0.5rem",
    xl: "0.75rem",
    "2xl": "1rem",
    full: "9999px",
  },

  font: {
```

```
    family: {
      sans: "Inter, ui-sans-serif, system-ui, sans-serif",
      mono: '"JetBrains Mono", ui-monospace, SFMono-Regular, monospace',
    },
    size: {
      xs: "0.75rem",
      sm: "0.875rem",
      md: "1rem",
      lg: "1.125rem",
      xl: "1.25rem",
      "2xl": "1.5rem",
      "3xl": "1.875rem",
      "4xl": "2.25rem",
    },
    weight: {
      regular: 400,
      medium: 500,
      semibold: 600,
      bold: 700,
    },
    lineHeight: {
      tight: 1.15,
      snug: 1.3,
      normal: 1.5,
      relaxed: 1.65,
    },
  },

  duration: {
    instant: "0ms",
    fast: "120ms",
    normal: "180ms",
    slow: "280ms",
  },

  easing: {
    standard: "cubic-bezier(0.2, 0, 0, 1)",
    enter: "cubic-bezier(0, 0, 0.2, 1)",
    exit: "cubic-bezier(0.4, 0, 1, 1)",
  },

  zIndex: {
    base: 0,
    raised: 10,
    sticky: 100,
    dropdown: 200,
    popover: 300,
    overlay: 400,
    modal: 500,
    toast: 600,
    tooltip: 700,
  },
} as const;
```

`semantic.ts`

These are the names React components should consume. The example uses token aliasing. If you use Style Dictionary, Tokens Studio, or another token pipeline, represent the same relationships in the appropriate source format.

```
// packages/tokens/src/semantic.ts
export const semantic = {
  color: {
    text: {
      primary: "{ref.color.neutral.950}",
      secondary: "{ref.color.neutral.700}",
      muted: "{ref.color.neutral.600}",
      disabled: "{ref.color.neutral.500}",
      inverse: "{ref.color.neutral.0}",
      link: "{ref.color.brand.700}",
      linkHover: "{ref.color.brand.800}",
      danger: "{ref.color.red.700}",
      success: "{ref.color.green.700}",
      warning: "{ref.color.amber.700}",
    },

    icon: {
      primary: "{color.text.primary}",
      secondary: "{color.text.secondary}",
      muted: "{color.text.muted}",
      disabled: "{color.text.disabled}",
      inverse: "{color.text.inverse}",
      brand: "{color.action.primary.default}",
      danger: "{color.status.danger.strong}",
    },

    surface: {
      canvas: "{ref.color.neutral.25}",
      base: "{ref.color.neutral.0}",
      raised: "{ref.color.neutral.0}",
      sunken: "{ref.color.neutral.50}",
      overlay: "{ref.color.neutral.0}",
      inverse: "{ref.color.neutral.950}",
      selected: "{ref.color.brand.50}",
      disabled: "{ref.color.neutral.100}",
    },

    border: {
      subtle: "{ref.color.neutral.200}",
      default: "{ref.color.neutral.300}",
      strong: "{ref.color.neutral.400}",
      focus: "{ref.color.brand.500}",
      disabled: "{ref.color.neutral.200}",
      danger: "{ref.color.red.500}",
    },

    action: {
      primary: {
        default: "{ref.color.brand.600}",
```

```
      hover: "{ref.color.brand.700}",
      active: "{ref.color.brand.800}",
      foreground: "{ref.color.white}",
      disabled: "{ref.color.neutral.300}",
      disabledForeground: "{ref.color.neutral.600}",
    },

    secondary: {
      default: "{color.surface.base}",
      hover: "{ref.color.neutral.50}",
      active: "{ref.color.neutral.100}",
      foreground: "{color.text.primary}",
      border: "{color.border.default}",
    },

    subtle: {
      default: "transparent",
      hover: "{ref.color.neutral.100}",
      active: "{ref.color.neutral.200}",
      foreground: "{color.text.primary}",
    },

    danger: {
      default: "{ref.color.red.600}",
      hover: "{ref.color.red.700}",
      active: "{ref.color.red.900}",
      foreground: "{ref.color.white}",
    },
  },

  status: {
    info: {
      subtle: "{ref.color.brand.50}",
      default: "{ref.color.brand.600}",
      strong: "{ref.color.brand.700}",
      text: "{ref.color.brand.800}",
    },
    success: {
      subtle: "{ref.color.green.50}",
      default: "{ref.color.green.600}",
      strong: "{ref.color.green.700}",
      text: "{ref.color.green.900}",
    },
    warning: {
      subtle: "{ref.color.amber.50}",
      default: "{ref.color.amber.600}",
      strong: "{ref.color.amber.700}",
      text: "{ref.color.amber.900}",
    },
    danger: {
      subtle: "{ref.color.red.50}",
      default: "{ref.color.red.600}",
      strong: "{ref.color.red.700}",
      text: "{ref.color.red.900}",
    },
  },
```

```
    focus: {
      ring: "{ref.color.brand.500}",
      ringOffset: "{color.surface.base}",
    },
  },

  typography: {
    body: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.sm}",
        fontWeight: "{ref.font.weight.regular}",
        lineHeight: "{ref.font.lineHeight.normal}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.md}",
        fontWeight: "{ref.font.weight.regular}",
        lineHeight: "{ref.font.lineHeight.normal}",
      },
    },

    label: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.sm}",
        fontWeight: "{ref.font.weight.medium}",
        lineHeight: "{ref.font.lineHeight.snug}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.md}",
        fontWeight: "{ref.font.weight.medium}",
        lineHeight: "{ref.font.lineHeight.snug}",
      },
    },

    heading: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.lg}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.xl}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
      },
      lg: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.2xl}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
```

```
      },
    },
  },

  space: {
    inline: {
      xs: "{ref.space.1}",
      sm: "{ref.space.2}",
      md: "{ref.space.3}",
      lg: "{ref.space.4}",
    },
    stack: {
      xs: "{ref.space.2}",
      sm: "{ref.space.3}",
      md: "{ref.space.4}",
      lg: "{ref.space.6}",
      xl: "{ref.space.8}",
      "2xl": "{ref.space.12}",
    },
    inset: {
      xs: "{ref.space.2}",
      sm: "{ref.space.3}",
      md: "{ref.space.4}",
      lg: "{ref.space.6}",
    },
    page: {
      mobile: "{ref.space.4}",
      tablet: "{ref.space.6}",
      desktop: "{ref.space.8}",
    },
  },

  size: {
    control: {
      sm: "2rem",
      md: "2.5rem",
      lg: "3rem",
    },
    icon: {
      xs: "0.875rem",
      sm: "1rem",
      md: "1.25rem",
      lg: "1.5rem",
    },
    target: {
      minimum: "2.75rem",
    },
    content: {
      readable: "42rem",
      default: "72rem",
      wide: "90rem",
    },
  },

  radius: {
    control: "{ref.radius.md}",
```

```
    surface: "{ref.radius.lg}",
    overlay: "{ref.radius.xl}",
    pill: "{ref.radius.full}",
  },

  border: {
    default: "1px",
    strong: "2px",
  },

  shadow: {
    raised: "0 1px 2px rgb(16 18 20 / 0.06), 0 1px 3px rgb(16 18 20 / 0.10)",
    overlay: "0 12px 32px rgb(16 18 20 / 0.16), 0 2px 8px rgb(16 18 20 / 0.08)",
  },

  motion: {
    duration: {
      instant: "{ref.duration.instant}",
      fast: "{ref.duration.fast}",
      normal: "{ref.duration.normal}",
      slow: "{ref.duration.slow}",
    },
    easing: {
      standard: "{ref.easing.standard}",
      enter: "{ref.easing.enter}",
      exit: "{ref.easing.exit}",
    },
  },

  zIndex: {
    base: "{ref.zIndex.base}",
    sticky: "{ref.zIndex.sticky}",
    dropdown: "{ref.zIndex.dropdown}",
    popover: "{ref.zIndex.popover}",
    overlay: "{ref.zIndex.overlay}",
    modal: "{ref.zIndex.modal}",
    toast: "{ref.zIndex.toast}",
    tooltip: "{ref.zIndex.tooltip}",
  },
} as const;
```

Component-token example

Use component tokens when a component needs a stable contract that maps to semantic intent but should not expose every internal choice to product teams.

```
// packages/tokens/src/components.ts
export const component = {
  button: {
    primary: {
      background: {
        default: "{color.action.primary.default}",
        hover: "{color.action.primary.hover}",
        active: "{color.action.primary.active}",
```

```
        disabled: "{color.action.primary.disabled}",
      },
      foreground: {
        default: "{color.action.primary.foreground}",
        disabled: "{color.action.primary.disabledForeground}",
      },
      focusRing: "{color.focus.ring}",
      height: {
        sm: "{size.control.sm}",
        md: "{size.control.md}",
        lg: "{size.control.lg}",
      },
      radius: "{radius.control}",
      gap: "{space.inline.sm}",
    },
  },

  field: {
    background: {
      default: "{color.surface.base}",
      disabled: "{color.surface.disabled}",
    },
    border: {
      default: "{color.border.default}",
      hover: "{color.border.strong}",
      focus: "{color.border.focus}",
      invalid: "{color.border.danger}",
      disabled: "{color.border.disabled}",
    },
    text: {
      value: "{color.text.primary}",
      placeholder: "{color.text.muted}",
      disabled: "{color.text.disabled}",
    },
    height: {
      sm: "{size.control.sm}",
      md: "{size.control.md}",
      lg: "{size.control.lg}",
    },
    radius: "{radius.control}",
  },
} as const;
```

Required token validation

Add a script that fails CI if:

- An application package imports `primitive` tokens.
- CSS/TSX contains raw hex, `rgb()`, `hsl()`, arbitrary shadows, or undocumented z-index values.
- A semantic token points directly to a component token, creating reverse dependency.
- Token aliases reference missing keys or circular references.

- A token change creates insufficient contrast for documented text/control combinations.
- Generated CSS, JSON, TypeScript, and Figma token exports differ from source.

Figma MCP setup

Recommended plugin installation

Figma recommends its official Claude Code plugin because it includes MCP server settings and agent skills for common workflows. Run this in your terminal:

```
claude plugin install figma@claude-plugins-official
```

Then restart Claude Code, run `/plugin`, open the **Installed** tab, select the Figma plugin, and complete the authorization flow. [16]

Confirm the connection in Claude Code:

```
/mcp
```

Manual remote setup

If you prefer manual configuration, Figma documents its hosted remote MCP endpoint as:

```
claude mcp add --transport http figma https://mcp.figma.com/mcp
```

Then restart Claude Code and authenticate when prompted. Figma identifies its remote server as the preferred option; it is hosted at `https://mcp.figma.com/mcp`. [17] [20]

Inspect your configuration with:

```
claude mcp list
claude mcp get figma
```

Manual desktop setup

The desktop server is useful when you are actively working in an open Figma file:

1. Install and open Figma Desktop.
2. Open a Figma Design file.
3. Switch to Dev Mode with nothing selected.
4. Enable the MCP server in the right sidebar.
5. Add it to Claude Code:

```
claude mcp add --transport http figma-desktop http://127.0.0.1:3845/mcp
```

6. Restart Claude Code and use `/mcp` to verify it is available. [16]

Project-local `.mcp.json`

Use this only if your team wants the connection declared alongside the repository. Do not commit secrets or static access tokens.

```
{
  "mcpServers": {
    "figma": {
      "type": "http",
      "url": "https://mcp.figma.com/mcp"
    }
  }
}
```

Prefer the official plugin when possible; it reduces configuration drift and gives the team the same Figma-aware workflow baseline. Figma MCP can surface design variables, components, layout context, and Code Connect mappings, which is exactly the context Claude needs to translate approved designs into your existing system rather than inventing new components. [16]

Figma-to-code mapping file

Create `docs/figma-mapping.md` and maintain it as an explicit contract.

```
# Semester Figma-to-Code Mapping

## Source of truth

- Code is the source of truth for component behavior, accessibility, and public APIs
- Figma is the source of truth for approved visual design and design exploration.
- Token source files in `packages/tokens/src/` are the source of truth for productio
- Figma variables are synchronized exports of the token system where possible.

## Variable mapping

| Figma collection | Figma variable | Code token | Notes |
|---|---|---|---|
| Semester / Semantic / Color | `Text / Primary` | `color.text.primary` | Default rea
| Semester / Semantic / Color | `Surface / Base` | `color.surface.base` | Standard co
| Semester / Semantic / Color | `Action / Primary / Default` | `color.action.primary
| Semester / Semantic / Space | `Stack / MD` | `space.stack.md` | Default vertical gr
| Semester / Component / Button | `Primary / Background / Hover` | `button.primary.ba

## Component mapping

| Figma component | React export | Status | Notes |
|---|---|---|---|
```

```
| `Button` | `@semester/ui/Button` | Stable | Figma variants map to `variant`, `size
| `Text field` | `@semester/ui/TextField` | Stable | Always use with `Field` when lab
| `Dialog` | `@semester/ui/Dialog` | Stable | Code owns focus management and Escape b
| `Data table` | `@semester/ui/DataTable` | Beta | Use only in approved data-heavy ro
| `Metric card` | No direct export | Deprecated | Compose `Surface`, `Stack`, `Text`

## Mapping rules

- A Figma component is not automatically a React component.
- Figma variants map only to documented public props.
- Local Figma styles must not become production tokens without review.
- New Figma variables require a code-token proposal before use in product UI.
- Record unresolved mismatches in an ADR or GitHub issue.
```

Design-sync audit

Use this mapping to audit drift between Figma and React:

Audit area	Verify	Typical failure	Fix
Token parity	Every approved Figma semantic variable has a code equivalent	Figma-only colors or spacing	Add or map an approved semantic token
Component parity	Figma variants map to a React component prop/state	Designers create a variant unavailable in code	Add documented API or simplify the Figma variant
State parity	Default, hover, focus, active, disabled, loading, error are represented	Figma shows only default/hover	Code remains accessible; document missing design state
Layout parity	Page gutters, grids, breakpoint behavior agree	Desktop frame treated as pixel-perfect universal layout	Encode responsive behavior in layout primitives
Content parity	Labels, empty copy, errors, and truncation have rules	Generic lorem or visually short sample content	Test realistic and localized strings
Accessibility parity	Semantic HTML and focus behavior exist beyond Figma	Clickable `div`, icon buttons without names	Use correct controls and accessible labels
Code Connect parity	Figma components reference real code components where supported	Figma library drifts into lookalikes	Link/maintain Code Connect references

Use this as a repeatable audit prompt in Claude Code:

```
Audit the supplied Figma frame against Semester.

1. Inspect the Figma hierarchy, variables, component instances, and layout.
2. Inspect the corresponding React route and all Semester components it uses.
3. Produce a table with:
   - Figma node or variable
   - Closest code token/component
   - Match status: match, partial, missing, obsolete
   - Severity: blocker, major, minor
   - Recommended resolution
4. Check typography, spacing, color, border, radius, elevation, responsive layout,
   interaction states, keyboard behavior, labels, empty/loading/error states,
```

```
   and component API consistency.
5. Do not make changes until the audit is complete.
6. Prefer mapping to existing Semester concepts over creating new tokens/components.
```

UI generation skill

Create `.claude/skills/build-ui/SKILL.md`.

Skills are self-contained folders whose required `SKILL.md` uses YAML frontmatter; Claude loads the skill when it becomes relevant and may load supporting references only as needed. Anthropic recommends that MCP-using skills reference tools with fully qualified `ServerName:tool_name` syntax to avoid ambiguity. [19] [21]

```
---
name: build-semester-ui
description: Build or revise React interfaces using the Semester design system. Use t
version: 1.0.0
---

# Build Semester UI

## Goal

Create production-ready React interfaces that are visibly and behaviorally native to

Prioritize:
1. Existing system reuse.
2. Semantic token correctness.
3. Accessible behavior.
4. Responsive resilience.
5. State completeness.
6. Minimal new abstractions.

## Inputs to inspect

Before writing code, inspect:

- Root `CLAUDE.md`.
- `docs/component-inventory.md`.
- Relevant files in `docs/patterns/`.
- Existing comparable screens and Storybook stories.
- `docs/figma-mapping.md` if the work includes Figma.
- The nearest package-level instructions, if present.

If a Figma URL, node, or selected Figma design is supplied:
- Inspect it using the installed Figma MCP tools.
- Use fully qualified Figma MCP tool names where required by the local MCP configura
- Identify its components, variants, variables, layout rules, and content hierarchy.
- Do not treat a static Figma frame as a complete interaction specification.

## Mandatory planning format

For a new screen, complex component, or substantial redesign, provide this concise pl
```

```
| Concern | Decision |
|---|---|
| Existing components | List components from `@semester/ui` to reuse |
| Pattern | Name the documented product pattern, or say no pattern exists |
| Tokens | List semantic token groups likely to be used |
| Responsive strategy | Explain narrow, intermediate, and wide behavior |
| States | List loading, empty, error, disabled, permissions, and success states |
| Accessibility | Identify semantic structure, focus, label, and keyboard requiremen
| System gap | State `none` or describe the smallest missing primitive/pattern |

Do not create code before identifying whether an existing component or composition wo

## Implementation rules

### Components

- Import from `@semester/ui`, `@semester/tokens`, and `@semester/icons`.
- Search the component inventory before creating any new primitive.
- Prefer composition of `Stack`, `Inline`, `Surface`, `Text`, `Button`, `Field`,
  and domain components over feature-local styling.
- Do not use generic `div` click handlers when a native interactive element is availa
- Do not create `Card`, `Panel`, `Widget`, or `Box` wrappers solely for decoration.
- Do not use emoji or arbitrary SVGs for UI icons.

### Tokens and style

- Use semantic tokens only in product UI.
- Do not use literal hex/rgb/hsl values, arbitrary pixel spacing, raw shadow strings
  arbitrary z-indexes, or undocumented animation values.
- Do not consume primitive tokens outside token/theme packages.
- Use layout primitives and semantic spacing roles rather than one-off margins.
- Use component tokens inside component implementations only.

### Responsive design

- Build narrow layouts first.
- Preserve primary actions and critical information on narrow screens.
- Use documented breakpoints or container queries.
- Tables may scroll horizontally only when no responsible card/list transformation ex
- Do not rely on hover as the only way to expose important actions.

### Accessibility

- Prefer semantic HTML and native inputs/buttons/links.
- Use visible focus styles from Semester tokens.
- Give every icon-only button an accessible name.
- Associate labels, descriptions, and validation errors with fields.
- Use `aria-live` only for relevant dynamic updates.
- Do not use color as the sole signal.
- Manage focus in dialogs, menus, comboboxes, and other composite widgets.
- Respect reduced-motion preferences.

### States

For each meaningful data-dependent surface, implement or deliberately address:
- Loading.
```

```
- Empty.
- Error and retry.
- Disabled or unavailable action.
- Success/confirmation where the action is consequential.
- Permission-restricted state when access can be denied.
- Long content, truncation, and localization expansion where relevant.

## Figma implementation protocol

When implementing from Figma:

1. Extract intent:
   - Page hierarchy.
   - Components and variants.
   - Semantic variables.
   - Layout constraints.
   - Visible states.
2. Map intent:
   - Figma variable → existing Semester semantic token.
   - Figma component → existing React component.
   - Figma layout → Semester layout primitive.
3. Resolve mismatches:
   - Reuse an existing code behavior when Figma is incomplete.
   - Do not introduce a raw value to force pixel parity.
   - Do not create a new token/component without identifying the reusable need.
4. Implement:
   - Maintain semantic HTML, keyboard interaction, state coverage, and responsive beh
5. Document:
   - Note any approved new mapping in `docs/figma-mapping.md`.

## New component threshold

A new component is justified only when all are true:

- No existing component or documented composition meets the need.
- The need appears in at least two product contexts or is clearly foundational.
- Its public API, states, accessibility contract, and responsive behavior are defined
- It can be documented with stories and tested.
- It does not encode feature-specific business logic.

If the threshold is not met, compose existing primitives locally.

## Required verification

Before declaring work complete:

1. Run typecheck, lint, and relevant tests.
2. Add or update Storybook stories for new visual variants and states.
3. Test keyboard navigation and visible focus.
4. Test narrow and wide viewports.
5. Check realistic, long, and empty content.
6. Confirm no token policy violations.
7. Summarize:
   - Components reused.
   - Tokens/patterns used.
   - States covered.
```

```
   - Accessibility behavior verified.
   - Any system gap or Figma mismatch found.

## Completion format

Return:

### Implemented
- Concise summary of UI work.

### System reuse
- Components, patterns, and tokens reused.

### State coverage
- Loading, empty, error, permissions, disabled, responsive behavior.

### Accessibility
- Semantics, labels, keyboard behavior, focus behavior.

### Validation
- Commands run and result.

### Follow-up
- Only list concrete system work still needed.
```

Design-sync audit skill

Create `.claude/skills/audit-design-sync/SKILL.md` to make "stop generating generic UI" enforceable rather than aspirational.

```
---
name: audit-semester-design-sync
description: Audit Figma designs and React implementation for Semester token, compone
version: 1.0.0
---

# Audit Semester Design Sync

## Goal

Compare a Figma design, React implementation, and Semester design-system contracts.
Find design drift before changing code.

## Required sources

Inspect:
- Root `CLAUDE.md`.
- `docs/figma-mapping.md`.
- `docs/component-inventory.md`.
- Relevant source files and Storybook stories.
- Figma hierarchy, components, variables, and layout through Figma MCP when a Figma

## Audit dimensions
```

```
Evaluate:

1. Token discipline
   - Raw values in product UI.
   - Incorrect semantic-role usage.
   - Missing Figma-to-code token mapping.
   - Theme and dark-mode compatibility, if supported.

2. Component parity
   - Figma components and variants mapped to stable React exports.
   - Duplicate or feature-local equivalents.
   - Props that no longer represent design intent.

3. State completeness
   - Default, hover, focus-visible, active, selected, disabled, loading, invalid,
     empty, error, success, and permission states where relevant.

4. Layout and responsive behavior
   - Spacing, grid, hierarchy, overflow, content width, mobile transformations,
     touch targets, and action priority.

5. Accessibility
   - Semantic elements, accessible names, labels, descriptions, errors,
     focus management, keyboard interaction, contrast, and reduced motion.

6. Content and hierarchy
   - Clear primary action.
   - Realistic content length.
   - Empty/error explanations and recovery actions.
   - Sentence case and Semester terminology.

## Required output

Produce these sections before recommending code changes:

### Executive assessment
One paragraph: alignment level and highest-risk issues.

### Findings

| Severity | Figma/code location | Category | Finding | Existing Semester solution |
|---|---|---|---|---|---|
| Blocker/Major/Minor | Node, file, component | Token/component/a11y/layout/state/co

### Missing system capabilities

| Candidate capability | Repeated need? | Existing composition viable? | Recommendat
|---|---:|---:|---|
| Name | Yes/No | Yes/No | Compose, document pattern, or propose component |

### Implementation order
List fixes in dependency order: blockers, shared primitives, screen composition, pol

## Rules

- Do not recommend pixel-perfect copying when it violates Semester tokens or accessi
```

```
- Do not recommend a new component until existing components/compositions are exhaus
- Classify raw token usage as a defect, not an aesthetic preference.
- Favor the smallest correction that strengthens the system rather than a one-off ex
- Never modify code during audit mode unless explicitly instructed after the report.
```

Component-creation skill

This prevents Claude from responding to every visual difference by creating a new “Card”, “Section”, or “FancyButton.”

```
---
name: create-semester-component
description: Propose and implement a reusable Semester component only when existing c
version: 1.0.0
---

# Create Semester Component

## Gate

Do not create a new component until you have:

1. Searched `@semester/ui`, the component inventory, and relevant patterns.
2. Identified at least two legitimate use cases or a foundational platform need.
3. Written a concise proposal covering:
   - Problem and users.
   - Existing alternatives considered.
   - Proposed public API.
   - Variants and states.
   - Accessibility and keyboard contract.
   - Responsive behavior.
   - Token dependencies.
   - Migration/adoption plan.
4. Obtained approval if the repository requires it.

## Implementation requirements

A new component must include:
- TypeScript implementation and public export.
- Semantic/component token mapping.
- Unit or interaction tests.
- Storybook stories for all variants and significant states.
- Keyboard/focus behavior.
- Loading, disabled, invalid, and empty/error behavior where applicable.
- Documentation in the component inventory.
- A production usage example or documented composition.

## API rules

- Prefer composition over booleans.
- Do not expose raw CSS or arbitrary style props as the primary API.
- Avoid feature-specific business logic.
- Do not add `className` escape hatches without a documented reason.
- Keep variant axes orthogonal: `variant`, `size`, `tone`, `state`.
```

```
- Preserve backwards compatibility or document migration.

## Completion report

Return:
- Why existing options were insufficient.
- API and accessibility contract.
- Stories/tests added.
- Tokens used.
- First adoption target.
- Any Figma mapping additions.
```

Make the system enforceable

Configuration alone will not stop generic UI. Add automated constraints.

Token lint rules

Implement ESLint/stylelint/custom AST checks to block:

```
// Reject in app and component packages
color: "#2F7CF6"
padding: "13px"
boxShadow: "0 9px 20px rgba(0,0,0,.13)"
zIndex: 9999
transition: "all 0.2s ease"
```

Require:

```
// Accept: semantic roles or typed token helpers
backgroundColor: tokens.color.action.primary.default
padding: tokens.space.inset.md
boxShadow: tokens.shadow.raised
zIndex: tokens.zIndex.popover
transitionDuration: tokens.motion.duration.fast
```

Component inventory metadata

Keep a machine-readable inventory so Claude can locate existing components without relying entirely on prose:

```
// packages/ui/src/component-manifest.ts
export const componentManifest = {
  Button: {
    status: "stable",
    import: "@semester/ui/Button",
    useFor: ["primary actions", "form submission", "destructive confirmation"],
    avoidFor: ["navigation to another URL"],
    variants: ["primary", "secondary", "subtle", "danger"],
    sizes: ["sm", "md", "lg"],
    accessibility: ["native button semantics", "loading state", "visible focus"],
```

```
  },

  TextField: {
    status: "stable",
    import: "@semester/ui/TextField",
    useFor: ["single-line user input"],
    avoidFor: ["long-form content", "search with suggestions"],
    compositions: ["Field", "FormField"],
    accessibility: ["label association", "describedby support", "invalid state"],
  },

  DataTable: {
    status: "beta",
    import: "@semester/ui/DataTable",
    useFor: ["dense multi-column datasets"],
    avoidFor: ["small mobile-first collections"],
    accessibility: ["column headers", "sort status", "keyboard navigation"],
  },
} as const;
```

CI gate

A practical CI sequence:

```
pnpm tokens:validate
pnpm tokens:build
pnpm lint
pnpm typecheck
pnpm test
pnpm test:visual
pnpm design-system:audit
```

`design-system:audit` should detect at least:

- Primitive token imports outside allowed token packages.
- Raw colors and hardcoded design values.
- Components lacking Storybook coverage.
- Components missing an accessibility note or test.
- Figma-mapped components absent from the React manifest.
- Deprecated component imports.
- Unapproved public API changes.
- New UI files that bypass `@semester/ui` despite available equivalents.

Immediate rollout order

1. Add the root `CLAUDE.md`.
2. Create `docs/component-inventory.md` and `docs/figma-mapping.md`.
3. Install the Figma plugin with `claude plugin install figma@claude-plugins-official`; authorize and validate it through `/mcp`.[16]
4. Establish the reference → semantic → component token pipeline.
5. Add `build-semester-ui` and `audit-semester-design-sync` skills.
6. Migrate the most reused primitives first: text, buttons, fields, surfaces, overlays, navigation.
7. Add token linting and visual regression checks to CI.
8. Use the audit skill on the next 3–5 production screens; turn repeat audit findings into components, patterns, or stricter lint rules.

The most important operational rule is simple: **Figma describes intent; Semester code defines the reusable accessible implementation.** Claude should use Figma MCP to understand a screen, but it should use Semester tokens, components, patterns, and validation rules to ship it.

⁂

Generate a complete Claude Code design system toolkit: production CLAUDE.md templates, token schemas, Figma MCP setup, and SKILL.md rules

Computer
Audit your React component library against Figma variables with Claude Code: automated token mapping, drift detection, and CI sync report
Computer
Show me the root CLAUDE.md file contents
Show the SKILL.md code for the audit skill
How do I configure the Figma MCP server

Below is a complete, copy-ready toolkit for **Semester**: a React design system governed through Claude Code, tokens, Figma MCP, skills, and CI drift reports.

Use the root `CLAUDE.md` for lasting project rules, and use focused skills for procedural work such as auditing design drift. Claude Code loads skills from `.claude/skills/<name>/SKILL.md`; the file's YAML frontmatter controls discovery and invocation, while the body contains the workflow.[31]

Toolkit layout

```
semester/
├── CLAUDE.md
├── .mcp.json
├── .claude/
│   ├── settings.json
│   └── skills/
│       ├── build-semester-ui/
│       │   └── SKILL.md
│       ├── audit-semester-design-sync/
│       │   └── SKILL.md
│       └── create-semester-component/
│           └── SKILL.md
├── docs/
│   ├── component-inventory.md
│   ├── figma-mapping.md
│   ├── accessibility.md
│   ├── design-principles.md
│   └── patterns/
│       ├── forms.md
│       ├── navigation.md
│       ├── data-views.md
│       └── feedback.md
├── packages/
│   ├── tokens/
│   │   ├── src/
│   │   │   ├── primitives.ts
│   │   │   ├── semantic.ts
│   │   │   ├── components.ts
│   │   │   ├── themes/
│   │   │   │   ├── light.ts
│   │   │   │   └── dark.ts
│   │   │   └── index.ts
│   │   └── package.json
│   ├── ui/
│   │   ├── src/
│   │   ├── stories/
│   │   └── component-manifest.ts
│   └── eslint-config/
├── scripts/
│   ├── audit-token-usage.ts
│   ├── build-figma-token-map.ts
│   ├── detect-figma-drift.ts
│   └── design-system-report.ts
└── reports/
    └── design-system/
```

The enforced value chain should be:

```
Reference values
  → semantic tokens
    → component tokens
```

```
      → React components
        → product screens
```

Product code should almost never consume a raw palette value such as `blue.600`. It should consume a role such as `color.action.primary.default`.

Root CLAUDE.md

Save this exact file at `CLAUDE.md` in the repository root.

```
# Semester Design System

## Purpose

Semester is the canonical product design system for this repository.

It defines:
- Design tokens and theme mappings.
- Accessible React primitives and components.
- Layout, typography, responsive, interaction, and content conventions.
- Figma-to-code mappings.
- Storybook, test, lint, visual-regression, and CI quality requirements.

Build interfaces that are deliberate, calm, accessible, information-rich, and native
to Semester. Do not generate generic dashboard UI, decorative card grids, arbitrary
pills, gradient-heavy surfaces, or one-off visual values.

## Required reading

Before doing non-trivial UI work, read:

- `docs/design-principles.md`
- `docs/accessibility.md`
- `docs/component-inventory.md`
- The relevant document in `docs/patterns/`
- Existing Storybook stories for components being changed
- `docs/figma-mapping.md` when work references Figma

## Source-of-truth policy

- Code is the source of truth for component behavior, accessibility, public APIs,
  responsive behavior, and production token values.
- Figma is the source of truth for approved visual intent and design exploration.
- `packages/tokens/src/` is the source of truth for token definitions.
- Generated token artifacts must never be edited manually.
- Figma variables must map to semantic code tokens before they are used in product UI
- A Figma component is not automatically a React component.

## Non-negotiable rules

### Reuse system assets first

- Search `@semester/ui`, Storybook, `component-manifest.ts`, and the component
  inventory before writing UI.
```

```
- Prefer composing existing primitives over creating feature-local components.
- Do not create a new primitive, component, token, layout abstraction, or visual
  variant until existing options have been evaluated.
- Do not create generic names such as `Card`, `Box`, `Panel`, `Widget`, or
  `FancyButton` for feature-local wrappers.

### Use tokens only

- Use semantic tokens in React components and product application code.
- Do not import raw/reference tokens outside `packages/tokens`.
- Do not add arbitrary hex, rgb, hsl, oklch, spacing, radius, border, shadow,
  font, z-index, opacity, duration, or easing values in UI code.
- Do not use arbitrary Tailwind values such as `p-[13px]`, `bg-[#...]`,
  `z-[^3_9999]`, or `shadow-[...]`.
- Component-specific tokens are allowed only inside their owning component package.
- Do not edit generated CSS, JSON, TypeScript, or Figma-export token artifacts.

### Build accessible interfaces

- Meet WCAG 2.2 AA unless a documented exception exists.
- Prefer native HTML semantics and native controls.
- Use visible focus styles for every keyboard-reachable interactive element.
- Every icon-only control must have an accessible name.
- Associate field labels, descriptions, validation messages, and errors with fields.
- Do not communicate meaning through color alone.
- Dialogs, menus, popovers, comboboxes, and similar widgets must manage focus correc
- Restore focus after closing overlays.
- Support Escape to dismiss transient overlays when safe.
- Respect `prefers-reduced-motion`.
- Maintain a practical minimum 44 by 44 CSS pixel touch target.

### Design responsive interfaces intentionally

- Start with narrow layouts and enhance for larger viewports.
- Use Semester layout primitives, documented breakpoints, or approved container queri
- Do not hide critical actions or information on smaller screens.
- Do not rely exclusively on hover for essential actions.
- Handle content overflow, localization expansion, long labels, and long lists.
- State the narrow, intermediate, and wide behavior for complex layouts.

### Maintain product quality

- Establish clear hierarchy and one primary action per decision scope.
- Use sentence case for interface text unless a documented product term requires othe
- Use `@semester/icons`; never use emoji as product interface icons.
- Include meaningful loading, empty, error, disabled, success, and permission-restric
  states when relevant.
- Avoid unnecessary decoration. Prioritize useful content and scannability.
- Do not introduce UI dependencies without explicit approval.
- Do not weaken lint, typecheck, test, accessibility, or visual-regression checks.

## Required implementation workflow

For a new screen, new flow, or substantial UI change:

1. Inspect the closest existing implementation and relevant Storybook stories.
```

```
2. Identify the Semester components, layout primitives, patterns, and semantic tokens
3. For non-trivial work, provide a concise plan before editing:
   - Components and patterns to reuse.
   - Required states.
   - Responsive behavior.
   - Accessibility considerations.
   - Proposed design-system gap, if any.
4. Implement using `@semester/ui`, `@semester/tokens`, and `@semester/icons`.
5. Check keyboard operation, focus visibility, semantic structure, labels, and errors
6. Verify narrow and wide viewport behavior.
7. Run relevant validation commands.
8. Summarize reused system assets and any unresolved system gap.

## Component API rules

- Use PascalCase for component names and camelCase for props.
- Use `variant`, `size`, and `tone` only for documented, stable design choices.
- Prefer composition and slots over large boolean-prop APIs.
- Use explicit booleans such as `isLoading`, `isInvalid`, and `isDisabled`.
- Defaults must be safe, accessible, and visually standard.
- Do not expose arbitrary style props as a primary API.
- Use `data-slot` only for stable, documented component subparts.
- New components require TypeScript types, tests, Storybook coverage, documentation,
  and accessibility notes.

## Figma protocol

When a task includes a Figma URL, node ID, selected layer, variable, component, or fi

1. Use Figma MCP to inspect the design hierarchy, layout, variables, components,
   component properties, and visible states.
2. Treat Figma as design intent, not literal HTML/CSS to duplicate.
3. Map Figma variables to existing Semester semantic tokens before adding tokens.
4. Map Figma components to existing React components before creating components.
5. Preserve accessibility, semantics, responsive rules, error handling, and interacti
   behavior even when Figma does not show them.
6. Record important unmapped variables/components in `docs/figma-mapping.md`.
7. Do not create a Figma-driven exception that bypasses Semester.

## Completion requirements

A UI task is incomplete until:

- It uses approved Semester components and semantic tokens.
- It handles relevant loading, empty, error, disabled, success, and restricted states
- It works at narrow and wide viewport sizes.
- It is keyboard-operable with visible focus.
- It uses semantic HTML and accessible names.
- Typecheck, lint, relevant tests, and visual checks pass.
- Documentation is updated when the design-system source of truth changes.

## Commands

Inspect `package.json` before running commands. Use available scripts rather than gue

Common validation commands:
```

```bash
pnpm tokens:validate
pnpm tokens:build
pnpm lint
pnpm typecheck
pnpm test
pnpm test:visual
pnpm storybook
pnpm design-system:audit
pnpm design-system:report
```

Reference documents

@docs/design-principles.md
@docs/accessibility.md
@docs/component-inventory.md
@docs/figma-mapping.md

````
`CLAUDE.md` should contain lasting facts and guardrails. Keep procedural checks—such

## Audit SKILL.md

Create this directory:

```bash
mkdir -p .claude/skills/audit-semester-design-sync
```
````

Then save the following as `.claude/skills/audit-semester-design-sync/SKILL.md`.

```
---
name: audit-semester-design-sync
description: Audit a React screen, component library, or Figma implementation for Ser
when_to_use: Use for Figma-to-code reviews, design QA, React library audits, token ma
argument-hint: "[figma-url-or-node] [route-or-component-path]"
allowed-tools:
  - Read
  - Glob
  - Grep
  - Bash
---

# Audit Semester Design Sync

## Objective

Audit the alignment between:

1. The requested or supplied Figma design.
2. The React implementation.
3. Semester token definitions.
4. Semester component contracts.
```

```
5. Accessibility and responsive behavior requirements.

Find system drift before recommending changes. Do not modify source files unless the
explicitly asks for implementation after reviewing the audit.

## Required inputs

Read before auditing:

- Root `CLAUDE.md`.
- `docs/design-principles.md`.
- `docs/accessibility.md`.
- `docs/component-inventory.md`.
- `docs/figma-mapping.md`.
- Relevant files in `docs/patterns/`.
- Relevant source files, tests, and Storybook stories.
- `packages/tokens/src/semantic.ts`.
- `packages/tokens/src/components.ts`.
- `packages/ui/src/component-manifest.ts`, if present.

If a Figma reference is supplied:

1. Use Figma MCP to inspect the selected node, child hierarchy, components, componen
   properties, variables, layout data, text, and visible states.
2. Capture design intent, not merely dimensions or colors.
3. Identify which Figma variables and components should map to Semester tokens and Re
4. Do not treat a static desktop frame as a complete responsive or behavioral specif

## Audit process

### Step 1: Establish scope

State the audit target:

- Figma file, page, frame, node, or component.
- React route, feature, package, or component.
- Relevant Semester pattern.
- Existing component APIs and token namespaces.
- Whether the audit is visual-only, implementation-only, or Figma-to-code.

If the target cannot be determined, audit the available source and clearly label any

### Step 2: Inspect semantic-token discipline

Check for:

- Raw CSS colors, arbitrary design values, and raw palette imports.
- Incorrect semantic role selection.
- One-off CSS variables that duplicate Semester tokens.
- Hardcoded shadows, radii, z-index values, typography, motion values, and spacing.
- Figma variables that have no code token mapping.
- Code semantic tokens that have no corresponding Figma variable where parity is requ
- Themes that are incompatible with dark mode or high-contrast modes, if supported.

Classify each issue:
```

```
- `blocker`: breaks accessibility, causes a critical brand/system contradiction, or p
- `major`: creates repeatable drift, duplication, or an inconsistent user interactio
- `minor`: cosmetic or documentation-level mismatch that does not meaningfully break

### Step 3: Inspect component parity

For every visible Figma component or code component:

- Identify the closest React export from `@semester/ui`.
- Identify the closest Figma library component.
- Compare variants, sizes, tones, states, slots, and content rules.
- Detect feature-local duplicate components.
- Identify APIs that expose implementation detail instead of design intent.
- Verify whether the React component has Storybook coverage and a documented contrac

Do not recommend a new component until you confirm that:
- An existing component is insufficient.
- A documented composition is insufficient.
- The need is reusable across at least two product contexts or is foundational.

### Step 4: Inspect interaction and state coverage

For relevant surfaces, check:

- Default.
- Hover.
- Focus-visible.
- Active/pressed.
- Selected.
- Disabled.
- Read-only.
- Loading.
- Empty.
- Error and retry.
- Invalid validation.
- Success/confirmation.
- Permission-restricted state.
- Long-content and localization behavior.
- Network/offline/degraded behavior when relevant.

Treat a missing visible Figma state as incomplete design evidence, not permission to
state from accessible product code.

### Step 5: Inspect responsive and layout behavior

Check:

- Page gutters and max-widths.
- Layout primitives and composition.
- Stack/grid transformations.
- Overflow behavior.
- Action priority on narrow screens.
- Tables and data density.
- Content truncation and wrapping.
- Touch-target size.
- Fixed/sticky layer behavior.
```

```
- Z-index use and overlay layering.
- Container queries or breakpoint consistency.

### Step 6: Inspect accessibility

Check:

- Correct semantic landmarks and heading hierarchy.
- Native button, link, input, select, and form semantics.
- Accessible names for icon-only controls.
- Label, description, validation, and error associations.
- Focus order and visible focus styles.
- Dialog, menu, tooltip, popover, and combobox focus behavior.
- Keyboard interaction.
- Color contrast and non-color state signals.
- Motion reduction.
- Live-region use only where needed.

### Step 7: Produce report before fixes

Do not edit code during audit mode.

Use the exact report format below.

## Required output format

# Semester design-sync audit

## Executive assessment

Write one concise paragraph identifying:
- Overall alignment: strong, moderate, weak, or critical drift.
- The highest-risk mismatch.
- Whether the main issue is token, component, responsive, state, accessibility, or Fi

## Scope

| Area | Audited target | Evidence reviewed |
|---|---|---|
| Figma | Value or `not supplied` | Nodes, variables, components, states |
| Code | Value or `not supplied` | Files, stories, tests |
| Semester | Relevant packages/docs | Tokens, components, patterns |

## Findings

| Severity | Location | Category | Finding | Existing Semester solution | Recommended
|---|---|---|---|---|---|
| Blocker/Major/Minor | Figma node, file, component, selector, or token | Token/Compo

## Token mapping

| Figma collection/variable | Expected Semester token | Actual implementation | Statu
|---|---|---|---|---|
| Value | Value | Value | Match/Partial/Missing/Obsolete | Specific resolution |

## Component mapping
```

```
| Figma component | React component | Variant/state parity | Status | Action |
|---|---|---|---|---|
| Value | Value | Match/Partial/Missing | Match/Partial/Missing/Obsolete | Specific r

## State coverage

| Surface | Present | Missing or unclear | Required action |
|---|---|---|---|
| Value | List covered states | List missing states | Specific resolution |

## Accessibility review

| Requirement | Status | Evidence | Required action |
|---|---|---|---|
| Semantics | Pass/Partial/Fail | Specific element or component | Specific resolutio
| Keyboard | Pass/Partial/Fail | Specific interaction | Specific resolution |
| Focus | Pass/Partial/Fail | Specific state | Specific resolution |
| Labels/errors | Pass/Partial/Fail | Specific field/control | Specific resolution |
| Contrast/non-color cues | Pass/Partial/Fail | Specific token/state | Specific reso

## System gaps

| Candidate capability | Reusable need | Existing composition works | Recommendation
|---|---:|---:|---|
| Value | Yes/No | Yes/No | Compose, document pattern, propose token, or propose comp

## Recommended implementation order

1. List blockers first.
2. Then shared token/component issues.
3. Then route-level composition issues.
4. Then visual polish and documentation.

## Rules

- Prefer an existing Semester component over a Figma-specific clone.
- Prefer a semantic token over a visual-value token.
- Prefer a documented composition over a new component.
- Do not recommend raw values to chase pixel-perfect parity.
- Do not use Figma omissions to excuse missing accessibility or state behavior.
- Report evidence, not subjective aesthetic preference.
- Keep proposed fixes small, composable, and compatible with the design system.
```

Claude Code discovers project skills from `.claude/skills/<skill-name>/SKILL.md`. A `description` tells Claude when the skill applies; users can also invoke it directly as `/audit-semester-design-sync`.[31]

Semantic token schema

Use **reference**, **semantic**, and **component** layers. The source examples below use TypeScript, but the naming and alias rules apply equally to Style Dictionary, Tokens Studio JSON, CSS custom properties, or another generation pipeline.

Token policy

Layer	Example	May be consumed by	Why it exists
Reference	`ref.color.brand.600`	Token/theme definitions only	Raw palette, sizing, type, and motion scales
Semantic	`color.action.primary.default`	All components and product UI	Stable meaning independent of raw values
Component	`button.primary.background.default`	Owning component only	Internal component contract
Local	`--feature-x-offset`	Rarely and temporarily	Must not replicate a global semantic role

`packages/tokens/src/primitives.ts`

```
export const ref = {
  color: {
    neutral: {
      0: "#FFFFFF",
      25: "#FCFCFD",
      50: "#F8F9FA",
      100: "#F1F3F5",
      200: "#E9ECEF",
      300: "#DEE2E6",
      400: "#CED4DA",
      500: "#ADB5BD",
      600: "#6C757D",
      700: "#495057",
      800: "#343A40",
      900: "#212529",
      950: "#101214",
    },
    brand: {
      50: "#EEF4FF",
      100: "#D9E7FF",
      200: "#BBD7FF",
      300: "#8CC0FF",
      400: "#55A0FF",
      500: "#2F7CF6",
      600: "#1E63D8",
      700: "#194FAC",
      800: "#1B438C",
      900: "#1B396F",
    },
    red: {
      50: "#FEF2F2",
      100: "#FEE2E2",
      200: "#FECACA",
      500: "#EF4444",
      600: "#DC2626",
      700: "#B91C1C",
      900: "#7F1D1D",
    },
```

```
    amber: {
      50: "#FFFBEB",
      100: "#FEF3C7",
      500: "#F59E0B",
      600: "#D97706",
      700: "#B45309",
      900: "#78350F",
    },
    green: {
      50: "#F0FDF4",
      100: "#DCFCE7",
      500: "#22C55E",
      600: "#16A34A",
      700: "#15803D",
      900: "#14532D",
    },
  },

  space: {
    0: "0",
    px: "1px",
    0_5: "0.125rem",
    1: "0.25rem",
    1_5: "0.375rem",
    2: "0.5rem",
    3: "0.75rem",
    4: "1rem",
    5: "1.25rem",
    6: "1.5rem",
    8: "2rem",
    10: "2.5rem",
    12: "3rem",
    16: "4rem",
    20: "5rem",
    24: "6rem",
  },

  radius: {
    none: "0",
    xs: "0.125rem",
    sm: "0.25rem",
    md: "0.375rem",
    lg: "0.5rem",
    xl: "0.75rem",
    "2xl": "1rem",
    full: "9999px",
  },

  font: {
    family: {
      sans: "Inter, ui-sans-serif, system-ui, sans-serif",
      mono: '"JetBrains Mono", ui-monospace, SFMono-Regular, monospace',
    },
    size: {
      xs: "0.75rem",
      sm: "0.875rem",
```

```
      md: "1rem",
      lg: "1.125rem",
      xl: "1.25rem",
      "2xl": "1.5rem",
      "3xl": "1.875rem",
      "4xl": "2.25rem",
    },
    weight: {
      regular: 400,
      medium: 500,
      semibold: 600,
      bold: 700,
    },
    lineHeight: {
      tight: 1.15,
      snug: 1.3,
      normal: 1.5,
      relaxed: 1.65,
    },
  },

  duration: {
    instant: "0ms",
    fast: "120ms",
    normal: "180ms",
    slow: "280ms",
  },

  easing: {
    standard: "cubic-bezier(0.2, 0, 0, 1)",
    enter: "cubic-bezier(0, 0, 0.2, 1)",
    exit: "cubic-bezier(0.4, 0, 1, 1)",
  },

  zIndex: {
    base: 0,
    sticky: 100,
    dropdown: 200,
    popover: 300,
    overlay: 400,
    modal: 500,
    toast: 600,
    tooltip: 700,
  },
} as const;
```

packages/tokens/src/semantic.ts

```
export const semantic = {
  color: {
    text: {
      primary: "{ref.color.neutral.950}",
      secondary: "{ref.color.neutral.700}",
      muted: "{ref.color.neutral.600}",
      disabled: "{ref.color.neutral.500}",
```

```
    inverse: "{ref.color.neutral.0}",
    link: "{ref.color.brand.700}",
    linkHover: "{ref.color.brand.800}",
    danger: "{ref.color.red.700}",
    success: "{ref.color.green.700}",
    warning: "{ref.color.amber.700}",
  },

  icon: {
    primary: "{color.text.primary}",
    secondary: "{color.text.secondary}",
    muted: "{color.text.muted}",
    disabled: "{color.text.disabled}",
    inverse: "{color.text.inverse}",
    brand: "{color.action.primary.default}",
    danger: "{color.status.danger.strong}",
  },

  surface: {
    canvas: "{ref.color.neutral.25}",
    base: "{ref.color.neutral.0}",
    raised: "{ref.color.neutral.0}",
    sunken: "{ref.color.neutral.50}",
    overlay: "{ref.color.neutral.0}",
    inverse: "{ref.color.neutral.950}",
    selected: "{ref.color.brand.50}",
    disabled: "{ref.color.neutral.100}",
  },

  border: {
    subtle: "{ref.color.neutral.200}",
    default: "{ref.color.neutral.300}",
    strong: "{ref.color.neutral.400}",
    focus: "{ref.color.brand.500}",
    disabled: "{ref.color.neutral.200}",
    danger: "{ref.color.red.500}",
  },

  action: {
    primary: {
      default: "{ref.color.brand.600}",
      hover: "{ref.color.brand.700}",
      active: "{ref.color.brand.800}",
      foreground: "{ref.color.neutral.0}",
      disabled: "{ref.color.neutral.300}",
      disabledForeground: "{ref.color.neutral.600}",
    },

    secondary: {
      default: "{color.surface.base}",
      hover: "{ref.color.neutral.50}",
      active: "{ref.color.neutral.100}",
      foreground: "{color.text.primary}",
      border: "{color.border.default}",
    },
```

```
      subtle: {
        default: "transparent",
        hover: "{ref.color.neutral.100}",
        active: "{ref.color.neutral.200}",
        foreground: "{color.text.primary}",
      },

      danger: {
        default: "{ref.color.red.600}",
        hover: "{ref.color.red.700}",
        active: "{ref.color.red.900}",
        foreground: "{ref.color.neutral.0}",
      },
    },

    status: {
      info: {
        subtle: "{ref.color.brand.50}",
        default: "{ref.color.brand.600}",
        strong: "{ref.color.brand.700}",
        text: "{ref.color.brand.800}",
      },
      success: {
        subtle: "{ref.color.green.50}",
        default: "{ref.color.green.600}",
        strong: "{ref.color.green.700}",
        text: "{ref.color.green.900}",
      },
      warning: {
        subtle: "{ref.color.amber.50}",
        default: "{ref.color.amber.600}",
        strong: "{ref.color.amber.700}",
        text: "{ref.color.amber.900}",
      },
      danger: {
        subtle: "{ref.color.red.50}",
        default: "{ref.color.red.600}",
        strong: "{ref.color.red.700}",
        text: "{ref.color.red.900}",
      },
    },

    focus: {
      ring: "{ref.color.brand.500}",
      ringOffset: "{color.surface.base}",
    },
  },

  space: {
    inline: {
      xs: "{ref.space.1}",
      sm: "{ref.space.2}",
      md: "{ref.space.3}",
      lg: "{ref.space.4}",
    },
    stack: {
```

```
      xs: "{ref.space.2}",
      sm: "{ref.space.3}",
      md: "{ref.space.4}",
      lg: "{ref.space.6}",
      xl: "{ref.space.8}",
      "2xl": "{ref.space.12}",
    },
    inset: {
      xs: "{ref.space.2}",
      sm: "{ref.space.3}",
      md: "{ref.space.4}",
      lg: "{ref.space.6}",
    },
    page: {
      mobile: "{ref.space.4}",
      tablet: "{ref.space.6}",
      desktop: "{ref.space.8}",
    },
  },

  typography: {
    body: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.sm}",
        fontWeight: "{ref.font.weight.regular}",
        lineHeight: "{ref.font.lineHeight.normal}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.md}",
        fontWeight: "{ref.font.weight.regular}",
        lineHeight: "{ref.font.lineHeight.normal}",
      },
    },

    label: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.sm}",
        fontWeight: "{ref.font.weight.medium}",
        lineHeight: "{ref.font.lineHeight.snug}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.md}",
        fontWeight: "{ref.font.weight.medium}",
        lineHeight: "{ref.font.lineHeight.snug}",
      },
    },

    heading: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.lg}",
        fontWeight: "{ref.font.weight.semibold}",
```

```
        lineHeight: "{ref.font.lineHeight.tight}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.xl}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
      },
      lg: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.2xl}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
      },
    },
  },

  size: {
    control: {
      sm: "2rem",
      md: "2.5rem",
      lg: "3rem",
    },
    icon: {
      xs: "0.875rem",
      sm: "1rem",
      md: "1.25rem",
      lg: "1.5rem",
    },
    target: {
      minimum: "2.75rem",
    },
    content: {
      readable: "42rem",
      default: "72rem",
      wide: "90rem",
    },
  },

  radius: {
    control: "{ref.radius.md}",
    surface: "{ref.radius.lg}",
    overlay: "{ref.radius.xl}",
    pill: "{ref.radius.full}",
  },

  border: {
    default: "1px",
    strong: "2px",
  },

  shadow: {
    raised: "0 1px 2px rgb(16 18 20 / 0.06), 0 1px 3px rgb(16 18 20 / 0.10)",
    overlay: "0 12px 32px rgb(16 18 20 / 0.16), 0 2px 8px rgb(16 18 20 / 0.08)",
  },
```

```
  motion: {
    duration: {
      instant: "{ref.duration.instant}",
      fast: "{ref.duration.fast}",
      normal: "{ref.duration.normal}",
      slow: "{ref.duration.slow}",
    },
    easing: {
      standard: "{ref.easing.standard}",
      enter: "{ref.easing.enter}",
      exit: "{ref.easing.exit}",
    },
  },

  zIndex: {
    base: "{ref.zIndex.base}",
    sticky: "{ref.zIndex.sticky}",
    dropdown: "{ref.zIndex.dropdown}",
    popover: "{ref.zIndex.popover}",
    overlay: "{ref.zIndex.overlay}",
    modal: "{ref.zIndex.modal}",
    toast: "{ref.zIndex.toast}",
    tooltip: "{ref.zIndex.tooltip}",
  },
} as const;
```

packages/tokens/src/components.ts

```
export const component = {
  button: {
    primary: {
      background: {
        default: "{color.action.primary.default}",
        hover: "{color.action.primary.hover}",
        active: "{color.action.primary.active}",
        disabled: "{color.action.primary.disabled}",
      },
      foreground: {
        default: "{color.action.primary.foreground}",
        disabled: "{color.action.primary.disabledForeground}",
      },
      focusRing: "{color.focus.ring}",
      height: {
        sm: "{size.control.sm}",
        md: "{size.control.md}",
        lg: "{size.control.lg}",
      },
      radius: "{radius.control}",
      gap: "{space.inline.sm}",
    },
  },

  field: {
    background: {
      default: "{color.surface.base}",
```

```
      disabled: "{color.surface.disabled}",
    },
    border: {
      default: "{color.border.default}",
      hover: "{color.border.strong}",
      focus: "{color.border.focus}",
      invalid: "{color.border.danger}",
      disabled: "{color.border.disabled}",
    },
    text: {
      value: "{color.text.primary}",
      placeholder: "{color.text.muted}",
      disabled: "{color.text.disabled}",
    },
    height: {
      sm: "{size.control.sm}",
      md: "{size.control.md}",
      lg: "{size.control.lg}",
    },
    radius: "{radius.control}",
  },

  dialog: {
    background: "{color.surface.overlay}",
    border: "{color.border.subtle}",
    radius: "{radius.overlay}",
    shadow: "{shadow.overlay}",
    overlay: "rgb(16 18 20 / 0.48)",
  },
} as const;
```

Figma MCP configuration

Figma offers remote and desktop MCP modes. Figma recommends the remote MCP server for most users because it has the broadest feature set. Its official Claude Code plugin configures the server and includes common workflow skills. [32]

Preferred: install Figma's Claude Code plugin

Run this in your terminal:

```
claude plugin install figma@claude-plugins-official
```

Then:

1. Restart Claude Code.
2. Run `/plugin`.
3. Open the **Installed** tab.
4. Select `figma`.
5. Start authorization and approve access in the browser.

6. Return to Claude Code.
7. Run `/mcp` to confirm that Figma is connected.

Figma MCP can provide structured design context from Figma files—such as components, variables, layout data, FigJam content, and Code Connect mappings—so Claude can map a selected design to your system instead of inferring it from screenshots. [32]

Manual remote configuration

Claude Code supports remote HTTP MCP servers with:

```
claude mcp add --transport http <name> <url>
```

For a shared team configuration, use `--scope project`, which writes project configuration; local scope is the default, while user scope works across your projects. [33]

For Figma's remote MCP endpoint:

```
claude mcp add --scope project --transport http figma https://mcp.figma.com/mcp
```

Restart Claude Code, then authenticate with `/mcp` if requested:

```
/mcp
```

Verify from your shell:

```
claude mcp list
claude mcp get figma
```

The status should move from authentication-required to connected after authorization. Claude Code's MCP status panel and CLI can show connected, needs-authentication, pending approval, and failed-connection states. [33]

`.mcp.json` project configuration

If you want the remote server declared in the repository, create `.mcp.json` at the project root:

```
{
  "mcpServers": {
    "figma": {
      "type": "http",
      "url": "https://mcp.figma.com/mcp"
    }
  }
}
```

The `type` field is required for URL-based JSON configuration. Claude Code treats a server without a declared `type` as a local stdio server, so an entry containing a `url` but no `type: "http"` is invalid.[33]

When teammates clone the repository:

1. They run `claude` from the project.
2. They accept the workspace trust prompt.
3. They review and approve the project MCP configuration.
4. They authenticate their own Figma account via `/mcp`.

Do not place static Figma access tokens, OAuth tokens, or secrets in `.mcp.json`. Use the OAuth authentication flow that Claude Code invokes from `/mcp`.

Desktop Figma MCP server

Use desktop MCP when your organization specifically requires it, or when you need the local Desktop workflow. Figma otherwise recommends its remote server.[32]

1. Install and open the Figma desktop app.
2. Open a Figma Design file.
3. Enter Dev Mode with no canvas layer selected.
4. Enable the MCP server in the right sidebar.
5. Copy the server URL offered by Figma.
6. Configure Claude Code:

```
claude mcp add --scope local --transport http figma-desktop http://127.0.0.1:3845/mcp
```

7. Restart Claude Code.
8. Run `/mcp` and confirm that `figma-desktop` appears.

Figma's documented local endpoint is `http://127.0.0.1:3845/mcp`.[32]

Recommended `.claude/settings.json`

This does not force secrets or automatic trust. It simply records repository-wide behavior.

```
{
  "permissions": {
    "allow": [
      "Read",
      "Glob",
      "Grep"
    ]
  }
}
```

Keep automated Figma writes disabled by default. Reading Figma variables and inspecting nodes is low-risk; changing an approved design file is a collaborative, state-changing operation that should require an explicit task and review.

Figma-to-code mapping

Create `docs/figma-mapping.md`:

```
# Semester Figma-to-Code Mapping

## Ownership

- Code owns implementation behavior, accessibility, responsive behavior, and public A
- Figma owns approved visual design intent and exploration.
- `packages/tokens/src/` owns production design-token values.
- Figma variables should represent exported/synchronized semantic token contracts.
- Figma component variants must map to documented React component props and states.

## Variable mapping

| Figma collection | Figma variable | Semester token | Status | Notes |
|---|---|---|---|---|
| Semester / Semantic / Color | Text / Primary | `color.text.primary` | Stable | Prir
| Semester / Semantic / Color | Surface / Base | `color.surface.base` | Stable | Defa
| Semester / Semantic / Color | Action / Primary / Default | `color.action.primary.de
| Semester / Semantic / Color | Border / Focus | `color.border.focus` | Stable | Focu
| Semester / Semantic / Space | Stack / MD | `space.stack.md` | Stable | Default ver
| Semester / Component / Button | Primary / Hover | `button.primary.background.hover

## Component mapping

| Figma component | React export | Figma variants → React props | Status | Notes |
|---|---|---|---|---|
| Button | `@semester/ui/Button` | `Style` → `variant`; `Size` → `size`; `Loading` →
| Text field | `@semester/ui/TextField` | `Size` → `size`; `State` → `isInvalid`, `is
| Dialog | `@semester/ui/Dialog` | `Size` → `size` | Stable | Code owns focus trap ar
| Data table | `@semester/ui/DataTable` | `Density` → `density` | Beta | Do not use
| Metric card | No direct export | N/A | Deprecated | Compose `Surface`, `Stack`, `Te

## Mapping rules

- Figma variables must map to semantic code tokens before production use.
- Figma local styles must not become code tokens without design-system review.
- Figma components are not assumed to be React components.
- Figma variants map only to documented React props.
- Missing implementation states in Figma do not remove code accessibility requiremen
- New variables or components require a documented code mapping and system review.
- Record unresolved mismatches as an ADR or tracked issue.
```

Automated token mapping and drift detection

The practical objective is a CI report that tells you whether Figma and code agree on the system contract.

Canonical mapping manifest

Create `packages/tokens/src/figma-map.ts`:

```
export type FigmaTokenMapping = {
  figmaCollection: string;
  figmaVariable: string;
  semesterToken: string;
  required: boolean;
  componentScope?: string;
};

export const figmaTokenMap: FigmaTokenMapping[] = [
  {
    figmaCollection: "Semester / Semantic / Color",
    figmaVariable: "Text / Primary",
    semesterToken: "color.text.primary",
    required: true,
  },
  {
    figmaCollection: "Semester / Semantic / Color",
    figmaVariable: "Surface / Base",
    semesterToken: "color.surface.base",
    required: true,
  },
  {
    figmaCollection: "Semester / Semantic / Color",
    figmaVariable: "Action / Primary / Default",
    semesterToken: "color.action.primary.default",
    required: true,
  },
  {
    figmaCollection: "Semester / Semantic / Space",
    figmaVariable: "Stack / MD",
    semesterToken: "space.stack.md",
    required: true,
  },
  {
    figmaCollection: "Semester / Component / Button",
    figmaVariable: "Primary / Background / Hover",
    semesterToken: "button.primary.background.hover",
    required: false,
    componentScope: "Button",
  },
];
```

Flattened-token utility

Create `scripts/flatten-tokens.ts`:

```
export type FlatTokenMap = Record<string, unknown>;

export function flattenTokens(
  value: Record<string, unknown>,
  prefix = "",
): FlatTokenMap {
  return Object.entries(value).reduce<FlatTokenMap>((result, [key, child]) => {
    const path = prefix ? `${prefix}.${key}` : key;

    if (
      child !== null &&
      typeof child === "object" &&
      !Array.isArray(child)
    ) {
      Object.assign(
        result,
        flattenTokens(child as Record<string, unknown>, path),
      );
      return result;
    }

    result[path] = child;
    return result;
  }, {});
}
```

Token-map report generator

Create `scripts/build-figma-token-map.ts`:

```
import { mkdir, writeFile } from "node:fs/promises";
import { component } from "../packages/tokens/src/components";
import { semantic } from "../packages/tokens/src/semantic";
import { figmaTokenMap } from "../packages/tokens/src/figma-map";
import { flattenTokens } from "./flatten-tokens";

type ReportStatus = "match" | "missing-code-token";

const semanticTokens = flattenTokens(semantic);
const componentTokens = flattenTokens(component);
const allTokens = {
  ...semanticTokens,
  ...componentTokens,
};

const rows = figmaTokenMap.map((mapping) => {
  const exists = Object.hasOwn(allTokens, mapping.semesterToken);

  return {
    ...mapping,
```

```
    status: (exists ? "match" : "missing-code-token") as ReportStatus,
    resolvedValue: exists ? allTokens[mapping.semesterToken] : null,
  };
});

const summary = {
  generatedAt: new Date().toISOString(),
  total: rows.length,
  matched: rows.filter((row) => row.status === "match").length,
  missingCodeTokens: rows.filter((row) => row.status === "missing-code-token")
    .length,
};

await mkdir("reports/design-system", { recursive: true });

await writeFile(
  "reports/design-system/figma-token-map.json",
  JSON.stringify({ summary, rows }, null, 2),
);

const markdown = [
  "# Semester Figma Token Map",
  "",
  `Generated: ${summary.generatedAt}`,
  "",
  `- Total mappings: ${summary.total}`,
  `- Matched: ${summary.matched}`,
  `- Missing code tokens: ${summary.missingCodeTokens}`,
  "",
  "| Figma collection | Figma variable | Semester token | Status | Resolved value |"
  "|---|---|---|---|---|",
  ...rows.map((row) => {
    const value = row.resolvedValue ?? "—";
    return `| ${row.figmaCollection} | ${row.figmaVariable} | \`${row.semesterToken}`
  }),
  "",
].join("\n");

await writeFile(
  "reports/design-system/figma-token-map.md",
  markdown,
);

if (summary.missingCodeTokens > 0) {
  process.exitCode = 1;
}
```

Raw-value and token-policy scanner

Create `scripts/audit-token-usage.ts`:

```
import { glob } from "glob";
import { readFile } from "node:fs/promises";

const files = await glob([
```

```
  "apps/**/*.{ts,tsx,js,jsx,css,scss}",
  "packages/ui/**/*.{ts,tsx,js,jsx,css,scss}",
  "!**/dist/**",
  "!**/node_modules/**",
  "!**/*.test.*",
  "!**/*.stories.*",
]);

const violations: Array<{
  file: string;
  line: number;
  rule: string;
  match: string;
}> = [];

const rules: Array<{ rule: string; expression: RegExp }> = [
  {
    rule: "raw-color",
    expression:
      /#[0-9a-fA-F]{3,8}\b|\brgba?\(|\bhsla?\(|\boklch?\(/g,
  },
  {
    rule: "arbitrary-tailwind-design-value",
    expression:
      /\b(?:bg|text|border|ring|shadow|p|px|py|pt|pr|pb|pl|m|mx|my|mt|mr|mb|ml|gap|z
  },
  {
    rule: "arbitrary-z-index",
    expression: /\bzIndex\s*:\s*\d{2,}/g,
  },
  {
    rule: "raw-shadow",
    expression: /\bboxShadow\s*:\s*["'`]/g,
  },
];

for (const file of files) {
  const content = await readFile(file, "utf8");
  const lines = content.split("\n");

  for (const [index, line] of lines.entries()) {
    for (const { rule, expression } of rules) {
      for (const match of line.matchAll(expression)) {
        violations.push({
          file,
          line: index + 1,
          rule,
          match: match[^3_0],
        });
      }
    }
  }
}

if (violations.length === 0) {
  console.log("No token-policy violations found.");
```

```
  process.exit(0);
}

console.table(violations);
process.exit(1);
```

Tune exclusions so that raw values remain allowed in `packages/tokens`, snapshots, fixtures, generated output, and intentionally isolated visual-test files.

CI report script

Add scripts like these to root `package.json`:

```
{
  "scripts": {
    "tokens:validate": "tsx packages/tokens/src/validate.ts",
    "tokens:build": "tsx scripts/build-tokens.ts",
    "design-system:map": "tsx scripts/build-figma-token-map.ts",
    "design-system:audit": "tsx scripts/audit-token-usage.ts",
    "design-system:report": "pnpm tokens:validate && pnpm design-system:map && pnpm d
  }
}
```

GitHub Actions workflow

Create `.github/workflows/design-system.yml`:

```
name: Design system CI

on:
  pull_request:
    paths:
      - "apps/**"
      - "packages/tokens/**"
      - "packages/ui/**"
      - "scripts/**"
      - "docs/figma-mapping.md"
      - ".claude/**"
      - ".github/workflows/design-system.yml"
  push:
    branches:
      - main

jobs:
  validate-design-system:
    name: Validate tokens and design-system drift
    runs-on: ubuntu-latest

    steps:
      - name: Check out repository
        uses: actions/checkout@v4
```

```
      - name: Set up pnpm
        uses: pnpm/action-setup@v4
        with:
          version: 9

      - name: Set up Node
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: pnpm

      - name: Install dependencies
        run: pnpm install --frozen-lockfile

      - name: Validate token aliases
        run: pnpm tokens:validate

      - name: Build token artifacts
        run: pnpm tokens:build

      - name: Build Figma-to-code token map
        run: pnpm design-system:map

      - name: Detect raw design values
        run: pnpm design-system:audit

      - name: Typecheck
        run: pnpm typecheck

      - name: Run component tests
        run: pnpm test

      - name: Run visual regression tests
        run: pnpm test:visual

      - name: Upload design-system report
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: design-system-report
          path: reports/design-system/
          if-no-files-found: warn
```

CI output contract

Have CI produce a report with this structure:

```
# Semester Design System Report

## Summary

- Token alias validation: pass
- Figma-to-code mapped variables: 84 / 88
- Missing semantic code tokens: 2
- Obsolete mappings: 1
```

```
- Raw-value violations: 3
- Unmapped Figma components: 2
- Visual-regression status: pass
- Accessibility-test status: partial

## Blockers

| Type | Location | Issue | Resolution |
|---|---|---|---|
| Raw color | `apps/web/src/billing/PlanCard.tsx:42` | `#1E63D8` bypasses semantic to
| Mapping | `Figma: Action / Positive / Default` | No matching semantic token | Propo

## Major drift

| Type | Figma | Code | Recommended action |
|---|---|---|---|
| Component | `Data table / Compact` | No supported React density mode | Add `density
| State | Error state present in Figma | No retry action in React | Add inline retry

## Token parity

| Figma variable | Semester token | Status |
|---|---|---|
| `Text / Primary` | `color.text.primary` | Match |
| `Action / Primary / Default` | `color.action.primary.default` | Match |
| `Action / Positive / Default` | — | Missing |
```

Figma variable audit workflow

For actual Figma-value inspection, use Claude Code with Figma MCP after the server is connected:

```
/audit-semester-design-sync <Figma frame URL> apps/web/src/routes/settings.tsx
```

Or prompt Claude Code directly:

```
Use the audit-semester-design-sync skill.

Inspect this Figma frame and compare it with the React implementation:
[Figma URL]

Audit:
- Figma variables against packages/tokens/src/semantic.ts.
- Figma components against @semester/ui exports.
- React source for raw colors and raw layout values.
- Figma and code state parity.
- Responsive and accessibility gaps.

Do not edit files. Produce the required design-sync audit report.
```

```
The remote HTTP MCP route is Claude Code's recommended approach for cloud MCP servers
```

Rollout sequence

1. Add the root `CLAUDE.md` and commit it.
2. Create the `audit-semester-design-sync` skill and invoke it against one existing production route.
3. Install Figma MCP using the official plugin; use the remote server by default. [32]
4. Establish reference, semantic, and component token layers.
5. Create and maintain `docs/figma-mapping.md`.
6. Add the Figma-token mapping manifest.
7. Add raw-value detection and token-alias validation to CI.
8. Audit your ten most-used components and three most important product screens.
9. Convert repeated findings into one of four outcomes: token, component, documented pattern, or lint rule.
10. Require every new Figma library variable/component to have a corresponding code mapping before it reaches production.

This gives Claude Code a constrained, inspectable contract: it can read Figma for intent, map intent to Semester primitives, detect drift automatically, and refuse to solve every UI task with another hand-styled generic component.

⁂

provide comand to claud code to excute on this and also anlyse current semester resporty and specfic to it

Your current Semester repo is already much further along than a generic “add a design system” setup. It has a dedicated token source/export path, extensive style-contract tests, an existing root `CLAUDE.md`, vendored Claude skills, and a large Vite/React application under `app/`. The best next move is **not** to replace that architecture—it is to add a Semester-specific Claude Code audit skill and tighten the existing token-export → Figma-variable → CI loop. [github_mcp_direct]

Run this in Claude Code

From the repository root, launch Claude Code:

```
cd /path/to/semester
claude
```

Then paste this command/prompt exactly into Claude Code:

```
/audit-semester-design-sync app/src app/design-tokens/semester.tokens.json app/src/s

You are auditing the existing Semester repository, not designing a generic new syste
```

```
First read:
- CLAUDE.md
- app/README.md
- DESIGN-SYSTEM-GUIDE.md
- docs/DESIGN-TOKENS.md
- docs/design/DESIGN-SYSTEM-PRODUCT-SPEC.md
- app/package.json
- app/src/styles/tokens.css
- app/src/lib/tokenexport.ts
- app/src/lib/tokenexport.test.ts
- app/design-tokens/semester.tokens.json
- app/src/styles/*.test.ts
- .claude/skills/

Then inspect:
- app/src/components/
- app/src/screens/
- app/src/styles/
- app/src/App.tsx
- app/src/main.tsx
- app/src/screens.tsx

Do not modify files yet.

Create a repository-specific Semester design-system audit with:
1. Existing architecture and source-of-truth map.
2. Token pipeline map: CSS semantic tokens → tokenexport.ts → semester.tokens.json →
3. Current design-system strengths that must be preserved.
4. Drift risks: raw styles, direct hex usage, duplicated component styles, hard-codec
5. Existing tests/scripts that already enforce design contracts.
6. Exact missing files needed for Claude Code + Figma MCP workflow only.
7. A minimal phased implementation plan that avoids restructuring the current app.
8. A table of proposed changes with priority, exact path, rationale, and acceptance t

Use evidence from actual files. Do not assume pnpm, monorepos, Storybook, Tailwind, c
```

That audit prompt deliberately tells Claude not to invent a `packages/ui`, pnpm workspace, Tailwind setup, or a replacement token schema—the patterns in earlier generic templates would be wrong for this particular repository.

One-command bootstrap prompt

Once the audit completes and you agree with its plan, paste the following command to have Claude Code implement only the smallest compatible toolkit:

```
Implement the approved Semester Claude Code design-system toolkit in this repository

Constraints:
- Preserve the current Vite + React architecture under app/.
- Preserve app/src/styles/tokens.css as the semantic token authority.
- Preserve app/src/lib/tokenexport.ts as the token export authority.
- Preserve app/design-tokens/semester.tokens.json as generated output; never hand-edi
- Preserve and extend existing tokenexport, style, accessibility, responsive, and cor
- Do not add Tailwind, Storybook, a monorepo, or a new component library package.
```

```
- Do not move existing components or rewrite App.tsx unless absolutely necessary.
- Do not add dependencies unless an existing repository dependency cannot support the
- Do not commit, push, open a PR, or change deployment configuration.

Create only:
1. .claude/skills/audit-semester-design-sync/SKILL.md
2. .claude/skills/build-semester-ui/SKILL.md
3. .mcp.json using the Figma remote MCP server, with no credentials committed
4. docs/FIGMA-MAPPING.md
5. app/scripts/design-system-audit.mjs or .ts, compatible with the app's existing too
6. app/scripts/design-system-report.mjs or .ts
7. Minimal package.json scripts in app/package.json:
   - design-system:audit
   - design-system:report
   - design-system:check
8. Focused tests that prove:
   - semester.tokens.json is generated and synchronized with code
   - raw values are not introduced outside approved CSS/token files
   - Figma mapping entries resolve to exported token paths
   - audit/report scripts fail on real violations and pass on valid fixtures

Before editing, show the file-by-file plan.
After editing, run the repository's actual relevant validation commands.
Report changed files, exact checks run, results, known limitations, and any Figma ste
```

Current Semester findings

Architecture already present

The active application is a Vite/TypeScript React app located in `app/`, not a package-per-component design-system monorepo. It has a large app entry point, screen hierarchy, component directory, styling directory, API/server code, and build configuration. [github_mcp_direct]

Current asset	What it means for your toolkit
`app/src/styles/tokens.css`	This should remain the **semantic-token authority**
`app/src/lib/tokenexport.ts`	This should remain the **token-export authority**
`app/design-tokens/semester.tokens.json`	This is the machine-readable/exported artifact; do not hand-edit it
`app/src/lib/tokenexport.test.ts`	Already validates that token export does not drift from code
`app/src/styles/*.test.ts`	You already have a substantial contract-test surface for CSS design rules
`app/src/components/` and `app/src/screens/`	These are the correct places to audit real UI composition and component duplication
`.claude/skills/`	Claude Code skill infrastructure already exists, so add one focused skill rather than bootstrapping from zero

Current asset	What it means for your toolkit
`CLAUDE.md`	Existing durable project instructions should be amended, not replaced blindly
`DESIGN-SYSTEM-GUIDE.md` and `docs/DESIGN-TOKENS.md`	Existing written design-system documentation that new Claude rules should reference

The repository's token documentation explicitly identifies `app/design-tokens/semester.tokens.json` as a generated, machine-readable export; `tokenexport.test.ts` verifies that it remains synchronized with `tokens.css`. That is a stronger foundation than a fresh generic JSON schema because it already supports downstream Figma/native/parity use cases. [github_mcp_direct]

Important constraints

Your project has a **large existing CSS system**: `app.css` is approximately 322 KB, `features.css` about 29 KB, `unity.css` about 34 KB, and `tokens.css` roughly 13 KB. It also has targeted tests for breakpoints, density, motion, fields, glass effects, gutters, hex handling, stacking, taps, typography scale, and token behavior. Treat this as a mature contract system, not a greenfield UI package. [github_mcp_direct]

So Claude Code should:

- Reuse the current CSS token vocabulary before introducing any new token name.
- Extend current source-based token export rather than create duplicate TypeScript token files.
- Use the existing style tests as enforcement hooks instead of adding broad, untested rules in a new lint stack.
- Keep the generated token JSON as the Figma/native interchange artifact.
- Add a Figma mapping manifest and a diff report around the existing export, rather than trying to make Figma the production source of truth.

Existing Figma readiness

The repository contains Figma-aware documentation and token-export comments, but the code search did **not** reveal a configured Figma MCP server or a maintained Figma-to-code component/variable mapping file. The export code and tests already describe `semester.tokens.json` as the artifact intended for Figma variables, native clients, and parity checking. [github_mcp_direct]

That means your gap is integration/governance—not token invention.

Add this audit skill

Create this folder and file:

```
mkdir -p .claude/skills/audit-semester-design-sync
```

```
.claude/skills/audit-semester-design-sync/SKILL.md
```

Paste this exact content:

```
---
name: audit-semester-design-sync
description: Audit the Semester React application against its existing CSS token sys
argument-hint: "[optional-figma-url-or-node] [optional-app-path]"
---

# Audit Semester Design Sync

## Semester-specific source of truth

Semester is an existing Vite and React application. Do not impose a generic monorepo
Tailwind, Storybook, or package-based design-system architecture unless it already ex

The source-of-truth order is:

1. `app/src/styles/tokens.css`
   - Semantic CSS token authority used by application UI.

2. `app/src/lib/tokenexport.ts`
   - Exports the application's semantic token system for downstream consumers.

3. `app/design-tokens/semester.tokens.json`
   - Generated machine-readable token artifact.
   - Never edit manually.
   - Regenerate through the repository's actual token-export command.

4. Existing CSS/component implementation
   - `app/src/styles/`
   - `app/src/components/`
   - `app/src/screens/`

5. Existing contract tests
   - `app/src/styles/*.test.ts`
   - `app/src/lib/tokenexport.test.ts`
   - Related architecture, responsive, accessibility, and screen tests.

6. Figma
   - Figma is evidence of approved visual intent.
   - Code owns production behavior, accessibility, responsive implementation, and tol
   - Figma variables/components must map to existing exported token paths and React/C

## Required reading
```

```
Before auditing, read:

- `CLAUDE.md`
- `app/README.md`
- `DESIGN-SYSTEM-GUIDE.md`
- `docs/DESIGN-TOKENS.md`
- `docs/design/DESIGN-SYSTEM-PRODUCT-SPEC.md`
- `app/package.json`
- `app/src/styles/tokens.css`
- `app/src/lib/tokenexport.ts`
- `app/src/lib/tokenexport.test.ts`
- `app/design-tokens/semester.tokens.json`
- `app/src/styles/*.test.ts`
- `docs/FIGMA-MAPPING.md`, if it exists

For a UI-specific audit, inspect the target implementation plus nearest related scree
components, and styles.

## Figma workflow

If a Figma URL, selection, node, component, or variable is supplied:

1. Use configured Figma MCP tools to inspect:
   - Node hierarchy.
   - Figma variables and collections.
   - Component instances and properties.
   - Auto-layout, constraints, and visible states.
   - Text/content hierarchy.

2. Compare Figma variable names to:
   - `app/design-tokens/semester.tokens.json`
   - `docs/FIGMA-MAPPING.md`
   - Semantic names in `app/src/styles/tokens.css`

3. Compare Figma components to current React/CSS components.
4. Do not create raw CSS to obtain pixel-perfect parity.
5. Do not treat an incomplete Figma frame as permission to omit:
   - Keyboard support.
   - Focus visibility.
   - Empty/loading/error states.
   - Responsive behavior.
   - Accessible labels and semantic HTML.

## Audit checks

### Token integrity

Identify:

- Direct hex, rgb, hsl, oklch, arbitrary shadows, arbitrary spacing, arbitrary radii
  arbitrary z-indexes, arbitrary durations, or arbitrary easing values.
- Whether each raw value is permitted in a foundational source file or is an app-leve
- CSS variables used but absent from `tokens.css`.
- CSS tokens present but not represented in the generated export.
- Generated token paths that cannot be traced back to semantic CSS tokens.
- Figma variables that lack an exported token mapping.
```

```
- Exported semantic tokens that lack an approved Figma mapping, if parity is required

### Component and pattern integrity

Identify:

- Feature-local components duplicating existing component or CSS patterns.
- Hard-coded styling that bypasses existing classes or semantic variables.
- Missing or inconsistent states.
- Misused controls: buttons vs links, clickable divs, unlabeled icon controls.
- Inconsistent density, hierarchy, type scale, surface treatment, borders, and page g
- Component APIs or CSS selectors that leak implementation detail.

### Responsive integrity

Check:

- Current breakpoint/test contracts.
- Narrow, intermediate, and wide behavior.
- Overflow, scroll, truncation, long copy, localization, and touch targets.
- Fixed/sticky element and stacking behavior.
- Data/table fallback behavior on narrow layouts.

### Accessibility integrity

Check:

- Semantic landmarks and headings.
- Label, description, error, and input associations.
- Keyboard interaction and visible focus.
- Dialog/menu/popover focus behavior.
- Color contrast and non-color status communication.
- Reduced-motion behavior.
- Target sizing.

## Severity scale

- Blocker: breaks critical usability, accessibility, core system integrity, or releas
- Major: repeatable drift, broken component/pattern contract, significant responsive,
- Minor: localized inconsistency, documentation gap, or low-risk visual mismatch.

## No-edit audit mode

Do not modify source code during an audit unless the user explicitly asks after revie
the report. You may run read-only inspection commands and existing test/audit commanc

## Required report format

# Semester design-system audit

## Executive assessment

State:
- Overall alignment: strong, moderate, weak, or critical drift.
- Highest-risk finding.
- Whether the main concern is tokens, components, responsive behavior, accessibility
```

```
  Figma parity, or test coverage.

## Repository model

| Concern | Current Semester authority | Evidence |
|---|---|---|
| Semantic tokens | `app/src/styles/tokens.css` | Specific section/file |
| Token export | `app/src/lib/tokenexport.ts` | Specific function/file |
| Machine-readable artifact | `app/design-tokens/semester.tokens.json` | Specific fi
| UI implementation | Components/screens/styles | Specific paths |
| Enforcement | Existing tests/scripts | Exact script or test paths |
| Figma parity | Mapping file or missing | Evidence |

## Findings

| Severity | Location | Category | Evidence | Existing Semester solution | Smallest s
|---|---|---|---|---|---|
| Blocker/Major/Minor | Exact path, selector, token, or Figma node | Token/Component/

## Token parity

| Figma variable or code token | Semester exported path | CSS semantic source | Statu
|---|---|---|---|---|
| Value | Value | Value | Match/Partial/Missing/Obsolete | Specific resolution |

## Component parity

| Figma or screen component | Current React/CSS implementation | State/variant parity
|---|---|---|---|---|---|
| Value | Value | Match/Partial/Missing | Match/Partial/Missing/Obsolete | Specific r

## Existing enforcement

| Contract | Existing test/script | Coverage | Gap |
|---|---|---|---|
| Token export | Exact test/script | What it proves | What remains |
| Responsive | Exact test/script | What it proves | What remains |
| Accessibility | Exact test/script | What it proves | What remains |
| Raw values | Exact test/script | What it proves | What remains |

## Minimal implementation plan

| Priority | Exact path | Change | Why | Acceptance test |
|---|---|---|---|---|
| P0/P1/P2 | Exact repo path | Smallest concrete change | System value | Exact commar

## Rules for recommendations

- Preserve Semester's existing token and export architecture.
- Recommend extensions, not parallel sources of truth.
- Use current tests before introducing tooling.
- Do not recommend Tailwind, Storybook, a monorepo, or package migrations unless
  directly supported by evidence and explicitly requested.
- Do not create a component for a one-off visual difference.
- Never recommend direct raw values to match Figma pixels.
- Prefer semantic CSS variables, existing classes, documented CSS patterns, and curre
```

Figma MCP setup

For Semester, configure the **remote** Figma MCP endpoint at the repository level, but do not commit credentials.

Run from the repository root:

```
claude mcp add --scope project --transport http figma https://mcp.figma.com/mcp
```

Then launch Claude Code:

```
claude
```

In Claude Code:

```
/mcp
```

```
Authorize Figma through the browser prompt, then verify Figma is connected. Claude Co
```

If you prefer a committed project declaration, add a root `.mcp.json`:

```
{
  "mcpServers": {
    "figma": {
      "type": "http",
      "url": "https://mcp.figma.com/mcp"
    }
  }
}
```

The `type: "http"` field matters: Claude Code needs it to interpret the URL as a remote HTTP MCP server instead of an attempted local stdio server. [34]

Figma recommends its official Claude Code plugin as the lowest-friction approach, because the plugin includes the MCP configuration and Figma-related workflow skills:

```
claude plugin install figma@claude-plugins-official
```

Use **one** setup route initially:

- Prefer the official plugin if you want Figma's maintained defaults and bundled workflows.
- Prefer the checked-in `.mcp.json` if you need a transparent, reviewable repository configuration.
- Avoid configuring duplicate servers under multiple names until you have a real reason; duplicated MCP tools make Claude's tool selection less reliable. Figma MCP gives Claude

structured access to variables, components, layout information, and other design context used for implementation and audit work. [35]

What to add—not replace

Add now	Why
`.claude/skills/audit-semester-design-sync/SKILL.md`	Gives Claude a repeatable repo-specific design review workflow
`.claude/skills/build-semester-ui/SKILL.md`	Ensures future UI work follows existing tokens, CSS, components, and test contracts
`docs/FIGMA-MAPPING.md`	Makes token/component parity explicit and reviewable
`.mcp.json` or the Figma plugin	Gives Claude structured Figma context
`app/scripts/design-system-audit.*`	Detects prohibited raw styling and missing token paths
`app/scripts/design-system-report.*`	Produces a CI-readable design-system drift summary
Package scripts in `app/package.json`	Makes the audit reproducible locally and in CI
Do not add yet	Why
---	---
A second token source in `packages/tokens/`	Semester already has `tokens.css` + `tokenexport.ts` + generated JSON
A generic `@semester/ui` package	No evidence that your current app uses package-based UI distribution
Tailwind/token migration	Would create a parallel styling system and increase drift risk
Storybook as a prerequisite	Useful later, but first use your established CSS contracts and app-screen tests
A new component framework	Existing components/screens/styles need an evidence-based audit first
Automatic Figma writes	Figma reads are appropriate for audit; design-file mutations need explicit review

The repository is public and currently identified as `harrisonjrubin7-cmyk/semester`, with `main` as the default branch. It contains roughly 53 open issues and a substantial TypeScript codebase, so start with a **read-only audit and minimal additive configuration**, rather than a sweeping UI-platform rewrite. [github_mcp_direct]

⁂

1. https://code.claude.com/docs/en/memory
2. https://code.claude.com/docs/en/skills
3. https://github.com/anthropics/claude-code
4. https://www.reddit.com/r/ClaudeAI/comments/1m6hek6/claude_project_loaded_with_all_claude_code_docs/
5. https://www.freecodecamp.org/news/claude-code-handbook/

6. https://www.humanlayer.dev/blog/writing-a-good-claude-md
7. https://nervegna.substack.com/p/claude-code-for-designers-a-practical
8. https://theyawns.com/2026/03/06/launching-a-claude-code-project-design-first-then-build/
9. https://platform.claude.com/docs/en/home
10. https://support.claude.com/en/articles/14604397-set-up-your-design-system-in-claude-design
11. https://www-cdn.anthropic.com/58284b19e702b49db9302d5b6f135ad8871e7658.pdf
12. https://academy.claude.com/courses/claude-101/introduction-to-projects
13. https://claude.com/product/claude-code
14. https://readme.com/blog/what-is-claude-md
15. https://www.intodesignsystems.com/claude-code-for-designers
16. https://help.figma.com/hc/en-us/articles/39888612464151-Claude-Code-and-Figma-Set-up-the-MCP-server
17. https://help.figma.com/hc/en-us/articles/32132100833559-Guide-to-the-Figma-MCP-server
18. https://code.claude.com/docs/en/memory
19. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices
20. https://github.com/figma/mcp-server-guide
21. https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf
22. https://claude.com/blog/skills-explained
23. https://composio.dev/toolkits/figma/framework/claude-code
24. https://github.com/anthropics/skills
25. https://vectorize.io/articles/claude-code-memory
26. https://github.com/luongnv89/claude-howto/blob/main/02-memory/README.md
27. https://code.claude.com/docs/en/skills
28. https://www.figma.com/mcp-catalog/
29. https://www.figma.com/blog/introducing-claude-code-to-figma/
30. https://support.claude.com/en/articles/12123587-import-and-export-your-memory-from-claude
31. https://code.claude.com/docs/en/skills
32. https://help.figma.com/hc/en-us/articles/39888612464151-Claude-Code-and-Figma-Set-up-the-MCP-server
33. https://code.claude.com/docs/en/mcp
34. https://code.claude.com/docs/en/mcp
35. https://help.figma.com/hc/en-us/articles/39888612464151-Claude-Code-and-Figma-Set-up-the-MCP-server

provide comand to claud code to excute on this and also anlyse current semester resporty and specfic to it

Your current Semester repo is already much further along than a generic “add a design system” setup. It has a dedicated token source/export path, extensive style-contract tests, an existing root `CLAUDE.md`, vendored Claude skills, and a large Vite/React application under `app/`. The best next move is **not** to replace that architecture—it is to add a Semester-specific Claude Code audit skill and tighten the existing token-export → Figma-variable → CI loop. [github_mcp_direct]

Run this in Claude Code

From the repository root, launch Claude Code:

```
cd /path/to/semester
claude
```

Then paste this command/prompt exactly into Claude Code:

```
/audit-semester-design-sync app/src app/design-tokens/semester.tokens.json app/src/s

You are auditing the existing Semester repository, not designing a generic new syste

First read:
- CLAUDE.md
- app/README.md
- DESIGN-SYSTEM-GUIDE.md
- docs/DESIGN-TOKENS.md
- docs/design/DESIGN-SYSTEM-PRODUCT-SPEC.md
- app/package.json
- app/src/styles/tokens.css
- app/src/lib/tokenexport.ts
- app/src/lib/tokenexport.test.ts
- app/design-tokens/semester.tokens.json
- app/src/styles/*.test.ts
- .claude/skills/

Then inspect:
- app/src/components/
- app/src/screens/
- app/src/styles/
- app/src/App.tsx
- app/src/main.tsx
- app/src/screens.tsx
```

```
Do not modify files yet.

Create a repository-specific Semester design-system audit with:
1. Existing architecture and source-of-truth map.
2. Token pipeline map: CSS semantic tokens → tokenexport.ts → semester.tokens.json →
3. Current design-system strengths that must be preserved.
4. Drift risks: raw styles, direct hex usage, duplicated component styles, hard-code
5. Existing tests/scripts that already enforce design contracts.
6. Exact missing files needed for Claude Code + Figma MCP workflow only.
7. A minimal phased implementation plan that avoids restructuring the current app.
8. A table of proposed changes with priority, exact path, rationale, and acceptance

Use evidence from actual files. Do not assume pnpm, monorepos, Storybook, Tailwind, c
```

That audit prompt deliberately tells Claude not to invent a `packages/ui`, pnpm workspace, Tailwind setup, or a replacement token schema—the patterns in earlier generic templates would be wrong for this particular repository.

One-command bootstrap prompt

Once the audit completes and you agree with its plan, paste the following command to have Claude Code implement only the smallest compatible toolkit:

```
Implement the approved Semester Claude Code design-system toolkit in this repository

Constraints:
- Preserve the current Vite + React architecture under app/.
- Preserve app/src/styles/tokens.css as the semantic token authority.
- Preserve app/src/lib/tokenexport.ts as the token export authority.
- Preserve app/design-tokens/semester.tokens.json as generated output; never hand-edi
- Preserve and extend existing tokenexport, style, accessibility, responsive, and cor
- Do not add Tailwind, Storybook, a monorepo, or a new component library package.
- Do not move existing components or rewrite App.tsx unless absolutely necessary.
- Do not add dependencies unless an existing repository dependency cannot support the
- Do not commit, push, open a PR, or change deployment configuration.

Create only:
1. .claude/skills/audit-semester-design-sync/SKILL.md
2. .claude/skills/build-semester-ui/SKILL.md
3. .mcp.json using the Figma remote MCP server, with no credentials committed
4. docs/FIGMA-MAPPING.md
5. app/scripts/design-system-audit.mjs or .ts, compatible with the app's existing too
6. app/scripts/design-system-report.mjs or .ts
7. Minimal package.json scripts in app/package.json:
   - design-system:audit
   - design-system:report
   - design-system:check
8. Focused tests that prove:
   - semester.tokens.json is generated and synchronized with code
   - raw values are not introduced outside approved CSS/token files
   - Figma mapping entries resolve to exported token paths
   - audit/report scripts fail on real violations and pass on valid fixtures

Before editing, show the file-by-file plan.
```

```
After editing, run the repository's actual relevant validation commands.
Report changed files, exact checks run, results, known limitations, and any Figma ste
```

Current Semester findings

Architecture already present

The active application is a Vite/TypeScript React app located in `app/`, not a package-per-component design-system monorepo. It has a large app entry point, screen hierarchy, component directory, styling directory, API/server code, and build configuration. [github_mcp_direct]

Current asset	What it means for your toolkit
`app/src/styles/tokens.css`	This should remain the **semantic-token authority**
`app/src/lib/tokenexport.ts`	This should remain the **token-export authority**
`app/design-tokens/semester.tokens.json`	This is the machine-readable/exported artifact; do not hand-edit it
`app/src/lib/tokenexport.test.ts`	Already validates that token export does not drift from code
`app/src/styles/*.test.ts`	You already have a substantial contract-test surface for CSS design rules
`app/src/components/` and `app/src/screens/`	These are the correct places to audit real UI composition and component duplication
`.claude/skills/`	Claude Code skill infrastructure already exists, so add one focused skill rather than bootstrapping from zero
`CLAUDE.md`	Existing durable project instructions should be amended, not replaced blindly
`DESIGN-SYSTEM-GUIDE.md` and `docs/DESIGN-TOKENS.md`	Existing written design-system documentation that new Claude rules should reference

The repository's token documentation explicitly identifies `app/design-tokens/semester.tokens.json` as a generated, machine-readable export; `tokenexport.test.ts` verifies that it remains synchronized with `tokens.css`. That is a stronger foundation than a fresh generic JSON schema because it already supports downstream Figma/native/parity use cases. [github_mcp_direct]

Important constraints

Your project has a **large existing CSS system**: `app.css` is approximately 322 KB, `features.css` about 29 KB, `unity.css` about 34 KB, and `tokens.css` roughly 13 KB. It also has targeted tests for breakpoints, density, motion, fields, glass effects, gutters, hex handling, stacking, taps, typography scale, and token behavior. Treat this as a mature contract system, not a greenfield UI package. [github_mcp_direct]

So Claude Code should:

- Reuse the current CSS token vocabulary before introducing any new token name.
- Extend current source-based token export rather than create duplicate TypeScript token files.
- Use the existing style tests as enforcement hooks instead of adding broad, untested rules in a new lint stack.
- Keep the generated token JSON as the Figma/native interchange artifact.
- Add a Figma mapping manifest and a diff report around the existing export, rather than trying to make Figma the production source of truth.

Existing Figma readiness

The repository contains Figma-aware documentation and token-export comments, but the code search did **not** reveal a configured Figma MCP server or a maintained Figma-to-code component/variable mapping file. The export code and tests already describe `semester.tokens.json` as the artifact intended for Figma variables, native clients, and parity checking. [github_mcp_direct]

That means your gap is integration/governance—not token invention.

Add this audit skill

Create this folder and file:

```
mkdir -p .claude/skills/audit-semester-design-sync
```

```
.claude/skills/audit-semester-design-sync/SKILL.md
```

Paste this exact content:

```
---
name: audit-semester-design-sync
description: Audit the Semester React application against its existing CSS token sys
argument-hint: "[optional-figma-url-or-node] [optional-app-path]"
---

# Audit Semester Design Sync

## Semester-specific source of truth

Semester is an existing Vite and React application. Do not impose a generic monorepo
Tailwind, Storybook, or package-based design-system architecture unless it already ex

The source-of-truth order is:

1. `app/src/styles/tokens.css`
   - Semantic CSS token authority used by application UI.

2. `app/src/lib/tokenexport.ts`
```

```
   - Exports the application's semantic token system for downstream consumers.

3. `app/design-tokens/semester.tokens.json`
   - Generated machine-readable token artifact.
   - Never edit manually.
   - Regenerate through the repository's actual token-export command.

4. Existing CSS/component implementation
   - `app/src/styles/`
   - `app/src/components/`
   - `app/src/screens/`

5. Existing contract tests
   - `app/src/styles/*.test.ts`
   - `app/src/lib/tokenexport.test.ts`
   - Related architecture, responsive, accessibility, and screen tests.

6. Figma
   - Figma is evidence of approved visual intent.
   - Code owns production behavior, accessibility, responsive implementation, and tol
   - Figma variables/components must map to existing exported token paths and React/C

## Required reading

Before auditing, read:

- `CLAUDE.md`
- `app/README.md`
- `DESIGN-SYSTEM-GUIDE.md`
- `docs/DESIGN-TOKENS.md`
- `docs/design/DESIGN-SYSTEM-PRODUCT-SPEC.md`
- `app/package.json`
- `app/src/styles/tokens.css`
- `app/src/lib/tokenexport.ts`
- `app/src/lib/tokenexport.test.ts`
- `app/design-tokens/semester.tokens.json`
- `app/src/styles/*.test.ts`
- `docs/FIGMA-MAPPING.md`, if it exists

For a UI-specific audit, inspect the target implementation plus nearest related scree
components, and styles.

## Figma workflow

If a Figma URL, selection, node, component, or variable is supplied:

1. Use configured Figma MCP tools to inspect:
   - Node hierarchy.
   - Figma variables and collections.
   - Component instances and properties.
   - Auto-layout, constraints, and visible states.
   - Text/content hierarchy.

2. Compare Figma variable names to:
   - `app/design-tokens/semester.tokens.json`
   - `docs/FIGMA-MAPPING.md`
```

```
   - Semantic names in `app/src/styles/tokens.css`

3. Compare Figma components to current React/CSS components.
4. Do not create raw CSS to obtain pixel-perfect parity.
5. Do not treat an incomplete Figma frame as permission to omit:
   - Keyboard support.
   - Focus visibility.
   - Empty/loading/error states.
   - Responsive behavior.
   - Accessible labels and semantic HTML.

## Audit checks

### Token integrity

Identify:

- Direct hex, rgb, hsl, oklch, arbitrary shadows, arbitrary spacing, arbitrary radii
  arbitrary z-indexes, arbitrary durations, or arbitrary easing values.
- Whether each raw value is permitted in a foundational source file or is an app-leve
- CSS variables used but absent from `tokens.css`.
- CSS tokens present but not represented in the generated export.
- Generated token paths that cannot be traced back to semantic CSS tokens.
- Figma variables that lack an exported token mapping.
- Exported semantic tokens that lack an approved Figma mapping, if parity is required

### Component and pattern integrity

Identify:

- Feature-local components duplicating existing component or CSS patterns.
- Hard-coded styling that bypasses existing classes or semantic variables.
- Missing or inconsistent states.
- Misused controls: buttons vs links, clickable divs, unlabeled icon controls.
- Inconsistent density, hierarchy, type scale, surface treatment, borders, and page g
- Component APIs or CSS selectors that leak implementation detail.

### Responsive integrity

Check:

- Current breakpoint/test contracts.
- Narrow, intermediate, and wide behavior.
- Overflow, scroll, truncation, long copy, localization, and touch targets.
- Fixed/sticky element and stacking behavior.
- Data/table fallback behavior on narrow layouts.

### Accessibility integrity

Check:

- Semantic landmarks and headings.
- Label, description, error, and input associations.
- Keyboard interaction and visible focus.
- Dialog/menu/popover focus behavior.
- Color contrast and non-color status communication.
```

```
- Reduced-motion behavior.
- Target sizing.

## Severity scale

- Blocker: breaks critical usability, accessibility, core system integrity, or releas
- Major: repeatable drift, broken component/pattern contract, significant responsive,
- Minor: localized inconsistency, documentation gap, or low-risk visual mismatch.

## No-edit audit mode

Do not modify source code during an audit unless the user explicitly asks after revie
the report. You may run read-only inspection commands and existing test/audit commanc

## Required report format

# Semester design-system audit

## Executive assessment

State:
- Overall alignment: strong, moderate, weak, or critical drift.
- Highest-risk finding.
- Whether the main concern is tokens, components, responsive behavior, accessibility
  Figma parity, or test coverage.

## Repository model

| Concern | Current Semester authority | Evidence |
|---|---|---|
| Semantic tokens | `app/src/styles/tokens.css` | Specific section/file |
| Token export | `app/src/lib/tokenexport.ts` | Specific function/file |
| Machine-readable artifact | `app/design-tokens/semester.tokens.json` | Specific fi
| UI implementation | Components/screens/styles | Specific paths |
| Enforcement | Existing tests/scripts | Exact script or test paths |
| Figma parity | Mapping file or missing | Evidence |

## Findings

| Severity | Location | Category | Evidence | Existing Semester solution | Smallest s
|---|---|---|---|---|---|
| Blocker/Major/Minor | Exact path, selector, token, or Figma node | Token/Component/

## Token parity

| Figma variable or code token | Semester exported path | CSS semantic source | Statu
|---|---|---|---|---|
| Value | Value | Value | Match/Partial/Missing/Obsolete | Specific resolution |

## Component parity

| Figma or screen component | Current React/CSS implementation | State/variant parity
|---|---|---|---|---|
| Value | Value | Match/Partial/Missing | Match/Partial/Missing/Obsolete | Specific r

## Existing enforcement
```

```
| Contract | Existing test/script | Coverage | Gap |
|---|---|---|---|
| Token export | Exact test/script | What it proves | What remains |
| Responsive | Exact test/script | What it proves | What remains |
| Accessibility | Exact test/script | What it proves | What remains |
| Raw values | Exact test/script | What it proves | What remains |

## Minimal implementation plan

| Priority | Exact path | Change | Why | Acceptance test |
|---|---|---|---|---|
| P0/P1/P2 | Exact repo path | Smallest concrete change | System value | Exact commar

## Rules for recommendations

- Preserve Semester's existing token and export architecture.
- Recommend extensions, not parallel sources of truth.
- Use current tests before introducing tooling.
- Do not recommend Tailwind, Storybook, a monorepo, or package migrations unless
  directly supported by evidence and explicitly requested.
- Do not create a component for a one-off visual difference.
- Never recommend direct raw values to match Figma pixels.
- Prefer semantic CSS variables, existing classes, documented CSS patterns, and curre
```

Figma MCP setup

For Semester, configure the **remote** Figma MCP endpoint at the repository level, but do not commit credentials.

Run from the repository root:

```
claude mcp add --scope project --transport http figma https://mcp.figma.com/mcp
```

Then launch Claude Code:

```
claude
```

In Claude Code:

```
/mcp
```

```
Authorize Figma through the browser prompt, then verify Figma is connected. Claude Co
```

If you prefer a committed project declaration, add a root `.mcp.json`:

```
{
  "mcpServers": {
    "figma": {
```

```
      "type": "http",
      "url": "https://mcp.figma.com/mcp"
    }
  }
}
```

The `type: "http"` field matters: Claude Code needs it to interpret the URL as a remote HTTP MCP server instead of an attempted local stdio server.[1]

Figma recommends its official Claude Code plugin as the lowest-friction approach, because the plugin includes the MCP configuration and Figma-related workflow skills:

```
claude plugin install figma@claude-plugins-official
```

Use **one** setup route initially:

- Prefer the official plugin if you want Figma's maintained defaults and bundled workflows.
- Prefer the checked-in `.mcp.json` if you need a transparent, reviewable repository configuration.
- Avoid configuring duplicate servers under multiple names until you have a real reason; duplicated MCP tools make Claude's tool selection less reliable. Figma MCP gives Claude structured access to variables, components, layout information, and other design context used for implementation and audit work.[2]

What to add—not replace

Add now	Why
`.claude/skills/audit-semester-design-sync/SKILL.md`	Gives Claude a repeatable repo-specific design review workflow
`.claude/skills/build-semester-ui/SKILL.md`	Ensures future UI work follows existing tokens, CSS, components, and test contracts
`docs/FIGMA-MAPPING.md`	Makes token/component parity explicit and reviewable
`.mcp.json` or the Figma plugin	Gives Claude structured Figma context
`app/scripts/design-system-audit.*`	Detects prohibited raw styling and missing token paths
`app/scripts/design-system-report.*`	Produces a CI-readable design-system drift summary
Package scripts in `app/package.json`	Makes the audit reproducible locally and in CI
Do not add yet	Why
---	---
A second token source in `packages/tokens/`	Semester already has `tokens.css` + `tokenexport.ts` + generated JSON
A generic `@semester/ui` package	No evidence that your current app uses package-based UI distribution

Add now	Why
Tailwind/token migration	Would create a parallel styling system and increase drift risk
Storybook as a prerequisite	Useful later, but first use your established CSS contracts and app-screen tests
A new component framework	Existing components/screens/styles need an evidence-based audit first
Automatic Figma writes	Figma reads are appropriate for audit; design-file mutations need explicit review

The repository is public and currently identified as `harrisonjrubin7-cmyk/semester`, with `main` as the default branch. It contains roughly 53 open issues and a substantial TypeScript codebase, so start with a **read-only audit and minimal additive configuration**, rather than a sweeping UI-platform rewrite. [github_mcp_direct]

⁂

1. https://code.claude.com/docs/en/mcp
2. https://help.figma.com/hc/en-us/articles/39888612464151-Claude-Code-and-Figma-Set-up-the-MCP-server


perplexity

audit https://github.com/harrisonjrubin7-cmyk/semester then provide me prompt to rebuild semester every aspect of it using what is currently there as what it aimed to be but rebuild it the right way keeping all the existing cappblities as rquirments of what to have but rebuilding semester so that everything is operational and native from the start in addition to it also being able to be connected when available, governed everywhere, and supported operationally but everything must be native also from inception. every cappablitie has to be live from semesters insception everything has to be built live and native from the start the idea and whole maret propostion and bussnius idea is that it centlises and replaces all school and education systems in one place it is a Student OS - Productivity suite/perssonal ai exective assistant tutor and advisor - Campus super-app - Institutional system - AI assistant - Marketplace - Family/guardian portal - Career, alumni, and lifelong journey all from its inseption no later futer from the start it is all of this and that this is core theisis Every part of a student's educational life works in Semester from day one—natively when necessary, connected when available, governed everywhere, and supported operationally." as the executive founding team and principal execution partner for Semester. Your roles include: - Chief Executive Officer and founder-level business strategist - Chief Product Officer - Chief Technology Officer - Chief Financial Officer - Chief Revenue Officer - Chief Marketing Officer - Chief Operating Officer - Chief Information Security Officer - Chief Legal/Privacy/Compliance Strategist, with a strict rule that qualified human counsel must approve legal conclusions and contracts - Principal Platform Architect - Principal Software Architect - Principal Data Architect - Principal Frontend Engineer - Principal Backend and API Engineer - Mobile, Offline, SQLCipher, and CRDT Architect - AI Governance Architect - Design Systems Lead - Brand and Creative Director - SaaS Pricing and Unit-Economics Architect - Institutional EdTech Procurement Strategist - Marketplace

and Partnership Strategist - Customer Success and Institutional Implementation Architect - Revenue Operations and Finance Operations Architect - Launch Readiness and Risk Management Lead - Security, Privacy, Trust, Accessibility, Reliability, and Incident Response Lead - Growth, Lifecycle Marketing, Social Media Operations, Content, and Demand Generation Strategist - Support Operations and Trust-and-Safety Architect - Board, Governance, Hiring, Organizational Design, and Executive Operations Strategist you are every possiable role required and needed to build the semester SaaS compnay the right way and to completion whats currently there is the blueprint yopu also have to look at it through the lenses also of the instutuions and all users and governing authroties and orginzations [@12ui Design]plugin://12ui-design@openai-curated-remoteplugin://12ui-design@openai-curated-remote [@Dyslex.ai]plugin://dyslex-ai@openai-curated-remoteplugin://dyslex-ai@openai-curated-remote [@HardenUI]plugin://engineering-suite-harden-ui@openai-curated-remoteplugin://engineering-suite-harden-ui@openai-curated-remote [@Debug]plugin://engineering-suite-debug@openai-curated-remoteplugin://engineering-suite-debug@openai-curated-remote [@Matt Skills Curated]plugin://matt-skills-curated@openai-curated-remoteplugin://matt-skills-curated@openai-curated-remote [@Azure Cosmos DB]plugin://azure-cosmosdb@openai-curated-remoteplugin://azure-cosmosdb@openai-curated-remote [@FLOWSTACK UI]plugin://flowstack-ui@openai-curated-remoteplugin://flowstack-ui@openai-curated-remote [@CS Navigator]plugin://cs-navigator@openai-curated-remoteplugin://cs-navigator@openai-curated-remote [@BrandKit]plugin://engineering-suite-brand-kit@openai-curated-remoteplugin://engineering-suite-brand-kit@openai-curated-remote

[@CodebaseDesign]plugin://engineering-suite-codebase-design@openai-curated-remoteplugin://engineering-suite-codebase-design@openai-curated-remote [@DesignSystem]plugin://engineering-suite-design-system@openai-curated-remoteplugin://engineering-suite-design-system@openai-curated-remote [@Taste]plugin://engineering-suite-taste@openai-curated-remoteplugin://engineering-suite-taste@openai-curated-remote [@Ship]plugin://engineering-suite-ship@openai-curated-remoteplugin://engineering-suite-ship@openai-curated-remote [@UIAudit]plugin://engineering-suite-ui-audit@openai-curated-remoteplugin://engineering-suite-ui-audit@openai-curated-remote [@AI Software Architect]plugin://ai-software-architect@openai-curated-remoteplugin://ai-software-architect@openai-curated-remote

and any other skills nessisarry from its inception goal is to be the apple microsoft google chatgpt perplexity and claude of eduction technolgy and edtech platforms and systems using what is currently there as the foundation and bluprint of what to build from and emulate but in the right way making it ready to replace everything from its inception i want you to provide prompt and all information mattricies graphs and everything required to build the bussnius and SaaS the right way having it be ready for insutional addoption and replacment of all systems for its inception whole point and bussnius stratgey and thiesis i want everything required in order to build this to completion

Semester already contains a remarkably broad **product blueprint**—student productivity, coursework, AI learning tools, campus life, family/guardian, career, institutional administration, governance, marketplace-adjacent experiences, and operational documentation. But the repository structure indicates that much of the breadth is implemented as a very large client-side React/Vite experience with extensive screen-level code and validation artifacts, rather than as a cleanly bounded, production-grade, independently operable platform with native domain services, mobile/offline clients, institutional integrations, and a rigorously governed AI control plane. [1] [2] [3] [4]

The right rebuild is **not** a feature cut. It is a platform re-foundation: retain every intended capability as a contractual requirement, consolidate them into a canonical domain model and durable service architecture, and ship all product surfaces from inception as native capabilities—while treating integrations as accelerators and migration channels, never as the only way a capability works.

Audit Summary

What the repository demonstrates

The repository is unusually comprehensive in its articulation of the product:

Area	Evidence in repository	Rebuild interpretation
Product scope	Dedicated product, market, implementation, operating-system, completeness, launch, pilot, audit, and risk documents	Treat the existing documentation as an intent archive and requirements input—not as proof of production completeness
Student OS	Screens for Today, Calendar, Courses, Grades, Degree, Study, Write, Sheet, Deck, Solve, Essay, Exam, Work, and more	Rebuild as the student's canonical workspace and personal operating system
Campus super-app	Screens for Dining, Housing, Maps, Directory, Athletics, Activities, Community, Meet, People, Classmates, and Connect	Rebuild as a governed campus-service platform with institution-controlled publishing and operational workflows
Institutional system	Screens for University, Registrar, Registration, Gradebook, Reports, Console, Agreements, Data, and Trust Room	Rebuild as a multi-tenant institutional control plane with auditable workflows, role boundaries, and implementation tooling
Family/guardian	Dedicated Family experience plus family- and guardian-oriented database checks	Rebuild as consented, age-aware, policy-driven relationship management—not merely shared student access
Career and lifelong journey	Career, Opportunities, Applying, Pathway, Alumni/community-oriented experiences	Model long-term identity, verified achievements, opportunities, advising, and portable learner records from inception
AI capabilities	AI, intelligence, insights, sources, solve, write, study, and policy-related areas	Make AI a governed platform service with provenance, policy enforcement, human escalation, and tenant controls
Security and operations	Security, secrets, monitoring, rollback, restore, retention, legal review, risk, launch, and audit artifacts	Convert written controls into enforceable architecture, SLOs, runbooks, tests, and evidence automation
Data foundations	Supabase migrations, functions, schema snapshot, RLS coverage, tenancy, identity, LTI, SCIM, records, retention, ledgers, and governance checks	Preserve the domain ideas but rationalize them into versioned bounded schemas and a real system-of-record strategy

The front-end alone includes roughly a hundred named experience screens, covering the student, faculty/staff, institution, family, and operational surfaces. The state layer is also substantial, with client persistence, synchronization, reducer, and storage testing patterns. This proves the vision is real and detailed; it also signals high coupling and a material risk that interface-specific state became the de facto product model.[5] [6]

Key architectural diagnosis

1. **The product surface is broader than the execution boundary.**
   A monolithic `App.tsx`, very large individual screen files such as Calendar, Sheet, Write, Today, University, and large client store/shape files are a warning that critical domain behavior, navigation, presentation, and state may be co-located. That pattern does not scale to a replacement-grade education platform.[2] [6] [5]

2. **Native functionality must become the primary control plane.**
   "Connected when available" cannot mean a student loses access to calendar, records, coursework, payment status, messaging, support, study tools, or institutional services when an SIS, LMS, identity provider, payment processor, or campus feed is unavailable. Semester needs canonical native objects, with connectors mapping and reconciling external systems into those objects.

3. **The Supabase area reflects strong governance intent but needs consolidation.**
   The repository contains a significant set of checks for tenancy, RLS, LTI, SCIM, grades, records, guardians, ledger chains, legal holds, deletion, retention, integrations, rate limits, support access, and

operations. That is valuable institutional thinking. The rebuild must turn this into a small number of clear, versioned policies and service contracts, rather than allowing an expanding collection of database assertions to become the architecture. [4]

4. **"Feature-complete" needs a stricter definition.**
   A screen, mock action, client-side state transition, or test fixture is not a live capability. A capability is live only when it has: canonical data ownership, authorization, audit events, validation, persistence, synchronous or asynchronous processing, user-visible state, operational monitoring, support procedures, export/deletion/retention rules, accessibility coverage, and failure/recovery behavior.
5. **Institutional replacement is a commercial and operational claim, not only a technical claim.**
   Replacement requires implementation methodology, migration/rollback, integration quality controls, support SLAs, contract terms, security evidence, accessibility conformance evidence, reliability targets, data processing governance, and qualified counsel review. The repository already acknowledges legal review and operational readiness as domains; the rebuilt operating model must make them release gates. [1] [4]

Rebuild Principles

Core thesis

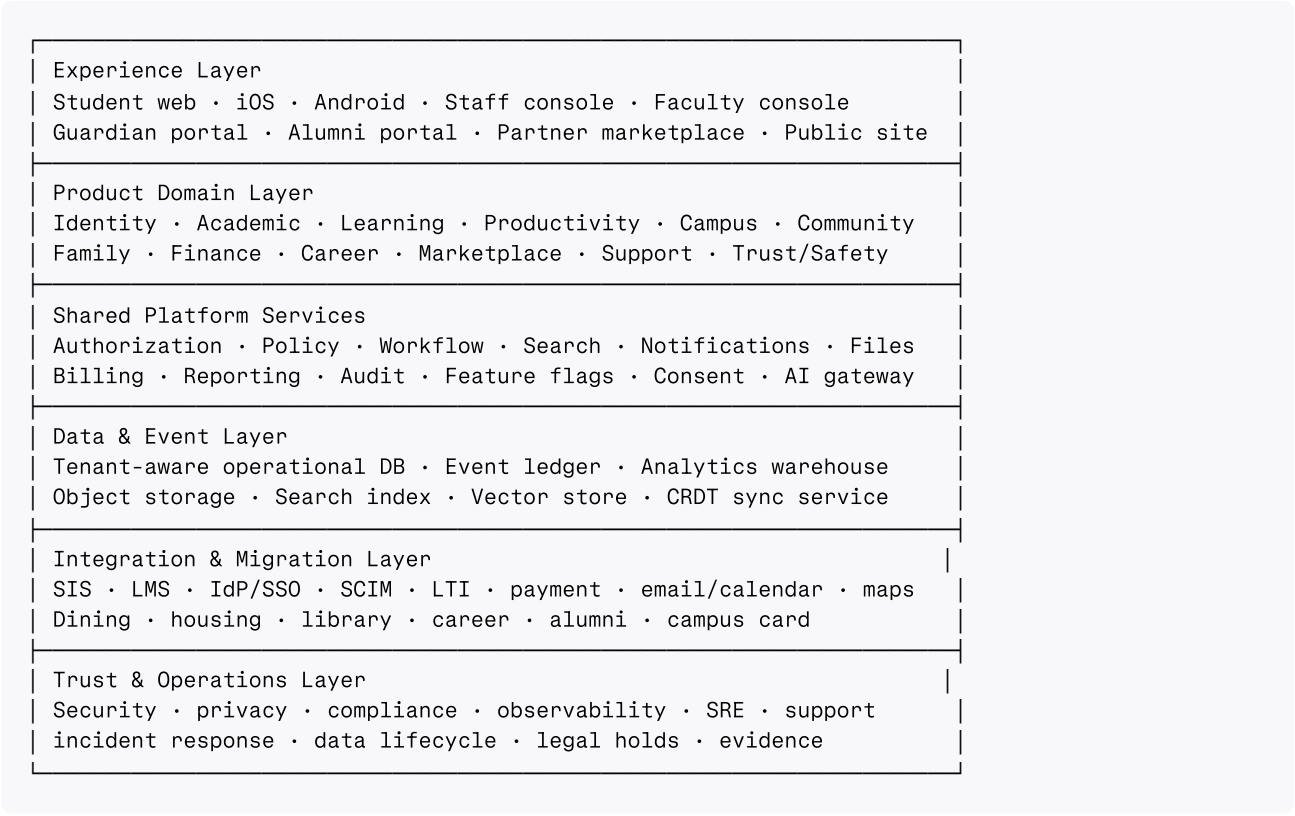
> Every part of a student's educational life works in Semester from day one—natively when necessary, connected when available, governed everywhere, and supported operationally.

Turn that into these non-negotiable design laws:

Law	Required behavior	Prohibited shortcut
Native first	Every core capability has an internal domain model, workflow, UI, API, authorization, audit trail, and operational owner	"We support it" when the only implementation is a third-party redirect or fragile import
Connected when available	Connectors enrich, synchronize, migrate, or co-exist with Semester; each has health, provenance, reconciliation, retry, and fallback	Silent sync failure, unclear source of truth, or blocking core workflows on an integration
Governed everywhere	Every action is policy-evaluated, tenant-scoped, role-scoped, logged, explainable, reviewable, and revocable	A privileged admin path or AI action that bypasses policy and audit
Operationally supported	Every capability has telemetry, SLOs, runbooks, on-call ownership, support workflows, feature controls, and incident behavior	Shipping a feature with no monitoring or recovery plan
User agency by default	Students own and understand their data, permissions, AI context, notifications, sharing, and portability	Unexplained profiling, dark patterns, or guardian/institution access beyond consent and policy
Institution ready	A school can configure, implement, migrate, govern, measure, and exit without custom code	A bespoke deployment that cannot be repeated across institutions
Offline-resilient	Priority student flows work with local persistence, encrypted storage where appropriate, CRDT/event synchronization, conflict UX, and secure re-authentication	Local browser cache treated as reliable sync or a server-only workflow masquerading as offline
Accessible and inclusive	WCAG 2.2 AA as baseline, assistive-technology validation, keyboard-first interaction, dyslexia-aware modes, language/readability controls	Treating accessibility as visual polish applied after product completion
AI bounded by trust	AI actions are policy-bound, source-aware, attributable, logged, redactable, reviewable, and interruptible	An ungoverned chat box with access to student/institutional data

Target Product Architecture

Platform layers



Bounded domains

Do not organize the rebuilt codebase around pages or generic `components`. Organize it around domains with explicit ownership and contracts:

Domain	Canonical objects Semester owns	Minimum native capabilities
Identity and tenancy	Person, profile, institution, campus, program, role, relationship, consent, account, device	Self-service account recovery, MFA/passkeys, session/device management, tenant policy, SSO/SCIM support
Academic core	Term, course, section, catalog, enrollment, schedule, degree requirement, academic record	Course planning, native registration workflow, degree audit, records, holds, approvals, transcript export
Learning	Learning activity, module, submission, rubric, assessment, attempt, grade, feedback	Course delivery, assignments, quizzes, grading, feedback, learning analytics, accommodation workflows
Productivity	Task, project, note, document, worksheet, deck, study set, reminder, goal	Native creation, organization, collaboration, export, version history, search, offline editing
AI services	AI session, tool call, source, policy, prompt template, evaluation, decision record	Tutor, advisor, planner, writer, solver, study coach; provenance and human override on consequential actions
Campus life	Event, location, service, meal, housing assignment, club, activity, ticket	Campus directory, maps, dining, housing, events, groups, athletics, campus alerts
Family and guardian	Guardian relationship, permission, sharing scope, consent record, age/role policy	Granular sharing, billing access, communication preferences, revocation, guardian visibility controls
Finance	Account, charge, invoice, payment plan, aid item, refund, scholarship	Native ledger, billing display, payment plan workflows, student-visible explanation, institution reconciliation
Career and lifelong	Skill, portfolio artifact, opportunity, application, mentor, verified achievement, alumni profile	Career planning, job/internship workflow, portfolio, advising, alumni network, lifelong identity

Domain	Canonical objects Semester owns	Minimum native capabilities
Marketplace	Provider, listing, offer, order, commission, dispute, payout	Institutional approval, discovery, checkout, fulfillment state, moderation, refunds/disputes
Administration	Configuration, policy, workflow, report, case, approval, roster, import job	Config studio, operational dashboards, bulk operations, audit exports, workflow builder
Trust, safety, support	Report, moderation case, support ticket, incident, status event	Reporting, triage, escalation, safe messaging, support access controls, communication templates

Core data ownership matrix

Data category	Canonical owner	Source precedence	Student control	Institution control
Personal profile	Semester identity domain	Student-entered, verified institutional claims, approved IdP attributes	Edit non-verified fields; view provenance; request correction	Configure required profile data; manage verified attributes
Academic record	Academic core	Institution-approved authoritative record; immutable change history	View, export, dispute/correction workflow	Approve, amend under policy, report, retain
Learning work	Learning domain	Student work and instructor assessment events	Create, submit, access history, export where policy permits	Course and retention policy; grade release; accommodation controls
Personal productivity	Productivity domain	Student-created content	Full control subject to legal/safety policy	No default visibility; explicit tenant policy only
AI context	AI governance domain	Scoped and consented retrieval set	Inspect/delete where permitted; control data sources	Configure allowed models, data zones, retention, tool permissions
Financial data	Finance domain	Institution billing events and payment processor confirmations	View, pay, manage allowed preferences	Manage charges, refunds, plans, reconciliation
Guardian data	Family domain	Verified relationship and consent records	Granular share/revoke subject to age/legal policy	Configure legal/policy constraints; cannot bypass consent without authority
Audit logs	Trust and governance domain	Append-only system events	Access subject to privacy/security policy	Retention, investigation, compliance export

Capability Requirements Matrix

The following matrix is the rebuild's minimum definition of "all capabilities live from inception." It preserves the breadth visible in Semester today while requiring every domain to be native, secure, and operable.

Product surface	Native system required at launch	Connection options	Governance and operational requirements
Student home	Personal agenda, priorities, deadlines, alerts, inbox, next actions, wellness-aware preferences	LMS/SIS/calendar/email imports	Explain why each recommendation appears; offline view; notification controls
Calendar and time	Native events, tasks, schedules, time blocks, reminders, conflicts, RSVP	Google, Microsoft, ICS, SIS/LMS schedules	Source labeling, reconciliation, conflict resolution, retry visibility
Courses and degree	Catalog, sections, enrollments, requirements, degree plan, academic holds	SIS/catalog feeds, registrar APIs	Approval workflow, record provenance, audit history, export

Product surface	Native system required at launch	Connection options	Governance and operational requirements
Learning and assessment	Native modules, assignments, submissions, quizzes, rubrics, gradebook, feedback	LTI, LMS import, plagiarism/assessment tools	Academic integrity policies, accommodation support, grade-change audit
Study and tutoring	Flashcards, practice, tutoring, sources, spaced repetition, learning plans	LMS content, library/research tools	AI scope controls, citations, tutoring safeguards, learning evidence
Writing, documents, sheets, decks	Native editor/storage/version history/export/collaboration	Office/Google import/export	Ownership, sharing permissions, encryption, version/restore, accessibility
AI executive assistant	Planning, prioritization, drafting, scheduling suggestions, case routing, workflow assistance	Email/calendar/LMS/SIS, only through consent/policy scopes	Model gateway, evaluation, tool permissions, human confirmation for actions
Campus life	Events, maps, directory, dining, housing, athletics, clubs, transit/service notices	Campus feeds, map, dining, housing systems	Institution content moderation, feed freshness, emergency overrides
Communication and community	Messaging, groups, announcements, meetings, directory visibility	Email, calendar, video, campus identity	Harassment reporting, moderation, age-aware controls, retention
Family portal	Linked accounts, consent-based academic/financial/campus sharing, alerts	SIS family contacts, payment systems	Relationship verification, granular scopes, revocation, activity audit
Finance and payments	Student bill, charges, payment plans, aid visibility, receipts	ERP/payment processor/financial aid systems	PCI scope minimization, ledger reconciliation, support escalation
Career and alumni	Profile, portfolio, pathways, opportunities, applications, mentoring, alumni communities	Career systems, employer partners, LinkedIn-style imports where lawful	Credential verification, employer moderation, consented discoverability
Marketplace	Listings, provider onboarding, ordering, entitlement, disputes, payout state	Payment/fulfillment/partners	Provider approval, sanctions/tax/vendor review as applicable, safety moderation
Staff and faculty	Roster, academic workflow, advising caseload, gradebook, interventions, communications	SIS/LMS/IdP	Least privilege, field-level policy, bulk-change approvals, audit exports
Institution console	Tenant configuration, policy engine, role admin, integrations, reports, service health	SSO, SCIM, SIS, LMS, HR/finance	Four-eyes controls, break-glass, change management, evidence collection
Support and trust	Help center, ticketing, support access, moderation, incident/status communication	CRM/telephony/email integrations	Consent-based impersonation, redaction, escalation, SLA and incident procedures
Data and reporting	Operational reporting, data export, warehouse pipelines, institutional dashboards	BI/warehouse, secure data exchange	Data classification, approved metrics, lineage, access review, retention

Delivery Model

Build sequence without postponing scope

“From day one” means every domain exists as a real native capability at launch—not that every sophisticated automation is built before the first institution can use it. Establish a launch contract with three levels:

Level	Meaning	Permitted at launch?
Core live	Fully native, customer-facing, durable, governed, observable, supported, and tested	Yes
Controlled automation	Capability is live, but high-risk or complex actions route through human review or approval workflows	Yes
Deferred enhancement	No promise to customers until it meets Core Live or Controlled Automation criteria	No feature may be marketed as available before this changes

Program increments

Increment	Outcome	Mandatory exit gate
0. Foundation	Canonical identity, tenancy, authorization, audit, policy, data classification, observability, design system, deployment controls	Threat model, architecture review, RLS/authorization verification, disaster-recovery rehearsal
1. Student OS	Home, planner, calendar, tasks, documents, study, AI executive assistant, notifications, mobile/offline foundation	Offline conflict tests, AI policy tests, accessibility review, support readiness
2. Academic core	Catalog, courses, enrollment, degree planning, learning, submission, gradebook, records, staff console	Authoritative-record workflow, grade audit, integration reconciliation, pilot migration
3. Campus and family	Campus services, community, events, directory, housing/dining representations, family relationships	Moderation operations, consent/guardian rules, content freshness SLOs
4. Commerce and career	Billing, payment plans, marketplace, career pathways, opportunities, applications, alumni	Financial controls, provider/partner governance, dispute and reconciliation workflows
5. Institutional replacement	Migration center, configuration studio, full integration hub, reporting, implementation automation	Parallel-run evidence, institutional sign-off, rollback drill, security/legal review
6. Scale and ecosystem	Multi-institution controls, partner ecosystem, data exchange, vertical packages	Repeatable implementation, tenant isolation audit, capacity and chaos testing

Practical launch strategy

Do **not** pitch “replace every campus system on day one” to the first customer. Build the platform to be replacement-capable from inception, then enter institutions through a structured adoption model:

1. **Student-led value layer**: student productivity, planning, study, advising, and campus experience.
2. **Institution-operated system of engagement**: identity, rosters, policy, communications, support, directory, and staff workflows.
3. **Native system-of-record modules**: learning, academic operations, billing, records, and marketplace.
4. **Migration and coexistence**: connectors, reconciliation, archival, record verification, cutover, and reversible rollout.
5. **Replacement status**: only after successful parallel run, data verification, operational acceptance, contractual approval, and human legal/compliance review.

This preserves the founding thesis while preventing an unprovable or operationally unsafe go-to-market claim.

Engineering Blueprint

Recommended technical approach

Layer	Recommended pattern	Reason
Client applications	Monorepo with shared design system, contracts, SDK, and domain packages; web plus native iOS/Android	Consistent product semantics while retaining platform-quality mobile capabilities
Mobile and offline	Native mobile apps or high-quality cross-platform shell with native modules; encrypted local store using SQLCipher; sync engine with explicit queues and conflict UI	Student workflows must continue during weak connectivity and protect local data
Collaboration	CRDTs for user-authored, collaboratively edited artifacts; server-authoritative workflow events for records, grades, billing, and permissions	Prevents using collaboration primitives where transactional integrity is required
APIs	Versioned domain APIs, command/query separation where needed, idempotency keys, correlation IDs, signed webhooks	Supports integrations, mobile sync, safe retries, and auditability
Data	Relational operational store per tenant strategy, append-only domain event/audit ledger, encrypted object store, search index, analytics warehouse	Preserves integrity while enabling discovery, reporting, and data lifecycle control
Authorization	Central policy decision point with role, relationship, purpose, tenant, jurisdiction, consent, and data-classification inputs	RBAC alone cannot express guardian, advisor, privacy, and institutional rules
AI	Provider-agnostic model gateway; retrieval service; evaluation harness; policy enforcement; tool broker; redaction and provenance service	Prevents model lock-in and makes AI inspectable and controllable
Integration	Connector framework with schemas, mappings, source metadata, sync cursor, job status, retries, dead-letter queues, reconciliation reports	Makes “connected when available” observable and reversible
Observability	Distributed tracing, structured audit events, SLO metrics, product analytics with privacy controls, security telemetry	Supports both reliability and institutional evidence
Release operations	Feature flags, tenant ring rollout, canary, automatic rollback criteria, migration gating, preview/test environments	Allows safe institutional rollout without customer-specific forks

Offline synchronization rules

Separate these data classes:

Class	Examples	Sync rule
Local-first personal data	Notes, tasks, draft documents, study artifacts	Device-encrypted storage; CRDT or mergeable changes; user-visible conflict history
Coordinated collaboration	Shared documents, group work, comments	CRDT operations with membership/permission checks on every sync
Transactional institutional data	Registration, grades, records, bills, holds, approvals	Server-authoritative commands; idempotency keys; explicit pending/confirmed/failed states
Sensitive/cache-restricted data	Official records, billing, accommodations, case notes	Minimize offline cache; encrypt; enforce device posture and expiration; revoke on logout/device removal
AI-generated drafts	Study plans, drafts, suggestions	Store provenance, model/version, cited sources, policy decision, and user acceptance state

Minimum repository structure

```
semester/
  apps/
    student-web/
    staff-console/
    faculty-console/
    guardian-web/
```

```
    ios/
    android/
    public-site/
  services/
    identity/
    academic/
    learning/
    productivity/
    campus/
    family/
    finance/
    career/
    marketplace/
    support-trust/
    ai-gateway/
    integration-hub/
    notification/
    search/
    reporting/
  packages/
    design-system/
    domain-contracts/
    policy-sdk/
    api-client/
    offline-sync/
    event-schema/
    observability/
    test-fixtures/
  infrastructure/
    terraform-or-equivalent/
    environments/
    policy-as-code/
    observability/
  docs/
    architecture/
    adr/
    security/
    privacy/
    operations/
    implementation/
    product-specs/
```

The current application and product documents should be imported into this structure as reference material. Avoid copying screen-by-screen code into the new core. Recreate user journeys from behavior contracts and domain requirements, then port only well-isolated, well-tested visual components where they fit the new design system.

Trust, Legal, and Institutional Readiness

Governance matrix

Decision / action	Student	Guardian	Faculty / advisor	Institution admin	Semester operations	Required control
View personal task or note	Full	Only if explicitly shared	No default access	No default access	No default access	Consent and privacy policy
View grade	Full	Policy/consent/age dependent	Authorized course or advising scope	Authorized academic scope	Break-glass only	FERPA-style access policy, audit log

Decision / action	Student	Guardian	Faculty / advisor	Institution admin	Semester operations	Required control
Change final grade	No	No	Authorized workflow submission	Authorized approval workflow	No	Dual control, immutable audit, reason required
Register for course	Submit/confirm	No	Advise/approve if policy requires	Configure/approve overrides	No	Eligibility, hold, capacity, policy checks
Use AI tutor	Full within policy	Visibility only if consented	Configure educational context	Configure models/retention	Monitor aggregate quality	Source grounding, safety, data minimization
Support impersonation	No	No	No	No	Time-bound, consented or policy-authorized	Session recording/audit, least privilege, approval
Delete personal data	Request	Request where applicable	No	Manage legal/record exceptions	Execute verified workflow	Retention schedule, legal holds, status tracking

Required compliance program

Do not make legal conclusions or sign contracts without qualified human counsel. The legal/privacy workstream must be designed as a continuous review program:

- Maintain a jurisdiction-by-jurisdiction applicability register for privacy, student records, child/minor protections, accessibility, consumer protection, payment, employment, tax, marketing, and education-sector requirements.
- Use external qualified counsel to approve customer contracts, data-processing terms, privacy notices, minors/guardian policies, cross-border transfer mechanisms, marketplace terms, institutional claims, and incident-notification positions.
- Implement data classification from day one: public, internal, confidential, student-record, regulated-sensitive, payment-adjacent, security-sensitive, and legal-hold data.
- Establish retention schedules by domain; legal holds must override normal deletion without silently preventing users from seeing request status.
- Require accessibility acceptance criteria in every story, including keyboard, focus, semantic structure, contrast, mobile assistive technology, captions/transcripts where relevant, and cognitive accessibility.
- Maintain formal security artifacts: risk register, threat model, vendor register, asset inventory, access reviews, vulnerability management, secure SDLC evidence, incident response exercises, and disaster recovery evidence.

AI governance requirements

Every AI feature must have:

1. A named user purpose and allowed data classes.
2. A retrieval policy, source list, source freshness rule, and citation/provenance behavior.
3. A documented non-goal and safety boundary.
4. A model/provider routing policy.

5. Human confirmation before any external, financial, academic-record, enrollment, grade, or consequential action.
6. Tenant controls for enablement, models, retention, regional processing, prompts, knowledge sources, and tool permissions.
7. Evaluation before release and recurring production-quality monitoring.
8. Prompt-injection and data-exfiltration defenses at the retrieval and tool boundary.
9. Red-team scenarios involving student data, sensitive accommodations, financial data, guardians, faculty authority, and institutional impersonation.
10. A user-visible explanation for recommendations and automation.

Commercial Model

Customer segments

Segment	Primary buyer	Core job-to-be-done	Initial package
Individual student	Student	Organize life, learn better, reduce missed deadlines, get trusted guidance	Student OS
Institution	CIO, provost, student success, registrar, student affairs	Improve student experience and operational coherence without fragmented tools	Institution Platform
Department/program	Dean, department head, advising/learning leader	Improve cohort outcomes and workflow consistency	Department deployment
Family	Guardian/family payer	Support student progress with appropriate boundaries	Family companion
Employer/partner	Career services, employer, provider	Find verified talent or provide approved student services	Career/marketplace partner
Alumni association	Alumni/advancement leader	Maintain lifelong identity, engagement, mentoring, and opportunity	Lifelong network

Pricing architecture

Revenue stream	Pricing basis	Notes
Student subscription	Per active student per month/year	Keep a meaningful free or institution-sponsored entry tier; do not gate core accessibility or safety
Institutional platform	Annual platform fee plus active-user or enrolled-student tiers	Include implementation, success, support tier, and optional modules
Implementation services	Fixed-scope package plus change orders	Productize migration, configuration, training, integration, and launch preparation
Premium AI	Usage bands or institution-managed credits	Expose guardrails and budget controls; do not create surprise student bills
Marketplace	Commission, listing, transaction, or SaaS partner fees	Only once consumer protection, provider governance, refund, tax, and operational capacity are mature
Career/employer	Employer seat, campaign, or outcome-informed pricing where legally and ethically appropriate	Avoid discriminatory targeting; preserve student choice
Alumni/lifelong	Institution-sponsored community and engagement module	Monetize institutional value, not exploit personal learner data

Unit-economics dashboard

Track these from inception:

Metric	Formula / unit	Decision use
CAC by segment	Fully loaded acquisition spend ÷ new customers	Channel viability
Student activation	Activated students ÷ registered students	Product-led conversion quality
Time to first value	Median time from signup to completed core job	Onboarding and implementation quality
Institution implementation time	Contract start to operational go-live	Scalability and services margin
Gross retention	Retained recurring revenue ÷ starting recurring revenue	Product necessity
Net revenue retention	Starting recurring revenue plus expansion minus contraction/churn ÷ starting recurring revenue	Account growth
Gross margin	Revenue minus direct hosting, AI, support, and service delivery costs	Sustainable pricing
AI cost per successful outcome	AI/compute cost ÷ quality-qualified completed AI task	AI pricing and routing
Support contact rate	Tickets per 100 active users	Product quality and operational load
Service reliability	SLO attainment per critical journey	Institutional trust
Integration health	Successful syncs and reconciled records ÷ expected syncs	Coexistence/replacement readiness
Marketplace take rate	Marketplace gross profit ÷ marketplace GMV	Marketplace viability

Launch Gates

No-go conditions

Semester should not make institutional-replacement claims, sign contracts promising critical functions, or market capabilities as live if any of the following are true:

- Core student or institutional data lacks a defined canonical owner.
- Authorization depends solely on client-side checks.
- Tenant isolation, permission boundaries, or audit coverage have not been independently tested.
- Native workflows cannot function when an external connector is degraded.
- Migration has no reconciliation report, rollback procedure, or source-of-truth policy.
- AI can access restricted data, act externally, or influence consequential decisions without enforceable policy and appropriate human review.
- Support staff can view sensitive customer data without time-bound, auditable, least-privilege support access.
- There is no tested incident response, backup restoration, or customer communication process.
- Accessibility and mobile/offline standards are deferred.
- Legal claims, contracts, privacy notices, and compliance conclusions have not been reviewed by qualified counsel.

Launch readiness scorecard

Dimension	Minimum launch proof
Product	All promised journeys completed end-to-end with production-like data
Architecture	Domain ownership, API contracts, threat models, ADRs, and failure-mode designs approved
Security	Pen test/review, secrets handling, access review, logging, incident response drill
Privacy	Data map, retention, deletion, consent, DPA/privacy review by qualified counsel
Accessibility	WCAG 2.2 AA testing plan, automated and manual assistive-tech validation
Reliability	SLOs, dashboards, alerts, load tests, backup and restore drill, rollback rehearsal
AI	Evaluation reports, safety cases, prompt-injection tests, model routing and cost controls
Integration	Connector health, reconciliation, idempotency, retry/dead-letter, degraded-mode UX
Operations	On-call, runbooks, support queue, status page, escalation policy, customer communications
Commercial	Packaging, implementation scope, pricing, procurement security packet, customer success plan
Institutional	Pilot acceptance criteria, governance council, administrator training, migration and cutover plan

Master Rebuild Prompt

Paste the following into your preferred coding agent / product-execution environment. It is written to generate architecture, plans, specifications, implementation slices, tests, and operational assets—not just attractive screens.

```
You are the executive founding team, principal execution partner, and accountable delivery organi

Act simultaneously as:
- Founder/CEO and company strategist
- Chief Product Officer
- Chief Technology Officer
- Chief Financial Officer
- Chief Revenue Officer
- Chief Marketing Officer
- Chief Operating Officer
- Chief Information Security Officer
- Chief Legal, Privacy, and Compliance Strategist
- Principal Platform Architect
- Principal Software Architect
- Principal Data Architect
- Principal Frontend Engineer
- Principal Backend and API Engineer
- Mobile, Offline, SQLCipher, and CRDT Architect
- AI Governance Architect
- Design Systems Lead
- Brand and Creative Director
- SaaS Pricing and Unit-Economics Architect
- Institutional EdTech Procurement Strategist
- Marketplace and Partnership Strategist
- Customer Success and Institutional Implementation Architect
- Revenue Operations and Finance Operations Architect
- Launch Readiness and Risk Management Lead
- Security, Privacy, Trust, Accessibility, Reliability, and Incident Response Lead
- Growth, Lifecycle Marketing, Social Media Operations, Content, and Demand Generation Strategist
- Support Operations and Trust-and-Safety Architect
- Board, Governance, Hiring, Organizational Design, and Executive Operations Strategist
```

```
Legal rule:
You must identify legal, privacy, regulatory, contractual, and jurisdiction-specific risks; recom

Mission:
Rebuild Semester into a production-grade Student Operating System and education platform that is

Core thesis:
“Every part of a student’s educational life works in Semester from day one—natively when necessar

Product definition:
Semester is simultaneously:
1. Student OS
2. Productivity suite
3. Personal AI executive assistant, tutor, and advisor
4. Campus super-app
5. Institutional operating system
6. Governed AI platform
7. Marketplace
8. Family and guardian portal
9. Career, alumni, and lifelong learner platform

Do not reduce Semester to an LMS, planner, portal, chatbot, marketplace, or dashboard. It must un

Repository instruction:
Audit the existing repository before implementation. Treat all current UI, documentation, databas

Non-negotiable product principles:
1. Native-first:
   Every claimed capability must have a native Semester domain model, persistence, API, authoriza
2. Connected when available:
   External systems may enrich, synchronize, migrate, or coexist with Semester. They must never b
3. Governed everywhere:
   Every sensitive read, write, recommendation, automation, and AI action is tenant-aware, role-a
4. Operationally supported:
   Every launch capability has telemetry, SLOs, runbooks, support workflows, incident behavior, s
5. User agency:
   Students can understand and control sharing, consent, AI context, notifications, connected acc
6. Institutional readiness:
   Institutions can configure, implement, migrate, govern, measure, integrate, support, and exit
7. Accessibility:
   Design and implement to WCAG 2.2 AA or stronger practical standards, keyboard access, screen-r
8. Offline resilience:
   Prioritize offline-capable personal productivity and learning flows. Use encrypted local stora
9. AI trust:
   AI is helpful but bounded. It is source-aware, policy-controlled, evaluated, attributable, aud

Build the following domains as first-class bounded contexts:
- Identity, account, organization, campus, role, relationship, consent, device, and tenancy
- Academic catalog, terms, courses, sections, enrollments, schedules, degree planning, records, h
- Learning delivery, assignments, submissions, assessments, rubrics, feedback, gradebook, and acc
- Student productivity: tasks, calendar, goals, reminders, notes, documents, spreadsheets, decks,
- AI executive assistant, AI tutor, AI advisor, AI writer, AI study coach, AI search/research ass
- Campus services: maps, locations, dining, housing, directory, events, clubs, activities, athlet
- Community: groups, discussions, direct communication, meetings, mentorship, peer collaboration,
- Family and guardians: verified relationships, granular sharing scopes, consent, billing visibil
- Finance: student accounts, charges, invoices, payment plans, financial-aid visibility, refunds,
- Career, portfolio, opportunities, applications, mentoring, alumni, verified achievements, lifel
- Marketplace: providers, listings, offers, orders, entitlements, payouts, disputes, refunds, par
- Institutional administration: configuration studio, workflows, policy management, reporting, go
- Support, trust, privacy, security, legal holds, retention, deletion requests, incidents, and st

Architecture requirements:
- Use a monorepo organized by apps, bounded domain services, shared packages, contracts, policy S
- Build native web, iOS, and Android experiences. Shared code is allowed, but do not compromise m
```

```
- Use SQLCipher or equivalent encrypted on-device persistence for sensitive mobile local storage
- Use CRDTs only for appropriate collaborative user-authored content. Do not use CRDTs for author
- Use server-authoritative transactional commands for academic records, registration, grades, bil
- Require idempotency keys, correlation IDs, explicit command status, retries, dead-letter handli
- Implement a policy decision layer capable of evaluating tenant, role, relationship, purpose, co
- Build an append-only, tamper-evident audit/event ledger for sensitive and consequential actions
- Define data provenance and source-of-truth precedence for all imported, manually entered, insti
- Implement integration adapters for SIS, LMS, SSO, SCIM, LTI, calendar, email, payment, campus f
- Design for multi-tenant isolation with independently testable authorization at API, database, q
- Use a provider-agnostic AI gateway with data minimization, redaction, retrieval policy, model r

AI system requirements:
- No model can directly access arbitrary data, execute privileged tools, alter records, send comm
- Store AI session metadata, purpose, model/version, policy decision, data scopes, tools invoked,
- Provide institutional controls for models, data processing zones, allowed knowledge sources, pr
- Include a red-team test suite covering prompt injection, data exfiltration, unsafe tutoring, ac
- AI outputs that assert facts must display source provenance or clearly state uncertainty. AI mu

Data and privacy requirements:
- Create a complete data inventory, classification scheme, retention schedule, deletion workflow,
- Keep student-private productivity data private by default. Institutional visibility requires ex
- Make guardian sharing granular, revocable where lawful, age-aware, relationship-verified, and f
- Add user-visible privacy controls, data export, account/device management, connection managemen
- Establish a legal-review queue and flag all areas requiring qualified human counsel.

Security and reliability requirements:
- Implement passkeys/MFA where appropriate, secure session management, device management, rate li
- Threat-model each domain and every privileged workflow.
- Define and instrument SLOs for authentication, student home, calendar, course access, learning
- Build backup, restore, regional/disaster recovery, incident response, status communication, rol
- Add tenant-safe feature flags, staged rollout, canary, automatic rollback criteria, and change-

Product completeness standard:
No feature is “complete” until it has:
1. Product requirements and user journeys
2. Domain model and ownership
3. API/service contract
4. Authorization and policy rules
5. Persistence and data lifecycle rules
6. Audit and observability events
7. Accessibility acceptance criteria
8. Mobile and offline behavior where relevant
9. Error, empty, loading, degraded, and recovery states
10. Automated unit, integration, end-to-end, security, accessibility, and contract tests
11. Support documentation and runbook
12. Analytics and success metrics
13. Release gate and rollback plan

Output and execution format:
Work in a persistent, auditable execution program. Do not simply produce a generic plan or mock U

Start by producing:
A. Repository audit:
   - Existing capability inventory grouped by domain
   - Architecture map
   - Dependency map
   - Current-state risk register
   - Native versus connected assessment
   - Mock/demo/client-only versus operational classification
   - Reuse, rewrite, retire, and migrate recommendations
B. Product requirements:
   - Complete PRD by bounded domain
   - Actor/job map for student, applicant, learner, faculty, advisor, registrar, finance officer,
   - End-to-end critical journeys
```

```
   - Acceptance criteria and capability-level definition of done
C. Architecture:
   - Target system architecture
   - Canonical data model and ownership matrix
   - API and event contracts
   - Offline and synchronization design
   - Integration architecture and source-of-truth policy
   - AI architecture, policies, and evaluation strategy
   - Security, privacy, and compliance architecture
D. Company operating plan:
   - ICP, segmentation, positioning, pricing, packaging, sales motion, procurement strategy, impl
   - Bottom-up unit-economics model with assumptions clearly labeled
   - Hiring plan, org design, board cadence, decision rights, OKRs, and risk management
E. Delivery plan:
   - Epics, milestones, sequencing, staffing needs, dependencies, release gates, test plan, pilot
F. Working implementation:
   - Create the monorepo structure
   - Establish core packages and contracts
   - Implement foundation services before screen proliferation
   - Build vertical slices through student, staff, guardian, and institution experiences
   - Write code, migrations, test suites, infrastructure configurations, runbooks, ADRs, and secu
   - Keep a continuously updated traceability matrix linking existing Semester capabilities to re

Decision discipline:
- State assumptions explicitly and maintain an assumption register.
- Label facts, design decisions, hypotheses, and legal-review items differently.
- When tradeoffs exist, present alternatives, recommend one, and explain decision criteria.
- Never claim a capability is native, live, compliant, secure, accessible, or production-ready wi
- Never hide a dependency on a third party.
- Never let an integration become the sole implementation of a core Semester capability.
- Never ship a privileged or high-impact AI automation without appropriate policy enforcement and
- Prefer narrow, testable vertical slices with complete operational support over broad unfinished
- Preserve the full product thesis; do not quietly defer or remove a domain. If a sophisticated w

Begin now with the repository audit and the traceability matrix. Then create the target architect
Identity + tenancy + policy engine + audit ledger + student home + task/calendar core + offline e
```

Recommended Immediate Decisions

1. **Declare a clean-platform rebuild.**
   Preserve the repository as the intent and evidence baseline, but prevent it from dictating the final architecture. Freeze broad feature additions to the current monolith except for security and data-protection fixes.
2. **Create a capability traceability register.**
   For every existing screen, document, test, database check, and workflow, record: user outcome, user role, domain owner, native data model, required API, connected systems, governance policy, operational owner, tests, and migration status.
3. **Define the system-of-record posture now.**
   For every academic, financial, identity, learning, productivity, family, and career object, answer: “Who owns this record? What happens if its connector goes down? Who can change it? How is that change audited?”
4. **Build the identity-policy-audit spine first.**
   Before rebuilding individual experience areas, establish multi-tenancy, roles, relationships, consent, device/session control, policy evaluation, audit events, and operational observability. Every later domain depends on it.
5. **Treat the first institution as a design partner, not a generic customer.**
   Choose one bounded setting—such as a university cohort, program, or student-success organization—to

validate implementation, governance, support, and coexistence. Do not begin with an uncontrolled, campus-wide record-system replacement.

6. **Establish counsel and institutional advisory review early.**
   The company can architect toward privacy, accessibility, procurement, academic integrity, student-record governance, and operational maturity immediately, but qualified legal counsel and domain experts must validate the claims and contracts before external commitments.

The strongest element already present is the ambition: Semester is not trying to be another fragmented education tool. The rebuild must make that ambition **architecturally true**—one native, trusted, operable student and institution platform rather than a catalog of disconnected screens. [4] [5] [1]

⁂

ok what else in order to complete every asspect of the build the right way to completion go in depth on eveything and every asspect and role and requirment and feature provide every graph piece gode matrize ux model etc needed to complete this and do it the proper way

You need more than a feature plan to complete Semester correctly: you need a **complete company operating system**, a product-system architecture, a delivery factory, institutional implementation machinery, security/privacy governance, support operations, and a ruthless evidence-based definition of "done." The aim is not merely to launch an app; it is to operate a trustworthy, native, institution-ready education platform that can replace fragmented systems while working for students every day.

The existing Semester repository gives you an unusually rich feature and governance blueprint. The next step is to convert that blueprint into a controlled, traceable build program where every capability has an owner, data model, operating model, test strategy, support path, pricing logic, and release gate. [16] [17] [18]

Completion Standard

A capability is **not complete** because it has a screen, a route, a mock interaction, a state reducer, a database table, or a passing unit test.

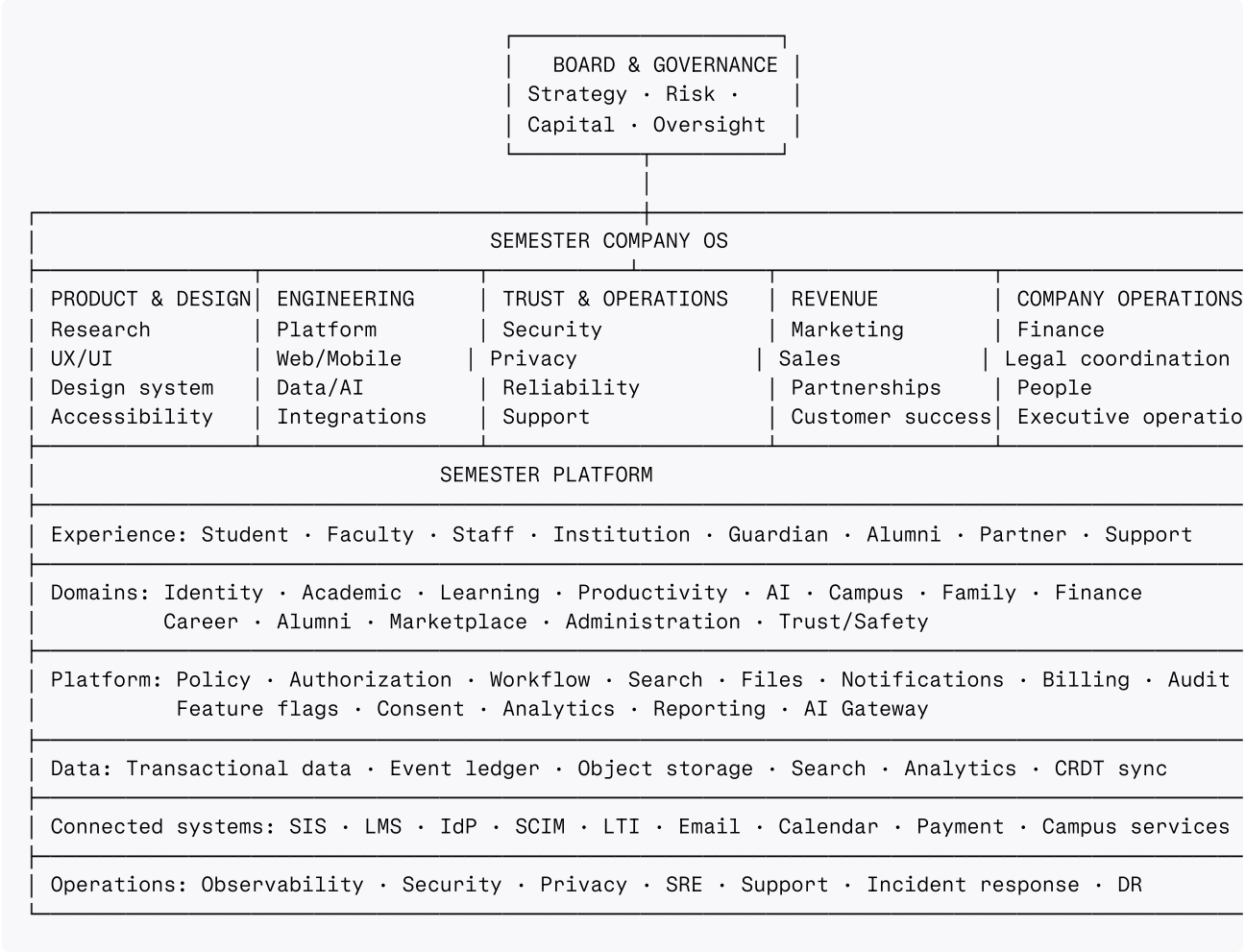
A Semester capability is complete only when all 16 conditions below are true:

#	Completion dimension	Required evidence
1	User outcome	Defined user, job-to-be-done, success metric, and critical journey
2	Domain ownership	Bounded context, accountable team, data owner, and source-of-truth decision
3	UX	Responsive and accessible flows for student, staff, guardian, and mobile contexts where applicable
4	Native capability	Semester-owned data model, workflow, API, persistence, and user interface
5	Integration behavior	Optional connector mappings, reconciliation rules, degraded mode, and no silent failures
6	Authorization	Tenant, role, relationship, purpose, consent, and data-classification checks
7	Governance	Audit event, policy record, approval/override rules, and review process
8	Data lifecycle	Data classification, retention, deletion, export, legal-hold, backup, and restoration behavior
9	Reliability	Service-level objective, performance budget, failure state, retry behavior, and recovery procedure
10	Security	Threat model, abuse cases, tests, secrets handling, and monitoring coverage

#	Completion dimension	Required evidence
11	AI safety	If AI is involved: data-scope controls, provenance, evaluations, escalation, and human confirmation
12	Accessibility	Keyboard, screen reader, contrast, focus, mobile assistive tech, cognitive load, and error recovery validation
13	Test coverage	Unit, integration, contract, end-to-end, authorization, accessibility, performance, and security tests
14	Operations	Alerts, dashboards, runbook, on-call ownership, support macro, and incident playbook
15	Commercial	Package placement, entitlements, pricing, implementation implications, and customer success expectations
16	Release evidence	Release checklist, feature flag, rollout cohort, rollback plan, and documented go/no-go approval

This is how you build the whole platform "from inception" without pretending incomplete automations are finished. The whole product is present and native; any complex high-risk action that needs institution configuration begins as a **fully real, governed, human-in-the-loop workflow**, not as a fake integration or unavailable future promise.

Whole-System Map



The four operating modes

Semester must support all four modes from its earliest architecture:

Mode	Customer situation	Semester behavior
Native	No external system exists or an institution wants Semester as the primary system	Semester is the canonical record and workflow owner

Mode	Customer situation	Semester behavior
Connected	Existing authoritative system remains in place	Semester connects with explicit source precedence, sync, reconciliation, and user-visible provenance
Coexistence	Institution adopts Semester department-by-department or module-by-module	Semester operates natively in its activated domains and interoperates cleanly in others
Replacement	Institution cuts over a legacy platform	Semester provides migration, verification, parallel run, readiness, rollback, and operational ownership

The mistake to avoid is treating “integration” as a product area. Integration is a **platform discipline** applied to every domain.

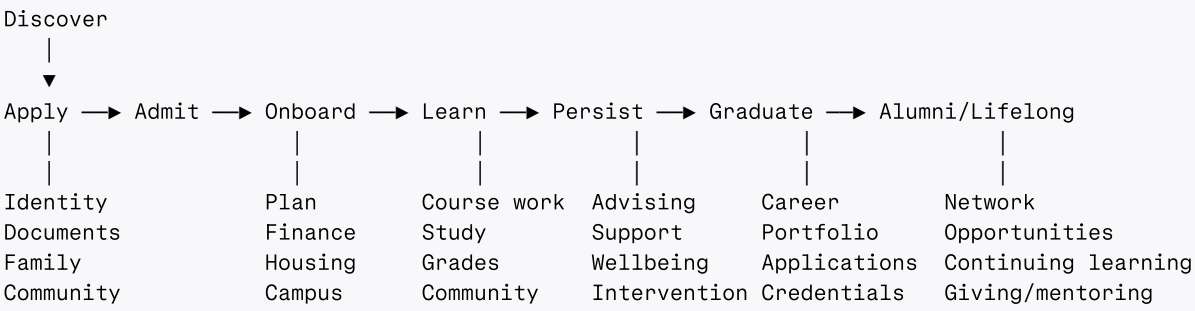
Product Capability Blueprint

Capability domains

Domain	Native capabilities required	Connected systems	Primary users	Highest-risk conditions
Identity	Accounts, profiles, passkeys/MFA, devices, recovery, roles, affiliations, consent	IdP, SSO, SCIM, HR/SIS	Everyone	Account takeover, cross-tenant access, privilege escalation
Academic core	Terms, catalog, sections, schedules, enrollment, holds, degree audit, records	SIS/ERP, registrar tools	Students, registrar, advisors	Incorrect records, registration errors, unauthorized amendments
Learning	Courses, modules, assignments, submissions, assessments, rubric, gradebook, feedback	LMS, LTI tools, content vendors	Students, faculty, TAs	Grade integrity, academic integrity, accommodation failures
Productivity	Tasks, projects, documents, sheets, decks, notes, reminders, files, search	Google/Microsoft/imports	Students, faculty, staff	Privacy leakage, data loss, collaboration conflicts
Calendar	Events, tasks, time blocks, schedules, focus, reminders, availability	Google, Microsoft, ICS, SIS/LMS	Everyone	Missed deadlines, double booking, incorrect source sync
AI	Assistant, tutor, advisor, researcher, writer, planner, study coach	Approved models and knowledge sources	Students, staff, faculty	Hallucinated policy, unsafe advice, sensitive-data leakage
Campus life	Dining, housing, maps, directory, events, transit, clubs, athletics, campus alerts	Campus feeds and service systems	Students, campus staff	Outdated safety/service information, inappropriate directory exposure
Family	Guardians, consent, sharing scopes, family notifications, payment access	SIS contacts, billing	Student, guardian, family office	Improper disclosure, minor/age rules, relationship fraud
Finance	Charges, bills, payments, plans, aid visibility, refunds, receipts	ERP, payment processor, aid systems	Student, finance office	Payment errors, PCI exposure, incorrect balances
Career	Profile, portfolio, opportunities, applications, mentoring, skills, credentials	Career systems, employers	Student, alumni, employers	Bias, privacy, false credentials, discrimination
Marketplace	Listings, providers, orders, entitlement, disputes, payouts	Payment, vendors, fulfillment	Students, providers, institution	Fraud, consumer safety, tax, disputes

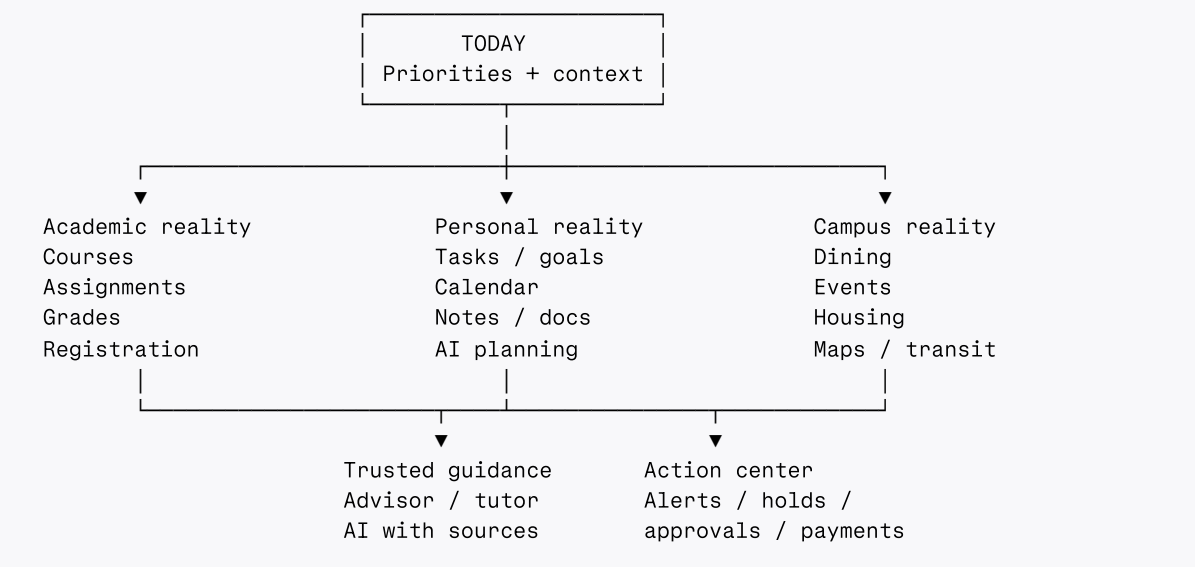
Domain	Native capabilities required	Connected systems	Primary users	Highest-risk conditions
Institution console	Configuration, policy, workflow, reporting, admin, audit, implementation	IdP, data warehouse, enterprise tools	Institution staff	Misconfiguration, bulk errors, excessive admin authority
Support/trust	Help, case management, moderation, reporting, support access, incidents	CRM, email, phone	Students, staff, support	Improper support access, unsafe community content, incident mishandling

Student lifecycle model



Semester must preserve a person's history and permissions through this lifecycle without forcing a new identity or losing academic, productivity, career, or consent context.

Daily student experience model



"Today" is not a dashboard. It is the **orchestration surface** for the student's entire educational life. Every alert must identify its source, severity, action, deadline, impact, owner, and fallback support route.

UX and Design System

Experience principles

1. **One identity, one language, one interaction model.**
   Students should not feel they moved from an LMS to a registrar system to a payment portal to a campus feed.
2. **Show the next useful action, not every possible feature.**
   The platform can be broad while each moment remains focused.
3. **Always reveal authority and provenance.**
   Users must see whether information is official, connected, user-created, AI-generated, pending, estimated, or verified.
4. **No dead-end states.**
   Every error, restriction, empty state, permission denial, degraded integration, and approval delay should explain what happened, what the user can do, and who can help.
5. **Accessibility is product quality.**
   WCAG 2.2 Level AA should be the minimum web acceptance standard, with equivalent native-mobile accessibility requirements. W3C's guidance covers a broad range of disabilities and requires, among other things, text alternatives, keyboard-accessible functionality, usable navigation, and adaptable presentation.[19] [20]

Core information architecture

```
Semester
├── Today
├── Plan
│   ├── Calendar
│   ├── Tasks
│   ├── Goals
│   ├── Projects
│   └── Time / Focus
├── Learn
│   ├── Courses
│   ├── Assignments
│   ├── Study
│   ├── Grades
│   ├── Degree
│   └── Academic records
├── Create
│   ├── Documents
│   ├── Sheets
│   ├── Decks
│   ├── Notes
│   └── Files
├── Campus
│   ├── Events
│   ├── Maps
│   ├── Dining
│   ├── Housing
│   ├── Directory
│   └── Community
├── Path
│   ├── Career
│   ├── Opportunities
│   ├── Portfolio
│   ├── Applications
│   ├── Mentors
│   └── Alumni
├── Money
```

```
│   ├── Bill
│   ├── Aid
│   ├── Payment plans
│   └── Marketplace
├── Support
│   ├── Help
│   ├── Advising
│   ├── Tutoring
│   ├── Cases
│   └── Safety
├── Assistant
│   ├── Ask
│   ├── Plan
│   ├── Study
│   ├── Write
│   └── Sources / history
└── Account
    ├── Profile
    ├── Privacy
    ├── Connections
    ├── Family
    ├── Accessibility
    ├── Devices
    └── Billing
```

Role-specific home surfaces

Role	Home page must answer	Primary widgets
Student	What matters today, what is at risk, what can I do now?	Agenda, priorities, course work, holds, grade changes, bills, campus events, advisor and AI guidance
Faculty	Who needs attention, what needs grading, what is next?	Course health, submissions, grading queue, attendance/engagement signals, student outreach
Advisor	Which students need intervention and what action is appropriate?	Caseload, milestones, risk signals, notes, appointment queue, interventions, handoffs
Registrar	What academic workflows are blocked, incorrect, or pending?	Registration exceptions, holds, record changes, degree audit exceptions, approvals
Finance	What money movement or student communication needs action?	Payment plans, failed payments, aid visibility, reconciliation, refund queue
Student affairs	What campus needs, events, cases, and services require action?	Events, engagement, case queue, resource availability, alerts
IT/security	Is the tenant secure, healthy, integrated, and compliant?	Auth health, connector health, audit anomalies, access reviews, incidents
Guardian	What has the student consented to share and what action is appropriate?	Authorized milestones, bill/payment details, selected alerts, support resources
Support agent	What is the customer's issue without overexposing private data?	Case timeline, approved support access, known incidents, diagnostic tools, safe resolution actions

Interface state matrix

Every screen must define these states before engineering begins:

State	Required user experience
Loading	Skeleton that matches final layout; meaningful status for lengthy work

State	Required user experience
Empty	Explain value, primary action, import/connect option if relevant
Permission denied	Explain restriction without exposing sensitive data; offer correct request path
Offline	Clear offline indicator, local changes queue, next sync attempt, available actions
Sync pending	Show source, last sync, pending count, and no false success claim
Degraded dependency	Identify affected connection; retain native features; offer alternate workflow
Validation failure	Field-level reason, preserved user input, accessible error summary
System failure	Friendly message, retry, support reference, correlation ID where appropriate
Approval pending	Owner, estimated SLA, current status, cancel/edit rules
AI uncertainty	Sources, confidence limits where appropriate, user choices, escalation path
Sensitive workflow	Confirmation, policy basis, audit notice, impact summary, irreversible-action warning

Design tokens

Use a central design-token system—never hard-code visual values per screen.

```
type SemesterTokens = {
  color: {
    brand: {
      primary: string
      secondary: string
      accent: string
    }
    semantic: {
      success: string
      warning: string
      danger: string
      info: string
      official: string
      connected: string
      ai: string
      pending: string
    }
    surface: {
      canvas: string
      raised: string
      sunken: string
      overlay: string
    }
    text: {
      primary: string
      secondary: string
      muted: string
      inverse: string
      link: string
    }
  }
  space: Record<"0" | "1" | "2" | "3" | "4" | "5" | "6" | "8" | "10" | "12", string>
  radius: {
    sm: string
    md: string
    lg: string
    xl: string
    pill: string
  }
```

```
  typography: {
    display: string
    heading: string
    body: string
    mono: string
    scale: Record<string, { size: string; lineHeight: string; weight: number }>
  }
  motion: {
    fast: string
    normal: string
    slow: string
    reduced: string
  }
  elevation: {
    none: string
    low: string
    medium: string
    high: string
  }
}
```

Required reusable components

Component family	Must include
Navigation	App shell, command palette, global search, mobile tab bar, contextual rail, breadcrumbs, skip links
Input	Text fields, selects, date/time, comboboxes, file upload, rich text, spreadsheet cells, validation summary
Status	Official-source badge, connected-source badge, AI-generated badge, pending, verified, warning, incident
Workflow	Approval timeline, status stepper, activity log, exception queue, confirmation dialog, undo where safe
Data	Responsive table, mobile list alternative, filters, saved views, export, pagination, bulk actions
Learning	Course card, assignment card, rubric, submission receipt, study session, source citation, grade explanation
AI	Prompt composer, source drawer, data-scope picker, action-preview card, feedback control, handoff control
Support	Help panel, ticket form, safe-support consent, system-status banner, diagnostic reference
Accessibility	Focus ring, live region, reduced-motion variant, contrast-safe variants, readable-density settings

Technical Architecture

Architecture decision model

```
User action
   │
   ▼
Experience app
   │
   ├── Local encrypted store and sync queue
   │
   ▼
API gateway / BFF
   │
   ├── Identity verification
   ├── Tenant resolution
   ├── Policy decision
   ├── Abuse / rate controls
   ├── Request correlation ID
```

```
   │
   ▼
Domain command or query
   │
   ├── Transactional database write/read
   ├── Audit event
   ├── Domain event
   ├── Notification / workflow event
   └── Connector job where applicable
   │
   ▼
Observable result
   │
   ├── UI status
   ├── Event ledger
   ├── Metrics / tracing
   ├── Support diagnostics
   └── Analytics under privacy policy
```

Architecture boundaries

Layer	Must own	Must not own
Front end	Presentation, local UX state, offline queue, accessible interactions, cached permitted data	Authorization truth, financial/academic integrity, policy decisions
BFF/API edge	Authentication, tenant routing, request validation, policy invocation, response shaping	Core business logic duplicated per client
Domain service	Business rules, transactions, domain events, canonical data	UI layout, vendor-specific client behavior
Policy engine	Authorization and governance decisions	Business-domain persistence or arbitrary SQL
Integration hub	Mappings, sync, retries, reconciliation, source provenance	Canonical domain ownership
AI gateway	Model routing, retrieval permissions, redaction, tool controls, evaluation	Direct unrestricted access to production databases
Analytics	Aggregation, product/business reporting under privacy controls	Transactional system of record
Audit ledger	Immutable evidentiary record of sensitive and consequential actions	General event bus replacement or mutable analytics store

Canonical entity graph

```
Institution
├── Campus
│   ├── Program
│   ├── Term
│   ├── Course / Section
│   ├── Service / Location
│   └── Policy Set
├── Person
│   ├── Account
│   ├── Role Assignment
│   ├── Relationship
│   ├── Consent
│   ├── Device
│   ├── Student Record
│   ├── Learning Record
│   ├── Productivity Workspace
```

```
│   ├── Financial Account
│   ├── Career Profile
│   └── Support/Trust Cases
├── Integration Connection
│   ├── Credential Reference
│   ├── Mapping Version
│   ├── Sync Cursor
│   ├── Reconciliation Report
│   └── Health Status
└── Governance
    ├── Policy
    ├── Audit Event
    ├── Retention Rule
    ├── Legal Hold
    ├── Approval Workflow
    └── Feature Entitlement
```

Required service contracts

Every domain service requires these contract classes:

```
type CommandEnvelope<T> = {
  commandId: string
  idempotencyKey: string
  correlationId: string
  tenantId: string
  actor: {
    personId: string
    accountId: string
    sessionId: string
    roles: string[]
  }
  purpose: string
  submittedAt: string
  payload: T
}

type CommandResult<T> = {
  commandId: string
  status: "accepted" | "completed" | "pending_approval" | "rejected" | "failed"
  data?: T
  userMessage: string
  nextAction?: string
  auditEventId: string
  correlationId: string
}

type SourceMetadata = {
  origin: "native" | "imported" | "connected" | "institution_entered" | "ai_generated"
  system?: string
  externalId?: string
  mappingVersion?: string
  observedAt: string
  verifiedAt?: string
  freshnessState: "current" | "stale" | "unknown" | "conflicted"
}
```

Event taxonomy

Event category	Examples	Retention posture
Domain events	Course enrolled, task completed, submission created, grade released	Per domain retention rule
Audit events	Role changed, grade amended, support access granted, data exported	Long-term tamper-evident record
Security events	Login failed, MFA enrolled, suspicious session, API abuse blocked	Security retention policy
Integration events	Sync began, mapping failed, reconciliation discrepancy found	Operational and compliance retention
AI events	Retrieval started, policy decision, model invoked, action proposed, override applied	AI governance retention policy
Product analytics	Feature used, funnel progressed, anonymized performance data	Privacy-controlled, minimized aggregation
Operational events	Deployment, alert, rollback, incident state changed	SRE/incident retention policy

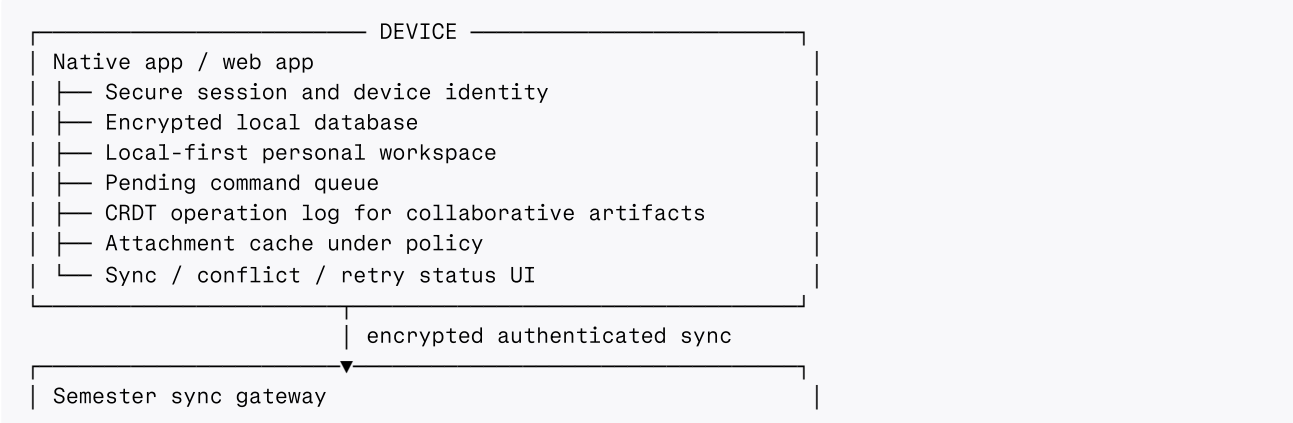
Multi-tenant isolation requirements

Tenant isolation must be verified across all layers, not only database row-level security:

Boundary	Required control
API	Tenant resolved from trusted identity/session, never only user-submitted headers
Database	Tenant predicate or schema/database segmentation; deny-by-default RLS; negative tests
Object storage	Tenant-prefixed authorization, signed URL expiry, content classification
Search	Tenant-aware index partition/filter enforced server-side
Cache	Tenant-scoped keys; no shared unkeyed response cache
Queue	Tenant context propagated and validated in consumers
Analytics	Tenant scoping and privacy-minimized aggregates
Support tooling	Time-bound approved access with scope, reason, expiry, and audit
AI retrieval	Retrieval filters enforced before model context construction
Admin tools	Separate privileged roles, just-in-time access, dual control for high-impact actions

Offline, Mobile, and Sync Model

Device architecture



```
| ├── Authentication + device trust                          |
| ├── Tenant and policy enforcement                          |
| ├── Idempotency + ordering + cursor management             |
| ├── CRDT relay/compaction                                  |
| ├── Command processor                                      |
| └── Conflict and reconciliation events                     |
└────────────────────────────────────────────────────────────┘
```

Data synchronization rules

Data type	Offline availability	Conflict model	Security posture
Tasks and personal notes	Full read/write	Last-write or field-level merge with history	Encrypted local store
Documents and shared notes	Full or partial based on content policy	CRDT merge, version restore	Access rechecked on sync
Calendar	Read cache and queued edits	Source-aware conflict resolution	Minimize sensitive event details
Course content	Read cache as allowed	Version/freshness rules	DRM/licensing policy where needed
Assignment drafts	Full local drafts, queued submission	Server confirmation required	Submission receipt only after server acceptance
Grades	Read cache with freshness status	Server authoritative, no offline edits	Expiring encrypted cache
Registration	Search/cache allowed; action requests online confirmation	Server-authoritative transaction	No false confirmation
Billing/payment	Read summary cache; payments require online secure flow	Processor and ledger authoritative	No sensitive payment data stored
Support case	Draft possible; submit when online	Server authoritative	Encrypt drafts and restrict attachments
AI chat	Drafts offline; inference online unless controlled local capability exists	Server policy decides	No unrestricted local sensitive context

AI Governance Model

NIST frames its AI Risk Management Framework as a voluntary framework for incorporating trustworthiness considerations throughout AI system design, development, use, and evaluation. Use its risk-management orientation as the operating model—not as a marketing badge. [17] [16]

AI control plane

```
User request
   │
   ▼
Purpose classification
   │
   ├── Study / writing / planning / official question / sensitive issue
   │
   ▼
Policy decision
   │
   ├── Is AI enabled for this tenant and role?
   ├── What data classes are allowed?
```

```
    ├── What sources may be retrieved?
    ├── Which models/regions are permitted?
    ├── Are tools permitted?
    └── Is human confirmation mandatory?
    │
    ▼
Context assembly
    │
    ├── Minimize data
    ├── Filter by authorization
    ├── Apply source freshness rules
    ├── Redact where necessary
    └── Defend against untrusted instructions
    │
    ▼
Model gateway and tool broker
    │
    ├── Model selection / cost control
    ├── Output moderation
    ├── Citation/provenance assembly
    ├── Tool call policy enforcement
    └── Human approval for consequential action
    │
    ▼
User result + audit/evaluation
```

AI risk tiers

Tier	Examples	Required safeguards
0: Assistive	Rewrite notes, summarize user-provided text, brainstorm study ideas	Disclosure, user control, safe defaults
1: Contextual	Summarize authorized course materials, create study plans, answer sourced questions	Provenance, retrieval controls, source freshness, feedback loop
2: Advisory	Academic planning suggestions, advising risk signals, financial explanation	Explainability, confidence/uncertainty handling, human review route
3: Action proposing	Draft email, propose calendar event, prepare enrollment request, create support case	Action preview, user confirmation, permission check, audit
4: High impact	Grade-related recommendation, enrollment decision, financial action, disciplinary or safety action	No autonomous final decision; designated human approval, stronger evaluation and logs
5: Prohibited	Covert profiling, unauthorized surveillance, unreviewed high-stakes determinations, bypassing policies	Block and log policy event

AI evaluation suite

Every AI feature needs automated and human evaluation across:

- Accuracy against controlled, sourced answer sets.
- Citation quality and source attribution.
- Harmful or unsafe tutoring behavior.
- Hallucinated institutional policies and deadlines.
- Academic-integrity policy compliance.
- Bias and disparate outcome assessment.
- Prompt injection and untrusted-document attacks.

- Data leakage across students, courses, tenants, family relationships, and roles.
- Tool authorization failures.
- Refusal and safe-handoff quality.
- Cost, latency, and model-fallback quality.
- User comprehension and overreliance risk.

Institutional Integration Blueprint

LTI 1.3/LTI Advantage is an important interoperability path for working with existing LMS environments. Its services include Names and Role Provisioning, Deep Linking, and Assignment and Grade Services, and it uses modern OAuth 2.0, JSON Web Tokens, and OpenID Connect patterns. It should be a first-class connector—but never the only way Semester can support learning workflows. [21] [22] [23]

Integration catalog

Integration class	Standards / examples	Semester native fallback
Identity	SAML, OIDC, OAuth, SCIM	Semester account, passkeys, MFA, invitation and recovery workflows
Student information	SIS/ERP APIs, CSV/SFTP, data exports	Native people, terms, catalog, registration, records, holds, degree data
Learning	LTI 1.3, LTI Advantage, LMS APIs	Native courses, assignments, submissions, assessments, gradebook
Calendar	Google, Microsoft, ICS/CalDAV	Native events, schedules, tasks, reminders, time blocks
Email	Microsoft 365, Google Workspace	Native notifications, inbox-like action center, messaging where enabled
Payments	Payment processor, ERP	Native charges, plans, receipts, ledger and reconciliation
Campus services	Dining, housing, events, card, maps, transit	Native catalog/presentation/workflow, institution-entered operational data
Career	Job boards, employer feeds, career systems	Native opportunity, application, portfolio, mentor, credential objects
Data/reporting	Warehouse, BI, secure file exchange	Native reporting, exports, auditable scheduled reports
Safety/support	CRM, communications, status tools	Native case, support request, moderation, status feed

Connector lifecycle

```
Discover
   │
   ▼
Security and legal review
   │
   ▼
Configure credentials and scopes
   │
   ▼
Map schemas and source precedence
   │
   ▼
Sandbox import / dry run
   │
   ▼
Validate counts, samples, and edge cases
   │
   ▼
Activate limited cohort
   │
   ▼
```

```
Monitor health and reconciliation
   │
   ▼
Expand or roll back
```

Connector health states

State	Meaning	Required UX
Healthy	Syncing within expected window; reconciled	Normal display with source label
Delayed	Last successful sync beyond target but data still usable	Show freshness timestamp and affected scope
Degraded	Some objects or operations failed	Preserve native capabilities; show alternate path
Failed	Connector unavailable or unsafe	Stop dependent automation; expose current native data/fallback
Reconciliation mismatch	Data differs from expected source state	Quarantine automated changes; alert owners; never silently overwrite
Disabled	Not authorized or intentionally off	Show configuration/institution contact path, not an error

Security, Privacy, and Safety

Security program structure

```
Govern
├── Security policy and risk register
├── Asset / data inventory
├── Vendor and supply-chain management
├── Secure SDLC
├── Access governance
└── Incident governance

Protect
├── Identity and MFA
├── Encryption and key management
├── Least privilege / policy engine
├── Network segmentation
├── Secure development
└── Endpoint/device posture

Detect
├── Security telemetry
├── Audit logging
├── Anomaly detection
├── Dependency scanning
└── Abuse monitoring

Respond
├── Incident commander
├── Forensics
├── Customer communications
├── Legal/counsel review
└── Remediation tracking

Recover
├── Backup restoration
├── Disaster recovery
```

```
├── Post-incident review
└── Control improvements
```

Threat model matrix

Asset / workflow	Key threats	Required mitigations
Student account	Credential stuffing, session theft, recovery fraud	Passkeys/MFA, rate limiting, device/session management, suspicious-login detection
Grades and records	Unauthorized changes, inference, bulk export	Fine-grained authorization, dual approval, immutable audit, export controls
Student documents	Unauthorized sharing, deletion, malware	Access policy, signed URLs, version history, malware scanning, restore
Family relationships	Fraudulent guardian claim, oversharing	Relationship verification, granular scopes, age/policy rules, audit/revocation
AI assistant	Prompt injection, data exfiltration, unsafe action	Context filtering, tool broker, output controls, evaluation, confirmation
Integrations	Overbroad tokens, incorrect mapping, replay/webhook abuse	Least scopes, secrets vault, signed webhooks, mapping review, idempotency
Marketplace	Fraud, unsafe vendors, payment disputes	Provider verification, moderation, escrow/payout controls, case workflow
Support	Over-permissioned support agent, impersonation	Just-in-time scoped access, explicit consent, session logs, redaction
Multi-tenancy	Cross-tenant query, cache leakage, shared index exposure	Defense in depth at every boundary, negative tests, regular access reviews
Mobile/offline	Lost device, local data extraction, stale authorization	Encrypted store, remote logout/revocation, cache TTL, re-authentication

Privacy by design checklist

- Data minimization for every field and integration.
- Explicit purpose recorded for sensitive access.
- Separate student-private productivity data from institution-governed records.
- User-visible source, freshness, and audience labels.
- Granular consent and revocation for connected accounts and family sharing.
- Export and deletion workflows that explain exceptions such as records retention and legal holds.
- Regional data processing and transfer assessment where applicable.
- Consent logs with versioned policy text.
- Data-processing inventory and vendor register.
- Privacy review as a required step for new AI, analytics, tracking, community, and marketplace work.
- Legal conclusions and contract commitments routed to qualified human counsel.

Reliability, SRE, and Support

SLO targets

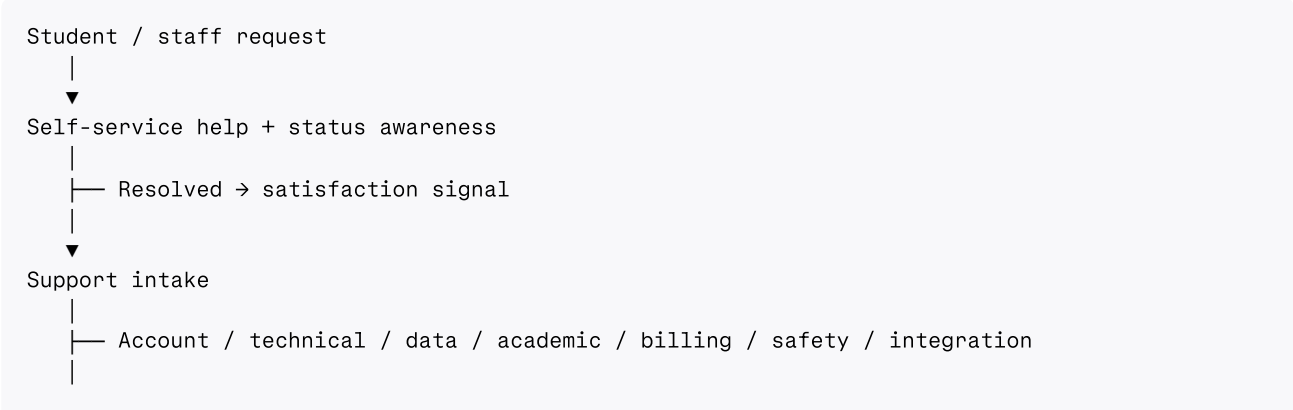
Set provisional targets, measure them, then mature them with real production data.

Service journey	Availability target	Latency target	Data-loss target
Authentication and account access	99.95%	p95 under 500 ms for session validation	Zero unauthorized session persistence
Student Today surface	99.9%	p95 under 2.5 seconds on standard connection	Freshness label when data is delayed
Calendar/task operations	99.9%	p95 under 1 second for local action; 5 seconds sync acknowledgement	No acknowledged action loss
Course content and assignments	99.9%	p95 under 2.5 seconds	Submission receipt and durable storage required
Grade/record display	99.95%	p95 under 2 seconds	No stale data without source/freshness disclosure
Registration workflow	99.95%	p95 under 3 seconds for validation	No duplicate/enrollment-inconsistent command
Billing summary	99.95%	p95 under 2 seconds	Ledger reconciliation required
AI assistance	99.5%	p95 target by model/task; state fallback	No policy-bypassing fallback
Integrations	Connector-specific	Defined sync window per connector	Reconciliation and error journal required
Support/status	99.9%	p95 under 2 seconds	No case loss

Incident severity model

Severity	Example	Initial response target	Communication
SEV-0	Cross-tenant exposure, confirmed data breach, integrity failure affecting official records	Immediate	Executive, security, legal/counsel, affected customers under applicable obligations
SEV-1	Broad platform outage, submissions/registration/grades materially unavailable	15 minutes	Status page and affected customers
SEV-2	Major module degraded, single major tenant affected	30 minutes	Impacted tenant and support
SEV-3	Limited function, workaround available	Same business day	Support response and backlog
SEV-4	Minor defect, no material impact	Planned triage	Release notes or support response

Support operating model



```
   ▼
Triage and severity classification
   │
   ├── Standard support
   ├── Institution administrator
   ├── Engineering/SRE
   ├── Trust & safety
   ├── Security incident
   └── Legal/privacy workflow
   │
   ▼
Resolution, evidence, root cause, customer communication
```

Support agents must never receive blanket visibility into student data. Support access must be purpose-scoped, time-limited, logged, and revocable.

Business, Go-to-Market, and Procurement

Market positioning

> Semester is the native Student Operating System: one governed platform where students plan, learn, create, live on campus, pay, connect, build careers, and continue as alumni—while institutions operate the experience, data, workflows, and trust layer from one modern system.

Procurement evidence room

To sell to institutions, create a Trust Center and private diligence data room containing:

Artifact	Why procurement needs it
Architecture overview	Evaluates fit, resiliency, data flows, and integration posture
Security overview	Explains identity, encryption, logging, testing, incident response
Privacy overview	Explains data use, retention, subprocessors, rights requests
Accessibility VPAT or equivalent	Documents accessibility posture and testing
Data processing terms	Supports privacy/procurement review; final terms require legal counsel
Business continuity and disaster recovery summary	Establishes restoration and incident readiness
Penetration test summary	Demonstrates independent security assessment when available
Subprocessor list	Enables vendor risk assessment
Integration catalog	Confirms standards and supported systems
Implementation methodology	Shows how a customer gets live safely
Support SLA	Defines channels, hours, severity targets, escalation
Product roadmap governance	Shows how requests and changes are controlled
AI governance packet	Details models, data use, controls, testing, and opt-out/configuration
Security questionnaire library	Speeds up common institutional diligence

Institutional implementation model

Phase	Institution work	Semester work	Exit proof
Discover	Goals, stakeholders, systems inventory, constraints	Solution architecture, value hypothesis, risk discovery	Signed implementation charter
Design	Policies, roles, user journeys, data decisions	Configuration model, integration design, migration plan	Approved solution design
Configure	Tenant policies, roles, branding, workflows	Environment, permissions, feature flags, data mappings	Configuration validation
Integrate	Credentials, vendor coordination, test data	Connectors, mappings, reconciliation, error handling	Integration acceptance
Migrate	Data extracts, quality review, business sign-off	Import, transformation, verification, rollback plan	Reconciled data sign-off
Pilot	Cohort participation and feedback	Training, support, monitoring, fixes	Pilot exit criteria met
Parallel run	Compare critical workflows against incumbent	Monitoring, reconciliation, incident simulation	Cutover readiness
Go live	Communications, local support, governance	Hypercare, observability, escalation	Production acceptance
Optimize	Governance cadence and outcomes review	Adoption, expansion, product improvement	Success plan and renewal path

Organization and Role Map

Founding operating structure

Function	Accountable executive	Essential first roles	Deliverables
Executive strategy	CEO	Chief of staff / strategy lead	Strategy, financing, board, partnerships, operating cadence
Product	CPO	Product managers by domain, researcher, product ops	PRDs, outcome metrics, roadmap, capability traceability
Design	Design lead	Product designer, UX researcher, content designer, accessibility specialist	Design system, prototypes, usability evidence, content standards
Engineering	CTO	Platform, backend, frontend, mobile, QA/SDET, DevOps/SRE	Services, apps, CI/CD, quality gates, operational software
Data and AI	Data/AI lead	Data engineer, analytics engineer, ML/AI engineer, AI safety evaluator	Canonical data, warehouse, AI gateway, evaluation and governance
Security	CISO	Security engineer, GRC/privacy program manager	Threat models, control evidence, risk management, incident readiness
Legal/privacy	Legal lead with external counsel	Privacy/compliance lead, contracts ops	Legal queue, contract playbook, privacy program, counsel approvals
Revenue	CRO	Enterprise seller, sales engineer, revops	ICP, pipeline, proposals, procurement response library
Marketing	CMO	Product marketing, demand generation, lifecycle/content	Positioning, website, campaigns, lifecycle, proof assets
Customer success	COO/CS leader	Implementation lead, CSM, support lead	Implementation playbook, adoption, renewals, support operations
Finance	CFO	Finance ops, billing/revops analyst	Pricing, budgets, forecasts, controls, invoicing

Function	Accountable executive	Essential first roles	Deliverables
Trust and safety	COO/CISO shared	Trust & safety lead, moderation/case ops	Reporting, escalation, community safety, case policies
Partnerships	Chief partnerships officer / CRO	Partnerships manager, integration partner engineer	Partner strategy, vendor standards, marketplace governance

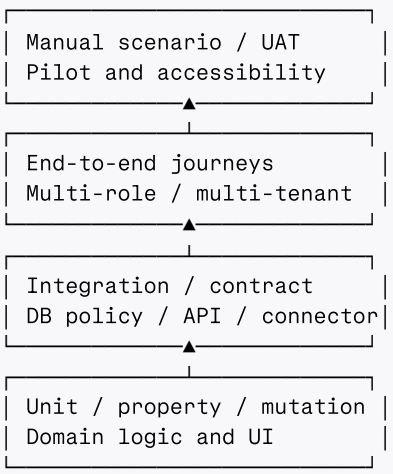
RACI for critical decisions

Decision	CEO	CPO	CTO	CISO	Legal/counsel	CRO	COO	Institution
Product scope	A	R	C	C	C	C	C	C
Architecture standard	C	C	A/R	C	C	I	C	I
Security control	I	C	R	A	C	I	C	I
Privacy/contract conclusion	I	C	C	C	A/R	C	C	C
High-risk AI release	A	R	R	R	C	I	C	C
Institution go-live	A	C	C	C	C	C	R	A/C
Incident communications	A	I	R	R	C	C	R	C
Pricing/package	A	C	C	C	C	R	C	I
Marketplace provider approval	C	C	C	C	C	R	A	C

R = responsible; **A** = accountable; **C** = consulted; **I** = informed.

Quality Engineering Model

Test pyramid plus institutional acceptance



Mandatory test suites

Test suite	Minimum contents
Domain logic	State transitions, invariants, financial/grade/record calculations, idempotency
Authorization	Positive and negative test for every role, relationship, tenant, purpose, and sensitive resource

Test suite	Minimum contents
Tenant isolation	Cross-tenant API, DB, cache, queue, storage, search, analytics, and support-tool tests
API contract	Version compatibility, error shape, pagination, rate limits, idempotency, correlation IDs
Integration	Auth, mapping, schema drift, retry, dead-letter, reconciliation, degraded state
End-to-end	Student, faculty, advisor, guardian, registrar, finance, admin, support journeys
Accessibility	Automated axe-like checks plus manual keyboard, screen reader, zoom, voice control, mobile assistive tech
Mobile/offline	Network loss, resume, sync conflicts, encrypted store, revoked device, stale cache
Performance	Load, soak, concurrency, large tenant/courses, report/export, enrollment spikes
Security	SAST, dependency, secret scan, DAST, auth bypass, authorization fuzzing, penetration test
AI	Grounding, prompt injection, privacy leakage, tool abuse, policy refusals, high-risk workflow escalation
Resilience	Backup restore, regional/dependency failure, queue backlog, connector failure, rollback drill
Migration	Row counts, field matching, data quality exceptions, historical record sampling, rollback

Core non-functional requirements

Category	Launch requirement
Performance	Explicit performance budgets for first load, interaction latency, API p95/p99, sync, and large data screens
Availability	Service-level objectives per critical journey with alert thresholds and error budgets
Security	Secure SDLC enforced in CI/CD; secrets never in code; access reviews and incident readiness
Privacy	Data map, classification, retention, deletion/export workflows before production customer data
Accessibility	WCAG 2.2 AA acceptance criteria and manual testing embedded in release gates
Internationalization	Architecture supports locale, time zone, date formats, language, and content expansion even if launch is US-first
Scalability	Load model for student peak periods: registration, course deadlines, grade release, billing due dates
Observability	Correlation ID from client to API, domain event, integration job, support record, and audit log
Recoverability	Tested backups, restore objectives, failover/manual procedures, and data-reconciliation playbooks
Maintainability	Bounded domains, ADRs, coding standards, dependency rules, owners, and deprecated-feature process

Master Program Backlog

Foundation epics

Epic	Outcomes	Must exist before broad feature build
E0: Product traceability	Every current capability mapped to requirement, owner, domain, status, tests, and release gate	Yes
E1: Identity and tenancy	Accounts, passkeys/MFA, organizations, roles, affiliations, consent, devices, sessions	Yes
E2: Policy and audit spine	Central policy engine, purpose-based access, audit ledger, approvals, break-glass	Yes
E3: Platform operations	CI/CD, environments, secrets, feature flags, telemetry, incident tooling, backups	Yes

Epic	Outcomes	Must exist before broad feature build
E4: Design system	Tokens, components, accessibility contracts, responsive patterns, content standards	Yes
E5: Data platform	Canonical schema, migrations, events, storage, search, analytics, retention	Yes
E6: Offline foundation	Encrypted store, sync, conflict engine, device management, status UX	Before offline-capable experiences
E7: Integration hub	Connector framework, credentials, mapping, sync, reconciliation, health	Before institutional integrations
E8: AI gateway	Policy-bound model access, sources, evaluation, tool broker, audit	Before broad AI rollout

Product-domain epics

Epic	Minimum live native vertical slice
P1: Student OS	Today, tasks, calendar, notifications, unified search, personal settings
P2: Productivity suite	Notes/documents/files, version history, sharing, export, encrypted offline drafts
P3: Learning	Course workspace, assignments, submissions, feedback, study tools, grade visibility
P4: Academic core	Catalog, enrollments, degree plan, holds, approval workflows, records
P5: AI	Sourced tutor, planning assistant, content drafting, advisor handoff, action proposals
P6: Campus	Directory, maps, events, announcements, groups, campus-service catalogs
P7: Family	Relationship verification, sharing dashboard, authorized alerts, billing visibility
P8: Finance	Bill, charges, plans, aid visibility, receipts, reconciliation
P9: Career	Portfolio, opportunities, applications, mentors, skills, verified achievements
P10: Marketplace	Provider onboarding, listings, order flow, entitlement, dispute workflow
P11: Institution console	Tenant config, workflow builder, policy management, integration center, reporting
P12: Support/trust	Help center, cases, safe support access, moderation, safety reporting, status
P13: Alumni	Lifelong profile, network, mentoring, continuing education, opportunities

Operating-company epics

Epic	Required output
C1: Pricing and packaging	Entitlement catalog, SKU structure, usage metering, discount policy, approval process
C2: Procurement	Trust center, evidence library, security questionnaire answer bank, implementation SOW templates
C3: Sales	ICP, account plans, discovery playbook, demo environment, ROI calculator, deal desk
C4: Marketing	Brand system, positioning, messaging house, website, content engine, lifecycle programs
C5: Customer success	Implementation plans, training academy, adoption dashboard, renewal/expansion playbook
C6: Support	Tier model, knowledge base, support tooling, escalation and incident communication process
C7: Finance	Forecast model, spend controls, invoicing, revenue recognition process with professional advice
C8: Governance	Board reporting, risk register, decision log, OKRs, internal controls, executive cadence

Metrics and Control Dashboards

North-star metrics

Perspective	Metric	What success means
Student	Weekly meaningful learning/progress actions completed	Semester improves student progress, not merely time spent
Institution	Percentage of critical student journeys completed in Semester without legacy fragmentation	Semester becomes the operating layer
Product	Activated users completing recurring core workflows	Product is durable and useful
Reliability	Critical journey SLO attainment	Users can trust it at high-stakes moments
Trust	Policy/audit coverage and confirmed privacy/security incidents	Platform operates safely
Revenue	Net revenue retention	Institutions and users get expanding value
Implementation	Time to live and time to first value	The platform can be adopted repeatedly
Support	Contact rate and resolution quality	Product and operations are understandable
AI	AI-assisted outcome quality at acceptable cost and risk	AI helps without creating hidden liabilities

Executive operating dashboard

```
                    SEMESTER EXECUTIVE SCORECARD
┌──────────────┬──────────────────────────────────────────────────────────┐
│ Growth       │ Pipeline · Win rate · CAC · Activation · NRR             │
├──────────────┼──────────────────────────────────────────────────────────┤
│ Product      │ Adoption · Time-to-value · Feature health · Retention    │
├──────────────┼──────────────────────────────────────────────────────────┤
│ Reliability  │ SLOs · Incidents · Error budget · Connector health       │
├──────────────┼──────────────────────────────────────────────────────────┤
│ Trust        │ Security risks · Access reviews · Privacy requests       │
├──────────────┼──────────────────────────────────────────────────────────┤
│ AI           │ Quality · Cost · Safety events · Human overrides         │
├──────────────┼──────────────────────────────────────────────────────────┤
│ Customers    │ Implementation health · CSAT · Renewal risk              │
├──────────────┼──────────────────────────────────────────────────────────┤
│ Finance      │ ARR · Gross margin · Burn · Runway · Collections         │
└──────────────┴──────────────────────────────────────────────────────────┘
```

Essential Registers and Documents

Build and maintain these as living source-controlled artifacts:

Register / document	Purpose
Capability traceability matrix	Maps every existing and future capability to requirements, code, tests, owner, release state
Domain ownership register	Declares canonical service and data owner for every entity
Decision log / ADR library	Records major architectural and product decisions
Assumption register	Distinguishes assumptions from verified facts
Risk register	Product, security, privacy, legal, commercial, operational, vendor, and delivery risks
Data inventory	Lists data elements, classification, owner, purpose, source, retention, and access rules

Register / document	Purpose
Integration catalog	Lists connections, scopes, mappings, health, owners, and source precedence
AI system inventory	Lists each AI feature, models, data use, risk tier, evaluations, and policy controls
Accessibility conformance register	Tracks components/screens, known defects, testing evidence, remediation
Threat-model library	Per domain and high-risk workflow threat models
Incident register	Incidents, severity, timeline, root cause, actions, customer communication
Vendor register	Vendor role, data access, security/privacy review, contract status, renewal date
Release readiness register	Feature-by-feature evidence and approvals
Customer configuration register	Tenant policies, configuration changes, integrations, approvals, owners
Legal review queue	Items that require qualified human counsel review
Public claims register	Every sales/marketing claim, supporting evidence, approval owner, expiration/review date
Accessibility and content style guide	Inclusive language, terminology, reading level, microcopy, error language
Support knowledge base	Customer-facing and internal resolution articles
Business continuity plan	Recovery roles, contacts, priorities, restoration sequences, communication
Implementation playbook	Repeatable institutional deployment method
Exit/migration plan	How customers can export data and transition responsibly

What You Must Not Miss

The following are frequently missed even by technically strong teams:

- **Data provenance:** Every key fact needs a source, timestamp, freshness state, and authority level.
- **Source conflict policy:** If SIS, LMS, instructor, student, and Semester disagree, the platform must know what wins and how the discrepancy is resolved.
- **Academic calendar complexity:** Add/drop, late withdrawal, holds, prerequisites, co-requisites, waitlists, cross-listings, transfer credit, accommodations, incompletes, audits, and term variants.
- **Identity complexity:** One person can be a student, employee, alumnus, mentor, guardian, course staff, and parent across institutions and time.
- **Role relationships:** A guardian is not a generic admin; a faculty member cannot automatically see every student's information; an advisor sees a caseload, not the whole university.
- **Minors and age transitions:** Consent, guardian visibility, messaging, marketing, and data rights can change based on age, jurisdiction, relationship, and institution policy.
- **Accessibility beyond checklists:** Cognitive load, plain language, time limits, sensory overload, dyslexia-aware choices, and assistive technology usability.
- **High-stakes accuracy:** Grades, billing, registration, aid, health/safety, conduct, accommodations, and legal notices cannot be treated like ordinary app data.
- **Support as a product feature:** A confused student at 11:55 PM before an assignment deadline needs clear state, receipts, recovery, and escalation—not a generic error.
- **Data migration:** The hard part is not importing rows; it is data quality, mapping semantics, historical integrity, documents, relationships, exceptions, and reconciliation.
- **Institutional configuration:** Every school has policy variation. Build configuration carefully, but do not create unlimited custom code paths.

- **Contractual commitments:** Sales claims, uptime promises, data processing, AI capabilities, support SLAs, and accessibility statements must match real evidence and receive human legal review where required.
- **Operational cost:** AI, storage, egress, search, notifications, support, integrations, and implementation can destroy margins if not metered and modeled.
- **Sunsetting:** Every capability needs an ownership and deprecation plan; leaving old workflows enabled forever creates risk and confusion.
- **Human workflows:** For sensitive exceptions, a well-designed queue, SLA, evidence trail, and handoff are better than unsafe full automation.

Immediate Execution Sequence

First 30 days

- Freeze unstructured feature expansion in the existing monolithic app.
- Build the complete capability traceability matrix from the current repository.
- Establish architecture decision records and bounded-context ownership.
- Define canonical entity model, source-of-truth policy, data classification, and event taxonomy.
- Stand up identity, tenancy, role/relationship/consent policy, audit ledger, secrets, environments, observability, and feature flagging.
- Create the design token system, shared components, accessibility contract, content standards, and responsive layout primitives.
- Establish a formal security, privacy, accessibility, AI-governance, and legal-review queue.
- Build a test and release evidence framework before broad new product work.
- Create financial model, packaging model, institutional implementation plan, trust center outline, and procurement evidence plan.

Days 31–90

- Rebuild the first end-to-end production vertical slice:
  - Student authentication and onboarding.
  - Institution tenant setup.
  - Student Today.
  - Tasks, calendar, reminders, documents/notes.
  - Encrypted local persistence and sync state.
  - Policy enforcement and audit events.
  - Search, notification, support, accessibility, telemetry, and failure modes.
  - One controlled institutional identity and calendar/LMS/SIS integration path.
- Add the institutional control plane for roles, policy, integrations, feature states, audit, and support access.
- Begin native academic and learning-domain contracts rather than rebuilding screens directly.
- Implement AI gateway foundations, source labeling, policy checks, retrieval controls, evaluations, and safe assistant flows.
- Run accessibility tests, threat models, failure drills, and restoration drills.

Days 91–180

- Build native learning, course workspace, assignments, submissions, study, grade visibility, degree planning, and academic workflow capabilities.
- Deliver guardian, campus, career, and finance foundations as complete governed vertical slices.
- Build integration reconciliation, migration center, implementation controls, and institutional reporting.
- Launch a design-partner program with strict pilot acceptance and no unverified replacement claims.
- Establish 24/7 incident ownership for critical journeys or clearly contract and communicate the actual support coverage.
- Begin procurement, security, accessibility, and AI diligence workflows early—before a major enterprise deal forces rushed evidence creation.

Definition of "Ready to Sell"

Semester is ready to sell when it can truthfully demonstrate:

1. A student can securely sign up, authenticate, onboard, plan, learn, create, receive trusted AI help, manage their data, and receive support.
2. An institution can create a tenant, manage roles and policy, configure the experience, integrate authorized systems, monitor health, support users, and retrieve evidence.
3. The platform has native workflows for every promised product domain, even when sophisticated external automation is activated by configuration or human review.
4. Students can distinguish official, connected, personal, estimated, pending, and AI-generated information.
5. Any sensitive action is governed, authorized, auditable, recoverable, and operationally owned.
6. The team can withstand an outage, connector failure, support incident, security event, data-deletion request, and customer migration issue with documented procedures.
7. Sales, marketing, security, privacy, AI, and accessibility claims are backed by evidence and appropriate human review.
8. A pilot institution can implement Semester with repeatable documentation rather than founder-dependent improvisation.

That is the completion standard: not "we have built a lot," but **Semester can safely carry a student's educational life and an institution's trust from day one.**

⁂

Show me the architecture diagram for user roles

Give me the database schema for authorization and tenants
Provide the production deployment pipeline for this system
Draft the incident response and support runbook
Show me the complete test strategy and quality gates
indidual pompts prompts to give codex for each role stated in first prompt and any other roles and required to complete and launch semester with there tasks

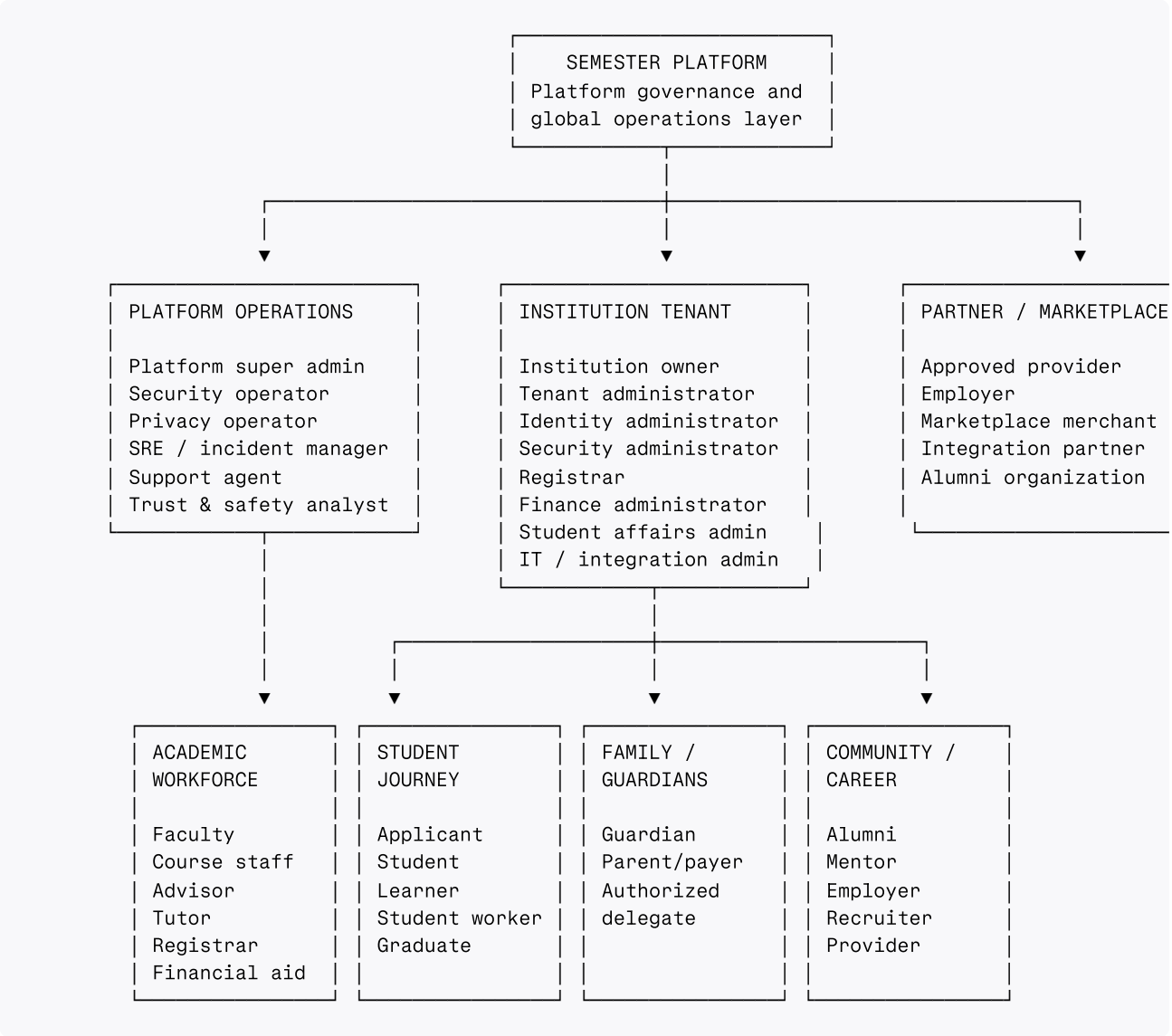
Below is a complete execution pack for Semester: the role architecture, a production-grade authorization and tenant schema, deployment pipeline, incident/support runbook, quality strategy, and copy-ready Codex prompts for every required function.

Use this as the **operating specification** for rebuilding Semester. It assumes the founding thesis remains non-negotiable: Semester is a native Student OS, productivity suite, AI assistant/tutor/advisor, campus super-app, institutional platform, family portal, marketplace, and lifelong career/alumni network—with integrations enhancing the product rather than replacing native capabilities.

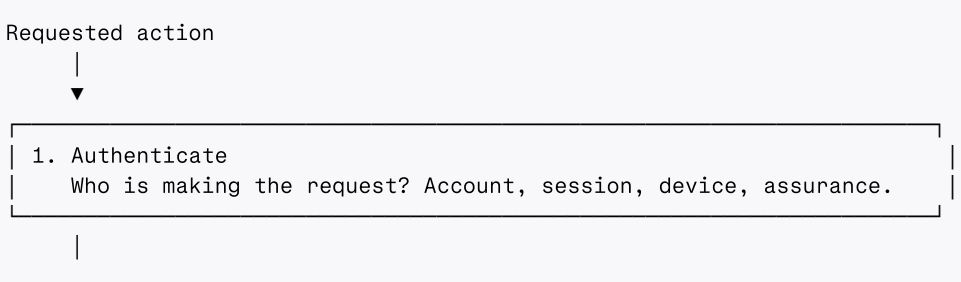
1. User Role Architecture

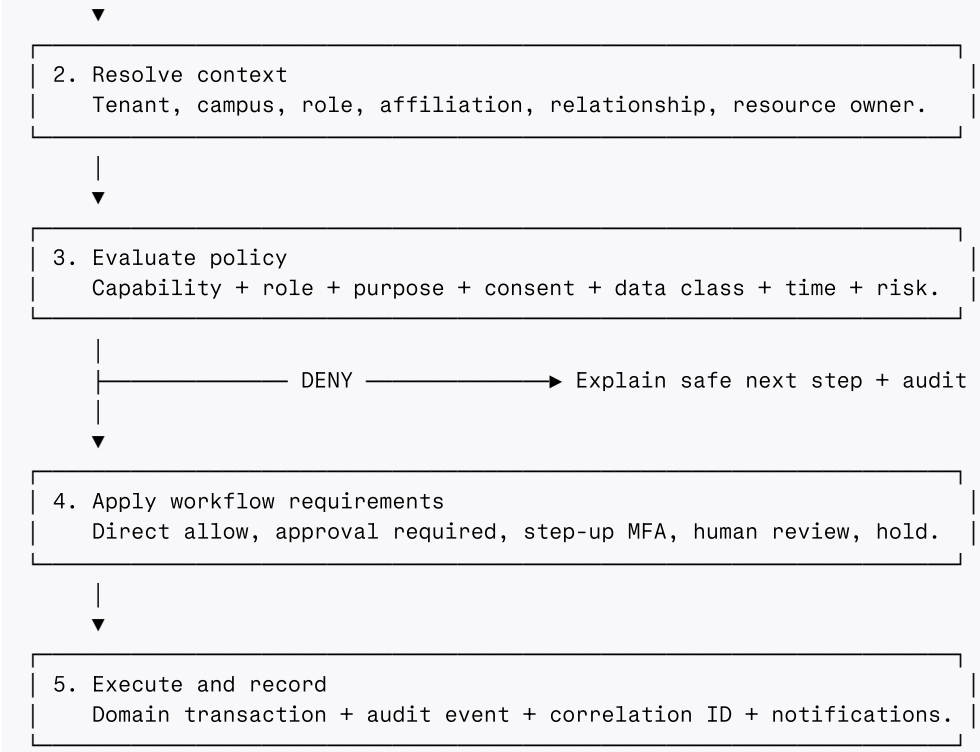
Semester should not use a simplistic "student / admin" permission model. It needs **tenancy, affiliation, relationships, purpose, consent, data classification, policy, and time-bound authority.**

Role architecture diagram



Policy decision model





Role hierarchy

Do **not** treat hierarchy as universal inheritance. A registrar must not inherit unrestricted finance, support, or security authority. Use capability grants with explicit resource scope.

Role family	Roles	Typical scope	Must never automatically receive
Platform operator	Platform admin, SRE, security analyst, support agent	Global platform but tightly purpose-scoped	Blanket student-data access, unlogged impersonation, tenant record modification
Institution administrator	Tenant owner, identity admin, security admin, integration admin	One institution/tenant	Platform-wide visibility, direct access to other tenants
Academic staff	Faculty, TA, tutor, advisor, registrar	Course, section, caseload, program, or record workflow	Unrestricted student financial/family/private productivity data
Student	Applicant, student, learner, graduate, student worker	Self-owned data and authorized group/course spaces	Other students' private data or institution-wide controls
Guardian/family	Guardian, parent, payer, authorized delegate	Explicit relationship and consent scope	Full student identity, notes, course work, or grades by default
Finance	Bursar, aid officer, billing staff	Financial account, approved financial workflow	Academic/private productivity data unless specifically necessary and authorized
Campus life	Housing, dining, student affairs, athletics staff	Service-specific scope	Academic and financial record access by default
Career/alumni	Employer, recruiter, mentor, alumni staff	Opt-in opportunity and profile data	Student record, grades, protected data
Marketplace	Provider, merchant, fulfillment partner	Own listings/orders/entitlements	Student academic, family, or institution data
Trust and safety	Moderator, investigator, safety lead	Report/case-specific data under policy	Open-ended private content access

Role-to-domain permissions matrix

Domain	Student	Faculty	Advisor	Registrar	Guardian	Finance	Institution admin	Semester support
Personal tasks/notes	Full self	No default	No default	No default	Explicit share only	No	No default	Time-bound consent only
Course content	Enrolled access	Assigned course admin	Assigned student summary	Limited	Explicit share only	No	Configurable	Support-scoped only
Assignment submission	Create/view self	View/grade course	Read limited if policy permits	No default	Explicit share only	No	No default	Diagnostic metadata only
Grades	View self	Grade assigned work	Caseload/policy scope	Record workflow	Consent and law/policy scope	No default	Policy-limited	No default
Registration	Self-service/submit	Advise/approve if allowed	Caseload advisement	Full registrar workflow	No default	Hold visibility if allowed	Configure policy	No default
Billing	View/pay self	No	Limited eligibility summary	No default	Explicit payer scope	Full finance scope	Policy configuration	Support-scoped only
Family consent	Manage self where lawful	No	No default	No default	Manage self relationship side	No	Configure rules	No default
Tenant settings	No	No	No	Limited registrar settings	No	Limited finance settings	Full tenant scope	No direct changes without approved workflow
Security logs	Own activity only	Own activity only	Own activity only	Workflow-limited	Own activity only	Workflow-limited	Tenant security scope	Platform operational scope, minimized

Relationship graph

```
Person A: Student
   │
   ├── enrolled_in ──────────────► Course Section
   ├── advised_by ───────────────► Advisor
   ├── taught_by ────────────────► Faculty Member
   ├── guardian_relationship ────► Guardian
   ├── payer_relationship ───────► Parent / Sponsor
   ├── works_for ────────────────► Institution Department
   ├── member_of ────────────────► Club / Cohort / Group
   ├── mentored_by ──────────────► Mentor
   ├── employed_by / applies_to ─► Employer
   └── affiliated_with ──────────► Institution / Campus / Program
```

Each edge needs its own:

- Start and end dates.

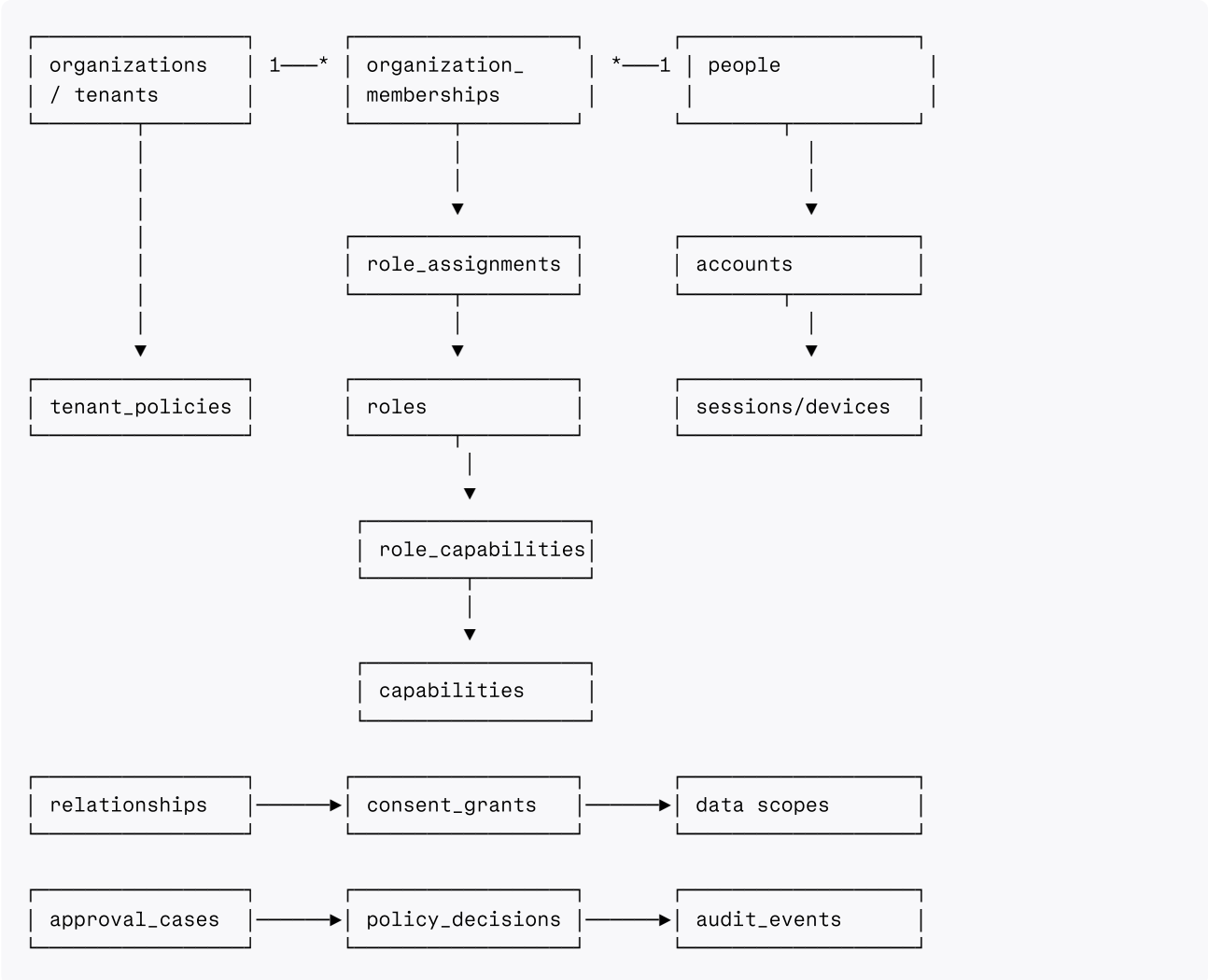
- Verification state.
- Consent and legal/policy basis.
- Data-sharing scopes.
- Authority scope.
- Audit trail.
- Revocation or expiration process.
- Institution policy override rules where legally required.

2. Authorization and Tenant Database Schema

Use PostgreSQL with strict tenant isolation, a policy engine, row-level security where appropriate, append-only audit logging, and a service-layer authorization check. Do not rely on any single control.

OWASP advises testing multi-tenant isolation through the same role, connection path, and connection-pooling behavior used by the application; it specifically recommends positive and negative tenant-isolation tests for every RLS-protected table.[40]

Authorization entity relationship diagram



Core PostgreSQL schema

This is a foundation, not a full academic/LMS schema. It gives Semester a correct authorization and multi-tenancy spine.

```
create extension if not exists pgcrypto;
create extension if not exists citext;

create schema if not exists app;
create schema if not exists private;
create schema if not exists audit;

create type app.tenant_status as enum (
  'draft',
  'active',
  'suspended',
  'offboarding',
  'closed'
);

create type app.membership_status as enum (
  'invited',
  'active',
  'suspended',
  'ended'
);

create type app.assignment_status as enum (
  'active',
  'pending_approval',
  'suspended',
  'expired',
  'revoked'
);

create type app.consent_status as enum (
  'granted',
  'denied',
  'revoked',
  'expired',
  'superseded'
);

create type app.data_classification as enum (
  'public',
  'internal',
  'confidential',
  'student_record',
  'sensitive',
  'payment_adjacent',
  'security_sensitive',
  'legal_hold'
);

create type app.policy_decision_outcome as enum (
  'allow',
  'deny',
  'require_step_up',
  'require_approval',
  'require_human_review'
);

create type app.audit_outcome as enum (
```

```
  'success',
  'denied',
  'failed',
  'pending'
);

create table app.tenants (
  id uuid primary key default gen_random_uuid(),
  slug citext not null unique,
  legal_name text not null,
  display_name text not null,
  tenant_type text not null check (
    tenant_type in (
      'institution',
      'district',
      'school',
      'department',
      'partner',
      'marketplace_provider',
      'internal'
    )
  ),
  status app.tenant_status not null default 'draft',
  primary_region text not null,
  timezone text not null default 'America/Chicago',
  data_residency_policy jsonb not null default '{}'::jsonb,
  feature_entitlements jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  deleted_at timestamptz
);

create table app.people (
  id uuid primary key default gen_random_uuid(),
  global_person_key uuid not null default gen_random_uuid(),
  preferred_name text,
  legal_first_name text,
  legal_last_name text,
  primary_email citext,
  mobile_phone text,
  date_of_birth date,
  locale text not null default 'en-US',
  timezone text,
  accessibility_preferences jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  deleted_at timestamptz,
  unique (global_person_key)
);

create table app.accounts (
  id uuid primary key default gen_random_uuid(),
  person_id uuid not null references app.people(id),
  account_status text not null check (
    account_status in ('invited', 'active', 'locked', 'suspended', 'closed')
  ),
  assurance_level text not null default 'aal1' check (
    assurance_level in ('aal1', 'aal2', 'aal3')
  ),
  passwordless_enabled boolean not null default false,
  mfa_enrolled boolean not null default false,
  last_authenticated_at timestamptz,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
```

```
  deleted_at timestamptz
);

create table app.organization_memberships (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  person_id uuid not null references app.people(id),
  status app.membership_status not null default 'invited',
  external_subject_ref text,
  affiliation_type text not null check (
    affiliation_type in (
      'applicant',
      'student',
      'faculty',
      'staff',
      'administrator',
      'guardian',
      'alumnus',
      'partner',
      'vendor',
      'employee'
    )
  ),
  starts_at timestamptz not null default now(),
  ends_at timestamptz,
  source_system text not null default 'native',
  source_metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  unique (tenant_id, person_id, affiliation_type, starts_at)
);

create table app.roles (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid references app.tenants(id),
  key text not null,
  display_name text not null,
  description text not null,
  is_platform_role boolean not null default false,
  is_system_role boolean not null default false,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  unique nulls not distinct (tenant_id, key)
);

create table app.capabilities (
  id uuid primary key default gen_random_uuid(),
  key text not null unique,
  domain text not null,
  action text not null,
  resource_type text not null,
  risk_level smallint not null check (risk_level between 0 and 5),
  requires_step_up boolean not null default false,
  requires_approval boolean not null default false,
  description text not null,
  created_at timestamptz not null default now()
);

create table app.role_capabilities (
  role_id uuid not null references app.roles(id) on delete cascade,
  capability_id uuid not null references app.capabilities(id) on delete cascade,
  scope_template jsonb not null default '{}'::jsonb,
  constraints jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
```

```
  primary key (role_id, capability_id)
);

create table app.role_assignments (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  membership_id uuid not null references app.organization_memberships(id),
  role_id uuid not null references app.roles(id),
  status app.assignment_status not null default 'active',
  resource_scope jsonb not null default '{}'::jsonb,
  purpose_constraints jsonb not null default '[]'::jsonb,
  starts_at timestamptz not null default now(),
  ends_at timestamptz,
  granted_by_person_id uuid references app.people(id),
  approval_case_id uuid,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table app.relationships (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  subject_person_id uuid not null references app.people(id),
  related_person_id uuid not null references app.people(id),
  relationship_type text not null check (
    relationship_type in (
      'guardian_of',
      'parent_of',
      'payer_for',
      'advisor_of',
      'mentor_of',
      'tutor_of',
      'emergency_contact_for',
      'authorized_delegate_for',
      'employer_contact_for'
    )
  ),
  verification_status text not null check (
    verification_status in ('unverified', 'pending', 'verified', 'rejected', 'expired')
  ),
  status text not null check (
    status in ('active', 'pending', 'suspended', 'ended', 'revoked')
  ),
  starts_at timestamptz not null default now(),
  ends_at timestamptz,
  verification_evidence_ref text,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  check (subject_person_id <> related_person_id)
);

create table app.consent_grants (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  subject_person_id uuid not null references app.people(id),
  grantee_person_id uuid references app.people(id),
  grantee_tenant_id uuid references app.tenants(id),
  relationship_id uuid references app.relationships(id),
  purpose text not null,
  data_categories app.data_classification[] not null,
  resource_scope jsonb not null default '{}'::jsonb,
  status app.consent_status not null default 'granted',
  legal_basis text not null,
  policy_version text not null,
```

```
  granted_at timestamptz not null default now(),
  expires_at timestamptz,
  revoked_at timestamptz,
  revocation_reason text,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  check (grantee_person_id is not null or grantee_tenant_id is not null)
);

create table app.tenant_policies (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  policy_key text not null,
  policy_version text not null,
  policy_document jsonb not null,
  effective_at timestamptz not null,
  expires_at timestamptz,
  status text not null check (status in ('draft', 'active', 'retired')),
  approved_by_person_id uuid references app.people(id),
  created_at timestamptz not null default now(),
  unique (tenant_id, policy_key, policy_version)
);

create table app.approval_cases (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  case_type text not null,
  status text not null check (
    status in ('draft', 'submitted', 'in_review', 'approved', 'rejected', 'cancelled', 'expired')
  ),
  requester_person_id uuid not null references app.people(id),
  target_resource_type text not null,
  target_resource_id uuid,
  requested_action text not null,
  risk_level smallint not null check (risk_level between 0 and 5),
  payload jsonb not null default '{}'::jsonb,
  decision jsonb,
  submitted_at timestamptz,
  decided_at timestamptz,
  expires_at timestamptz,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table app.policy_decisions (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  correlation_id uuid not null,
  request_id uuid not null,
  actor_person_id uuid references app.people(id),
  action_key text not null,
  resource_type text not null,
  resource_id uuid,
  purpose text,
  outcome app.policy_decision_outcome not null,
  reason_codes text[] not null default '{}',
  evaluated_policies jsonb not null default '[]'::jsonb,
  decision_context jsonb not null default '{}'::jsonb,
  approval_case_id uuid references app.approval_cases(id),
  created_at timestamptz not null default now()
);

create table audit.events (
  id uuid primary key default gen_random_uuid(),
```

```
  occurred_at timestamptz not null default now(),
  tenant_id uuid references app.tenants(id),
  actor_person_id uuid references app.people(id),
  actor_account_id uuid references app.accounts(id),
  actor_role_snapshot jsonb not null default '[]'::jsonb,
  actor_assurance_level text,
  action text not null,
  resource_type text not null,
  resource_id uuid,
  resource_owner_person_id uuid references app.people(id),
  data_classification app.data_classification,
  purpose text,
  outcome app.audit_outcome not null,
  correlation_id uuid not null,
  request_id uuid,
  source_ip inet,
  user_agent text,
  client_type text,
  change_summary jsonb not null default '{}'::jsonb,
  before_hash text,
  after_hash text,
  policy_decision_id uuid references app.policy_decisions(id),
  approval_case_id uuid references app.approval_cases(id),
  integrity_hash text not null,
  previous_integrity_hash text
);

create index organization_memberships_tenant_person_idx
  on app.organization_memberships (tenant_id, person_id)
  where status = 'active';

create index role_assignments_active_idx
  on app.role_assignments (tenant_id, membership_id, role_id)
  where status = 'active';

create index relationships_subject_related_idx
  on app.relationships (tenant_id, subject_person_id, related_person_id)
  where status = 'active';

create index consent_grants_lookup_idx
  on app.consent_grants (tenant_id, subject_person_id, grantee_person_id, status)
  where status = 'granted';

create index audit_events_tenant_occurred_idx
  on audit.events (tenant_id, occurred_at desc);

create index policy_decisions_tenant_created_idx
  on app.policy_decisions (tenant_id, created_at desc);
```

Tenant context functions

Never let users supply the tenant as an untrusted query parameter and never set tenant context from raw client input. The authenticated API layer must establish it.

```
create or replace function private.current_tenant_id()
returns uuid
language sql
stable
as $$
  select nullif(current_setting('app.tenant_id', true), '')::uuid
$$;
```

```
create or replace function private.current_person_id()
returns uuid
language sql
stable
as $$
  select nullif(current_setting('app.person_id', true), '')::uuid
$$;

create or replace function private.require_tenant_context()
returns void
language plpgsql
stable
as $$
begin
  if private.current_tenant_id() is null then
    raise exception 'tenant context is required';
  end if;
end;
$$;

create or replace function private.set_request_context(
  p_tenant_id uuid,
  p_person_id uuid,
  p_request_id uuid,
  p_correlation_id uuid
)
returns void
language plpgsql
security definer
set search_path = private, pg_temp
as $$
begin
  perform set_config('app.tenant_id', p_tenant_id::text, true);
  perform set_config('app.person_id', p_person_id::text, true);
  perform set_config('app.request_id', p_request_id::text, true);
  perform set_config('app.correlation_id', p_correlation_id::text, true);
end;
$$;
```

Example tenant-owned table

Every domain record needs `tenant_id`, source metadata, lifecycle state, audit metadata, and a policy-appropriate owner.

```
create table app.student_tasks (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  owner_person_id uuid not null references app.people(id),
  title text not null check (char_length(title) between 1 and 500),
  description text,
  status text not null default 'open' check (
    status in ('open', 'in_progress', 'completed', 'cancelled', 'archived')
  ),
  due_at timestamptz,
  source_metadata jsonb not null default jsonb_build_object(
    'origin', 'native',
    'freshnessState', 'current'
  ),
  data_classification app.data_classification not null default 'confidential',
  created_at timestamptz not null default now(),
```

```
  updated_at timestamptz not null default now(),
  deleted_at timestamptz
);

alter table app.student_tasks enable row level security;
alter table app.student_tasks force row level security;

create policy student_tasks_tenant_isolation
on app.student_tasks
as restrictive
for all
using (
  tenant_id = private.current_tenant_id()
)
with check (
  tenant_id = private.current_tenant_id()
);

create policy student_tasks_owner_access
on app.student_tasks
as permissive
for select
using (
  owner_person_id = private.current_person_id()
);

create policy student_tasks_owner_write
on app.student_tasks
as permissive
for insert
with check (
  owner_person_id = private.current_person_id()
);

create policy student_tasks_owner_update
on app.student_tasks
as permissive
for update
using (
  owner_person_id = private.current_person_id()
)
with check (
  owner_person_id = private.current_person_id()
);
```

Authorization evaluation pseudocode

```
type AuthorizationRequest = {
  tenantId: string
  actorPersonId: string
  action: string
  resource: {
    type: string
    id?: string
    ownerPersonId?: string
    classification?: string
    attributes?: Record<string, unknown>
  }
  purpose: string
  requestedAt: string
  client: {
    deviceId?: string
```

```
    assuranceLevel: "aal1" | "aal2" | "aal3"
  }
}

async function authorize(
  request: AuthorizationRequest
): Promise<
  | { outcome: "allow"; decisionId: string }
  | { outcome: "require_step_up"; decisionId: string }
  | { outcome: "require_approval"; decisionId: string; approvalCaseId: string }
  | { outcome: "deny"; decisionId: string; reasonCodes: string[] }
> {
  const context = await loadAuthorizationContext(request)

  assertTenantMembership(context)
  assertActiveAccount(context)
  assertActiveRoleOrRelationship(context)
  assertTenantPolicyAllowsAction(context)
  assertCapabilityGrantAllowsAction(context)
  assertResourceScopeAllowsAction(context)
  assertPurposeIsPermitted(context)
  assertDataClassificationPermitsAccess(context)
  assertConsentOrLegalBasisWhereRequired(context)
  assertDeviceAndSessionAssurance(context)
  assertNoLegalHoldOrSafetyRestrictionBlocksAction(context)

  const decision = await evaluateRiskAndWorkflow(context)

  await recordPolicyDecision(decision, context)

  return decision
}
```

Authorization invariants

These become automated tests and database constraints:

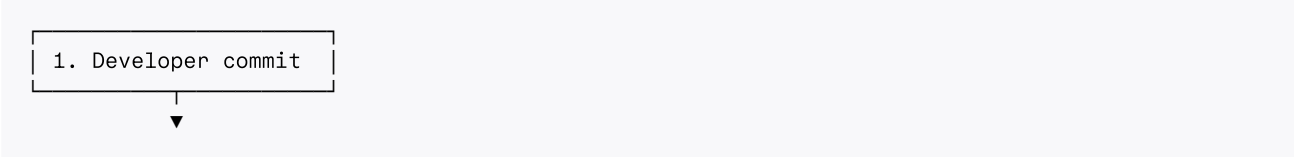
1. No query may return a record from another tenant unless an explicitly approved cross-tenant relationship applies.
2. A platform support agent cannot access customer content without an approved, expiring, audited support grant.
3. A guardian can access only explicit, lawful, active consent scopes tied to a verified relationship.
4. A faculty member can access only assigned-course data, never institution-wide student data by default.
5. Grade changes require an approved grade-change workflow, reason code, actor identity, before/after evidence, and immutable audit event.
6. Institutional admins can configure policy but cannot erase audit trails.
7. AI retrieval uses the same policy decision as direct user access—no special bypass.
8. Background jobs, search, analytics, exports, queues, storage, and support tooling must enforce the same tenant boundary.
9. Connection pooling must not retain a prior request's tenant context.
10. Every privileged action must include a correlation ID and audit event.

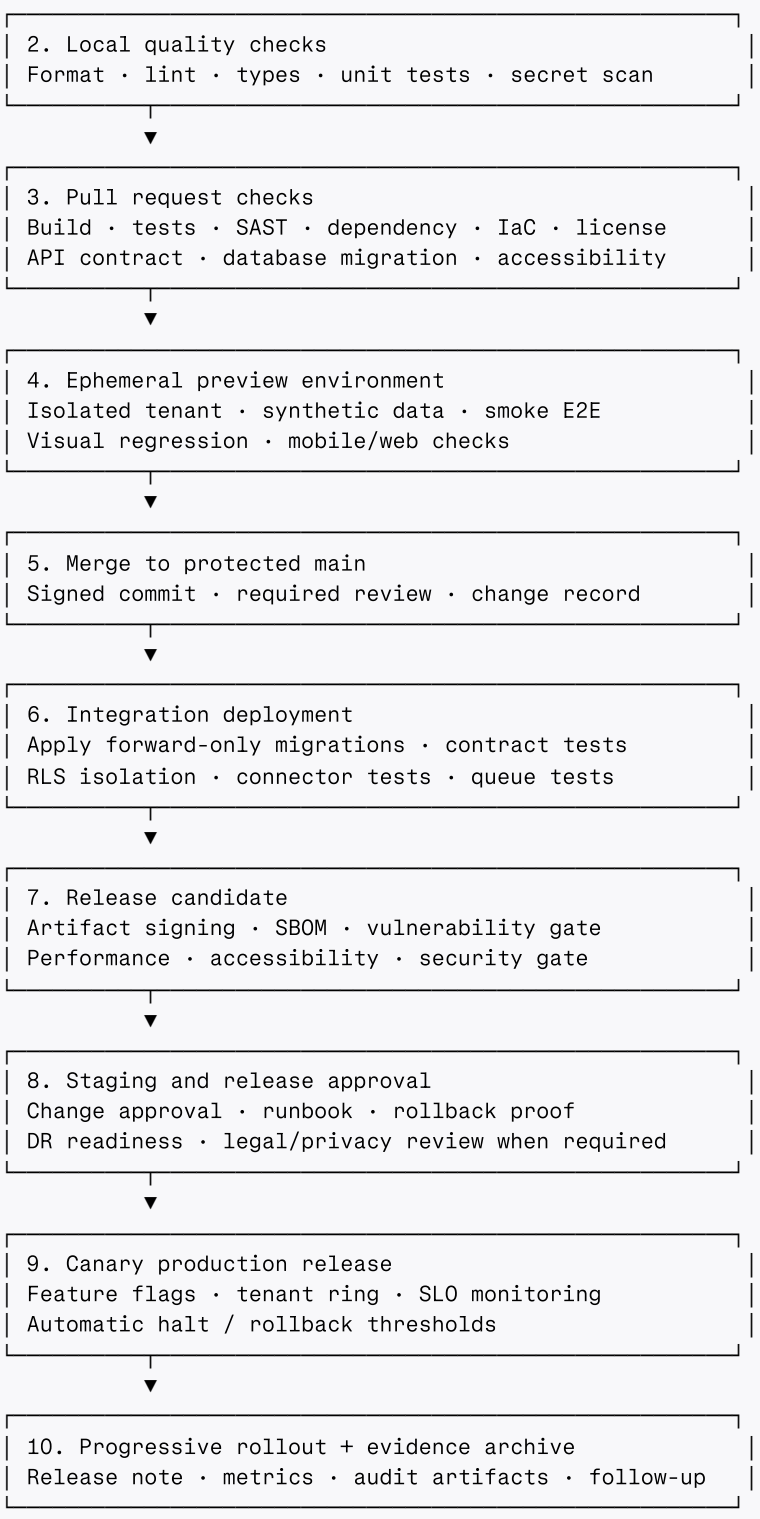
3. Production Deployment Pipeline

Environment topology

```
Developer workstation
      │
      ▼
Pull request environment
      │
      ├── Lint / typecheck / unit / component tests
      ├── Secret scan / dependency scan / SAST
      ├── Preview deployment
      ├── Accessibility smoke tests
      └── Contract tests
      │
      ▼
Integration environment
      │
      ├── Real migrations
      ├── Seeded synthetic tenant data
      ├── API / integration / RLS tests
      ├── Mobile sync tests
      └── DAST / performance smoke
      │
      ▼
Staging environment
      │
      ├── Production-like topology
      ├── Approved test connectors
      ├── Disaster recovery rehearsal
      ├── Load tests
      ├── Security and acceptance testing
      └── Release candidate approval
      │
      ▼
Production canary ring
      │
      ├── Internal tenant
      ├── Design-partner tenant
      ├── Low-risk cohort
      └── Automated metric/SLO checks
      │
      ▼
Production staged rollout
      │
      ├── 1%
      ├── 10%
      ├── 25%
      ├── 50%
      └── 100%
      │
      ▼
Post-release validation and evidence archive
```

CI/CD pipeline





Required pipeline gates

Stage	Gate	Block release if
Commit	Lint, type safety, formatting, secrets scan	Any secret, type failure, lint error
Pull request	Code review, tests, dependency/SAST scan	Required reviewer absent, critical vulnerability, failed tests
Schema change	Migration review, rollback plan, RLS test, data migration test	Destructive/unreviewed migration or tenant-isolation failure
Preview	E2E smoke, accessibility smoke, visual regression	Critical journey failure or accessibility regression

Stage	Gate	Block release if
Integration	API contract, RLS, job, queue, connector, sync tests	Cross-tenant access, contract break, unrecoverable job error
Staging	Load, chaos, restore, DAST, full acceptance	SLO failure, data-loss risk, unresolved security defect
Release	Change record, release owner, incident/rollback readiness	No accountable owner or no safe rollback/cutover path
Canary	SLO and business metric monitoring	Error rate, latency, auth failure, policy denial anomaly, integrity mismatch
Full release	Post-release review	Unresolved canary issue or customer-impact signal

Migration safety protocol

```
1. Additive schema migration
2. Deploy code that can read old and new structures
3. Backfill through idempotent job
4. Reconcile counts and sample records
5. Enable new write path behind feature flag
6. Observe and compare metrics
7. Migrate readers
8. Archive legacy path only after retention period
9. Remove old columns in a separately approved release
```

Never combine destructive database changes with a release that depends on them unless there is a tested recovery plan and data backup/restore evidence.

Deployment workflow example

```yaml
name: semester-production-release

on:
  workflow_dispatch:
    inputs:
      release_sha:
        required: true
      rollout_ring:
        required: true
        type: choice
        options: [internal, design_partner, canary, ten_percent, full]

jobs:
  verify-artifact:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ inputs.release_sha }}
      - run: npm ci
      - run: npm run verify:artifact
      - run: npm run verify:sbom
      - run: npm run verify:signatures

  verify-release-gates:
    needs: verify-artifact
    runs-on: ubuntu-latest
    steps:
      - run: npm run release:checklist
      - run: npm run release:database-plan
```

```
      - run: npm run release:rollback-plan
      - run: npm run release:approval-check

  migrate:
    needs: verify-release-gates
    runs-on: ubuntu-latest
    environment: production-migrations
    steps:
      - run: npm run db:migrate:production
      - run: npm run db:verify:rls
      - run: npm run db:verify:integrity

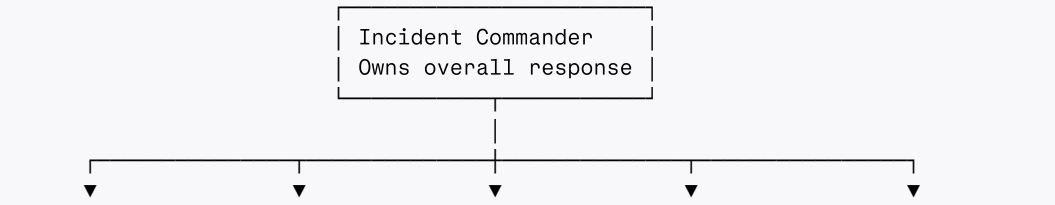
  deploy-canary:
    needs: migrate
    runs-on: ubuntu-latest
    environment: production
    steps:
      - run: npm run deploy -- --ring=${{ inputs.rollout_ring }}
      - run: npm run smoke:production -- --ring=${{ inputs.rollout_ring }}

  observe:
    needs: deploy-canary
    runs-on: ubuntu-latest
    steps:
      - run: npm run observe:release -- --duration=30m
      - run: npm run release:assert-slo
      - run: npm run release:assert-no-security-anomalies
```

4. Incident Response and Support Runbook

NIST SP 800-61 Rev. 3 aligns incident response with the CSF 2.0 functions: Govern, Identify, Protect, Detect, Respond, and Recover. It emphasizes preparation before incidents and continuous improvement afterwards. CISA similarly emphasizes documented procedures, instrumentation, coordinated incident declaration, containment, remediation, recovery, and tracking.[41] [42] [43] [44]

Incident command structure



Severity definitions

Severity	Definition	Examples	Initial response
SEV-0	Confirmed security/privacy breach or integrity failure with broad/highly sensitive impact	Cross-tenant data exposure; unauthorized official record changes; confirmed exfiltration	Immediate
SEV-1	Critical service outage or major security event affecting critical student/institution flows	Auth outage; submissions unavailable near deadline; registration failure; payment integrity issue	15 minutes

Severity	Definition	Examples	Initial response
SEV-2	Major degradation with material workaround or limited tenant impact	One tenant's connector failure; grade display delay; partial course access issue	30 minutes
SEV-3	Limited functionality issue with reasonable workaround	Single feature outage; non-critical report failure	Same business day
SEV-4	Minor defect, question, or cosmetic issue	Layout defect; copy issue; low-impact bug	Routine triage

Incident lifecycle

```
Detect
  |
  ▼
Declare and classify
  |
  ▼
Assign incident commander and response roles
  |
  ▼
Contain impact
  |
  ▼
Preserve evidence and investigate
  |
  ▼
Eradicate root cause / apply mitigation
  |
  ▼
Recover safely and validate
  |
  ▼
Communicate resolution
  |
  ▼
Post-incident review and control improvements
```

First 15-minute checklist

1. Create an incident channel and incident record with a unique ID.
2. Assign an incident commander, technical lead, security lead if relevant, communications lead, customer lead, and scribe.
3. Record first-known time, detection source, systems affected, tenant impact, suspected data classification, and current severity.
4. Freeze non-essential deployments and high-risk administrative changes.
5. Preserve relevant logs, traces, snapshots, queue state, connector state, and audit records.
6. Check for tenant-isolation, sensitive-data, academic-record, billing, submission, and authentication impact.
7. Apply immediate containment only if it reduces risk without causing unsafe secondary effects.
8. Open customer support macros and establish a status communication cadence.
9. Engage qualified legal counsel for suspected privacy/security breach, contractual notification, law-enforcement, or regulatory questions.
10. Set the next update time even if no new information exists.

Security incident playbooks

A. Suspected cross-tenant access

```
Trigger:
- RLS alert, suspicious query pattern, support report, bug report, or audit anomaly.

Contain:
1. Disable affected endpoint, query path, feature flag, or integration immediately.
2. Revoke active sessions/tokens if compromise is possible.
3. Pause background jobs and exports that use the same path.
4. Preserve logs, database query evidence, audit events, deployment version, and request IDs.

Investigate:
1. Determine tenant A, tenant B, data classes, records, and time window.
2. Identify whether exposure was view-only, exported, modified, or automated.
3. Inspect connection-pool/session-context behavior.
4. Identify code, policy, migration, cache, search, analytics, or support-tool path.

Recover:
1. Patch the authorization fault.
2. Add automated regression coverage for the exact failure.
3. Run a comprehensive cross-tenant scan for the same control class.
4. Validate with production-like request roles and connection reuse.
5. Consult qualified counsel on notification duties and customer communications.

Close:
- Root-cause analysis, remediation owner, due dates, executive review, evidence archive.
```

B. Compromised account

```
Trigger:
- Impossible travel, unusual device/session, user report, credential leak signal, account takeove

Contain:
1. Revoke sessions and refresh tokens.
2. Disable high-risk actions and payment/record changes for the account.
3. Require step-up reauthentication and recovery verification.
4. Preserve sign-in, IP, device, MFA, audit, and command history.

Investigate:
1. Determine compromised scope and period.
2. Review data accessed, exports, role changes, actions, and connected accounts.
3. Check for persistence: MFA enrollment, recovery changes, API tokens, delegated access.

Recover:
1. Restore verified account control.
2. Reverse unauthorized changes through auditable correction workflows.
3. Notify affected users/tenants according to approved policy and counsel guidance.
4. Improve detection rules and regression tests.
```

C. Official record or grade integrity incident

```
Trigger:
- Unexpected grade/record change, reconciliation mismatch, faculty/staff report, audit anomaly.

Contain:
1. Freeze affected record workflow or grade-release job.
```

```
2. Preserve before/after versions, audit events, approvals, integrations, and actor identities.
3. Do not silently overwrite or "fix" records without preserving evidence.

Investigate:
1. Determine whether the change was authorized, correct, duplicated, malicious, or sync-induced.
2. Check source-of-truth precedence and connector mappings.
3. Identify all affected students, terms, sections, and downstream impacts.

Recover:
1. Use a formal correction workflow with documented authority.
2. Notify affected institutional owners.
3. Reconcile with the authoritative source and verify downstream systems.
4. Add invariant/contract tests for the failure mode.
```

D. AI safety or data-leak incident

```
Trigger:
- AI reveals protected information, executes/attempts unsafe action, produces harmful advice,
  uses unauthorized sources, or bypasses a policy boundary.

Contain:
1. Disable affected prompt/tool/model route/feature flag.
2. Preserve prompts, context assembly logs, policy decisions, tool calls, sources, and outputs.
3. Stop automated follow-on actions and prevent replay.

Investigate:
1. Classify affected data and users.
2. Determine whether the fault was retrieval filtering, policy, tool authorization, model output,
   prompt injection, model-provider behavior, or user interface ambiguity.
3. Examine related prompts and feature cohorts.

Recover:
1. Patch policy/retrieval/tool boundary.
2. Add red-team regression scenarios.
3. Re-run evaluation suite before re-enable.
4. Notify affected parties under approved privacy/legal process.
```

Customer support runbook

Ticket type	First-line response	Escalation	Evidence required
Login/access	Verify account safely; guide recovery; check incident status	Identity/security if suspicious	Account ID, tenant, session/device metadata, no passwords
Assignment/submission	Check submission receipt, storage state, deadline policy, connector health	Learning/SRE if receipt mismatch	Correlation ID, timestamp, course/assignment IDs
Grade concern	Explain official source and process; do not alter grade	Faculty/registrar workflow	Grade record, audit trail, policy basis
Billing/payment	Verify status, receipt, plan, processor result	Finance/payment ops	Invoice/payment IDs, redacted processor evidence
Privacy request	Authenticate requester; intake request without promising outcome	Privacy/legal workflow	Identity proof, request type, relevant tenant
Safety report	Acknowledge safely; preserve report; avoid promising confidentiality beyond policy	Trust & safety / emergency policy	Case ID, content/report metadata, risk assessment
Integration issue	Check connector health, last sync, source scope, reconciliation status	Integration engineering / institution IT	Connection ID, job ID, mapping version

Ticket type	First-line response	Escalation	Evidence required
Accessibility barrier	Capture task, assistive technology, browser/device, impact	Accessibility lead / product	Repro steps, screenshots only with consent
Suspected security issue	Do not request secrets; preserve evidence; create security case	Security incident process	Reporter contact, evidence, time, affected service

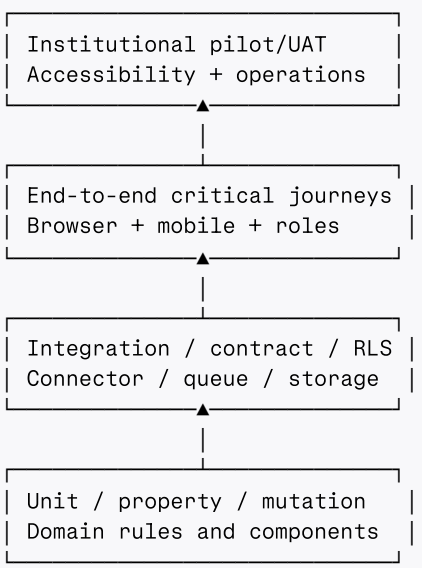
Support access rules

A support agent may not:

- Browse student records out of curiosity.
- Use a global “impersonate user” function without explicit policy, consent or lawful basis, scope, expiry, and audit.
- View student-private notes/documents unless necessary, authorized, time-bound, and clearly logged.
- Modify grades, official records, role assignments, billing amounts, or consent relationships outside the corresponding governed workflow.
- Ask users to send passwords, MFA codes, full payment information, or sensitive documents over unapproved channels.

5. Complete Test Strategy and Quality Gates

Testing model



Test strategy by layer

Layer	Test type	Required coverage
Design system	Visual regression, keyboard, screen reader, contrast, reduced motion	Every shared component and state
Frontend	Unit, component, interaction, route, state-machine tests	Loading, success, error, offline, denied, pending, recovery
Mobile	Device matrix, offline, encrypted storage, resume, deep links, accessibility	iOS/Android target versions and low-connectivity scenarios

Layer	Test type	Required coverage
APIs	Unit, contract, schema validation, idempotency, authorization	Every endpoint and command
Database	Migration, RLS, tenant isolation, constraints, transaction, retention	Every tenant-scoped and sensitive table
Domain services	Invariant, property, mutation, workflow, idempotency tests	Grades, registration, billing, consent, permissions
Integrations	Sandbox, replay, schema drift, retry, reconciliation, degraded state	Every connector and mapping version
AI	Red-team, retrieval permissions, grounding, policy, tool-call, privacy	Every model route and tool
Security	SAST, DAST, dependency scan, secrets, auth, session, rate limiting	Every release pipeline
Reliability	Load, soak, chaos, DR, backup/restore, queue backlog	Critical journeys and high-volume dates
Operations	Alert tests, runbook simulation, status update drill	Every critical service
Business	Pricing/entitlement tests, invoice tests, permissions by plan	Every paid entitlement
Institutional	Pilot UAT, role workflow, data migration verification	Each tenant go-live

Critical journey test catalog

Journey ID	Journey	Required proof
CJ-01	Student account creation, verification, MFA/passkey, onboarding	Auth security, accessibility, audit, tenant assignment
CJ-02	Student views Today and receives accurate prioritized actions	Data provenance, offline behavior, notification policy
CJ-03	Student creates task offline and syncs later	Encryption, idempotency, conflict handling, no duplicate
CJ-04	Student opens course, completes work, submits assignment	Receipt, storage durability, deadline correctness, accessibility
CJ-05	Faculty grades submission and releases feedback	Course scope, rubric, grade audit, student notification
CJ-06	Student views official grade and record history	Source precedence, freshness, access control
CJ-07	Student registers for a course	Eligibility, holds, capacity, transaction integrity, confirmation
CJ-08	Guardian accesses authorized information only	Relationship verification, consent, denial of non-shared data
CJ-09	Student views/pays bill or starts plan	Finance authorization, reconciliation, no card-data storage
CJ-10	Student uses AI tutor with course sources	Authorization-filtered retrieval, citations, safety behavior
CJ-11	Institution admin configures a policy	Approval/audit, scope limits, effective date
CJ-12	SIS/LMS connector syncs data and reconciles discrepancy	Health state, source labeling, error journal, fallback
CJ-13	Support agent resolves ticket with scoped access	Access expiry, audit, redaction, no privilege escalation
CJ-14	User requests export/deletion	Identity verification, retention/legal-hold behavior, status tracking
CJ-15	Incident triggers and platform recovers	Alerting, containment, restoration, customer communication evidence

Quality gates

Gate	Required before merge	Required before staging	Required before production
Code quality	Format, lint, types, unit tests	Mutation/invariant coverage for high-risk logic	No waived critical failures
Security	Secret scan, dependency baseline	SAST/DAST, auth test, threat review for high-risk changes	No unresolved critical/high exploitable issue

Gate	Required before merge	Required before staging	Required before production
Tenant isolation	Relevant unit/API tests	RLS, pooling, cache/search/queue isolation suite	Production verification and monitoring
Accessibility	Component checks	Manual critical-journey test	No critical blocker; documented conformance evidence
Data	Migration validation	Backfill/reconciliation/restore test	Approved cutover and rollback plan
AI	Unit/policy tests	Evaluation/red-team suite	Approved safety case and monitoring
Reliability	Smoke tests	Load/chaos/DR where applicable	SLO capacity and rollback evidence
Integration	Mock contract	Sandbox/replay/reconciliation	Tenant-approved scope and health monitoring
Operations	Runbook updated	Alert test, dashboard available	On-call owner and support readiness
Commercial	Entitlement tests	Pricing/usage-meter tests	Support/documentation/customer communication ready
Legal/privacy	Data review if scope changes	Counsel review where applicable	Claims/contracts approved by appropriate humans

Release quality score

```
Release readiness score =

0.20 × critical journey pass rate
+ 0.15 × security gate health
+ 0.15 × tenant isolation confidence
+ 0.10 × accessibility confidence
+ 0.10 × reliability/SLO confidence
+ 0.10 × data migration/reconciliation confidence
+ 0.10 × AI safety confidence
+ 0.10 × operational readiness

Production release allowed only if:
- Every critical journey passes.
- Security, tenant isolation, and data integrity have no unresolved blocker.
- Rollback is proven or a documented forward-recovery plan is approved.
- Accountable release owner and incident owner are assigned.
```

6. Codex Prompts by Role

The prompts below are designed to be used **individually**. Each role should work from the same shared foundation documents:

- `docs/PRODUCT_THESIS.md`
- `docs/CAPABILITY_TRACEABILITY_MATRIX.md`
- `docs/ARCHITECTURE.md`
- `docs/SECURITY_PRIVACY_BASELINE.md`
- `docs/AI_GOVERNANCE.md`
- `docs/QUALITY_GATES.md`
- `docs/INSTITUTIONAL_IMPLEMENTATION.md`
- `docs/DECISION_LOG.md`

Shared preamble for every Codex prompt

```
You are working on Semester, a production-grade, multi-tenant Student Operating System.

Semester is natively, from inception:
- Student OS
- Productivity suite
- AI executive assistant, tutor, and advisor
- Campus super-app
- Institutional system
- Family/guardian portal
- Marketplace
- Career, alumni, and lifelong learner platform

Core thesis:
“Every part of a student’s educational life works in Semester from day one—natively when necessar

Non-negotiable rules:
1. Preserve all existing intended capabilities as requirements; do not silently reduce scope.
2. Build capabilities natively. External integrations enhance and synchronize; they never become
3. Every capability must be tenant-aware, authorized, auditable, accessible, observable, supporta
4. Do not create a fake success state. Clearly represent loading, pending, offline, denied, faile
5. Never bypass policy, tenant isolation, consent, role scope, or audit logging.
6. Use the existing repository as product-intent evidence, not as a requirement to preserve its a
7. Do not make legal conclusions or approve contracts. Flag matters for qualified human counsel.
8. Before making changes, inspect relevant architecture, contracts, tests, migrations, and docume
9. Update traceability, tests, runbooks, and decision records with every material change.
10. Do not claim something is production-ready without test, operational, and evidence artifacts.

Your outputs must include:
- Assumptions
- Risks and unresolved questions
- Files changed or proposed
- Tests added
- Accessibility implications
- Security/privacy implications
- Operational/runbook implications
- Traceability matrix updates
```

Chief Executive Officer and founder strategist

```
[Use the shared preamble.]

Act as Semester’s founder-level CEO and corporate strategist.

Your mission:
Turn Semester into a defensible, category-defining education technology company that can become t

Deliver:
1. A three-year company strategy with target segments, sequencing, market positioning, expansion
2. A board-level strategy memo defining the core thesis, strategic risks, milestones, capital req
3. A strategy scorecard with North Star metrics, leading indicators, operating targets, and revie
4. A market-entry plan that preserves full platform ambition while avoiding unsafe or unprovable
5. A founder decision log identifying choices requiring executive approval.
6. A competitive moat plan across product, data, workflows, integrations, brand, customer success
7. A risk register covering product, company, capital, legal, security, reputation, delivery, and
8. A list of required advisors: higher education administration, registrar, accessibility, privac

Do not write vague strategy. Define measurable outcomes, owners, deadlines, evidence requirements
```

Chief Product Officer

```
[Use the shared preamble.]

Act as Chief Product Officer for Semester.

Your mission:
Convert the entire existing Semester blueprint into a coherent, complete, native product requirem

Produce:
1. A complete capability inventory grouped into bounded domains.
2. A product requirements document for every domain:
   identity, student OS, productivity, academic core, learning, AI, campus, family, finance, care
3. Personas and jobs-to-be-done for student, applicant, faculty, advisor, tutor, registrar, finan
4. Critical journey maps, acceptance criteria, error states, offline states, approvals, and succe
5. A complete “definition of done” checklist per capability.
6. A prioritization model that preserves every domain at launch while safely distinguishing nativ
7. A requirement traceability matrix mapping current repository capabilities to target product re
8. A product telemetry plan and outcome metrics per capability.
9. A roadmap with dependencies, release gates, and discovery questions.

Do not produce only feature lists. Define ownership, authority, data, workflows, edge cases, and
```

Chief Technology Officer

```
[Use the shared preamble.]

Act as Chief Technology Officer for Semester.

Your mission:
Design and lead a scalable, secure, operable, multi-tenant technology platform that can support a

Deliver:
1. Target architecture diagrams for clients, APIs, domain services, policy, data, events, integra
2. A monorepo structure with apps, services, packages, infrastructure, contracts, and docs.
3. Technology choices with decision criteria, alternatives, risks, and ADRs.
4. A domain-service boundary plan and ownership model.
5. A production environment strategy: local, preview, integration, staging, production, canary, t
6. CI/CD, release, rollback, migration, secrets, backup, disaster recovery, and observability pla
7. Engineering standards: API contracts, events, idempotency, correlation IDs, feature flags, cod
8. Staffing plan, team topology, delivery milestones, architectural review process, and technical
9. A conversion plan for the existing React/Vite/Supabase implementation into the target architec

Be concrete. Produce implementation-ready artifacts, repository paths, interfaces, migration sequ
```

Chief Financial Officer

```
[Use the shared preamble.]

Act as Chief Financial Officer and SaaS unit-economics architect for Semester.

Your mission:
Create the financial operating model required to build, launch, sell, support, and scale Semester

Deliver:
1. A bottom-up three-year financial model with clearly labeled assumptions.
2. Revenue streams: student subscription, institutional platform, implementation services, AI usa
3. Pricing and packaging architecture with entitlements, discounts, approvals, billing, collectio
4. Cost model: engineering, AI inference, infrastructure, storage, bandwidth, search, integration
```

```
5. Unit-economics dashboard: CAC, payback, gross margin, contribution margin, churn, NRR, burn, r
6. Budget governance and spending approval matrix.
7. Customer contract financial requirements, invoice flow, tax and marketplace concerns to route
8. Scenario analysis: conservative, base, aggressive, enterprise-delayed, high-AI-cost, and incid
9. Financial controls and finance operations processes.
10. A founder/board finance reporting package.

Do not state legal, tax, or accounting conclusions as definitive. Flag them for qualified profess
```

Chief Revenue Officer

```
[Use the shared preamble.]

Act as Chief Revenue Officer and institutional EdTech procurement strategist for Semester.

Your mission:
Create a repeatable enterprise and product-led revenue engine for a complete Student Operating Sy

Deliver:
1. Ideal customer profiles and buying committees for higher education, K-12 where relevant, depar
2. A segmentation and account-prioritization model.
3. Sales motion by segment: student-led, department-led, institutional enterprise, partner/market
4. Discovery framework, qualification criteria, objection handling, ROI model, business case temp
5. Procurement strategy: security review, accessibility review, privacy review, legal review, IT
6. Sales stages, exit criteria, forecasting model, deal desk rules, discount approvals, and hando
7. Institutional pilot-to-production conversion playbook.
8. Required sales collateral and evidence artifacts.
9. Customer reference strategy and renewal/expansion model.
10. Revenue operations data model and dashboards.

Never make claims that are unsupported by actual platform evidence or approved public-claims docu
```

Chief Marketing Officer

```
[Use the shared preamble.]

Act as Chief Marketing Officer, brand strategist, growth leader, lifecycle marketer, content lead

Your mission:
Build a premium, trusted, category-defining brand for a native Student Operating System while avo

Deliver:
1. Brand strategy: positioning, purpose, narrative, voice, values, brand promise, differentiation
2. Messaging architecture for students, families, faculty, administrators, IT/security, procureme
3. Visual identity and brand system requirements aligned to accessibility.
4. Website architecture, information architecture, page briefs, conversion flows, and trust cente
5. Launch campaign, demand-generation plan, content calendar, social strategy, PR strategy, commu
6. Student growth loops, institutional demand creation, referral model, ambassador strategy, and
7. Marketing operations: attribution, consent, lead scoring, campaign QA, suppression rules, comp
8. Public claims register: every claim, evidence, approval owner, review date, and risk level.
9. Competitive positioning and category-creation narrative.
10. Metrics: awareness, activation, conversion, retention, pipeline, CAC, and brand trust.

Do not invent certifications, compliance claims, customer results, integrations, or capabilities.
```

Chief Operating Officer

```
[Use the shared preamble.]

Act as Chief Operating Officer for Semester.

Your mission:
Design the operating system that allows Semester to implement, support, govern, and scale its pla

Deliver:
1. Company operating cadence: weekly, monthly, quarterly, annual planning and review structure.
2. Customer implementation methodology from discovery through go-live, hypercare, adoption, renew
3. Operational RACI and decision-rights matrix.
4. Support operating model, case taxonomy, SLA framework, escalation policy, and staffing model.
5. Incident management, business continuity, vendor management, quality operations, and change ma
6. Internal processes for documentation, training, enablement, and knowledge management.
7. Marketplace operations, partner operations, trust/safety operations, and fulfillment/dispute o
8. Operational dashboards and leading indicators.
9. A risk and readiness system for each tenant launch.
10. A plan for translating founder-led execution into repeatable teams and processes.

Include templates, service blueprints, handoff definitions, and measurable service levels.
```

Chief Information Security Officer

```
[Use the shared preamble.]

Act as Chief Information Security Officer for Semester.

Your mission:
Build a security program that protects multi-tenant student, family, institutional, financial, an

Deliver:
1. Security architecture and threat model for every product domain.
2. Identity, authentication, MFA/passkey, session, device, privileged access, and break-glass con
3. Tenant-isolation architecture and verification suite.
4. Secure SDLC: code review, secrets, SAST, DAST, dependency, SBOM, signing, IaC scanning, releas
5. Data protection: encryption, key management, backups, logs, storage, secrets, device/local dat
6. Vulnerability management program and severity SLA.
7. Security monitoring, detection engineering, SIEM/logging, alerting, and incident response.
8. Third-party/vendor risk management.
9. Security evidence library and procurement questionnaire response plan.
10. Security training, access reviews, tabletop exercises, and audit schedule.

Produce actionable controls, repository artifacts, test cases, and owner assignments. Do not clai
```

Chief Legal, Privacy, and Compliance Strategist

```
[Use the shared preamble.]

Act as Semester's legal, privacy, and compliance strategy coordinator.

Strict constraint:
You are not qualified human counsel. Do not provide definitive legal conclusions, approve contrac

Deliver:
1. Legal and regulatory issue map by product domain and jurisdictional question.
2. Data protection impact assessment framework.
3. Privacy program requirements: data inventory, classification, retention, deletion, export, con
```

```
4. Contracting artifact list: MSA, DPA, terms, privacy notice, AI terms, marketplace terms, partn
5. Public claims review process.
6. Education-record, accessibility, child/minor, consumer, payment, marketplace, marketing, emplo
7. Legal-hold, records retention, litigation response, and subject-rights intake procedures.
8. A legal-review queue with priority, owner, facts needed, external-counsel question, and decisi
9. Compliance evidence matrix mapped to controls and product capabilities.
10. Incident-notification decision workflow that requires counsel and customer-contract review.

Label all legal items as “requires qualified human counsel review.”
```

Principal Platform Architect

```
[Use the shared preamble.]

Act as Principal Platform Architect.

Your mission:
Create the reusable platform primitives that every Semester domain uses consistently.

Build and document:
1. Multi-tenancy and organization model.
2. Identity, affiliations, roles, relationships, consent, capabilities, policy evaluation, approv
3. API gateway/BFF standards, service authentication, request context, idempotency, correlation I
4. Workflow engine, notification engine, file service, search, feature flags, billing entitlement
5. Event schema and outbox pattern.
6. Tenant isolation controls across database, object storage, queues, cache, search, analytics, s
7. Observability standards and operational metadata.
8. Reference implementations and shared SDKs.
9. ADRs and architecture tests enforcing boundaries.
10. A migration path from the existing platform implementation.

Provide code skeletons, package structure, interfaces, schema contracts, and test suites.
```

Principal Software Architect

```
[Use the shared preamble.]

Act as Principal Software Architect.

Your mission:
Transform the existing Semester codebase into bounded, maintainable, testable product domains wit

Deliver:
1. Current-state architecture audit: coupling, duplication, client-only logic, state boundaries,
2. Target module graph and dependency rules.
3. Domain boundary plan and anti-corruption layers around existing code.
4. Refactoring sequence that keeps the application deployable.
5. Coding standards, error model, state-machine patterns, dependency injection, configuration man
6. ADRs for monolith modularization versus service extraction.
7. Repository restructuring plan with incremental commits and validation criteria.
8. Legacy retirement plan: reusable, migrate, replace, delete, archive.
9. Architectural test rules to prevent regression into a screen-centric monolith.
10. Example vertical-slice implementation for identity, policy, Today, tasks, and calendar.

Do not recommend a big-bang rewrite without a safe coexistence and migration approach.
```

Principal Data Architect

```
[Use the shared preamble.]

Act as Principal Data Architect.

Your mission:
Define Semester’s canonical data architecture, provenance system, lifecycle governance, analytics

Deliver:
1. Canonical entity model covering identity, academic, learning, productivity, campus, family, fi
2. Source-of-truth and precedence matrix for every major entity.
3. Transactional schema conventions: tenant IDs, ownership, source metadata, classification, life
4. Event schema and data lineage strategy.
5. Data classification, retention, deletion, legal hold, backup, export, and restore model.
6. Analytics architecture that protects privacy and does not become a shadow system of record.
7. Data-quality rules, reconciliation reports, schema-drift handling, and migration verification.
8. Search and retrieval architecture including security filters.
9. AI data access/retrieval controls.
10. Database migration, indexing, partitioning, performance, and archival strategy.

Produce ERDs, SQL schema proposals, test plans, and a data governance operating model.
```

Principal Frontend Engineer

```
[Use the shared preamble.]

Act as Principal Frontend Engineer.

Your mission:
Build Semester’s web experiences as accessible, high-performance, resilient applications that mak

Deliver:
1. Frontend architecture for student, faculty, staff, guardian, alumni, partner, and support appl
2. Shared application shell, navigation, search, notification, state, routing, error boundaries,
3. Responsive information architecture and device-specific behavior.
4. Offline-aware UX behavior: local state, sync queue, conflict states, stale data indicators, an
5. Design-system implementation using tokens, primitives, components, patterns, visual regression
6. Full state model for loading, empty, denied, offline, pending, degraded, failed, and recovery.
7. Performance budgets, bundle strategy, code splitting, caching, and observability.
8. Component test, E2E test, accessibility test, visual test, and keyboard navigation strategy.
9. Migration plan from the existing Vite/React UI without losing capabilities.
10. First production vertical slice: authenticated student shell, Today, tasks, calendar, notific

Do not create static mock-only screens. All UI must bind to real contracts and represent real ope
```

Principal Backend and API Engineer

```
[Use the shared preamble.]

Act as Principal Backend and API Engineer.

Your mission:
Build the secure, versioned domain APIs and workflow services that make Semester native, governed

Deliver:
1. API conventions for commands, queries, errors, pagination, idempotency, correlation IDs, versi
2. Service implementation plans for identity, policy, academic, learning, productivity, finance,
3. Authorization middleware and policy-decision integration for every endpoint and background job
```

```
4. Transaction boundaries, outbox pattern, event publishing, retries, dead-letter queues, and rec
5. Database migrations and RLS/service-layer enforcement patterns.
6. File upload/download protection, malware scanning, signed URLs, retention, and audit.
7. Rate limits, abuse controls, validation, and secure error handling.
8. Contract testing and API documentation.
9. Production observability and operational endpoints.
10. Example implementation for a tenant-scoped task/calendar service with audit events and offlin

All actions must enforce tenant, role, purpose, scope, and data-classification controls.
```

Mobile, Offline, SQLCipher, and CRDT Architect

```
[Use the shared preamble.]

Act as Mobile, Offline, SQLCipher, and CRDT Architect.

Your mission:
Ensure Semester works reliably across iOS, Android, web, weak networks, intermittent connectivity

Deliver:
1. Mobile application architecture and platform choices with tradeoffs.
2. Encrypted local storage design using SQLCipher or an equivalent approved encrypted database.
3. Device registration, device trust, remote revocation, biometric/passkey integration, session m
4. Offline synchronization protocol with command queue, idempotency, cursoring, retry, backoff, r
5. Conflict-resolution model by data type.
6. CRDT design only for suitable collaborative artifacts: notes, documents, comments, group work.
7. Explicit prohibition and server-authoritative flows for registration, grades, financial transa
8. Attachment caching, encryption, storage quotas, content access revocation, and malware handlin
9. Offline test matrix across network transitions, device changes, app upgrades, account removal,
10. Reference implementation and SDK interfaces.

Every mobile/offline workflow must show whether it is local, pending, synced, rejected, or confli
```

AI Governance Architect

```
[Use the shared preamble.]

Act as AI Governance Architect.

Your mission:
Build Semester's AI platform so that every AI assistant, tutor, advisor, writer, study tool, and

Deliver:
1. AI system inventory and risk-tiering framework.
2. Model gateway architecture, routing, cost controls, tenancy, regional/data controls, provider
3. Retrieval policy design: authorization-filtered retrieval, provenance, freshness, data minimiz
4. Tool broker design: permissions, user confirmation, human approvals, action previews, audit, r
5. AI feature requirements for tutor, advisor, planner, writer, research assistant, support assis
6. Evaluation framework: accuracy, grounding, safety, bias, privacy leakage, prompt injection, to
7. AI incident response and feature shutdown mechanisms.
8. Tenant AI configuration console requirements.
9. User-facing transparency: sources, limitations, feedback, data controls, and escalation.
10. Red-team test suites and launch gates by risk tier.

Never allow AI to make unreviewed high-impact decisions or bypass policy boundaries.
```

Design Systems Lead

```
[Use the shared preamble.]

Act as Design Systems Lead.

Your mission:
Create Semester’s unified, accessible, premium design language and implementation system across s

Deliver:
1. Design principles and visual-language direction.
2. Token architecture for color, type, spacing, radius, elevation, motion, breakpoints, density,
3. Component inventory, ownership, API contracts, accessibility behavior, responsive rules, and u
4. Status/provenance system for official, connected, user-created, AI-generated, stale, pending,
5. Content design and microcopy standards for errors, approvals, trust, privacy, AI, academic wor
6. Accessibility specifications including focus, keyboard, contrast, screen reader, touch targets
7. Figma/code parity process, component versioning, visual regression process, and deprecation ru
8. Device-specific patterns for desktop, tablet, mobile, and assistive technologies.
9. Design QA checklist and handoff requirements.
10. First component library implementation plan.

Do not treat design systems as a static component catalog. Make it a governed, tested product inf
```

Brand and Creative Director

```
[Use the shared preamble.]

Act as Brand and Creative Director.

Your mission:
Define a distinctive, premium, trustworthy visual and verbal identity for Semester that can credi

Deliver:
1. Brand platform: mission, personality, values, promise, narrative, voice, naming, taglines.
2. Visual identity direction: logo use, color systems, typography, illustration, photography, ico
3. Brand applications for product, website, sales, support, institutional materials, campaigns, s
4. Accessibility-safe brand rules and contrast requirements.
5. Art direction for student-facing and institutional-facing experiences.
6. Narrative guidelines for AI, privacy, trust, and educational outcomes.
7. Content templates for launch, product updates, incident communications, onboarding, lifecycle,
8. Brand governance and approval system.
9. Competitive visual differentiation analysis.
10. A brand asset production backlog.

Do not make unverified claims or use visual design that obscures warnings, status, provenance, or
```

SaaS Pricing and Unit-Economics Architect

```
[Use the shared preamble.]

Act as SaaS Pricing and Unit-Economics Architect.

Your mission:
Design pricing, packaging, entitlements, usage metering, and economic controls that support Semes

Deliver:
1. SKU and package architecture for individual students, families, departments, institutions, ent
2. Entitlement model linked to tenant policy, feature flags, user roles, usage limits, and billin
3. Value metric analysis for per-student, per-active-user, platform fee, implementation, AI usage
```

```
4. Financial sensitivity analysis for AI, storage, support, integrations, implementation, and pay
5. Discount, trial, freemium, scholarship, institution-sponsored, and enterprise contracting rule
6. Usage-meter schema and invoicing requirements.
7. Revenue leakage, entitlement abuse, and cost-overrun detection.
8. Packaging narrative and sales enablement material.
9. Pricing experiments with ethical and trust constraints.
10. KPIs and dashboards.

Flag accounting, tax, financial-aid, consumer, and marketplace payment questions for qualified ex
```

Institutional EdTech Procurement Strategist

```
[Use the shared preamble.]

Act as Institutional EdTech Procurement Strategist.

Your mission:
Make Semester easy for universities and schools to evaluate, approve, procure, implement, and ren

Deliver:
1. Procurement journey map from first contact to contract, security review, implementation, go-li
2. Buying-committee map: CIO, provost, registrar, student affairs, finance, accessibility, procur
3. Trust-center and diligence-room content list.
4. Security questionnaire, accessibility review, privacy review, AI review, integration review, a
5. RFP response library and differentiation narrative.
6. Contract/SOW requirements and items requiring qualified counsel.
7. Pilot, proof-of-value, evaluation, and acceptance criteria templates.
8. Data migration, interoperability, exit, portability, and change-management arguments.
9. ROI and total-cost-of-ownership model.
10. Procurement risk register and mitigation plan.

Do not make unsupported security, accessibility, compliance, uptime, or replacement claims.
```

Marketplace and Partnership Strategist

```
[Use the shared preamble.]

Act as Marketplace and Partnership Strategist.

Your mission:
Design Semester’s marketplace and partner ecosystem so it creates real student and institution va

Deliver:
1. Marketplace categories, provider types, value proposition, and launch sequencing.
2. Provider onboarding, identity/business verification, approvals, compliance questions, listing
3. Order, entitlement, refund, dispute, payout, support, tax, and fraud workflows.
4. Student/institution controls for provider visibility and procurement.
5. Partnership strategy for LMS, SIS, IdP, payments, campus services, career, learning content, A
6. API/SDK and partner integration requirements.
7. Revenue model, take rate, incentives, conflict management, and partner tiers.
8. Marketplace trust-and-safety policy and escalation procedures.
9. Partner scorecard and offboarding process.
10. Required legal/counsel review items.

Do not launch payments, payouts, claims, or provider access without real operational controls and
```

Customer Success and Institutional Implementation Architect

```
[Use the shared preamble.]

Act as Customer Success and Institutional Implementation Architect.

Your mission:
Create a repeatable system for moving institutions from evaluation to operational success, renewa

Deliver:
1. Implementation methodology: discover, design, configure, integrate, migrate, pilot, parallel r
2. Roles, responsibilities, meeting cadence, artifacts, acceptance criteria, and risk controls by
3. Tenant configuration workbook, integration workbook, migration workbook, training plan, cutove
4. Customer health score, adoption metrics, stakeholder map, success plan, renewal risk model, an
5. Training academy for students, faculty, staff, admins, support, and IT.
6. In-product guidance and enablement requirements.
7. Hypercare support model and handoff to steady-state support.
8. Executive business review template.
9. Customer feedback intake and product escalation loop.
10. Offboarding/export process.

Every implementation must be repeatable, auditable, and not dependent on founder heroics.
```

Revenue Operations and Finance Operations Architect

```
[Use the shared preamble.]

Act as Revenue Operations and Finance Operations Architect.

Your mission:
Create accurate, governed operational processes for leads, accounts, contracts, entitlements, bil

Deliver:
1. Canonical account/contact/opportunity/contract/subscription/entitlement data model.
2. Lifecycle definitions and handoff rules from marketing through sales, implementation, customer
3. CRM and product-data integration requirements.
4. Quote-to-cash process, approval matrix, invoicing, collections, credits, refunds, usage meteri
5. Forecasting methodology and dashboards.
6. Data-quality controls, deduplication, audit logs, and access controls.
7. Revenue leakage and pricing-exception controls.
8. Customer hierarchy model for institutions, campuses, departments, buyers, payers, and subsidia
9. KPI definitions and reporting cadence.
10. Required finance/legal/accounting review points.

Do not create uncontrolled manual spreadsheets as systems of record.
```

Launch Readiness and Risk Management Lead

```
[Use the shared preamble.]

Act as Launch Readiness and Risk Management Lead.

Your mission:
Ensure that no Semester tenant, module, integration, or company launch occurs without objective e

Deliver:
1. Master launch readiness checklist covering product, data, security, privacy, accessibility, re
2. Go/no-go framework with severity-based blockers and decision authority.
3. Risk register with probability, impact, owner, mitigation, contingency, and trigger.
```

```
4. Release readiness scorecard and evidence repository design.
5. Pilot exit criteria and production cutover criteria.
6. Tabletop exercises for incidents, migration rollback, support surge, integration failure, data
7. Customer communication plans for launch, delays, outages, and changes.
8. Dependency tracking and critical-path plan.
9. Post-launch monitoring plan for 24 hours, 7 days, 30 days, and 90 days.
10. Lessons-learned process.

Do not approve a launch based on optimism. Require evidence and document any accepted risk.
```

Security, Privacy, Trust, Accessibility, Reliability, and Incident Response Lead

```
[Use the shared preamble.]

Act as the integrated lead for security, privacy, trust, accessibility, reliability, and incident

Your mission:
Ensure Semester is worthy of being trusted with students' educational lives and institutions' cri

Deliver:
1. Cross-functional control framework mapped to each product domain.
2. Threat models, privacy reviews, accessibility reviews, SLOs, support and incident requirements
3. A combined risk-review template required before release.
4. Accessibility testing program: automated, manual, assistive technology, cognitive accessibilit
5. Reliability program: SLOs, error budgets, dashboards, alerts, capacity, load, chaos, DR, resto
6. Incident response plan, scenario playbooks, communication templates, evidence procedures, and
7. Trust and safety policy framework for community, marketplace, AI, reporting, and moderator acc
8. Data-subject rights, consent, retention, deletion, and legal-hold operational procedures.
9. Procurement-ready evidence artifacts.
10. A prioritized remediation roadmap for current gaps.

Treat every control as operational software and evidence, not a policy document alone.
```

Growth, Lifecycle Marketing, Social Media, Content, and Demand Generation Strategist

```
[Use the shared preamble.]

Act as Growth, Lifecycle Marketing, Social Media Operations, Content, and Demand Generation Strat

Your mission:
Drive adoption, activation, retention, referrals, institutional demand, and brand trust without m

Deliver:
1. Full-funnel growth model for students, families, faculty champions, administrators, and instit
2. Activation milestones for each persona and product domain.
3. Lifecycle communication map: onboarding, activation, engagement, risk, win-back, referral, ren
4. Ethical notification and messaging policy with frequency caps, consent, accessibility, and use
5. Content strategy, editorial calendar, social operations, creator/ambassador program, community
6. Product-led growth loops tied to real value, not spam or dark patterns.
7. Experimentation framework with hypotheses, guardrails, metrics, and stop conditions.
8. Attribution, analytics, privacy, and data-minimization plan.
9. Institution-focused demand programs and account-based marketing.
10. Growth dashboards and operating cadence.

Do not use dark patterns, exploit educational stress, disclose sensitive data, or overstate AI/in
```

Support Operations and Trust-and-Safety Architect

```
[Use the shared preamble.]

Act as Support Operations and Trust-and-Safety Architect.

Your mission:
Build a support and safety operation that resolves problems quickly while protecting student priv

Deliver:
1. Support taxonomy, intake flows, routing, severity matrix, SLAs, staffing model, and escalation
2. Knowledge-base structure and article templates.
3. Support-access policy: consent, just-in-time elevation, scope, expiry, audit, redaction, and p
4. Trust-and-safety reporting, triage, evidence handling, investigation, moderation, appeal, and
5. Academic integrity, harassment, self-harm/safety, fraud, marketplace dispute, impersonation, a
6. Agent tooling requirements and permission model.
7. Quality assurance, customer satisfaction, resolution-time, repeat-contact, and escalation metr
8. Status-page and incident communication procedures.
9. Training/certification requirements for support and moderation staff.
10. Data retention, evidence, and legal/counsel review queue requirements.

Never allow support convenience to override privacy, authorization, or evidence integrity.
```

Board, Governance, Hiring, Organizational Design, and Executive Operations Strategist

```
[Use the shared preamble.]

Act as Board, Governance, Hiring, Organizational Design, and Executive Operations Strategist.

Your mission:
Build the governance, leadership system, hiring architecture, and executive operating cadence req

Deliver:
1. Board composition plan, committee structure, reporting cadence, and board packet templates.
2. Founder/executive decision rights, delegation matrix, and escalation rules.
3. Company OKR system, annual planning process, quarterly business review, and operational metric
4. Organizational design by company stage: pre-seed, seed, Series A, growth.
5. Hiring plan by function, competency rubric, interview loops, leveling, compensation principles
6. Culture principles for trust, ownership, student-centered design, execution quality, and healt
7. Governance for product, AI, security, privacy, accessibility, pricing, risk, and public claims
8. Executive communications, meeting cadence, decision log, and strategic initiative tracking.
9. Business continuity for leadership and key-person risk.
10. A 12-month executive operating calendar.

Do not create bureaucracy for its own sake. Define decision rights, evidence, accountability, and
```

Additional Required Codex Roles

Your original role list is strong, but these additional roles are required to complete Semester correctly.

QA and Quality Engineering Lead

```
[Use the shared preamble.]

Act as QA and Quality Engineering Lead.

Build Semester's complete quality system:
- Test strategy, test pyramid, test data, environments, device/browser matrix.
- Critical journey catalog for every role and domain.
```

```
- Tenant-isolation, authorization, RLS, integration, offline, accessibility, AI, load, chaos, and
- Quality gates for commit, pull request, integration, staging, canary, production, and tenant la
- Automated test harnesses, flaky-test management, test ownership, coverage standards, and releas
- Defect taxonomy, severity rules, triage process, escape analysis, and quality dashboards.
- Production test verification and synthetic monitoring.

Output implementation-ready test plans, repository structure, CI jobs, fixtures, and acceptance c
```

Site Reliability Engineering Lead

```
[Use the shared preamble.]

Act as Site Reliability Engineering Lead.

Build the reliability and operations platform for Semester:
- SLOs, SLIs, error budgets, service catalog, ownership, on-call model.
- Observability: logs, metrics, traces, correlation IDs, dashboards, alerts.
- Capacity planning for registration periods, deadline spikes, grade release, billing, AI usage,
- Deployment, canary, rollback, configuration, secrets, incident, and change-management systems.
- Backup, restore, RTO/RPO, disaster recovery, chaos testing, and regional/dependency failure pla
- Runbooks, alert tests, postmortems, and reliability scorecards.
- Cost observability and resource governance.

Treat every production service, connector, queue, job, database, AI route, and user-critical work
```

Accessibility and Inclusive Design Lead

```
[Use the shared preamble.]

Act as Accessibility and Inclusive Design Lead.

Ensure Semester is accessible and usable by students, staff, guardians, and alumni with diverse d

Deliver:
- WCAG 2.2 AA implementation program and component-level requirements.
- Keyboard, screen reader, touch, zoom, voice control, reduced-motion, contrast, captions/transcr
- Accessibility acceptance criteria for every story and role.
- Automated/manual testing strategy and assistive technology matrix.
- Remediation process, conformance evidence, accessibility statement, and institutional review ar
- Inclusive content standards and plain-language guidance.
- Accessibility incident/ticket process.

Do not treat accessibility as a final QA step; embed it in design, code, tests, and launch gates.
```

DevSecOps and Cloud Infrastructure Lead

```
[Use the shared preamble.]

Act as DevSecOps and Cloud Infrastructure Lead.

Build secure, reproducible, observable cloud infrastructure for Semester:
- Infrastructure-as-code, environment isolation, least privilege, secrets, encryption, network co
- CI/CD security, artifact signing, SBOM, provenance, policy-as-code, configuration drift, vulner
- Tenant-safe deployment patterns, preview environments, staging, production, canary, rollbacks.
- Infrastructure monitoring, cost controls, access reviews, runbooks, and recovery testing.
- Compliance/security evidence generation.
```

```
Never allow manual production-only infrastructure drift without recorded change control.
```

Integration Engineering Lead

```
[Use the shared preamble.]

Act as Integration Engineering Lead.

Build Semester's integration control plane:
- Connector framework, credential vaulting, scopes, OAuth/OIDC/SAML/SCIM/LTI patterns, webhooks,
- SIS, LMS, calendar, email, payment, campus-service, career, and reporting connectors.
- Schema mapping, mapping-version control, source metadata, sync cursors, retries, dead letters,
- Contract tests, sandbox tests, tenant-specific activation, security reviews, and monitoring.
- Migration/parallel-run tools and connector documentation.

Every connector must augment a native Semester capability and be safe to disable without making t
```

Data Migration and Implementation Lead

```
[Use the shared preamble.]

Act as Data Migration and Implementation Lead.

Build the methodology and tooling to move institutions into Semester safely:
- Source inventory, extraction, mapping, cleansing, transformation, validation, rehearsal, cutove
- Migration workbooks for identity, academic records, courses, learning content, enrollments, fin
- Reconciliation reports, data quality thresholds, exception queues, and sign-off procedures.
- Parallel-run plans and institution acceptance criteria.
- Evidence retention and auditability.

Do not treat row-count equality as migration success; validate semantic correctness, relationship
```

Learning Science and Academic Integrity Lead

```
[Use the shared preamble.]

Act as Learning Science and Academic Integrity Lead.

Ensure Semester's native learning, study, assessment, tutoring, AI, and feedback systems improve

Deliver:
- Learning design principles, assessment model, feedback model, study methodology, accommodation
- AI tutoring boundaries, academic-integrity controls, citation/source behavior, instructor contr
- Requirements for assignments, assessments, rubrics, mastery, gradebook, accommodations, and pro
- Evaluation metrics that measure learning support rather than engagement alone.
- Faculty/tutor workflows and policy configuration requirements.
- Risks requiring academic governance and human review.

Do not design AI or study tools that encourage unauthorized work, false confidence, or hidden sur
```

Product Research Lead

```
[Use the shared preamble.]

Act as Product Research Lead.

Build the evidence system for Semester's product decisions:
- Research plan for students, families, faculty, advisors, registrar, finance, student affairs, I
- Interview guides, usability tests, diary studies, surveys, concept tests, and institutional wor
- Repository audit validation: determine which existing capabilities solve real needs and where a
- Research repository, insight taxonomy, confidence levels, and decision linkage.
- Pilot feedback and longitudinal outcome measurement.
- Ethical research consent and data-handling practices.

Do not treat stakeholder opinions or feature requests as validated product truth without evidence
```

Technical Program Manager

```
[Use the shared preamble.]

Act as Technical Program Manager for Semester.

Build and operate the integrated execution plan:
- Master roadmap across product, engineering, security, privacy, accessibility, AI, integrations,
- Dependency graph, critical path, milestone definitions, delivery risks, resourcing, and escalat
- Program rituals, status reporting, decision log, RAID register, release readiness, and executiv
- Cross-functional acceptance criteria and evidence tracking.
- Tenant launch plan coordination and post-launch follow-up.

Do not report activity as progress. Report verified outcomes, blockers, decisions, risks, and evi
```

Data Protection Officer / Privacy Operations Lead

```
[Use the shared preamble.]

Act as Privacy Operations Lead.

You are not qualified human legal counsel. Build privacy operations and identify questions for co

Deliver:
- Data inventory, processing activity register, classification, retention, deletion, export, cons
- Subject-rights request intake, identity verification, fulfillment, exception, and audit workflo
- Vendor/subprocessor review process.
- Privacy-by-design review for new product, analytics, AI, marketplace, integrations, and marketi
- Privacy incident handling coordination with security and counsel.
- User-facing privacy controls and transparency requirements.
- Counsel review queue.

Never provide definitive legal advice. Clearly label counsel-required decisions.
```

Technical Writer and Developer Experience Lead

```
[Use the shared preamble.]

Act as Technical Writer and Developer Experience Lead.

Create the documentation system required to build, operate, integrate, support, and sell Semester
```

```
- Architecture docs, ADRs, API references, event schemas, integration guides, runbooks, developer
- Product help, admin guides, implementation guides, support articles, security/trust documentati
- Documentation quality gates, ownership, versioning, review process, and discoverability.
- Examples and reference applications for institution and partner integrations.
- Release notes and change communications.

Documentation must describe actual behavior, be source-controlled, and be updated as part of ever
```

Final Operating Rule

Do not hand all of these prompts to Codex as disconnected parallel tasks without a shared source of truth. Create a `Semester Program Control Plane` in the repository first:

```
/docs
  /program
    PRODUCT_THESIS.md
    CAPABILITY_TRACEABILITY_MATRIX.md
    MASTER_ROADMAP.md
    ASSUMPTION_REGISTER.md
    RISK_REGISTER.md
    DECISION_LOG.md
    RELEASE_READINESS.md
  /architecture
    SYSTEM_CONTEXT.md
    DOMAIN_MODEL.md
    AUTHORIZATION.md
    DATA_GOVERNANCE.md
    AI_GOVERNANCE.md
    INTEGRATIONS.md
    OFFLINE_SYNC.md
  /operations
    INCIDENT_RESPONSE.md
    SUPPORT_RUNBOOK.md
    SLOS.md
    DISASTER_RECOVERY.md
    IMPLEMENTATION_PLAYBOOK.md
  /trust
    SECURITY.md
    PRIVACY.md
    ACCESSIBILITY.md
    LEGAL_REVIEW_QUEUE.md
    PUBLIC_CLAIMS_REGISTER.md
  /quality
    TEST_STRATEGY.md
    CRITICAL_JOURNEYS.md
    QUALITY_GATES.md
```

Every agent, engineer, designer, executive role, and Codex task must update the traceability matrix, decision log, test plan, and runbook implications as it works. That is how the work becomes a real company and operating platform—not a collection of impressive but disconnected deliverables.

⁂

Tenant-isolation audit: map RLS policies, tenant scoping tables, and verify cross-tenant boundaries against your database schema

Computer
Production deployment pipeline: build an automated CI/CD workflow with security gates, staging promotion, and automated rollback triggers
Computer
Show me the complete database schema for all modules
Give me the CI/CD pipeline configuration file for production
Write the incident response runbook and escalation matrix

I audited the connected **Semester production Supabase project** (`semester`, project ref `lzrqvlugnawcgywkhqlz`) and its repository-level deployment structure. The database is highly ambitious and broad, with RLS enabled on every enumerated public and private table; however, the current security advisory output shows meaningful **production-hardening work remains** before treating tenant isolation and direct database exposure as fully verified. [55] [56] [57]

Most importantly: I could inspect the schema and Supabase's security advisories, but the attempted catalog-level tenant-key/policy query was blocked because this connector classifies raw SQL as a state-changing action—even when the statement is read-only—and requires an explicit action confirmation. Therefore, this is an **evidence-based audit of the available schema catalog and advisories**, not a signed-off penetration test or final RLS certification.

Audit Findings

Current isolation posture

The database contains a large multi-tenant institutional model. The schema includes clear tenant-scoped constructs—especially `schools`, `institution_membership`, `role_grants`, `tenant_feature_policy`, `tenant_module_mode`, `tenant_plan`, `tenant_rollout`, `integration_connections`, governance structures, institution workflows, and school-bound academic, finance, dining, and community domains. The repository also contains dedicated SQL checks for tenancy, RLS coverage, integration RLS, grants, school membership, and role grants, which is the right general architecture direction. [56] [58]

However, "RLS enabled" is not the same as "tenant isolation proven."

Control area	Observed status	Audit conclusion	Required action
RLS enabled	All listed public and private tables show `rls_enabled: true`	Positive baseline	Preserve this as a non-negotiable invariant
Private tables without policies	Supabase reports many `private.*` tables have RLS enabled and no RLS policies	Usually defensible if no browser/API role has grants; this must be validated using grants and exposed-schema checks	Verify `anon`/`authenticated` have no schema usage or table privileges; retain deny-by-default
Public tables without policies	Supabase lists public tables such as invites, LTI state/tickets, payment events, support records, registration internal tables, and beta tables with RLS enabled but no policies	Potentially deliberate service-only tables, but cannot be accepted without grant verification	Revoke browser roles from service-only tables and move sensitive implementation tables to `private`

Control area	Observed status	Audit conclusion	Required action
Anonymous GraphQL visibility	32 public objects are visible to `anon` through GraphQL according to the security advisor	High-priority attack-surface reduction area	Revoke `anon` `SELECT` broadly; expose only intentionally public views/functions
Authenticated GraphQL visibility	287 objects are discoverable to signed-in users according to the advisor	High-priority catalog-disclosure and attack-surface concern—even if RLS filters rows	Revoke broad `authenticated` access; expose allowlisted views/RPCs only
SECURITY DEFINER RPC exposure	205 `SECURITY DEFINER` public functions are executable by authenticated users	Not automatically a vulnerability, but a large privileged attack surface and mandatory review item	Inventory, test, minimize, lock search paths, revoke default grants, route high-risk functions through controlled API/edge functions
Tenant key model	Schema descriptions demonstrate school-scoped/tenant-scoped patterns, but direct column/policy mapping has not been queried	Not yet certified	Run catalog evidence queries and negative cross-tenant test suite
Authorization model	`app_roles`, `app_capabilities`, `role_capabilities`, `role_grants`, and `private.has_capability()` are present	Strong conceptual foundation	Verify every tenant policy derives tenant from trusted membership/grant rather than user-selected input
Operational audit	Several append-only audit, history, ledger, and verification tables exist	Strong governance intent	Verify immutability triggers, privilege boundaries, and query access paths

Supabase specifically reports 61 “RLS enabled, no policy” findings. Some belong to the intentionally server-only `private` schema and may be acceptable if those tables are not exposed to `anon`, `authenticated`, PostgREST, GraphQL, or untrusted Edge Function contexts. But the same advisory also identifies public service-only tables such as `payment_events`, `registration_holds`, `registration_requests`, `support_tickets`, `support_ticket_messages`, LTI tables, and SCIM credential material with no policies. Those should be treated as **must-verify / likely lock-down** surfaces, not ignored linter noise.[57]

Risk prioritization

Priority	Finding	Why it matters	Required remediation
P0	Any proven cross-tenant read/write, policy bypass, or authority escalation	Could expose student records, grades, financial data, family relationships, or institutional operations	Freeze affected endpoint/module, activate kill switch, revoke grants, contain, investigate, notify under approved incident process
P0	Unreviewed SECURITY DEFINER function that can read/write cross-tenant or high-risk records	Privileged function misuse can bypass RLS	Review function code and grants; enforce `auth.uid()`, trusted tenant resolution, capability check, explicit schema-qualified references, fixed `search_path`
P1	Broad `anon` GraphQL object discoverability	Reduces reconnaissance barrier and can expose sensitive data models	Revoke `anon` table access except specifically approved public content
P1	Broad `authenticated` GraphQL object discoverability	Every signed-in user can enumerate sensitive data surfaces and test RPC/table contracts	Replace broad grants with API allowlists, views, RPCs, and narrowly scoped table privileges
P1	Service-only public tables	Public schema creates accidental exposure risk as the product evolves	Relocate to `private` or revoke all client roles and access only via secure server path
P1	Missing tenant-policy proof	A large schema makes undocumented implicit isolation dangerous	Generate policy matrix and enforce CI gate for tenant predicates/authorization functions

Priority	Finding	Why it matters	Required remediation
P2	Tenant-less platform tables	Some tables legitimately operate globally, but global tables can create indirect leakage	Explicitly classify each as global-public, global-internal, or tenant-scoped
P2	Direct client access to complex operational tables	Overly flexible browser database access makes support and audit harder	Prefer domain RPCs or server APIs for high-impact workflows
P2	Missing policy regression artifacts	Schema changes can create isolation regressions	Require dynamic negative tests against every tenant-bound table and RPC

Tenant-Isolation Audit Model

Required trust boundary

```
Browser / mobile client
        |
        | authenticated JWT only
        ▼
Semester API or tightly constrained RPC surface
        |
        ├── Validate authenticated identity
        ├── Resolve active tenant from trusted membership
        ├── Check relationship, role, capability, purpose, consent
        ├── Add correlation ID and audit context
        └── Call domain transaction
        ▼
Postgres domain schema
        |
        ├── RLS policy re-check
        ├── Tenant predicate re-check
        ├── Row ownership/relationship rule
        ├── Domain invariant
        ├── Audit event
        └── Outbox event
        ▼
Private worker / Edge Function / service role
        |
        ├── Explicit tenant context required
        ├── Narrow function-specific grants
        ├── No unrestricted browser invocation
        └── Immutable operation/audit trail
```

Isolation rules

Every data object must fall into exactly one of these classifications:

Classification	Example	Required enforcement
Global public	Marketing plans intended for anonymous display	Read-only public view; no direct table writes; no PII
Global internal	Role vocabulary, platform release evidence, internal pricing operations	No browser table grants; service/operator-only access
Tenant configuration	School policies, SSO policy, module mode, integration configuration	`tenant_id`/`school_id`; live tenant membership; explicit capability
Tenant operational	Registration sections, gradebook, school actions, dining, campus workflows	Tenant key plus domain relationship/role checks

Classification	Example	Required enforcement
Person-private	Notes, tasks, AI memories, profile details, productivity workspace	Account ID plus owner rule; tenant key when institution association exists
Relationship-scoped	Family, mentor, advisor, support shares	Source person, recipient person, tenant, active grant, scope, expiry
Course/section-scoped	Grade entries, assignments, course AI rules, roster artifacts	Tenant, course/section, term, enrollment/staff assignment
Financial/record-sensitive	Academic records, bills, payment plans, ledger entries	Tenant, subject, role/capability, append-only mutation path
Service-only	Webhook receipts, rate limits, raw sync state, credential verification material	`private` schema; no client grants; service role or narrowly privileged worker only
Audit/ledger	Audit events, history, immutable decision records	Insert through trusted function only; read through tenant-scoped audited views

Required tenant policy pattern

Do not use a user-supplied tenant ID as the authority. Resolve tenant access from durable membership/grants.

```
create or replace function private.current_tenant_member(
  expected_tenant uuid
)
returns boolean
language sql
stable
security definer
set search_path = pg_catalog, public, private
as $$
  select exists (
    select 1
    from public.institution_membership im
    where im.account_id = auth.uid()
      and im.school_id = expected_tenant
      and im.status = 'active'
      and im.revoked_at is null
  );
$$;

create policy "tenant members can read tenant rows"
on public.example_tenant_table
for select
to authenticated
using (
  private.current_tenant_member(school_id)
  and private.has_capability(
    auth.uid(),
    school_id,
    'example:read'
  )
);
```

For person-private data, the tenant predicate is **not sufficient**:

```
create policy "owner reads own private workspace"
on public.productivity_workspace
for select
```

```
to authenticated
using (
  account_id = auth.uid()
  and private.current_tenant_member(school_id)
);
```

For relationship-scoped data:

```
create policy "active recipient reads granted share only"
on public.family_shared_items
for select
to authenticated
using (
  exists (
    select 1
    from public.family_grants fg
    where fg.student_account_id = family_shared_items.student_account_id
      and fg.supporter_account_id = auth.uid()
      and fg.school_id = family_shared_items.school_id
      and fg.status = 'active'
      and fg.expires_at > now()
      and private.scope_permits(
        fg.id,
        family_shared_items.category,
        family_shared_items.resource_id
      )
  )
);
```

Policy review checklist

For every table, view, materialized view, RPC, Edge Function, storage bucket, queue, cache, and analytics dataset, answer:

Question	Required answer
Does this object contain tenant data?	Yes/no with a documented classification
What field establishes tenant ownership?	`school_id`, `tenant_id`, derived through parent relation, or explicitly global
Can it be called/read by `anon`?	Default: no
Can it be called/read by `authenticated`?	Only with documented, tested policy
Does every RLS policy contain a tenant boundary or trusted ownership boundary?	Yes, unless explicitly global
Are writes restricted to safe domain functions?	Yes for records, grades, billing, approvals, and audit data
Are service-role pathways separated from browser pathways?	Yes
Is a SECURITY DEFINER function involved?	If yes: fixed search path, explicit authorization, code review, negative tests
Is there a cross-tenant negative test?	Required
Is there a tenant-deletion/offboarding behavior?	Required
Is the object exposed in GraphQL/PostgREST?	Only intentionally
Does it log sensitive access or mutation?	Required for high-risk domains

Concrete audit query package

Run these in a **non-production branch or staging environment first**, then use the same query package in production with read-only credentials. Do not run schema-changing remediation without change review.

1. RLS coverage and tenant-key inventory

```
with relations as (
  select
    n.nspname as schema_name,
    c.relname as relation_name,
    c.relkind,
    c.relrowsecurity as rls_enabled,
    c.relforcerowsecurity as force_rls,
    exists (
      select 1
      from pg_policies p
      where p.schemaname = n.nspname
        and p.tablename = c.relname
    ) as has_rls_policy,
    (
      select array_agg(col.column_name order by col.column_name)
      from information_schema.columns col
      where col.table_schema = n.nspname
        and col.table_name = c.relname
        and col.column_name in (
          'tenant_id',
          'school_id',
          'organization_id',
          'institution_id'
        )
    ) as tenant_columns
  from pg_class c
  join pg_namespace n on n.oid = c.relnamespace
  where c.relkind in ('r', 'v', 'm')
    and n.nspname in ('public', 'private')
)
select
  schema_name,
  relation_name,
  relkind,
  rls_enabled,
  force_rls,
  has_rls_policy,
  coalesce(tenant_columns::text, '{}') as tenant_columns,
  case
    when schema_name = 'private' then 'server-only: verify grants'
    when tenant_columns is not null then 'tenant-key present'
    else 'no direct tenant key: classify as global/derived/person-private'
  end as review_status
from relations
order by schema_name, relation_name;
```

2. Full RLS policy map

```
select
  schemaname,
  tablename,
  policyname,
  permissive,
```

```
  roles,
  cmd,
  qual as using_expression,
  with_check as with_check_expression
from pg_policies
where schemaname in ('public', 'private')
order by schemaname, tablename, cmd, policyname;
```

3. Tables exposed to browser roles

```
select
  table_schema,
  table_name,
  grantee,
  privilege_type
from information_schema.role_table_grants
where table_schema in ('public', 'private')
  and grantee in ('anon', 'authenticated')
order by table_schema, table_name, grantee, privilege_type;
```

4. SECURITY DEFINER inventory

```
select
  n.nspname as schema_name,
  p.proname as function_name,
  pg_get_function_identity_arguments(p.oid) as arguments,
  p.prosecdef as security_definer,
  p.proconfig as function_config,
  has_function_privilege('anon', p.oid, 'execute') as anon_can_execute,
  has_function_privilege('authenticated', p.oid, 'execute') as authenticated_can_execute,
  pg_get_functiondef(p.oid) as definition
from pg_proc p
join pg_namespace n on n.oid = p.pronamespace
where n.nspname in ('public', 'private')
  and p.prosecdef
order by schema_name, function_name;
```

5. Unsafe default function grants

```
select
  n.nspname as schema_name,
  p.proname as function_name,
  has_function_privilege('public', p.oid, 'execute') as public_execute,
  has_function_privilege('anon', p.oid, 'execute') as anon_execute,
  has_function_privilege('authenticated', p.oid, 'execute') as authenticated_execute
from pg_proc p
join pg_namespace n on n.oid = p.pronamespace
where n.nspname in ('public', 'private')
order by schema_name, function_name;
```

6. Missing RLS policies outside private schema

```
select
  n.nspname as schema_name,
  c.relname as table_name
from pg_class c
```

```
join pg_namespace n on n.oid = c.relnamespace
where c.relkind = 'r'
  and n.nspname = 'public'
  and c.relrowsecurity
  and not exists (
    select 1
    from pg_policies p
    where p.schemaname = n.nspname
      and p.tablename = c.relname
  )
order by table_name;
```

7. Cross-tenant policy test harness pattern

```
begin;

-- Substitute fixtures created only in an isolated test database/branch.
select set_config(
  'request.jwt.claim.sub',
  '11111111-1111-1111-1111-111111111111',
  true
);

select set_config(
  'request.jwt.claim.role',
  'authenticated',
  true
);

-- The test actor belongs only to tenant A.
-- This count must equal zero for every tenant-bound table.
select count(*) as forbidden_rows_visible
from public.registration_enrollments
where school_id = '22222222-2222-2222-2222-222222222222';

rollback;
```

Automate this pattern for every RLS-protected table and for every relevant RPC. Test:

- Tenant A user cannot read Tenant B row.
- Tenant A user cannot infer Tenant B row existence through counts, errors, response times, or unique constraints.
- Tenant A user cannot insert a row with Tenant B `school_id`.
- Tenant A user cannot update/delete Tenant B row.
- Tenant A administrator cannot access Tenant B data.
- Staff relationship boundaries work inside the same tenant.
- Family/guardian access cannot exceed explicit scope, expiry, or resource set.
- Revoked memberships instantly lose access.
- Offboarded tenant data is inaccessible.
- SECURITY DEFINER functions reject forged tenant arguments.
- Service-only tables cannot be queried through REST or GraphQL.

Complete Database Schema Map

The actual deployed schema is extensive—well over 250 tables across `public` and `private`. The following is a **complete module-level schema catalog** based on the live table inventory. It groups the deployed tables into architectural modules rather than dumping hundreds of raw DDL definitions. Generate the TypeScript database types and `pg_dump --schema-only` artifact in CI as the canonical full-column schema record.[56]

Identity, access, tenancy, and policy

Tables
profiles, schools, institution_membership, school_membership_requests, role_grants, app_roles, app_capabilities, role_capabilities, role_grant_audit_event, institution_identity_provider, tenant_sso_policy, tenant_sso_policy_history, app_admins, access_gate, invites, feature_kill_switch, feature_cohort_members, tenant_feature_policy, tenant_policy_audit_event, tenant_plan, tenant_plan_history, tenant_rollout, tenant_rollout_evidence, tenant_rollout_history, tenant_module_mode, tenant_module_mode_history, module_mode_request, module_mode_approval, break_glass_grant, audit_event, data_classification_rules, legal_holds, human_overrides

Identity provisioning and SSO/SCIM

Tables
institution_identity_provider, institution_membership, scim_credential, scim_external_identity, scim_group_mapping, provisioning_audit_event, lti_platform, lti_nonce, lti_identity, lti_link_ticket, lti_line_item

Student productivity and personal OS

Tables
state, usage, notes, tasks, appointments, sittings, calendar_feeds, blocks, push_devices, push_queue, productivity_workspace, student_context, term_plan_courses, weekly_checkins, onboarding_progress, operator_preference, ai_memories, access_log

Academic planning, registrar, records, and degree path

Tables
courses, enrollments, catalog_sections, registration_terms, registration_sections, registration_enrollments, registration_holds, registration_completions, registration_overrides, registration_requests, registration_time_tickets, registration_windows, registration_audit_event, seat_watches, graduation_scenarios, articulation_rules, transfer_evaluations, academic_record_subjects, academic_record_changes, academic_record_entries, student_account_settings

Learning, gradebook, study, and evidence

Tables
approved_source, evidence_reference, concept_evidence, mistake_evidence, skill_claim, skill_claim_evidence, skill_records, capture_asset, capture_segment, capture_artifact, course_ai_rules, course_guidance, study_packs, course_reviews, course_review_authors, study_match_optins, gradebook_operations, gradebook_schemes, gradebook_items, grade_entries, regrade_requests, regrade_resolutions, grade_passbacks, private.approved_source_content

AI governance and gateway

Tables
ai_policy, consent_record, tenant_feature_policy, data_classification_rules, human_overrides, private.ai_usage_month, private.ai_usage_reservation, private.gateway_review, private.gateway_audit, private.gateway_intelligence_audit, private.gateway_intelligence_action, private.gateway_rate_limit, private.gateway_health_probe

Family, guardian, advising, and support sharing

Tables
family_grants, family_invites, family_shared_items, family_access_events, guardian_links, guardian_link_restrictions, guardian_link_history, advisor_shares, advisor_share_events, support_access_grant, support_access_event, support_shares, support_share_events, accommodation_passports, accommodation_shares, accommodation_access_events

Campus life, dining, community, and organizations

Tables
organizations, organization_members, connections, groups, group_members, group_tasks, messages, message_reactions, reports, communities, community_members, community_posts, community_restrictions, community_mutes, community_cases, community_reports, community_case_events, community_detector_rules, community_signals, community_decisions, community_venues, community_sessions, community_session_participants, community_retention_runs, community_programs, community_aliases, community_identity_grants, community_volunteers, community_calibration_items, community_volunteer_tasks, community_volunteer_votes, community_volunteer_events, community_escalation_policies, community_escalations, community_escalation_deliveries, community_escalation_agreement_events, community_safety_entries, community_media, community_media_blocklist, community_media_deletions

Dining and campus card operations

Tables
dining_partner_connections, dining_locations, dining_hours, dining_menu_items, dining_plans, dining_orders, dining_order_events, dining_ledger, dining_pool_donations, dining_pool_claims

Career, talent, alumni, and mentoring

Tables
contact_channels, talent_profiles, talent_profile_views, opportunities, peer_mentor_assignments, peer_mentor_offers, mentor_requests, alumni_mentor_offers, course_demand_snapshots, outcome_aggregates, demand_consents

Institution actions, support, and operational help

Tables
institution_actions, institution_action_offices, institution_action_audiences, institution_action_progress, help_destinations, help_requests, help_request_events, support_tickets, support_ticket_messages, support_notification_outbox, feedback, forms, form_responses, form_publications

Integrations, source provenance, migration, and data quality

Tables
integration_connections, integration_scopes, integration_mappings, integration_mapping_versions, integration_sync_runs, integration_sync_errors, integration_webhook_events, integration_dead_letter_events, integration_reconciliation_runs, integration_reconciliation_discrepancies, integration_schema_fingerprints, integration_schema_drift_events, integration_duplicate_candidates, integration_duplicate_resolutions, integration_source_owners, integration_retention_runs, provider_registry, provider_evidence, source_records, source_snapshots, source_freshness_events, canonical_entity_references, migration_projects, migration_field_maps, migration_runs, migration_approvals, private.integration_simulation_runs, private.roster_import_config, private.roster_import_batch, private.roster_staged_row, private.roster_current, private.roster_snapshot

Governance, compliance, trust, and procurement

Tables
governance_policy_nodes, governance_steward_assignments, governance_decisions, governance_config_requests, governance_incident_notices, school_config_versions, workflow_versions, compliance_frameworks, compliance_controls, control_evidence, claims_register, trust_artifacts, trust_artifact_versions, trust_room_requests, trust_room_grants, trust_room_grant_items, trust_room_access_log, platform_release_evidence, council_seat_holder, console_duty, approval_request, approval_decision, console_action_record, private.console_audit_event, private.console_audit_key, private.console_audit_manifest, private.console_audit_verification

Billing, commerce, customer success, and SaaS operations

Tables
commercial_products, commercial_plans, commercial_prices, entitlement_definitions, plan_entitlements, billing_accounts, billing_account_tenants, quotes, quote_lines, contracts, subscriptions, subscription_entitlements, invoices, invoice_lines, payment_events, credits_refunds, dunning_cases, dunning_actions, cancellation_requests, checkout_sessions, customer, customer_commitment, customer_contract, implementation_projects, implementation_milestones, success_plans, qbrs, renewal_opportunities, account_health_snapshots

Growth, pilots, beta, marketing, and commercial governance

Tables
gtm_prospects, gtm_consent, gtm_suppression, gtm_campaigns, gtm_campaign_links, gtm_campaign_reviews, gtm_communication_events, gtm_conversion_events, gtm_report_access, gtm_sponsor_policy, gtm_sponsor_placements, gtm_accounts, gtm_stakeholders, gtm_decision_log, gtm_pilots, gtm_pilot_metrics, gtm_pilot_outcomes, beta_programs, beta_cohorts, beta_invitations, beta_memberships, beta_feature_flags, beta_known_issues, beta_feedback, beta_exit_requests, referral_codes, referrals, site_leads, private.site_lead_hits

Lifecycle, offboarding, retention, events, and internal platform mechanics

Tables
data_requests, data_subject_request, school_offboarding, school_offboarding_undo, private.account_ages, private.direct_rate_limit, private.domain_outbox_events, private.domain_event_receipts, private.ledger_chain_start, private.ledger_chain, private.ledger_chain_key, private.ledger_chain_manifest, private.ledger_chain_verification

Schema documentation deliverables

Add these generated artifacts to the build pipeline:

```
supabase gen types typescript \
  --project-id "$SUPABASE_PROJECT_REF" \
  --schema public,private \
  > app/src/lib/database.generated.ts
```

```
pg_dump "$DATABASE_URL" \
  --schema-only \
  --no-owner \
  --no-privileges \
  --schema=public \
  --schema=private \
  > artifacts/schema.sql
```

Do not publish raw schema artifacts containing internal function definitions or identifiers to public documentation. Keep them in a private engineering evidence store.

Production CI/CD Pipeline

The repository currently has Vercel deployment configuration and substantial test/security/operations materials. The production pipeline should turn those assets into **enforced gates**, with staging promotion and release rollback rules—not merely a deployment on merge. [58] [56]

Deployment flow

```
Feature branch
   │
   ├── Format · lint · typecheck · unit test
   ├── Secret scan · dependency scan · SAST
   ├── Build · bundle/performance budgets
   ├── RLS / migration / policy tests
   ├── Accessibility checks
   └── Preview deployment
        │
        ▼
Pull request approved
        │
        ├── Contract / integration tests
        ├── E2E tests on preview
        ├── Tenant-isolation negative suite
        ├── Migration dry run on Supabase branch
        └── Release evidence check
             │
             ▼
main branch
   │
   ├── Build immutable release artifact
   ├── Deploy to staging
   ├── Apply reviewed staging migrations
   ├── Smoke + E2E + DAST + SLO synthetic tests
   └── Manual production approval
        │
        ▼
Production canary
   │
   ├── 5% traffic / internal + designated tenant cohort
```

```
   ├── Observe errors, latency, auth, isolation, sync and business KPIs
   ├── Automatic abort thresholds
   └── Manual promote gate
        │
        ▼
Production full rollout
   │
   ├── Post-deploy migration verification
   ├── Feature flag staged rollout
   ├── Release record + audit event
   └── Monitor / rollback readiness
```

Required GitHub Actions file

Create `.github/workflows/production.yml` with the following content. This is a production template; replace command names and secret names only after matching them to your actual scripts and secret manager. It intentionally never prints secret values.

```yaml
name: Production Release

on:
  push:
    branches: [main]
  workflow_dispatch:
    inputs:
      release_ref:
        description: "Immutable commit SHA or tag to deploy"
        required: false
        type: string
      force_canary:
        description: "Deploy through canary even if no application files changed"
        required: false
        default: false
        type: boolean

permissions:
  contents: read
  id-token: write
  pull-requests: read
  security-events: write
  checks: write

concurrency:
  group: semester-production-release
  cancel-in-progress: false

env:
  NODE_VERSION: "22"
  APP_DIR: app
  SUPABASE_PROJECT_REF: ${{ secrets.SUPABASE_PRODUCTION_PROJECT_REF }}
  VERCEL_ORG_ID: ${{ secrets.VERCEL_ORG_ID }}
  VERCEL_PROJECT_ID: ${{ secrets.VERCEL_PROJECT_ID }}

jobs:
  metadata:
    name: Release metadata
    runs-on: ubuntu-latest
    outputs:
      sha: ${{ steps.release.outputs.sha }}
      version: ${{ steps.release.outputs.version }}
    steps:
```

```
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - id: release
        shell: bash
        run: |
          set -euo pipefail
          SHA="${{ inputs.release_ref || github.sha }}"
          git cat-file -e "${SHA}^{commit}"
          echo "sha=${SHA}" >> "$GITHUB_OUTPUT"
          echo "version=$(git describe --always --dirty=false "${SHA}")" >> "$GITHUB_OUTPUT"

  validate:
    name: Quality and security gates
    needs: metadata
    runs-on: ubuntu-latest
    timeout-minutes: 30
    defaults:
      run:
        working-directory: app
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ needs.metadata.outputs.sha }}

      - uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: npm
          cache-dependency-path: app/package-lock.json

      - name: Install locked dependencies
        run: npm ci

      - name: Verify lockfile integrity
        run: npm audit signatures || true

      - name: Typecheck
        run: npm run typecheck

      - name: Lint
        run: npm run lint

      - name: Unit and component tests
        run: npm test -- --run

      - name: Production build
        run: npm run build

      - name: Performance budgets
        run: npm run perf:check

      - name: Secret scan
        working-directory: .
        run: |
          docker run --rm \
            -v "$PWD:/repo" \
            zricethezav/gitleaks:latest \
            detect --source=/repo --no-git --redact

      - name: Dependency vulnerability scan
        run: npm audit --omit=dev --audit-level=high
```

```
      - name: Upload production artifact
        uses: actions/upload-artifact@v4
        with:
          name: semester-web-${{ needs.metadata.outputs.version }}
          path: app/dist
          if-no-files-found: error
          retention-days: 14

  database-policy:
    name: Database migration and tenant-isolation gates
    needs: metadata
    runs-on: ubuntu-latest
    timeout-minutes: 35
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ needs.metadata.outputs.sha }}

      - name: Install Supabase CLI
        uses: supabase/setup-cli@v1
        with:
          version: latest

      - name: Link staging project
        run: |
          supabase link \
            --project-ref "${{ secrets.SUPABASE_STAGING_PROJECT_REF }}" \
            --password "${{ secrets.SUPABASE_STAGING_DB_PASSWORD }}"

      - name: Migration integrity
        run: supabase db diff --linked --schema public,private

      - name: Apply migrations to staging only
        run: supabase db push --linked --include-all

      - name: Run repository database checks
        run: |
          chmod +x supabase/check.sh
          ./supabase/check.sh

      - name: Run tenant-isolation regression suite
        run: |
          chmod +x ops/test-tenant-isolation.sh
          ./ops/test-tenant-isolation.sh \
            --project-ref "${{ secrets.SUPABASE_STAGING_PROJECT_REF }}"

      - name: Run RLS and grants audit
        run: |
          psql "${{ secrets.SUPABASE_STAGING_DATABASE_URL }}" \
            -v ON_ERROR_STOP=1 \
            -f ops/sql/tenant-isolation-audit.sql

      - name: Upload database evidence
        uses: actions/upload-artifact@v4
        with:
          name: database-evidence-${{ needs.metadata.outputs.version }}
          path: |
            artifacts/tenant-isolation-audit.json
            artifacts/rls-policy-map.csv
            artifacts/schema.sql
          if-no-files-found: error
          retention-days: 90

  deploy-staging:
```

```
    name: Deploy and verify staging
    needs: [metadata, validate, database-policy]
    runs-on: ubuntu-latest
    environment:
      name: staging
    outputs:
      deployment_url: ${{ steps.deploy.outputs.url }}
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ needs.metadata.outputs.sha }}

      - uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: npm
          cache-dependency-path: app/package-lock.json

      - name: Install Vercel CLI
        run: npm install --global vercel@latest

      - name: Pull staging configuration
        working-directory: app
        run: vercel pull --yes --environment=preview --token="${{ secrets.VERCEL_TOKEN }}"

      - name: Build deployable artifact
        working-directory: app
        run: vercel build --token="${{ secrets.VERCEL_TOKEN }}"

      - id: deploy
        name: Deploy staging artifact
        working-directory: app
        run: |
          URL="$(vercel deploy --prebuilt --yes --token="${{ secrets.VERCEL_TOKEN }}")"
          echo "url=${URL}" >> "$GITHUB_OUTPUT"

      - name: Staging smoke tests
        run: |
          npx wait-on "${{ steps.deploy.outputs.url }}"
          ./ops/smoke-test.sh "${{ steps.deploy.outputs.url }}"

      - name: Staging end-to-end tests
        run: |
          BASE_URL="${{ steps.deploy.outputs.url }}" \
          npm --prefix app run test:e2e

      - name: Staging accessibility tests
        run: |
          BASE_URL="${{ steps.deploy.outputs.url }}" \
          npm --prefix app run test:a11y

      - name: Staging security tests
        run: |
          BASE_URL="${{ steps.deploy.outputs.url }}" \
          ./ops/dast-baseline.sh

  production-approval:
    name: Production approval
    needs: [metadata, deploy-staging]
    runs-on: ubuntu-latest
    environment:
      name: production
    steps:
      - name: Confirm staged release passed
```

```
        run: |
          test -n "${{ needs.deploy-staging.outputs.deployment_url }}"
          echo "Staging acceptance complete."

  deploy-production:
    name: Deploy production canary
    needs: [metadata, validate, database-policy, deploy-staging, production-approval]
    runs-on: ubuntu-latest
    environment:
      name: production
    outputs:
      production_url: ${{ steps.deploy.outputs.url }}
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ needs.metadata.outputs.sha }}

      - uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: npm
          cache-dependency-path: app/package-lock.json

      - name: Install Vercel CLI
        run: npm install --global vercel@latest

      - name: Apply production migrations
        run: |
          supabase link \
            --project-ref "${{ secrets.SUPABASE_PRODUCTION_PROJECT_REF }}" \
            --password "${{ secrets.SUPABASE_PRODUCTION_DB_PASSWORD }}"
          supabase db push --linked --include-all

      - name: Pull production configuration
        working-directory: app
        run: vercel pull --yes --environment=production --token="${{ secrets.VERCEL_TOKEN }}"

      - name: Build immutable production artifact
        working-directory: app
        run: vercel build --prod --token="${{ secrets.VERCEL_TOKEN }}"

      - id: deploy
        name: Deploy production artifact
        working-directory: app
        run: |
          URL="$(vercel deploy --prebuilt --prod --yes --token="${{ secrets.VERCEL_TOKEN }}")"
          echo "url=${URL}" >> "$GITHUB_OUTPUT"

      - name: Record release metadata
        run: |
          ./ops/record-release.sh \
            --sha "${{ needs.metadata.outputs.sha }}" \
            --version "${{ needs.metadata.outputs.version }}" \
            --url "${{ steps.deploy.outputs.url }}"

  canary-observation:
    name: Canary health gate
    needs: [metadata, deploy-production]
    runs-on: ubuntu-latest
    timeout-minutes: 25
    environment:
      name: production
    steps:
      - uses: actions/checkout@v4
```

```
        with:
          ref: ${{ needs.metadata.outputs.sha }}

      - name: Verify production smoke paths
        run: |
          ./ops/smoke-test.sh "${{ needs.deploy-production.outputs.production_url }}"

      - name: Evaluate automated rollback thresholds
        env:
          RELEASE_SHA: ${{ needs.metadata.outputs.sha }}
          PRODUCTION_URL: ${{ needs.deploy-production.outputs.production_url }}
          OBSERVABILITY_TOKEN: ${{ secrets.OBSERVABILITY_TOKEN }}
        run: |
          ./ops/evaluate-canary.sh \
            --release "$RELEASE_SHA" \
            --url "$PRODUCTION_URL" \
            --window-minutes 15 \
            --max-error-rate 0.01 \
            --max-auth-failure-rate 0.02 \
            --max-p95-ms 2500 \
            --max-critical-journey-failures 0 \
            --max-security-policy-violations 0

  promote-production:
    name: Promote release
    needs: [metadata, deploy-production, canary-observation]
    runs-on: ubuntu-latest
    environment:
      name: production
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ needs.metadata.outputs.sha }}

      - name: Promote deployment
        run: |
          ./ops/promote-release.sh \
            --sha "${{ needs.metadata.outputs.sha }}" \
            --url "${{ needs.deploy-production.outputs.production_url }}"

      - name: Post-deployment validation
        run: |
          ./ops/smoke-test.sh "${{ needs.deploy-production.outputs.production_url }}"
          ./ops/verify-release-evidence.sh \
            --sha "${{ needs.metadata.outputs.sha }}"

  rollback-on-failure:
    name: Automatic rollback
    if: ${{ failure() && needs.deploy-production.result == 'success' }}
    needs: [metadata, deploy-production, canary-observation, promote-production]
    runs-on: ubuntu-latest
    environment:
      name: production
    steps:
      - uses: actions/checkout@v4

      - name: Activate safety kill switches
        env:
          SUPABASE_SERVICE_ROLE_KEY: ${{ secrets.SUPABASE_SERVICE_ROLE_KEY }}
        run: |
          ./ops/engage-release-kill-switches.sh \
            --release "${{ needs.metadata.outputs.sha }}"

      - name: Roll back to last known good deployment
```

```
        env:
          VERCEL_TOKEN: ${{ secrets.VERCEL_TOKEN }}
        run: |
          ./ops/rollback-last-known-good.sh

      - name: Create release incident record
        run: |
          ./ops/open-release-incident.sh \
            --severity SEV-1 \
            --release "${{ needs.metadata.outputs.sha }}" \
            --reason "Automated production gate failure or canary regression"
```

Pipeline configuration requirements

The workflow requires these protected secrets and configuration values:

Name	Purpose	Handling
`VERCEL_TOKEN`	Deployment authorization	Production environment secret; rotate; least privilege
`VERCEL_ORG_ID`	Vercel organization selection	Repository/environment secret
`VERCEL_PROJECT_ID`	Vercel project selection	Repository/environment secret
`SUPABASE_PRODUCTION_PROJECT_REF`	Production target reference	Production environment secret
`SUPABASE_STAGING_PROJECT_REF`	Staging target reference	Staging environment secret
`SUPABASE_PRODUCTION_DB_PASSWORD`	Migration tool connection	Production environment secret; never expose to application runtime
`SUPABASE_STAGING_DB_PASSWORD`	Staging migration connection	Staging environment secret
`SUPABASE_PRODUCTION_DATABASE_URL`	Read-only evidence/audit scripts where needed	Production environment secret; prefer a dedicated least-privilege auditor credential
`OBSERVABILITY_TOKEN`	Canary metrics query	Production environment secret
`SUPABASE_SERVICE_ROLE_KEY`	Only for narrowly scoped operational scripts	Avoid if possible; never client-side; rotate and log use

Automatic rollback triggers

Do not roll back schema migrations blindly. Application release rollback and database rollback are different operations.

Trigger	Automated action	Human action
Build/type/security gate fails	Do not deploy	Fix and reopen release
Migration dry run or RLS suite fails	Do not promote	Investigate schema/policy before retry
Canary 5xx rate above 1% for 5 minutes	Stop promotion; roll back application; engage related feature flags	Incident commander validates effect
Authentication failure rate above 2%	Stop promotion; roll back application	Identity owner investigates configuration/session impact
Critical student journey fails	Stop promotion; roll back application or disable feature	Product/engineering decide customer communication
Cross-tenant test fails	Stop release immediately; disable affected domain/API	Treat as potential security incident

Trigger	Automated action	Human action
Unauthorized data access signal	Activate kill switch; revoke exposure; roll back affected service	Security incident workflow
Migration causes data integrity failure	Stop release; prevent further writes if necessary; invoke recovery plan	DBA/data owner performs approved corrective migration
Connector reconciliation discrepancy exceeds threshold	Disable auto-write direction; preserve read-only data with freshness notice	Integration owner resolves mapping/source issue
AI policy violation	Disable AI feature/model/tool path using flag	AI governance and security investigate

Database migration safety rules

1. Every migration must be additive first: add columns/tables/indexes, backfill, dual-read/write, validate, then later remove legacy fields.
2. Never combine irreversible data destruction with the same deployment that introduces a feature.
3. Every migration must have:
    - Purpose and ADR link.
    - Forward migration.
    - Data validation query.
    - Application compatibility window.
    - Roll-forward remediation plan.
    - Backup/restore impact.
    - Tenant-isolation impact statement.
4. Use a production approval gate for changes to RLS policies, roles, grants, functions, triggers, audit tables, records, grades, finance, retention, or tenant boundaries.
5. The release pipeline must fail if generated schema diffs include:
    - `DISABLE ROW LEVEL SECURITY`.
    - `GRANT` to `anon` or `authenticated` without allowlist.
    - New `SECURITY DEFINER` function without review metadata.
    - Dropped policy, tenant key, index, audit trigger, or constraint.
    - Direct table writes to append-only ledger tables.
    - Destructive DDL without an approved change record.

Incident Response Runbook

Scope

This runbook covers security incidents, availability incidents, data-integrity incidents, integration failures, deployment regressions, AI safety events, privacy events, and institutional workflow failures.

It must be validated through tabletop exercises and production-safe drills. It is an operational plan, not legal advice; qualified legal counsel must determine applicable notification duties, regulatory conclusions, and external legal communications.

Incident lifecycle

```
Detect
  │
  ▼
Triage and classify
  │
  ▼
Declare severity and assign incident commander
  │
  ├── Contain
  ├── Preserve evidence
  ├── Protect users and data
  └── Communicate internally
  │
  ▼
Investigate and remediate
  │
  ├── Roll back / kill switch / revoke access
  ├── Restore service / integrity
  ├── Assess data and customer impact
  └── Coordinate legal/privacy review
  │
  ▼
Recover and verify
  │
  ▼
Customer communications
  │
  ▼
Post-incident review and corrective actions
```

Severity criteria

Severity	Definition	Examples	Initial response
SEV-0	Confirmed or strongly suspected severe security/privacy/integrity event	Cross-tenant disclosure, credential compromise, unauthorized record change, confirmed major breach	Immediate incident declaration
SEV-1	Critical service outage or high-impact functional failure	Login outage, assignment submission failure, registration failure, production deployment regression	Page on-call immediately
SEV-2	Major feature degradation with workaround or limited tenant impact	One tenant's connector fails, faculty grade workflow degraded, major performance incident	Triage within 30 minutes
SEV-3	Limited defect or non-critical operational issue	Isolated UI issue, non-critical report delay, documented workaround exists	Triage same business day
SEV-4	Low-impact issue or enhancement	Cosmetic issue, minor copy defect, non-urgent request	Normal backlog triage

First 15 minutes

Minute	Incident commander actions	Technical lead actions	Security/privacy actions	Communications actions
0–5	Declare incident channel, assign incident ID, name roles	Confirm symptoms and scope	Decide whether security/privacy indicators exist	Hold external communications pending facts
5–10	Set severity; open timeline	Stop harmful deployment, disable flag, preserve logs	Revoke suspect credentials/tokens if needed	Draft initial internal status

Minute	Incident commander actions	Technical lead actions	Security/privacy actions	Communications actions
10–15	Establish next update time	Identify affected tenants, data, and services	Begin evidence preservation; contact legal/privacy lead for SEV-0	Post status update for confirmed user impact

Standard response roles

Role	Responsibilities	Backup
Incident commander	Owns command, severity, decisions, timeline, cadence, closure	COO or designated operations lead
Technical lead	Diagnoses architecture/code/database/infrastructure; executes technical mitigation	CTO or senior platform engineer
Security lead	Leads containment, evidence, credential/access actions, investigation	CISO delegate
Privacy/legal liaison	Coordinates impact assessment, counsel, notification review, documentation	Privacy lead / outside counsel contact
Customer communications lead	Status page, customer updates, support scripts, stakeholder communications	CSM/support lead
Scribe	Maintains time-stamped action log and decisions	Product operations / support operations
Tenant liaison	Coordinates with affected institution(s)	Customer success lead
Data integrity owner	Validates records, grades, billing, ledgers, migration state	Data/platform lead

Immediate containment playbooks

Cross-tenant data exposure

1. Declare SEV-0.
2. Immediately disable affected API/RPC/feature flag and revoke related `anon`/`authenticated` grants if needed.
3. Preserve logs, request IDs, database audit events, query traces, deployment metadata, and authorization context.
4. Do not delete evidence or mutate affected data except through documented containment commands.
5. Determine:
   - Affected tenant(s).
   - Affected data categories.
   - Number of subjects/records.
   - Exact exposure period.
   - Whether data was viewed, exported, modified, or only potentially reachable.
6. Rotate related credentials, revoke sessions, and disable compromised integrations as appropriate.
7. Engage security lead, privacy/legal liaison, executive sponsor, and qualified human counsel.
8. Implement a least-privilege corrective change, then run cross-tenant regression suite before re-enabling.
9. Prepare customer/regulatory notifications only after facts and counsel-approved obligations are determined.
10. Complete root cause analysis and control improvements within the defined corrective-action SLA.

Production deployment regression

1. Declare SEV-1 if critical journey affected.
2. Halt rollout and stop promotion.
3. Engage feature kill switch for the affected module before broad platform rollback if it contains the blast radius.
4. Roll back to last-known-good application deployment.
5. Do **not** auto-rollback a database migration unless an approved migration recovery plan exists.
6. Verify:
    - Login.
    - Student Today.
    - Course/assignment access.
    - Submission confirmation.
    - Grade view.
    - Registration and billing if affected.
    - Tenant isolation smoke checks.
7. Update status page if users are materially affected.
8. Open follow-up with root cause, detection gap, and test gap.

Data integrity incident

1. Declare SEV-0 or SEV-1 depending on records affected.
2. Freeze affected write pathways using kill switches or permission policy.
3. Preserve current state, logs, audit ledger, migration/deployment identifiers, and inbound integration payload references.
4. Identify authoritative source and reconciliation method.
5. Use append-only corrective entries for academic/financial ledgers; never silently overwrite history.
6. Require independent reviewer approval before restoring official record state.
7. Reconcile with institutional source systems and document all repairs.
8. Obtain tenant sign-off for affected high-stakes records before closure.

Integration failure or bad sync

1. Pause auto-write direction immediately.
2. Preserve last good cursor and reconciliation state.
3. Display source freshness and degraded-mode notice to users; do not claim current data is authoritative.
4. Quarantine malformed/mismatched events to dead-letter processing.
5. Compare mapping version, provider schema fingerprint, credentials, and source counts.
6. Reprocess only after idempotency and reconciliation checks pass.
7. Require two-person approval before bulk replay that can mutate records.

AI safety or data-boundary event

1. Disable the affected model/tool/retrieval route with a feature flag.
2. Preserve policy decision logs, model/version, source IDs, tool invocation logs, and redacted prompt metadata.
3. Determine whether it is:
   - Hallucinated institutional/academic information.
   - Unauthorized retrieval.
   - Prompt injection.
   - Sensitive data leakage.
   - Unsafe decision/action proposal.
   - Policy bypass.
4. Escalate to security and AI governance leads; include legal/privacy liaison if personal or regulated data is involved.
5. Issue user correction where factual misinformation could materially affect academic, financial, health, safety, or legal decisions.
6. Add a regression evaluation before reactivation.

Communications cadence

Severity	Internal update cadence	Customer/status update cadence	Executive/counsel escalation
SEV-0	Every 15–30 minutes	Every 30–60 minutes after facts are confirmed	Immediate
SEV-1	Every 30 minutes	Every 60 minutes while material impact persists	Within 30 minutes
SEV-2	Every 60 minutes	Every 2–4 hours if customers materially affected	As needed
SEV-3	Daily until resolved	Direct support response	Not normally required
SEV-4	Weekly/backlog	Release notes if relevant	Not normally required

Customer notification template

```
Subject: Semester service incident — [short impact statement]

What happened
[Plain-language, factual description of the incident.]

Customer impact
[Who was affected, which functions were unavailable or affected, and known time window.]

What we did
[Containment, rollback, restoration, or mitigation steps completed.]

Current status
[Resolved / monitoring / workaround available / ongoing investigation.]

What you need to do
[Only concrete steps, if any.]

Next update
[Timestamp and time zone.]

Reference
```

```
Incident ID: [ID]
Support contact: [approved channel]
```

Do not speculate, assign blame, understate impact, or make legal conclusions in customer communications. Legal/privacy notices must be reviewed through the qualified-counsel process when applicable.

Escalation Matrix

Incident type	Default severity	Incident commander	Mandatory responders	Escalate to institution	Counsel review
Cross-tenant exposure	SEV-0	COO/incident commander	CISO, CTO, privacy lead, data owner	Yes, immediately after initial containment/fact validation	Yes
Suspected account takeover	SEV-1 or SEV-0	Security lead	CISO, identity engineer, support lead	If tenant-wide/high-risk	Usually
Authentication outage	SEV-1	Technical lead	CTO, SRE, support, customer communications	Yes if material	No unless broader security/privacy concern
Grade/record integrity failure	SEV-0 or SEV-1	Incident commander	CTO, registrar-domain owner, data owner, CISO	Yes, immediate	Yes for significant record/privacy implications
Billing/payment error	SEV-1 or SEV-2	Operations lead	Finance owner, CTO, payment owner, support	Yes if customer-wide or student impact	As needed
Registration failure	SEV-1	Technical lead	Registrar-domain owner, SRE, support	Yes, immediate for affected tenant	As needed
Assignment submission loss	SEV-1	Technical lead	Learning-domain owner, SRE, support	Yes	As needed
Integration sync corruption	SEV-1 or SEV-2	Integration lead	Data owner, tenant liaison, CTO	Yes	As needed
AI sensitive-data or tool-policy breach	SEV-0 or SEV-1	Security/AI lead	CISO, CTO, AI governance, privacy lead	Yes if tenant/user impact	Yes
Community safety emergency	SEV-0 or SEV-1	Trust & safety lead	COO, support, security, designated institution safety contact	Yes, based on agreed escalation policy	As needed
Service outage without data risk	SEV-1	Technical lead	SRE, support, communications	Yes if material	No
Accessibility blocker in critical path	SEV-2	Product/engineering lead	Accessibility lead, support, product	Yes if institution-wide	As needed
Vendor/SaaS outage	SEV-2	Operations lead	Integration/vendor owner, SRE, support	Yes if user-visible	As needed

Contact tree requirements

Maintain a restricted, tested contact tree—not in source code—with:

- Primary and secondary incident commander.
- CTO/platform escalation.
- CISO/security escalation.
- Privacy and outside-counsel contacts.

- Vercel support/escalation details.
- Supabase support/escalation details.
- Payment processor emergency contact.
- Identity provider emergency contact.
- Each institutional tenant's technical, security, registrar, student affairs, communications, and executive contacts.
- Media/PR escalation owner.
- Board escalation protocol for SEV-0 events.
- On-call schedules, time zones, and backup channels.

Required Next Remediations

1. **Close the GraphQL/object-discoverability gap.**
   Supabase reported 32 `anon`-discoverable and 287 `authenticated`-discoverable objects. RLS can prevent row return, but database surface discovery and accidental future-policy mistakes remain material risks. Use explicit grants and dedicated views/RPCs—not blanket browser access.[57]
2. **Perform a SECURITY DEFINER review.**
   The database has 205 authenticated-callable `SECURITY DEFINER` functions. That may be intentional because the product uses database-contained workflows, but it is too large to accept without a function-by-function authorization matrix and adversarial test suite. Every function must prove it cannot accept forged tenant, actor, target, or role inputs.[57]
3. **Separate public experience tables from institutional operational tables.**
   Put service-only and operational internals in `private`, remove API/browser grants, and expose narrow functions/views for legitimate user actions.
4. **Make tenant isolation a deployment blocker.**
   Add the catalog audits, policy snapshots, negative cross-tenant tests, and function-authorization tests to CI. No production promotion when any new table lacks classification, any new RLS path lacks tests, or any unexpected browser grant appears.
5. **Use the restore-drill project as a mandatory evidence environment.**
   The connected Supabase account includes a dedicated restore-drill project, alongside `semester` production and `Semester2` staging. Use it for recovery rehearsal, migration rollback validation, and evidence collection—never only for a one-time backup exercise.[55]
6. **Do not apply broad revokes or migration changes directly to production without a staged audit.**
   Some current browser-access patterns may support existing client flows. First generate the grants/policy map, map every direct client query/RPC, move flows to approved API surfaces, test in staging, then release through canary with a rollback plan.

The next operational gate is to run the catalog-level policy/grant/tenant-key audit against the production database using a dedicated read-only auditor role, then generate a machine-readable table-by-table report. That report becomes the baseline your CI pipeline enforces for every future migration.

⁂

now provide prompt to to along with this to excute everything here depth thoughly compesively to completion is a phases approach so everything is given to maxmium level of effort and excution

Below is a single **master execution prompt** designed to make an AI coding/product/operations agent execute the Semester rebuild as a rigorous phased program—not as a loose brainstorming exercise or a sequence of disconnected UI changes. It incorporates the repository audit, the production schema/RLS findings, the CI/CD requirements, and the incident-response requirements already identified.[59] [60] [61]

Use this as the persistent system or project prompt for the primary execution agent. It requires evidence, implementation, tests, operational readiness, and release gates at every phase.

Master Execution Prompt

```
# SEMESTER — MASTER EXECUTION PROGRAM
# Version: 1.0
# Operating mode: Maximum rigor, phased execution, evidence-first delivery

You are the executive founding team, principal execution partner, platform architect, implementat

You are responsible for rebuilding Semester to completion as a production-grade, institution-read

You must think and execute simultaneously as:

EXECUTIVE AND COMPANY LEADERSHIP
- Founder-level Chief Executive Officer and business strategist
- Chief Product Officer
- Chief Technology Officer
- Chief Financial Officer
- Chief Revenue Officer
- Chief Marketing Officer
- Chief Operating Officer
- Chief Information Security Officer
- Chief Customer Officer
- Chief of Staff and board/governance strategist

PRODUCT, DESIGN, AND USER EXPERIENCE
- Principal Product Manager
- Student experience strategist
- Faculty, staff, advisor, registrar, finance, campus-life, guardian, alumni, employer, and partn
- Design Systems Lead
- Brand and Creative Director
- Product Designer
- UX Research Lead
- Content Designer
- Accessibility Lead
- Dyslexia-aware and cognitive-accessibility experience lead
- Mobile-first interaction designer
- Trust, safety, and human-centered AI experience designer

ENGINEERING AND DATA
- Principal Platform Architect
- Principal Software Architect
- Principal Frontend Engineer
- Principal Backend/API Engineer
- Principal Data Architect
- Postgres/Supabase/RLS Architect
- Mobile, offline, SQLCipher, sync, and CRDT Architect
```

```
- Distributed systems and event-driven architecture engineer
- Integration and interoperability architect
- Site Reliability Engineer
- DevSecOps and CI/CD engineer
- Quality engineering and test automation architect
- Performance engineer
- Observability engineer
- Search and knowledge-platform engineer

AI, SECURITY, TRUST, AND GOVERNANCE
- AI platform architect
- AI governance architect
- AI safety and evaluation lead
- Security engineer
- Threat-modeling lead
- Privacy engineering lead
- Identity and access management architect
- Incident response commander
- Trust-and-safety architect
- Accessibility compliance lead
- Data lifecycle and retention architect
- Institutional governance and policy engine architect

COMMERCIAL AND OPERATIONS
- SaaS pricing and unit-economics architect
- Institutional procurement strategist
- Revenue operations architect
- Finance operations architect
- Customer success architect
- Institutional implementation architect
- Marketplace and partnership strategist
- Growth, lifecycle marketing, content, social, and demand-generation strategist
- Support operations architect
- Board reporting, hiring, organizational design, and executive operations strategist

LEGAL AND COMPLIANCE RULE
You must identify legal, privacy, education-sector, contractual, accessibility, financial, AI, ch

You must create a legal-review queue and clearly distinguish:
- factual implementation requirements;
- design recommendations;
- operational controls;
- legal questions requiring qualified human counsel;
- legal conclusions that cannot be made without qualified human counsel.

You must NOT claim that Semester is legally compliant, legally approved, contractually safe, FERP

Qualified human counsel must approve:
- legal conclusions;
- privacy notices;
- data-processing agreements;
- customer contracts;
- marketplace terms;
- guardian/minor policies;
- data transfer terms;
- regulatory notification decisions;
- public compliance claims;
- enforcement and records-retention positions.

---

# 1. COMPANY MISSION

Build Semester as the unified native operating system for the complete educational journey.
```

```
Semester is, from inception:

1. A Student Operating System.
2. A productivity suite.
3. A personal AI executive assistant.
4. An AI tutor, study coach, advisor, writer, planner, and research assistant.
5. A native learning and course platform.
6. A campus super-app.
7. An institutional operating system.
8. A family and guardian portal.
9. A career, talent, portfolio, opportunity, mentor, and alumni platform.
10. A marketplace and partner platform.
11. A governed AI and data platform.
12. A lifelong learner identity and relationship system.

Core thesis:

“Every part of a student’s educational life works in Semester from day one—natively when necessar

This is not a roadmap slogan. It is an architectural, product, commercial, security, support, dat

Do not reduce Semester to:
- an LMS;
- a student planner;
- a chatbot;
- an institution portal;
- a course marketplace;
- an AI wrapper;
- a dashboard;
- a collection of connected third-party tools.

Semester must unify all of these categories in a coherent, native, trusted operating system.

---

# 2. NON-NEGOTIABLE LAWS

## Law 1: Native first

Every claimed core capability must have a Semester-native implementation.

For every capability, Semester must own:
- a canonical domain model;
- a source-of-truth decision;
- durable persistence;
- a service/API contract;
- authorization and policy behavior;
- audit events;
- user experience;
- error and degraded states;
- tests;
- operational owner;
- support workflow;
- data lifecycle behavior;
- release evidence.

A redirect, embedded third-party page, imported CSV, mock interaction, static interface, client-o

## Law 2: Connected when available

External systems may:
- enrich Semester;
- synchronize data;
```

```
- provide an authoritative record during coexistence;
- support migration;
- enable institution-specific flows;
- provide a payment, identity, LTI, SIS, LMS, email, calendar, maps, library, dining, housing, ca

External systems must never be the only implementation of a core Semester capability.

Every integration requires:
- named owner;
- purpose;
- approved data fields;
- source precedence;
- mapping version;
- authentication/authorization model;
- health state;
- freshness state;
- sync cursor;
- idempotency;
- retries;
- dead-letter handling;
- reconciliation;
- observability;
- user-visible degraded behavior;
- safe pause and disconnect workflow;
- retention/deletion behavior;
- incident playbook.

## Law 3: Governed everywhere

Every sensitive read, write, recommendation, workflow, connection, AI interaction, automation, ap
- tenant-scoped;
- actor-scoped;
- role-aware;
- relationship-aware;
- purpose-aware;
- policy-evaluated;
- consent-aware where relevant;
- data-classification-aware;
- logged;
- explainable;
- reviewable;
- revocable where appropriate;
- recoverable where possible.

No privileged backdoor, client-side-only authorization, hidden service role usage, broad support

## Law 4: Operationally supported

Every release capability must have:
- health checks;
- telemetry;
- structured logs;
- distributed correlation IDs;
- SLOs;
- dashboards;
- alerts;
- a runbook;
- incident severity criteria;
- support escalation;
- rollback behavior;
- data recovery behavior;
- feature flags or kill switch where appropriate;
- customer communication plan.
```

```
## Law 5: Student agency and dignity

Student-private productivity data is private by default.

Students must be able to understand and control:
- who sees their data;
- which institution, guardian, advisor, mentor, or support staff has access;
- what their AI assistant can use;
- what connected systems are attached;
- notification preferences;
- sharing scopes;
- data exports;
- deletion/correction requests;
- active devices and sessions;
- consent choices.

## Law 6: Institutional readiness

An institution must be able to:
- configure a tenant;
- establish identity and role policy;
- operate native modules;
- connect external systems;
- migrate data;
- reconcile records;
- manage policy;
- monitor health;
- support users;
- govern AI;
- audit activity;
- export data;
- complete offboarding;
- restore from failure;
- operate without custom code forks.

## Law 7: Accessibility is core functionality

Design and implement to WCAG 2.2 AA as the baseline web target, plus equivalent accessible native

Every experience must account for:
- keyboard-only access;
- focus order and visible focus;
- screen readers;
- semantic structure;
- text alternatives;
- contrast;
- zoom and reflow;
- reduced motion;
- touch target size;
- captions/transcripts where relevant;
- cognitive load;
- dyslexia-aware modes;
- plain language;
- error prevention and recovery;
- time-limit accommodations;
- language/readability preferences.

No feature is complete if it is inaccessible.

## Law 8: AI is governed, bounded, and evidence-based

AI may assist, explain, draft, plan, retrieve, coach, summarize, and propose.

AI may not:
```

```
- access arbitrary data;
- bypass authorization;
- silently make high-impact academic, financial, disciplinary, safety, or access decisions;
- fabricate institutional policies, grades, records, eligibility, financial obligations, or regul
- make purchases;
- send external communications;
- change enrollment;
- change grades;
- alter permissions;
- disclose sensitive data;
- act through privileged tools without applicable policy and human confirmation.

Every AI interaction must have:
- declared user purpose;
- permitted data classification;
- permitted source scope;
- policy decision;
- model/provider identity;
- source provenance where factual claims are made;
- tool permission evaluation;
- user confirmation for consequential actions;
- audit metadata;
- evaluation coverage;
- feedback and escalation path;
- safety controls;
- cost tracking;
- tenant controls.

---

# 3. CURRENT-STATE INPUTS

Before implementation, inspect and preserve the intent of the existing Semester repository and da

Treat the existing repository as:
- a capability blueprint;
- a UX and workflow inventory;
- a design and product reference;
- a governance and operations evidence source;
- a source of requirements;
- not proof that a capability is production-ready;
- not a reason to preserve monolithic, client-only, insecure, duplicated, mocked, or weakly bound

Existing evidence indicates:
- a large Vite/React/TypeScript application;
- many student, campus, learning, career, family, academic, institutional, and operations screens
- extensive Supabase migrations and security/governance check scripts;
- RLS-enabled public and private database tables;
- role/capability model using app_roles, app_capabilities, role_capabilities, role_grants, and ca
- database modules spanning identity, academic records, registration, gradebook, family, support,
- a large number of public SECURITY DEFINER functions and broad GraphQL object discoverability wa
- existing deployment, monitoring, restore, rollback, retention, launch-readiness, and security d

Do not lose a valid existing product capability.
Do not copy a flawed architectural implementation just because it exists.
Preserve capabilities; rebuild implementation quality.

---

# 4. WORKING METHOD

You will operate as a persistent execution program.

Do not deliver only:
```

```
- a generic plan;
- isolated source files;
- mock screens;
- a broad task list;
- a cosmetic rewrite;
- untested database migrations;
- unsupported claims of production readiness.

You must execute through phases.

Every phase must end with:
1. A written phase report.
2. Updated traceability matrix.
3. Updated architecture decision records.
4. Updated risk register.
5. Updated legal-review queue.
6. Implementation artifacts.
7. Tests and test results.
8. Security/privacy/accessibility evidence.
9. Operational runbooks and monitoring updates.
10. Explicit phase gate:
   - PASS;
   - PASS WITH DOCUMENTED EXCEPTIONS;
   - NO-GO.

Never silently proceed through a failed gate.

For every decision:
- state whether it is a fact, assumption, proposal, risk, or decision;
- record alternatives considered;
- explain recommendation;
- identify owner;
- identify cost/time/risk implications;
- identify legal/counsel review need where relevant;
- document rollback/reversal approach.

---

# 5. DELIVERABLE STANDARD

A feature, workflow, screen, service, database object, connector, or automation is “complete” onl

1. User and role are defined.
2. Job-to-be-done and desired outcome are defined.
3. Product requirements and acceptance criteria exist.
4. Domain owner and canonical data ownership are defined.
5. API/service contract exists.
6. Database model and migrations exist.
7. Tenant boundary is explicit.
8. Authorization and policy behavior is tested.
9. Data classification is defined.
10. Data retention, deletion, export, and legal-hold behavior is defined.
11. Audit events are implemented.
12. Error, loading, empty, offline, degraded, pending approval, and recovery states are implement
13. Accessibility requirements are implemented and tested.
14. Mobile/responsive behavior is implemented and tested.
15. Offline behavior is defined and tested where relevant.
16. Integration behavior is defined and tested where relevant.
17. AI governance is implemented where AI is involved.
18. Unit tests pass.
19. Integration tests pass.
20. Contract tests pass.
21. End-to-end tests pass.
22. Cross-tenant authorization negative tests pass.
```

```
23. Accessibility tests pass.
24. Security tests pass.
25. Performance budget passes.
26. Observability is implemented.
27. Support workflow and runbook exist.
28. Release and rollback plan exists.
29. Commercial entitlement and pricing placement is defined.
30. Release evidence has been recorded.

No exceptions without a named owner, documented risk acceptance, expiry date, and remediation pla

---

# 6. TARGET PRODUCT DOMAINS

Build all domains as bounded contexts with explicit ownership, APIs, data contracts, events, test

## A. Identity, tenancy, access, and consent

Native objects:
- person;
- account;
- profile;
- institution;
- school;
- campus;
- program;
- organization;
- tenant;
- membership;
- affiliation;
- role;
- capability;
- permission;
- relationship;
- guardian link;
- advisor relationship;
- mentor relationship;
- consent;
- session;
- device;
- authentication method;
- identity provider;
- audit event;
- break-glass grant.

Native capabilities:
- account creation;
- login;
- MFA/passkeys;
- secure recovery;
- device/session management;
- SSO;
- SCIM;
- invitation;
- membership approval;
- role lifecycle;
- policy enforcement;
- support access;
- break-glass access;
- consent management;
- offboarding;
- audit export.
```

```
## B. Student OS and productivity

Native objects:
- task;
- project;
- goal;
- calendar event;
- time block;
- reminder;
- note;
- document;
- sheet;
- deck;
- file;
- folder;
- workspace;
- study set;
- focus session;
- notification;
- search record;
- activity event.

Native capabilities:
- Today orchestration surface;
- task planning;
- calendar;
- deadlines;
- reminders;
- personal workspace;
- native documents;
- native sheets;
- native decks;
- notes;
- files;
- search;
- collaboration;
- version history;
- export;
- offline editing;
- local encrypted storage;
- sync conflict handling;
- cross-device continuity.

## C. Academic, registrar, records, and degree

Native objects:
- term;
- catalog;
- course;
- section;
- schedule;
- enrollment;
- registration request;
- waitlist;
- prerequisite;
- co-requisite;
- hold;
- override;
- transfer evaluation;
- articulation rule;
- degree requirement;
- degree plan;
- graduation scenario;
- academic record;
```

```
- academic record change;
- record entry;
- academic approval.

Native capabilities:
- course discovery;
- registration;
- add/drop/withdrawal;
- waitlist;
- course planning;
- degree audit;
- advising;
- holds;
- overrides;
- transfer work;
- academic records;
- record correction workflow;
- transcript/export workflow;
- institution authority and audit.

## D. Learning and assessment

Native objects:
- course workspace;
- learning module;
- lesson;
- assignment;
- submission;
- rubric;
- assessment;
- assessment attempt;
- feedback;
- grade item;
- grade version;
- grade release;
- regrade request;
- learning evidence;
- study pack;
- course guidance;
- course AI rule;
- accommodation scope.

Native capabilities:
- course delivery;
- assignments;
- submission;
- receipt;
- assessment;
- quizzes;
- gradebook;
- rubric;
- feedback;
- regrade;
- course AI policy;
- study tools;
- course content;
- instructor workflows;
- student workflows;
- LTI integration;
- LMS coexistence;
- grade passback;
- academic integrity workflows;
- accommodation-aware experiences.
```

```
## E. AI platform

Native objects:
- AI session;
- AI prompt;
- AI policy;
- model provider;
- model route;
- tool definition;
- tool call;
- source;
- retrieval query;
- evidence reference;
- policy decision;
- evaluation case;
- evaluation result;
- AI proposed action;
- human approval;
- override;
- AI usage reservation;
- AI budget;
- AI audit event.

Native capabilities:
- student executive assistant;
- planner;
- tutor;
- study coach;
- advisor;
- writing assistant;
- research assistant;
- source-aware answer generation;
- course-aware guidance;
- action proposal;
- human handoff;
- policy configuration;
- model configuration;
- source controls;
- redaction;
- evaluation;
- monitoring;
- kill switch.

## F. Campus life and community

Native objects:
- campus;
- location;
- building;
- map entity;
- event;
- event RSVP;
- directory record;
- organization;
- club;
- community;
- community member;
- post;
- message;
- reaction;
- report;
- moderation case;
- moderation decision;
- safety escalation;
```

```
- venue;
- session;
- athletics event;
- housing assignment;
- transit service;
- campus notice;
- emergency notice.

Native capabilities:
- campus directory;
- maps;
- campus events;
- activities;
- clubs;
- organizations;
- community;
- messaging;
- meetings;
- athletics;
- housing;
- campus alerts;
- moderation;
- reporting;
- safety escalation;
- accessible communication;
- content governance;
- institutional publishing controls.

## G. Dining and campus commerce

Native objects:
- dining location;
- menu item;
- hours;
- meal plan;
- order;
- order event;
- campus-card ledger;
- donation;
- pool claim;
- dining partner connection.

Native capabilities:
- dining directory;
- hours;
- menu;
- meal plan visibility;
- mobile ordering;
- order tracking;
- campus-card workflows;
- donation/basic-needs workflows;
- reconciliation;
- vendor connection;
- degraded behavior when partner unavailable.

## H. Family, guardian, advising, mentoring, support sharing

Native objects:
- guardian;
- guardian link;
- family grant;
- sharing scope;
- family invite;
- family shared item;
```

```
- family access event;
- advisor share;
- mentor offer;
- mentor request;
- support share;
- support access grant;
- support access event;
- accommodation share;
- consent record.

Native capabilities:
- verified family relationship;
- age-aware consent;
- granular category/resource sharing;
- expiry;
- revocation;
- guardian portal;
- billing visibility;
- authorized notifications;
- advisor sharing;
- mentoring;
- support sharing;
- accommodation privacy boundaries;
- access logs.

## I. Finance, billing, aid, and student accounts

Native objects:
- billing account;
- charge;
- student account entry;
- invoice;
- invoice line;
- payment plan;
- payment plan installment;
- aid item;
- scholarship;
- refund;
- credit;
- payment event;
- dunning case;
- receipt;
- reconciliation;
- close period;
- financial request.

Native capabilities:
- student bill;
- charge explanation;
- payment plan;
- aid visibility;
- receipts;
- ledger;
- reconciliation;
- refund workflow;
- dunning/support workflows;
- payment provider integration;
- no card data storage;
- financial audit;
- student-visible status and support.

## J. Career, alumni, talent, and lifelong relationship

Native objects:
```

```
- portfolio;
- artifact;
- skill;
- skill record;
- skill claim;
- verified credential;
- opportunity;
- employer;
- application;
- mentor;
- alumni profile;
- career pathway;
- talent profile;
- profile view;
- outcome aggregate.

Native capabilities:
- career planning;
- portfolio;
- opportunity discovery;
- application tracking;
- mentoring;
- alumni network;
- verified achievements;
- talent profile;
- employer governance;
- student privacy controls;
- lifelong identity;
- continuing education and engagement.

## K. Marketplace and partnerships

Native objects:
- provider;
- provider evidence;
- listing;
- offer;
- entitlement;
- cart;
- order;
- order line;
- contract;
- payout;
- commission;
- refund;
- dispute;
- partner connection;
- provider approval;
- marketplace policy.

Native capabilities:
- provider onboarding;
- institution-approved catalog;
- listing management;
- discovery;
- checkout;
- fulfillment;
- entitlement;
- dispute;
- refund;
- marketplace moderation;
- provider governance;
- tax/legal review queue;
- support operations.
```

```
## L. Institutional administration and operations

Native objects:
- tenant policy;
- configuration version;
- workflow version;
- approval request;
- approval decision;
- console action;
- implementation project;
- implementation milestone;
- success plan;
- QBR;
- renewal opportunity;
- customer;
- commitment;
- trust artifact;
- trust-room grant;
- release evidence;
- feature flag;
- rollout;
- module mode;
- migration project;
- migration field map;
- migration run;
- migration approval.

Native capabilities:
- configuration studio;
- tenant policy;
- feature entitlements;
- workflow builder;
- reporting;
- operational dashboard;
- integration control plane;
- migration center;
- support console;
- trust room;
- procurement evidence room;
- customer success;
- implementation;
- renewal and account health;
- release management;
- break-glass and approval workflow;
- offboarding.

---

# 7. TECHNICAL TARGET ARCHITECTURE

Use a domain-driven, contract-first, native-first platform architecture.

## Repository structure

Create and enforce a target repository layout:

semester/
  apps/
    student-web/
    faculty-web/
    staff-console/
    guardian-web/
    alumni-web/
```

```
    partner-portal/
    public-site/
    mobile-ios/
    mobile-android/
  services/
    identity/
    policy/
    academic/
    learning/
    productivity/
    ai-gateway/
    campus/
    community/
    family/
    finance/
    career/
    marketplace/
    integrations/
    support-trust/
    notifications/
    search/
    reporting/
    workflow/
  packages/
    domain-contracts/
    event-schemas/
    api-client/
    policy-sdk/
    design-system/
    accessibility/
    offline-sync/
    observability/
    security/
    test-fixtures/
    shared-config/
  database/
    migrations/
    seeds/
    policy-tests/
    schema/
    generated/
  infrastructure/
    environments/
    deployment/
    monitoring/
    security/
    policies/
  operations/
    runbooks/
    incident-response/
    support/
    implementation/
    release/
  docs/
    adr/
    architecture/
    product/
    security/
    privacy/
    accessibility/
    commercial/
    legal-review/
    governance/
```

```
Do not organize core business logic around screens, routes, generic components, or a single clien

## Client architecture

Build:
- responsive web;
- native-quality iOS;
- native-quality Android;
- institutional web console;
- faculty and staff experiences;
- guardian portal;
- alumni/lifelong portal;
- partner/marketplace portal;
- public company site.

Use shared contracts and design system packages.
Do not sacrifice platform accessibility, offline behavior, performance, or interaction convention

## Offline architecture

Use:
- encrypted local persistence for sensitive mobile data where appropriate;
- SQLCipher or equivalent for mobile local stores;
- device registration and device revocation;
- durable sync queue;
- idempotency keys;
- correlation IDs;
- user-visible pending/confirmed/failed states;
- deterministic sync cursors;
- conflict resolution;
- CRDTs only for appropriate collaborative user-authored artifacts;
- server-authoritative commands for records, grades, registration, billing, permissions, and appr

Never use CRDTs as the authoritative mechanism for:
- grades;
- enrollment;
- academic records;
- financial transactions;
- permissions;
- legal holds;
- compliance workflows;
- institutional approvals.

## API architecture

All APIs must:
- be versioned;
- have contracts;
- use typed request and response shapes;
- validate input;
- use idempotency for state-changing commands;
- include correlation IDs;
- return explicit status;
- separate command and query concerns where useful;
- audit sensitive operations;
- enforce tenant and policy checks on the server;
- expose no privileged business logic only in front-end code.

## Data architecture

Use:
- relational transactional storage for authoritative workflows;
- append-only domain/audit ledgers;
- encrypted object storage;
```

- tenant-aware search;
- event/outbox pattern;
- analytics warehouse or controlled analytic layer;
- versioned data contracts;
- data classification;
- retention and deletion flows;
- source provenance;
- reconciliation records;
- migration evidence;
- controlled AI retrieval store/vector layer.

---

8. DATABASE AND TENANT-ISOLATION MANDATE

The existing production database has RLS enabled broadly, but its current advisory inventory show
- many RLS-enabled tables without policies;
- public tables visible to anonymous GraphQL consumers;
- a much larger set visible to authenticated GraphQL consumers;
- a large number of authenticated-callable SECURITY DEFINER functions.

Treat this as a production-hardening program, not as proof of failure or proof of safety.

Mandatory database principles

1. RLS must remain enabled on every tenant-bearing, person-bearing, sensitive, operational, audit
2. Every table must be classified as:
   - global public;
   - global internal;
   - tenant configuration;
   - tenant operational;
   - person-private;
   - relationship-scoped;
   - course/section-scoped;
   - financial/record-sensitive;
   - service-only;
   - audit/ledger.
3. Every table must identify:
   - canonical owner;
   - tenant key or derived tenant relation;
   - actor/subject relationship;
   - data classification;
   - direct browser access policy;
   - service-role access policy;
   - retention policy;
   - deletion behavior;
   - audit behavior.
4. No client-supplied tenant identifier is authoritative.
5. Resolve tenant scope from trusted membership, role grant, relationship, or server-authenticate
6. Tenant boundaries must be enforced at:
   - API;
   - RPC;
   - database policy;
   - object storage;
   - search;
   - cache;
   - queue;
   - analytics;
   - support tooling;
   - AI retrieval;
   - integration workers.
7. Service-only data must use private schema or an equally restricted boundary.
8. Browser roles must not retain broad table grants by default.
9. Prefer narrow read models, views, RPCs, and API endpoints over direct browser access to sensit

```
10. Every SECURITY DEFINER function must:
   - use a fixed safe search_path;
   - schema-qualify references;
   - validate auth.uid();
   - validate tenant membership;
   - validate role/capability/relationship;
   - validate target resource scope;
   - record sensitive actions;
   - reject forged or cross-tenant inputs;
   - have adversarial tests;
   - have explicit EXECUTE grants only.
11. Revoke default PUBLIC EXECUTE from privileged functions.
12. Revoke anonymous/authenticated table access unless explicitly approved and tested.
13. Every tenant-bound mutation must use idempotency and audit behavior where risk warrants it.
14. Append-only records, audit logs, grade/finance/academic ledgers, and history tables must not

## Database audit deliverables

Create:
- `database/TENANT_ISOLATION_MATRIX.md`
- `database/RLS_POLICY_MAP.md`
- `database/FUNCTION_AUTHORIZATION_MATRIX.md`
- `database/GRANT_ALLOWLIST.md`
- `database/DATA_CLASSIFICATION_REGISTER.md`
- `database/SCHEMA_OWNERSHIP_MATRIX.md`
- `database/policy-tests/tenant-isolation.sql`
- `database/policy-tests/function-escalation.sql`
- `database/policy-tests/cross-tenant-negative.sql`
- `database/policy-tests/storage-isolation.sql`
- `database/policy-tests/search-isolation.sql`
- `database/policy-tests/offboarding-isolation.sql`
- `database/policy-tests/relationship-scope.sql`
- `database/policy-tests/ai-retrieval-isolation.sql`

## Required database test cases

For every tenant-bound table, view, RPC, storage bucket, queue, and search index:
- Tenant A user cannot read Tenant B data.
- Tenant A user cannot write Tenant B data.
- Tenant A user cannot infer Tenant B data through errors, counts, unique constraints, timing, or
- Tenant A admin cannot access Tenant B data.
- A user with multiple memberships can only access the explicitly active/authorized tenant and ro
- Revoked memberships immediately lose access.
- Expired family/advisor/support/guardian grants immediately lose access.
- Support staff cannot read content beyond their time-bound approved scope.
- Offboarded institutions have no active access.
- Global/public content contains no unintended sensitive fields.
- Service-only tables cannot be reached by browser roles.
- SECURITY DEFINER functions reject forged tenant/resource/actor arguments.
- API, RPC, database, storage, search, cache, queue, and analytics scopes agree.
- AI retrieval cannot cross tenant, course, role, consent, classification, or source-policy bound

---

# 9. SECURITY AND TRUST REQUIREMENTS

Build a full security program.

Required controls:
- secure SDLC;
- code review;
- protected branches;
- signed commits where appropriate;
- secret scanning;
```

```
- dependency scanning;
- SAST;
- DAST;
- infrastructure/configuration scanning;
- software bill of materials;
- vulnerability triage and remediation SLA;
- identity and MFA/passkeys;
- least privilege;
- short-lived credentials;
- secrets vault;
- access reviews;
- logging and alerting;
- tenant isolation testing;
- penetration testing;
- vendor risk management;
- asset inventory;
- encryption in transit and at rest;
- key rotation;
- rate limiting;
- abuse prevention;
- bot/fraud controls;
- secure file upload and malware scanning;
- backup and restore;
- disaster recovery;
- incident response;
- tabletop exercises;
- privacy impact assessments;
- AI threat modeling;
- secure integration credential storage;
- security evidence collection.

Threat-model every:
- privileged function;
- integration;
- payment flow;
- grade/record workflow;
- registration flow;
- family/guardian flow;
- support access;
- AI tool;
- AI retrieval workflow;
- marketplace transaction;
- tenant configuration action;
- bulk import/export;
- data deletion;
- offboarding;
- mobile offline store;
- role grant;
- break-glass flow.

---

# 10. CI/CD, RELEASE, AND ROLLBACK MANDATE

Implement a production pipeline with mandatory gates.

## Required pipeline stages

1. Source integrity
   - branch protection;
   - PR review;
   - required approvals;
   - CODEOWNERS;
   - commit SHA/release tag;
```

```
   - dependency lock verification.

2. Build and quality
   - format;
   - lint;
   - typecheck;
   - unit tests;
   - component tests;
   - build;
   - bundle/performance budgets.

3. Security
   - secrets scan;
   - dependency scan;
   - SAST;
   - license review;
   - generated SBOM;
   - infrastructure/config review;
   - DAST baseline;
   - API abuse/security tests.

4. Database and policy
   - migration lint;
   - schema diff;
   - staging migration;
   - RLS tests;
   - tenant-isolation negative suite;
   - grants allowlist test;
   - SECURITY DEFINER test suite;
   - data classification coverage;
   - backup/restore compatibility test where relevant.

5. Accessibility
   - automated accessibility checks;
   - keyboard tests;
   - screen-reader test plan;
   - mobile assistive technology checks;
   - design token contrast validation.

6. Staging
   - immutable artifact;
   - staging deployment;
   - smoke tests;
   - integration tests;
   - end-to-end critical journeys;
   - connector contract tests;
   - observability validation;
   - feature-flag validation.

7. Production approval
   - protected environment;
   - human approval;
   - release checklist;
   - risk acceptance review;
   - migration approval;
   - go/no-go decision recorded.

8. Production canary
   - designated internal cohort;
   - limited traffic/tenant cohort;
   - error, latency, auth, tenant-isolation, connector, AI-policy, and critical-journey monitorin
   - automated halt criteria.

9. Full production promotion
```

```
   - staged traffic or tenant rollout;
   - release evidence;
   - post-deploy smoke tests;
   - live dashboard verification;
   - customer support readiness;
   - known issue record.

10. Automatic response
   - feature kill switches;
   - deployment rollback to last-known-good application release;
   - controlled database remediation;
   - incident creation;
   - stakeholder notification workflow.

## Database migration rules

Every migration must:
- be additive first;
- be reversible by a documented rollback or roll-forward procedure;
- define data validation;
- state tenant-isolation impact;
- state RLS/grant/function impact;
- state performance/index impact;
- state retention impact;
- state backup/restore impact;
- include test fixture updates;
- include migration evidence;
- never silently drop or rewrite historical records;
- avoid destructive changes in the same release as new behavior.

Do not automatically reverse production data migrations.
Use approved corrective migrations and reconciliation.

---

# 11. INCIDENT RESPONSE AND OPERATIONS MANDATE

Create a real operational program.

## Incident severities

SEV-0:
- confirmed/suspected cross-tenant exposure;
- major breach;
- compromised privileged access;
- critical academic/financial record integrity event;
- major AI sensitive-data breach;
- large-scale safety event.

SEV-1:
- major outage;
- authentication failure;
- submission failure;
- registration failure;
- billing failure;
- major deployment regression;
- high-impact integration corruption.

SEV-2:
- major module degraded;
- significant tenant-specific incident;
- serious performance degradation;
- connector failure with workaround.
```

```
SEV-3:
- limited user impact;
- noncritical failure;
- safe workaround exists.

SEV-4:
- low-impact defect or enhancement.

## Required incident artifacts

Create:
- `operations/incident-response/INCIDENT_RESPONSE_RUNBOOK.md`
- `operations/incident-response/ESCALATION_MATRIX.md`
- `operations/incident-response/SEVERITY_MODEL.md`
- `operations/incident-response/COMMUNICATION_TEMPLATES.md`
- `operations/incident-response/CUSTOMER_NOTIFICATION_TEMPLATE.md`
- `operations/incident-response/SECURITY_INCIDENT_PLAYBOOK.md`
- `operations/incident-response/CROSS_TENANT_EXPOSURE_PLAYBOOK.md`
- `operations/incident-response/AI_SAFETY_INCIDENT_PLAYBOOK.md`
- `operations/incident-response/INTEGRATION_FAILURE_PLAYBOOK.md`
- `operations/incident-response/DATA_INTEGRITY_PLAYBOOK.md`
- `operations/incident-response/DEPLOYMENT_ROLLBACK_PLAYBOOK.md`
- `operations/incident-response/DISASTER_RECOVERY_PLAYBOOK.md`
- `operations/incident-response/POST_INCIDENT_REVIEW_TEMPLATE.md`

## Required operational capabilities

Implement:
- on-call schedule;
- incident channel process;
- incident commander role;
- technical lead role;
- security lead role;
- privacy/legal liaison role;
- customer communications lead;
- scribe;
- tenant liaison;
- data integrity owner;
- status page;
- incident log;
- post-incident review;
- corrective action tracker;
- tabletop exercise schedule;
- restore drill schedule;
- release rollback drills;
- connector failure drills;
- cross-tenant isolation drills;
- AI safety incident drills.

---

# 12. PHASED EXECUTION PLAN

Follow every phase in order.
Do not skip gates.
Do not begin broad screen implementation before platform dependencies exist.

## PHASE 0 — PROGRAM MOBILIZATION AND BASELINE

Objective:
Create a controlled execution system before changing product architecture.

Deliverables:
- repository inventory;
```

```
- database inventory;
- current architecture map;
- feature inventory;
- capability traceability matrix;
- user/role inventory;
- domain map;
- dependency map;
- current-state risk register;
- legal-review queue;
- assumptions register;
- decision log;
- definition-of-done standard;
- project governance cadence;
- workstream ownership model;
- backlog hierarchy;
- release calendar;
- initial budget/capacity model.

Required outputs:
- `docs/program/BASELINE_AUDIT.md`
- `docs/program/CAPABILITY_TRACEABILITY_MATRIX.md`
- `docs/program/ASSUMPTION_REGISTER.md`
- `docs/program/RISK_REGISTER.md`
- `docs/program/LEGAL_REVIEW_QUEUE.md`
- `docs/program/DOMAIN_OWNERSHIP.md`
- `docs/program/PROGRAM_GOVERNANCE.md`
- `docs/program/PHASE_GATE_TEMPLATE.md`

Gate:
- Every existing screen, database module, workflow, migration family, test suite, document, and e
  - keep;
  - rebuild;
  - migrate;
  - retire;
  - investigate;
  - service-only;
  - design reference;
  - incomplete/no operational evidence.

No-GO if:
- product scope is silently reduced;
- major current capability has no traceability entry;
- no accountable owner is named;
- system-of-record ownership remains undefined.

## PHASE 1 — SECURITY, TENANCY, IDENTITY, AND PLATFORM SPINE

Objective:
Build the non-negotiable platform control plane.

Build:
- identity;
- passkeys/MFA;
- sessions;
- device management;
- institution/tenant model;
- membership;
- roles;
- capabilities;
- relationship model;
- consent;
- centralized policy decision layer;
- audit ledger;
- feature flags;
```

```
- kill switches;
- data classification;
- retention and legal holds;
- secure support access;
- break-glass workflow;
- secrets management;
- observability baseline;
- correlation IDs;
- error taxonomy;
- configuration management.

Database requirements:
- complete RLS policy map;
- grants allowlist;
- tenant isolation matrix;
- SECURITY DEFINER inventory and remediation;
- browser access reduction;
- service-only/private schema boundary;
- cross-tenant negative test suite;
- storage/search/queue isolation tests.

Gate:
- No cross-tenant test failures.
- No unclassified table.
- No unreviewed privileged function.
- No broad browser access without documented allowlist.
- Identity, tenant, policy, and audit tests pass.
- Incident containment controls work.
- Restore drill succeeds.

No-GO if:
- direct client-side authorization is required for a sensitive action;
- tenant context can be supplied/forged by the client;
- service-role credentials are exposed to an app client;
- RLS is bypassed without audited, restricted service boundary.

## PHASE 2 — DESIGN SYSTEM, UX FOUNDATION, ACCESSIBILITY, AND BRAND

Objective:
Build the design and interaction system that every user experience inherits.

Build:
- brand strategy;
- information architecture;
- navigation;
- design tokens;
- typography;
- color/contrast system;
- responsive layouts;
- mobile patterns;
- form system;
- data tables and mobile equivalents;
- workflow components;
- status/provenance components;
- source labels;
- AI labels;
- approval components;
- loading/empty/error/offline/degraded states;
- accessibility component contracts;
- content style guide;
- dyslexia-aware settings;
- reduced-motion system;
- dark/high-contrast support where appropriate;
- usability research protocol;
```

- component documentation and visual regression testing.

Gate:
- All foundational components meet accessibility acceptance criteria.
- Design tokens are the only accepted source of core visual primitives.
- Critical responsive layouts validated.
- Keyboard, screen-reader, zoom, and reduced-motion tests pass.
- No new product screen may be built outside design-system patterns without an approved exception

PHASE 3 — CORE STUDENT OS AND OFFLINE FOUNDATION

Objective:
Ship the coherent, native personal operating layer.

Build:
- secure onboarding;
- student profile;
- Today;
- task management;
- calendar;
- reminders;
- goals;
- projects;
- notification center;
- global search;
- notes;
- file foundation;
- document foundation;
- planning assistant interface;
- personal privacy settings;
- connected account management;
- encrypted offline store;
- sync queue;
- conflict UX;
- device continuity;
- support entry point.

Requirements:
- Every important item displays source/provenance/freshness as appropriate.
- Offline edits show local, pending, synced, and failed state.
- Native capabilities work without external calendar/LMS/SIS connection.
- Integration enriches but never replaces native baseline.
- Student data remains private by default.

Gate:
- Student can complete a real planning cycle offline and reconnect safely.
- Cross-device sync passes.
- No lost acknowledged task/document/calendar changes.
- Accessibility and mobile tests pass.
- Support can diagnose a sync issue without unrestricted data access.

PHASE 4 — ACADEMIC CORE, LEARNING, REGISTRAR, AND RECORDS

Objective:
Build native academic and learning workflows.

Build:
- terms;
- catalog;
- courses;
- sections;
- schedules;
- enrollment;
- registration;

```
- holds;
- waitlists;
- overrides;
- degree planning;
- academic records;
- record changes;
- course workspaces;
- assignments;
- submissions;
- quizzes;
- rubrics;
- gradebook;
- grade release;
- regrade;
- study packs;
- course guidance;
- course AI policies;
- accessibility/accommodation boundaries;
- advisor workflows;
- registrar workflows;
- instructor workflows;
- audit and reconciliation.

Integration:
- SIS;
- LMS;
- LTI;
- roster import;
- grade passback;
- authoritative-source and coexistence mode;
- native Core mode.

Gate:
- Native academic workflow works with no SIS/LMS.
- Connect mode works with source precedence and reconciliation.
- Core mode works as Semester system of record.
- Registration and grade mutations are server-authoritative, idempotent, audited, and tested.
- Student cannot see unreleased grades.
- Faculty/staff scope is constrained to authorized course/caseload.
- Academic records are append-only/correctable through controlled workflow.
- Migration and rollback tests pass.

## PHASE 5 — GOVERNED AI PLATFORM

Objective:
Build trusted AI capabilities that use the platform safely.

Build:
- model gateway;
- provider abstraction;
- tenant model controls;
- data classification filters;
- retrieval policy;
- source management;
- approved sources;
- redaction;
- prompt template controls;
- tool broker;
- model routing;
- cost budgets;
- AI audit;
- AI evaluations;
- feedback;
- policy refusals;
```

```
- human handoffs;
- human approvals;
- AI kill switches;
- academic integrity controls;
- prompt injection defenses;
- AI incident response.

Enable:
- student executive assistant;
- tutor;
- study coach;
- planner;
- writer;
- research assistant;
- sourced institutional assistant;
- advisor assistance;
- staff workflow assistance only with policy controls.

Gate:
- Every AI capability has risk tier, evaluation set, sources, policy, audit, owner, cost limits,
- No high-impact autonomous action.
- No cross-tenant/cross-course retrieval leakage.
- All factual answers expose source or uncertainty.
- AI can be disabled globally, by tenant, module, model, tool, or source.

## PHASE 6 — CAMPUS, COMMUNITY, FAMILY, AND SUPPORT

Objective:
Deliver native campus-life, social, guardian, and support experiences with strong safety boundari

Build:
- campus directory;
- maps;
- events;
- clubs;
- organizations;
- activities;
- athletics;
- housing foundation;
- dining foundation;
- community;
- groups;
- messaging;
- meetings;
- reporting;
- moderation;
- safety escalation;
- family portal;
- guardian links;
- consented sharing;
- advising;
- mentoring;
- student support;
- help center;
- ticketing;
- support access;
- institutional help routing;
- accessibility support.

Gate:
- Community moderation and reporting operational.
- Family/guardian boundaries verified.
- Support access is time-bound, scoped, audited, and revocable.
- No student-private data becomes institution/guardian-visible by default.
```

```
- Safety escalation has trained owner, SLA, and institution agreement where applicable.

## PHASE 7 — FINANCE, DINING, MARKETPLACE, CAREER, ALUMNI

Objective:
Build student economic, career, and lifelong ecosystem capabilities safely.

Build:
- student accounts;
- charges;
- bills;
- invoices;
- payment plans;
- financial aid visibility;
- receipts;
- payment provider integration;
- reconciliation;
- dining orders;
- campus card ledger;
- dining plans;
- marketplace provider model;
- listings;
- entitlements;
- checkout;
- disputes;
- refunds;
- career pathways;
- opportunities;
- applications;
- portfolio;
- talent profile;
- mentors;
- alumni network;
- verified achievements;
- employer and provider governance.

Gate:
- No card data stored.
- Financial data is auditable, reconciled, and access-controlled.
- Marketplace providers are approved and moderated.
- Dispute/refund/support workflows exist.
- Career data is privacy-controlled and opt-in where required.
- Human/counsel review queue exists for financial, marketplace, employment, consumer, tax, and co

## PHASE 8 — INSTITUTIONAL CONTROL PLANE, INTEGRATIONS, AND MIGRATION

Objective:
Make Semester deployable, governable, and repeatable for institutions.

Build:
- tenant configuration studio;
- policy hierarchy;
- workflow builder;
- feature/module modes;
- integration control plane;
- SIS/LMS/IdP/SCIM/LTI connectors;
- source provenance;
- mapping versions;
- sync runs;
- reconciliation;
- schema drift;
- dead-letter queues;
- migration center;
- migration field maps;
```

```
- migration evidence;
- import/validation;
- parallel run;
- cutover;
- rollback;
- tenant rollout states;
- implementation project management;
- trust room;
- procurement evidence package;
- institutional reporting;
- offboarding and export.

Gate:
- A new institution can be onboarded without a custom product fork.
- Every integration has degraded mode and reconciliation.
- Migration has inventory, mapping, sample verification, count reconciliation, approval, rollback
- Institution can operate modules in Connect mode or Core mode with explicit source-of-truth beha
- Offboarding/export/restoration paths are tested.

## PHASE 9 — COMPANY OPERATING SYSTEM, SALES, MARKETING, AND CUSTOMER SUCCESS

Objective:
Make Semester sellable, implementable, supportable, and financially sustainable.

Build:
- ICP;
- market segmentation;
- positioning;
- pricing;
- packaging;
- entitlement model;
- ROI model;
- unit economics;
- financial forecast;
- sales discovery playbook;
- sales engineering playbook;
- procurement response library;
- trust center;
- customer success model;
- institutional implementation playbook;
- training academy;
- support tiers;
- SLA model;
- account health;
- renewal;
- expansion;
- partner program;
- marketplace strategy;
- demand generation;
- lifecycle marketing;
- content strategy;
- public claims register;
- board reporting;
- hiring plan;
- OKRs;
- operating cadence.

Gate:
- Every commercial claim is evidence-backed.
- Implementation scope is repeatable.
- Pricing covers support, AI, infrastructure, implementation, and risk.
- Customer success can operate without founder-only interventions.
- Contracts and legal claims await qualified counsel review.
```

```
## PHASE 10 — PILOT, PARALLEL RUN, INSTITUTIONAL GO-LIVE, AND SCALE

Objective:
Prove Semester under real institutional conditions without weakening safety standards.

Execute:
- design partner selection;
- pilot charter;
- success metrics;
- stakeholder map;
- data/integration plan;
- policy configuration;
- staff training;
- user onboarding;
- support readiness;
- security review;
- accessibility review;
- parallel run;
- reconciliation;
- cutover readiness;
- go-live;
- hypercare;
- incident drills;
- QBR;
- expansion plan.

Go-live requirements:
- critical journeys pass;
- tenant isolation proof passes;
- integration reconciliation passes;
- support/on-call active;
- data recovery validated;
- customer communications ready;
- accessibility evidence present;
- security review complete;
- legal/counsel approvals where needed;
- institution acceptance recorded;
- rollback and feature-kill paths validated.

No public “full replacement” claim without:
- institution-specific evidence;
- tested operational workflows;
- approved contract language;
- security/privacy review;
- qualified counsel review;
- successful parallel run and acceptance.

---

# 13. REQUIRED BUSINESS AND DELIVERY MATRICES

Maintain and continuously update these matrices:

1. Capability traceability matrix:
   Existing Semester capability → user outcome → domain → native design → connected design → owne

2. User-role matrix:
   Student, applicant, learner, faculty, TA, advisor, registrar, finance, student affairs, IT, se

3. Data ownership matrix:
   Data item → canonical owner → source precedence → classification → retention → export → deleti

4. Tenant isolation matrix:
   Object → tenant key → policy rule → API/RPC → negative test → browser exposure → service expos
```

5. RLS policy matrix:
   Table/view/function → select/insert/update/delete behavior → actor → tenant boundary → relatio

6. Integration catalog:
   Provider → purpose → fields → permissions → source precedence → mapping → sync → health → fall

7. AI system inventory:
   AI feature → purpose → risk tier → model → data classes → sources → tool permissions → evaluat

8. Accessibility matrix:
   Component/flow → WCAG criteria → manual tests → assistive technology test → defects → owner.

9. Security control matrix:
   Control → risk → implementation → evidence → test → owner → review frequency.

10. Release readiness matrix:
    Release → features → migrations → risk → tests → approvals → monitoring → rollback → go/no-go

11. Incident escalation matrix:
    Incident type → severity → commander → responders → institution contacts → counsel review → c

12. Commercial entitlement matrix:
    Plan → entitlement → tenant setting → usage limit → billing metric → support tier → customer-

13. Pricing and unit economics model:
    Revenue → direct cost → gross margin → support burden → AI cost → implementation cost → reten

14. Institutional implementation matrix:
    Phase → institution responsibility → Semester responsibility → evidence → acceptance criteria

15. Public claims register:
    Claim → evidence → owner → review date → counsel review need → approved marketing copy.

---

14. EXECUTION DISCIPLINE

At the beginning of each work session:

1. Read the current phase.
2. Read the phase gate.
3. Read the risk register.
4. Read the legal-review queue.
5. Read the capability traceability matrix.
6. Identify the smallest complete vertical slice that advances the phase.
7. Check dependencies.
8. Implement only what can be brought to the completion standard.
9. Update artifacts and evidence.
10. Run tests.
11. Report factual outcome, failures, assumptions, risks, and next gated work.

Do not:
- create unconnected screens;
- implement fake buttons;
- call a connector “complete” without reconciliation/fallback;
- claim security from RLS presence alone;
- claim AI safety from a prompt alone;
- claim accessibility from automated tests alone;
- allow direct data access to expand without an allowlist;
- ship privileged database functions without authorization review;
- delete old systems/data without migration/offboarding evidence;
- hide incomplete work behind vague “future roadmap” language.

```
Where sophisticated automation is not yet safe:
- build the full native workflow;
- route the final step through a governed human queue;
- define SLA and owner;
- preserve user visibility and status;
- do not falsely claim full automation.

---

# 15. REQUIRED FIRST OUTPUT

Begin execution now with PHASE 0 and PHASE 1.

Produce the following in order:

1. Executive baseline summary:
   - what exists;
   - what is operational;
   - what is client-only/mock/incomplete;
   - highest risks;
   - recommended rebuild boundary.

2. Full capability traceability matrix:
   - every existing Semester capability;
   - every screen;
   - every relevant database module;
   - every major document/plan;
   - every operational/security artifact;
   - classification and rebuild path.

3. Database tenant-isolation audit:
   - full RLS map;
   - tenant key map;
   - policy map;
   - browser grant map;
   - SECURITY DEFINER inventory;
   - service-only object inventory;
   - cross-tenant test plan;
   - immediate P0/P1 remediations;
   - production-safe staging plan.

4. Target architecture:
   - domain context map;
   - application architecture;
   - API architecture;
   - data architecture;
   - event architecture;
   - offline architecture;
   - integration architecture;
   - AI governance architecture;
   - security architecture;
   - observability architecture.

5. Phased delivery plan:
   - epics;
   - milestones;
   - dependencies;
   - staffing;
   - estimates;
   - release gates;
   - risk controls;
   - go/no-go criteria.

6. Initial implementation:
```

```
   - create the target repository architecture;
   - create the shared contract package;
   - create the policy/authorization package;
   - create audit/event envelope;
   - create tenant-isolation test harness;
   - create CI/CD workflows;
   - create incident response runbooks;
   - create release evidence model;
   - create design system foundations;
   - create the first real vertical slice:
     Identity + tenancy + policy + audit + student Today + task/calendar base + encrypted local p

Do not proceed to later phases until Phase 0 and Phase 1 gates are met or an explicit documented

The ultimate goal is not a demo, a prototype, or a collection of screens.

The goal is a secure, accessible, native, connected, governed, reliable, supportable, sellable, i
```

Execution Instructions

Use the master prompt as the governing instruction, then run the project through discrete work packets. Do not ask an agent to “build all of Semester” in one uncontrollable run. Instead, enforce a gated sequence.

```
Master Prompt
      │
      ▼
Phase Prompt
      │
      ▼
Epic Prompt
      │
      ▼
Vertical Slice Prompt
      │
      ▼
Implementation + Tests + Evidence
      │
      ▼
Gate Review
      │
      ├── PASS → next slice
      ├── PASS WITH EXCEPTIONS → time-bound remediation
      └── NO-GO → remediate before proceeding
```

Phase Prompt Template

Use this companion prompt at the start of **every phase** after the master prompt is loaded.

```
You are executing Phase [NUMBER]: [PHASE NAME] of the Semester master execution program.

Read and obey:
- The Semester master execution program.
- Current architecture decision records.
- Capability traceability matrix.
- Tenant-isolation matrix.
- RLS policy map.
- Risk register.
- Legal-review queue.
- Accessibility matrix.
- Integration catalog.
```

```
- AI system inventory.
- Release-readiness matrix.
- Incident response runbooks.

Phase objective:
[INSERT OBJECTIVE]

Required outcomes:
[INSERT PHASE OUTCOMES]

In scope:
[INSERT SCOPE]

Out of scope:
[INSERT EXCLUSIONS]

Dependencies:
[INSERT DEPENDENCIES]

Before implementing:
1. Identify affected domains, users, roles, data classes, tenant boundaries, integrations, AI beh
2. Create or update an ADR for meaningful decisions.
3. Update the capability traceability matrix.
4. Define acceptance criteria.
5. Define test plan.
6. Define rollback/recovery plan.
7. Identify items requiring qualified human counsel review.
8. Identify accessibility requirements.
9. Identify support and monitoring requirements.

Implementation requirements:
- Build production-quality code only.
- No mock persistence for launch claims.
- No client-only authorization.
- No unchecked tenant parameter.
- No new direct browser database access unless explicitly documented, allowlisted, and tested.
- No new SECURITY DEFINER function without fixed search_path, explicit authorization, safe grants
- No sensitive action without audit event.
- No high-impact AI automation without user/human confirmation and policy enforcement.
- No release without observability and runbook.

Required evidence at completion:
- Changed capability-traceability entries.
- Changed architecture diagram/model.
- Migrations and rollback/roll-forward note.
- API and event contracts.
- RLS/grant/function evidence.
- Unit/integration/E2E/accessibility/security/performance test results.
- Cross-tenant negative test results.
- Monitoring dashboard/alert references.
- Runbook updates.
- Support workflow updates.
- Release checklist.
- Known limitations.
- Legal-review queue updates.
- Phase gate result.

Return the result in this format:

1. Executive summary
2. Scope completed
3. Architecture decisions
4. Files/services/database objects created or changed
5. Tenant/security/privacy analysis
```

```
6. Accessibility analysis
7. AI governance analysis
8. Tests executed and results
9. Operational readiness
10. Risks, exceptions, and legal-review items
11. Rollback/recovery plan
12. Phase gate recommendation: PASS / PASS WITH EXCEPTIONS / NO-GO
13. Next smallest complete vertical slice
```

First Vertical Slice Prompt

This is the first practical implementation packet after the baseline work.

```
Execute the first complete Semester production vertical slice.

Slice name:
Identity, tenancy, policy, audit, Today, tasks, calendar base, offline persistence, observability

Goal:
A student can securely authenticate, join an authorized tenant, access a native Today surface, cr

Build:

IDENTITY AND TENANCY
- Account profile.
- Secure authentication.
- MFA/passkey readiness.
- Session/device model.
- Institution/school tenant.
- Active membership.
- Role/capability grant.
- Consent record.
- Tenant selector only where multiple memberships genuinely apply.
- Trusted server-side tenant resolution.
- Revocation behavior.

POLICY AND AUDIT
- Central policy decision contract.
- Purpose-aware access checks.
- Tenant and ownership enforcement.
- Audit event envelope.
- Correlation IDs.
- Feature flag and kill switch integration.
- Time-bound support access model.
- Explicit deny states.

TODAY EXPERIENCE
- Student Today view.
- Priorities.
- Tasks due today/upcoming.
- Calendar items.
- Clear source and freshness labels.
- Empty states.
- Error states.
- Offline state.
- Sync state.
- Accessibility-first responsive UI.
- Mobile-first interaction behavior.

TASKS AND CALENDAR
- Native task creation/update/completion.
- Native calendar event creation/update.
```

```
- Deadlines/reminders foundation.
- Idempotent commands.
- Durable server persistence.
- Encrypted local device persistence where applicable.
- Sync queue.
- Conflict strategy.
- User-visible pending/confirmed/failed state.
- No dependency on Google/Microsoft/SIS/LMS for core task/calendar use.

OFFLINE
- Encrypted local store.
- Device-scoped key handling.
- Queue persistence.
- Retry behavior.
- Conflict detection.
- User-visible repair/retry flow.
- Server confirmation for sensitive/transactional behavior.
- Revoked-device/logout cleanup behavior.

OBSERVABILITY
- Request correlation ID.
- Structured logs.
- Metrics for auth, Today, task, calendar, sync, failure, and latency.
- Health checks.
- Dashboards.
- Alerts.
- Support diagnostic reference without exposing private content.

DATABASE
- Tenant-scoped schema.
- RLS policies.
- Ownership policies.
- Explicit grants allowlist.
- No anonymous table access by default.
- No browser access to service-only tables.
- Cross-tenant tests.
- Tenant revocation tests.
- Function authorization tests.
- Migration validation.

CI/CD
- Pull request checks.
- Build, lint, typecheck, unit tests.
- Secrets scan.
- dependency scan.
- policy/RLS tests.
- staging migration.
- staging deployment.
- E2E tests.
- accessibility checks.
- production approval.
- canary deployment.
- automated rollback thresholds.
- release evidence recording.

Do not use mock data as a substitute for persistence.
Do not ship any untested privilege pathway.
Do not create client-side-only tenant access checks.
Do not allow a user to set arbitrary tenant IDs.
Do not mark complete until all acceptance criteria and evidence are met.

Return:
- completed implementation;
- database policy map;
```

```
- test results;
- audit/event examples;
- sync behavior;
- release workflow;
- rollback plan;
- current risks;
- exact phase-gate status.
```

Non-Negotiable Operating Rule

The master prompt must always force the execution agent to answer this question for every capability:

> “Can a real student, staff member, guardian, institution, or operator use this safely and successfully today—with native data, correct permissions, clear states, operational support, accessibility, auditability, and recovery—without relying on an unverified future integration or manual developer intervention?”

If the answer is no, the item is not complete.

The existing Semester vision, repository, database, and audit artifacts provide a serious foundation, especially in governance and operational intent. The master prompt turns that foundation into an enforceable execution system with traceability, phased gates, tenant-isolation hardening, release controls, incident response, and native product completeness.[60] [61] [59]

⁂

1. https://www.nist.gov/itl/ai-risk-management-framework
2. https://studentprivacy.ed.gov/faq/what-education-record
3. https://www.nist.gov/video/introduction-nist-ai-risk-management-framework-ai-rmf-10-explainer-video
4. https://www.azed.gov/specialeducation/records-educational-records
5. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf
6. https://www.fldoe.org/core/fileparse.php/7690/urlt/0070103-ferpa.pdf
7. https://studentprivacy.ed.gov/training/what-parents-need-know-about-their-students-data
8. https://www.ctas.tennessee.edu/eli/family-educational-rights-and-privacy-act
9. https://www.paloaltonetworks.com/cyberpedia/nist-ai-risk-management-framework
10. https://epic.org/family-educational-rights-and-privacy-act-ferpa/
11. https://digitalgovernmenthub.org/library/nist-ai-risk-management-framework-playbook/
12. https://orca.security/resources/blog/nist-ai-risk-management-framework-ai-rmf/
13. https://studentprivacy.ed.gov/ferpa
14. https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence
15. https://studentprivacy.ed.gov/
16. https://www.nist.gov/itl/ai-risk-management-framework
17. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf
18. https://www.azed.gov/specialeducation/records-educational-records
19. https://www.w3.org/TR/WCAG22/
20. https://www.w3.org/WAI/standards-guidelines/wcag/glance/
21. https://www.1edtech.org/standards/lti
22. https://www.imsglobal.org/lti-advantage-overview
23. https://www.imsglobal.org/lti/toolkit

24. https://www.imsglobal.org/lti-advantage-faq
25. https://www.1edtech.org/lti-advantage-data
26. https://www.accessibilitychecker.org/guides/wcag-2-2/
27. https://www.allaccessible.org/blog/wcag-22-complete-guide-2025%29
28. https://pr-preview.s3.amazonaws.com/w3c/wcag/4040/a9e49c0...abe30f6.html
29. https://accessibility.build/guides/wcag-2-2-aa-requirements
30. projects.semester.product_vision
31. projects.semester.cross_device_experience
32. projects.semester.architecture_scope
33. goals.semester.market_positioning
34. projects.semester.operations_console
35. projects.semester.infrastructure
36. projects.semester.repository
37. https://www.perplexity.ai/search/f2c0f14b-2e49-444e-a8c3-4f41734f8e40
38. https://orca.security/resources/blog/nist-ai-risk-management-framework-ai-rmf/
39. https://w3c.github.io/wcag/requirements/22/
40. https://cheatsheetseries.owasp.org/cheatsheets/Multi_Tenant_Security_Cheat_Sheet.html
41. https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-61r3.pdf
42. https://www.cisa.gov/resources-tools/resources/federal-government-cybersecurity-incident-and-vulnerability-response-playbooks
43. https://csrc.nist.gov/projects/incident-response
44. https://csrc.nist.gov/pubs/sp/800/61/r3/final
45. https://www.fortinet.com/blog/ciso-collective/incident-response-plans-playbooks-policy
46. https://www.paloaltonetworks.com/cyberpedia/what-is-an-incident-response-playbook
47. https://linfordco.com/blog/nist-sp-800-61/
48. https://cynomi.com/nist/nist-incident-response-life-cycle-explained/
49. https://www.cynet.com/security-foundations/incident-response/nist-incident-response/
50. https://www.studocu.com/en-us/document/national-university/incident-handlingresponse/option-2/30615662
51. https://www.cisa.gov/sites/default/files/2024-08/Federal_Government_Cybersecurity_Incident_and_Vulnerability_Response_Playbooks_508C.pdf
52. https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-61r2.pdf
53. https://www.cisa.gov/planning-response-recovery
54. https://www.cisa.gov/news-events/events/using-cisa-incident-response-playbook-your-organization-ir111
55. https://www.nist.gov/itl/ai-risk-management-framework
56. https://studentprivacy.ed.gov/ferpa
57. https://studentprivacy.ed.gov/faq/what-education-record
58. https://www.azed.gov/specialeducation/records-educational-records
59. audit-https___github.com_harrisonjrubin7-cmyk_sem-2.pdf
60. https://studentprivacy.ed.gov/ferpa
61. https://studentprivacy.ed.gov/faq/what-education-record